A

PROJECT REPORT

on

AI-ASSISTED HOUSE PLANNING AND VISUALIZATION SYSTEM

Submitted For Partial Fulfillment & Award of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

MOHAMMAD RAAFEY AZEEM [23XX1A0522]

S. SANJANA [24XX5A0524]

D. TULASIRAM [23XX1A0512]

SRIVATSA [24XX5A0503]

Under the Guidance of

Dr. MALLU PRAVEEN



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

(Established by Govt.Act No.30 of 2008)

UNIVERSITY COLLEGE OF ENGINEERING MAHABUBABAD

Indira Nagar,Mahababad (Dist)-506101,T.G.(India)

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

(Established by Govt.Act No.30 of 2008)

UNIVERSITY COLLEGE OF ENGINEERING MAHABUBABAD

Indira Nagar,Mahabubabad (Dist)-506101,T.G.(India)

CERTIFICATE

This is to certify that the project report entitled "AI-ASSISTED HOUSE PLANNING AND VISUALIZATION SYSTEM" is a bonafide work done by MOHAMMAD RAAFEY AZEEM (23XX1A0522), S. SANJANA (24XX5A0524), D. TULASIRAM (23XX1A0512), SRIVATSA (24XX5A0503). In partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in CSE from Jawaharlal Nehru Technological University Hyderabad During the Academic Year [2026-27].

Dr. MALLU PRAVEEN
NAME OF THE GUIDE

Dr. P. RAJU
HEAD OF THE DEPARTMENT

INTERNAL EXAMINER
DATE :

EXTERNAL EXAMINER
DATE:

ACKNOWLEDGEMENT

We are grateful to numerous individuals who contributed to the preparation of our Project Report.

We wish to express our sincere and heartfelt gratitude to our Project Guide Dr. Mallu Praveen, Computer Science and Engineering, who encouraged us to take up this project in sync with industry needs.

We thank and express a deep sense of gratitude to Dr. Raju, Head of the Department, Computer Science and Engineering, for his constant encouragement and cooperation during the project work.

We express our gratitude and utmost regards to Dr. A. Balaram Naik (M.E, Ph.D, M.I.E.E.E, MISTE) Principal, University College of Engineering Mahabubabad, for his constant support, encouragement and for providing the necessary permissions during the completion of the project.

We also thank the entire faculty members and fellow classmates who directly or indirectly helped us to complete this project.

MOHAMMAD RAAFEY AZEEM [23XX1A0522]

S. SANJANA [24XX5A0524]

D. TULASIRAM [23XX1A0512]

SRIVATSA [24XX5A0503]

DECLARATION

We hereby declare that the Project Report entitled "AI-ASSISTED HOUSE PLANNING AND VISUALIZATION SYSTEM" under the guidance of Dr. Mallu Praveen, is submitted in partial fulfillment of the requirements for the Award of the Degree of Bachelor of Technology in CSE. This is a record of bonafide work carried out by us and the results embodied in this Project Report have not been submitted to any other University or Institute for the Award of any other Degree.

MOHAMMAD RAAFEY AZEEM [23XX1A0522]

S. SANJANA [24XX5A0524]

D. TULASIRAM [23XX1A0512]

SRIVATSA [24XX5A0503]

DATE:

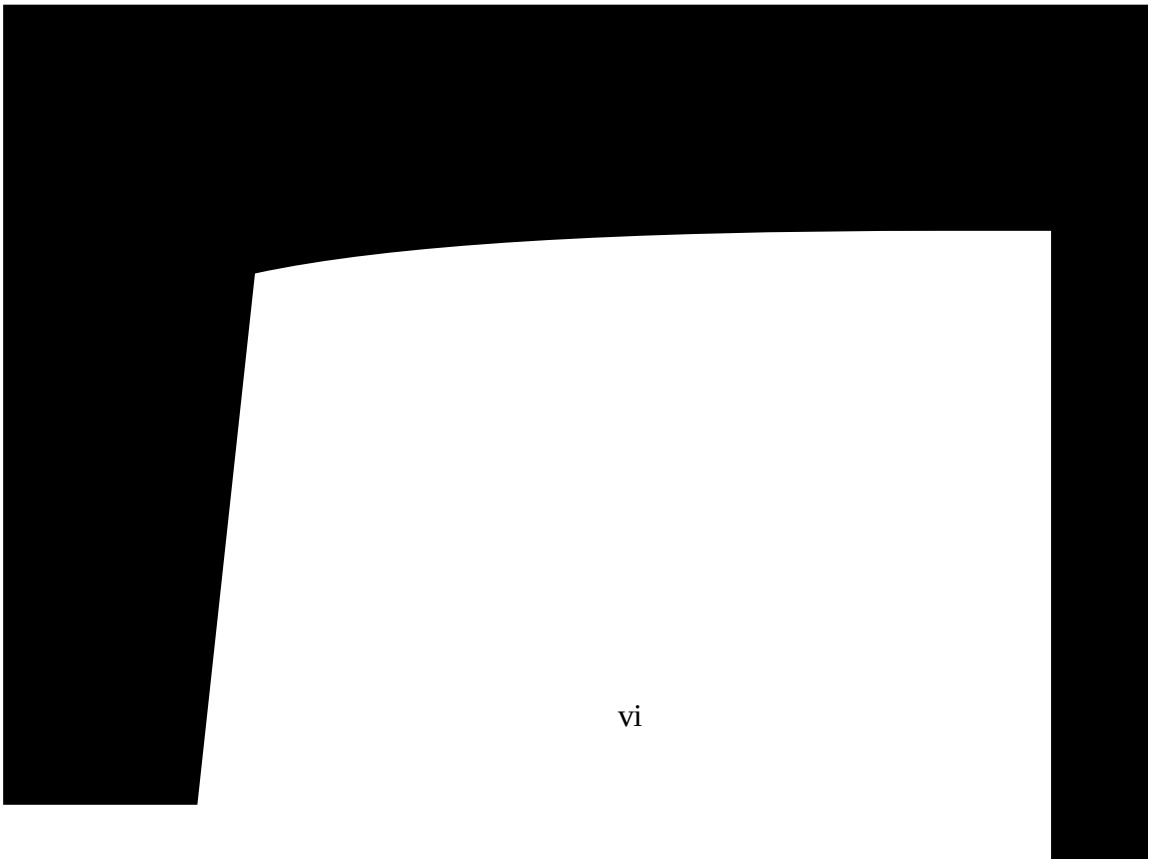
Department of Computer Science & Engineering

JNTUH, University College of Engineering, Mahabubabad

ABSTRACT

Planning a residential building usually involves separate tools and professionals for floor-plan drafting, interior selection, exterior finishing and construction cost estimation, so a change made in one stage is rarely reflected in the others. This project proposes the AI-Assisted House Planning and Visualization System, a web application that connects these stages in one workflow. A user enters plot dimensions, facing direction, number of floors and required rooms. The system generates a 2D floor plan using constraint-based planning with OR-Tools CP-SAT, validates it geometrically with Shapely, and can use Gemini for architectural reasoning while keeping a deterministic fallback.

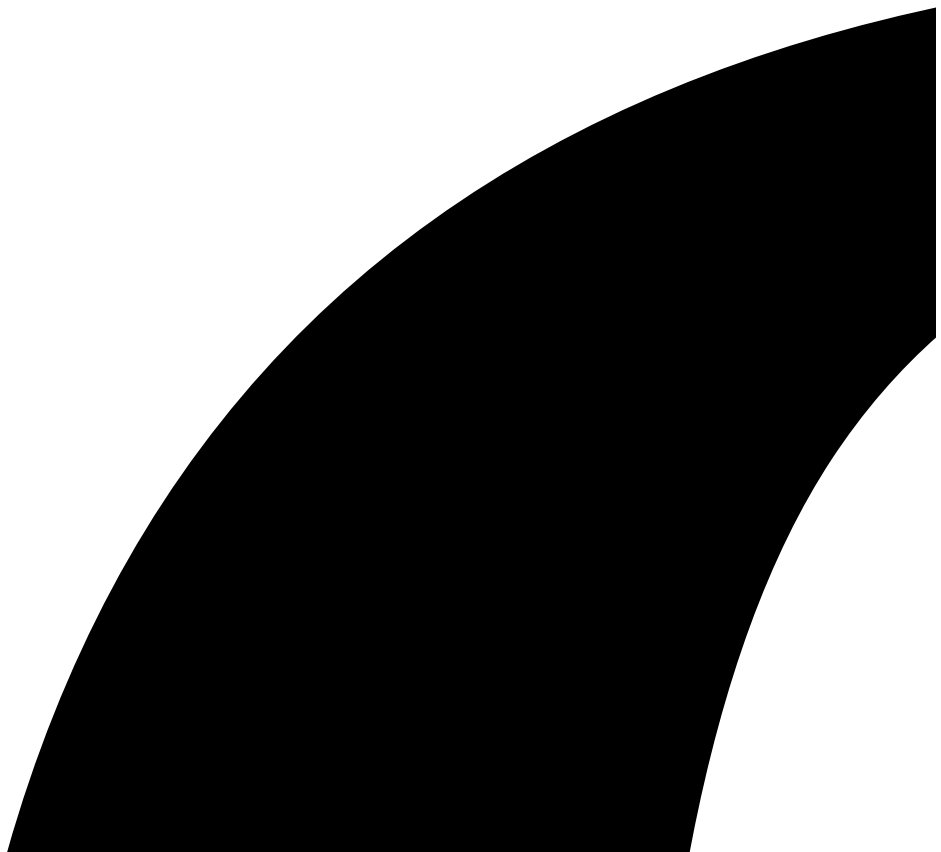
The user can then browse 32 interior design references across six room categories, select designs for the rooms that exist in the plan, and customize materials and lighting. A transparent cost module derives quantities from plan geometry, applies editable material and labour rates, adds contingency and contractor overhead, and recalculates without rerunning the floor-plan solver. The frontend uses React with Vite, and the backend uses FastAPI, Pydantic and Supabase. This Stage-1 report documents the system design and the partial work completed. According to the project checkpoint, 15 cost estimation tests, 36 combined interior, cost and geometry tests, and the production build passed; these results are reported and have not been independently verified. Exterior design, colour selection and coherent 3D visualization are planned for Stage-2.





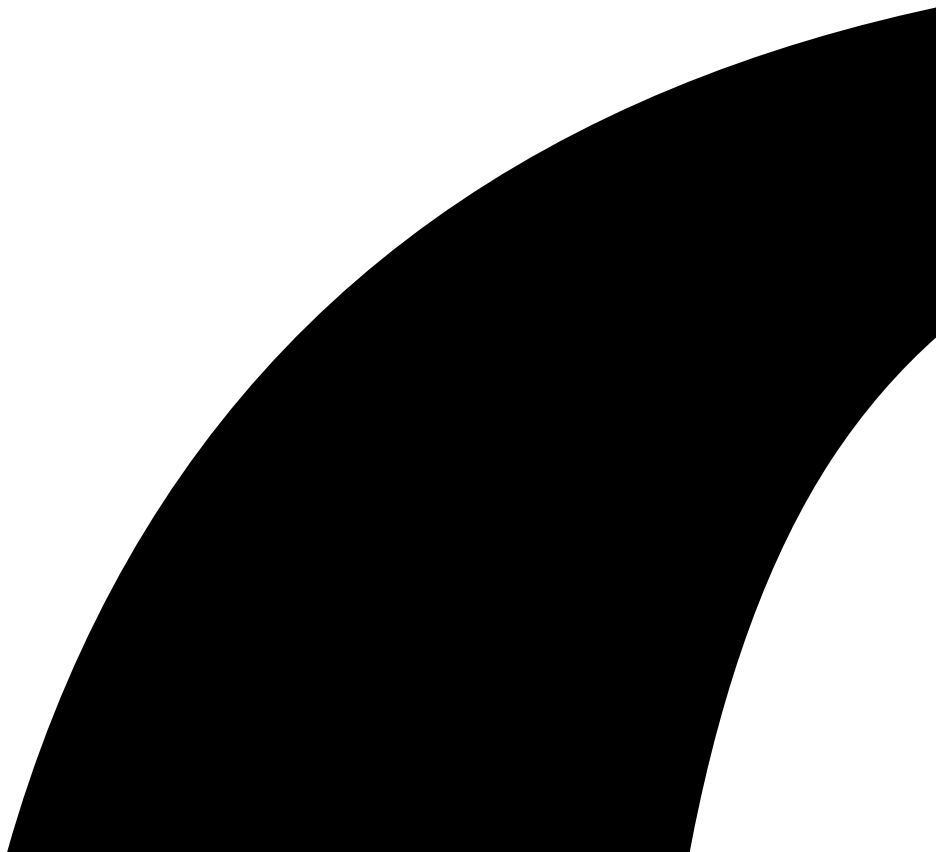




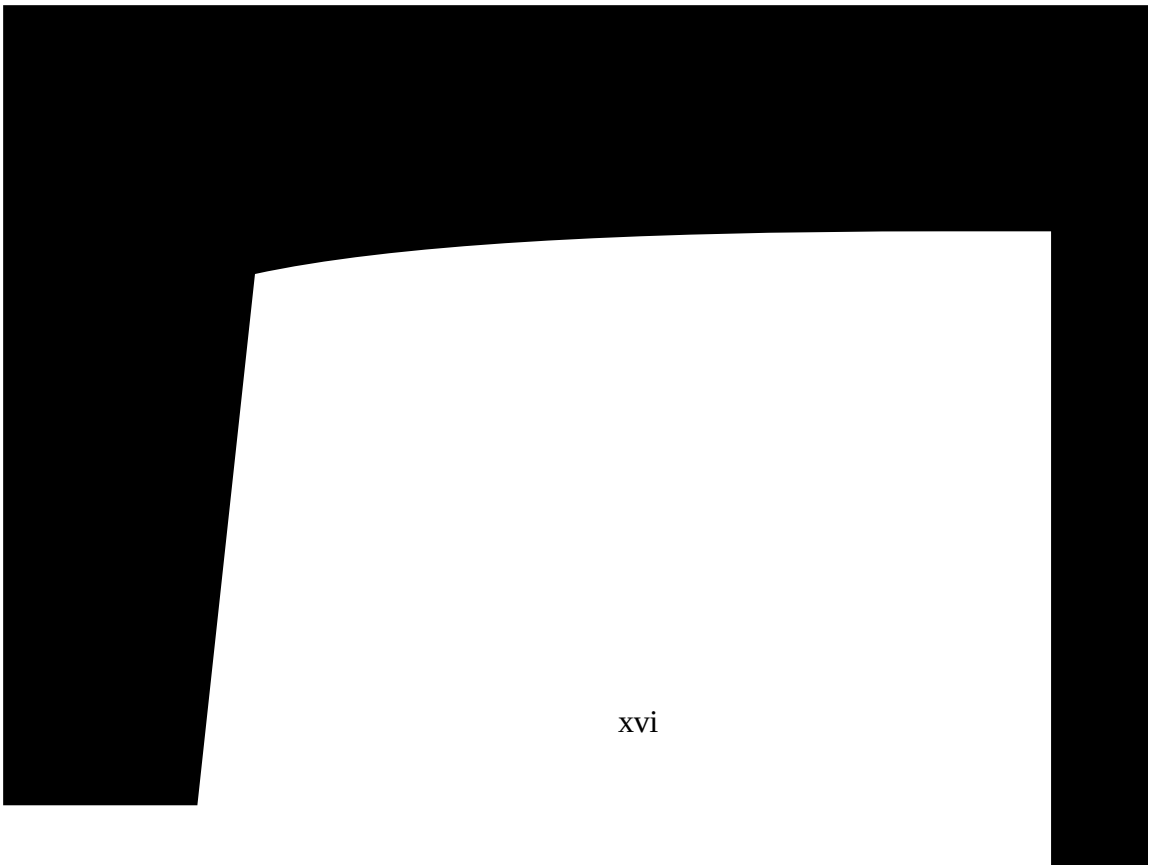






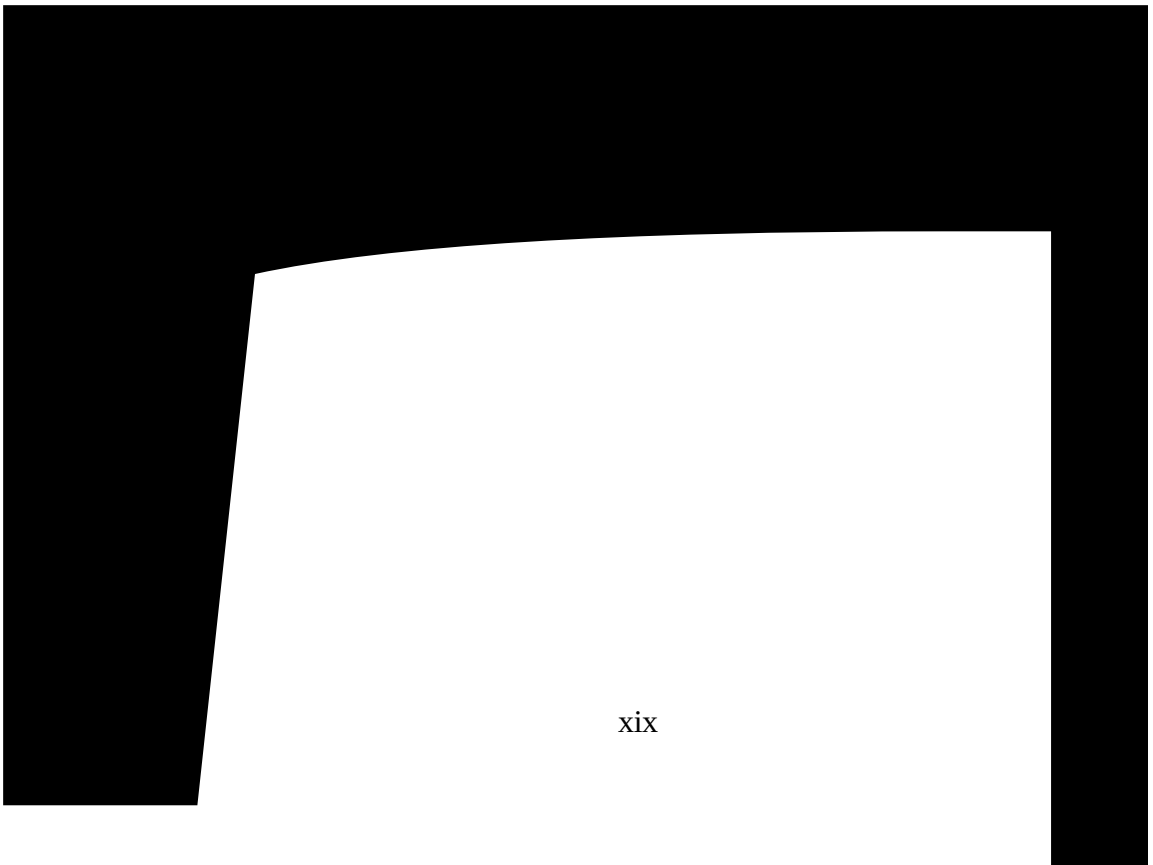


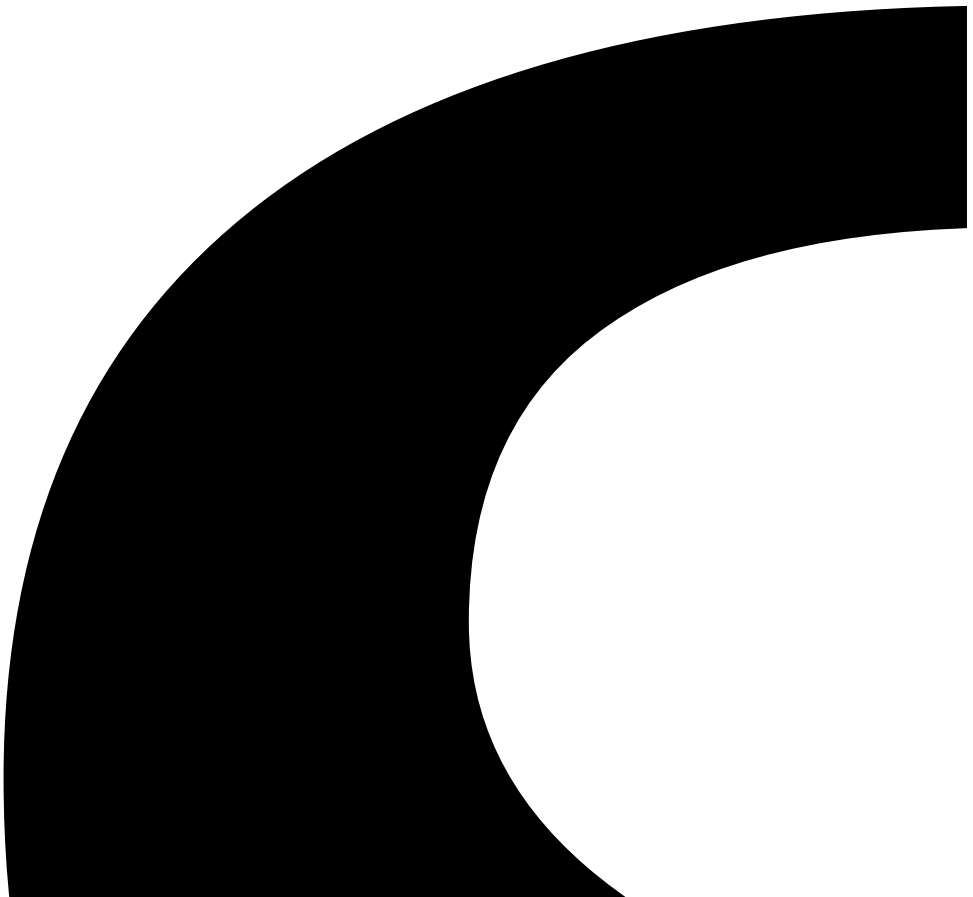


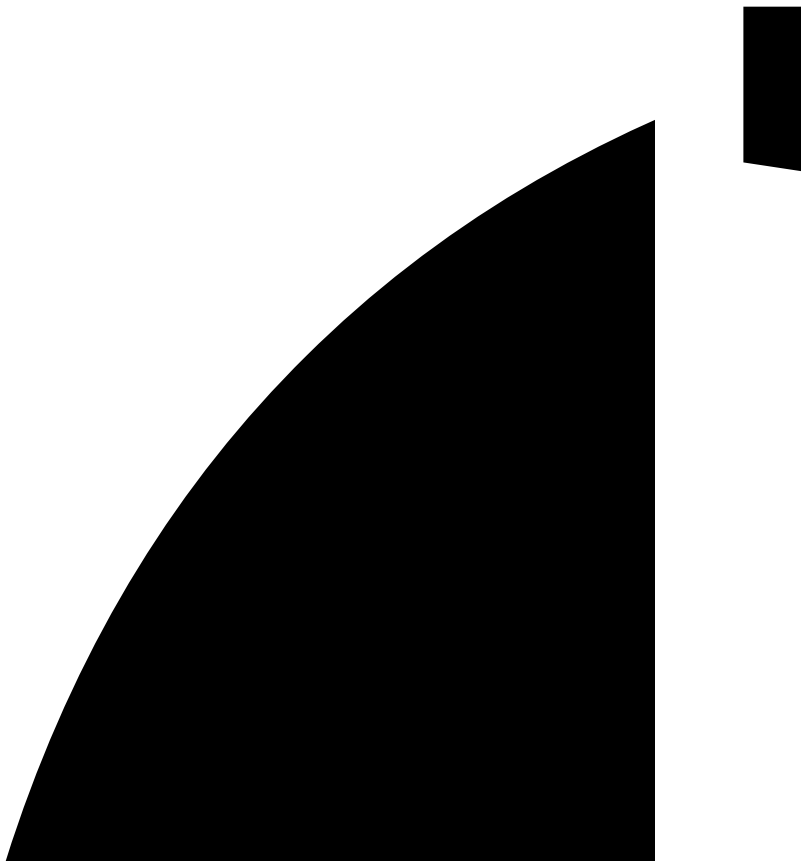


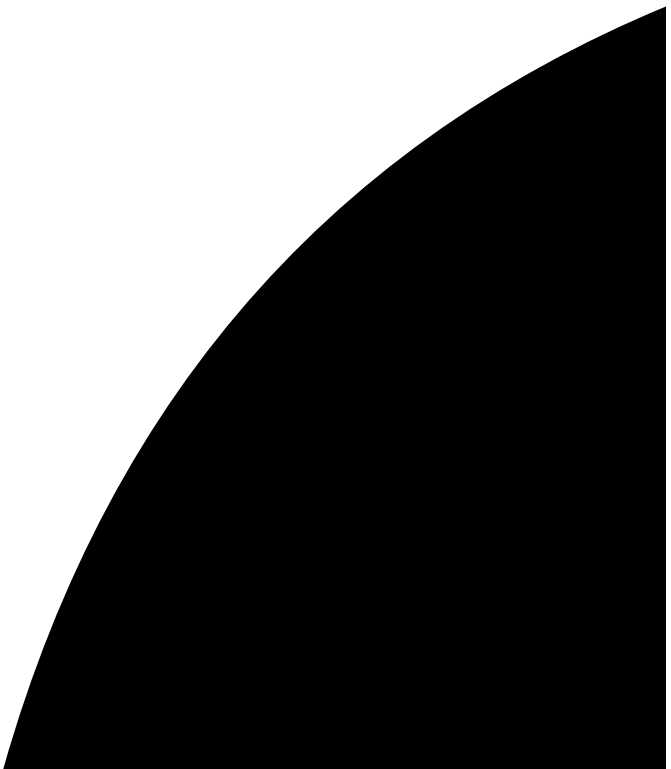


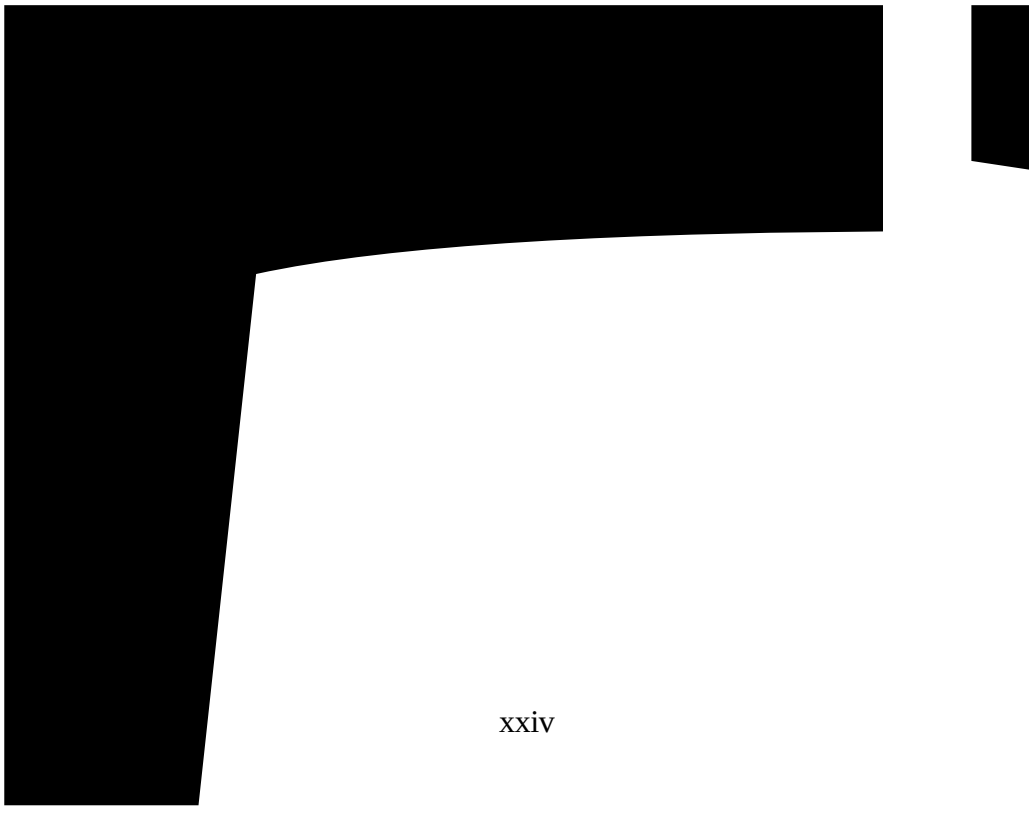




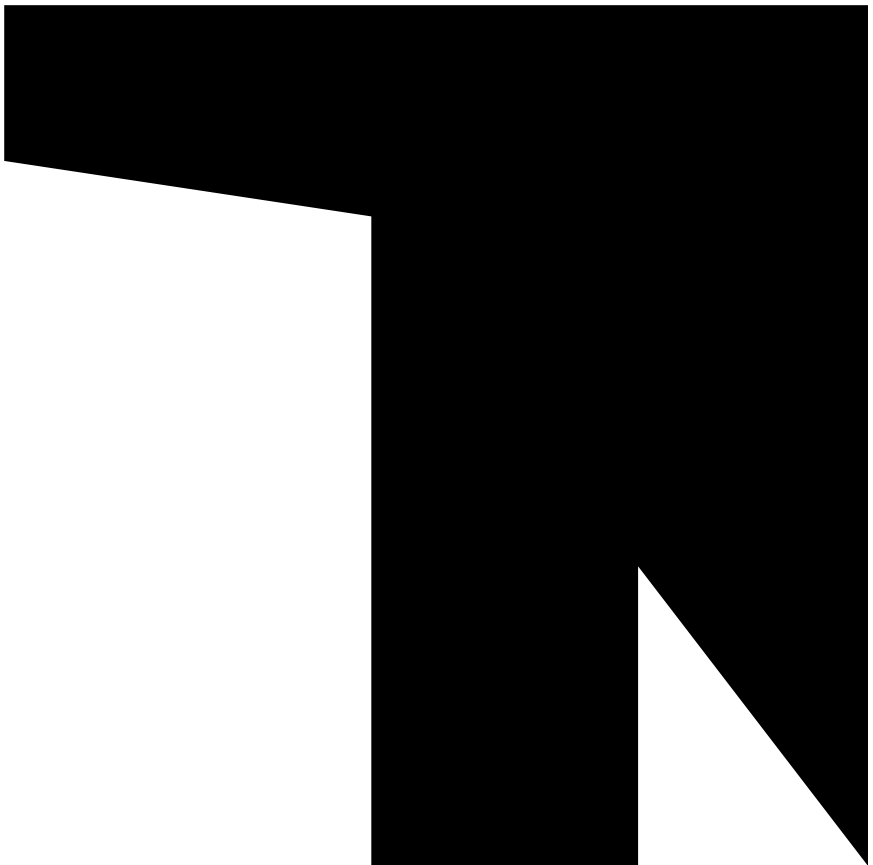


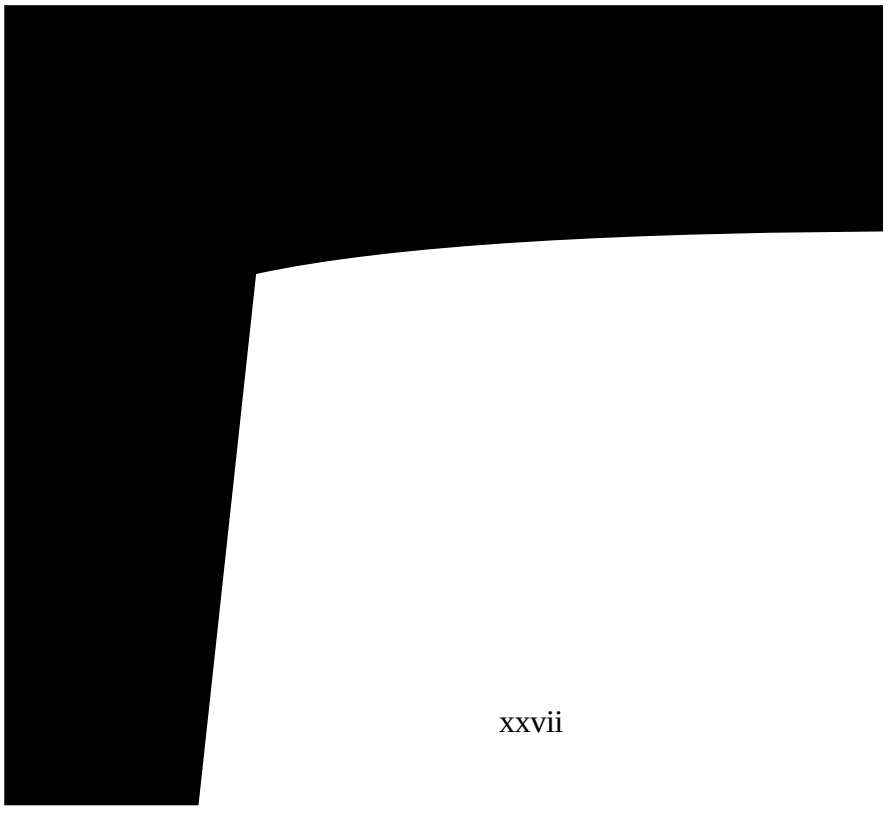


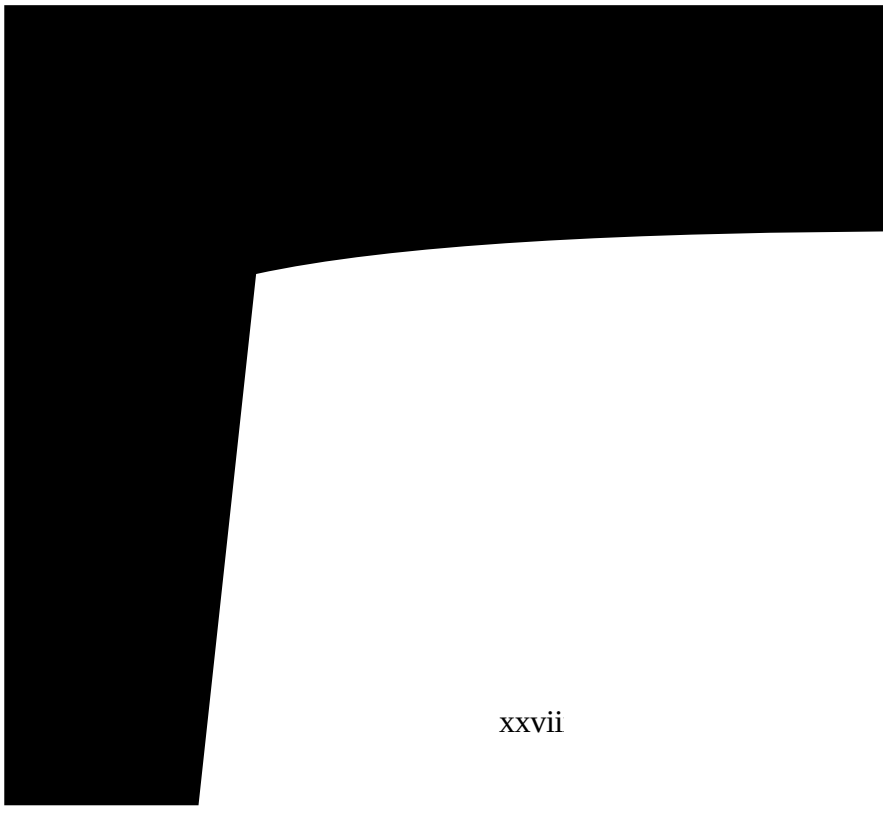


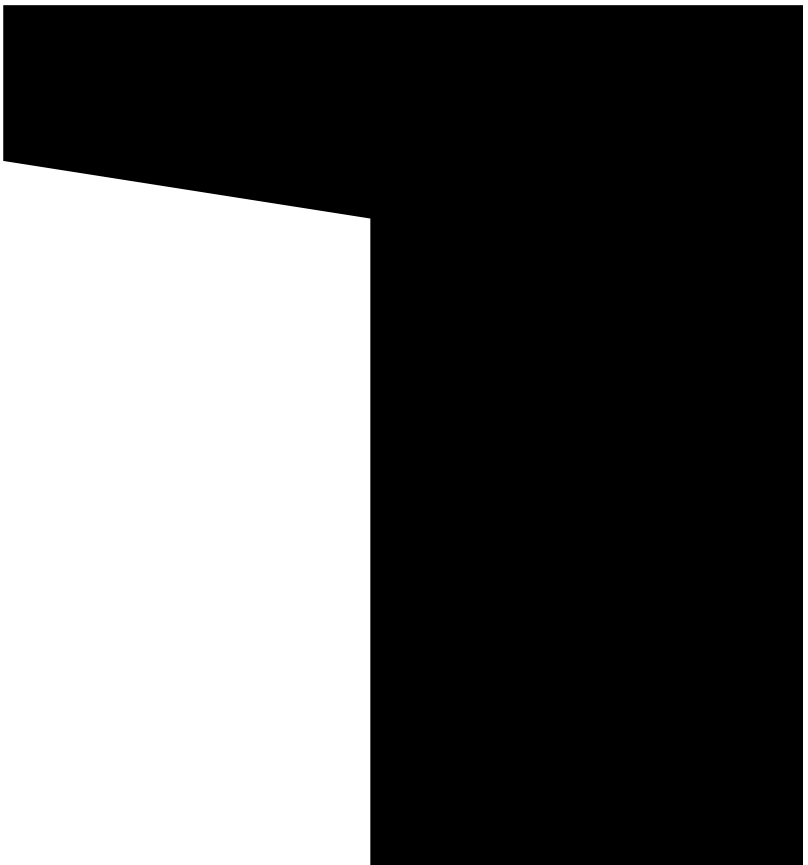




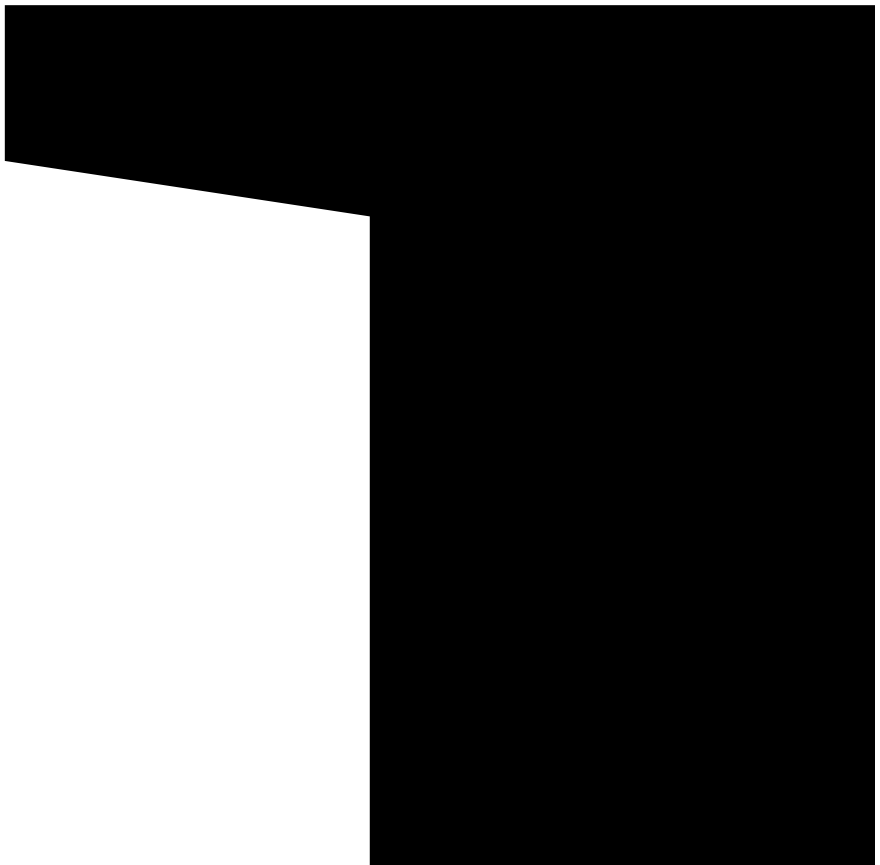


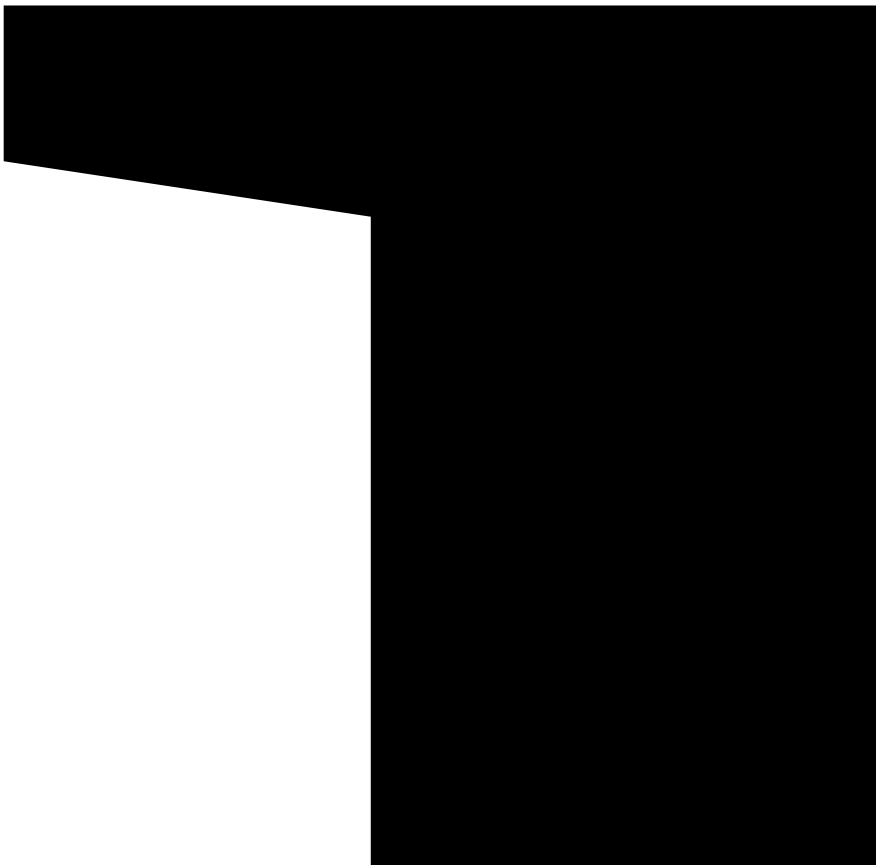


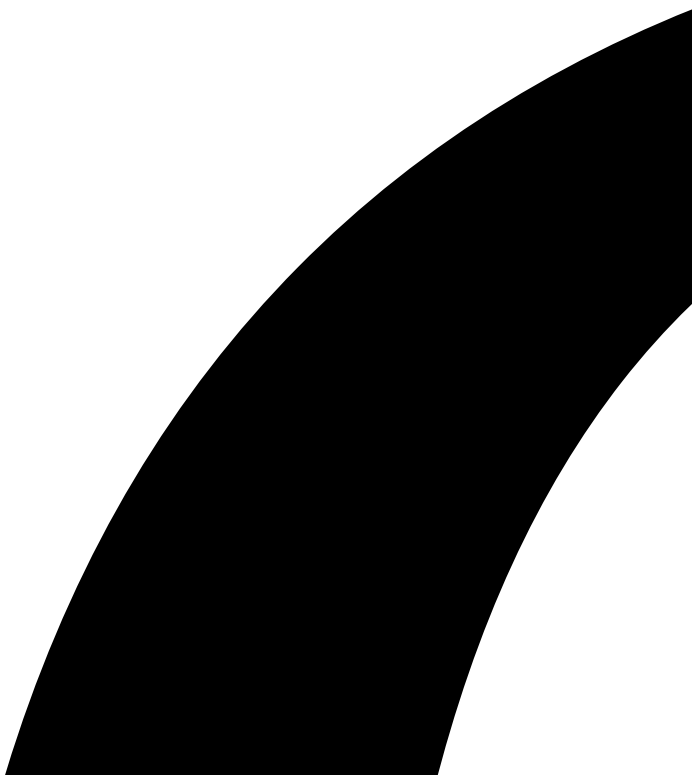


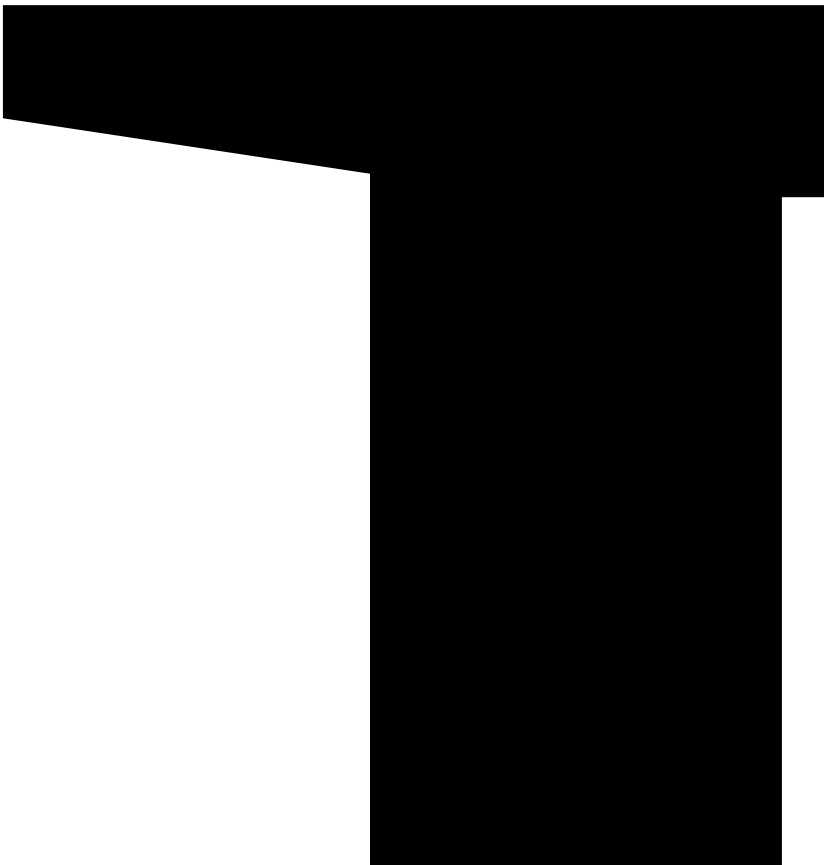


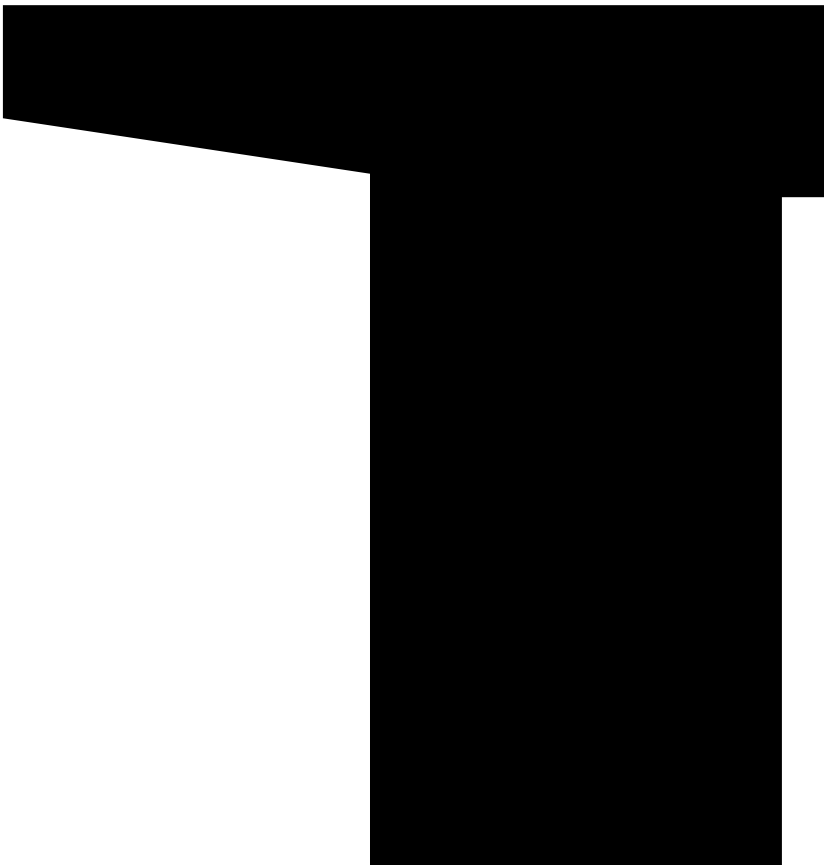




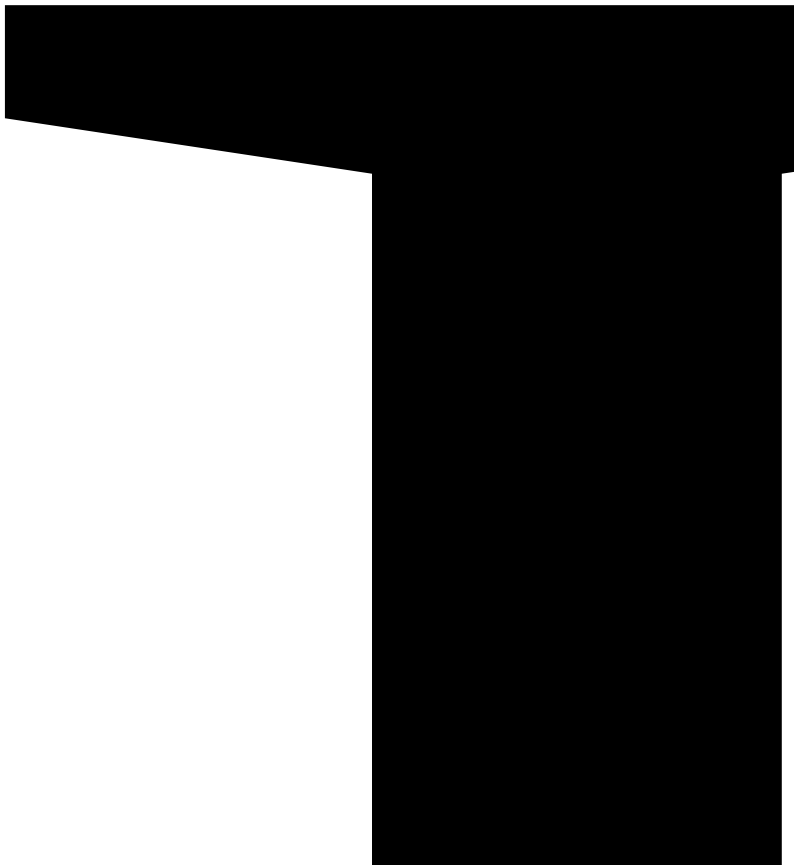


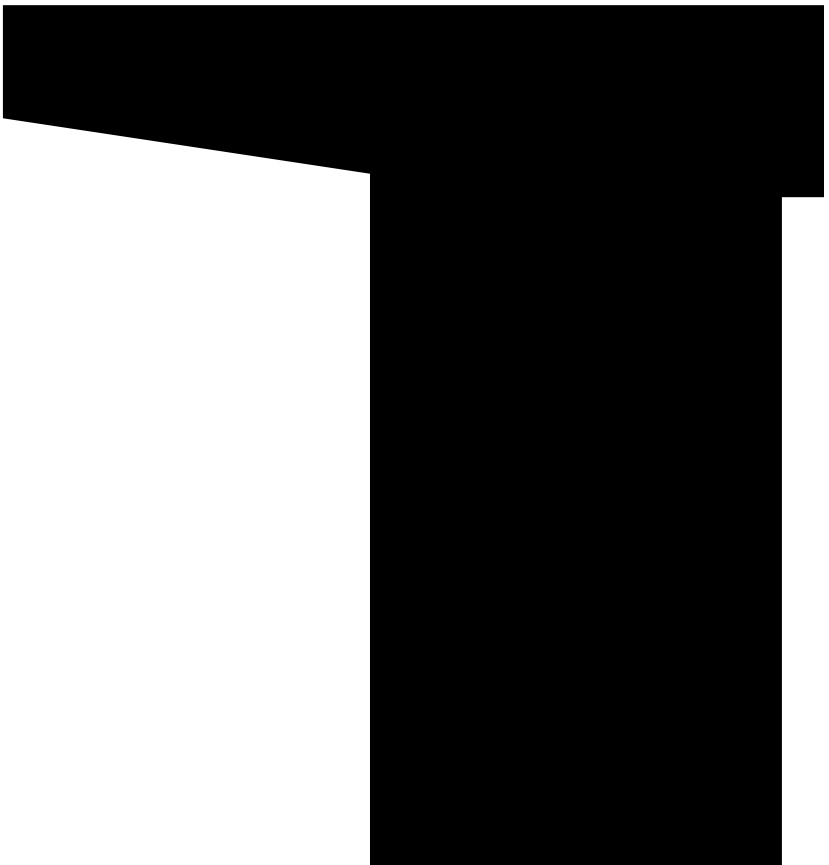


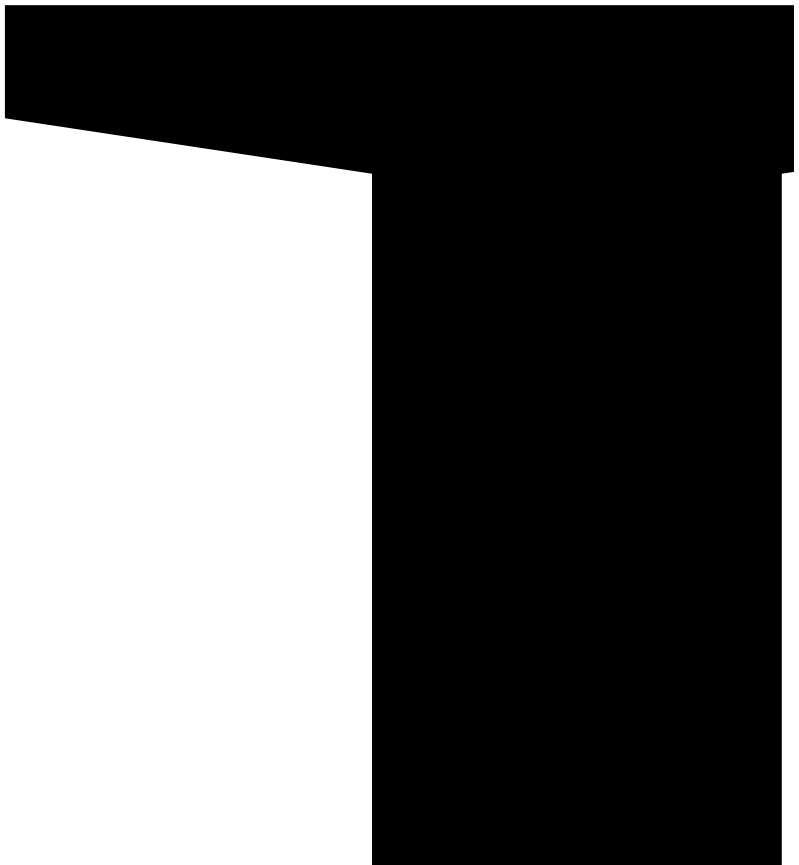


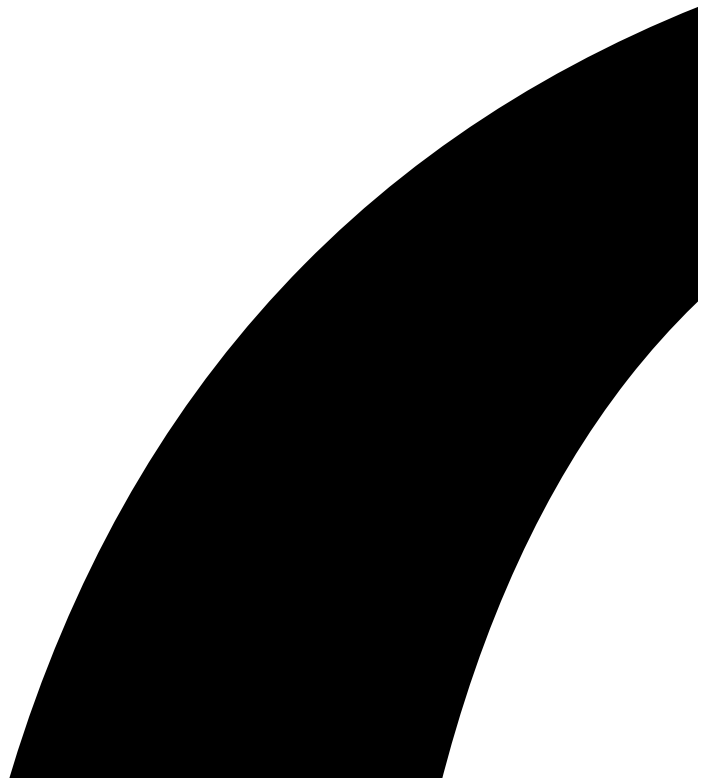




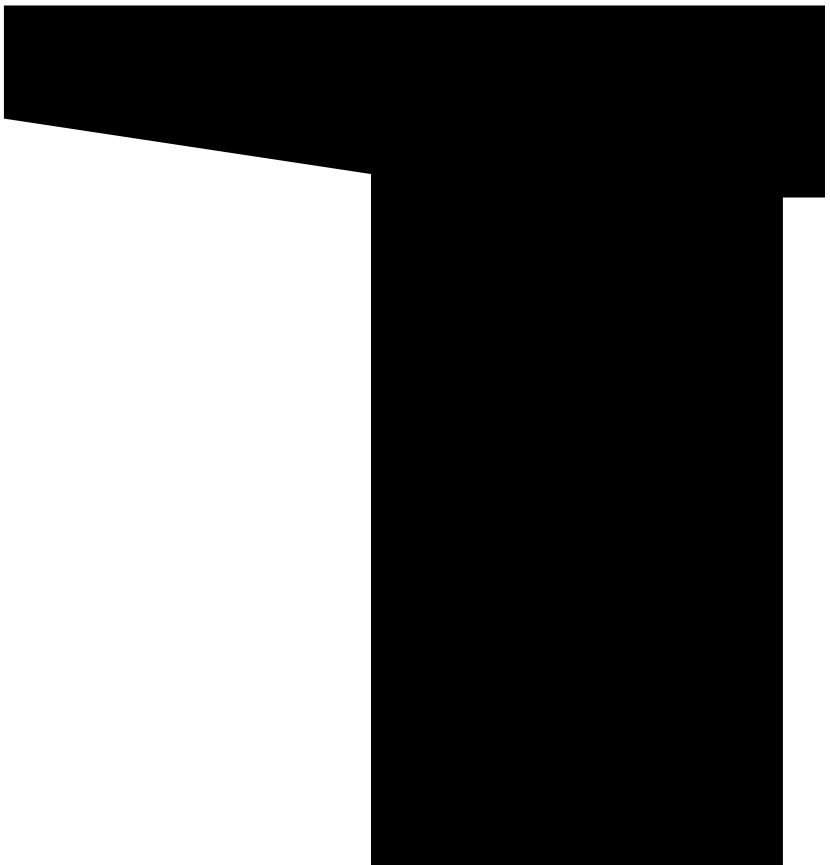


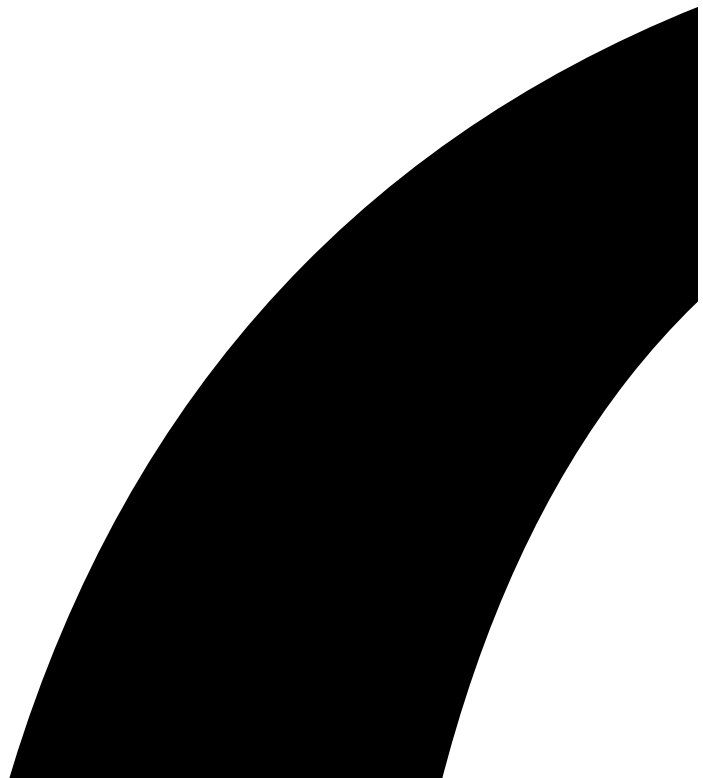


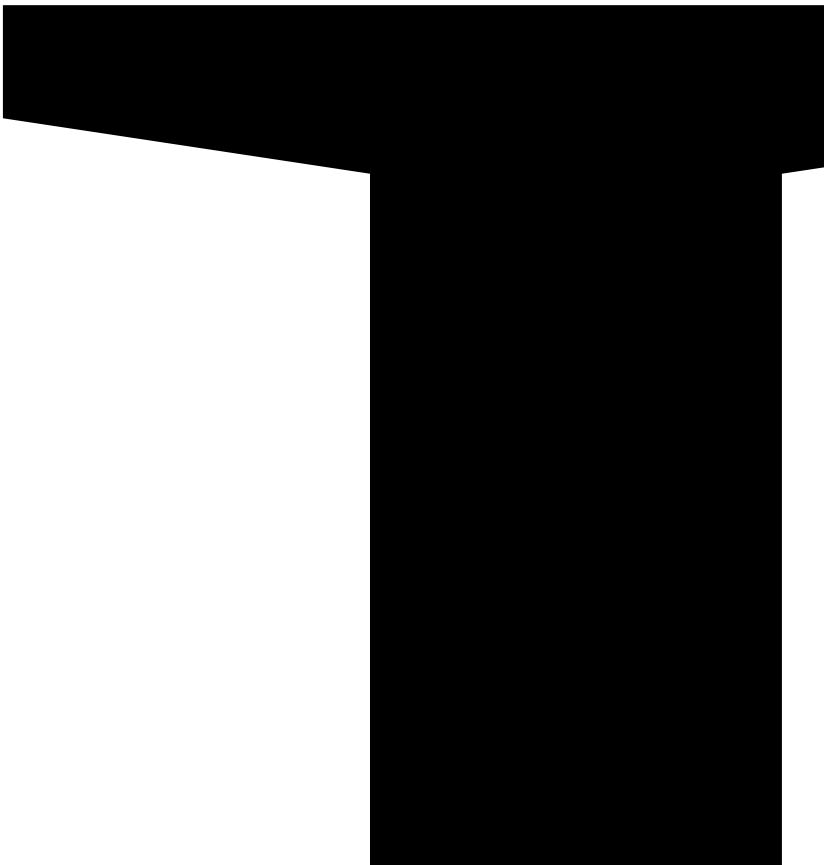


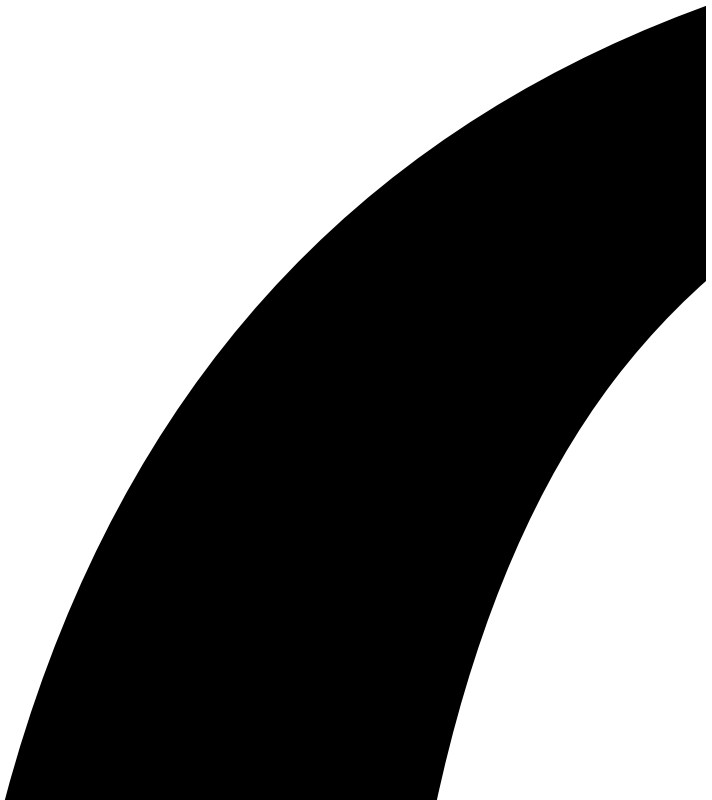




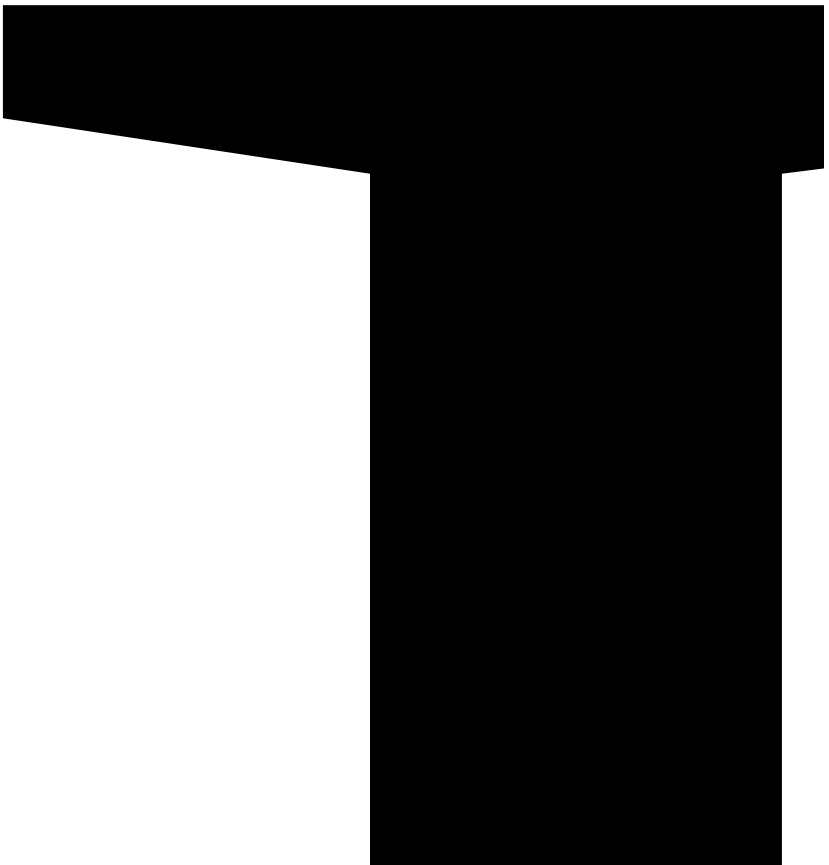


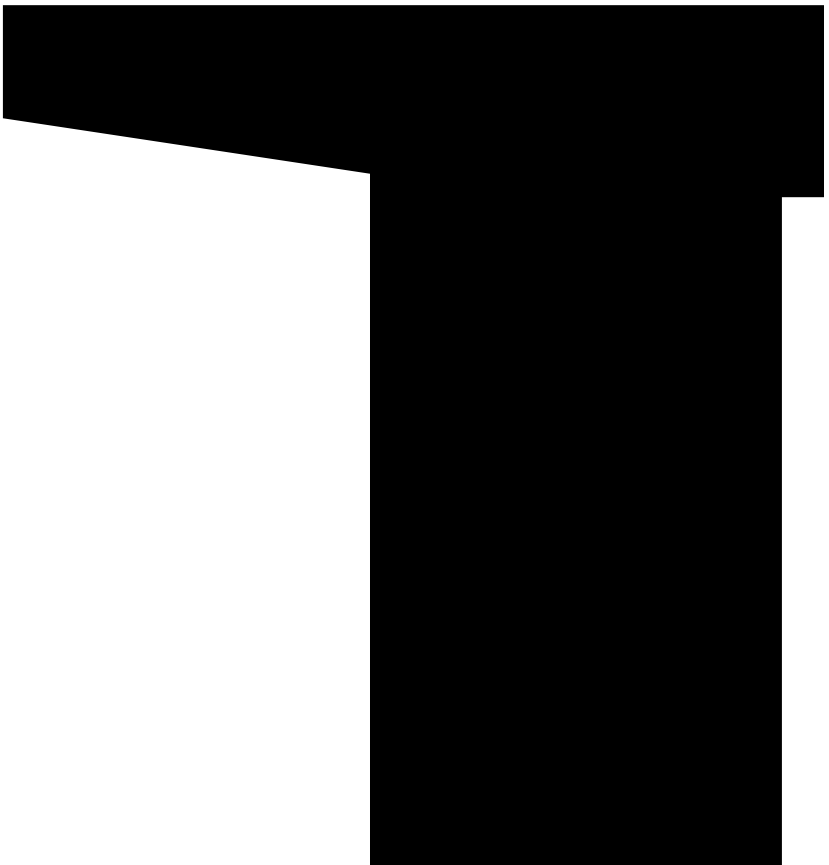


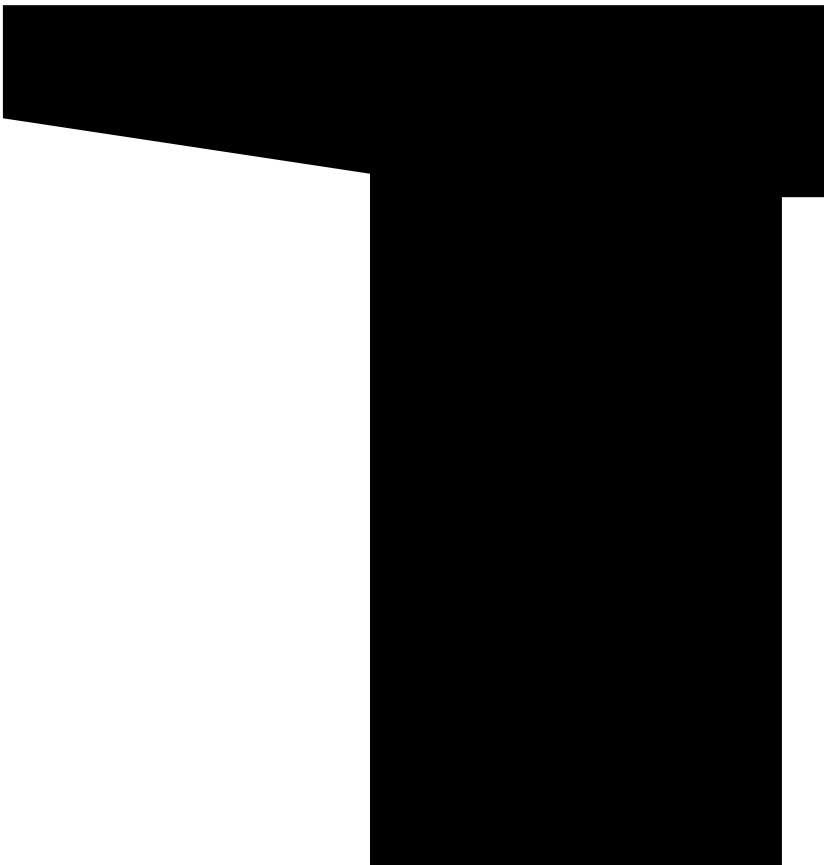


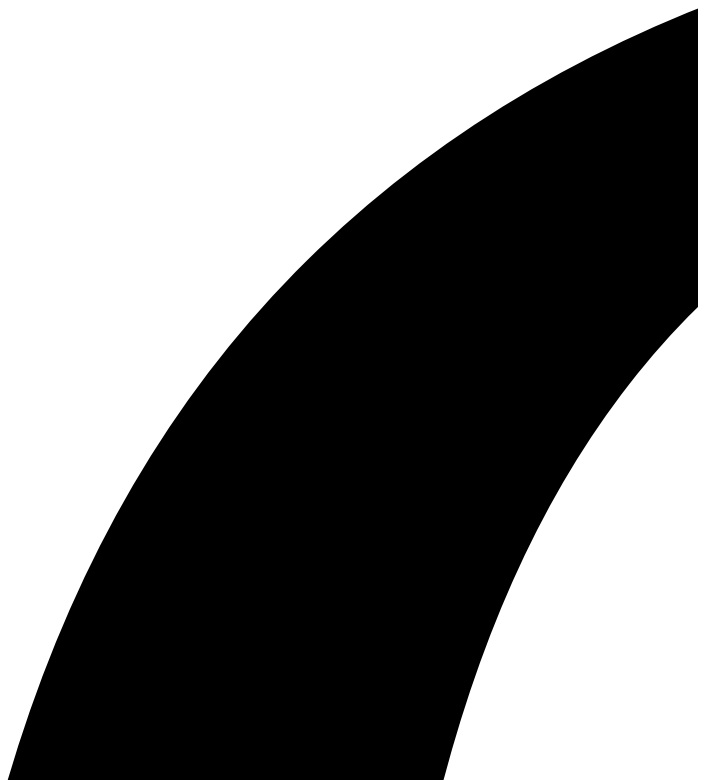


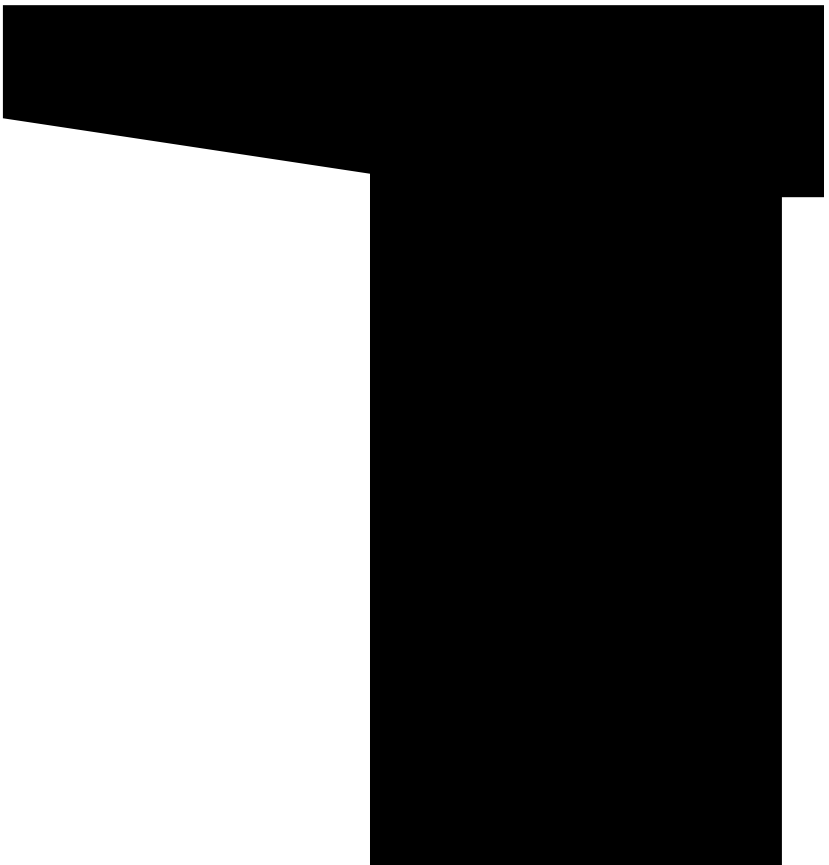


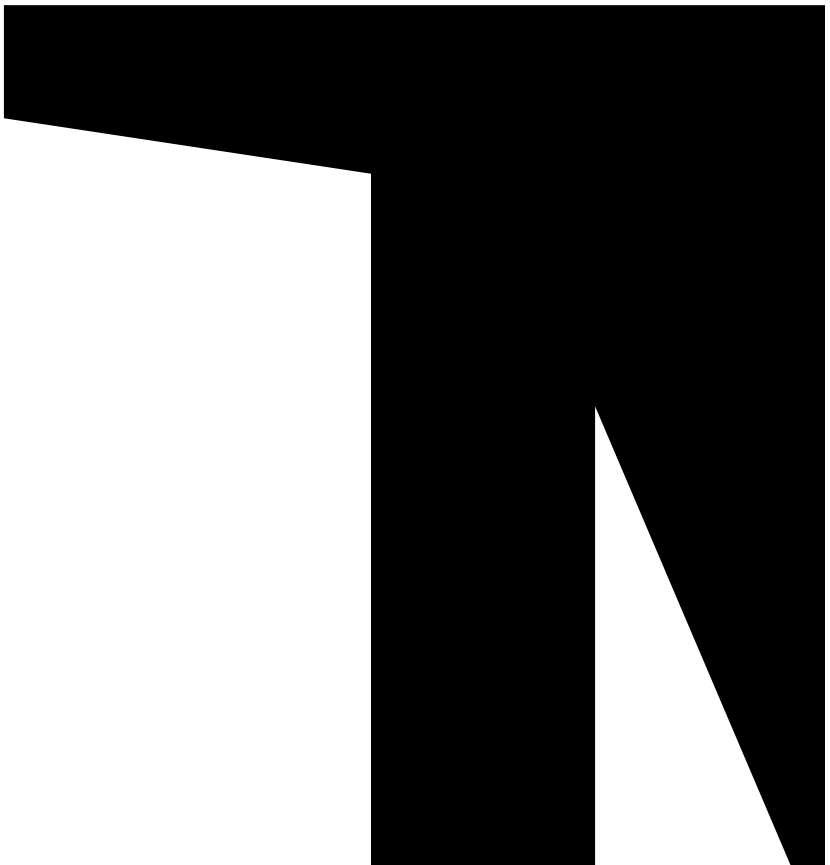


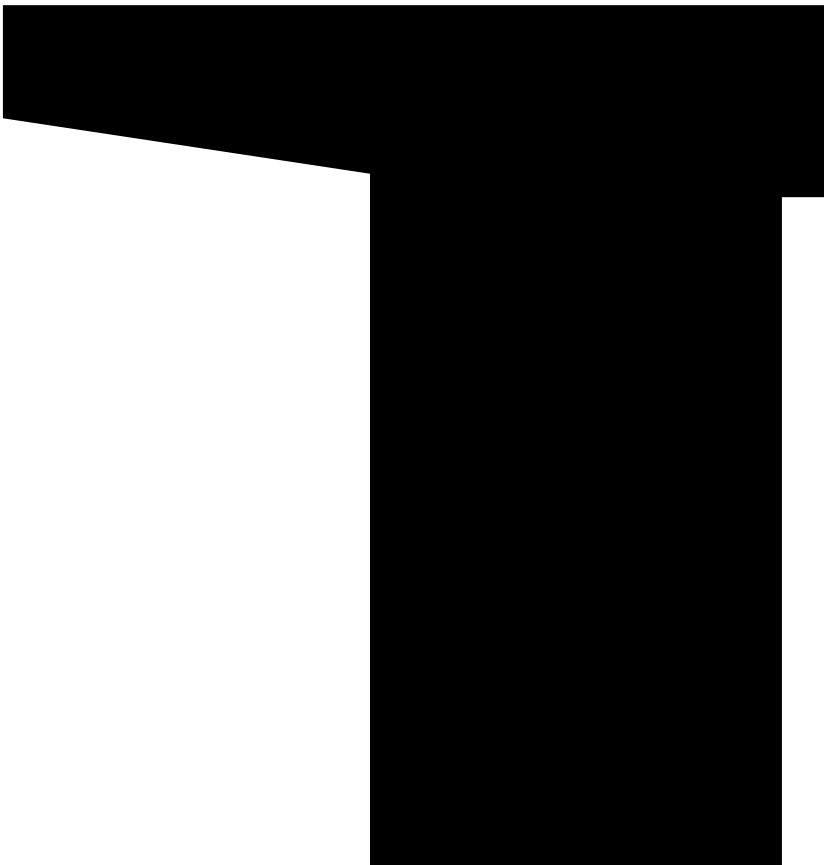




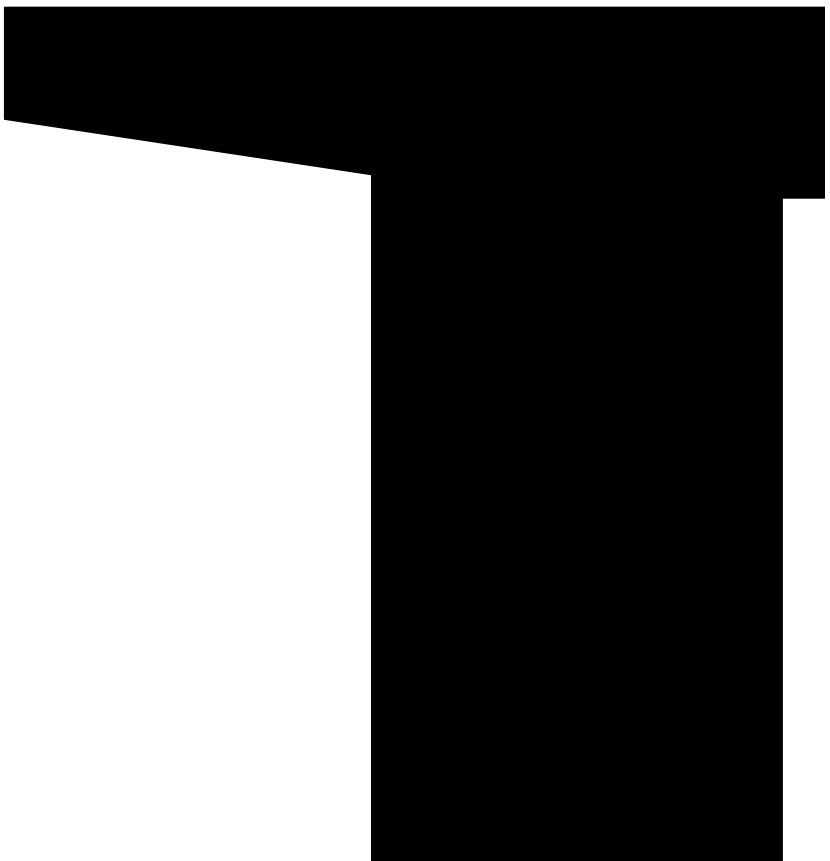


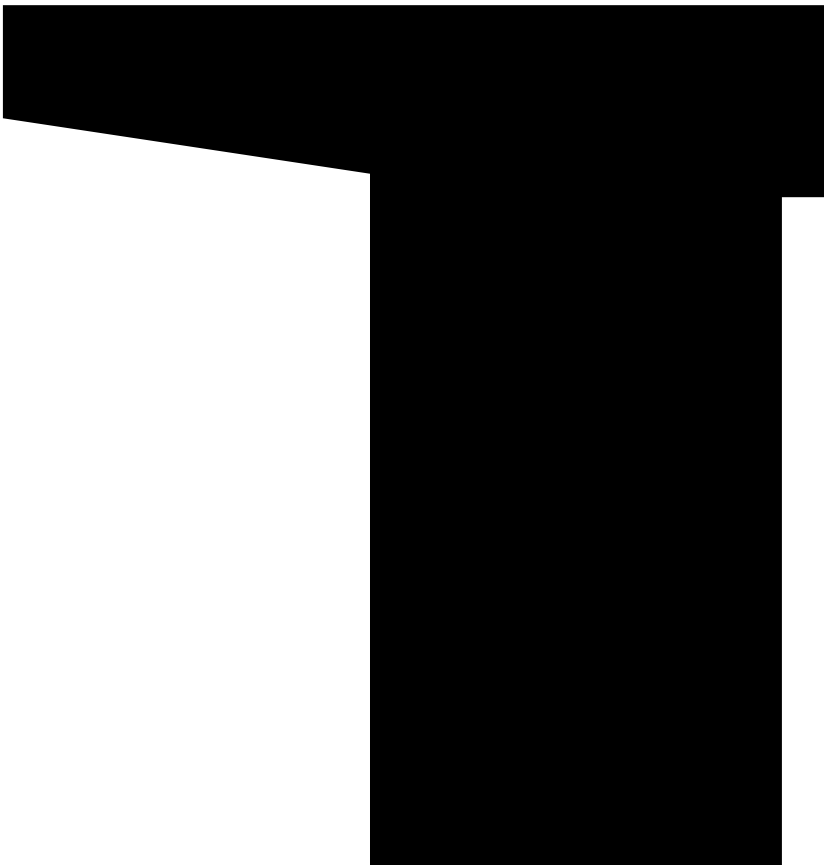


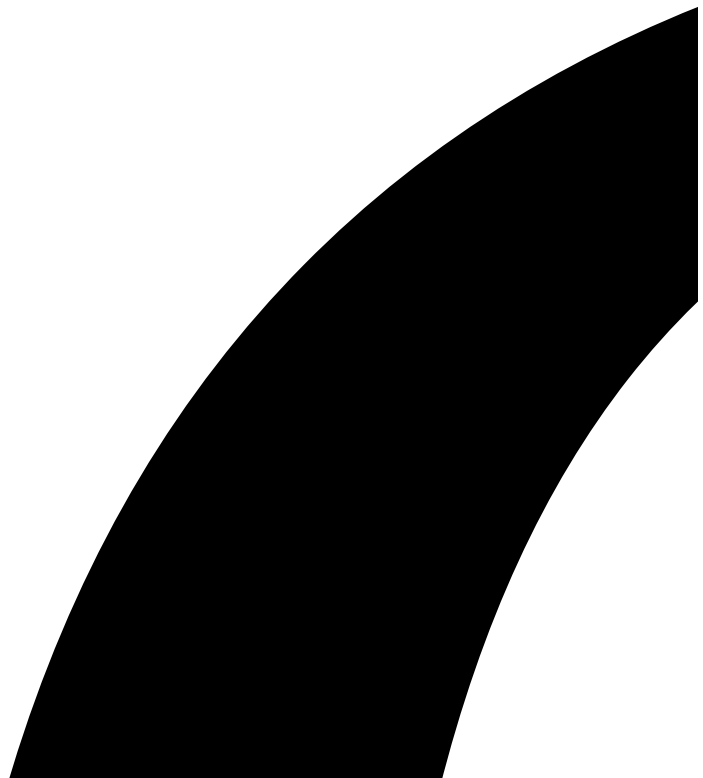


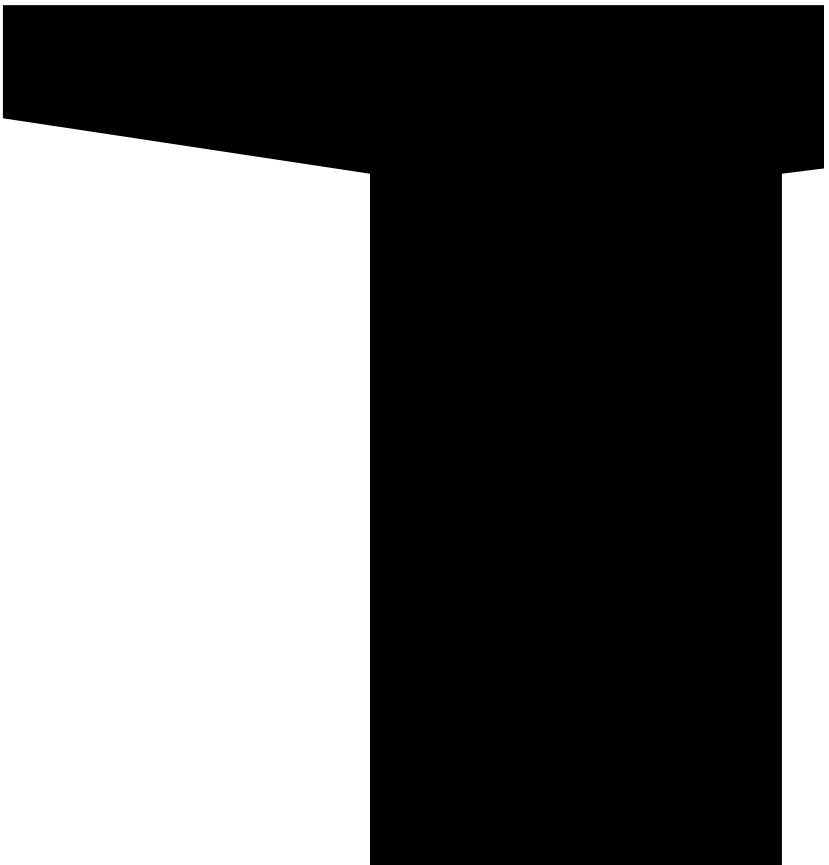


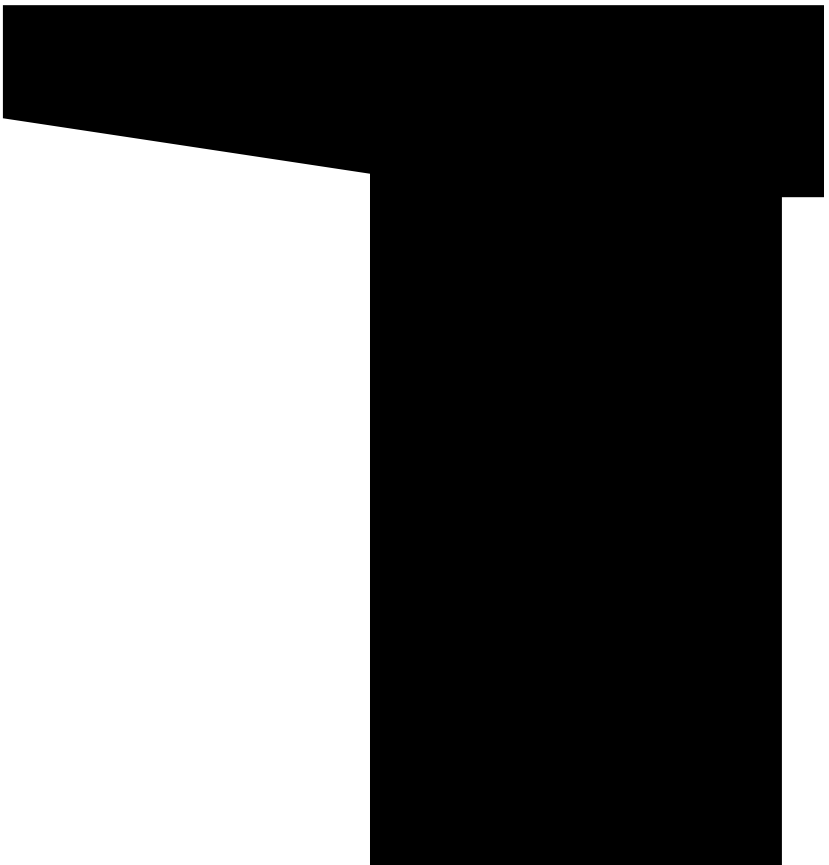




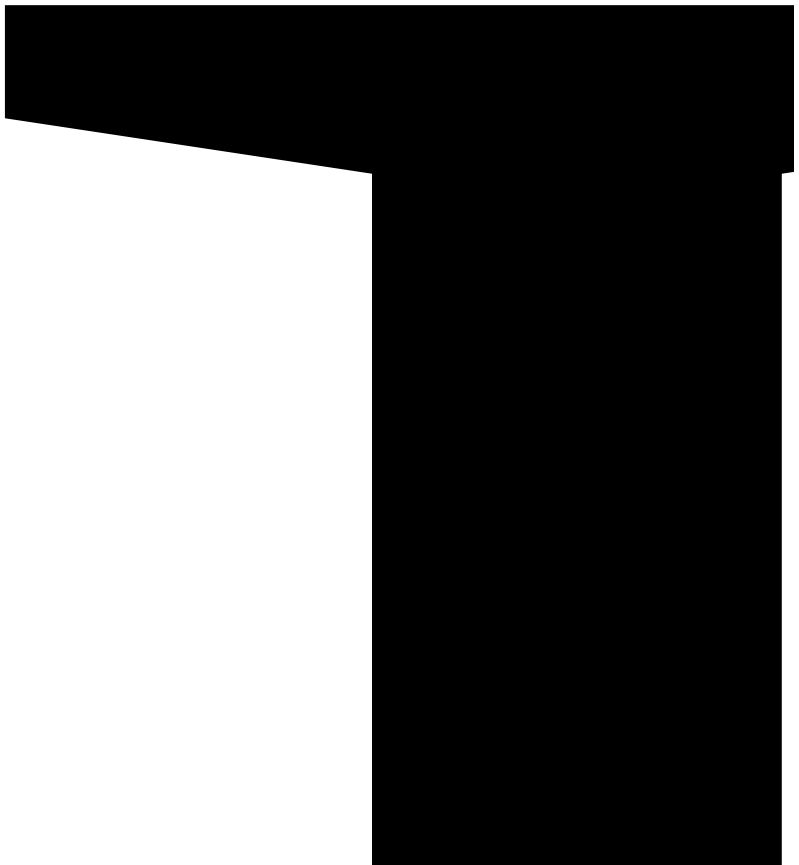


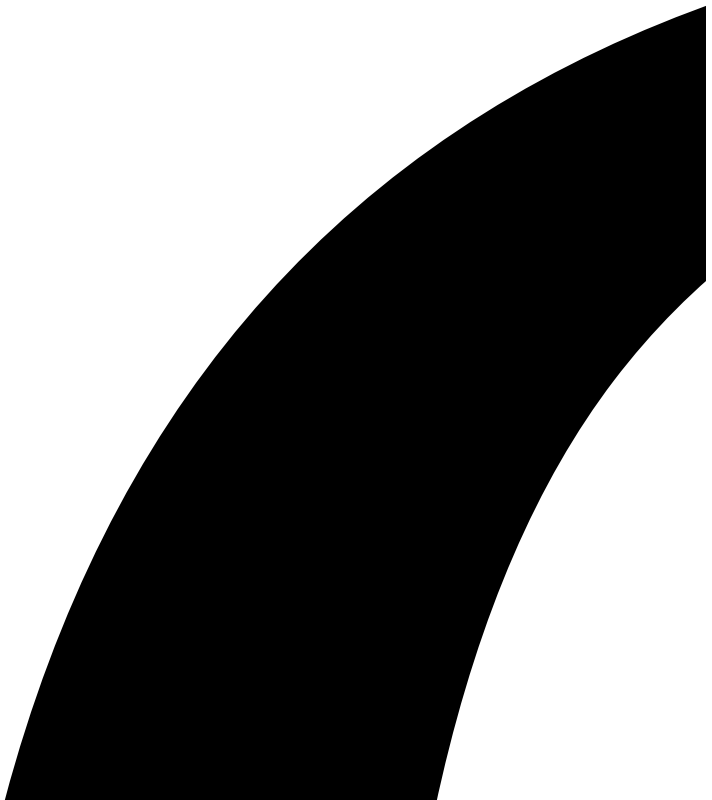


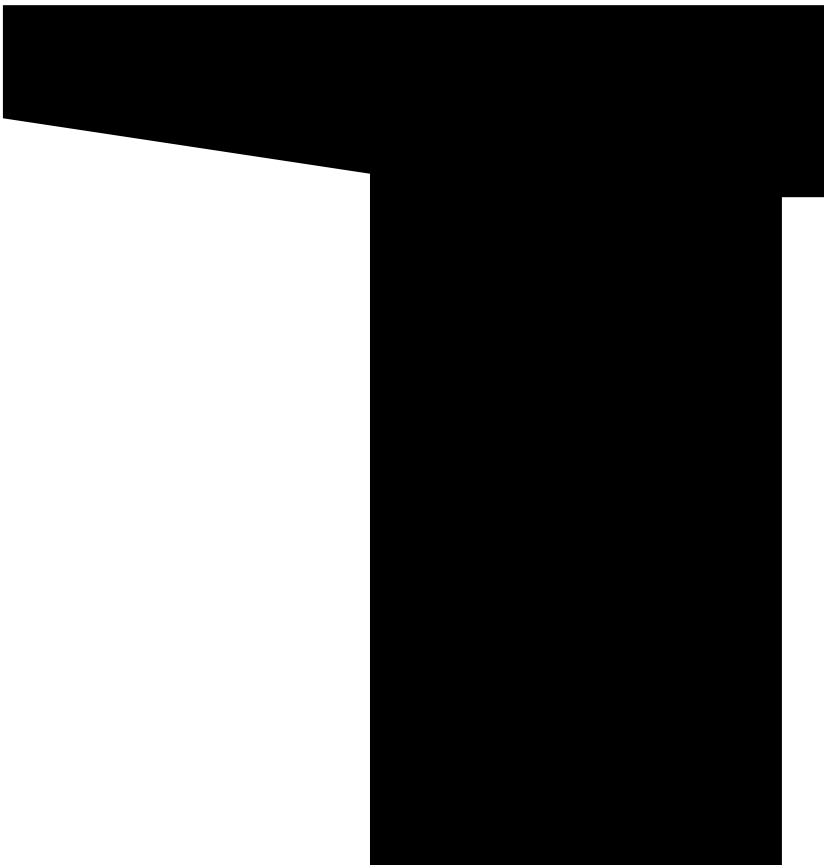


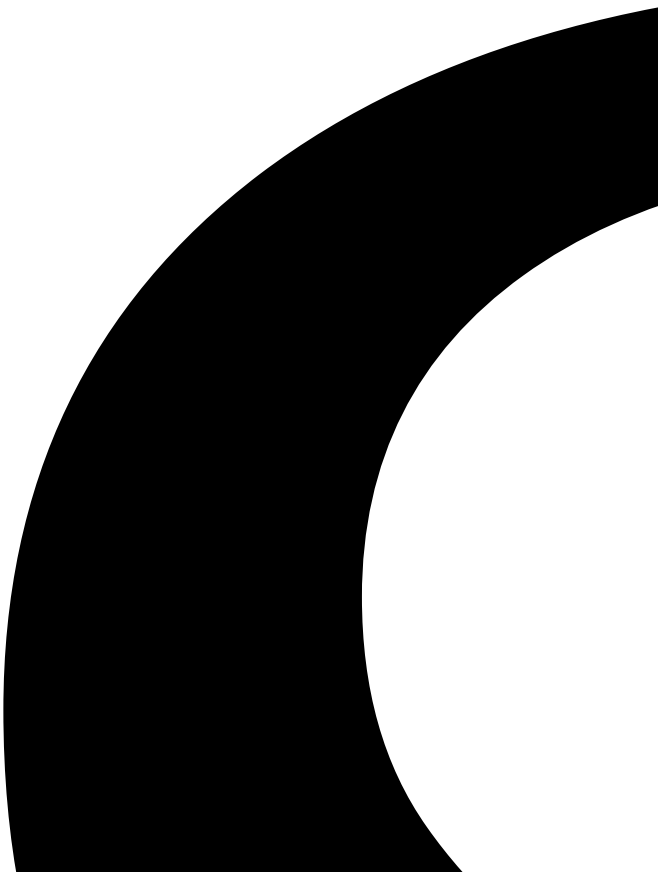




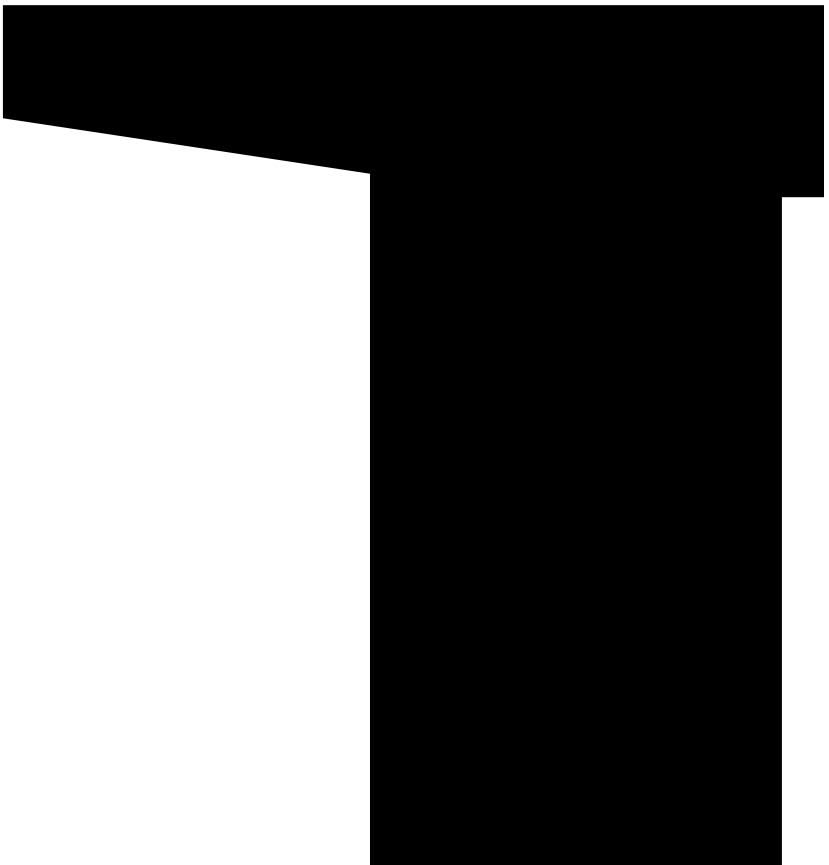


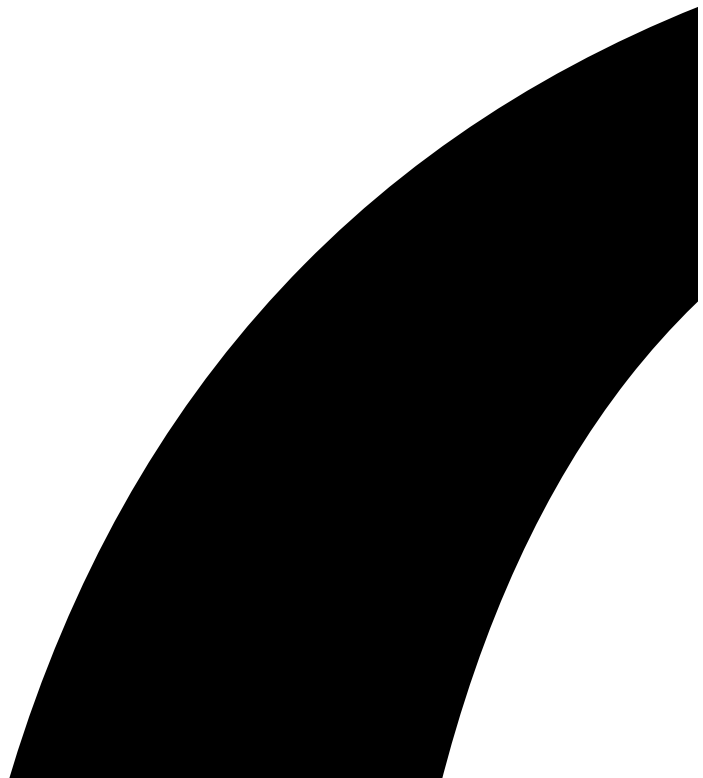






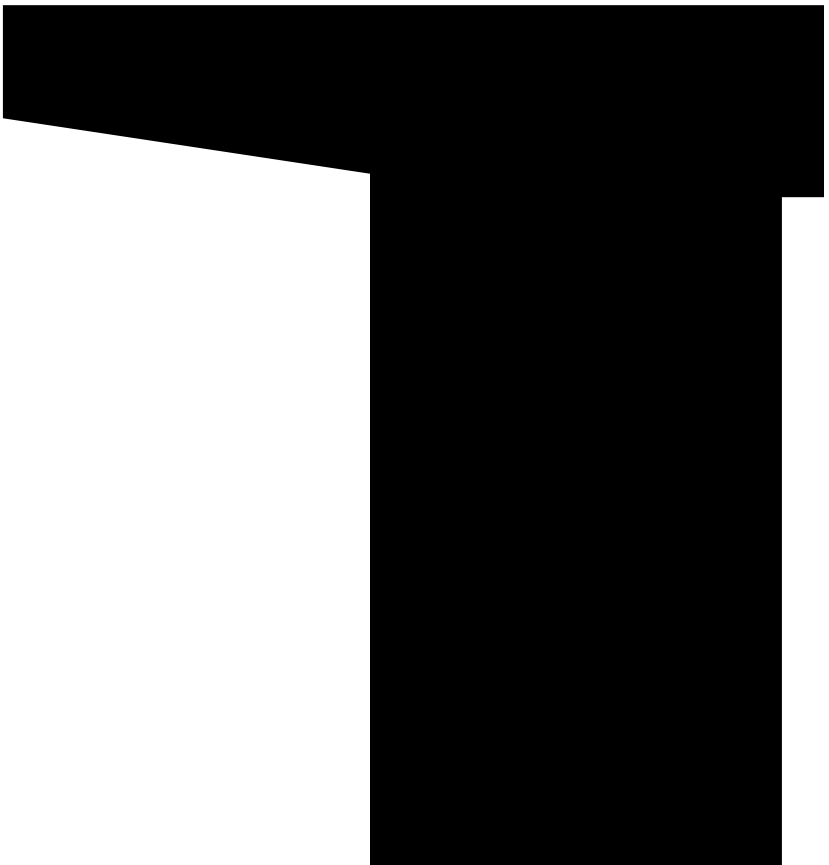


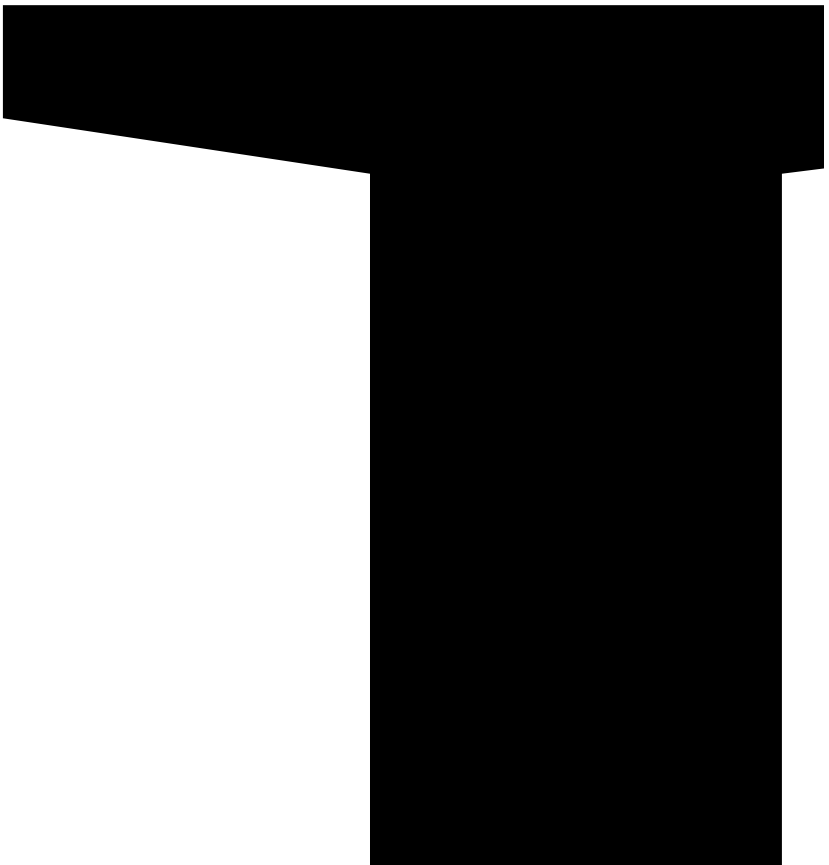


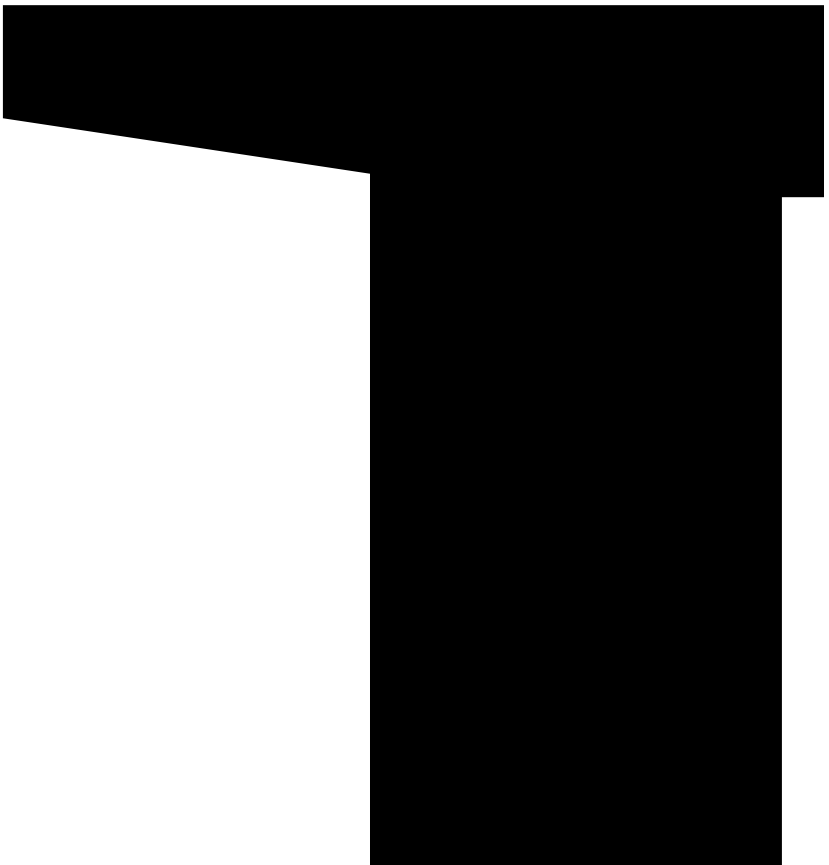


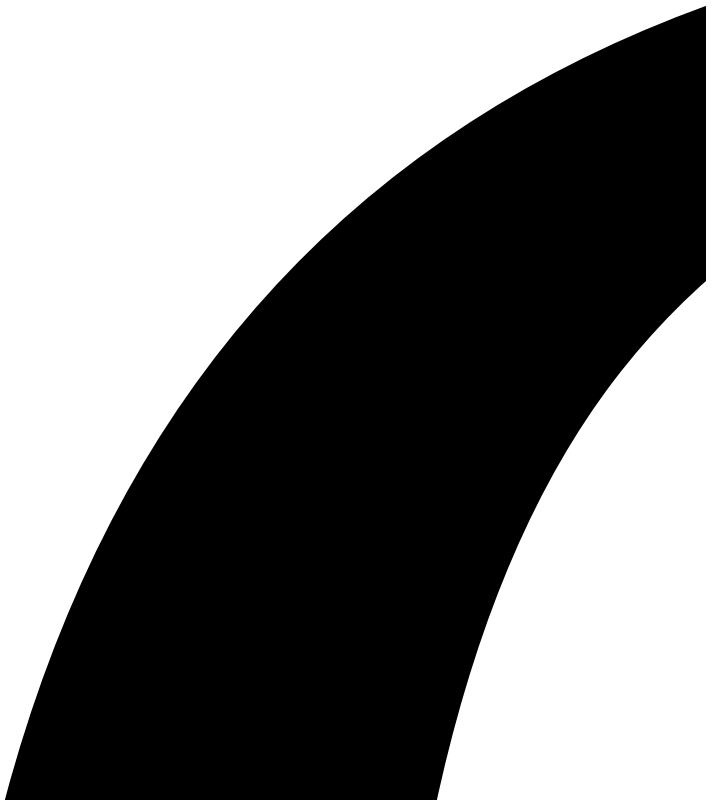


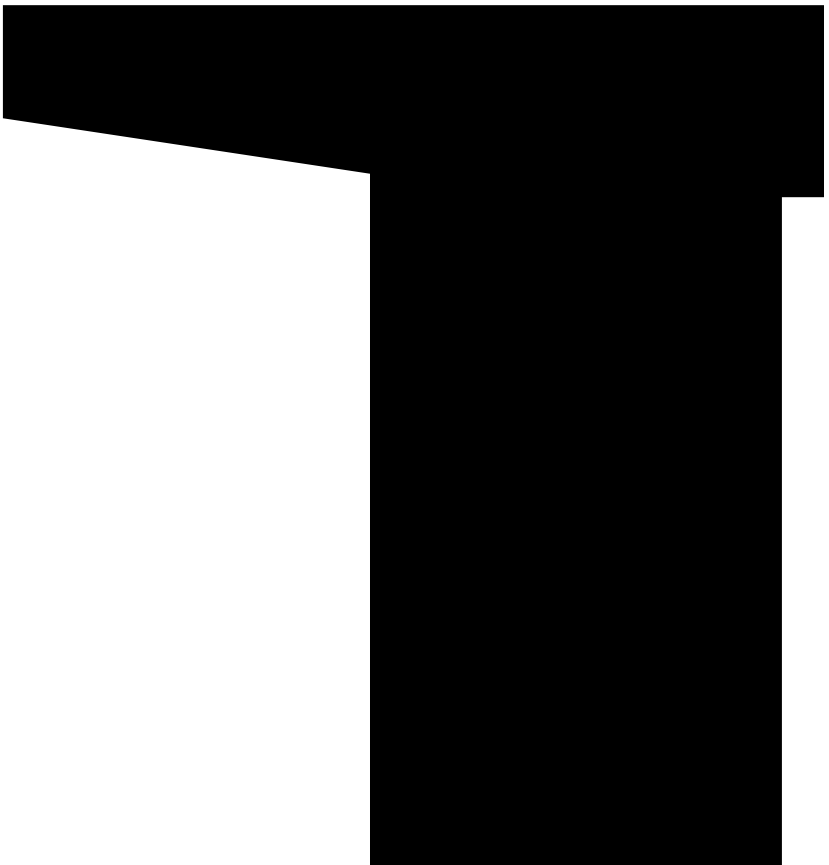


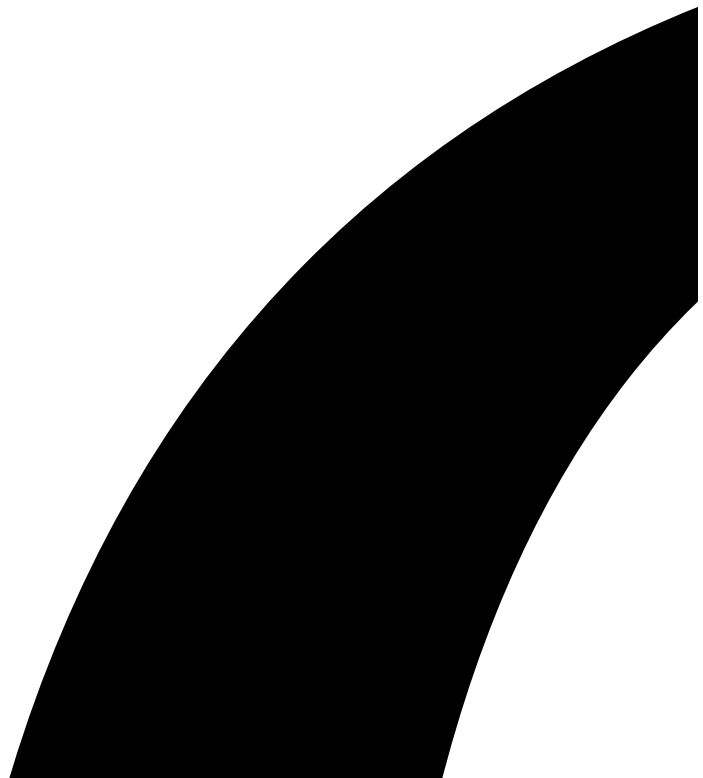


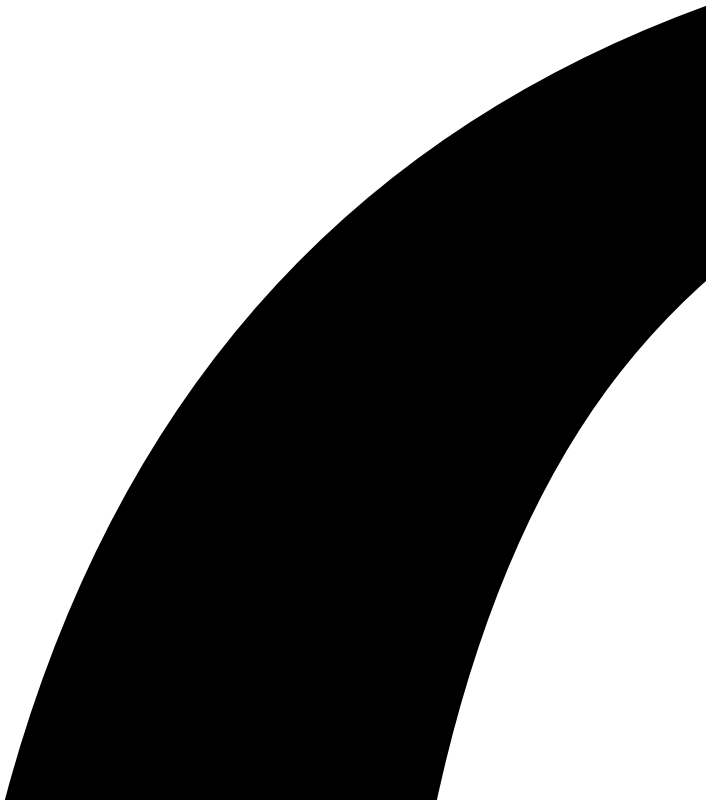




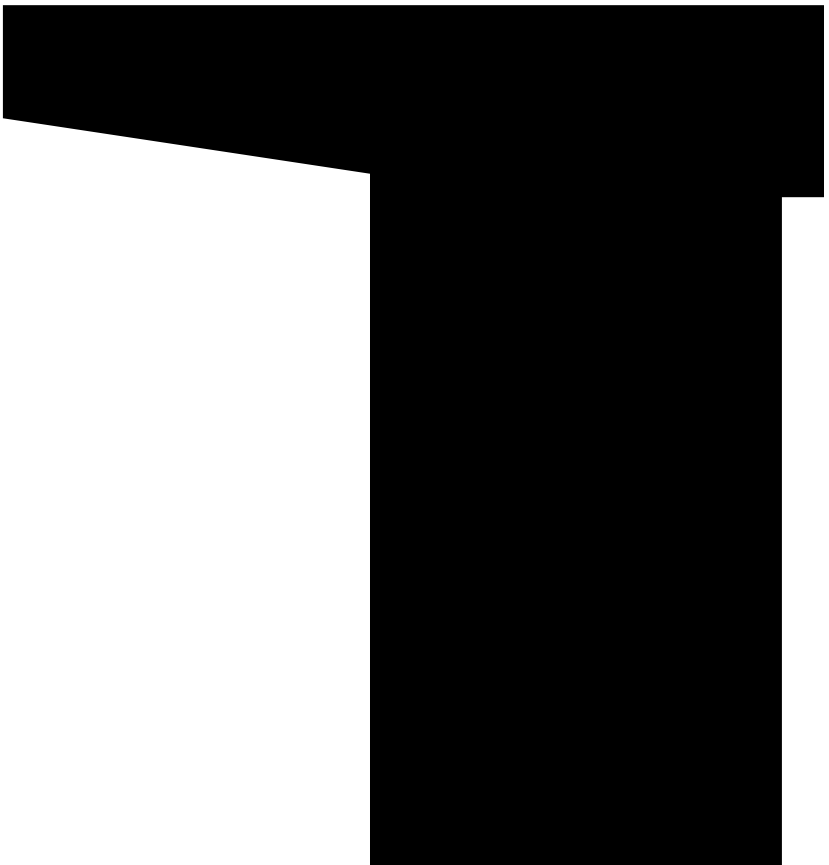




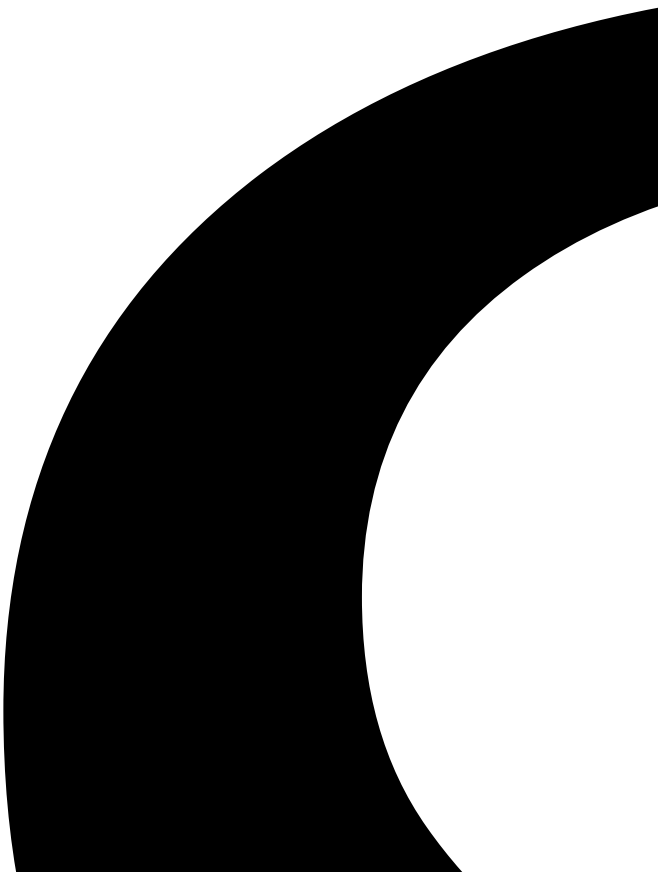


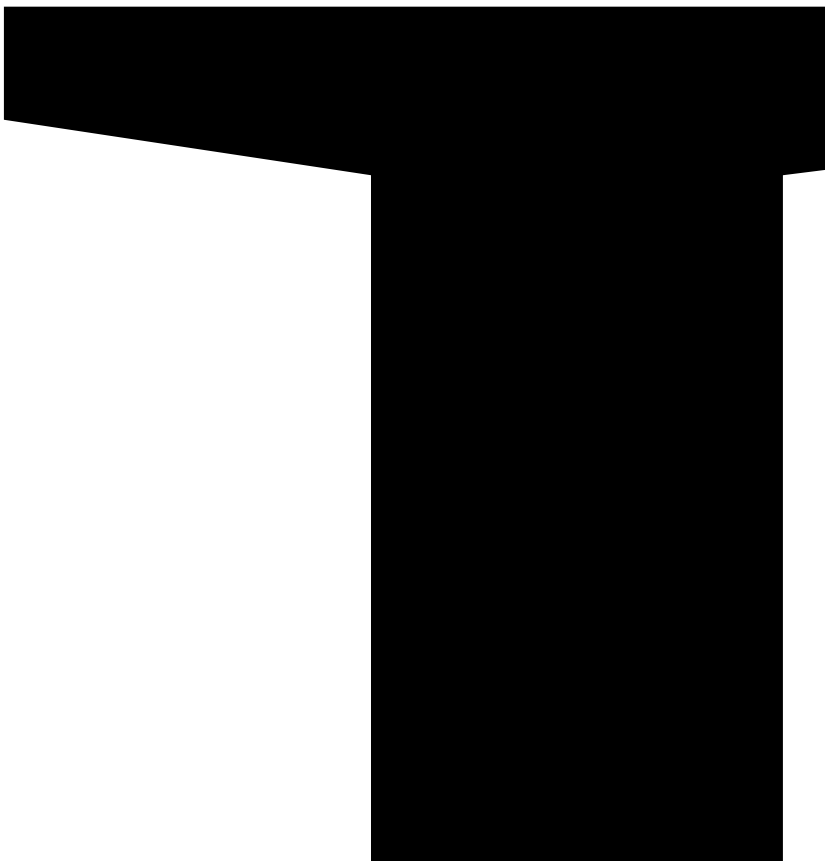


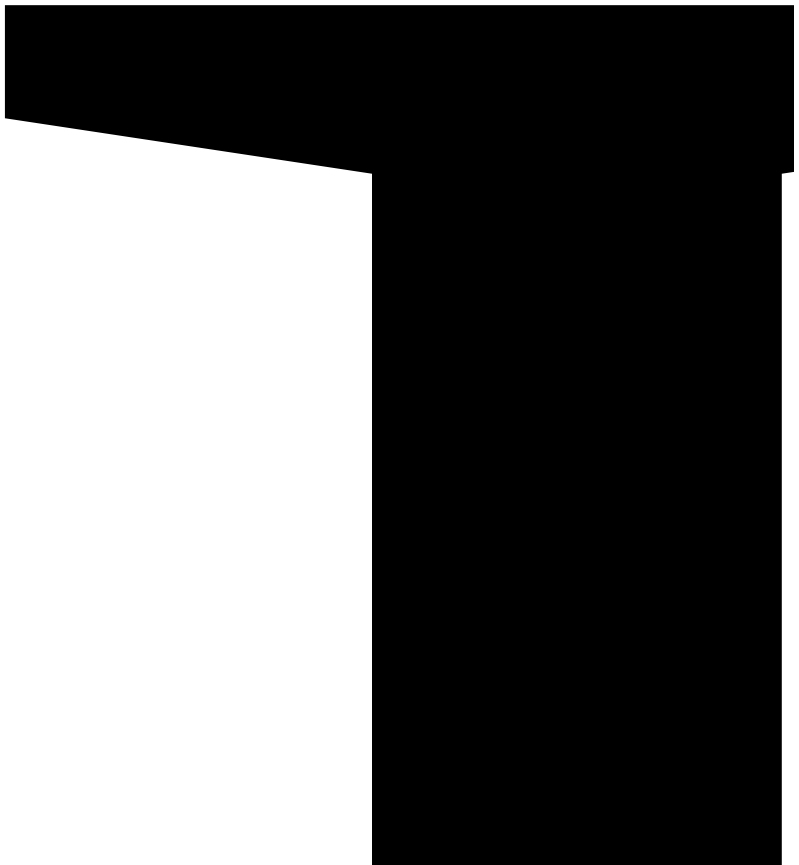


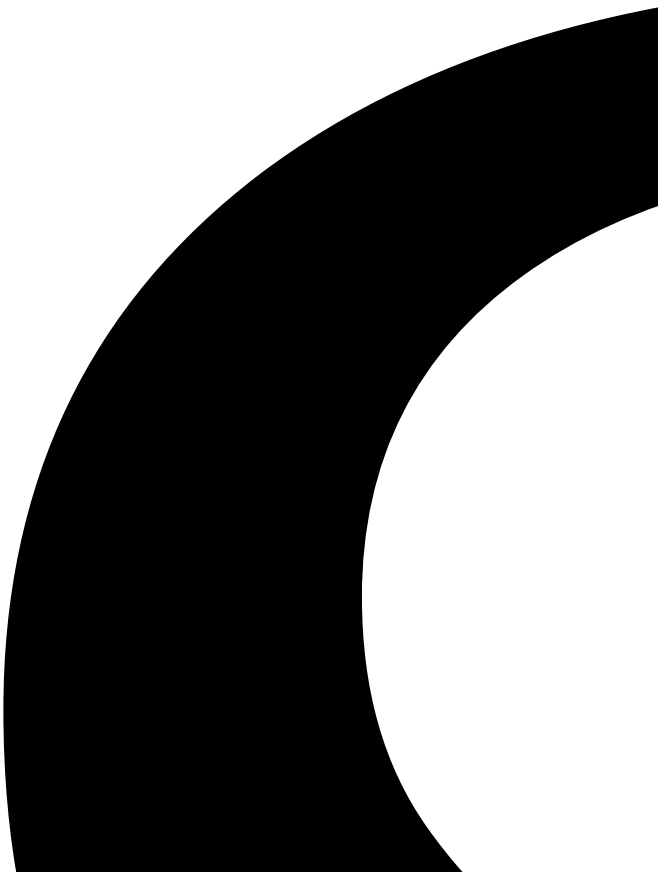




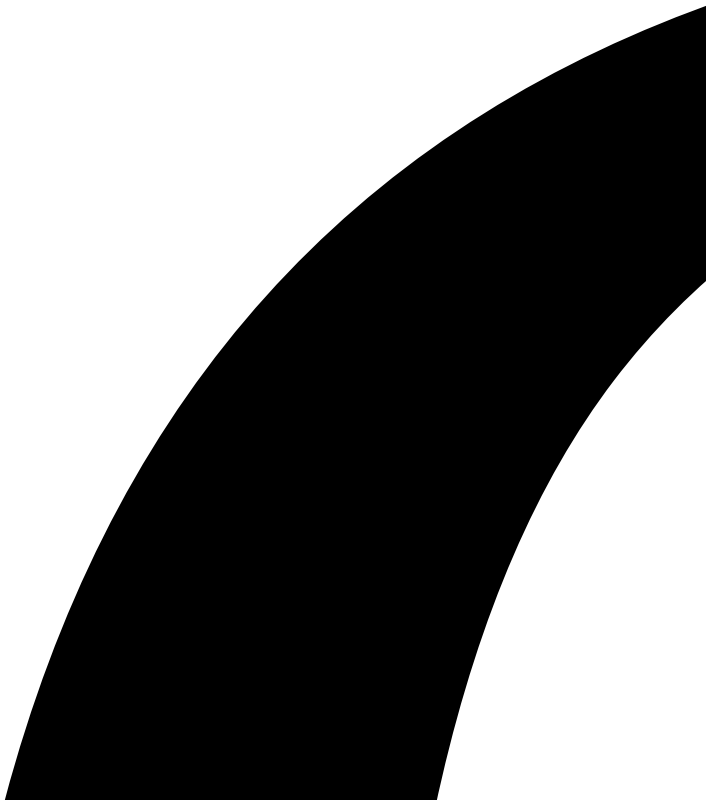


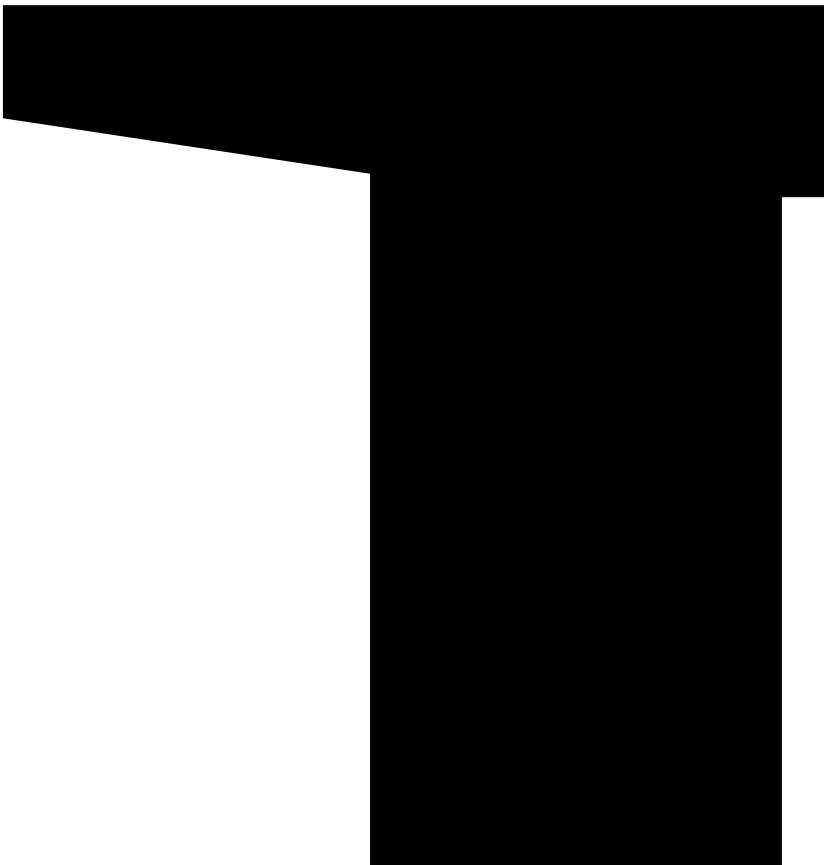


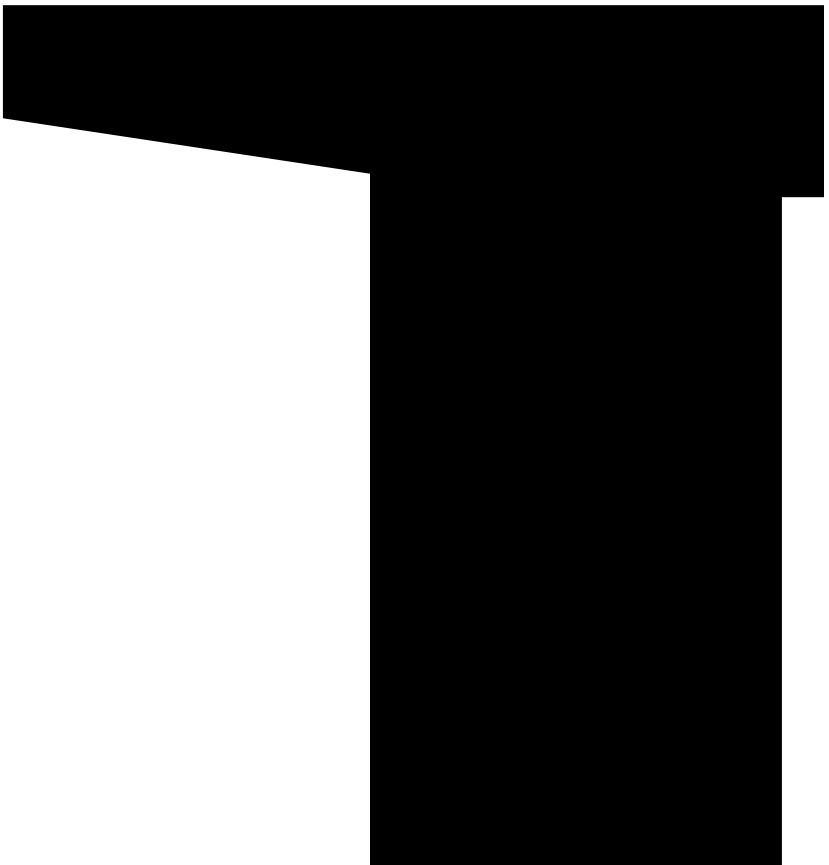


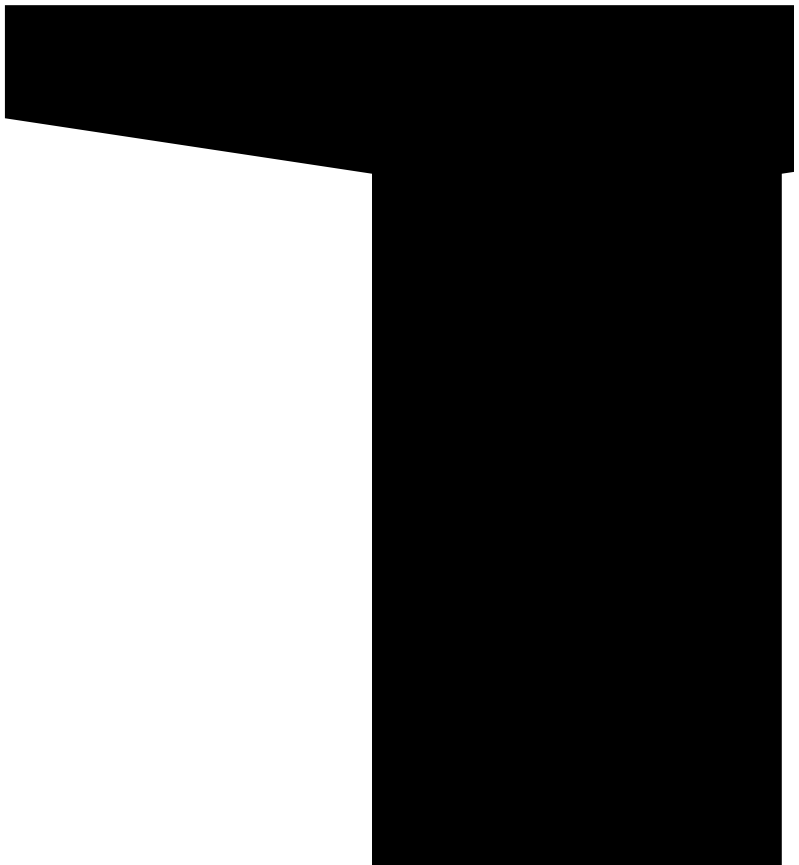


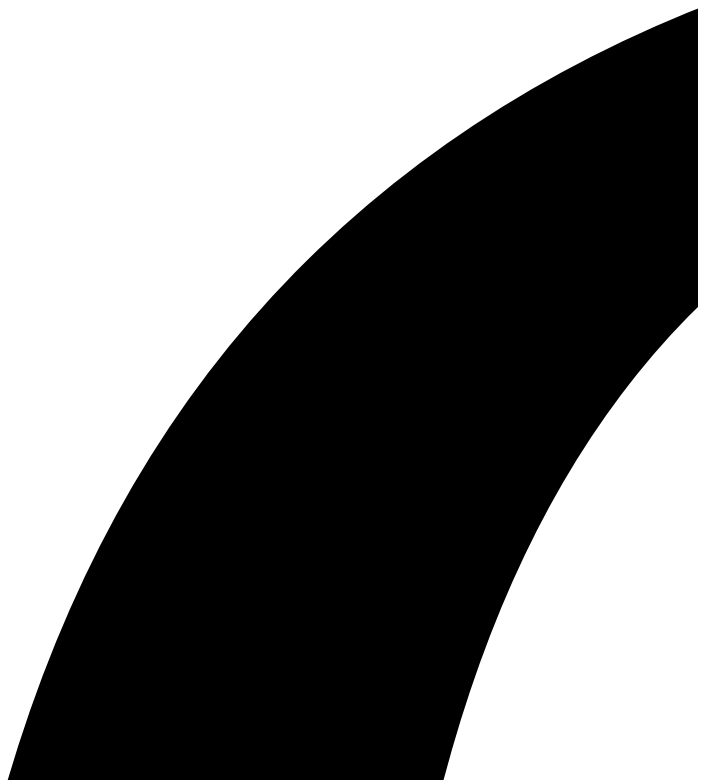


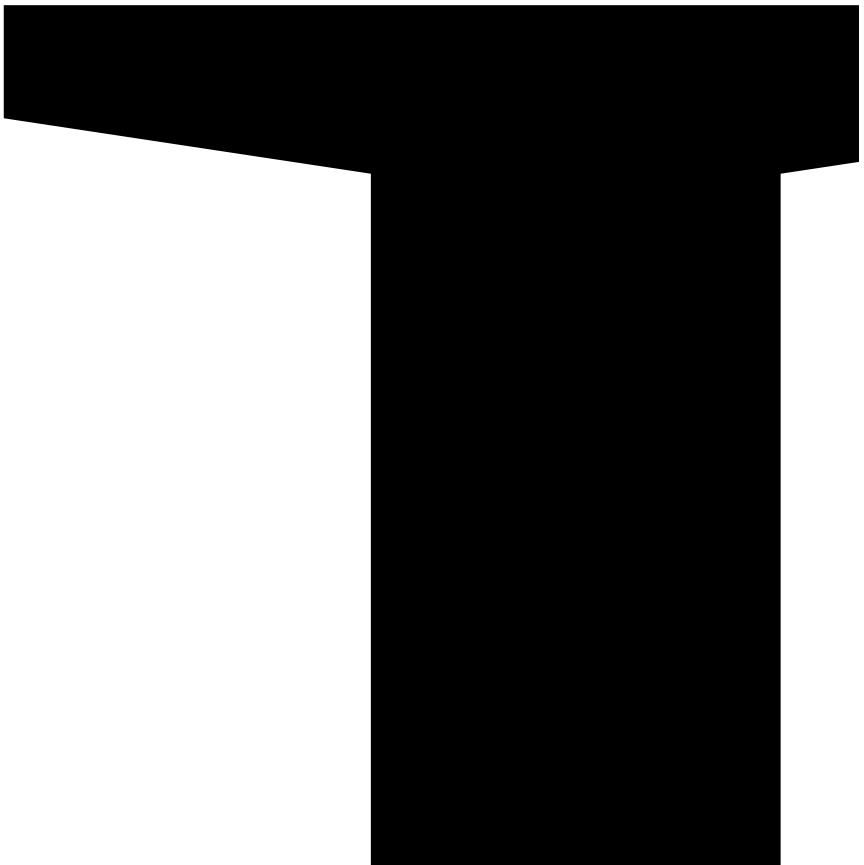


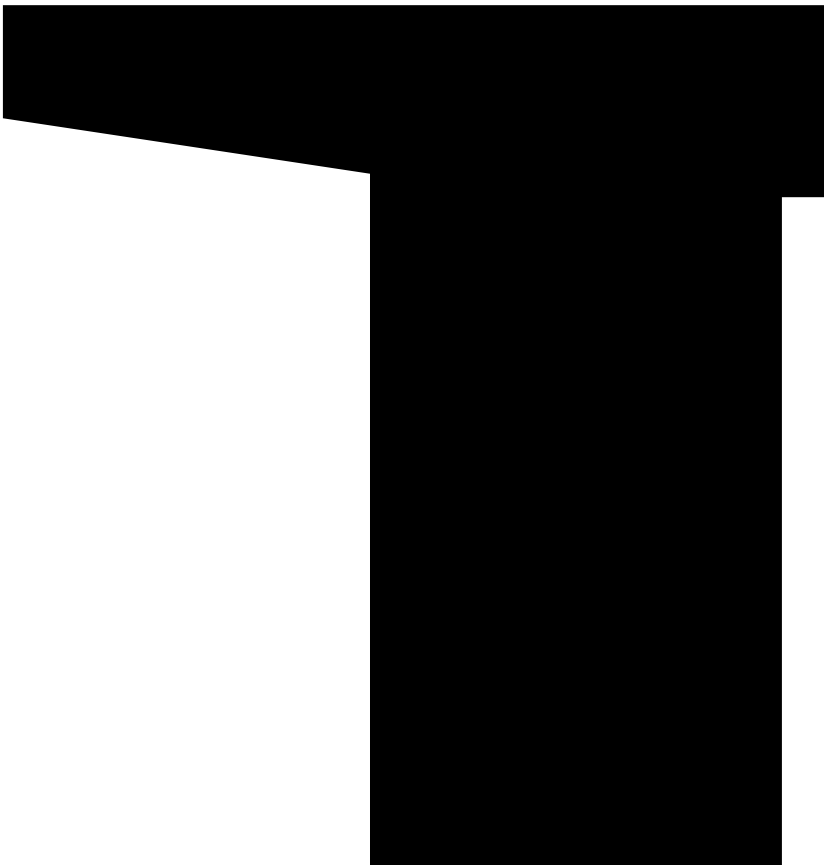


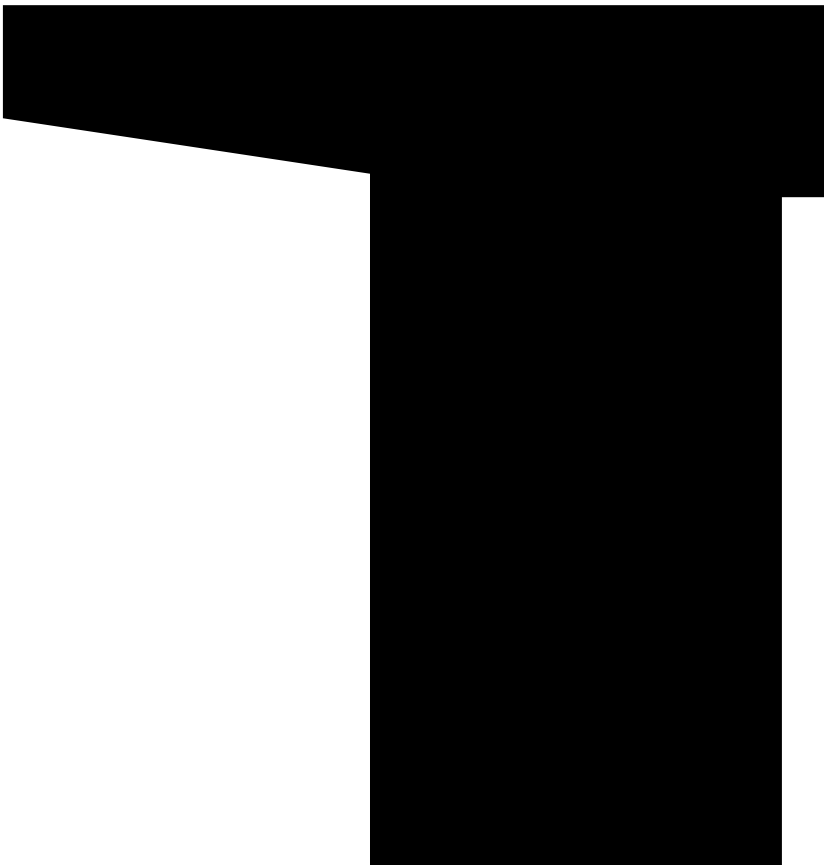


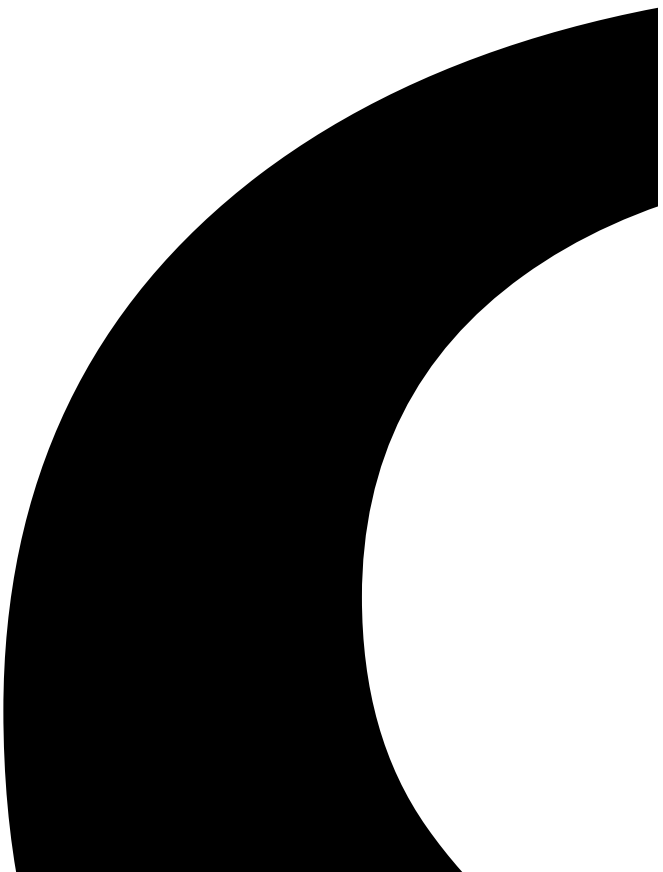


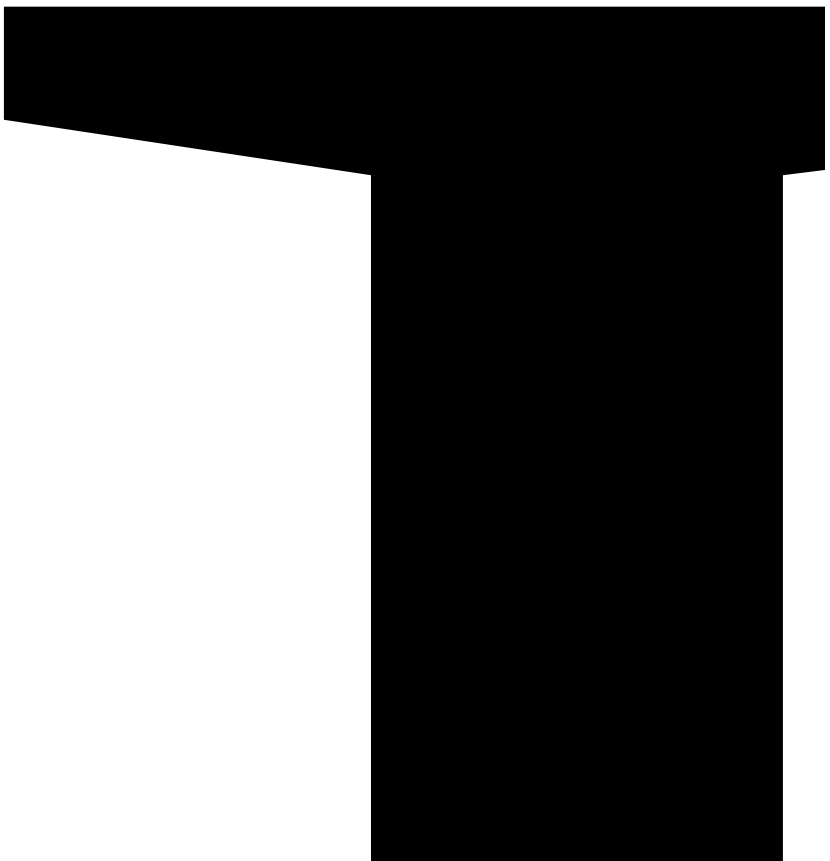


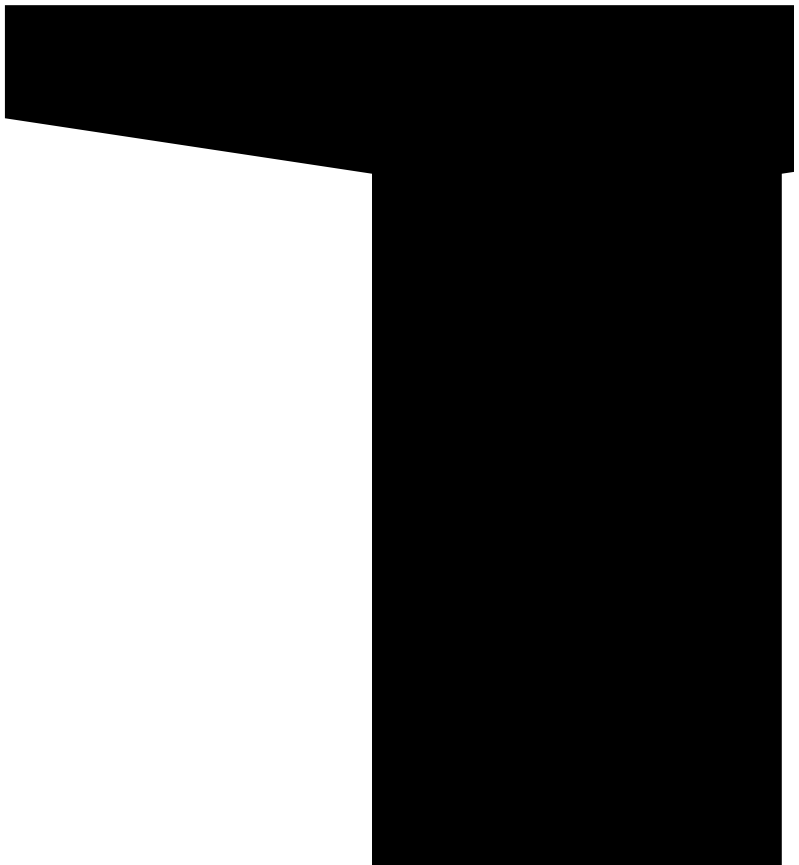


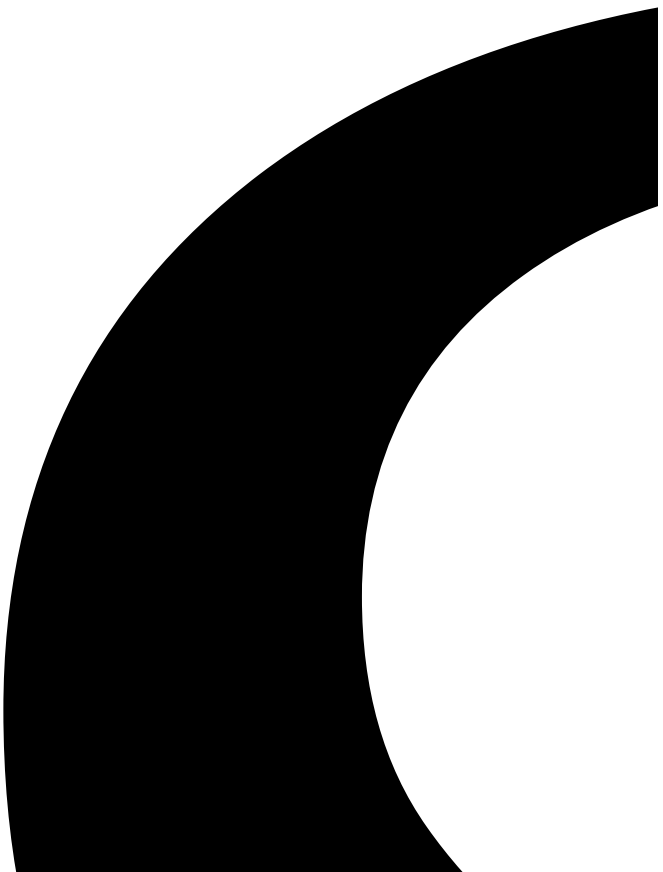




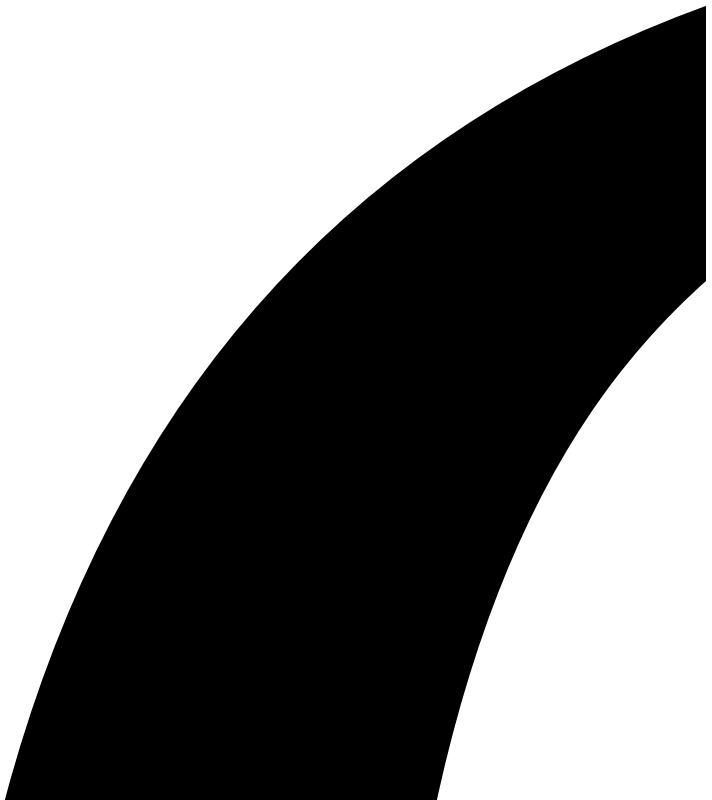


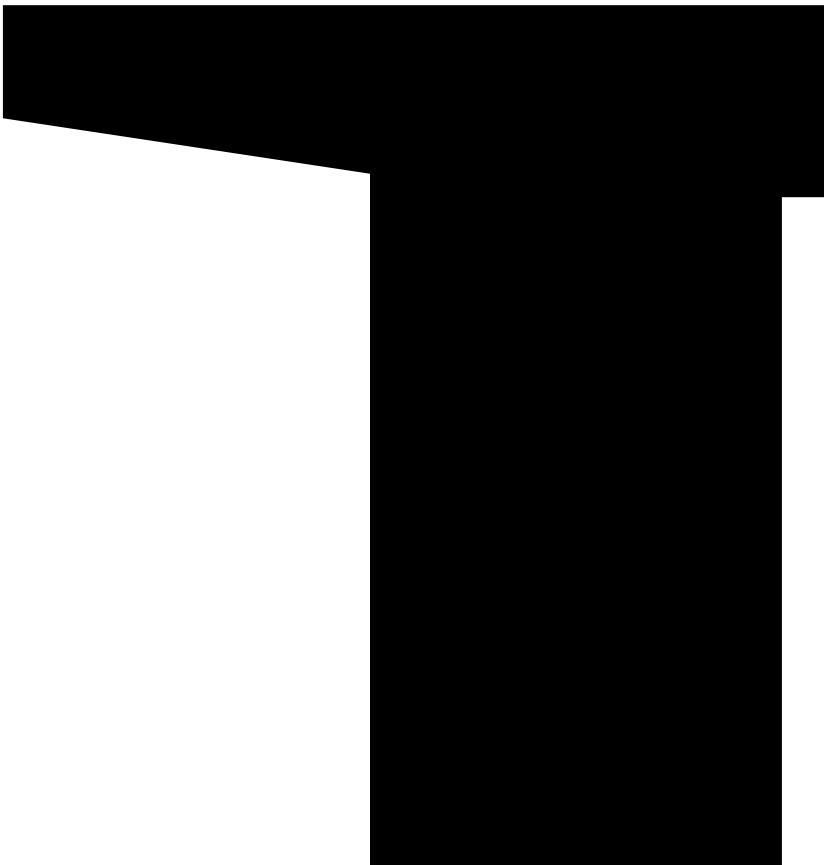




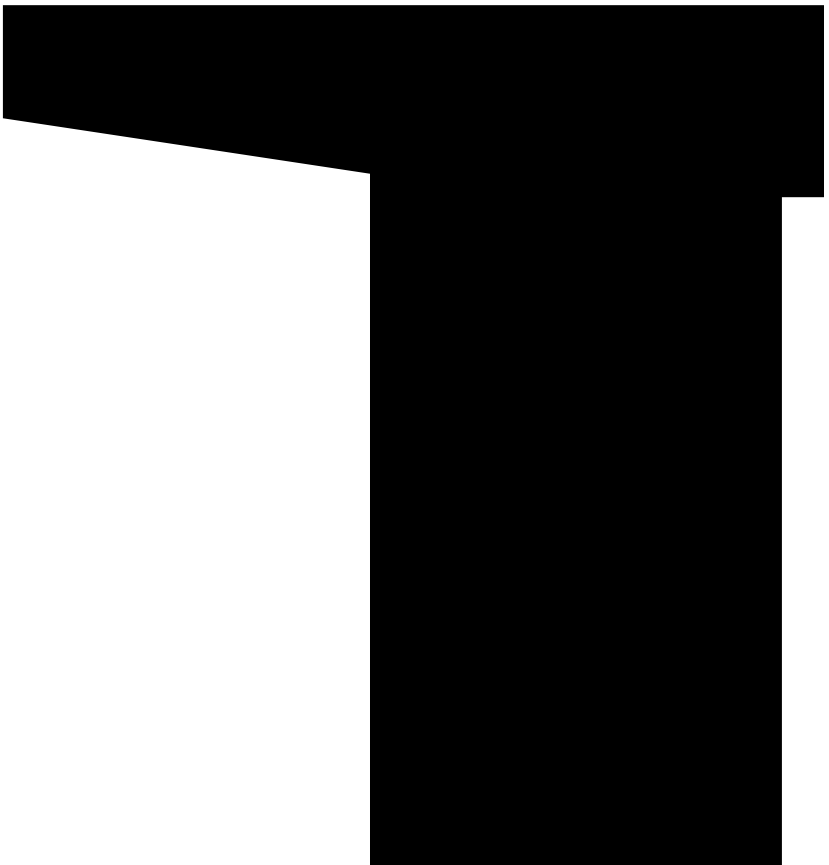


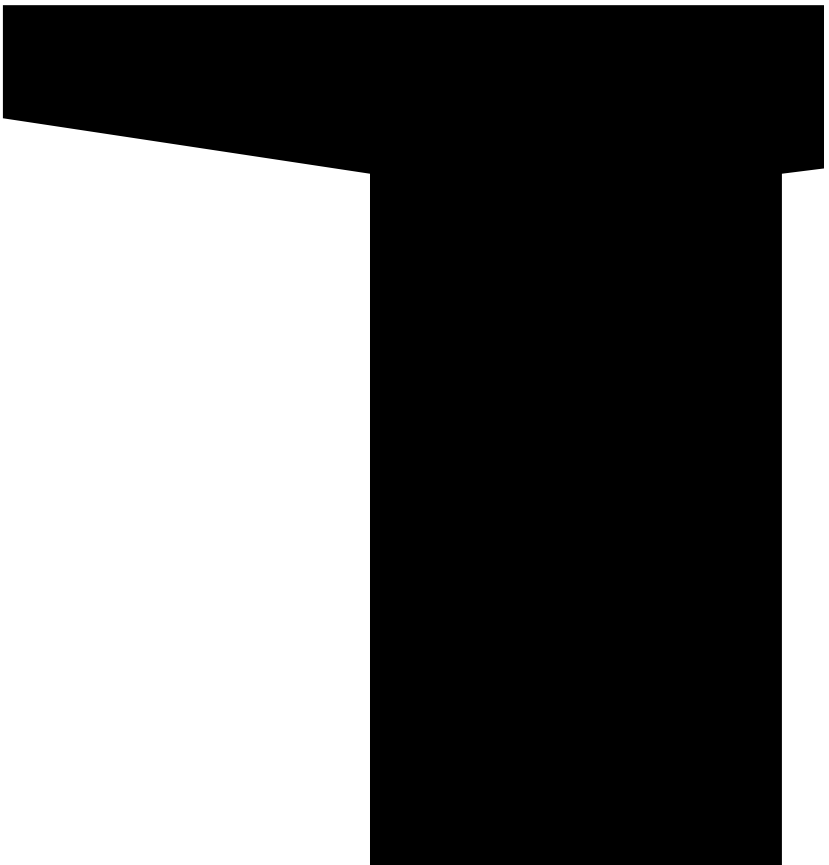


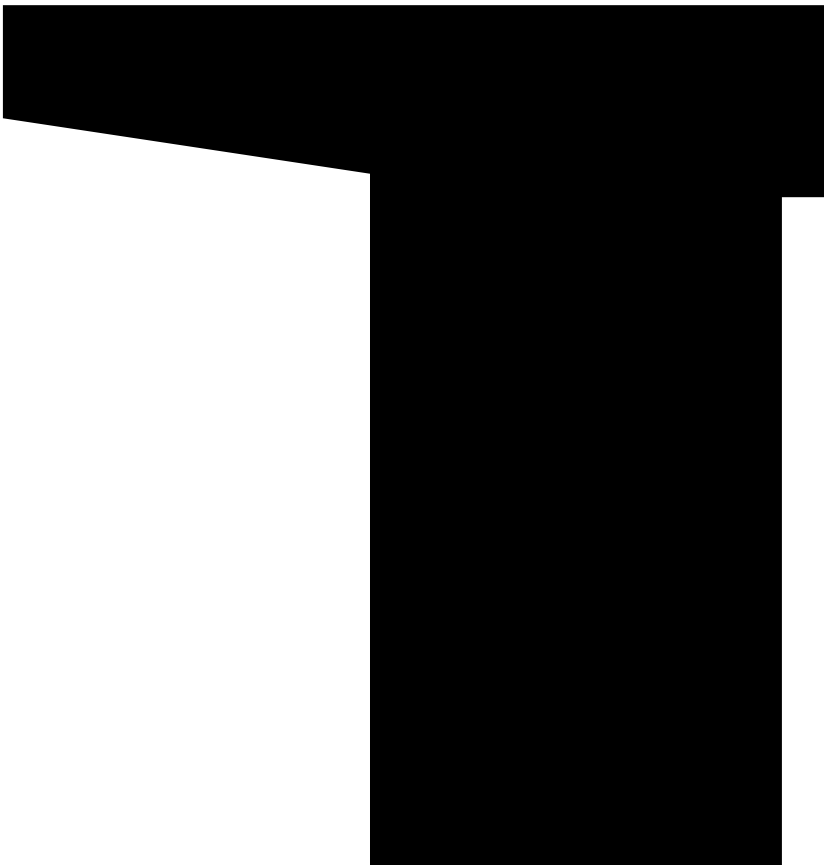


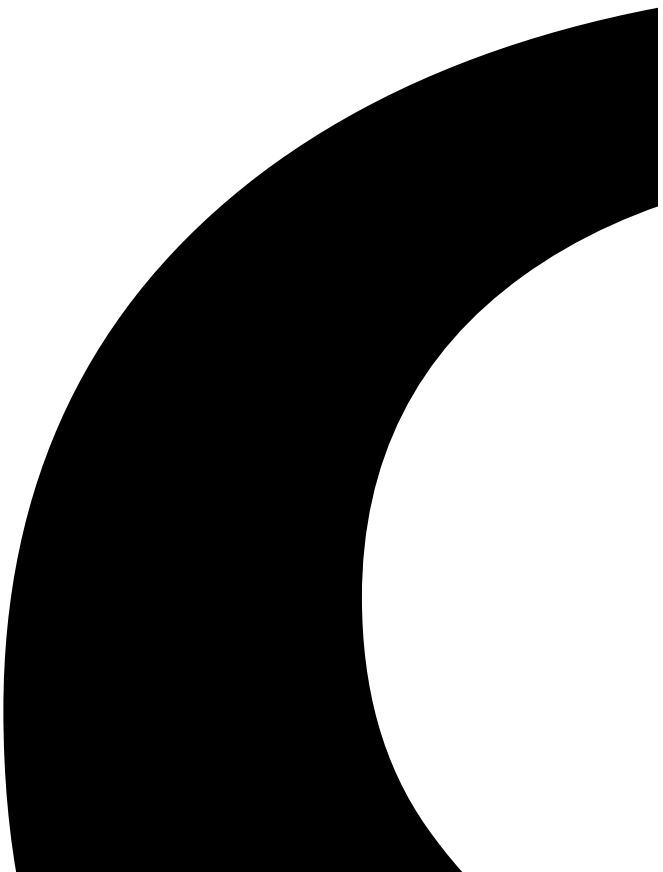


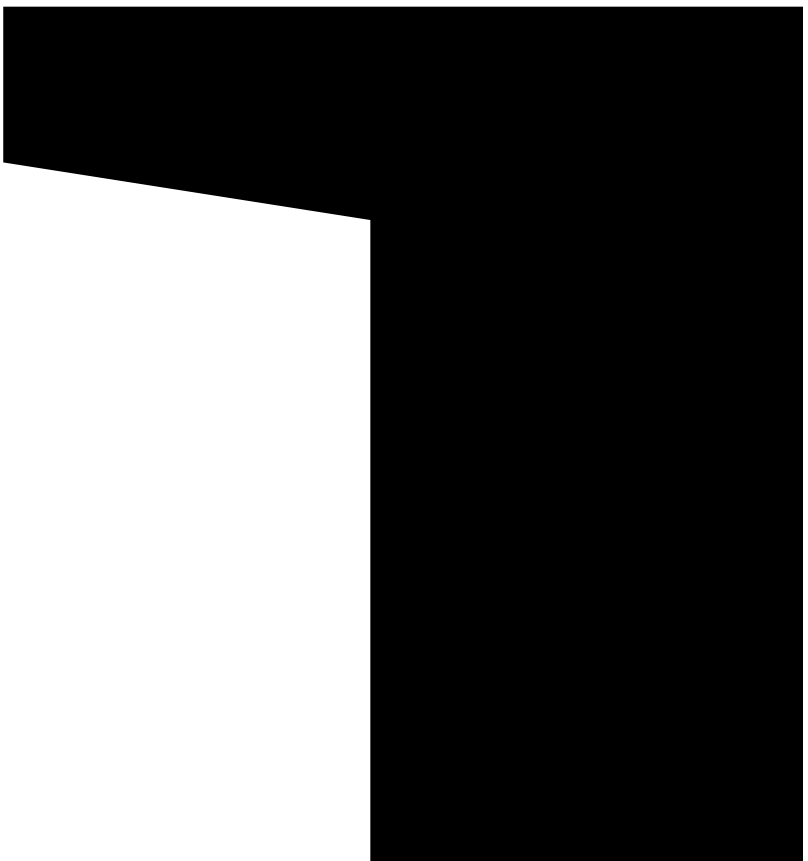


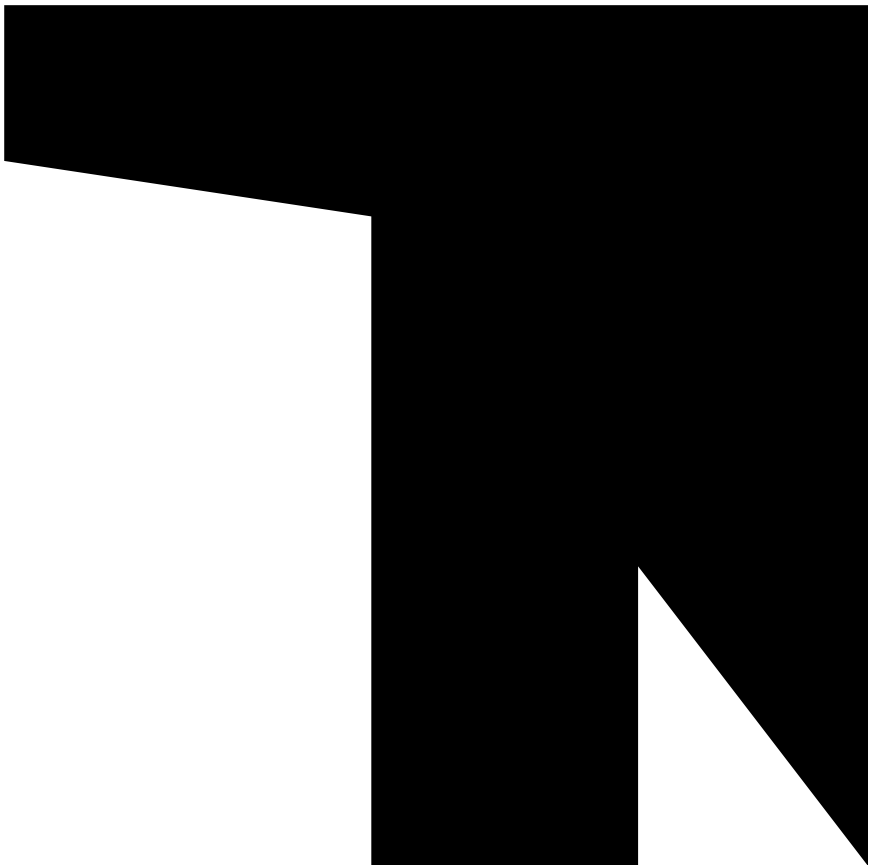




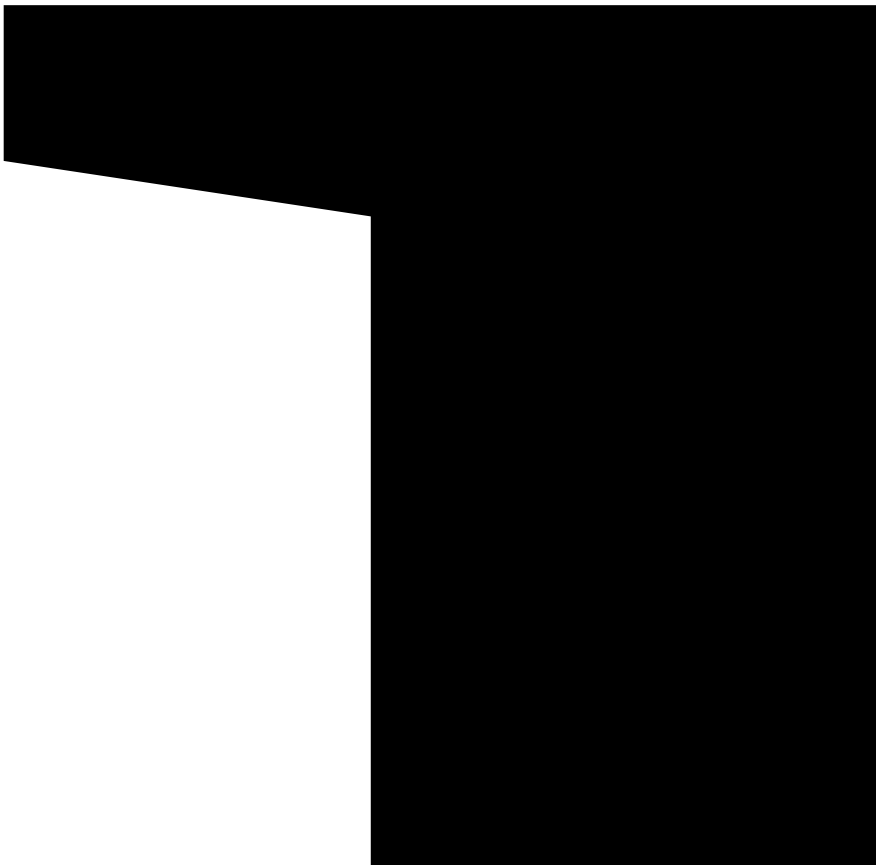


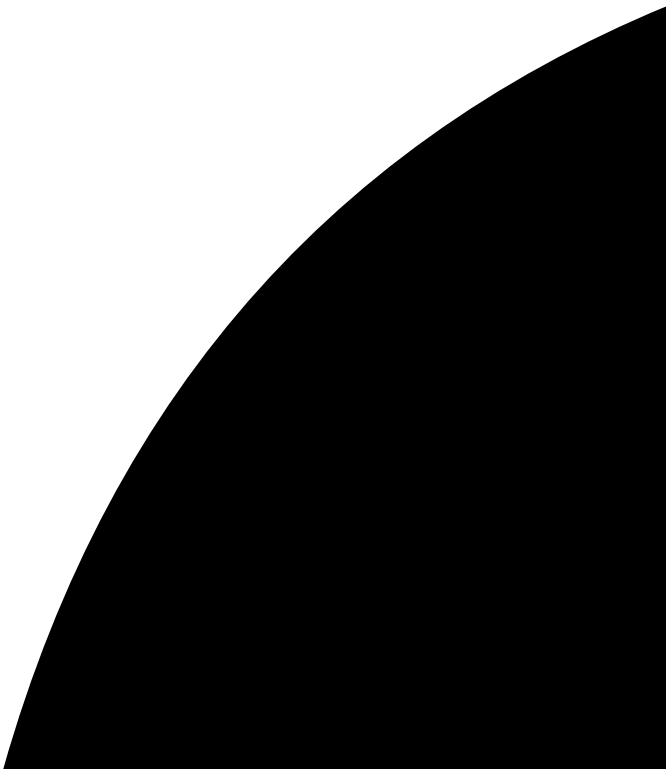


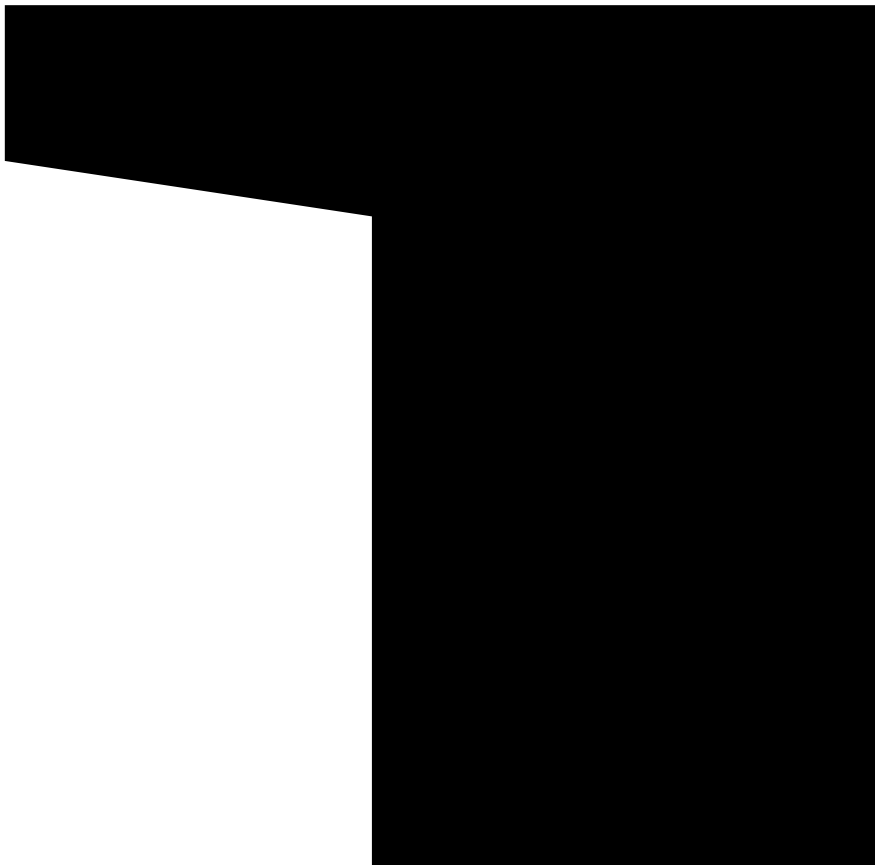


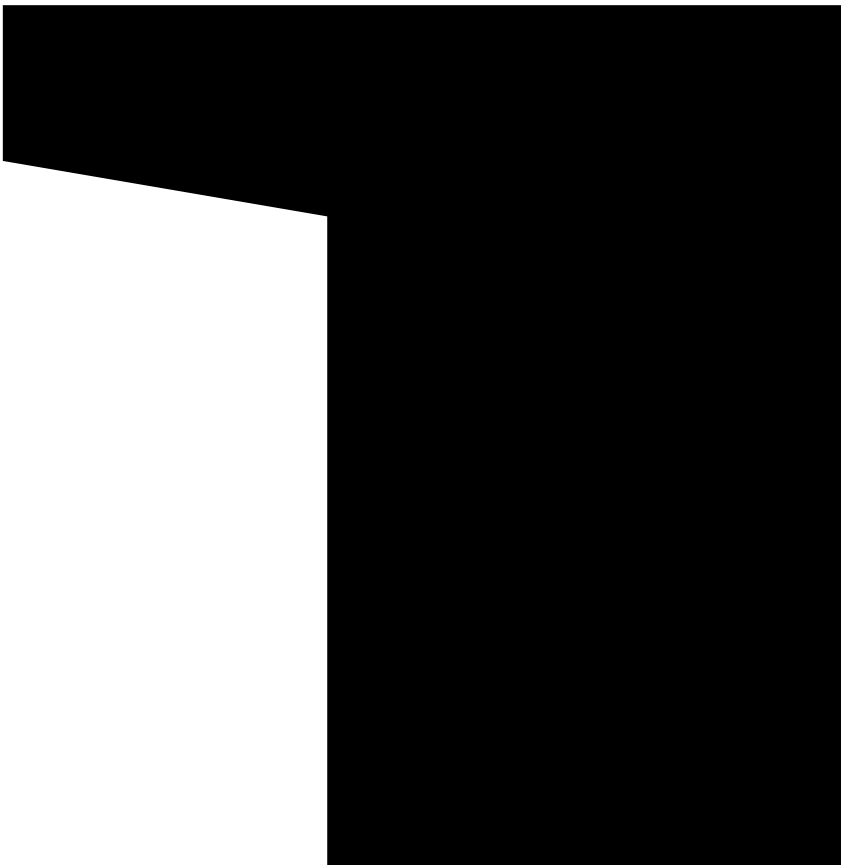


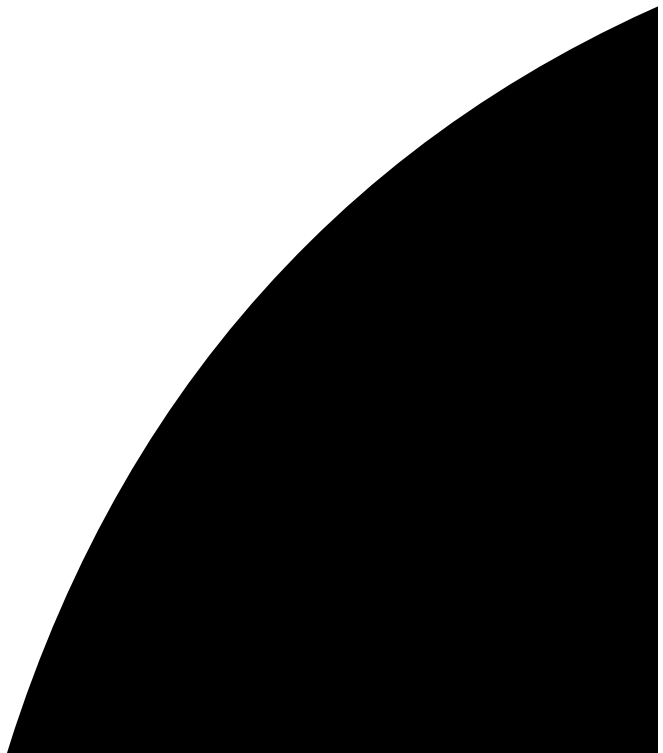


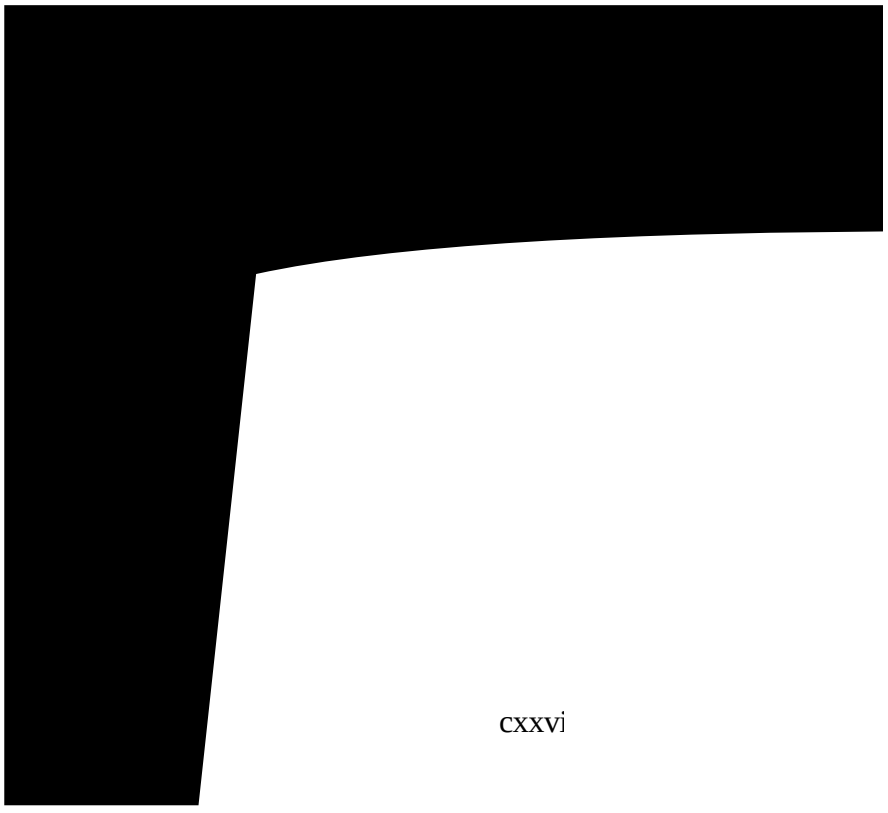


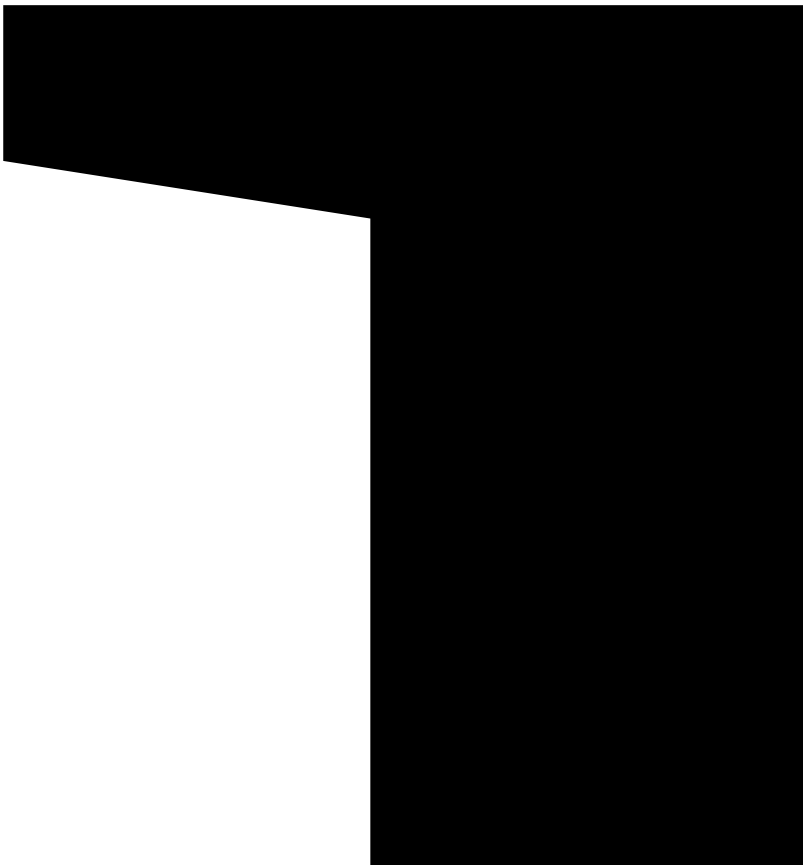


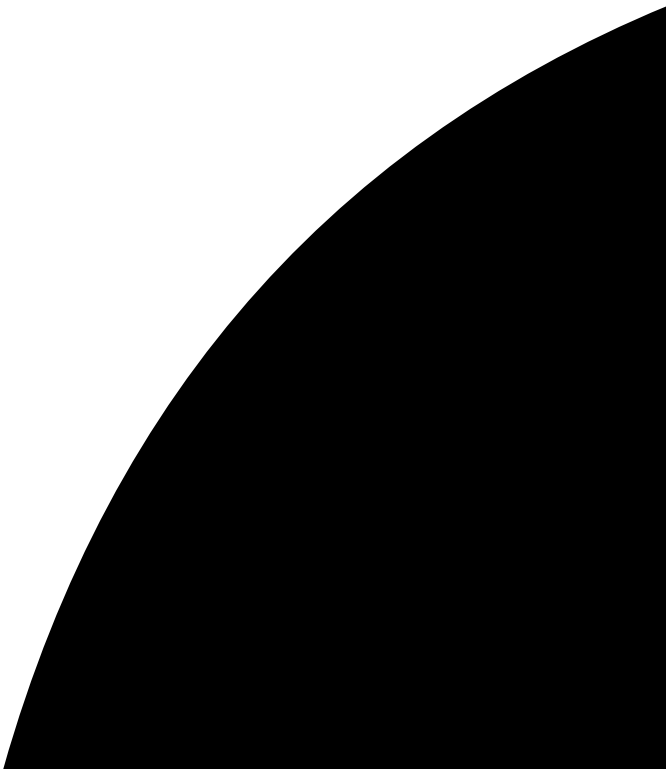


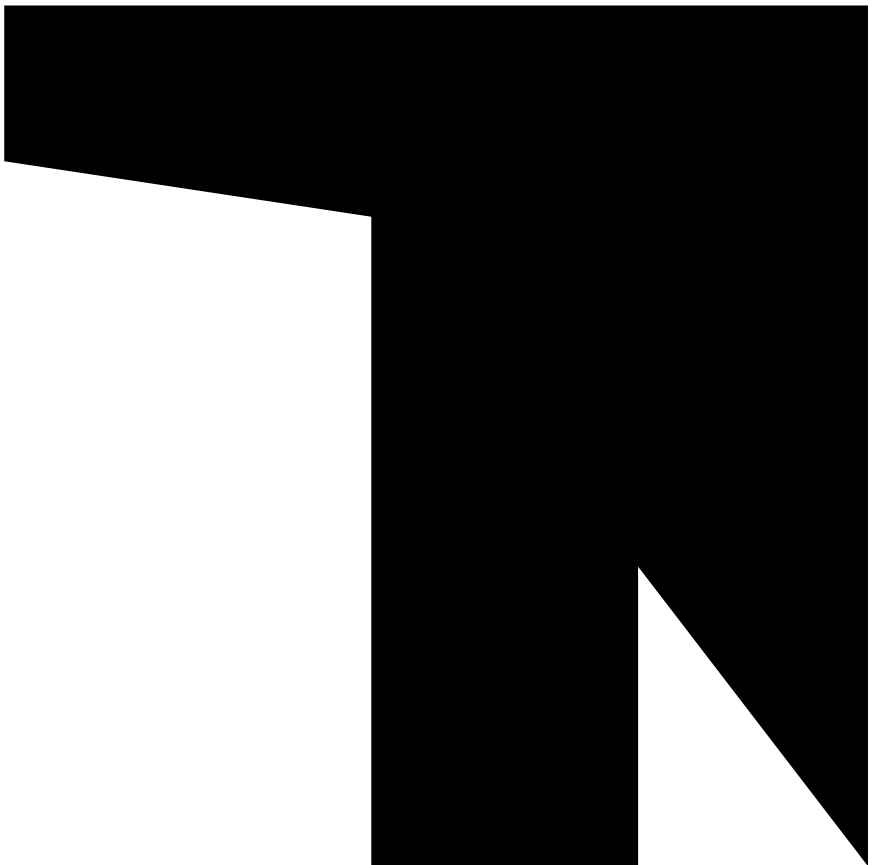


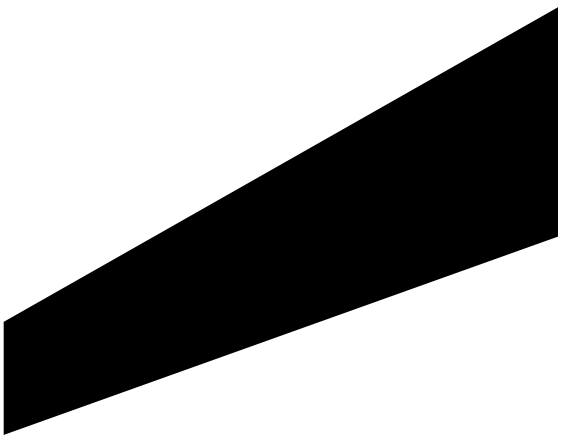


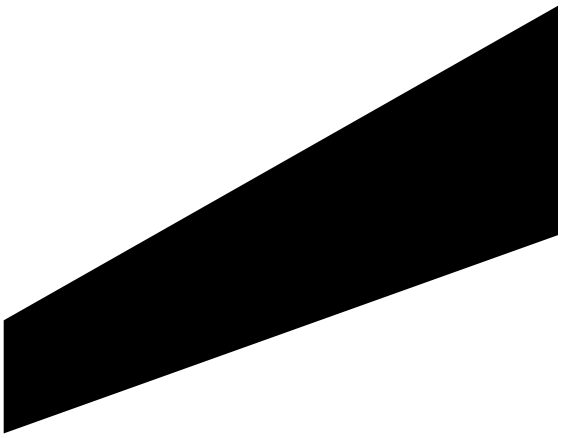


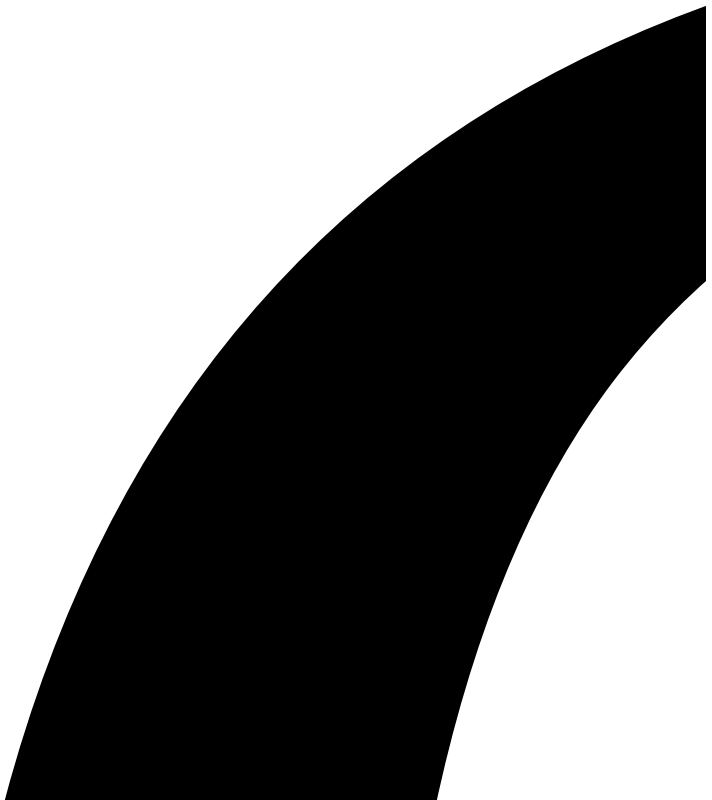




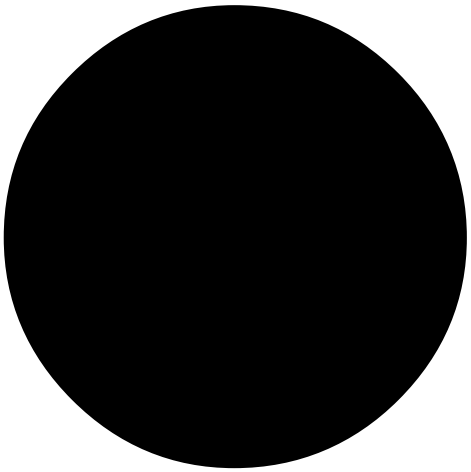


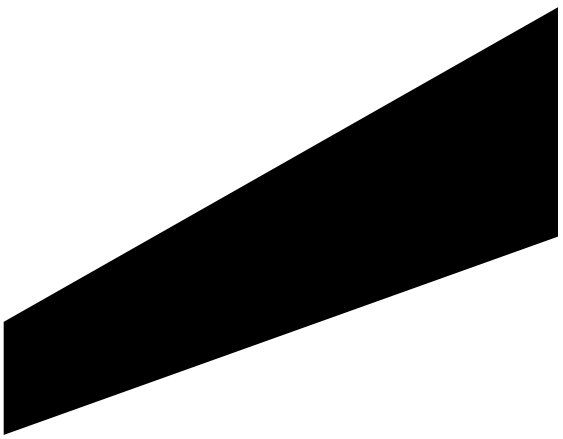


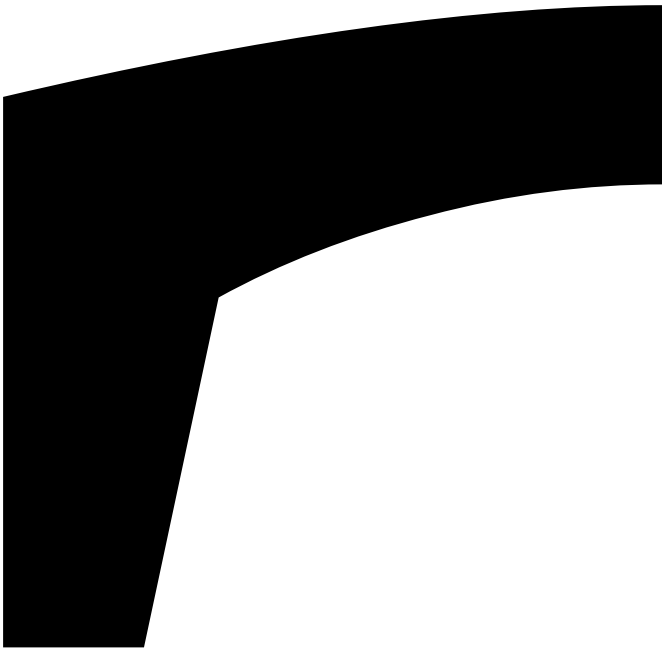


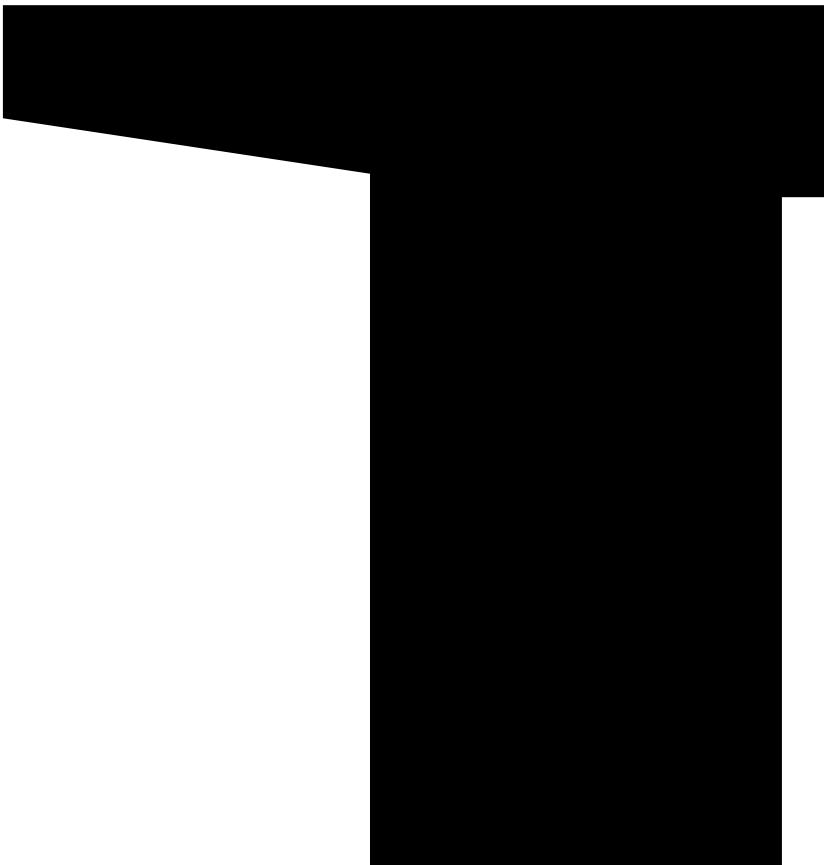




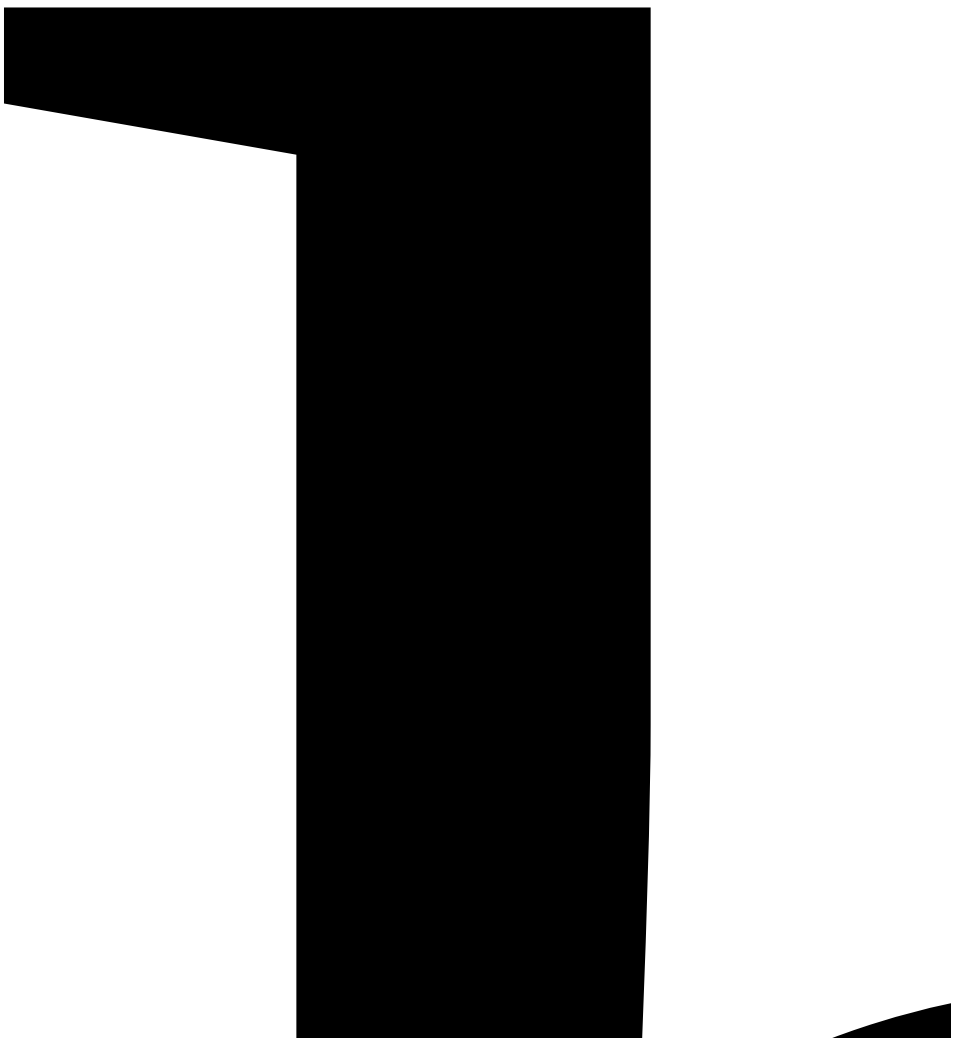






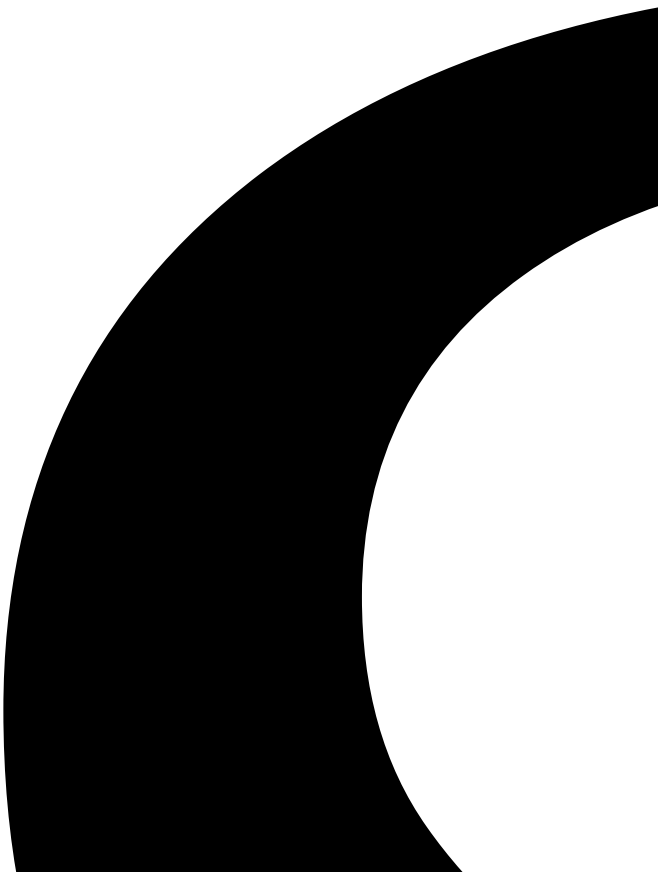








cl



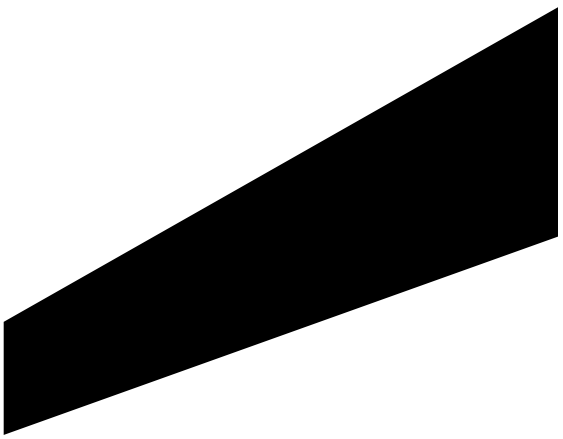


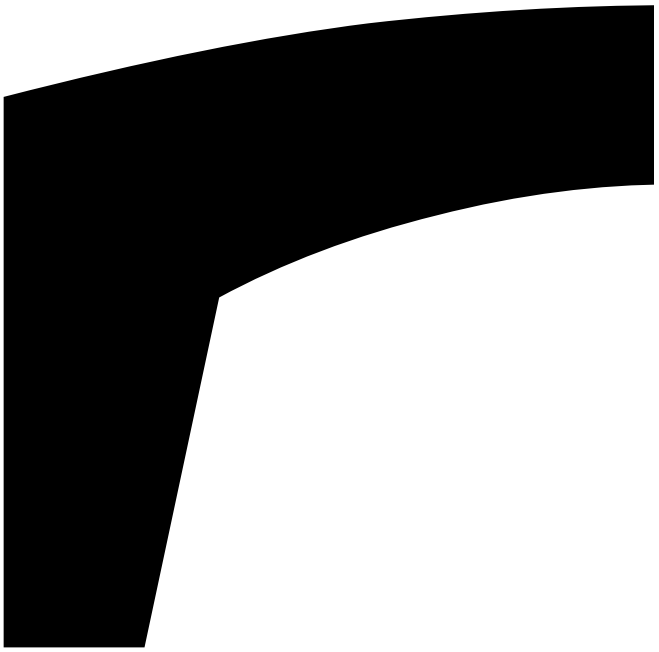


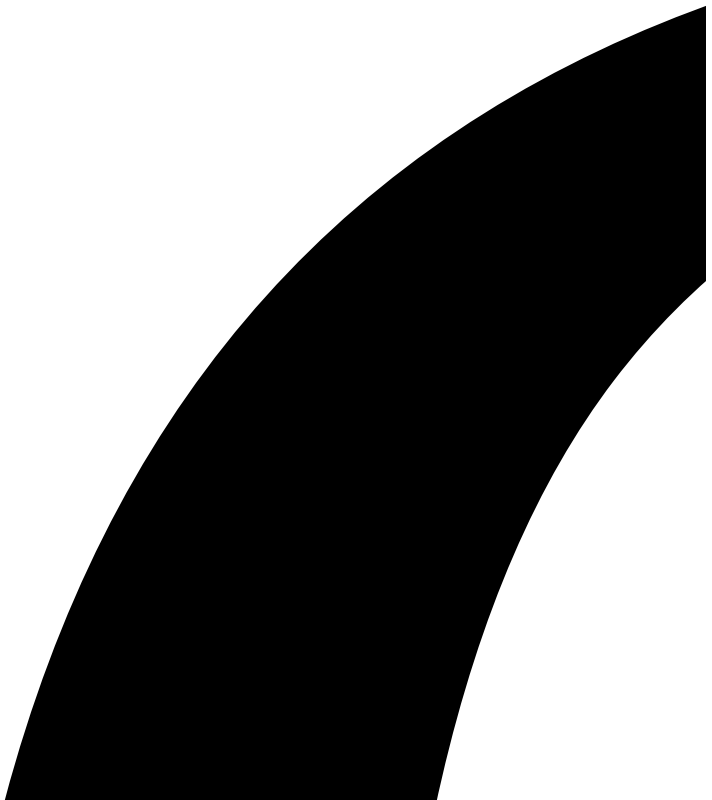


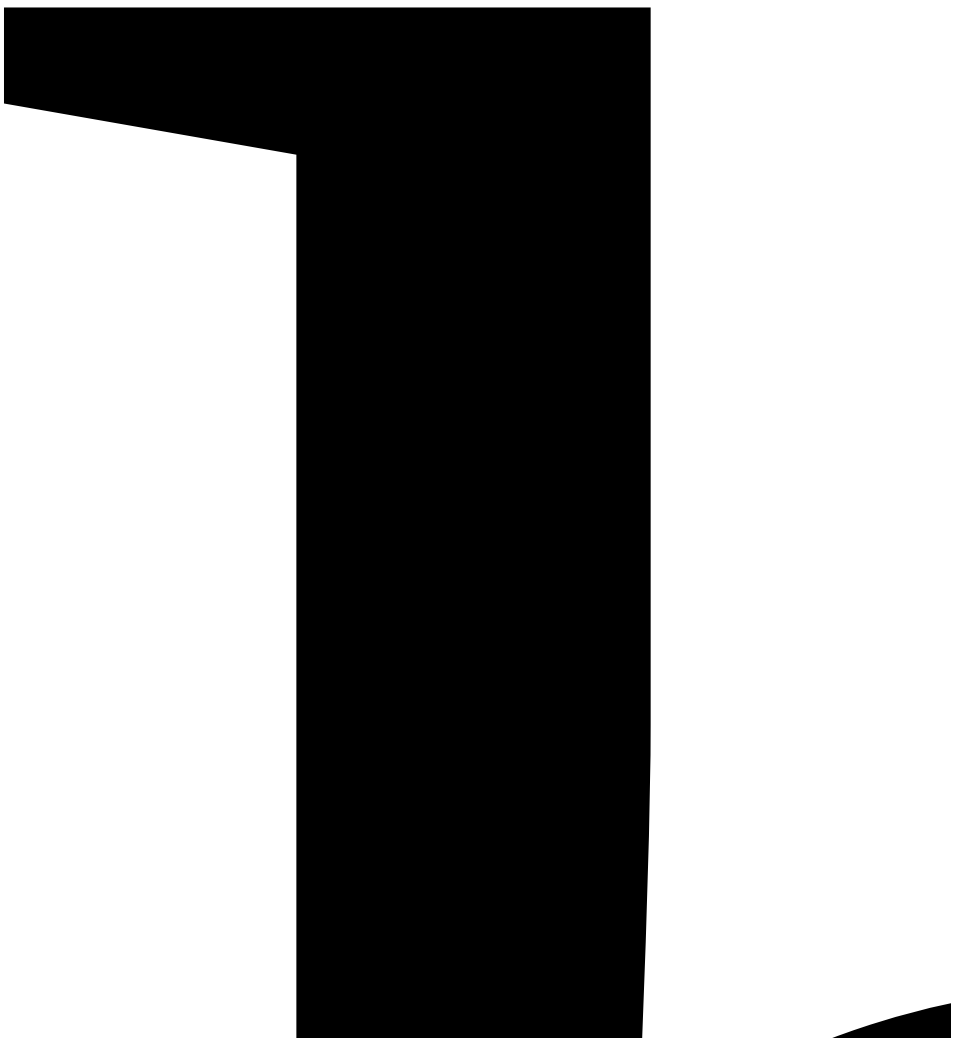


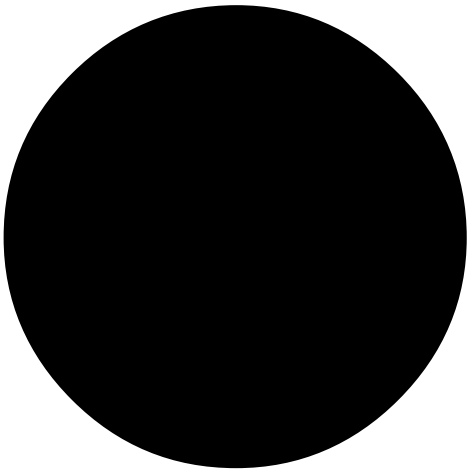






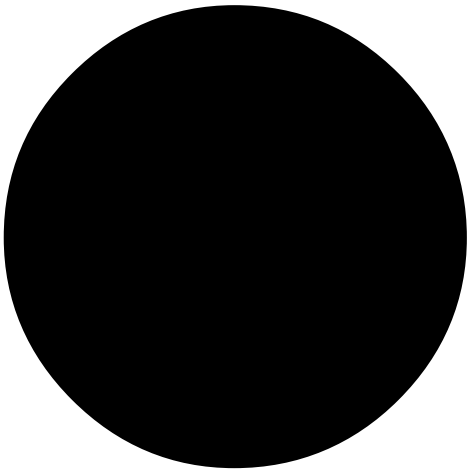




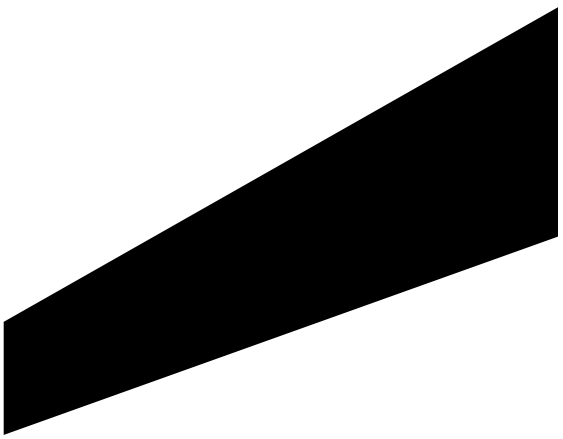


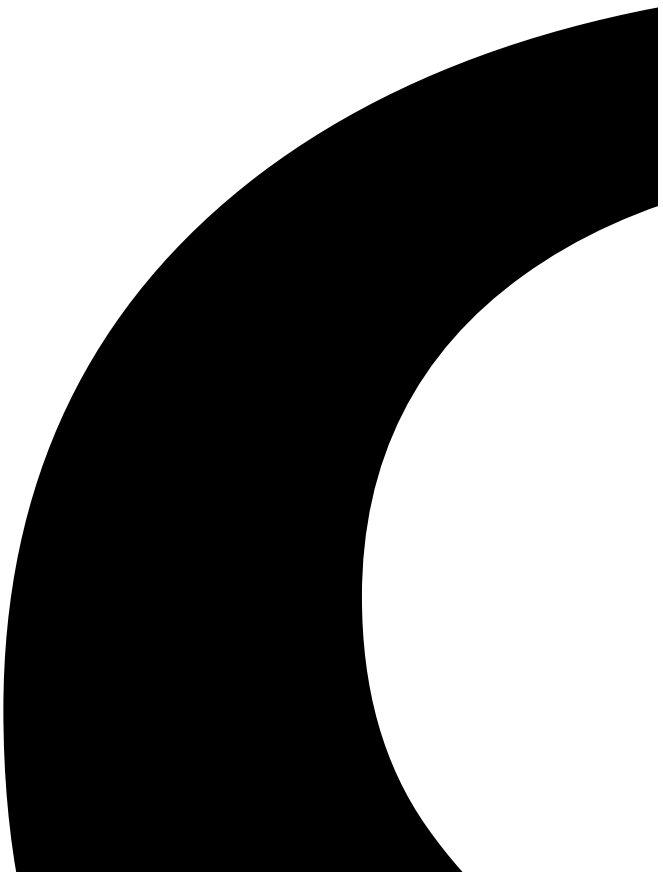


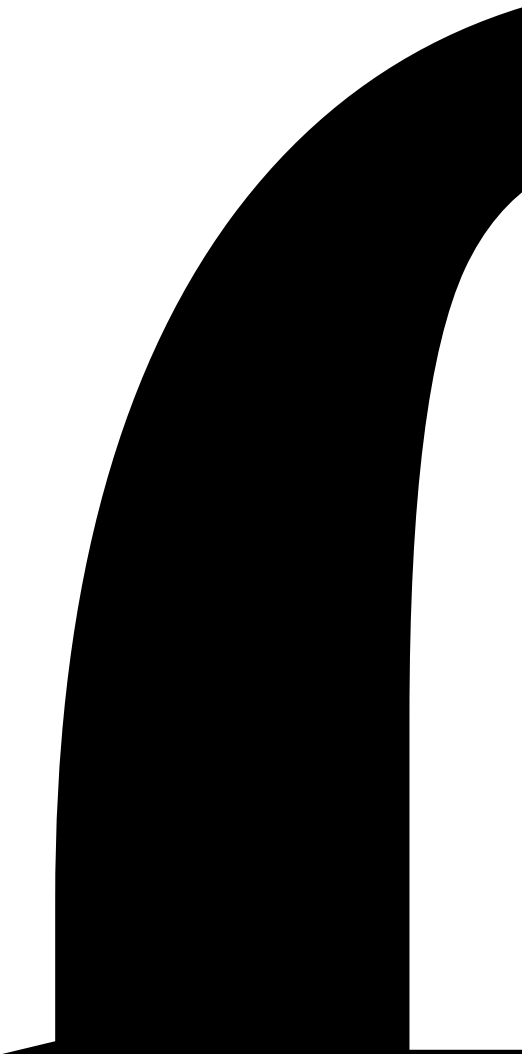






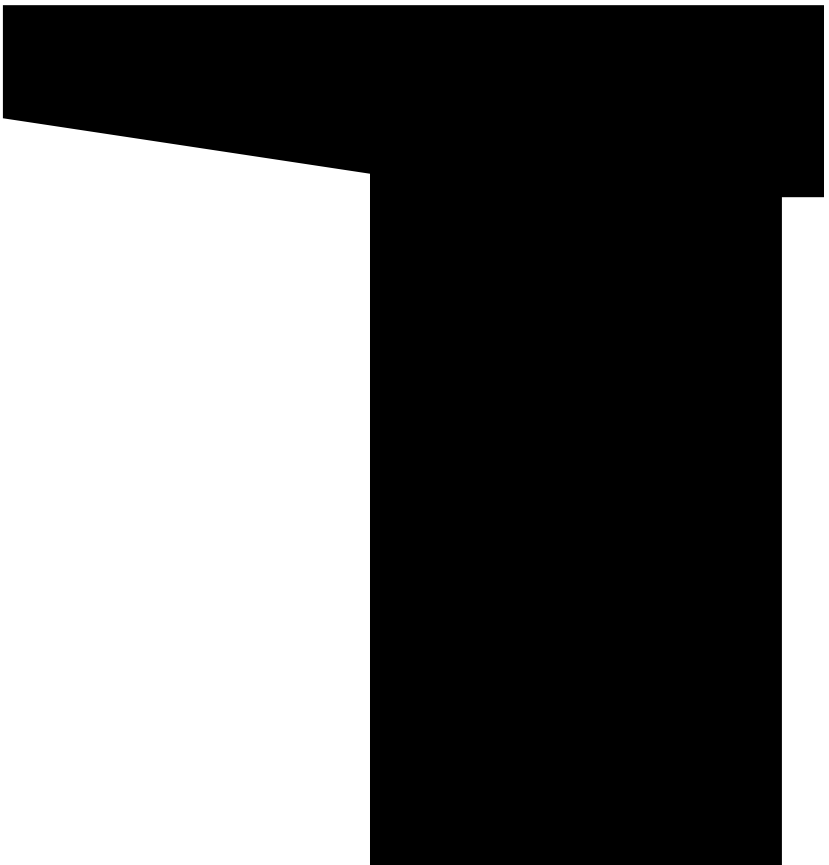


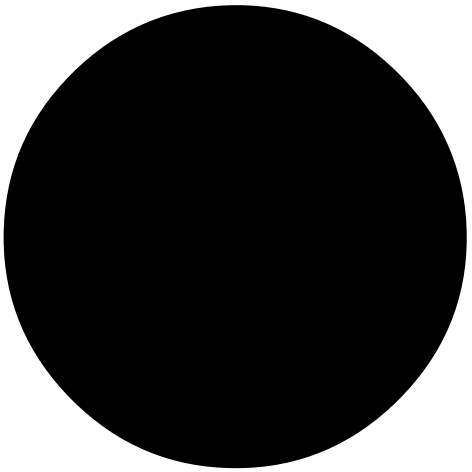






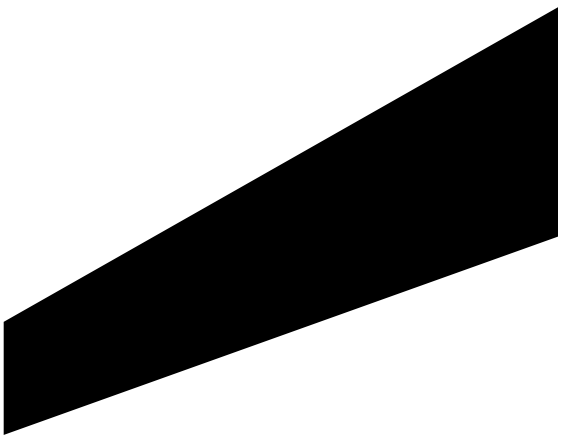


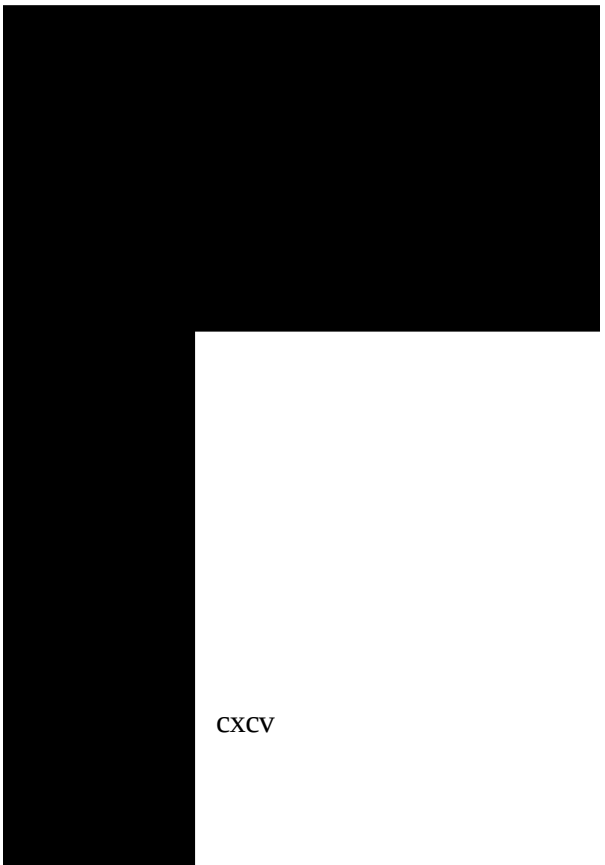




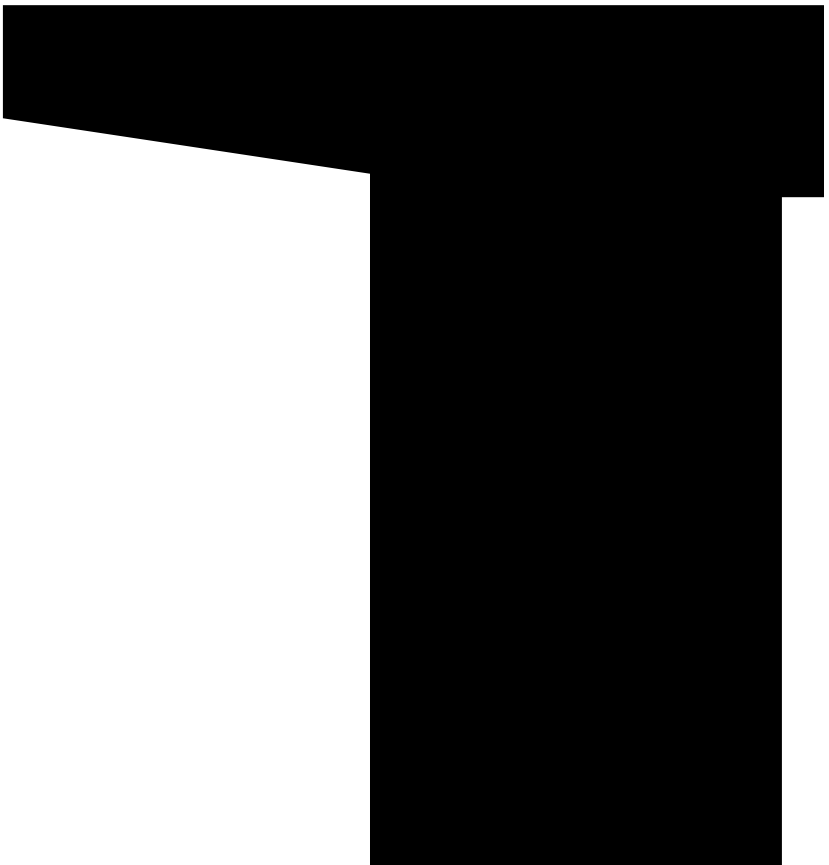








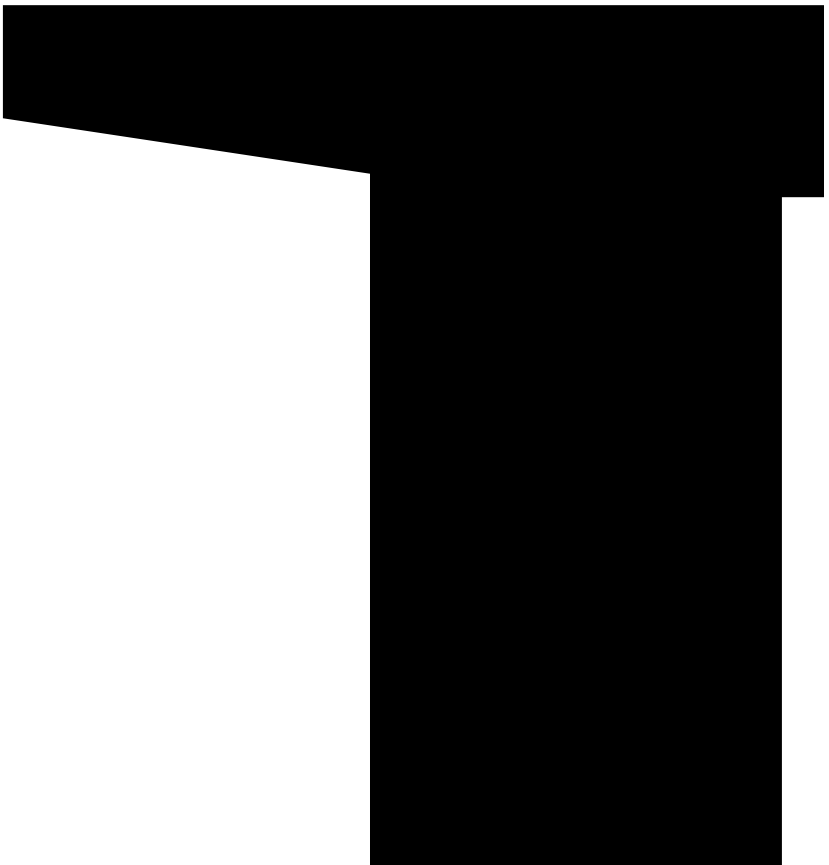
cxcv

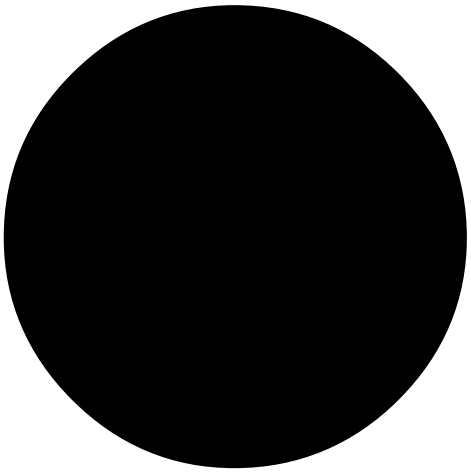




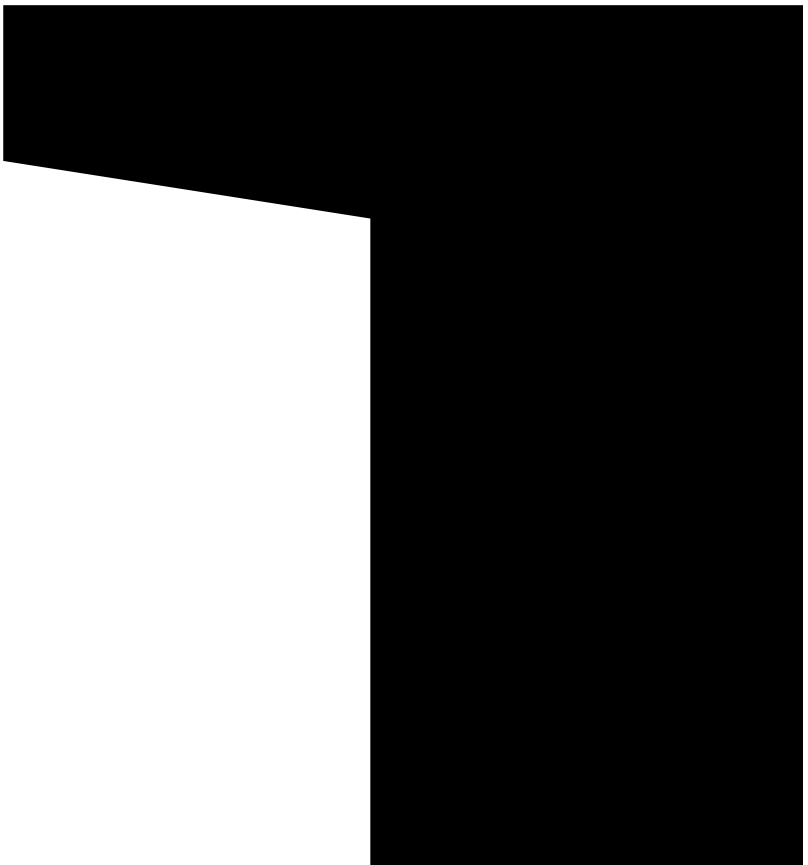


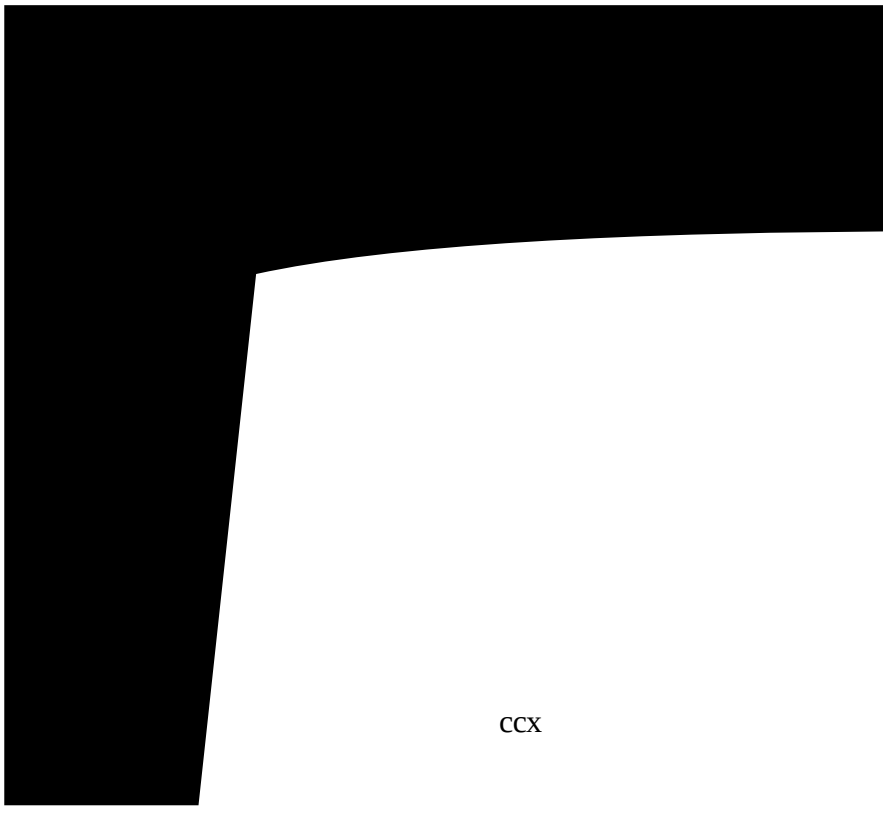


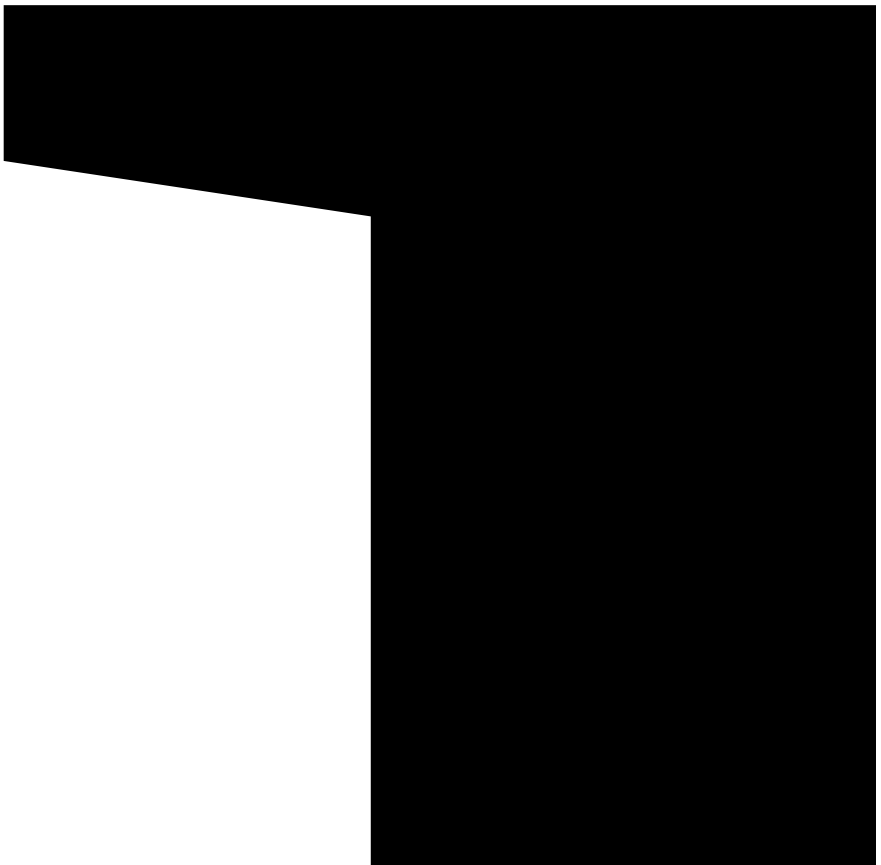


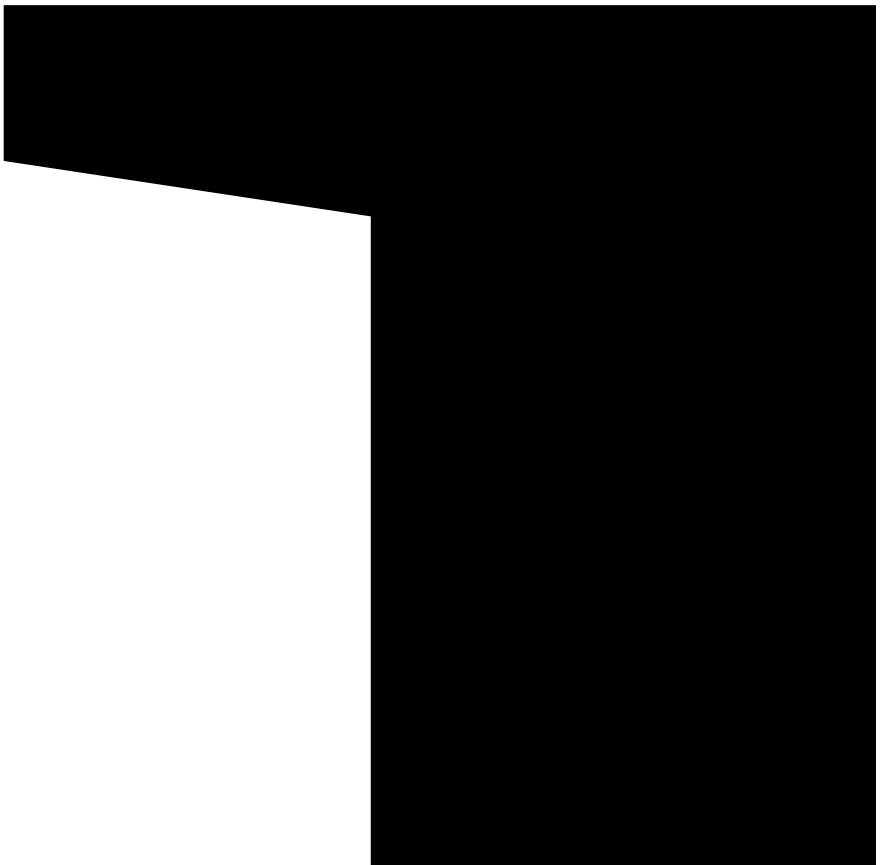


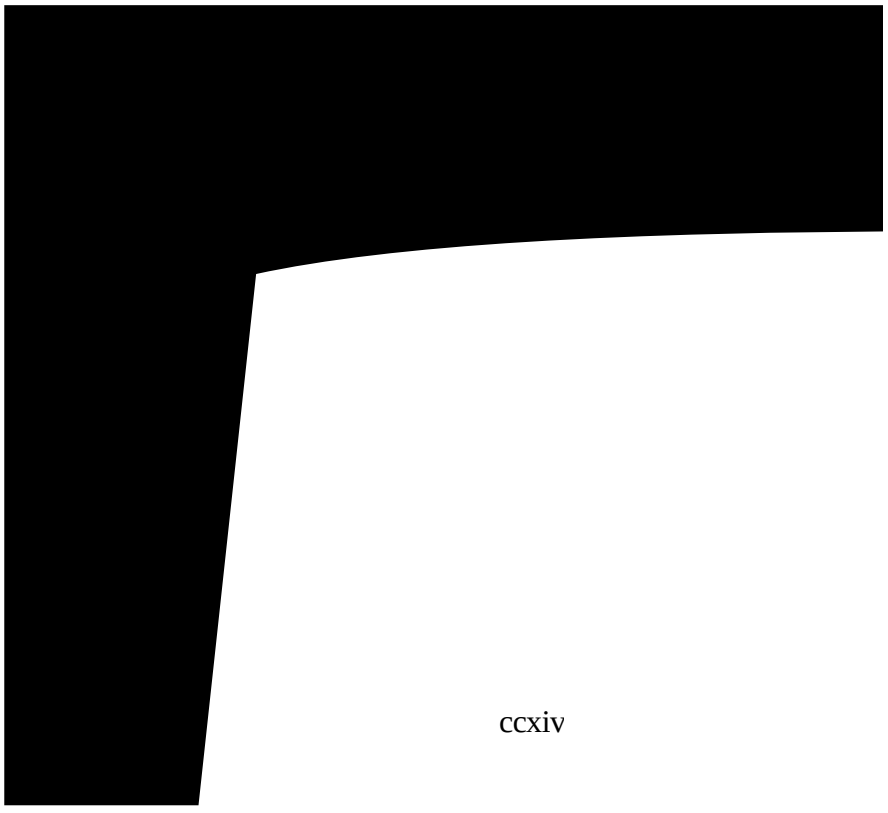


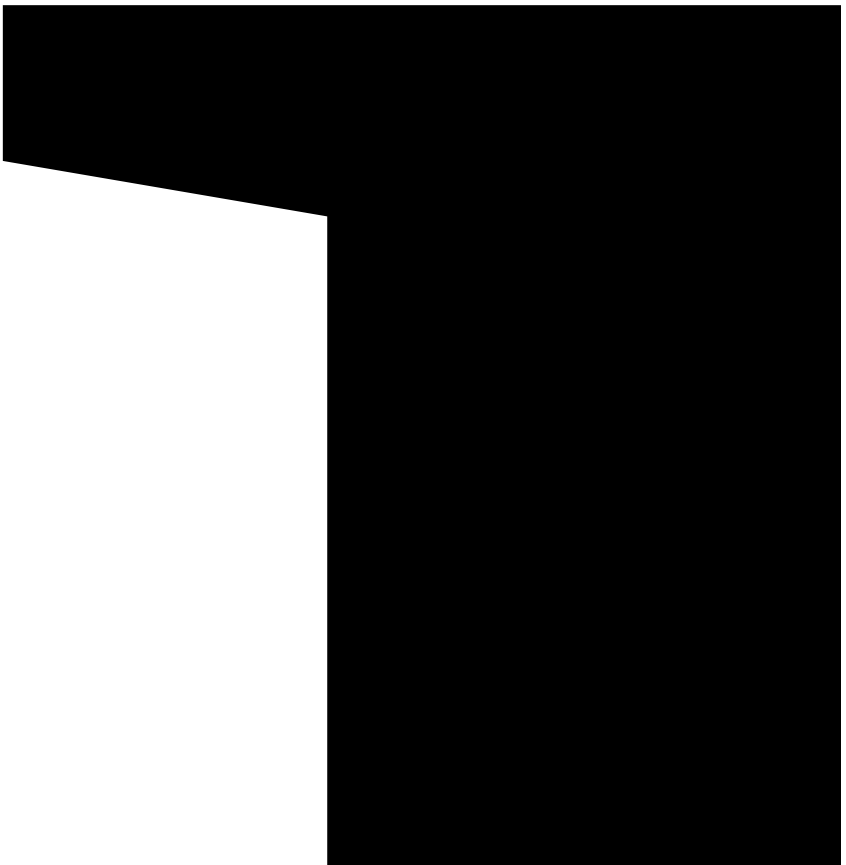


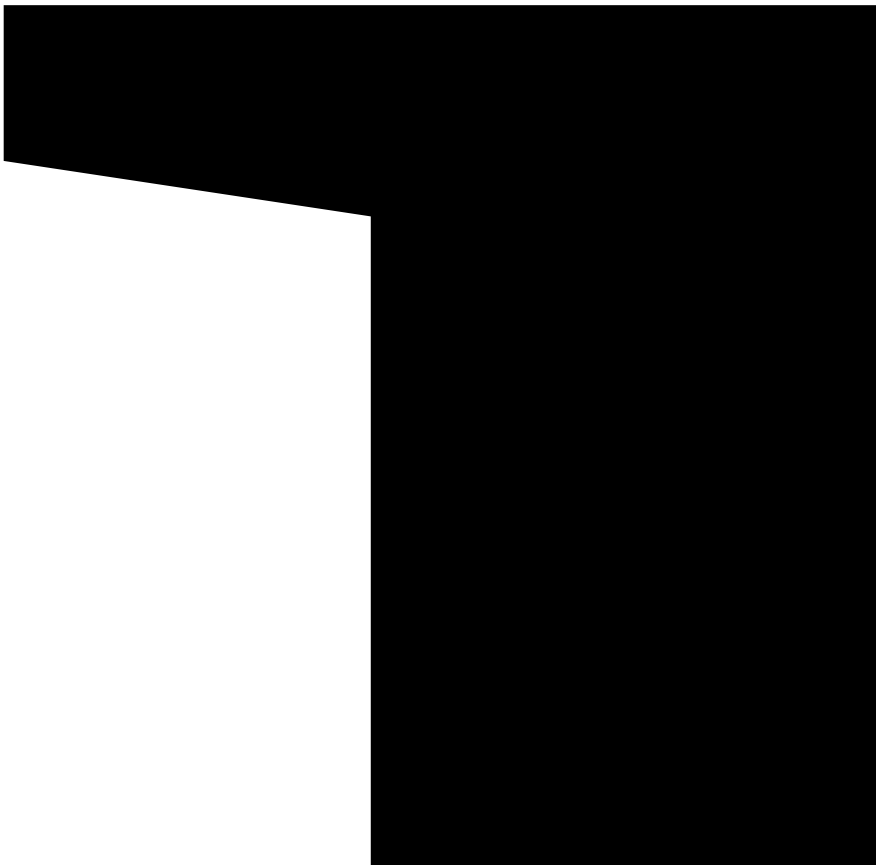


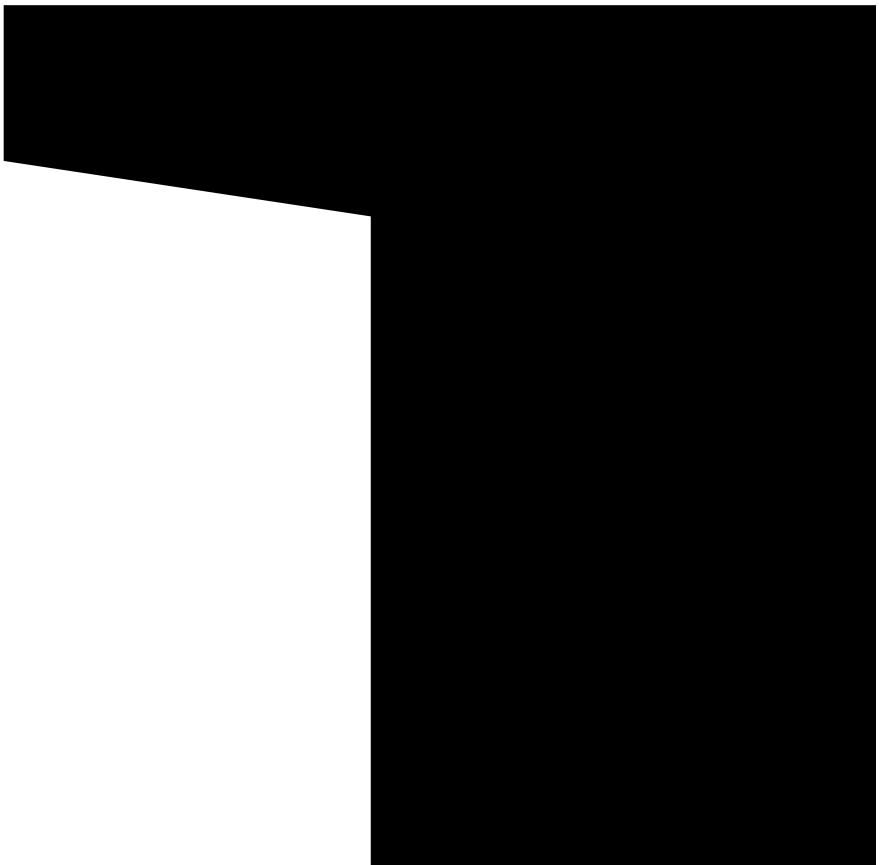


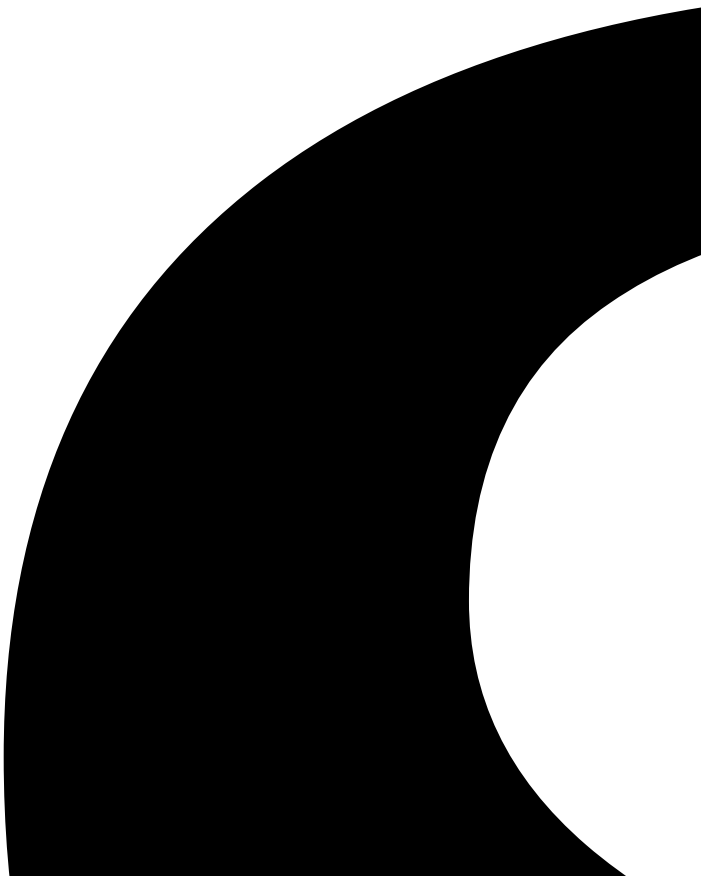


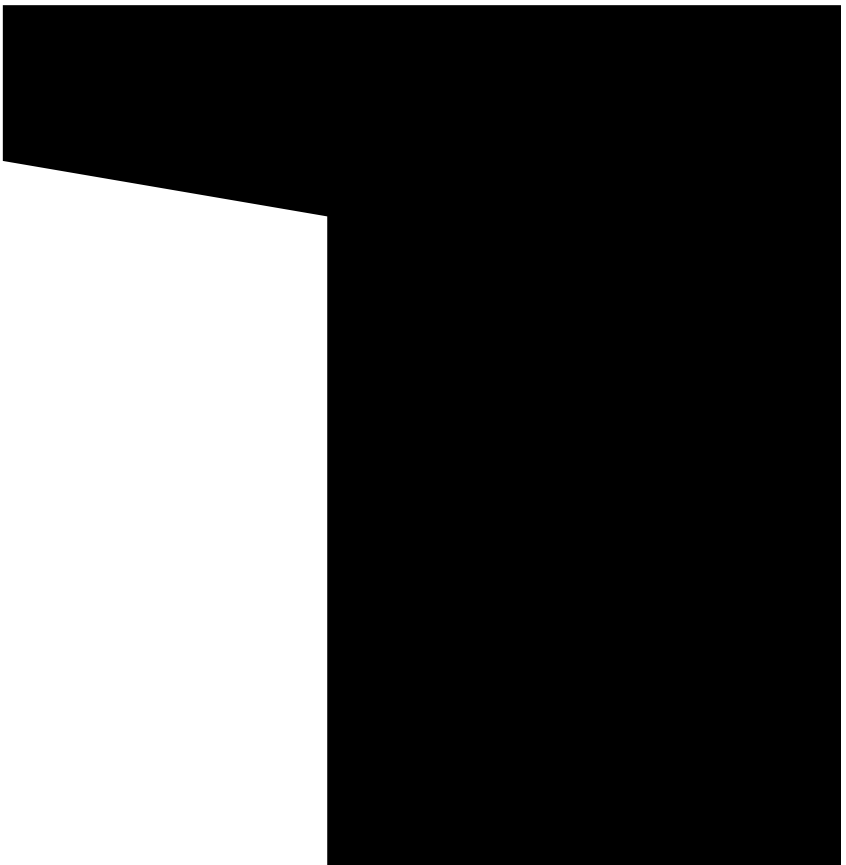


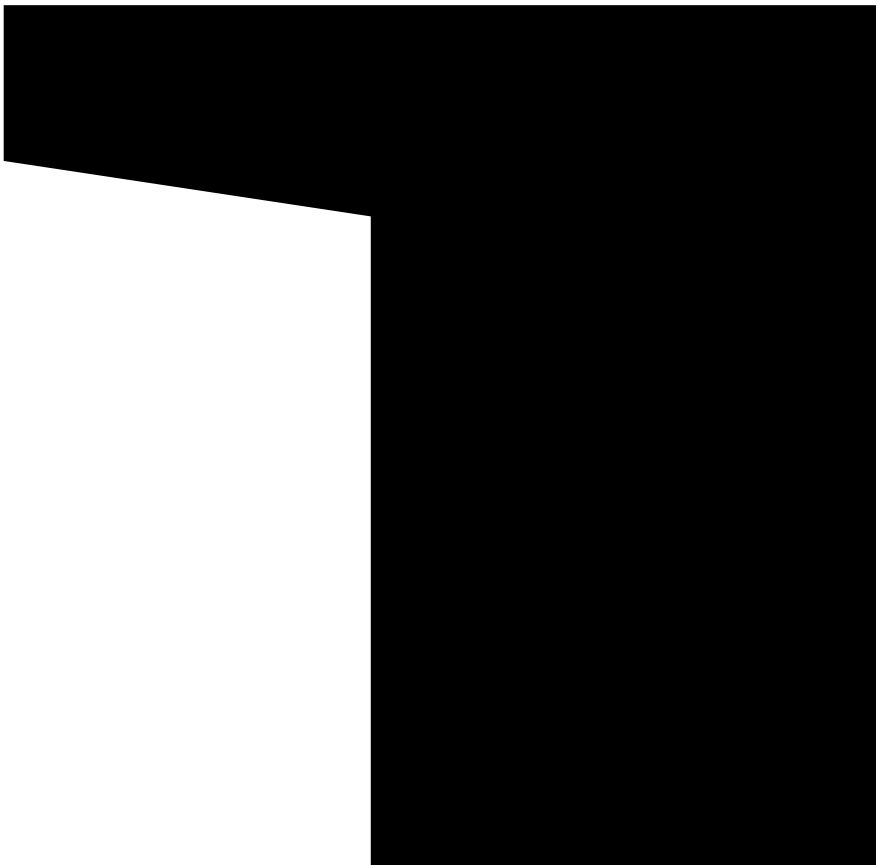




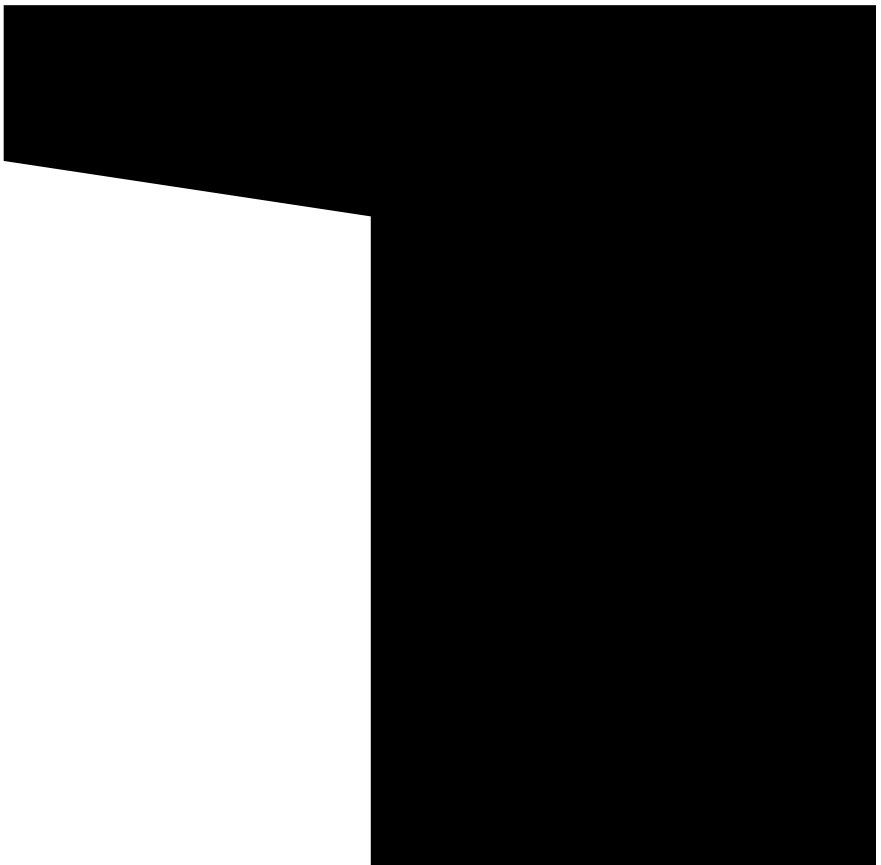


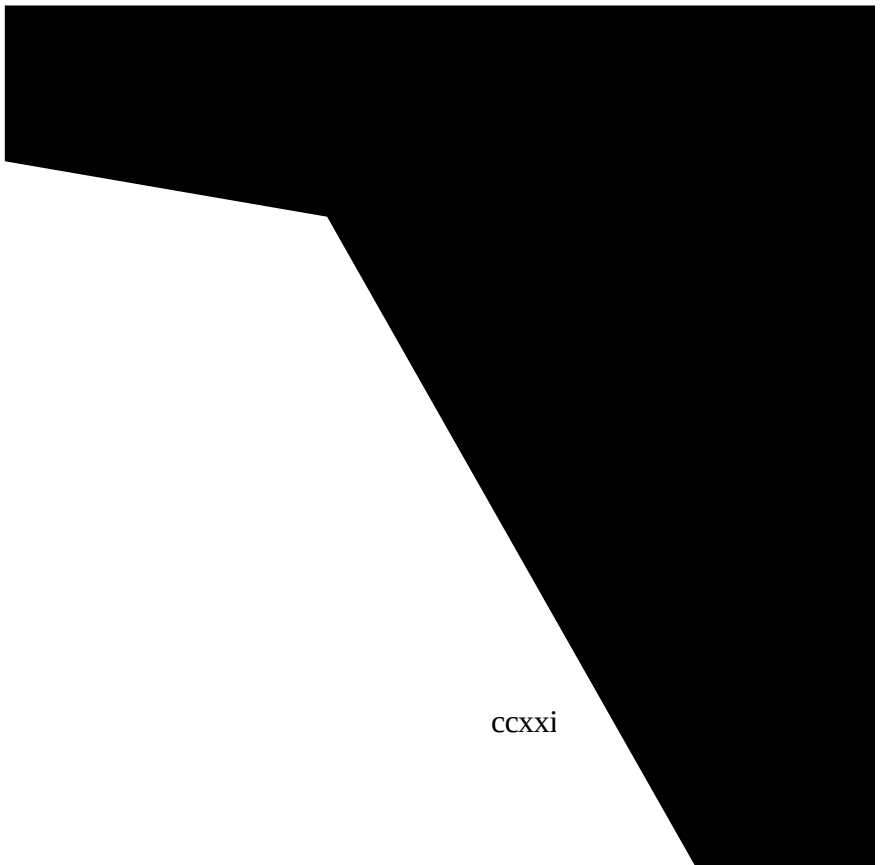


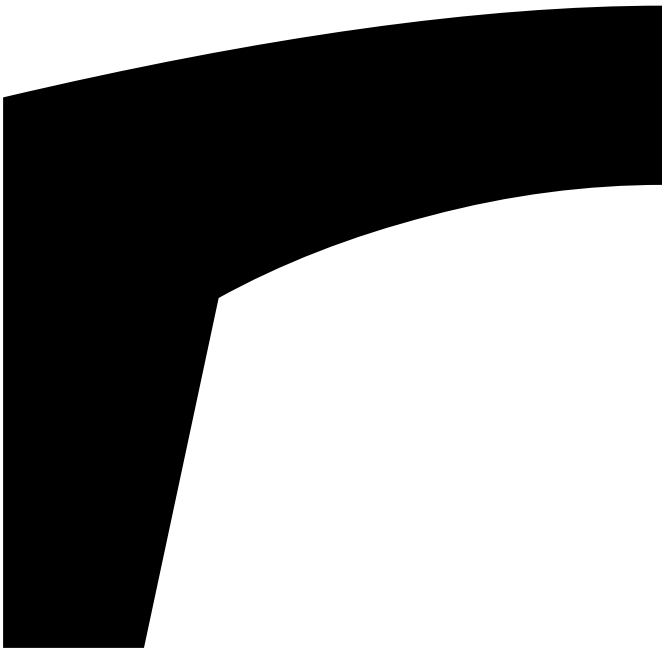


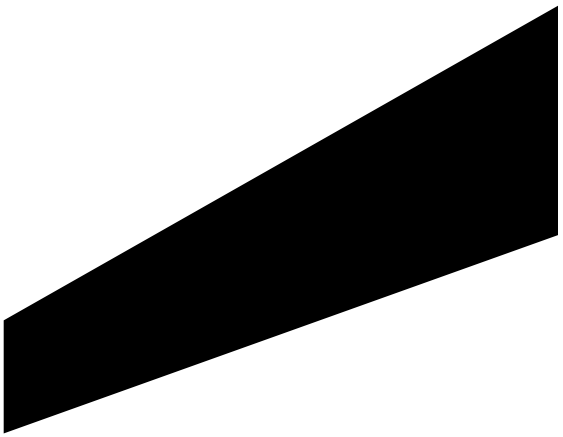


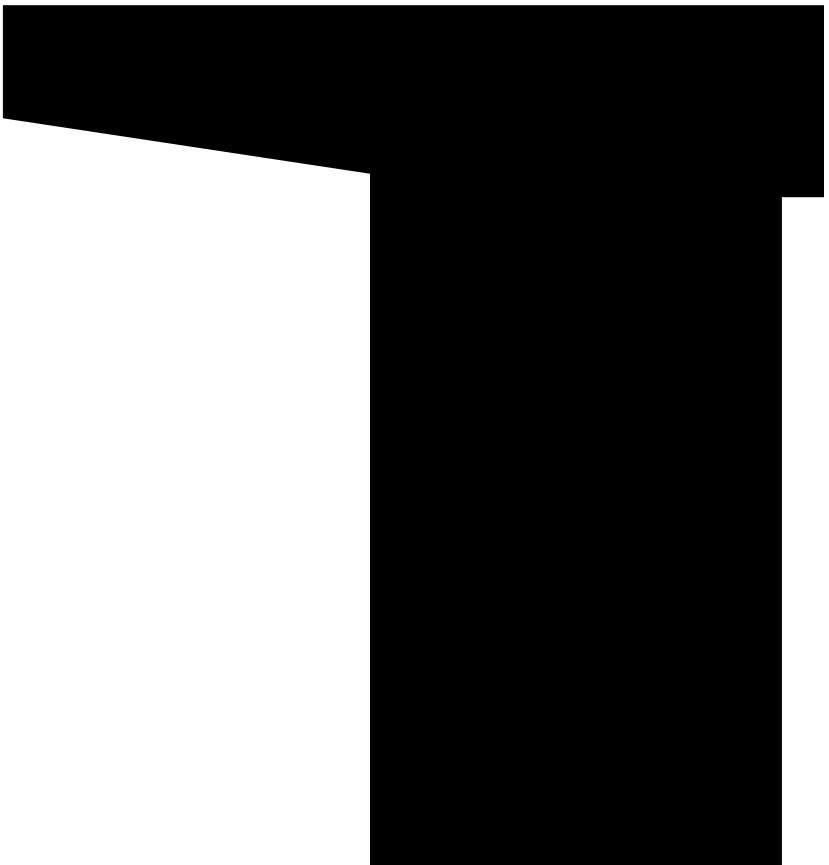




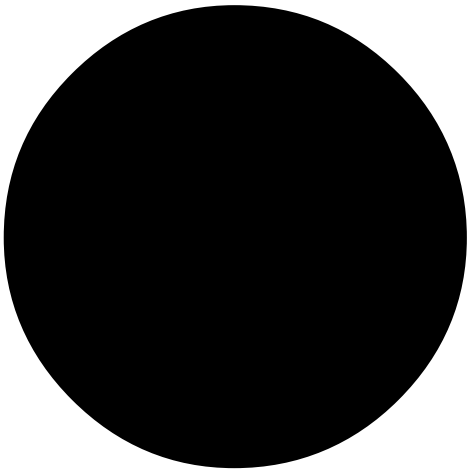


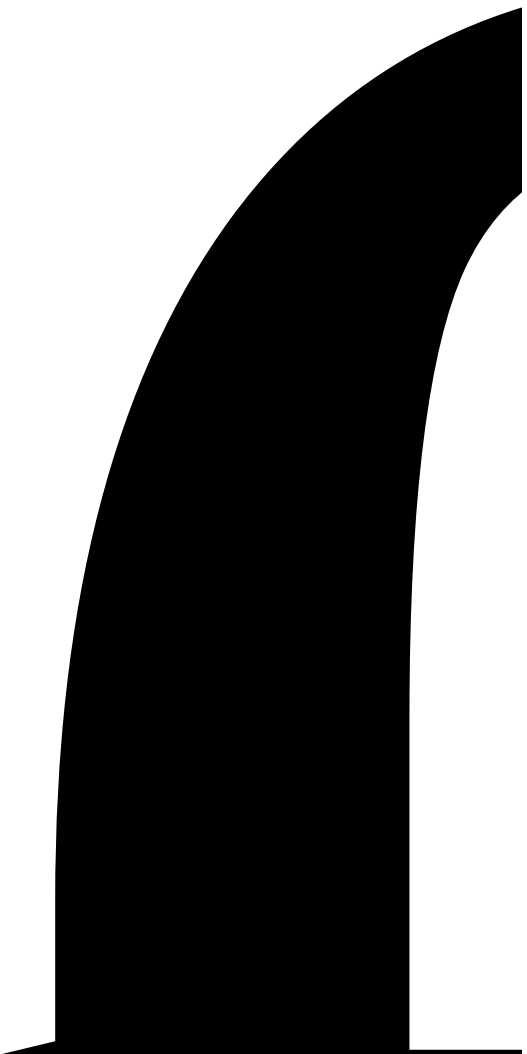






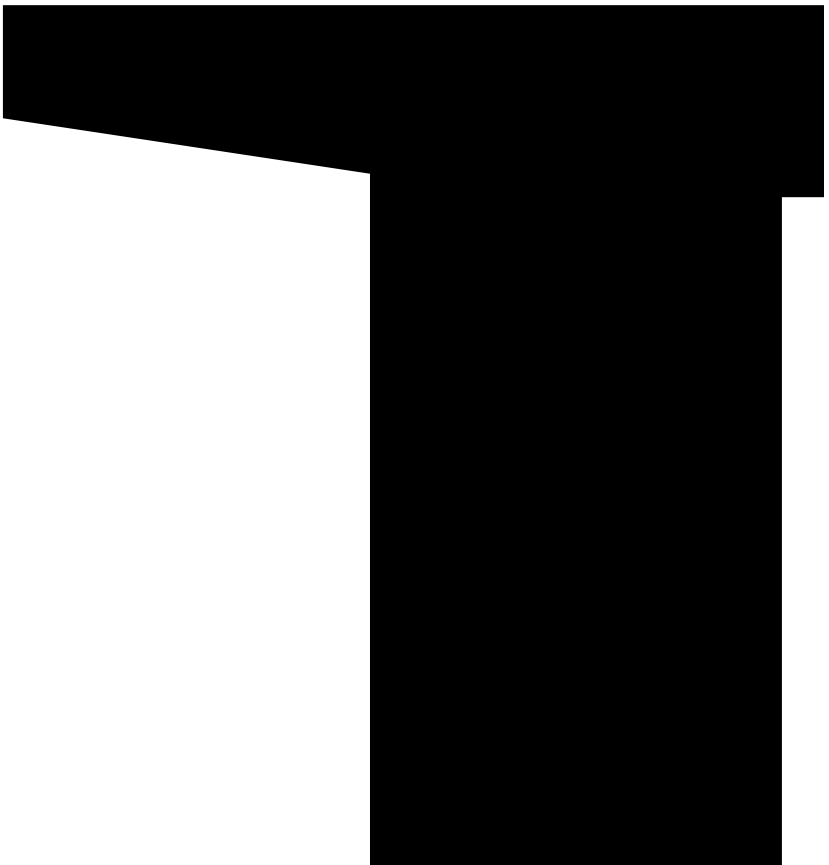


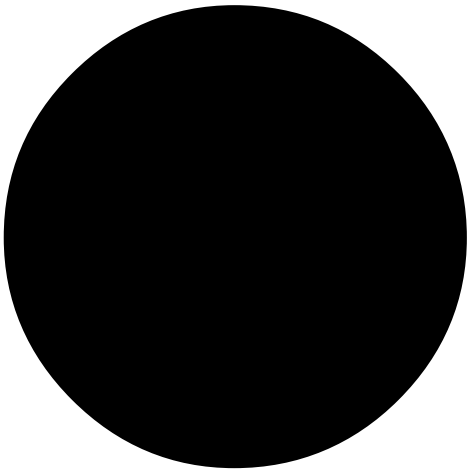






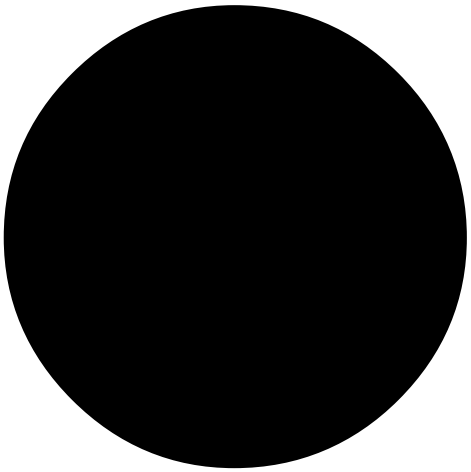






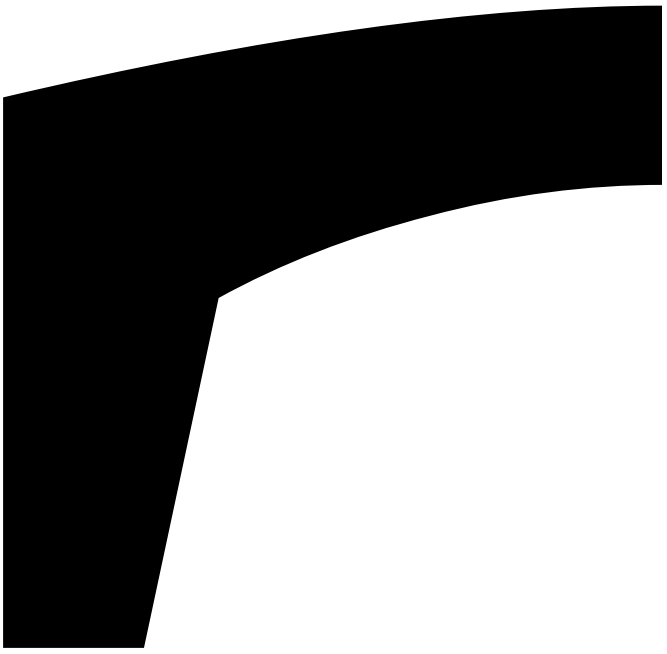




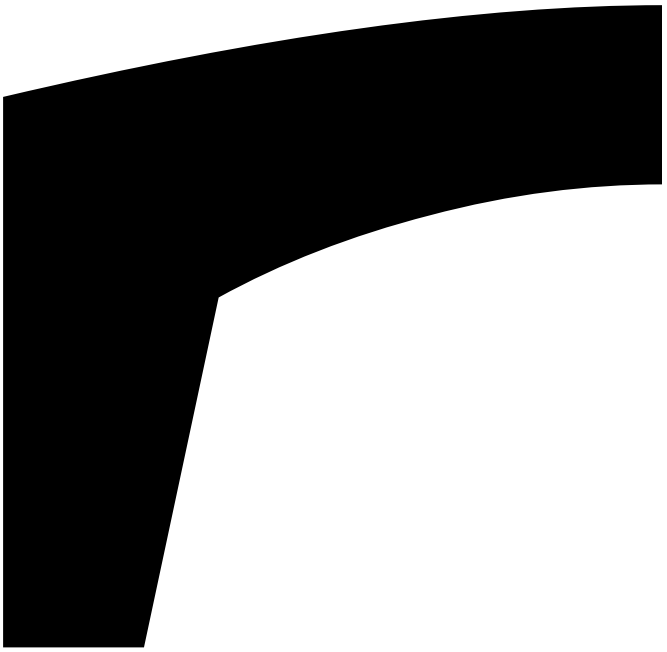


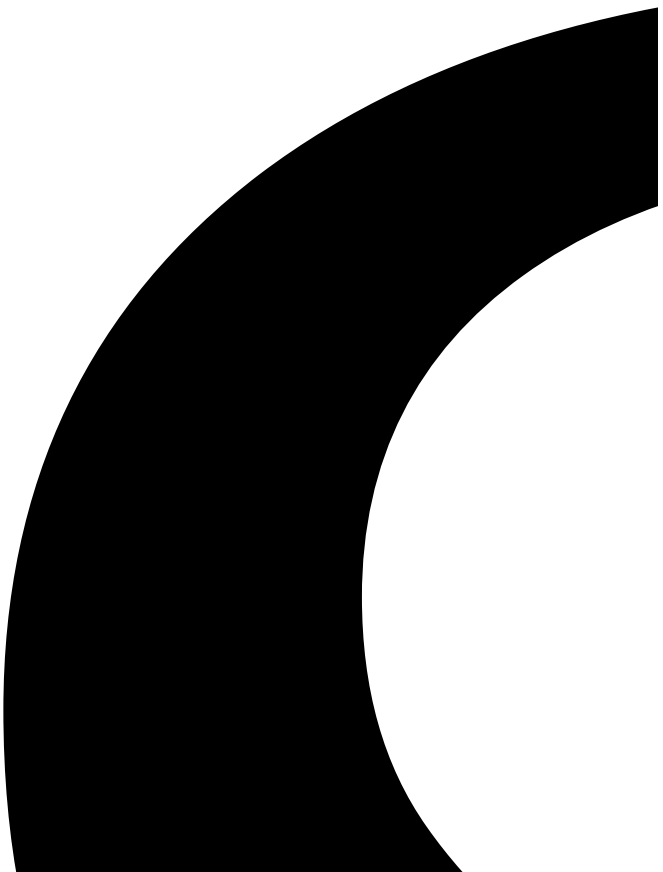


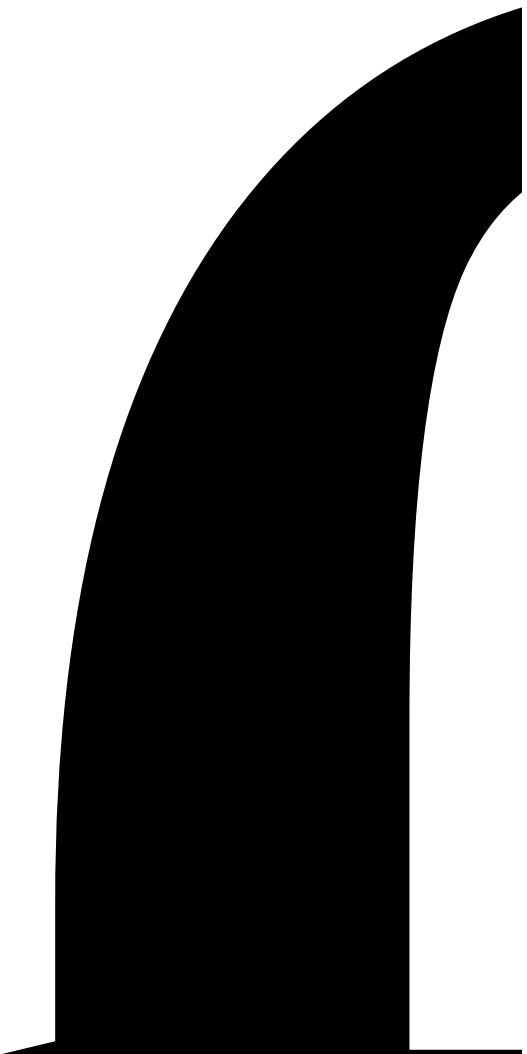




ccl

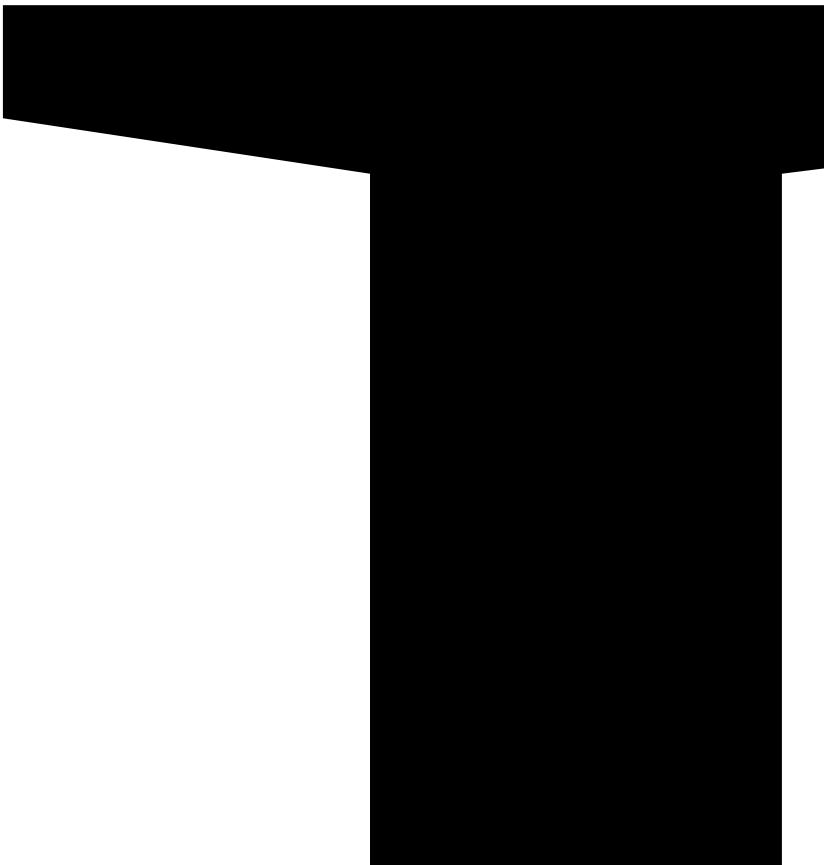


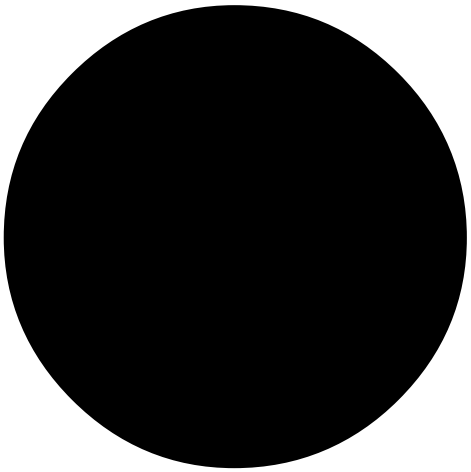








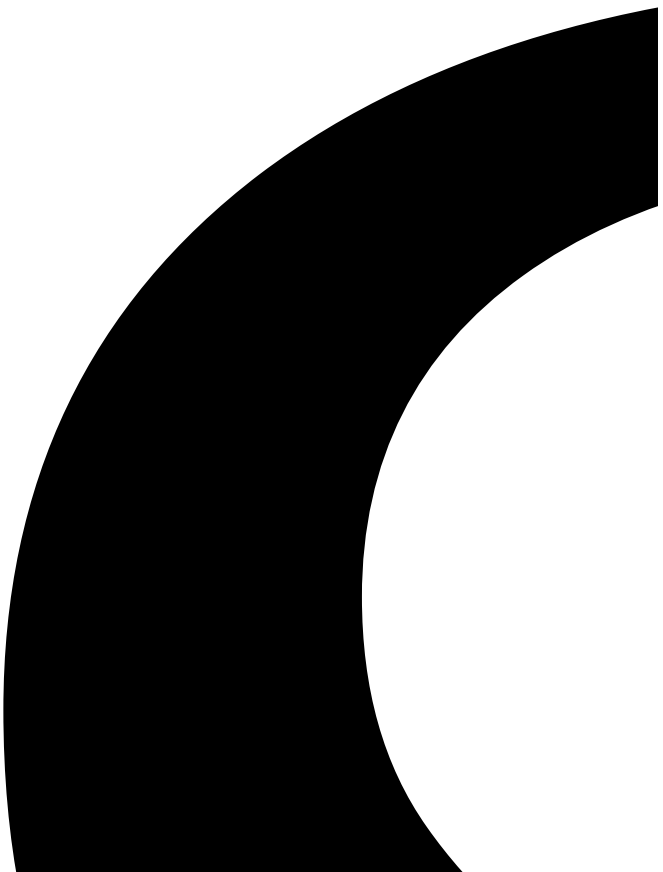




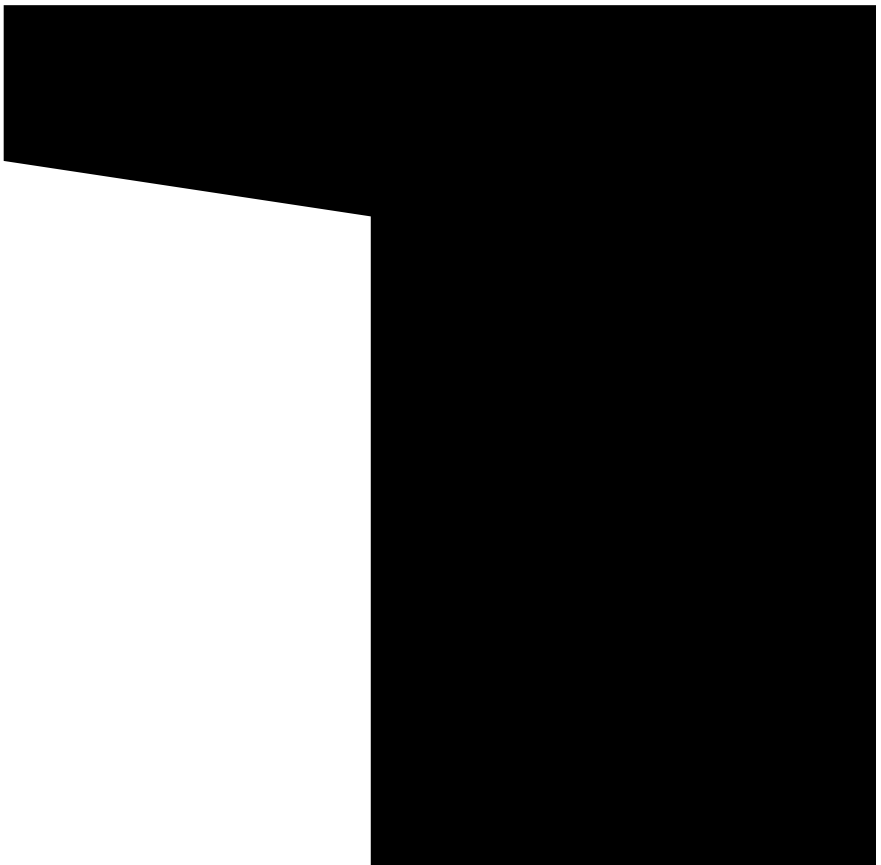


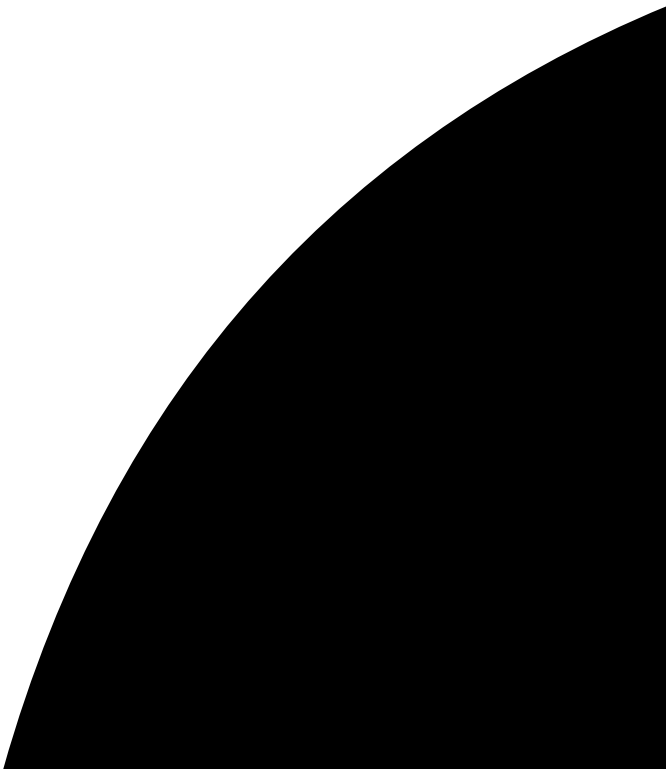


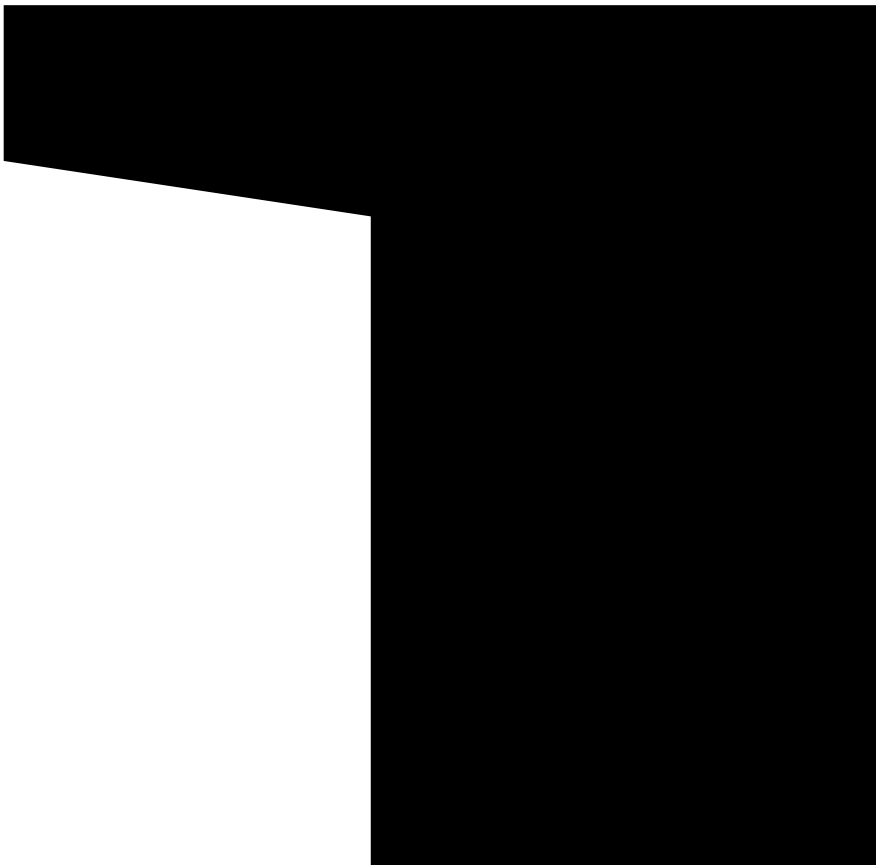


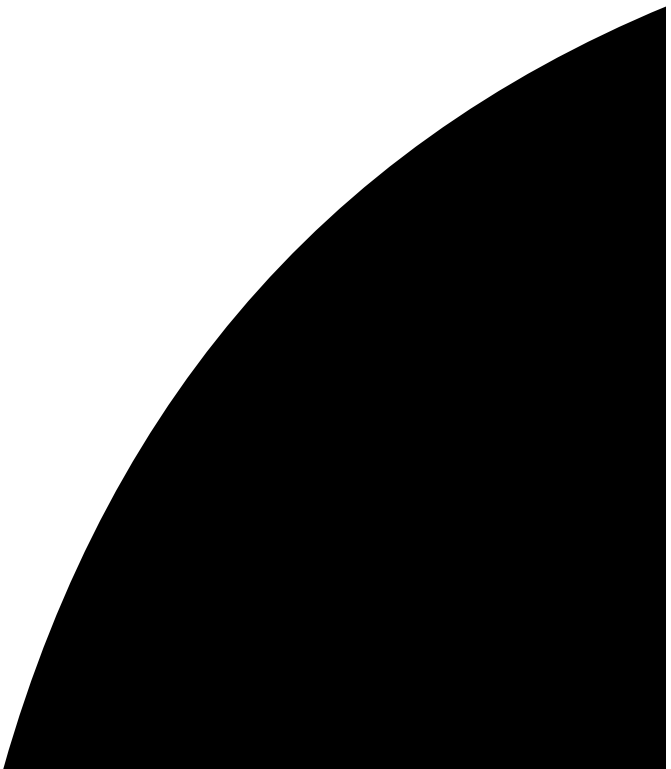


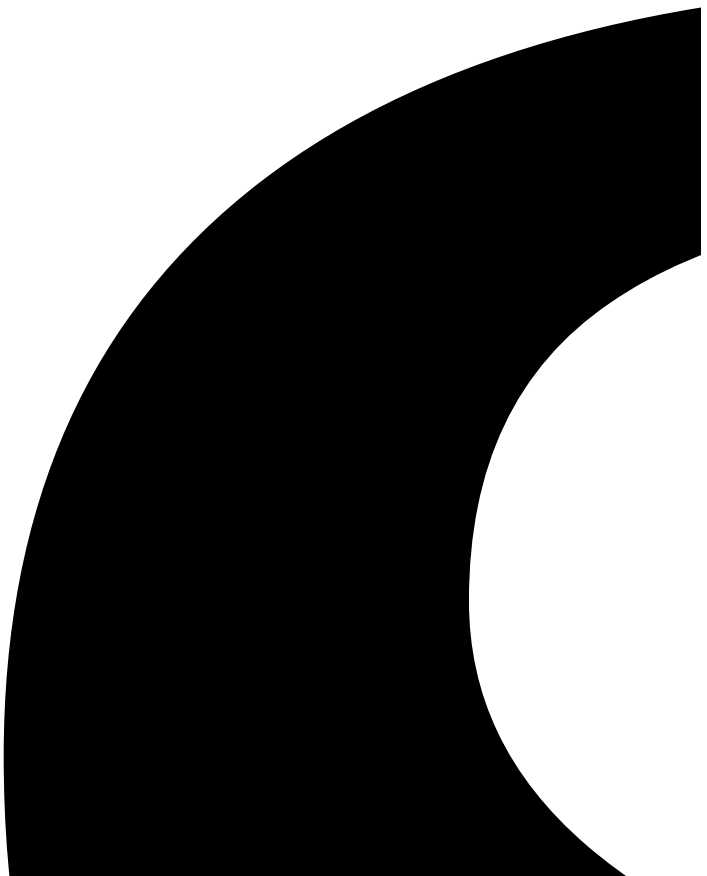


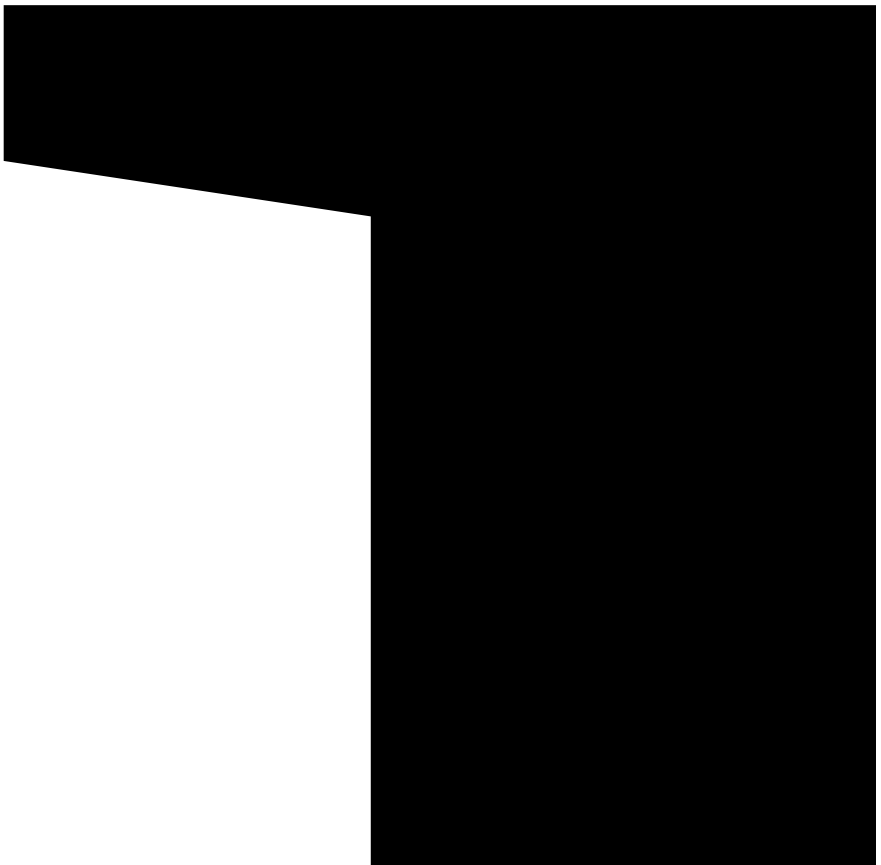


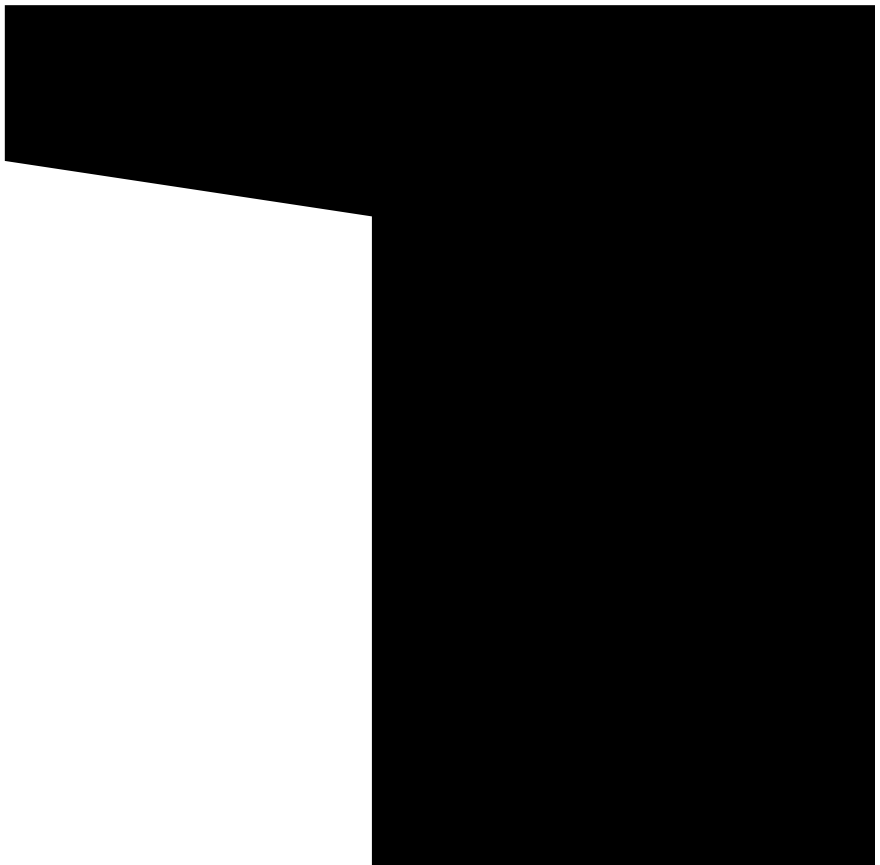




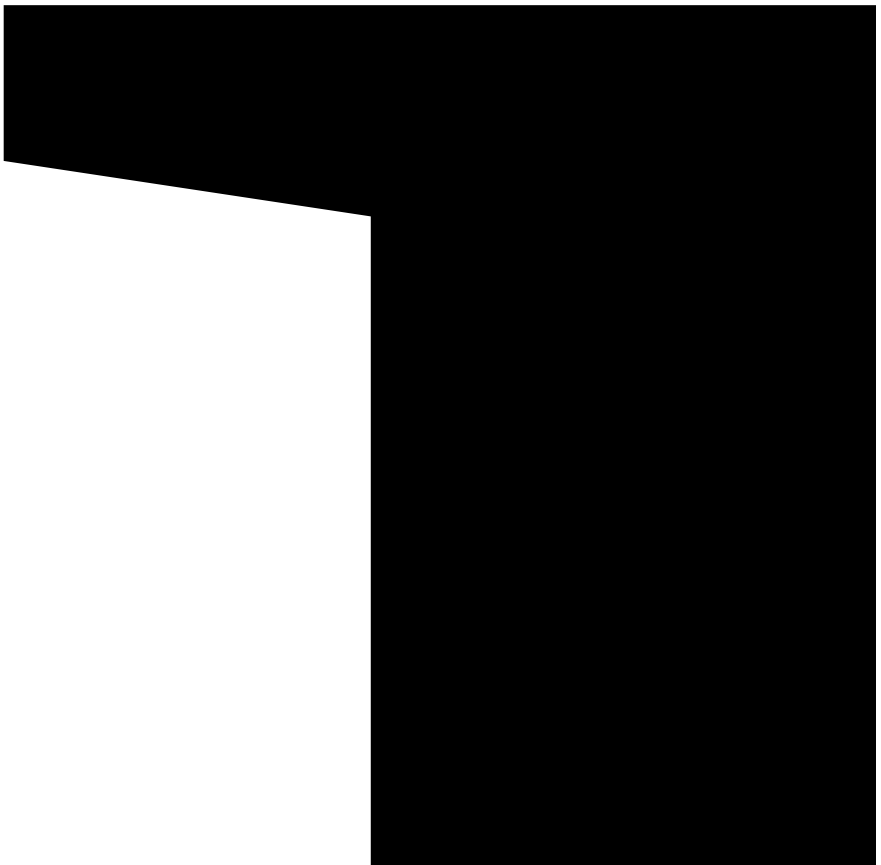


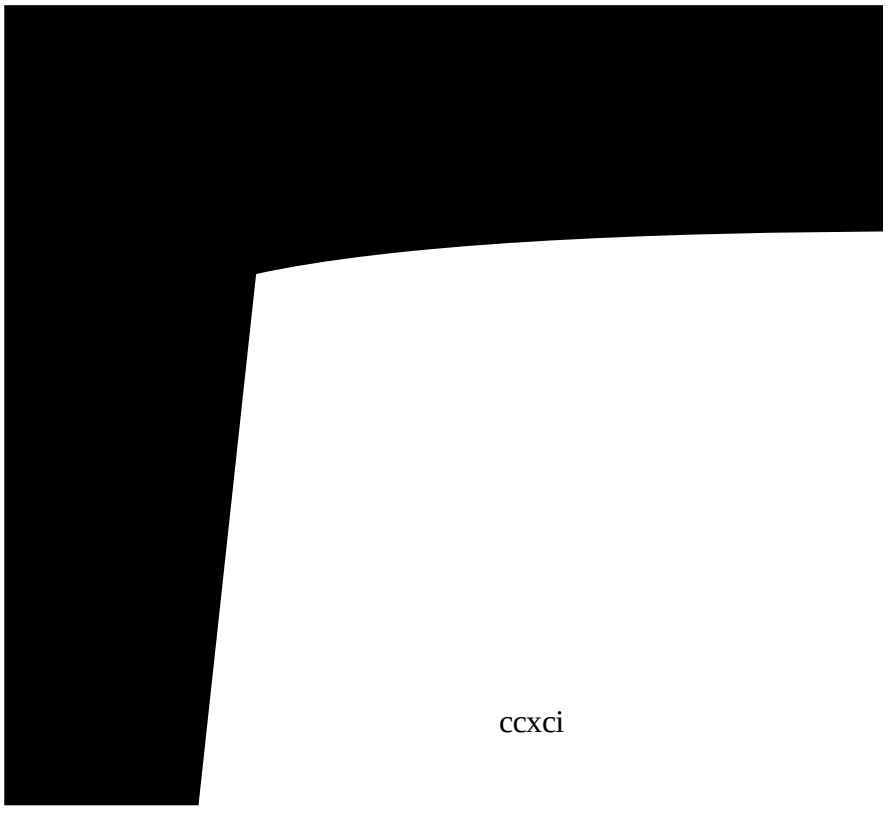


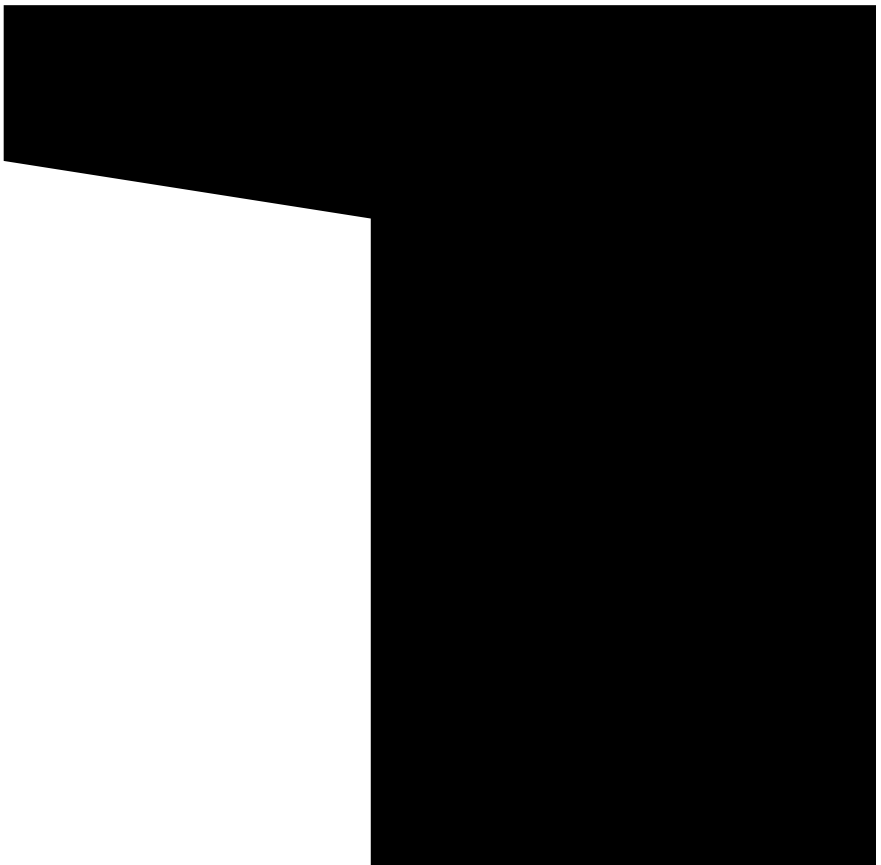


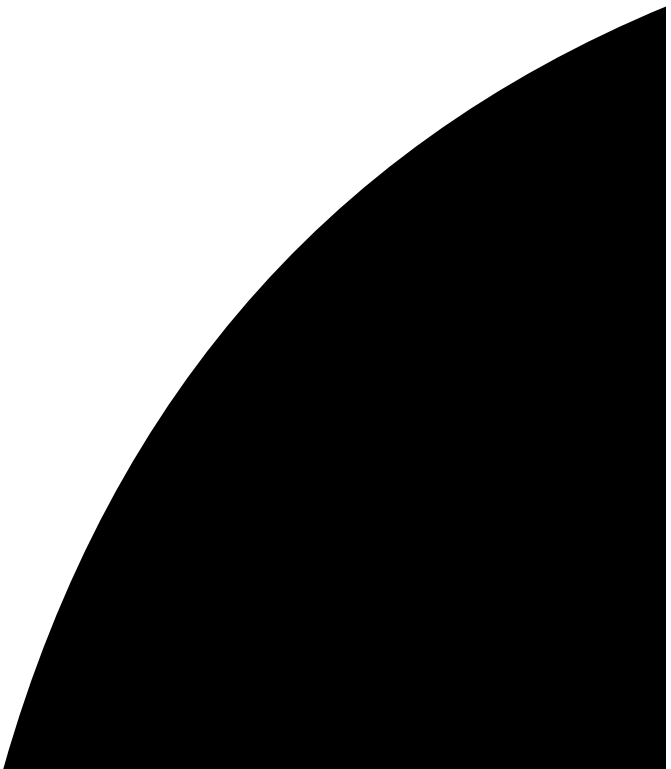


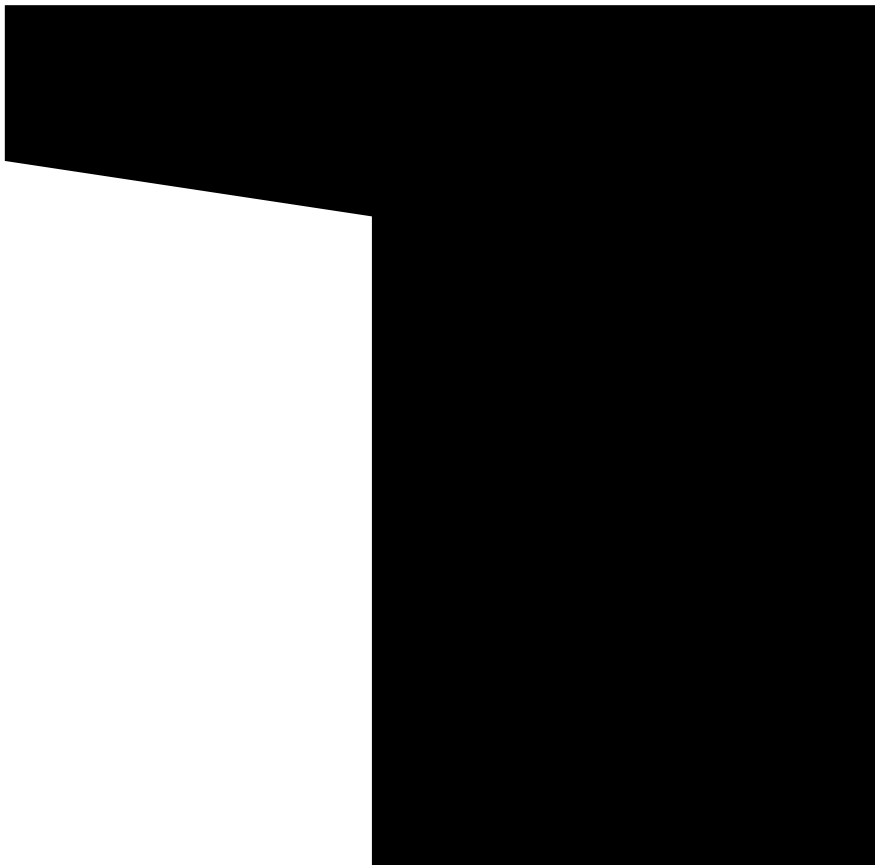


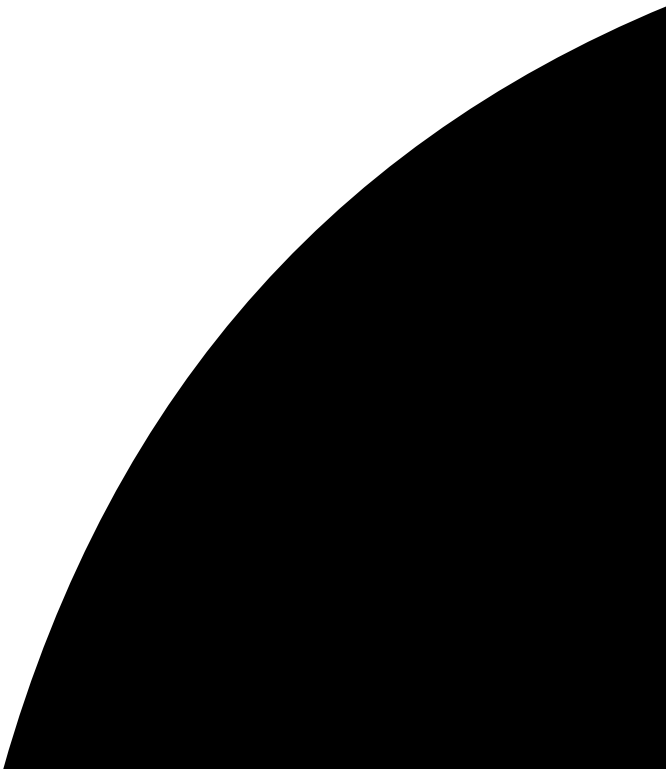


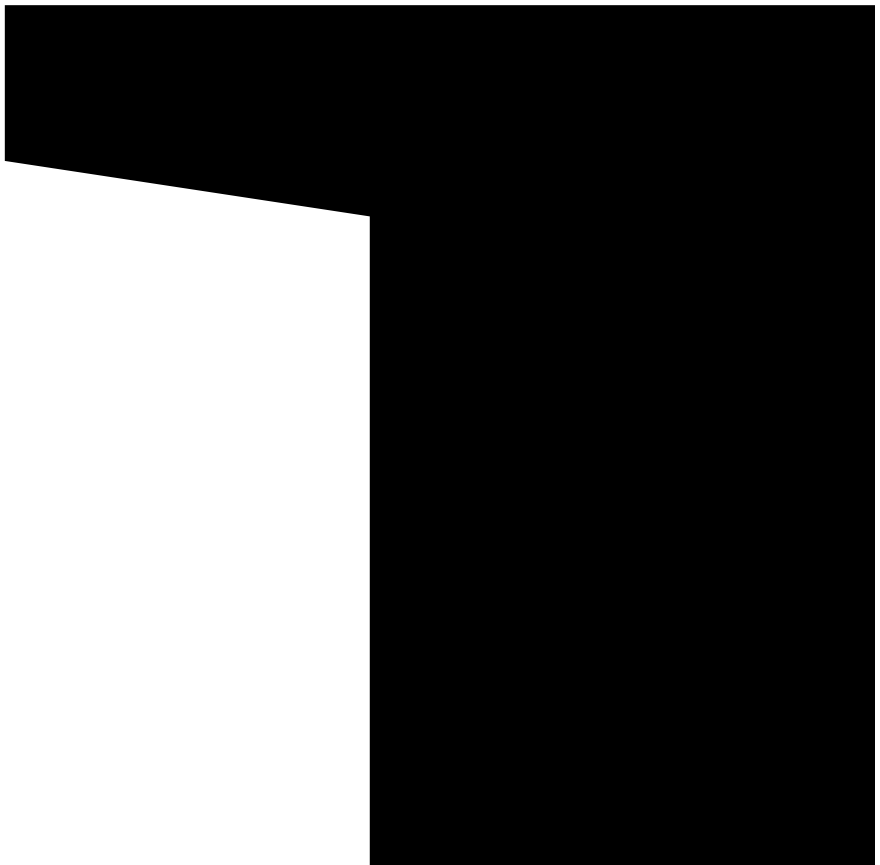


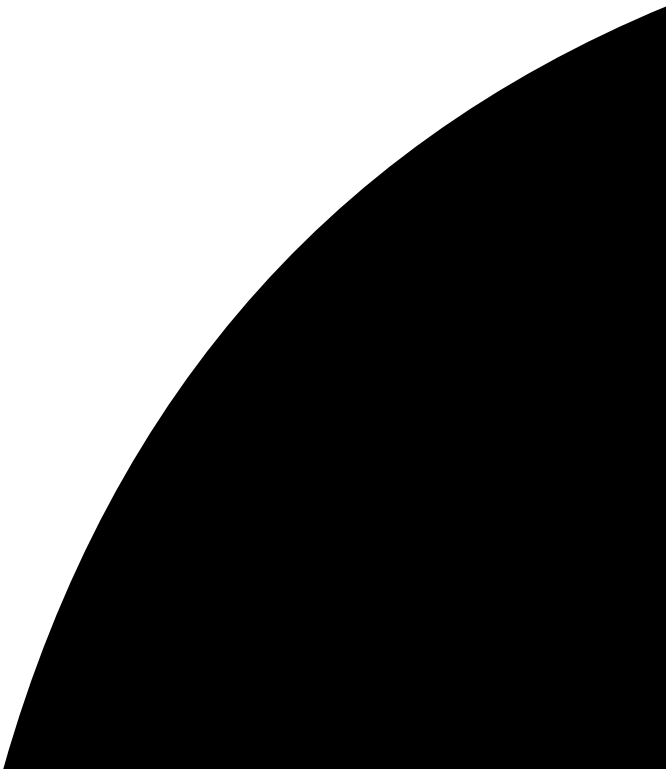


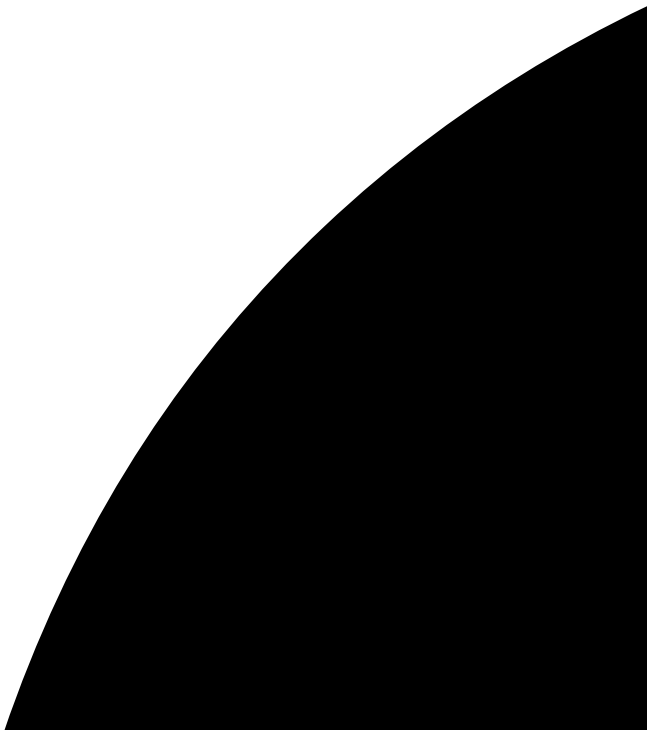


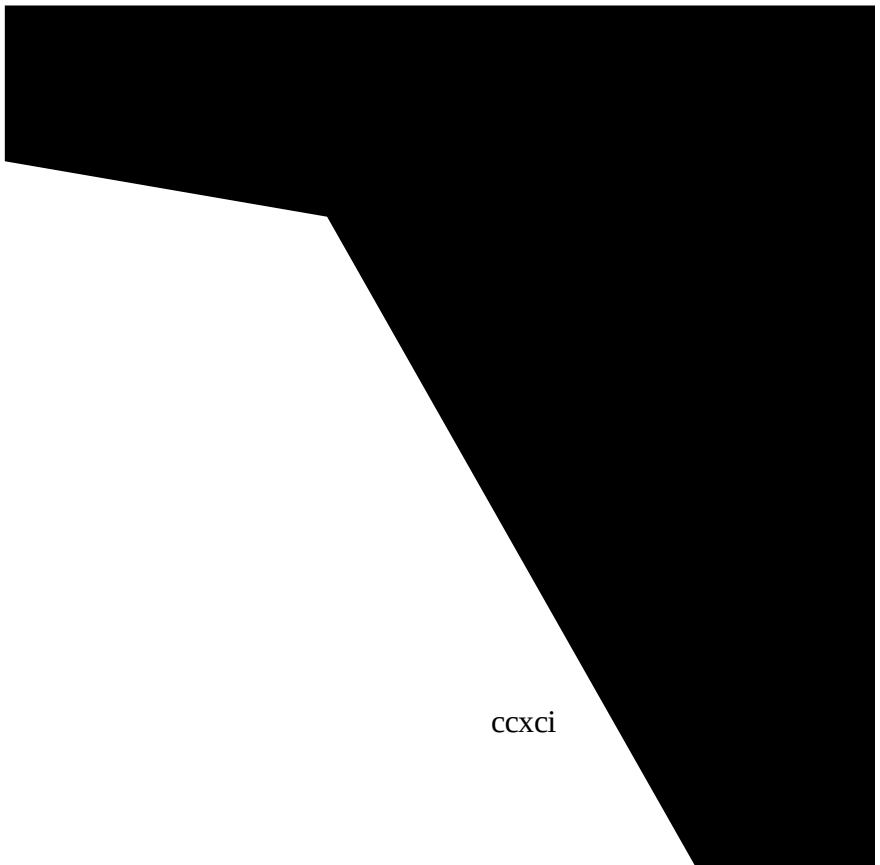


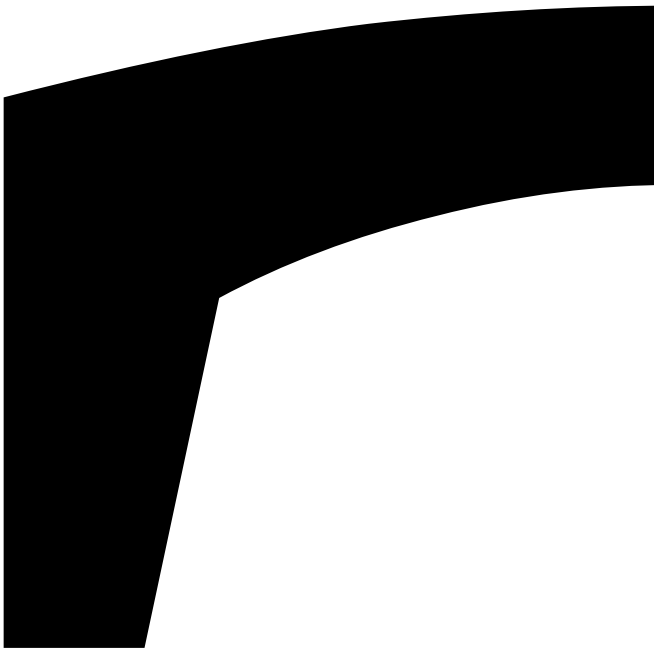




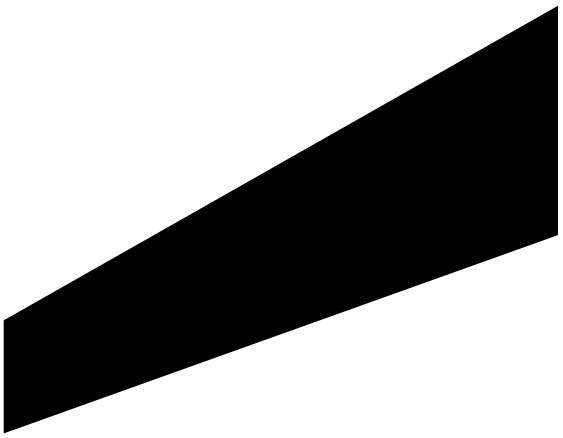


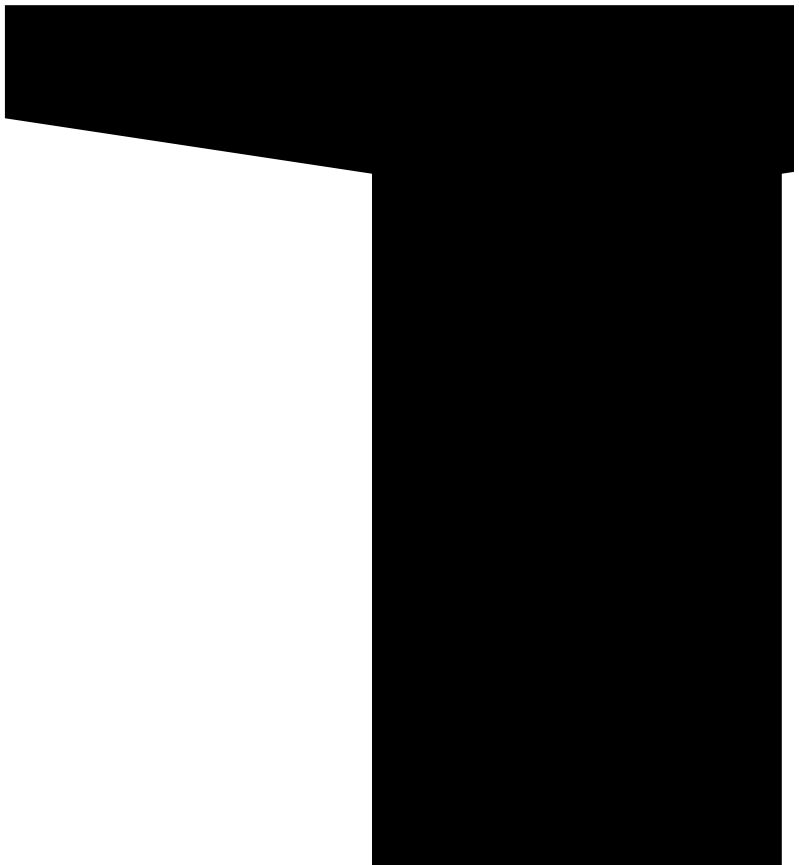






ccc



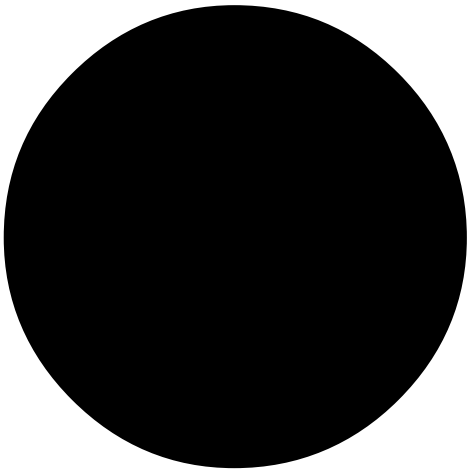


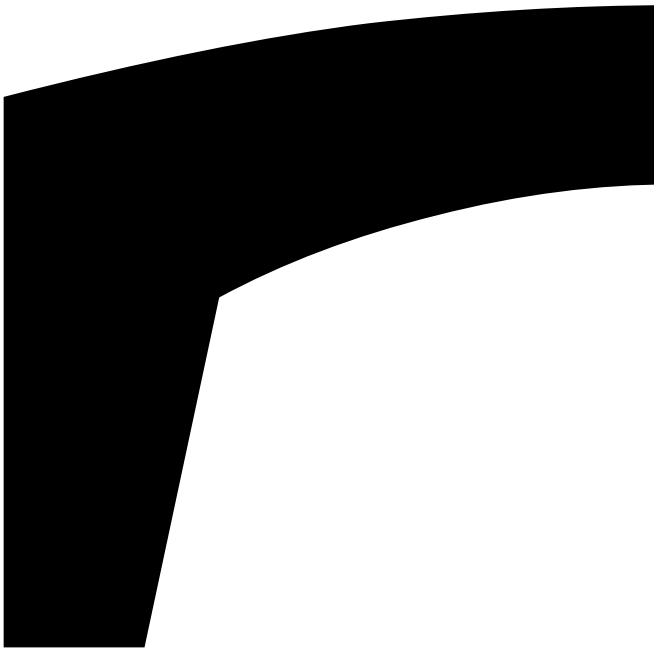


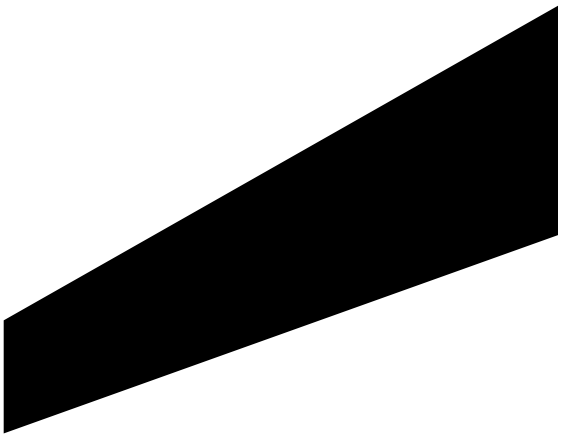


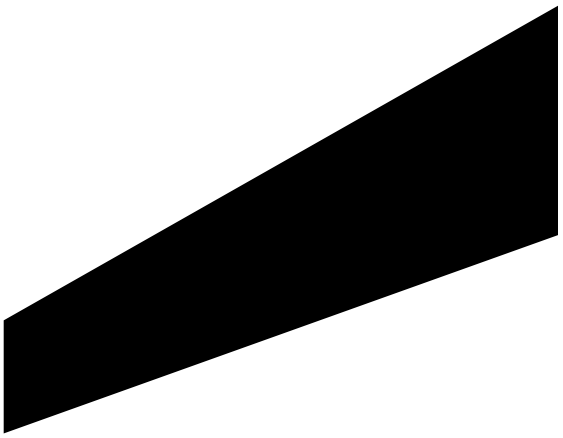


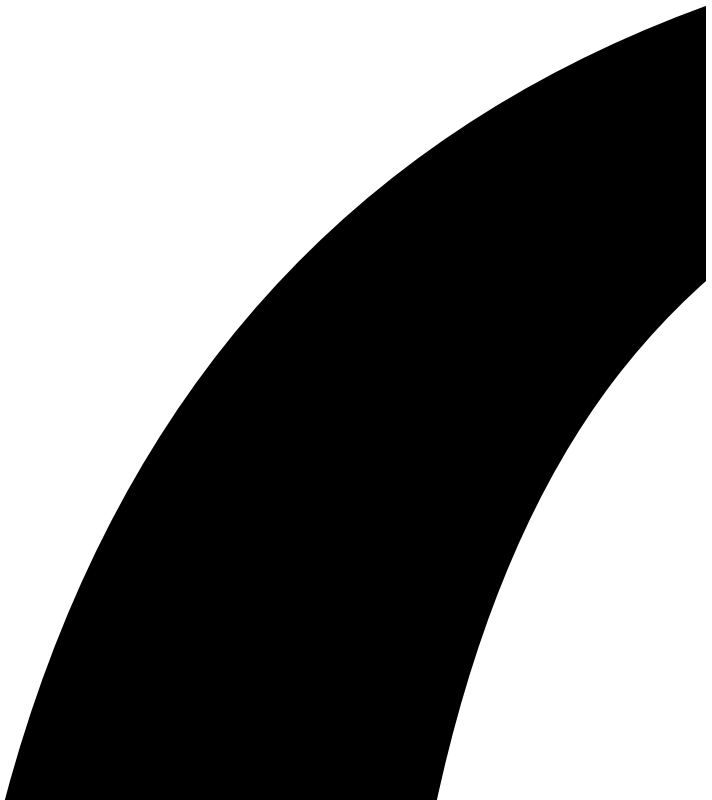


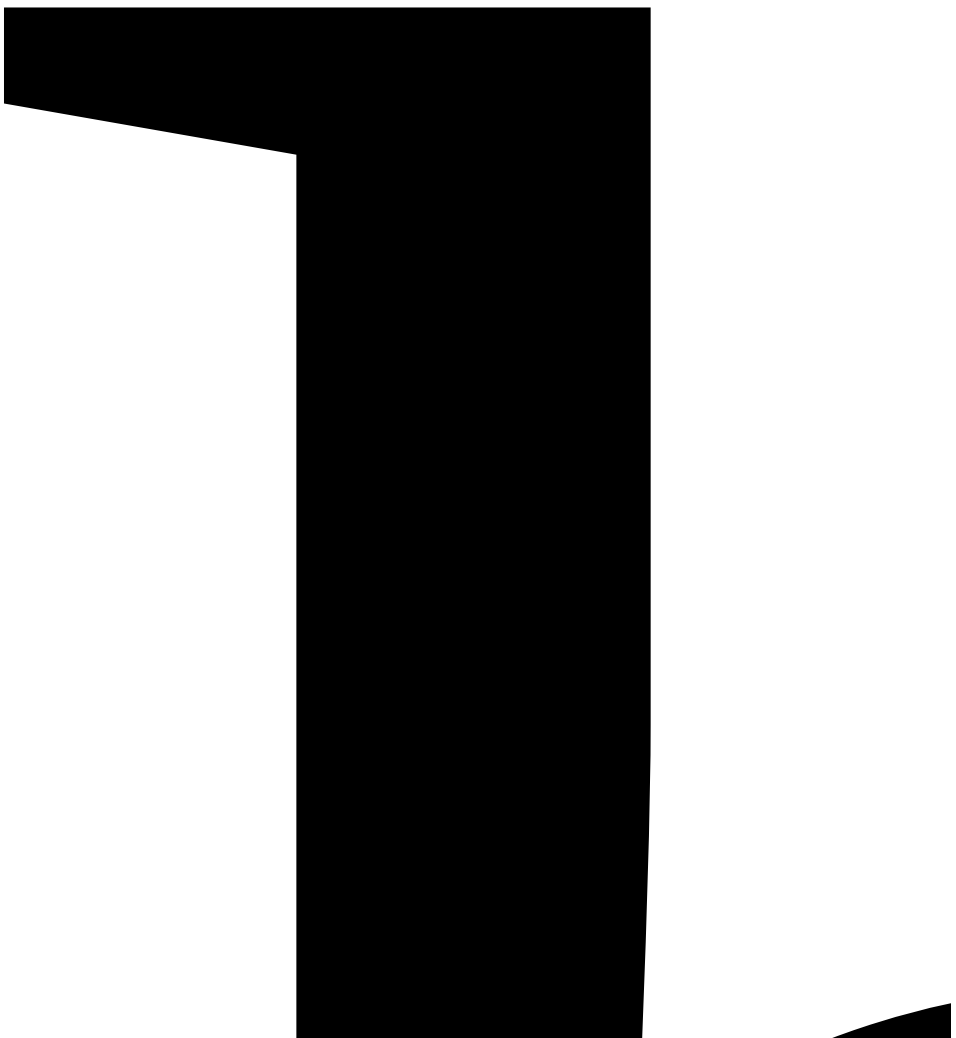


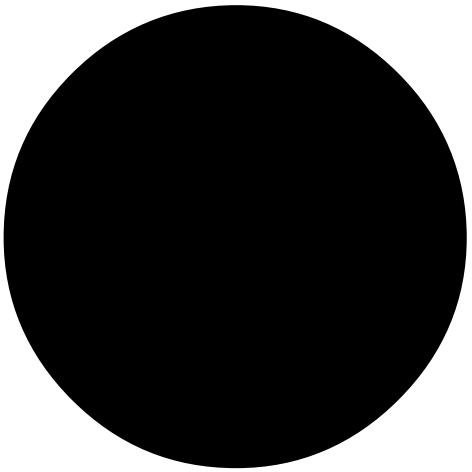






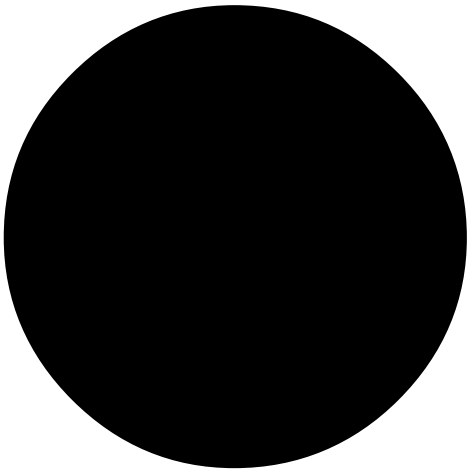




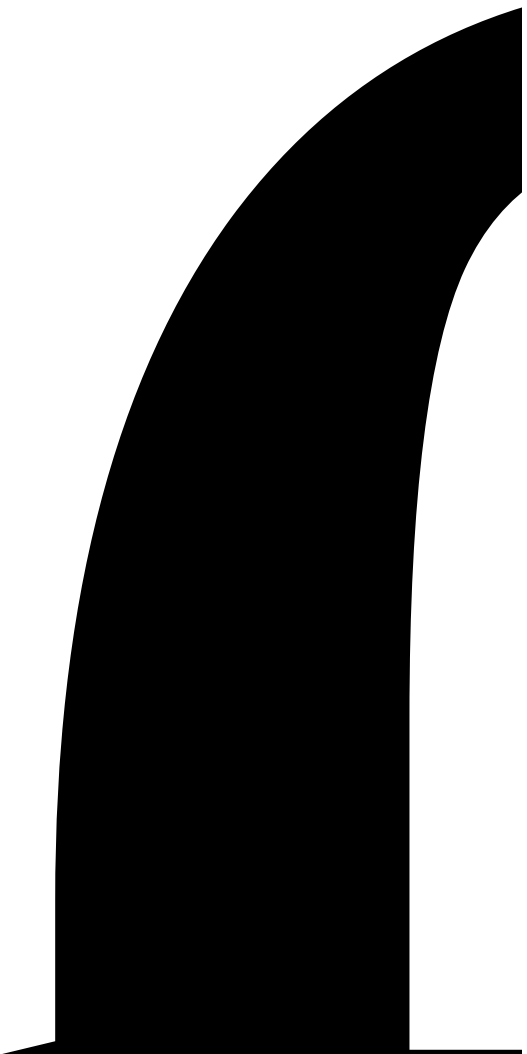








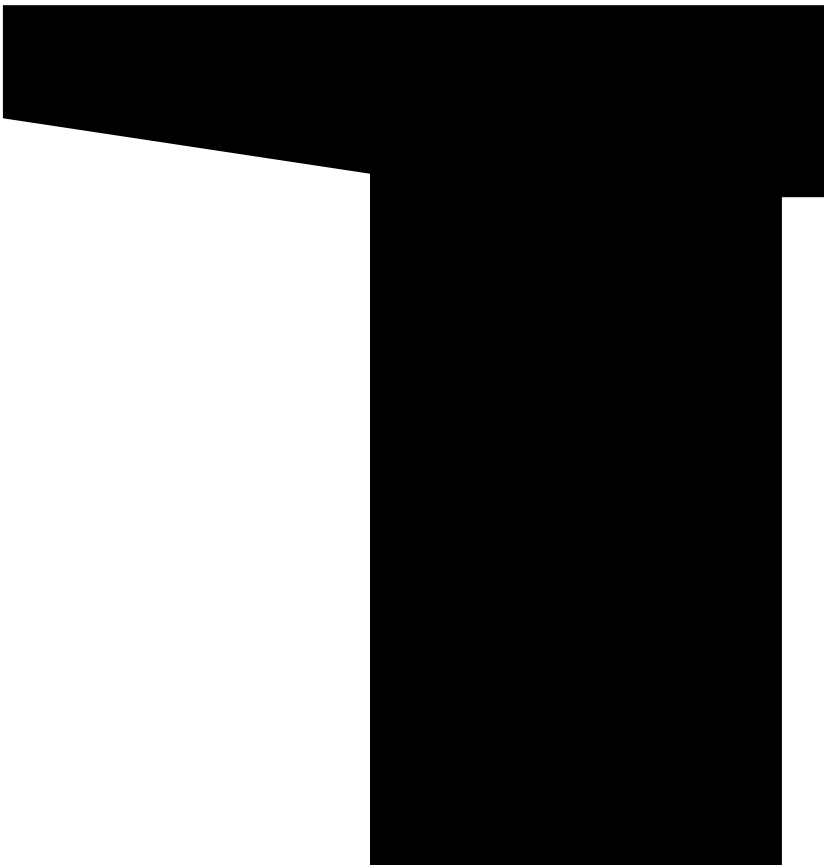






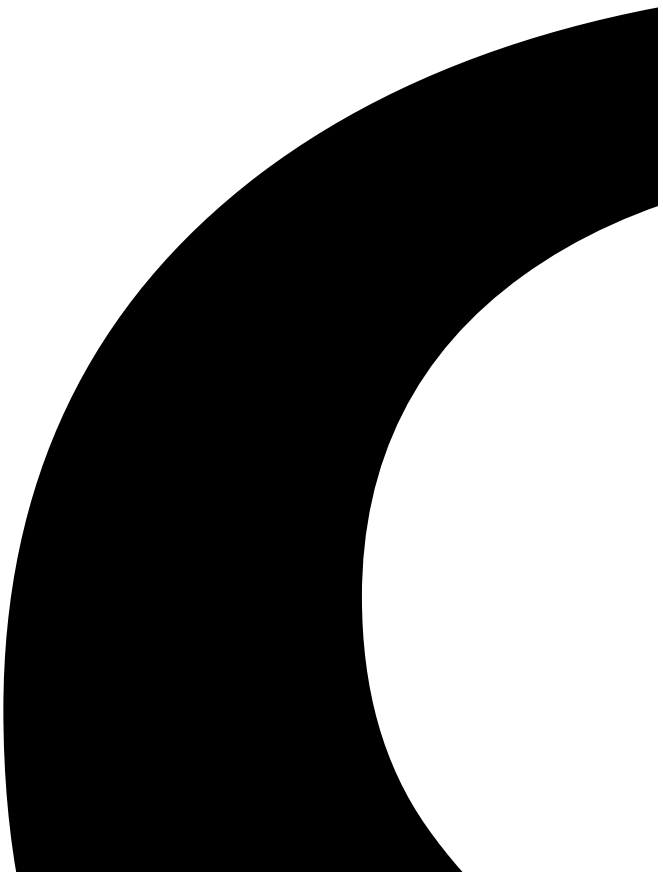




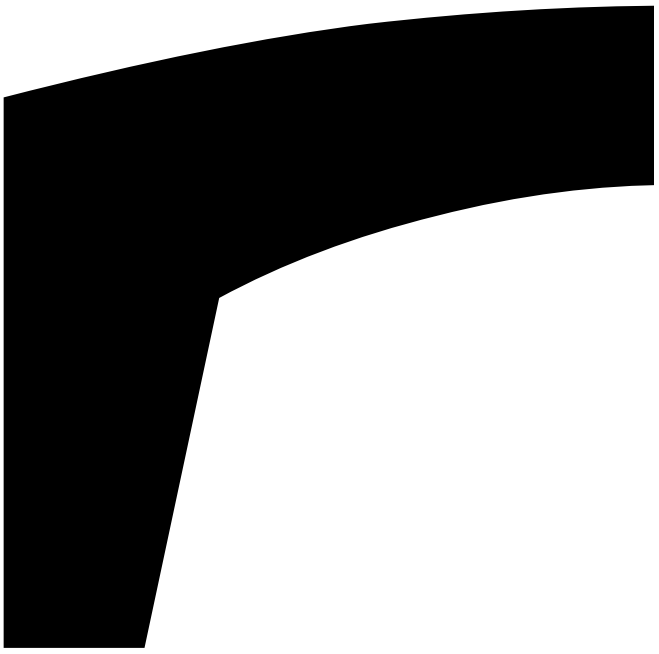




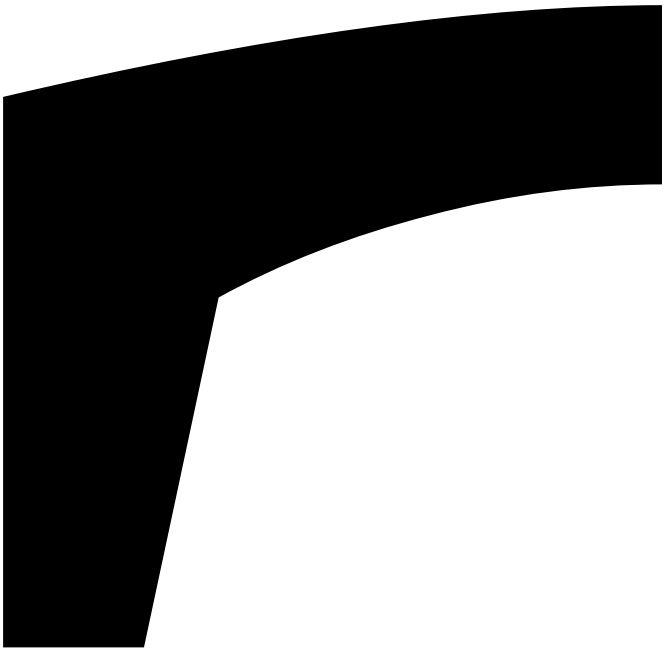


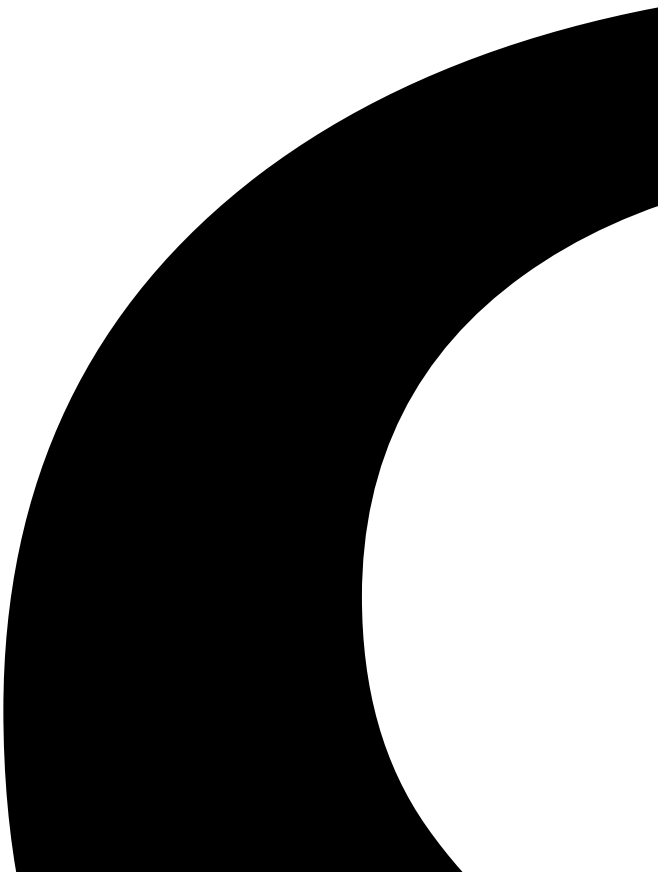






cccl

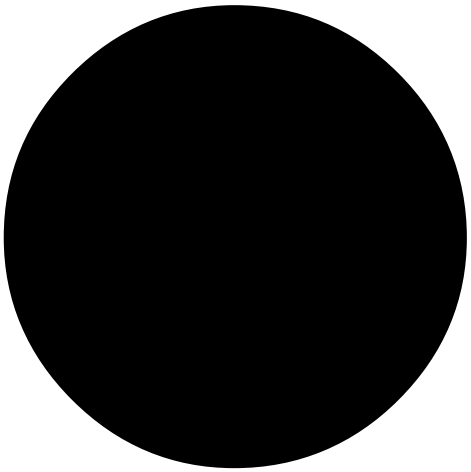






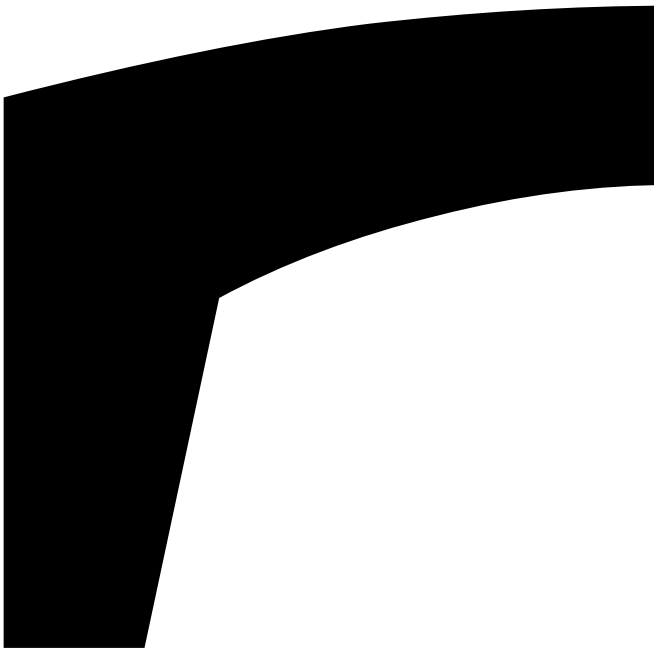


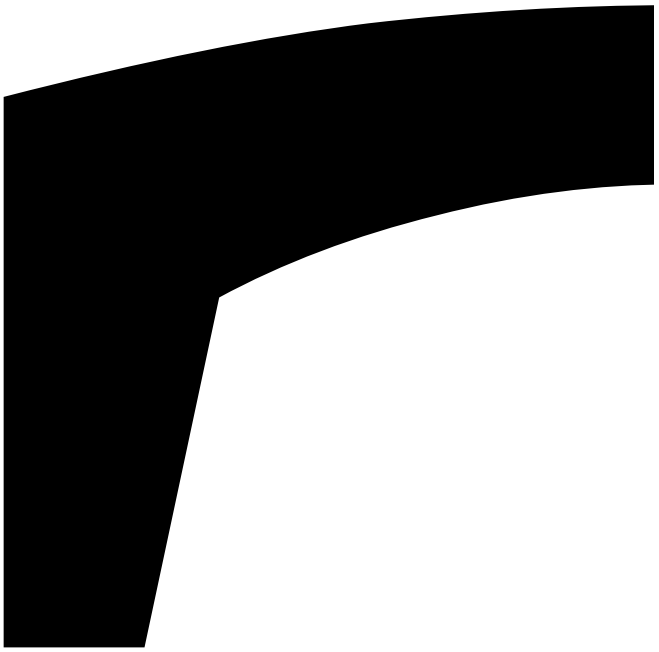


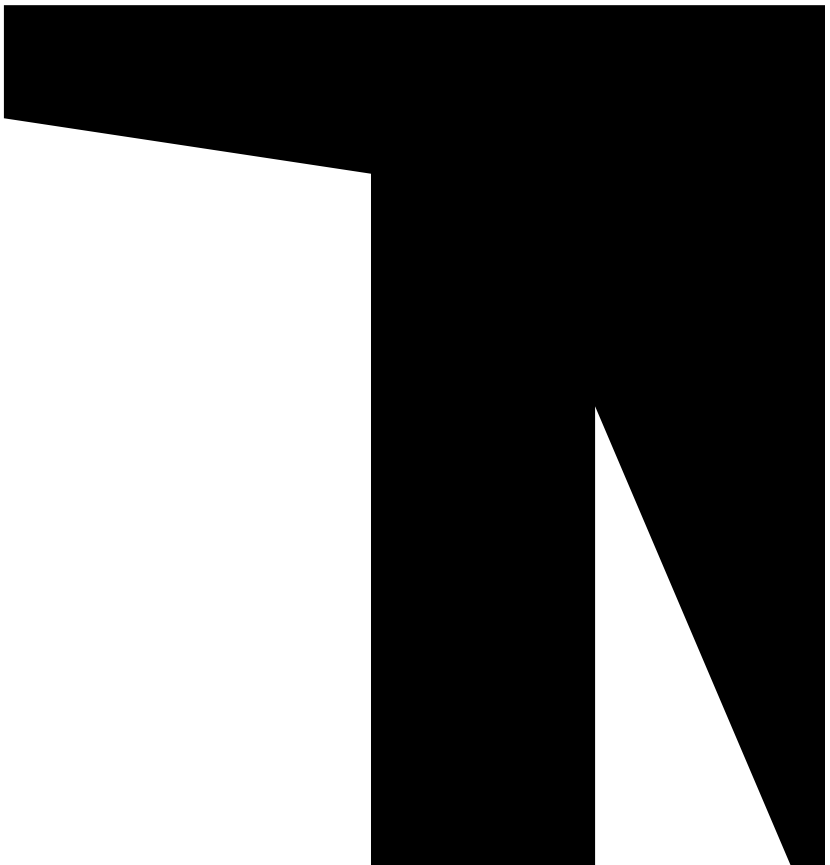




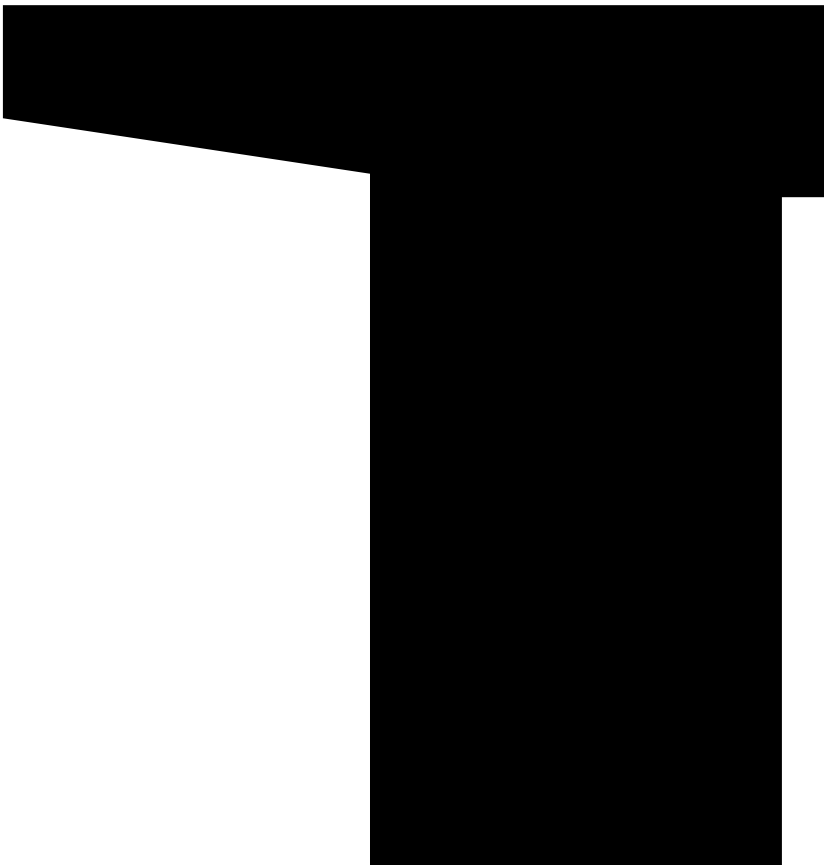


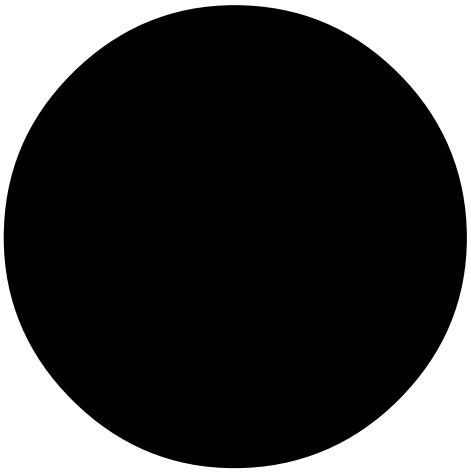






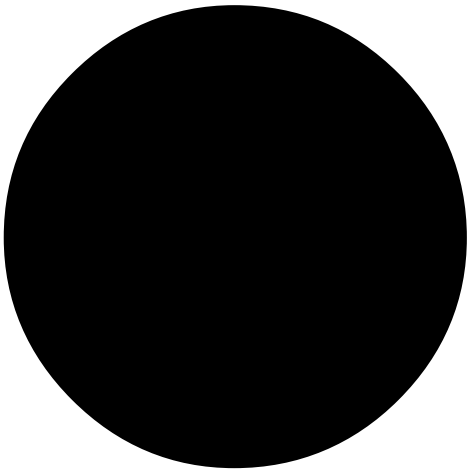




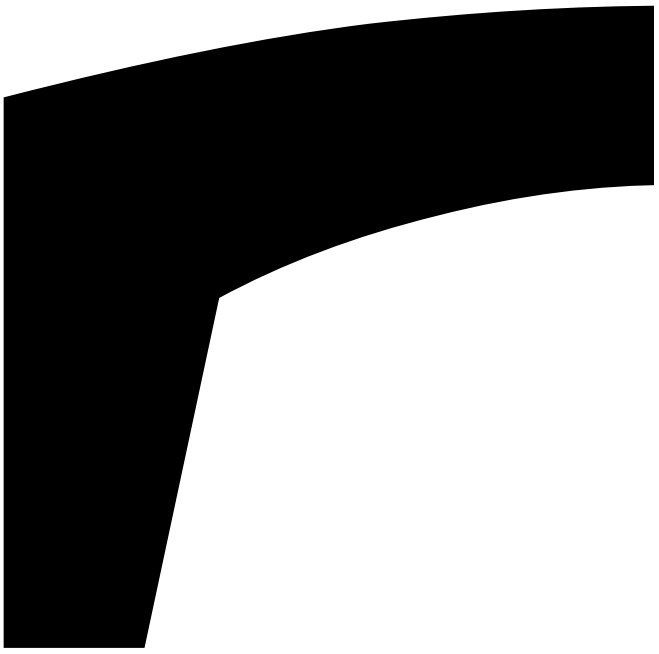








cccxc



cccxc

cccxc



cccxc

cccxc

cccxc



cccxc

cccxc

cd

cdi



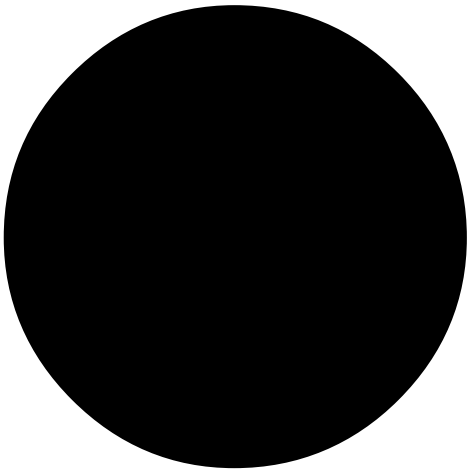
cdii

cdiv

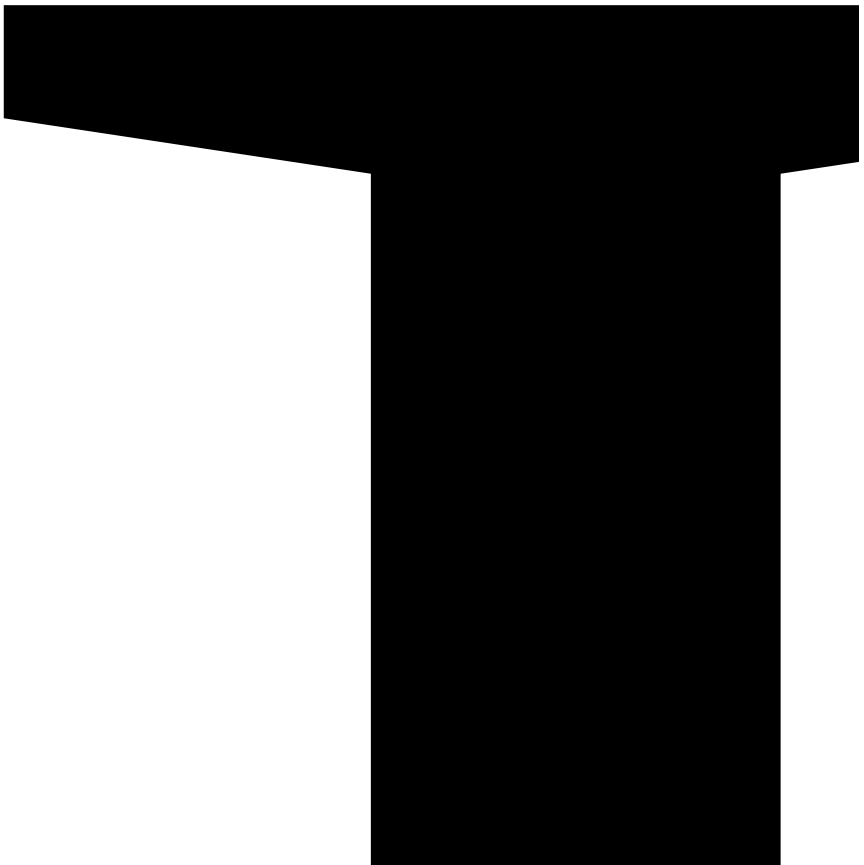


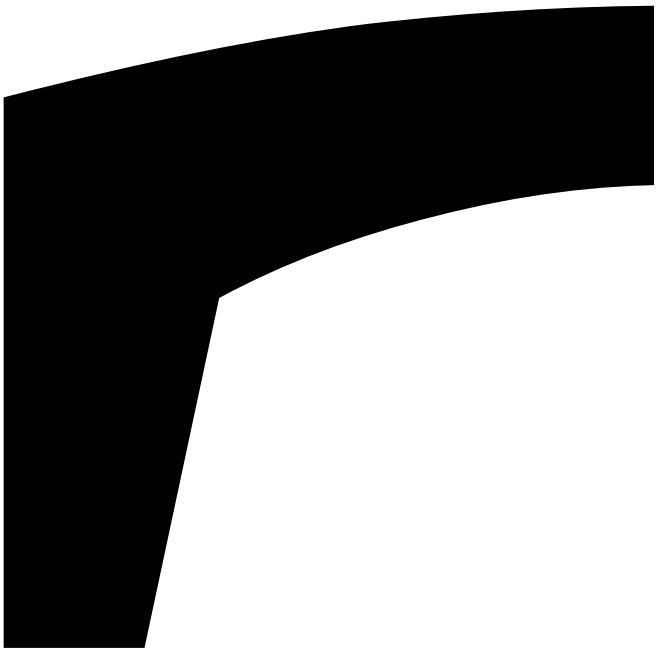


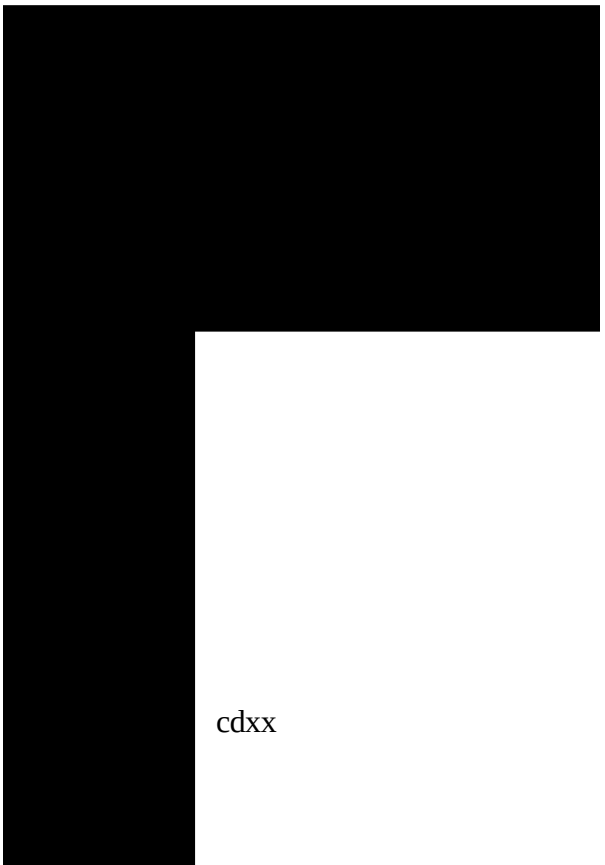




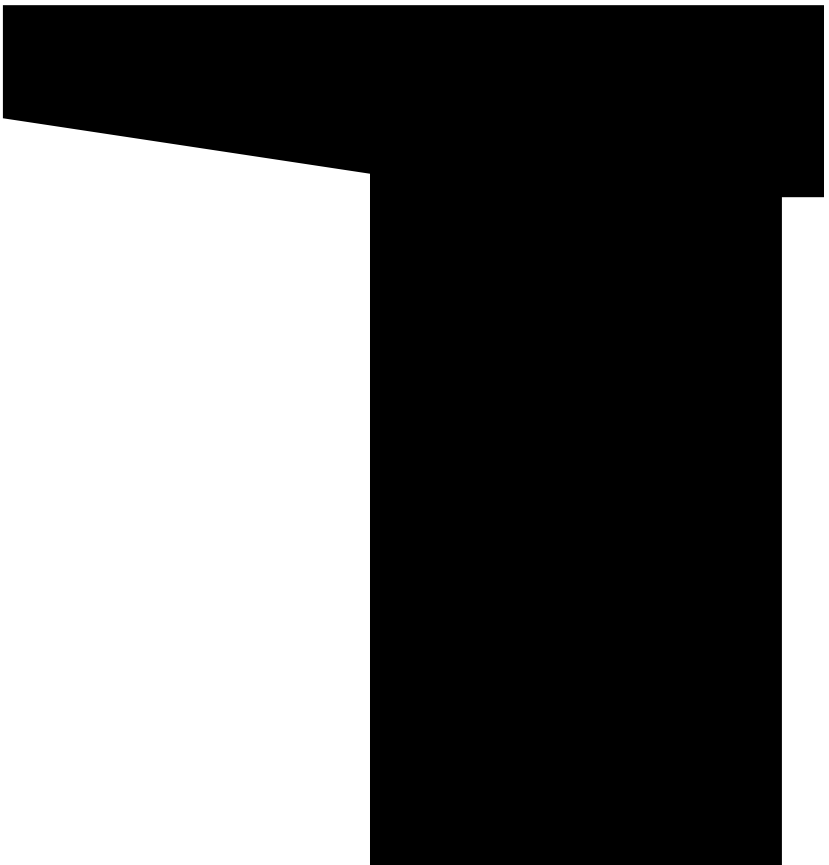




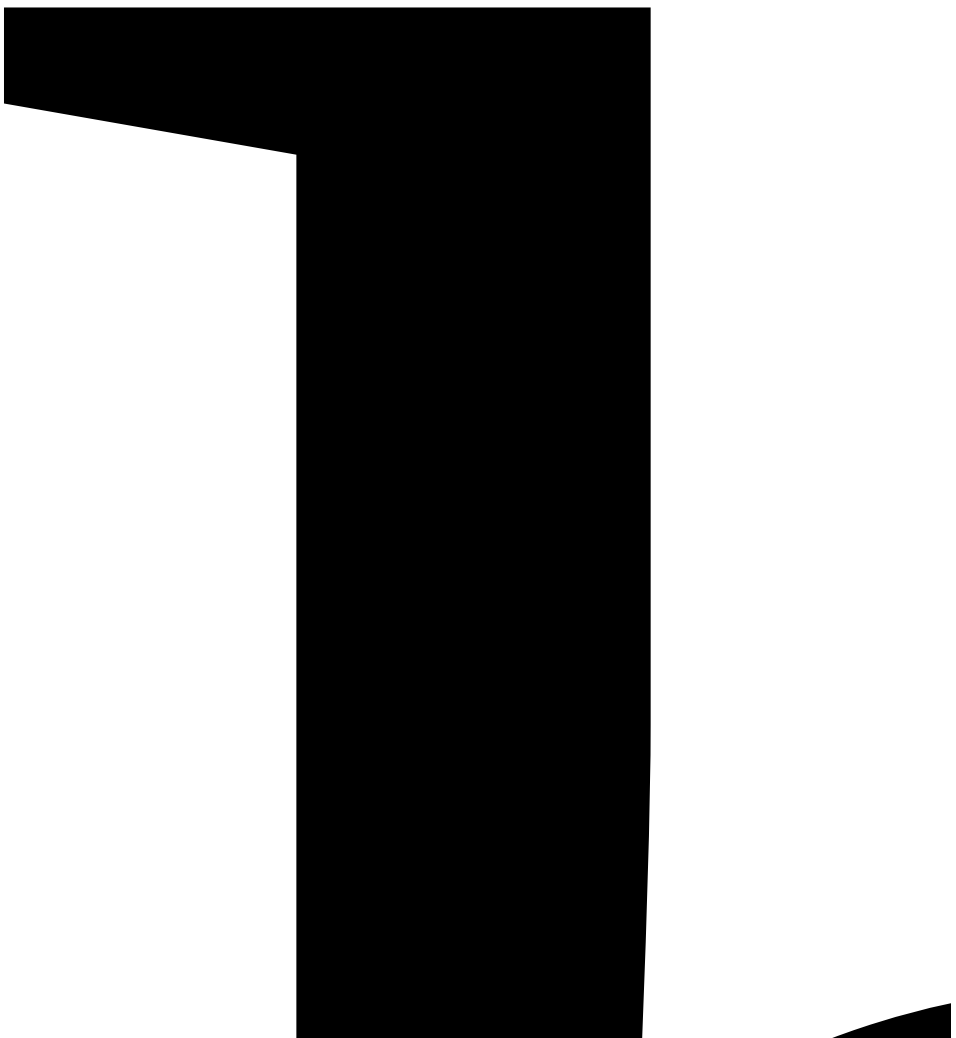


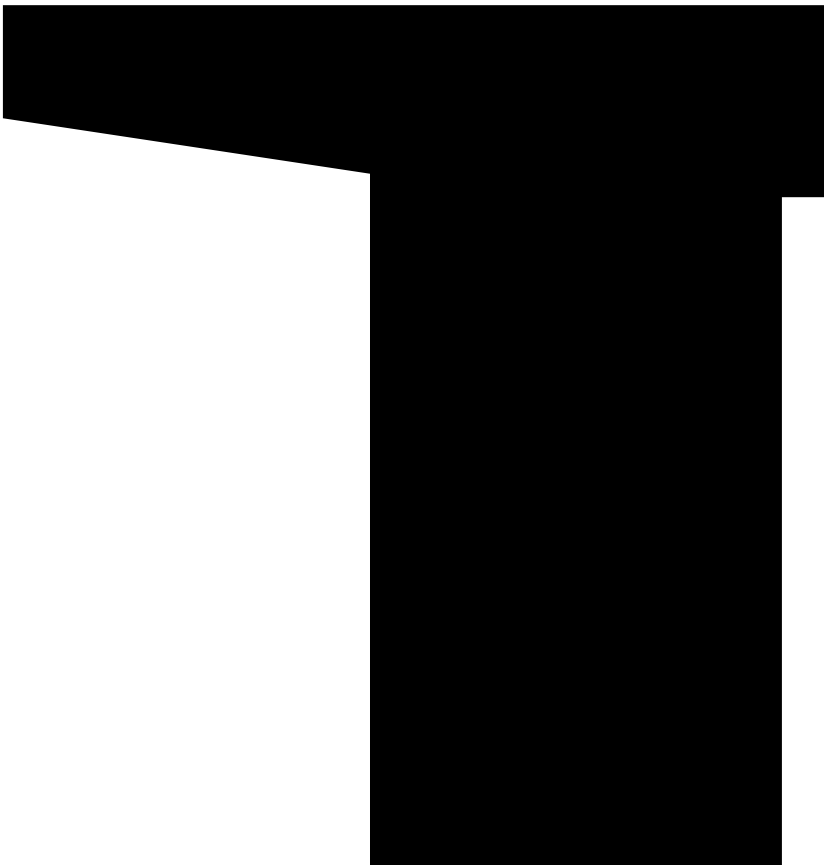


cdxx



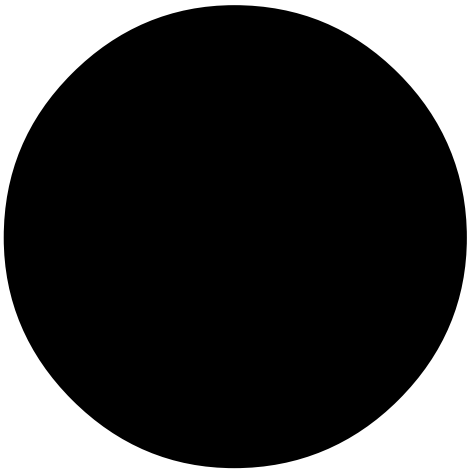






cdxx

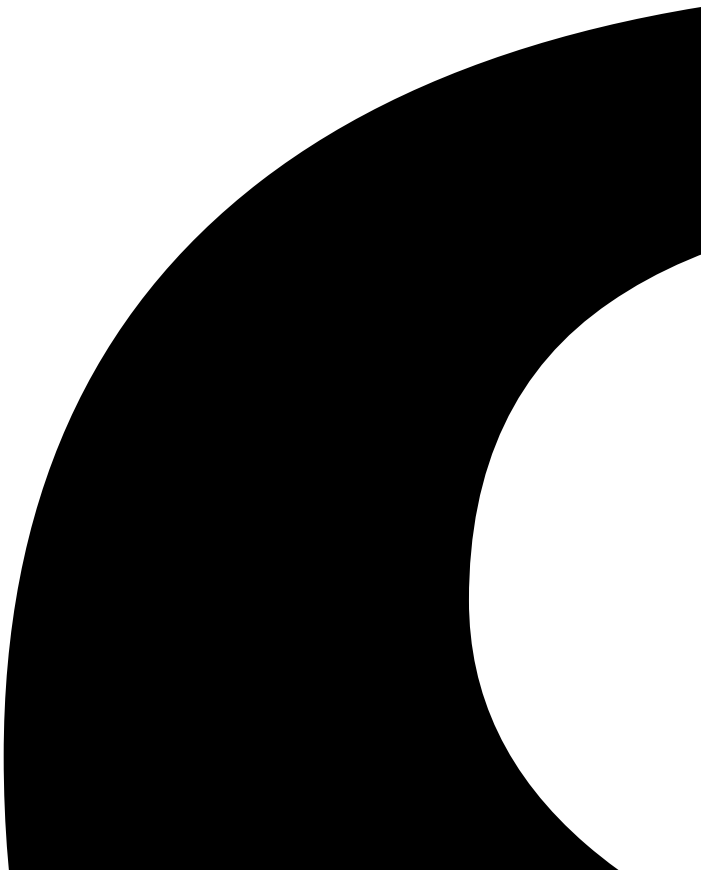
cdxx

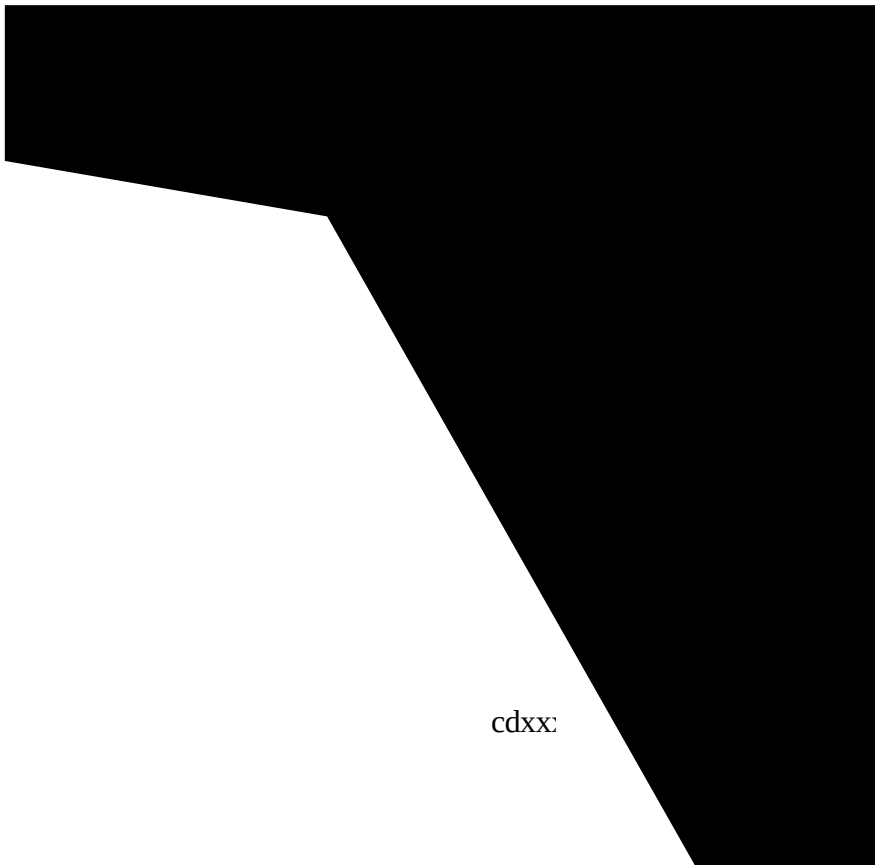


cdxx

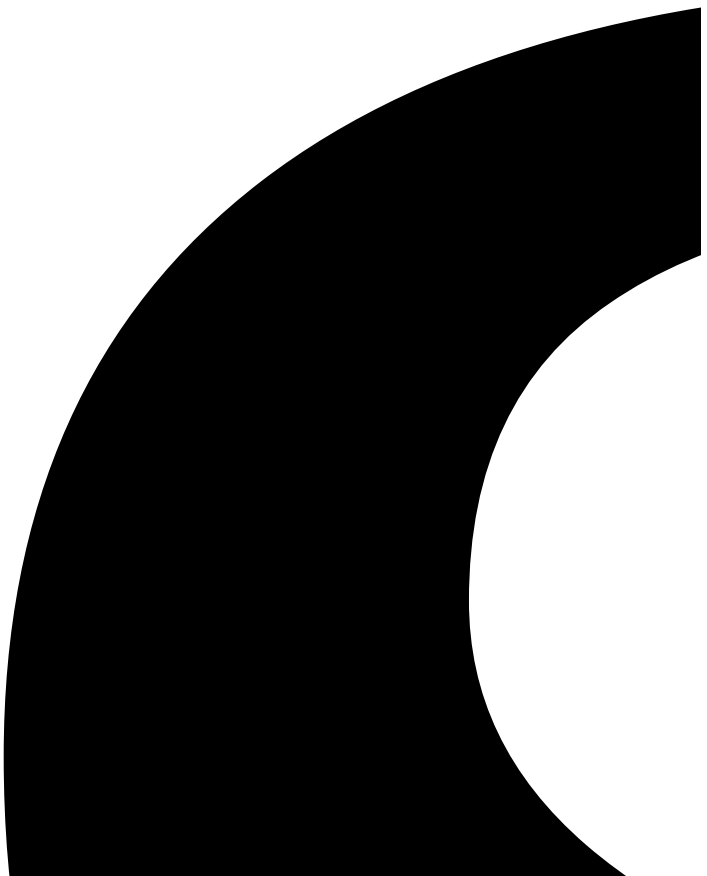
cdxx

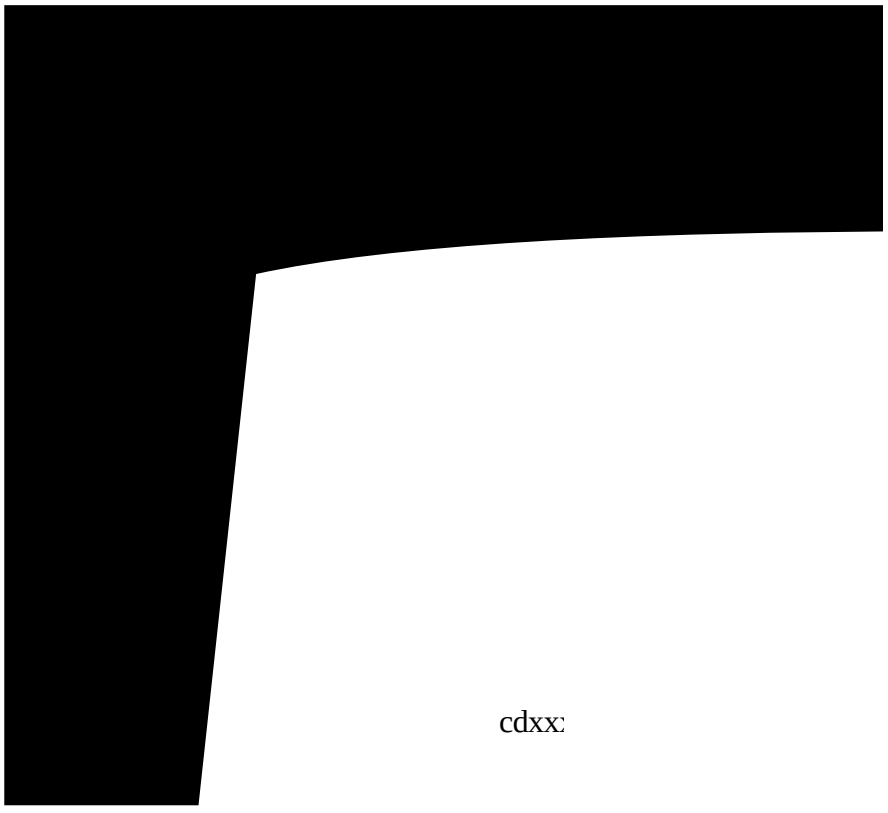
cdxx



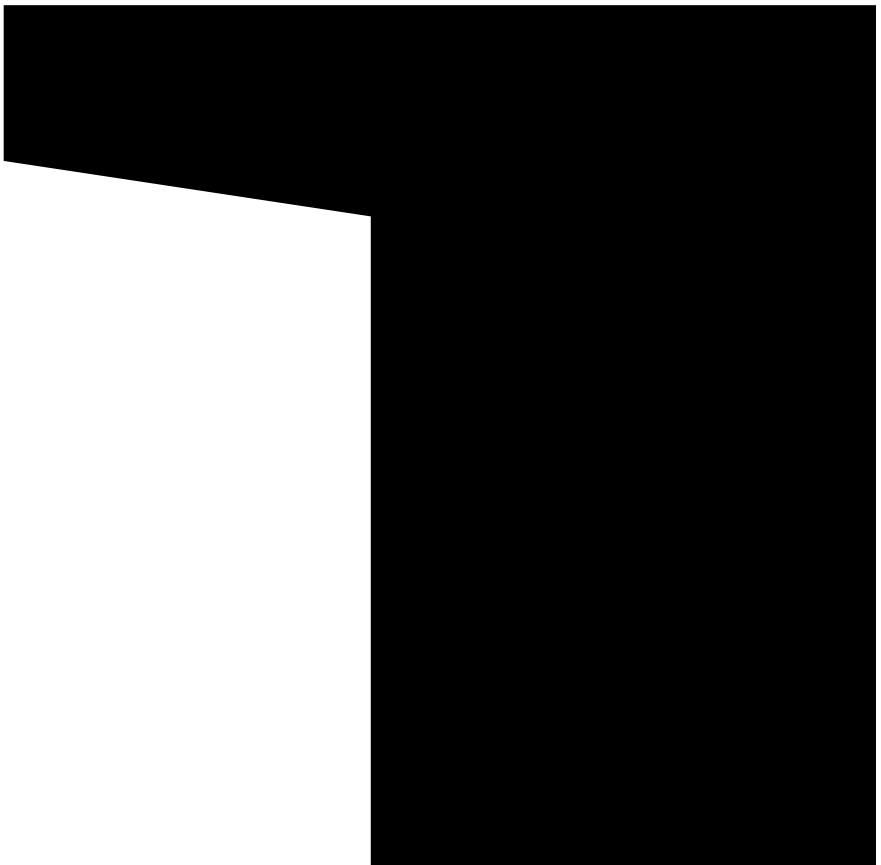


cdxx

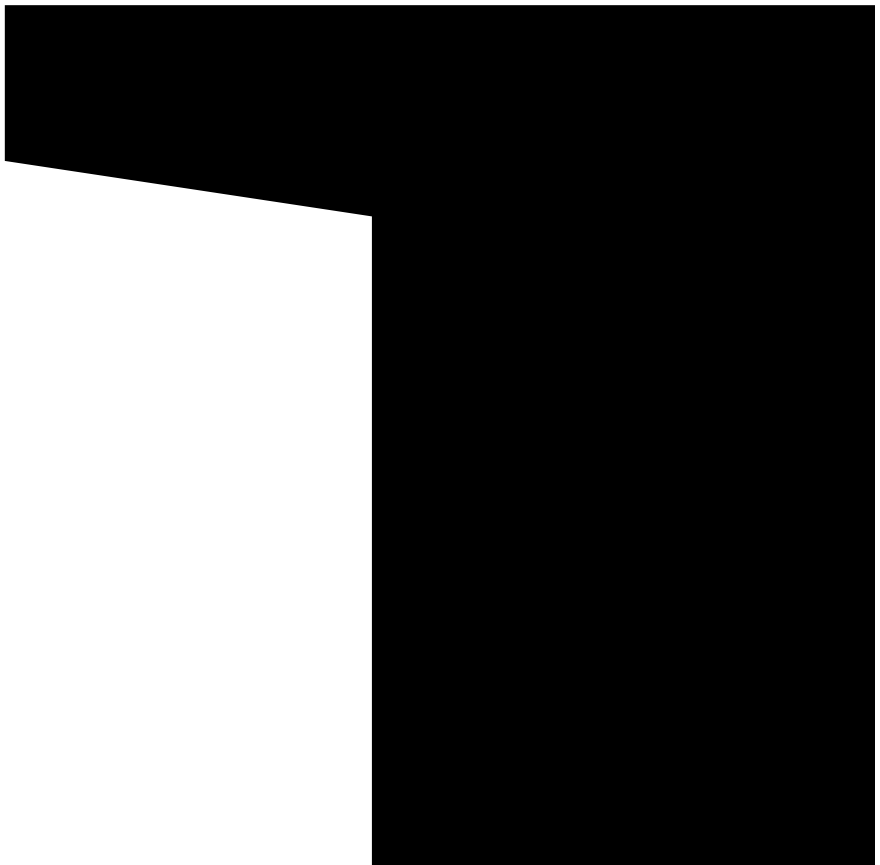


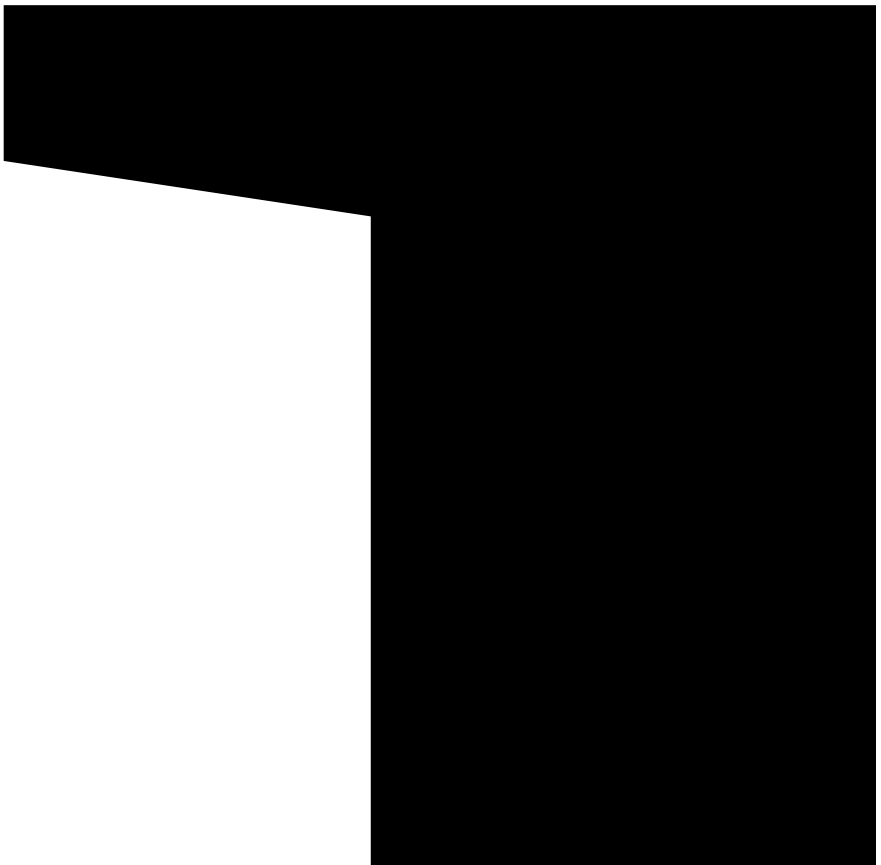


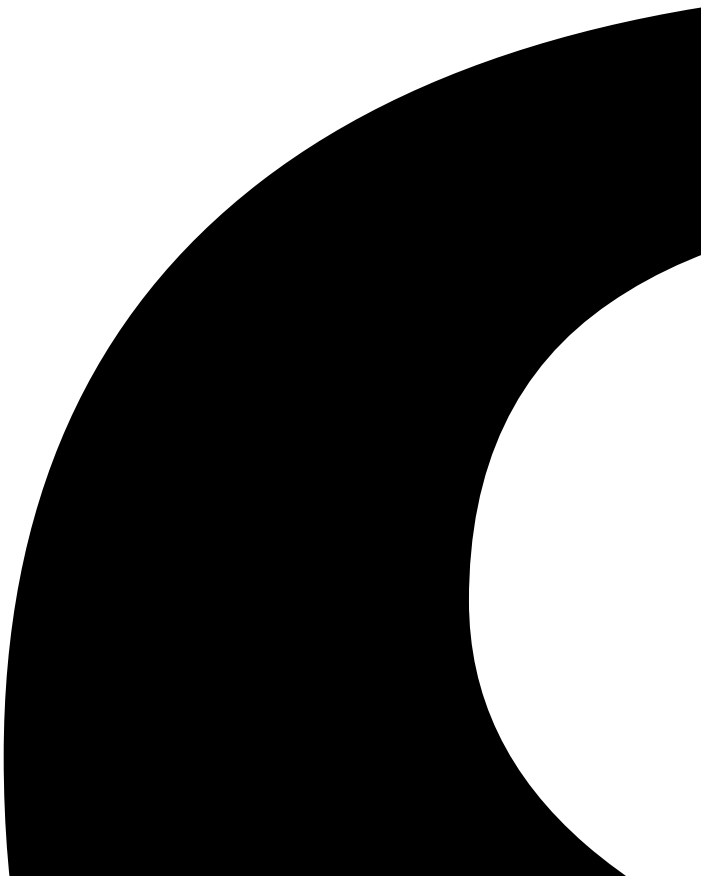
cdxx

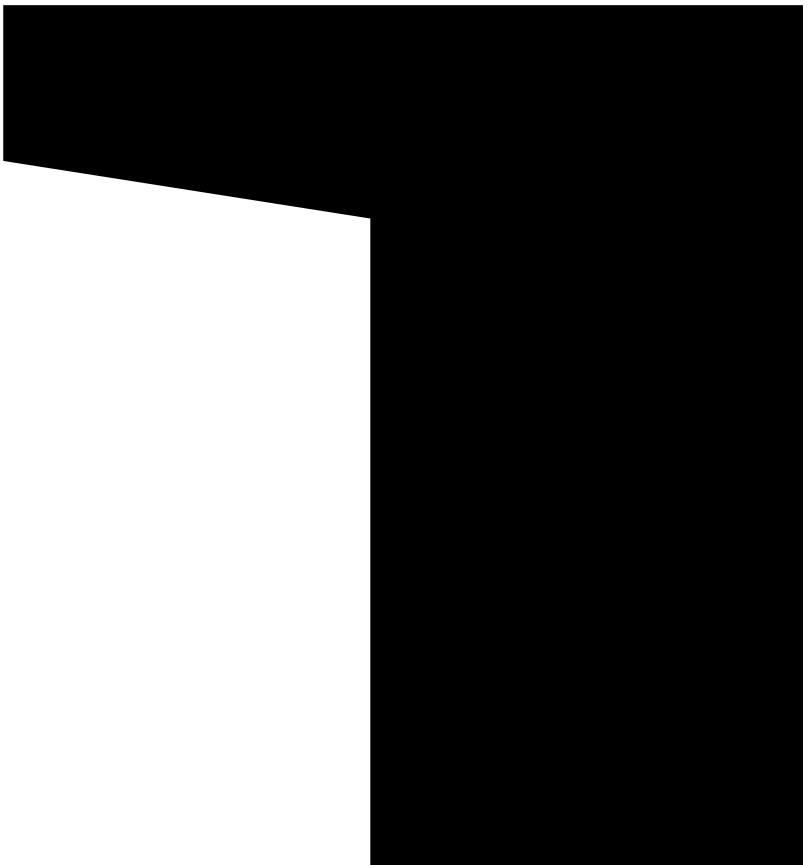


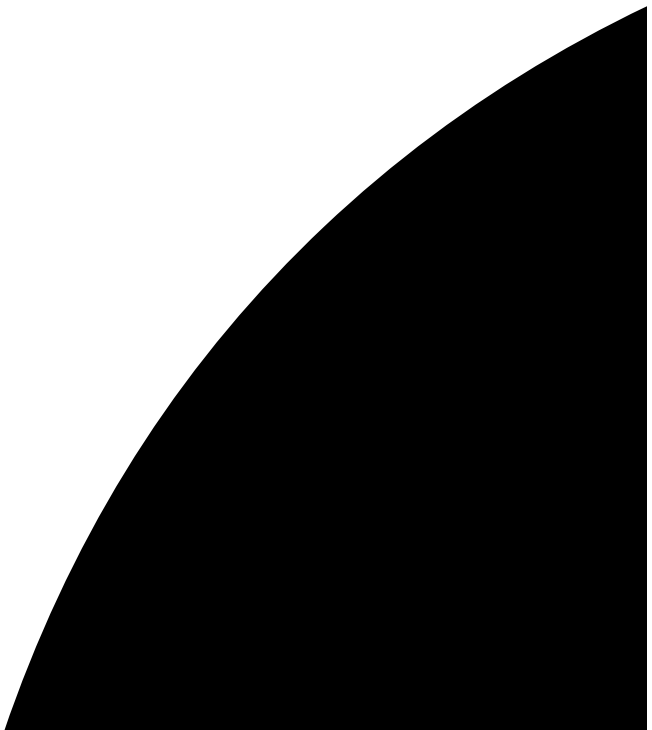


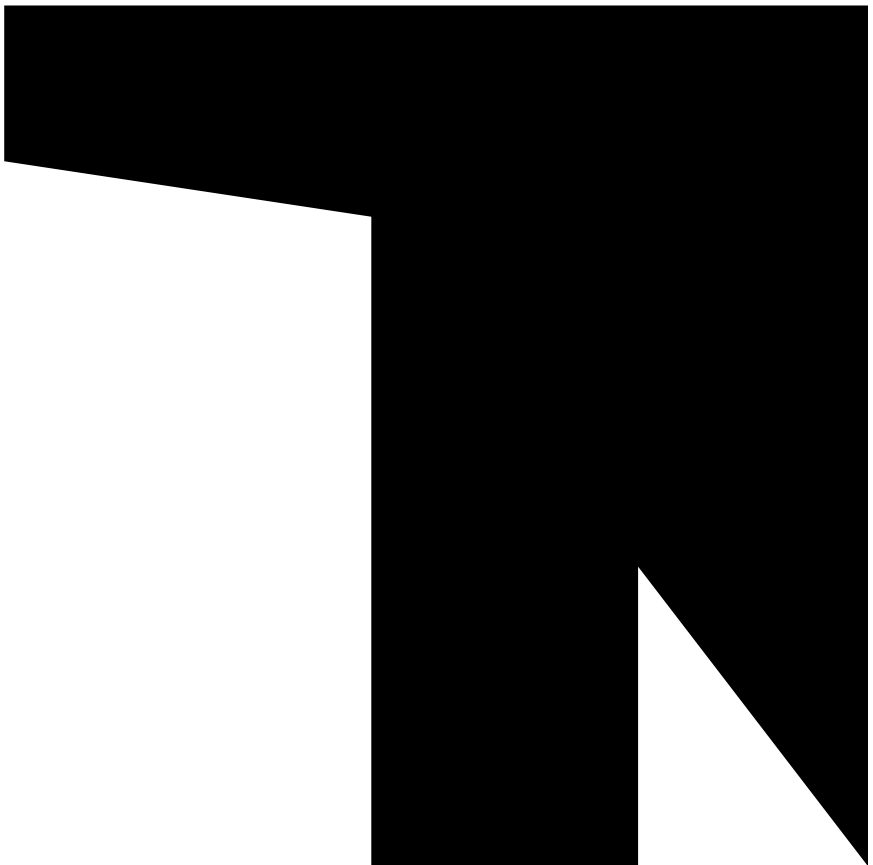


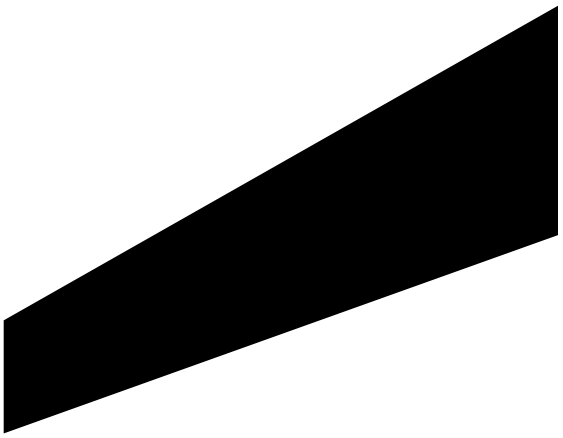


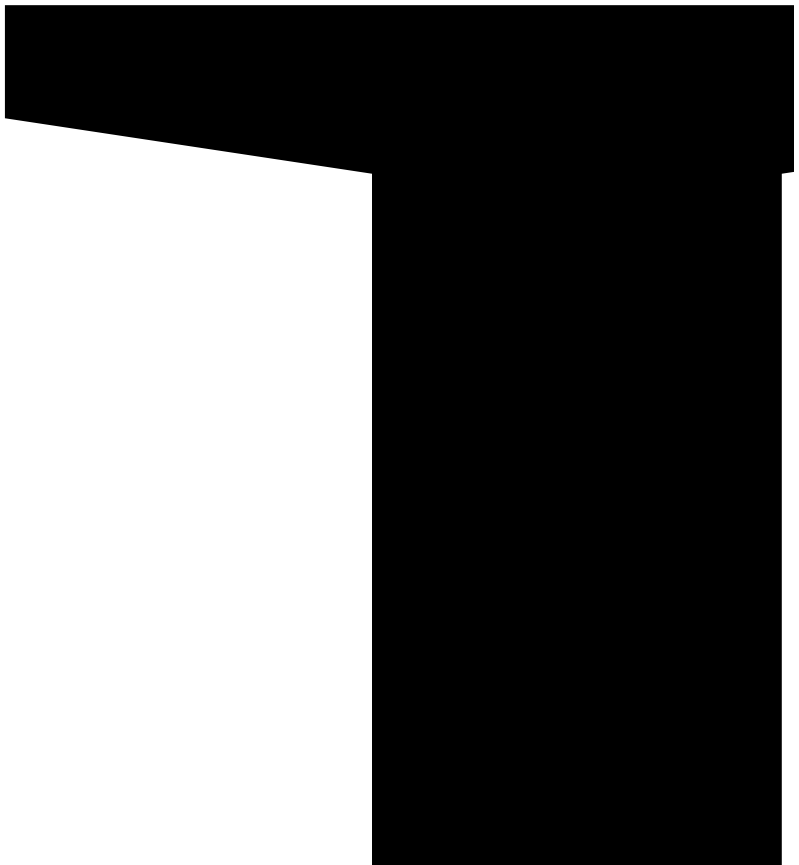








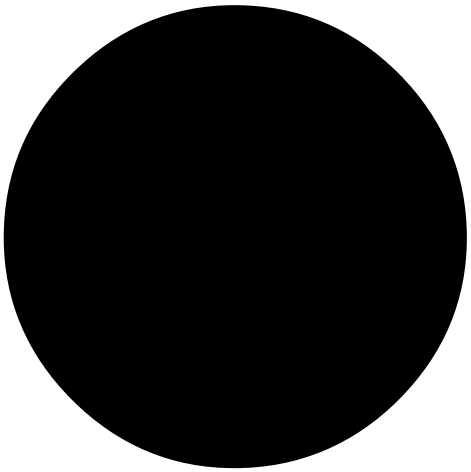


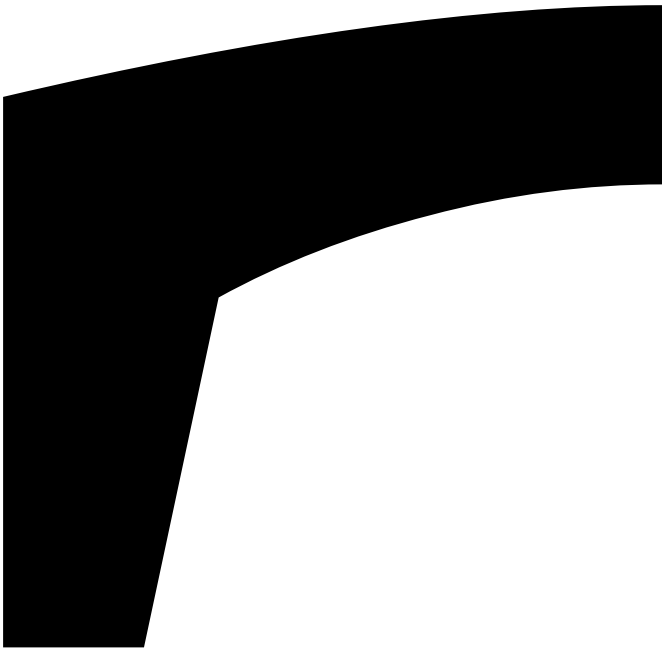


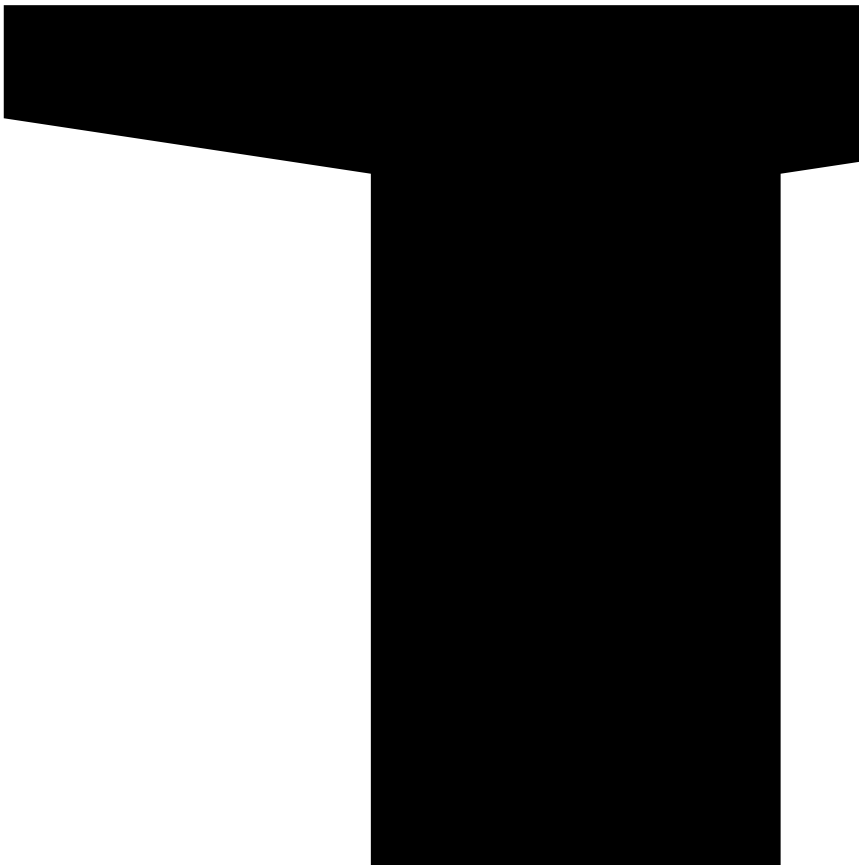


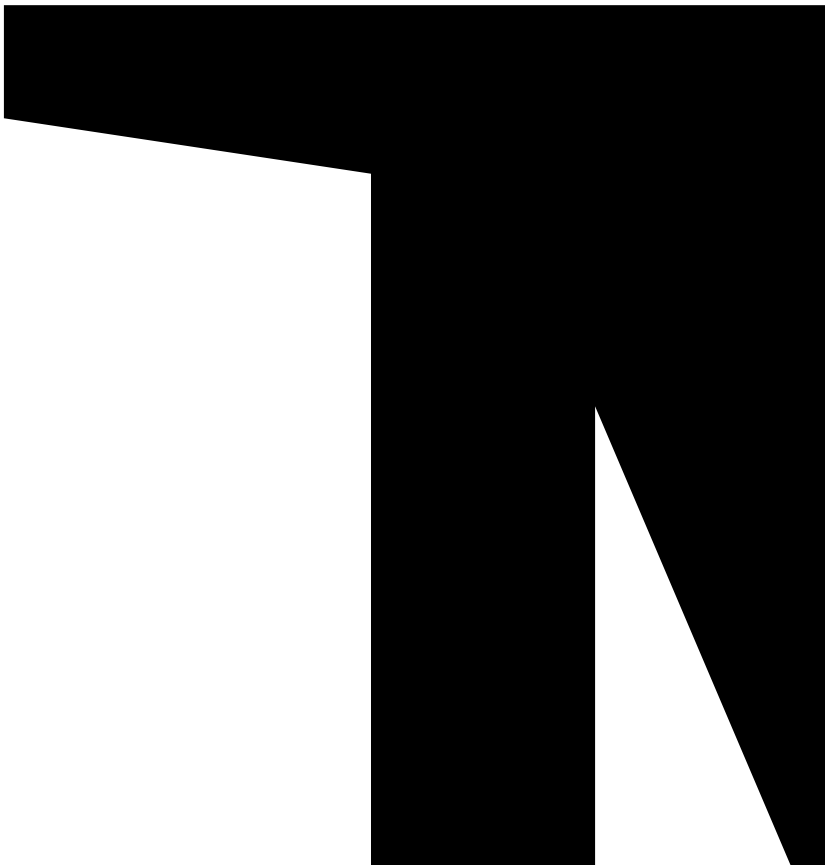


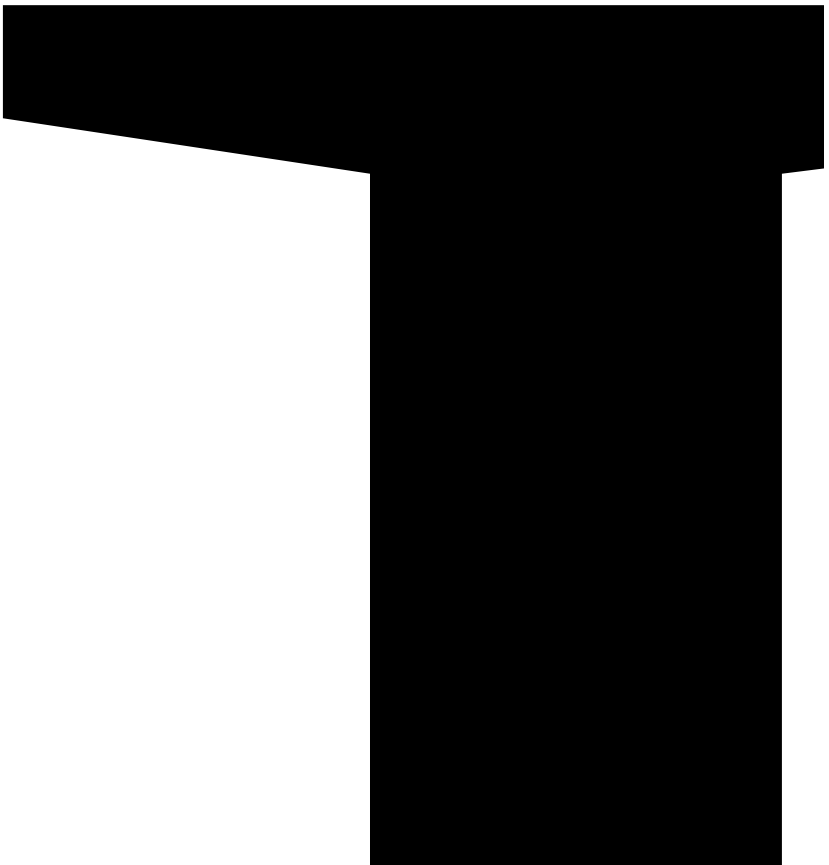


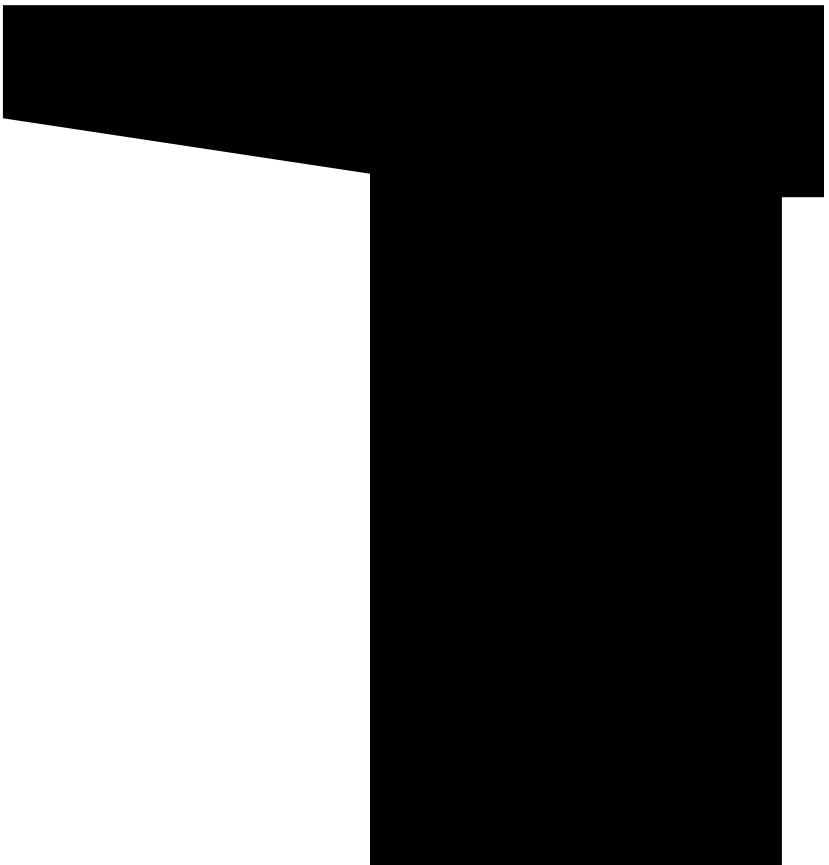


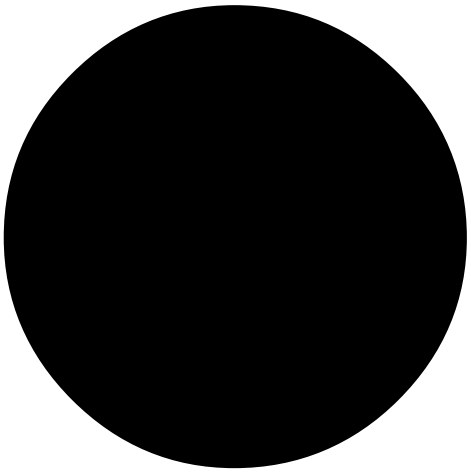


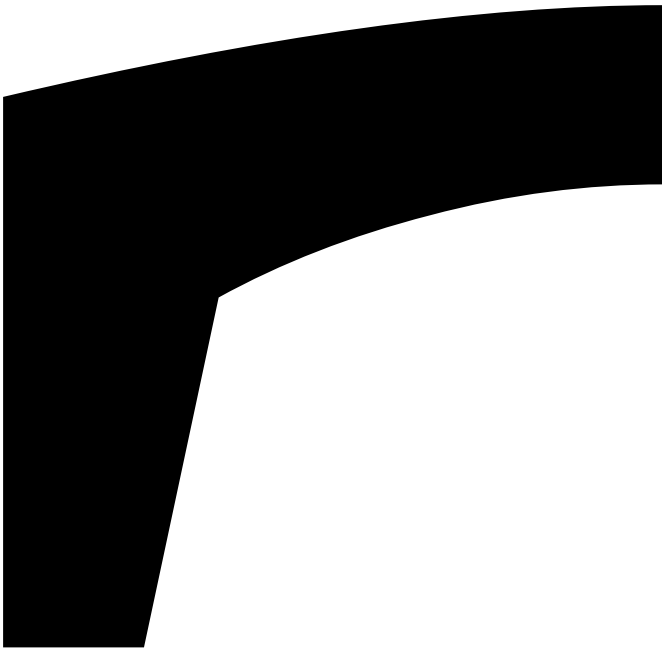




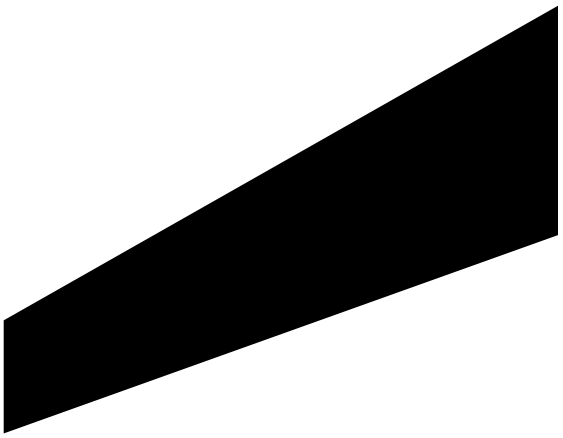


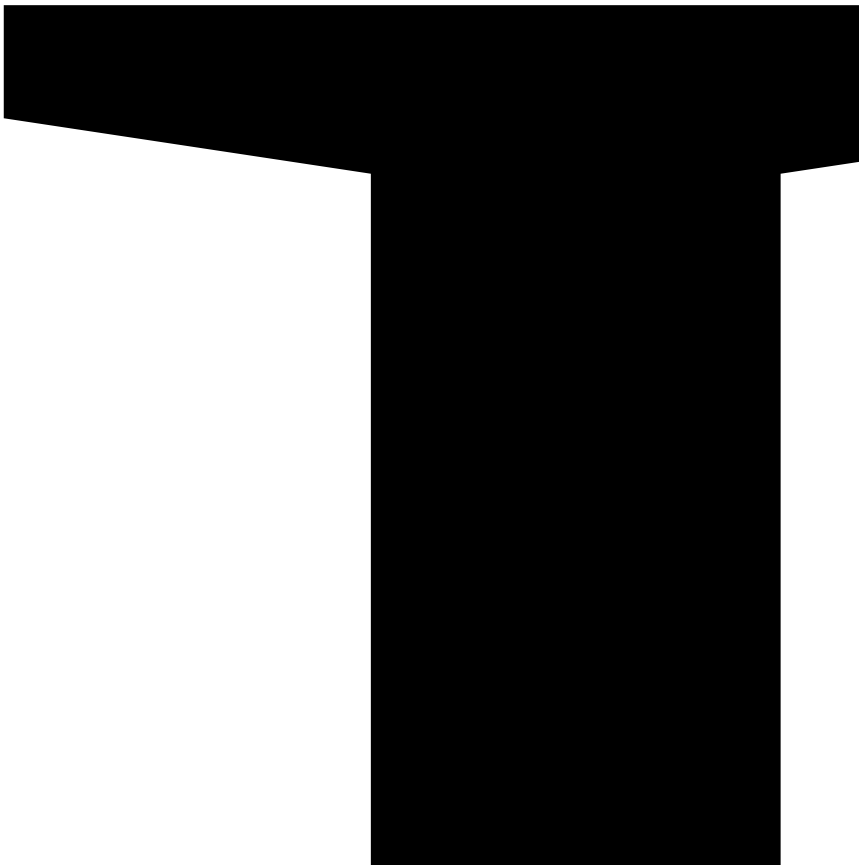


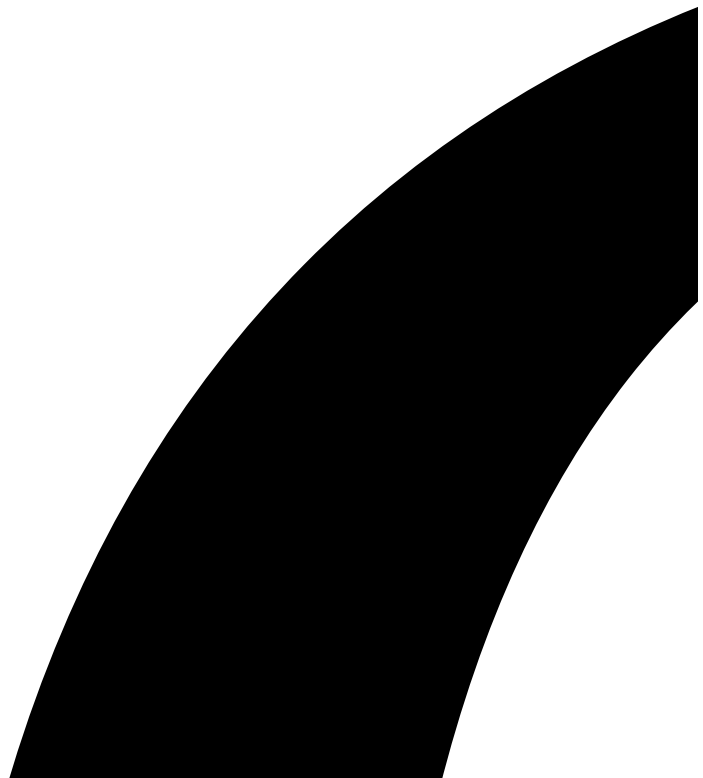


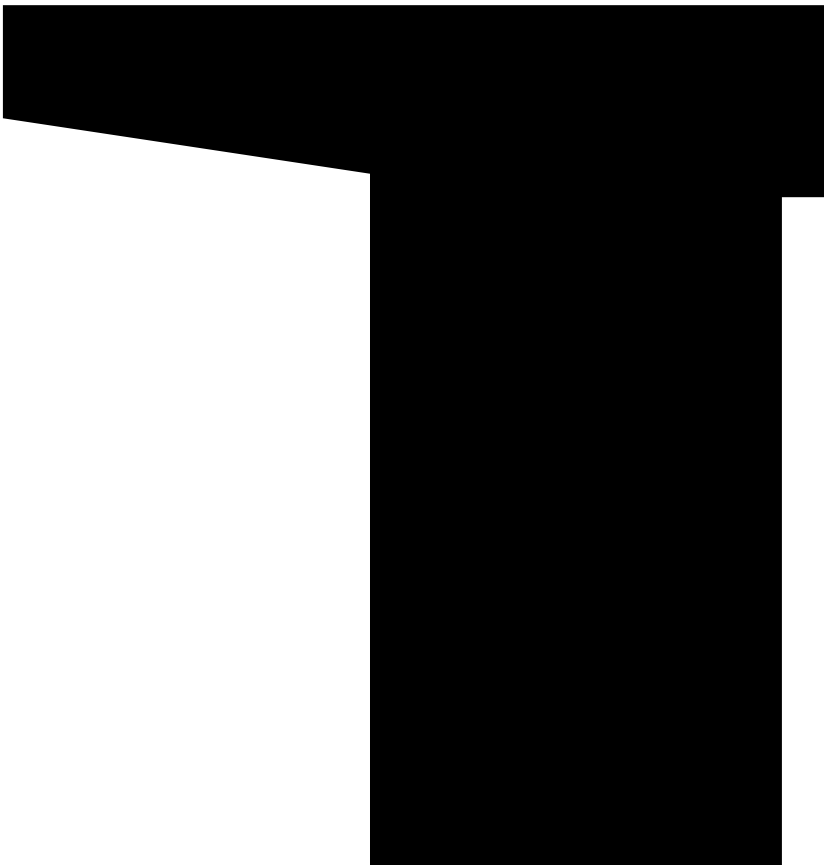


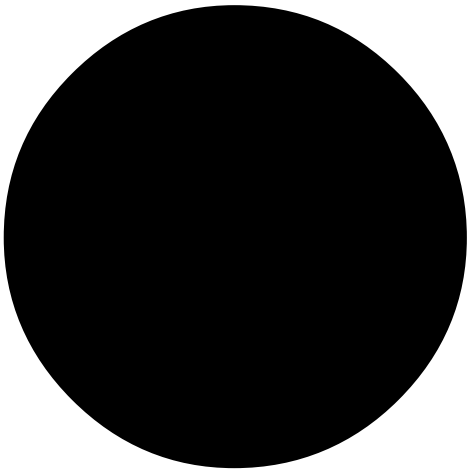
cdlxx











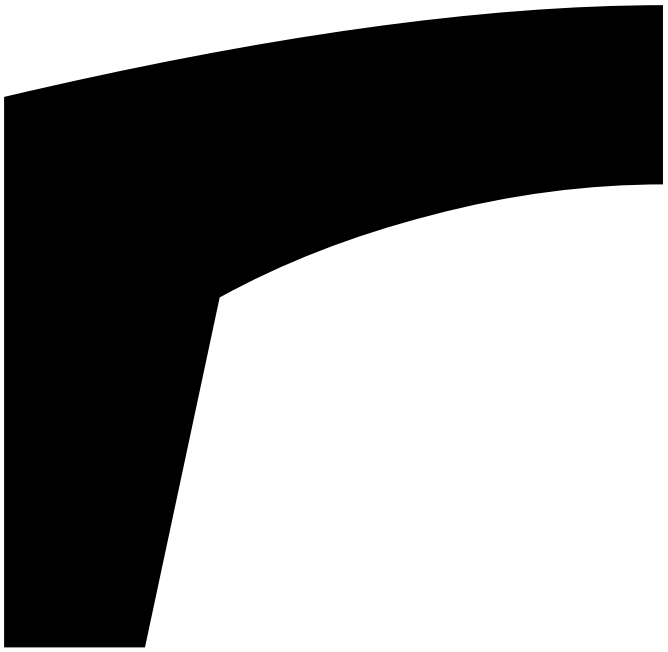
cdxc

cdxcv

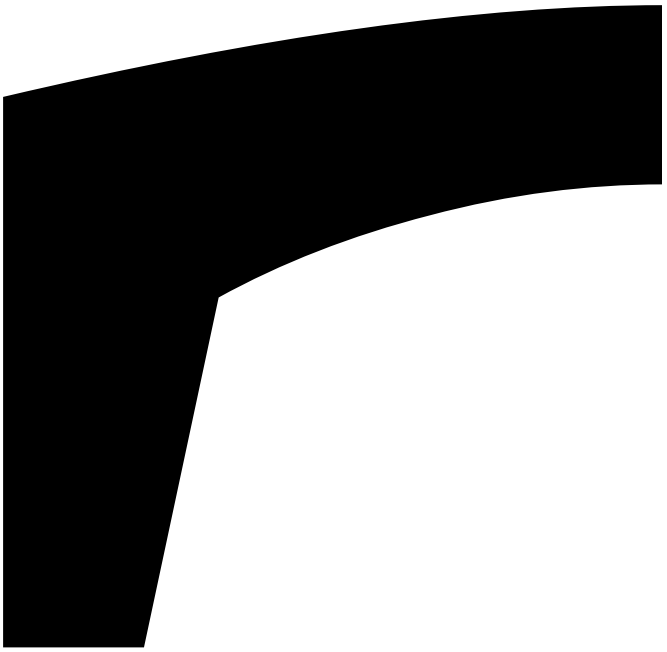
cdxcv

cdxcv

cdxcv

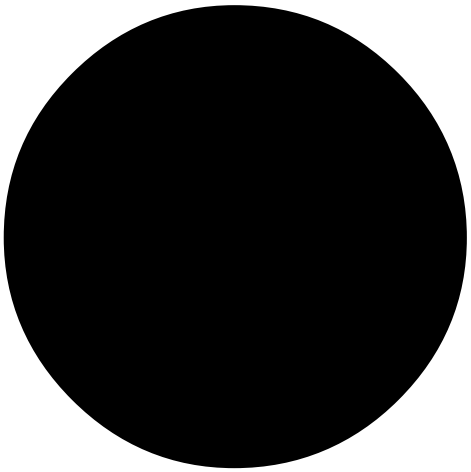


d



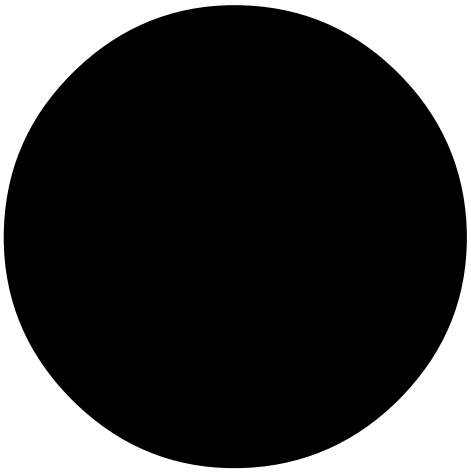
di



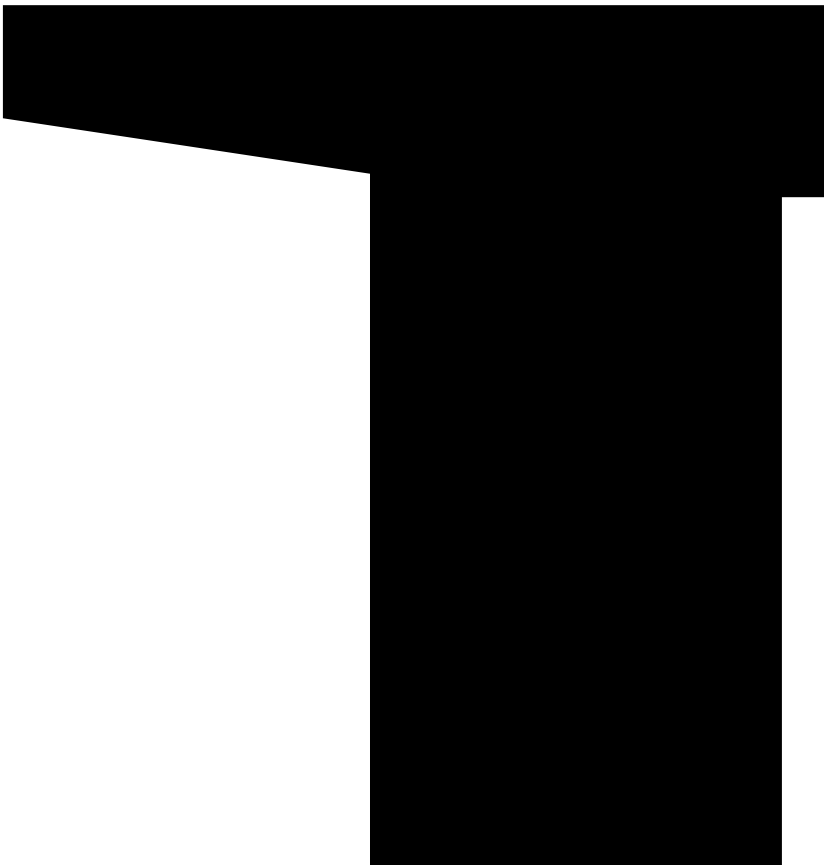


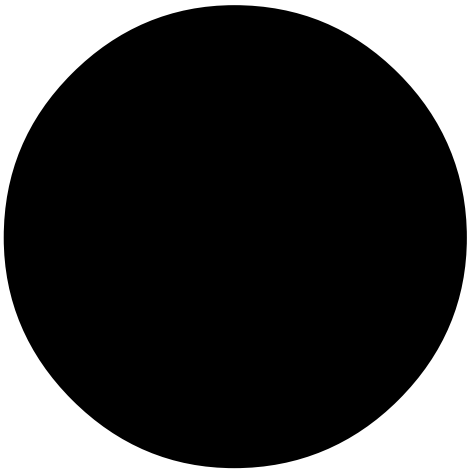
dv

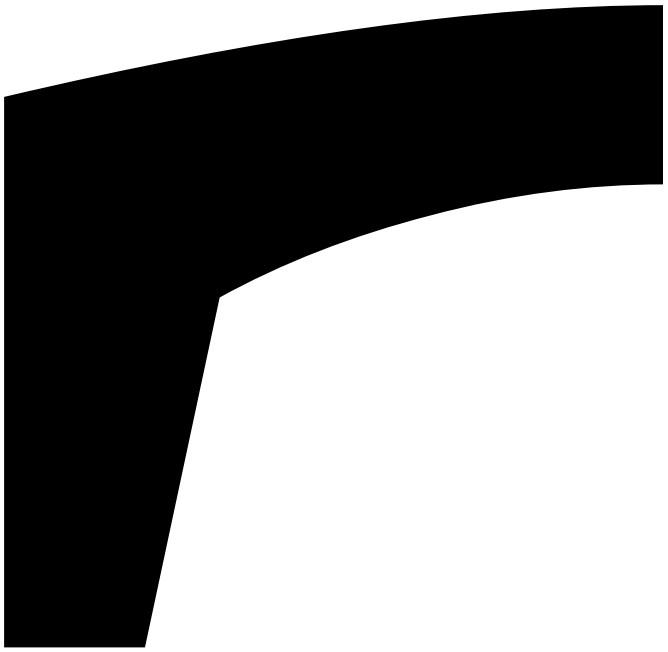
dvi



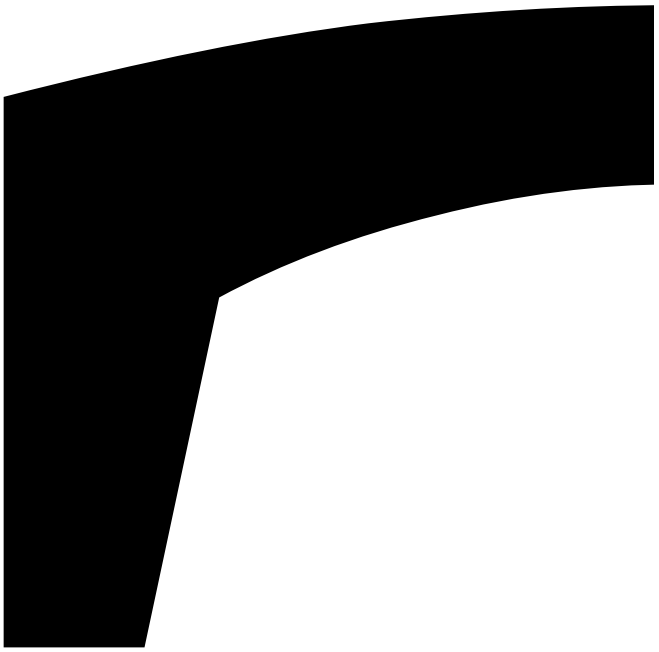


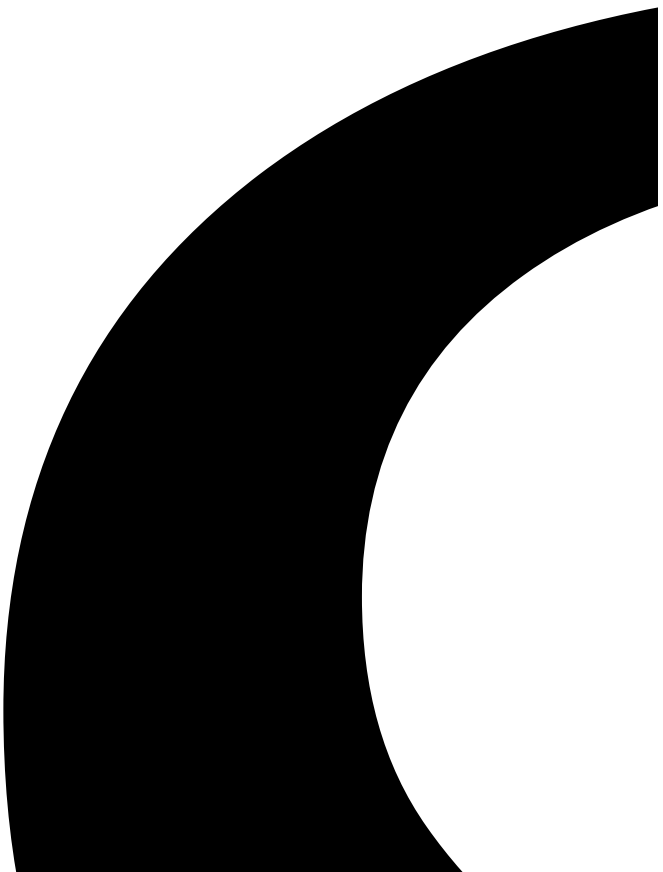




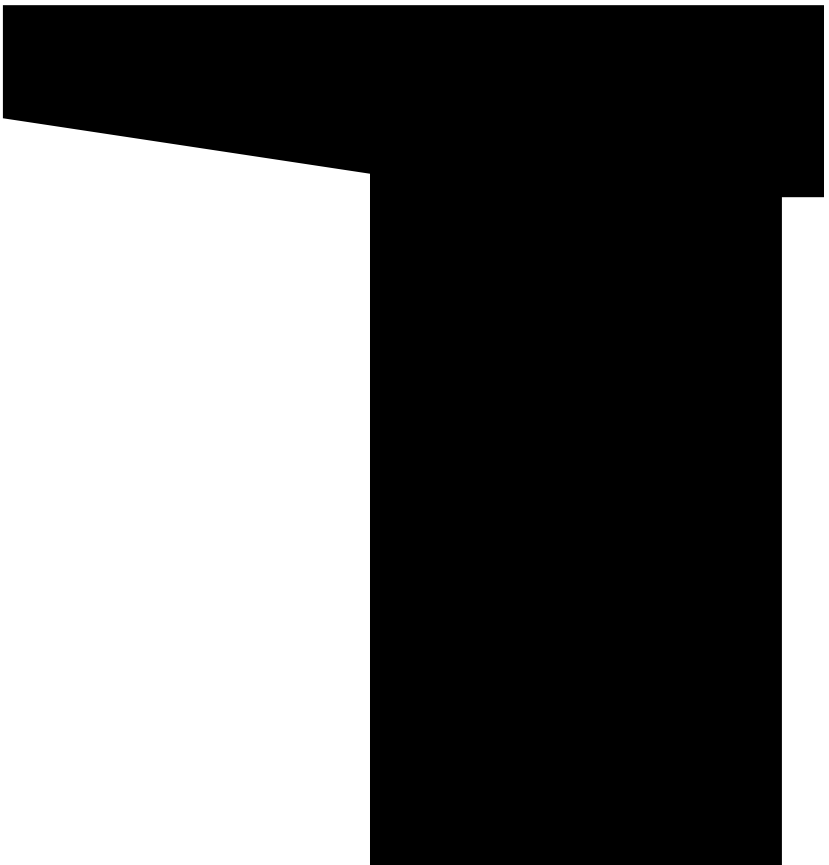


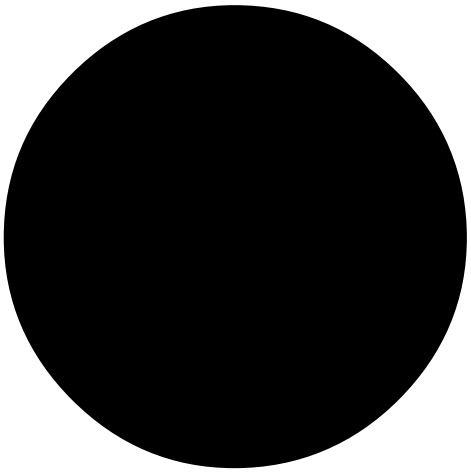
dxx





dxxx





dxxx

dxxx

dxxx

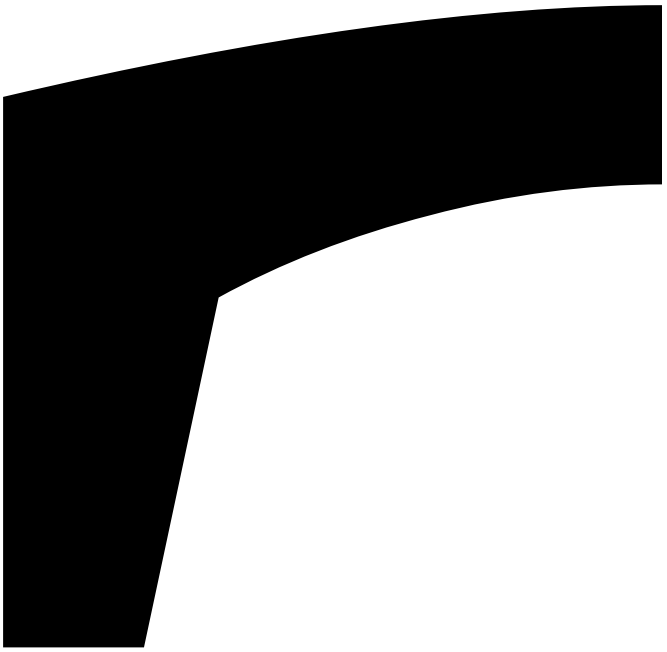
dxxx

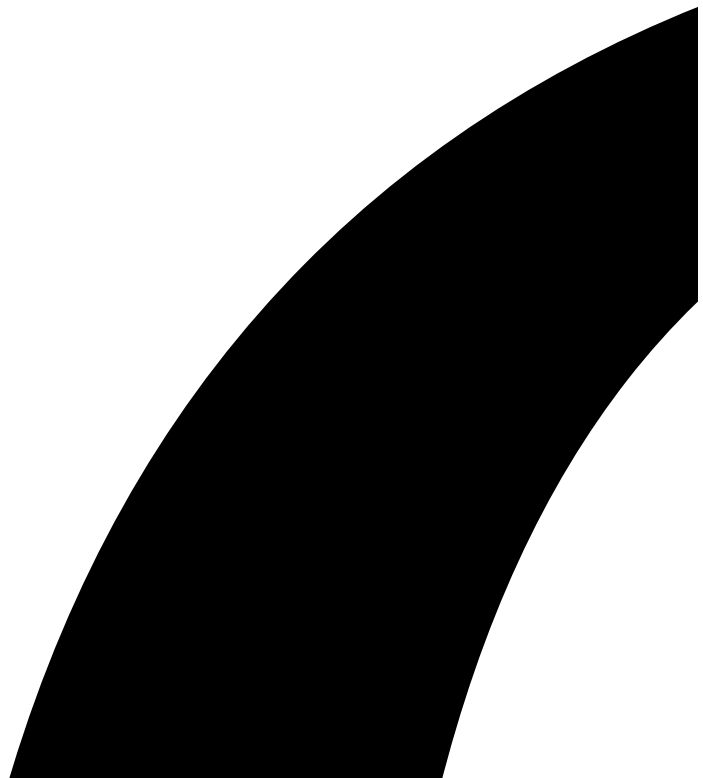
dxxx

dxxx

dxxx

dxl



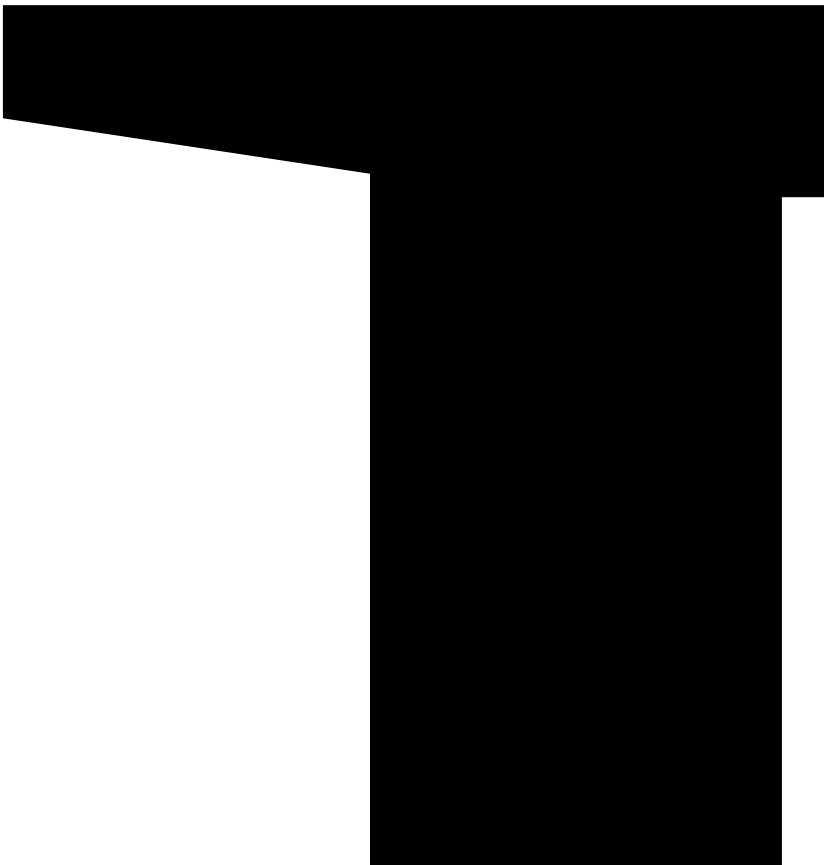


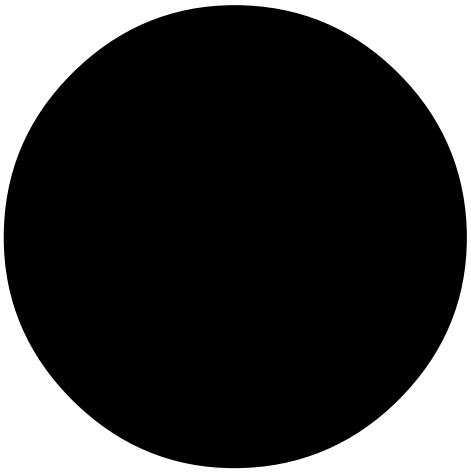


dl



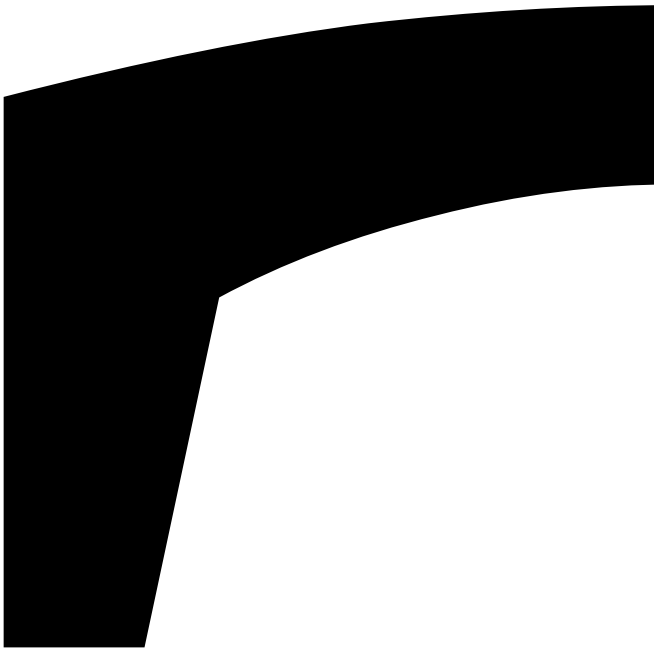


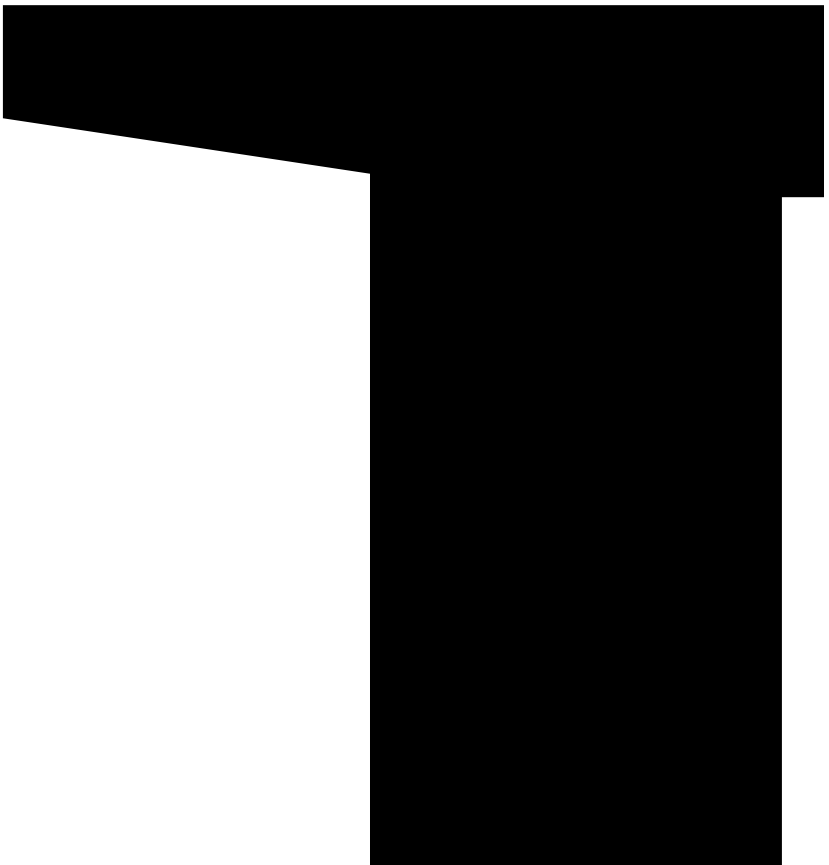




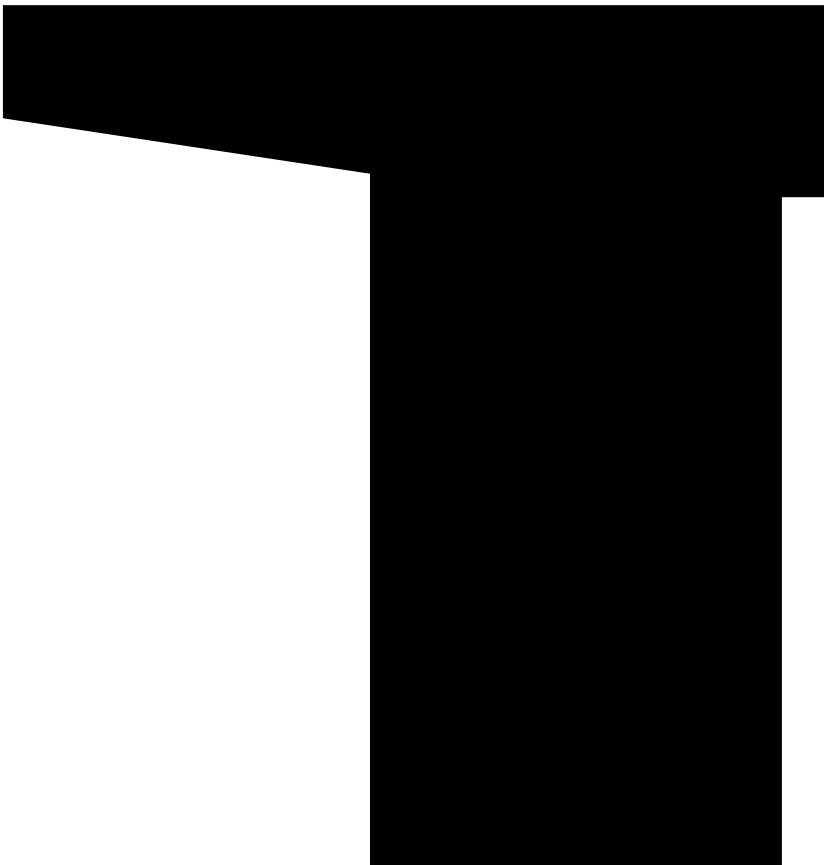
dlv

dlx





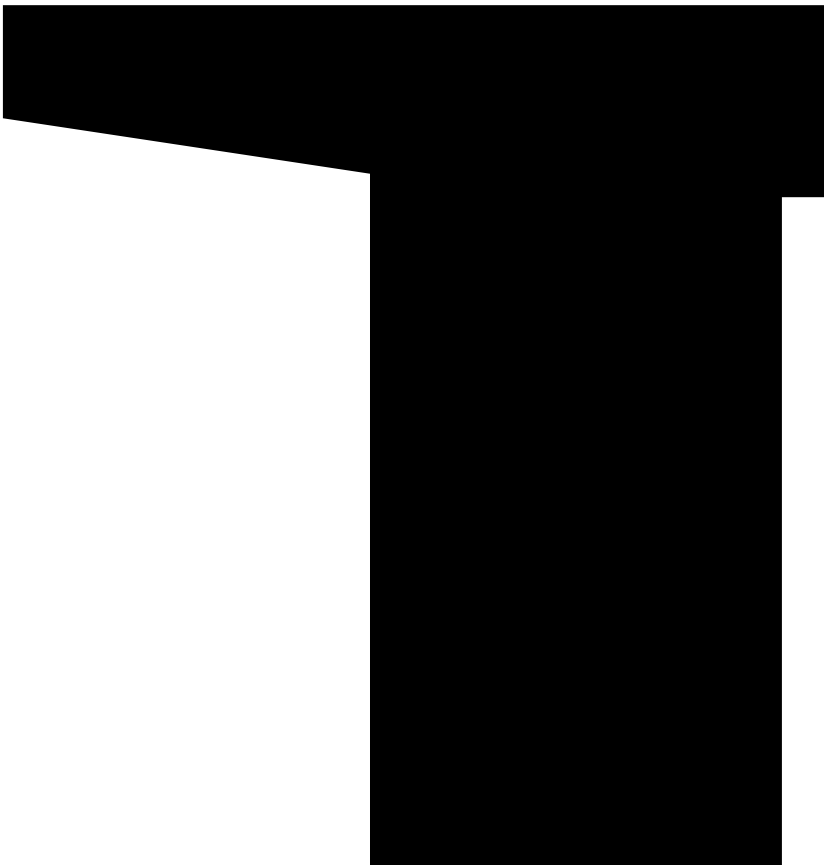


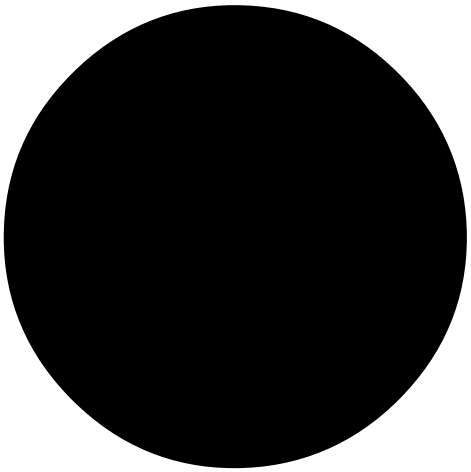




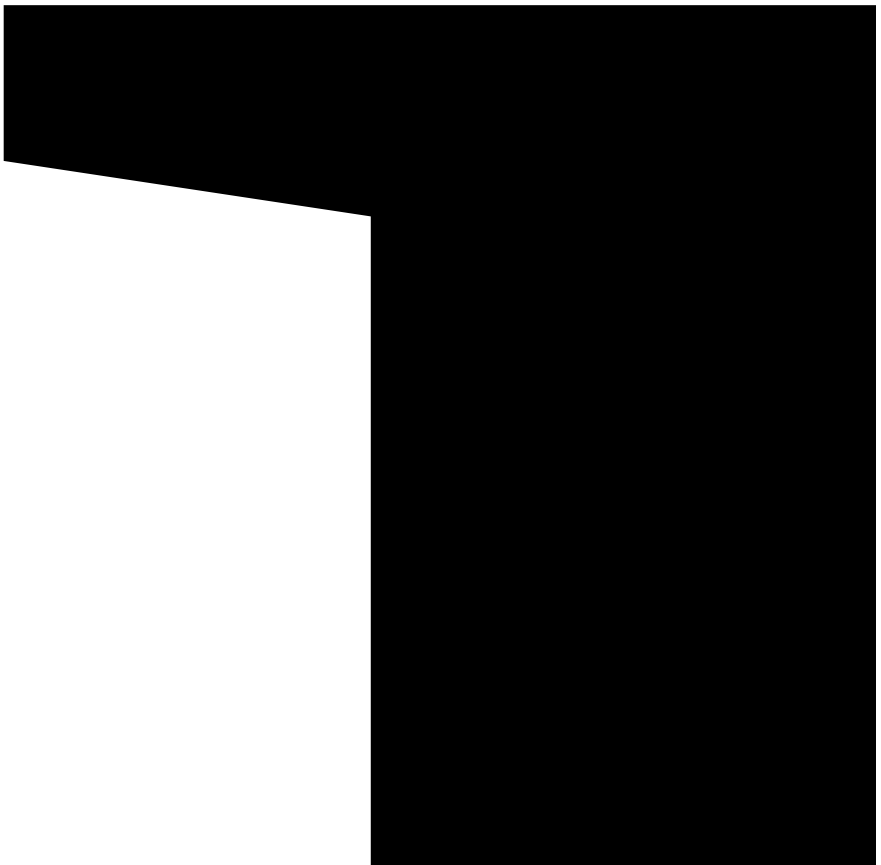
dlxx

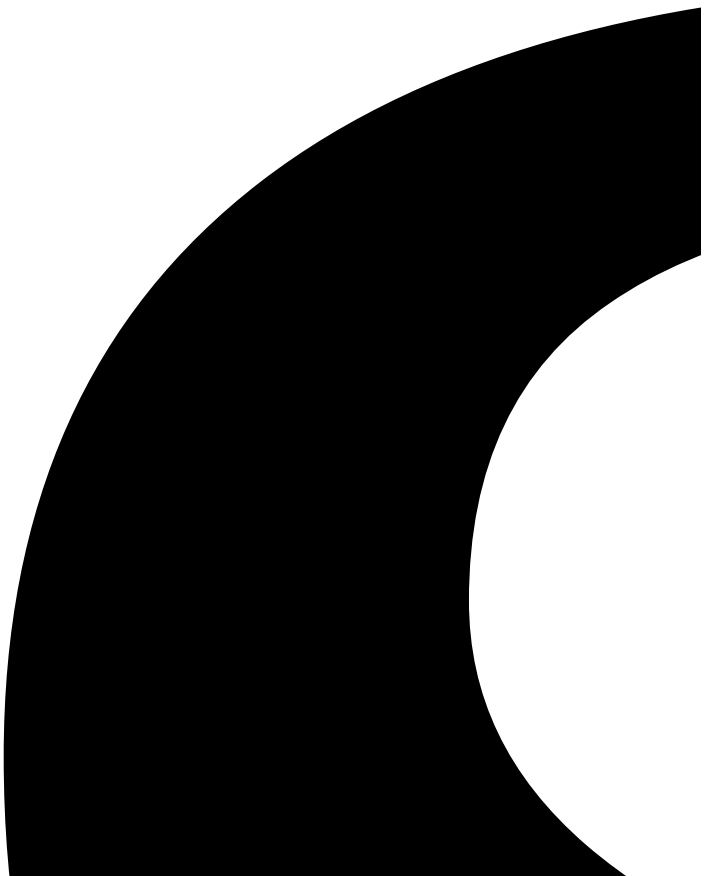


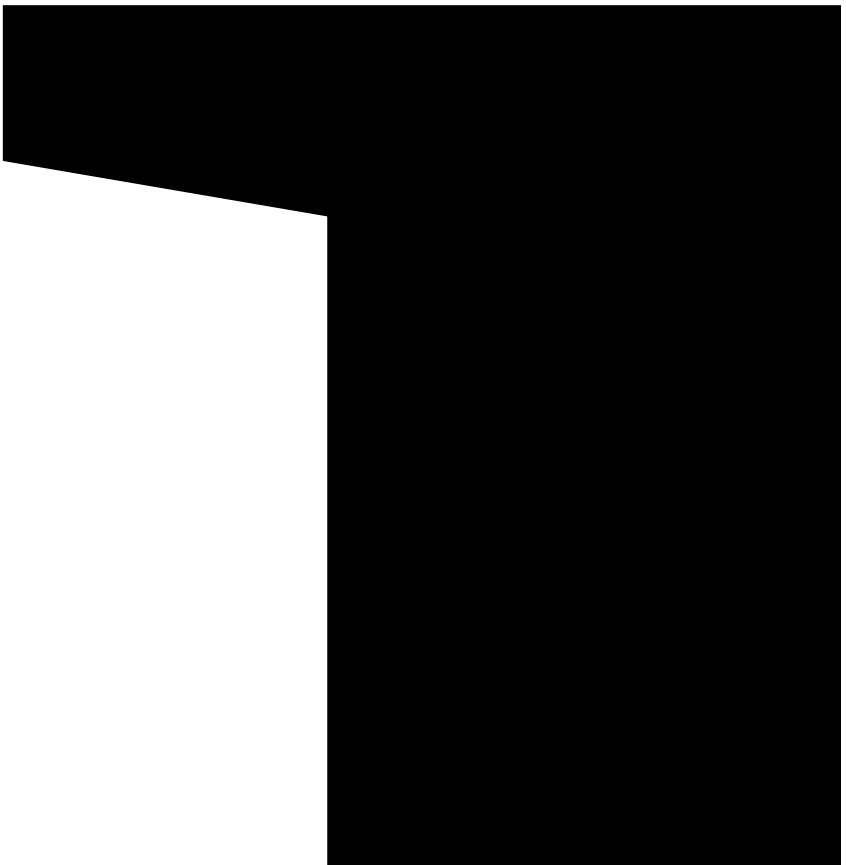


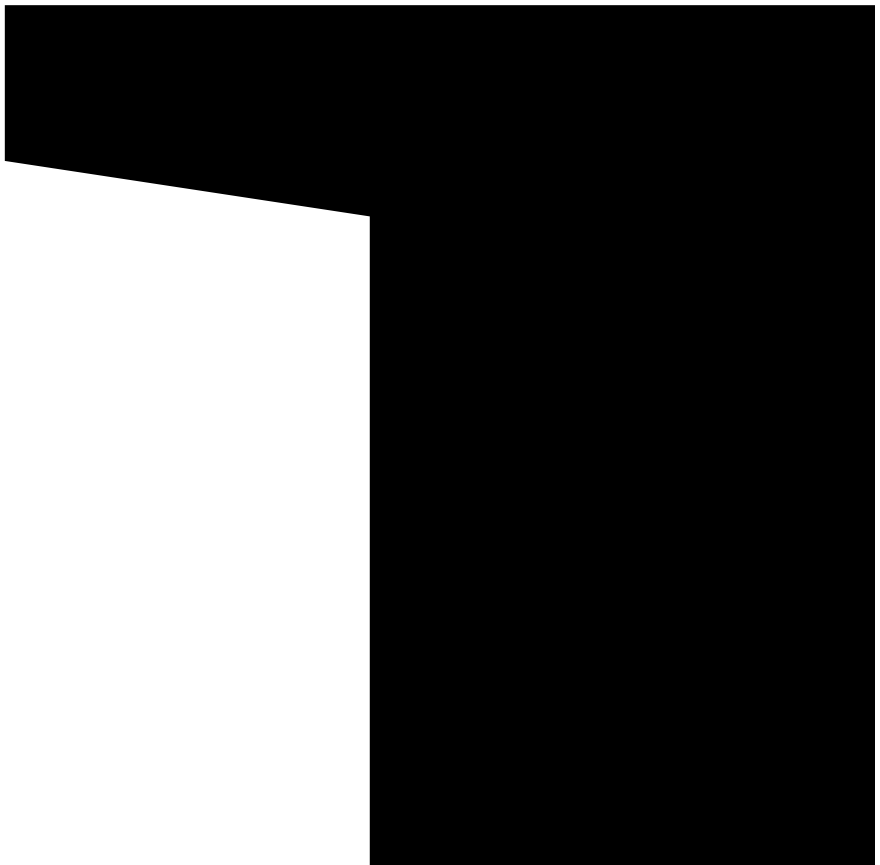








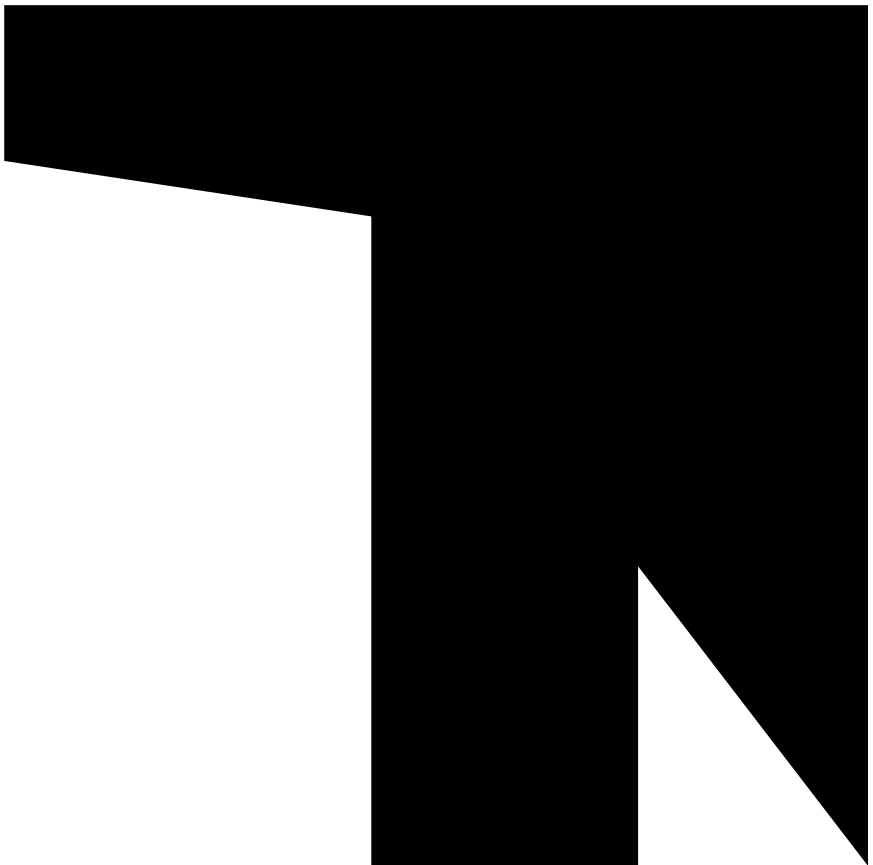




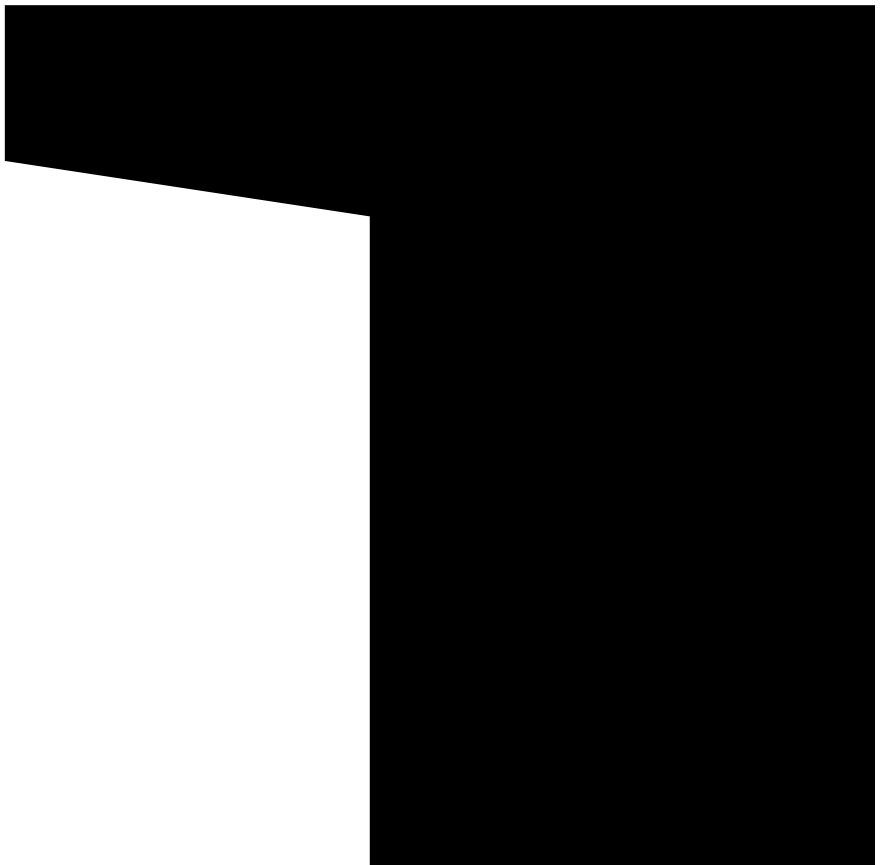


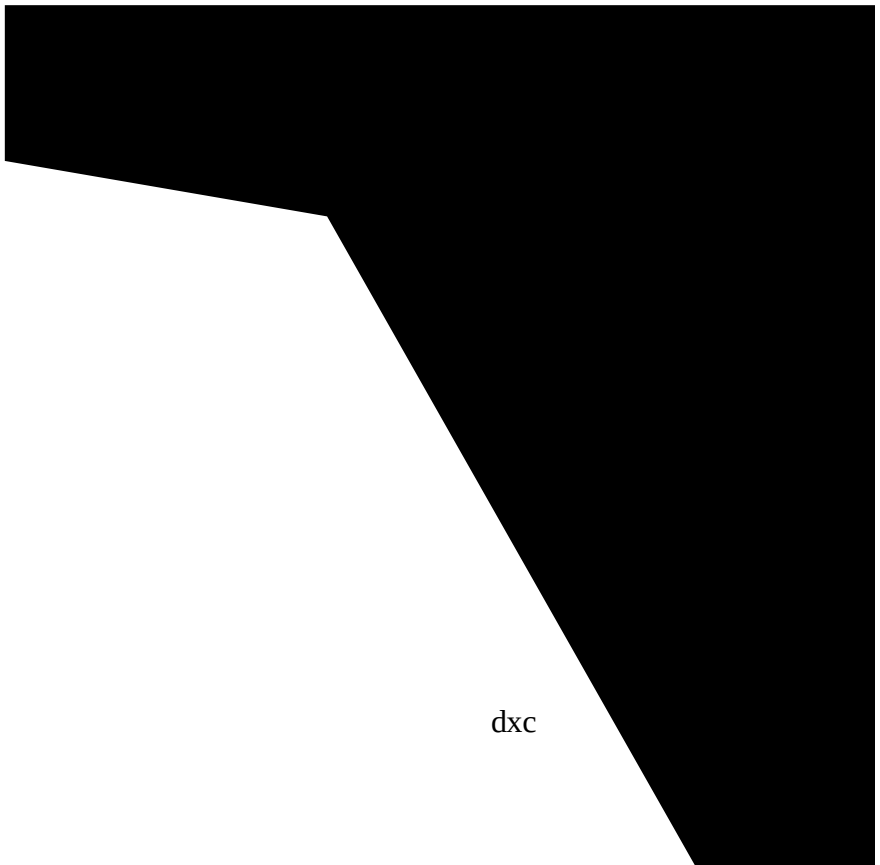
dlxxx

dlxxx

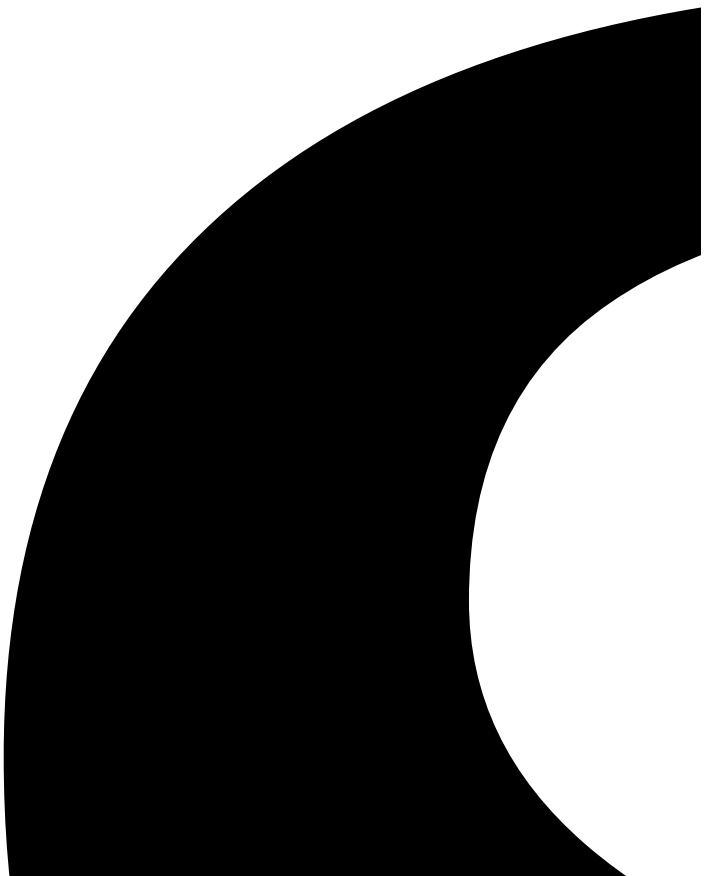


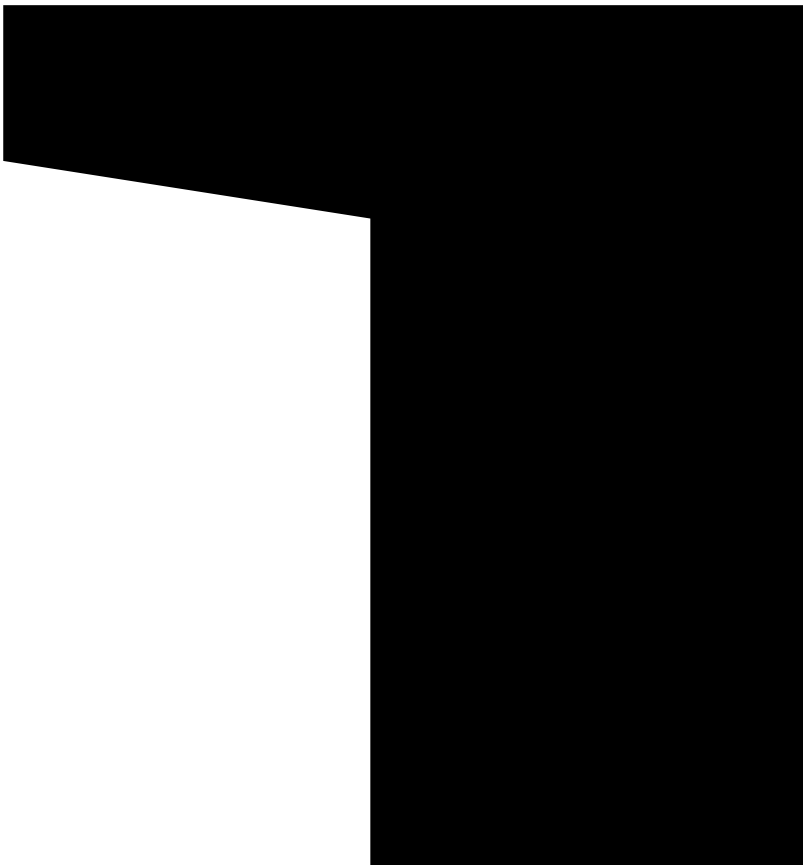
dlxxx

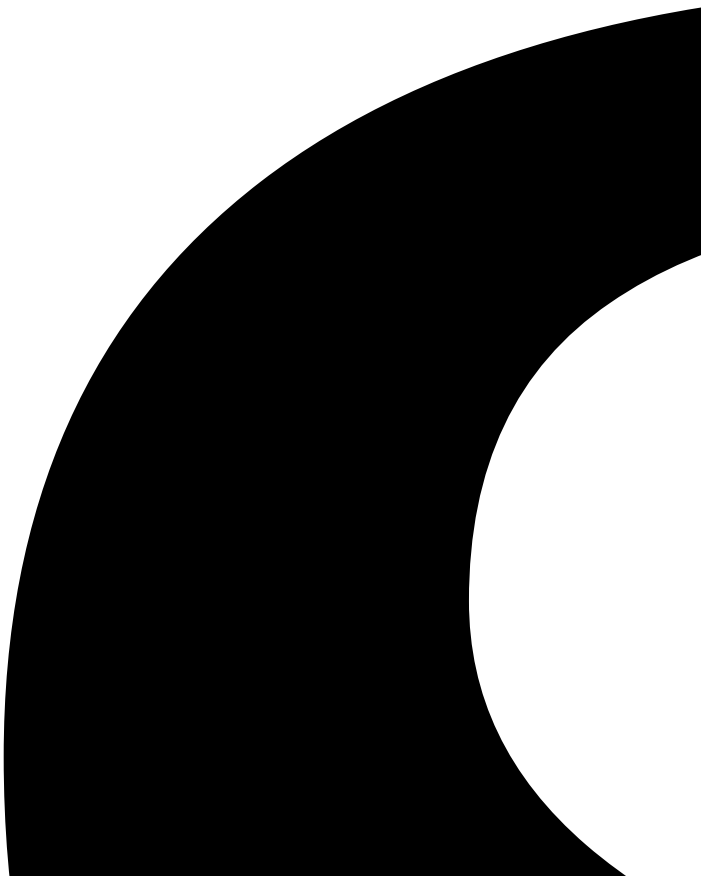


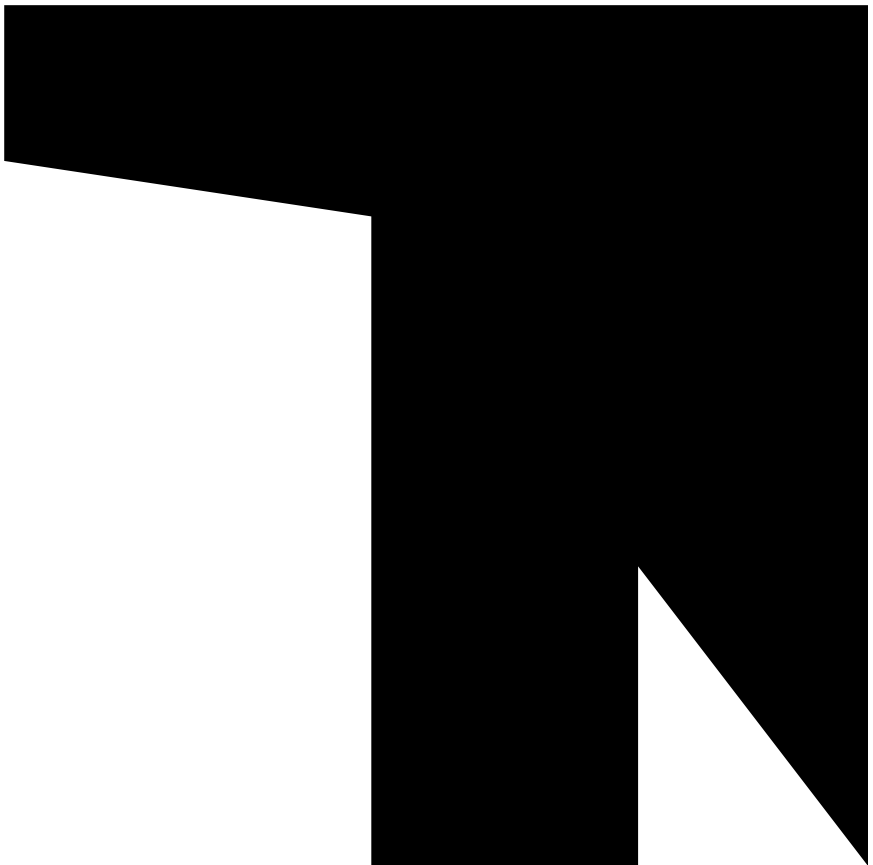


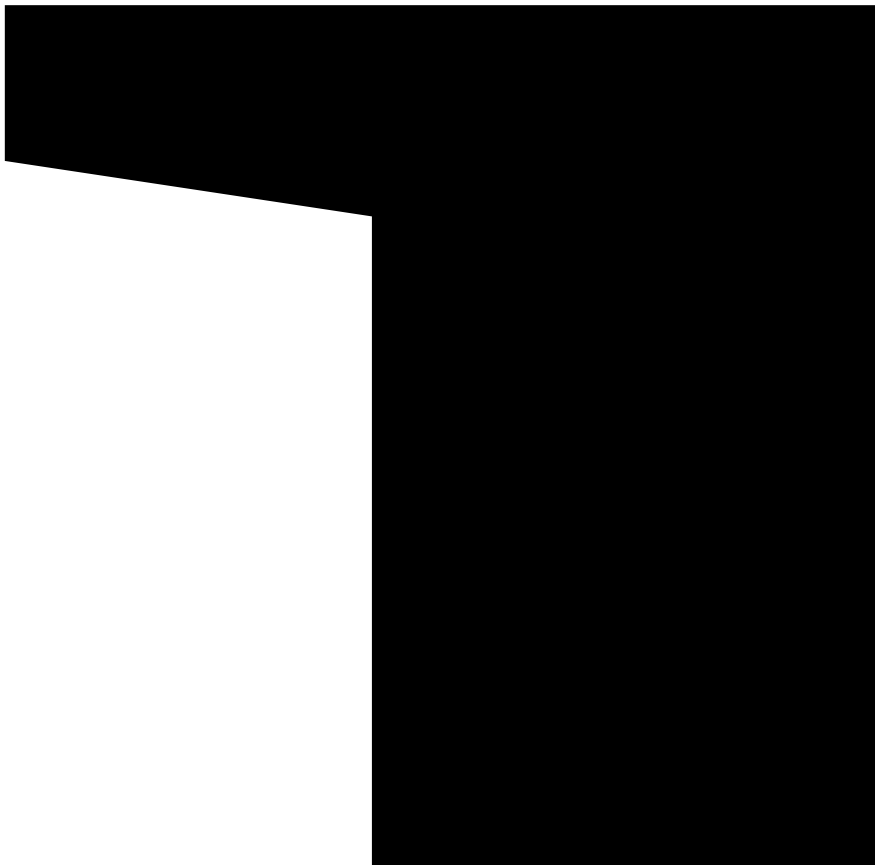
dxc

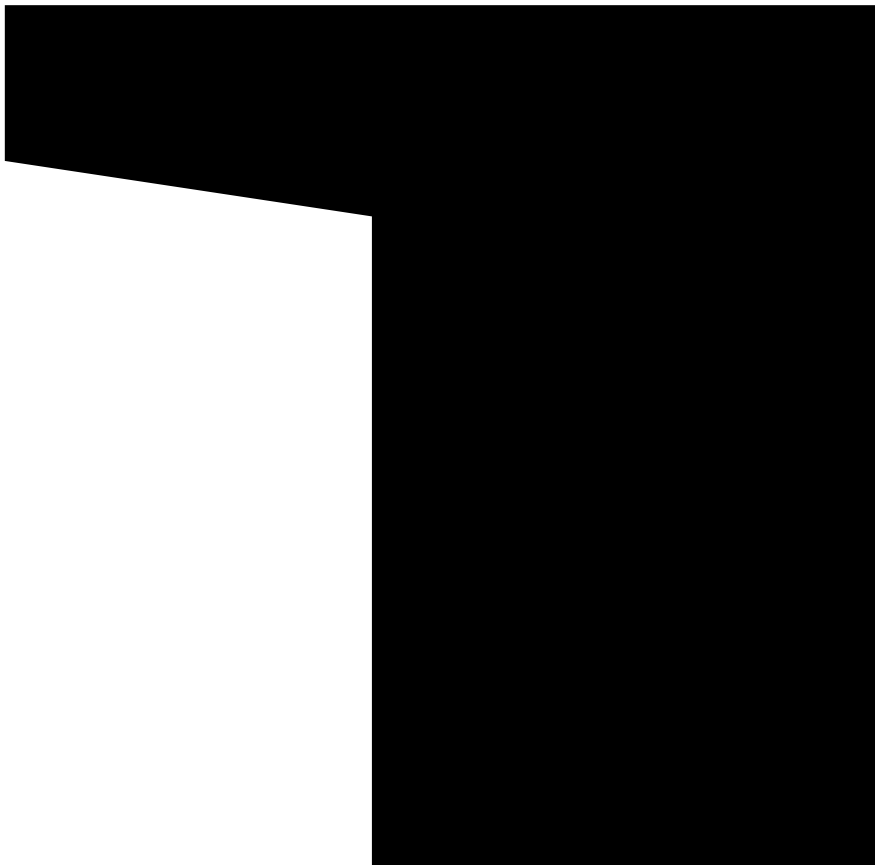


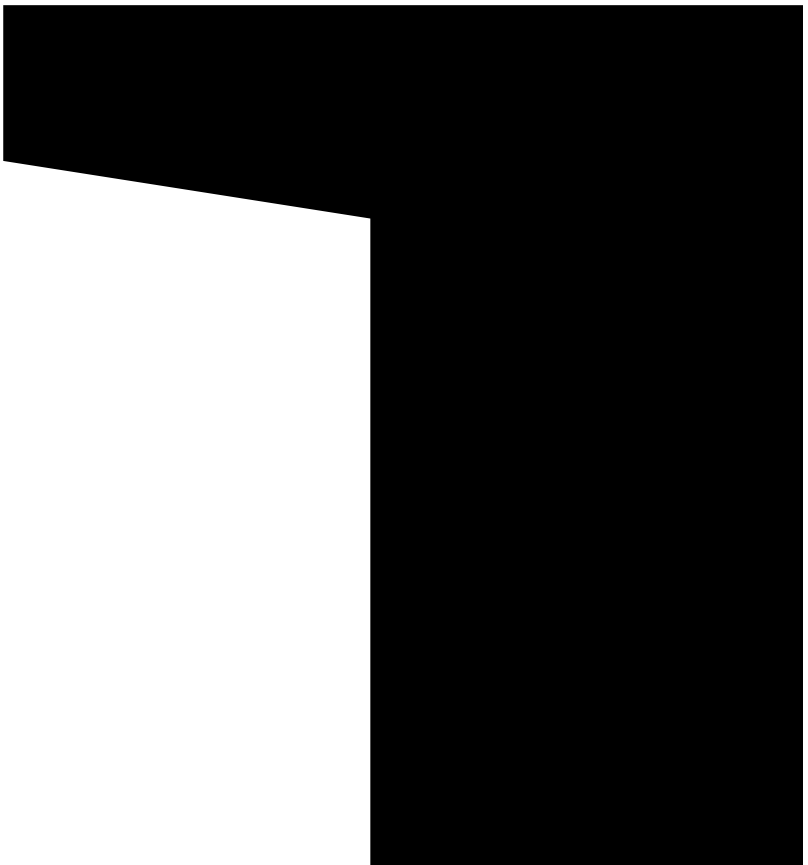


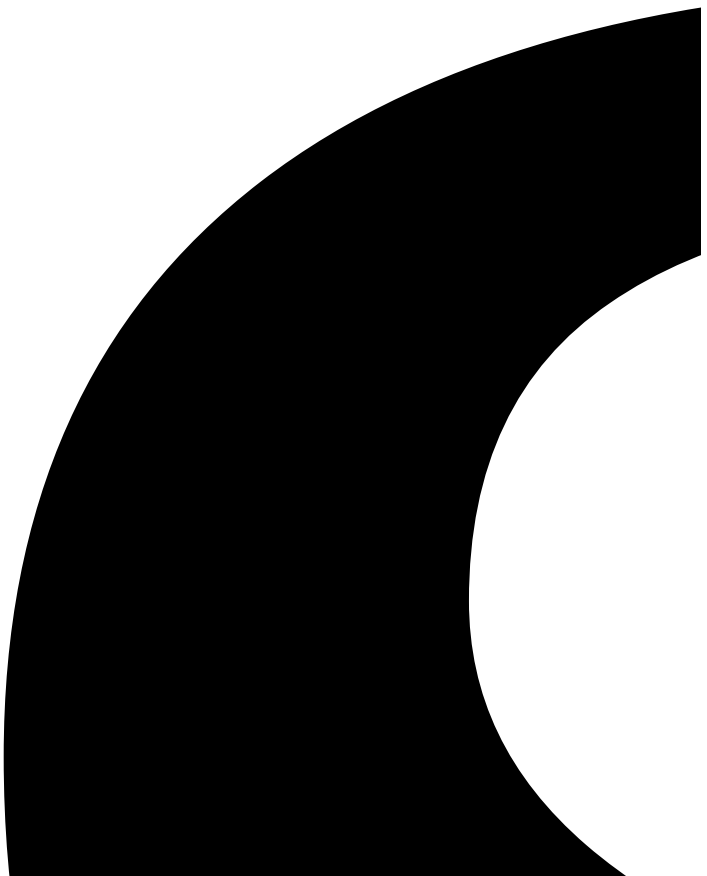


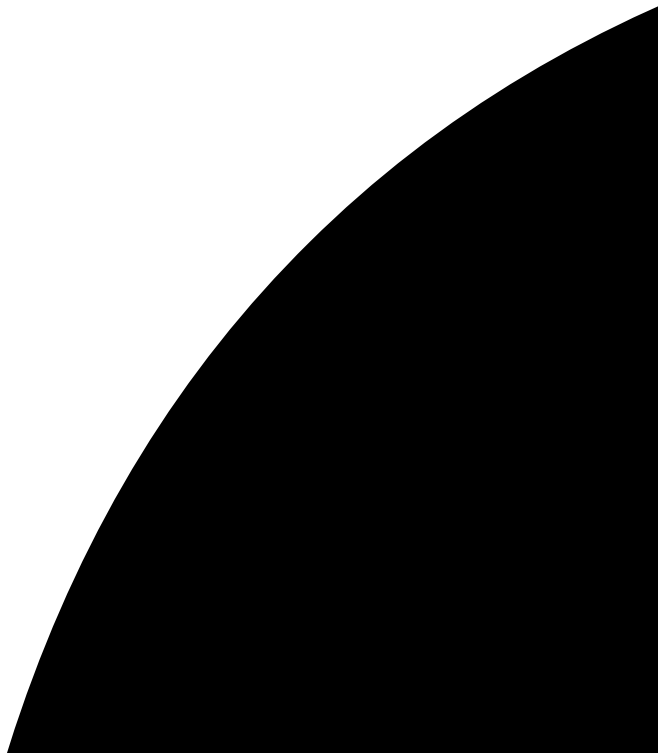


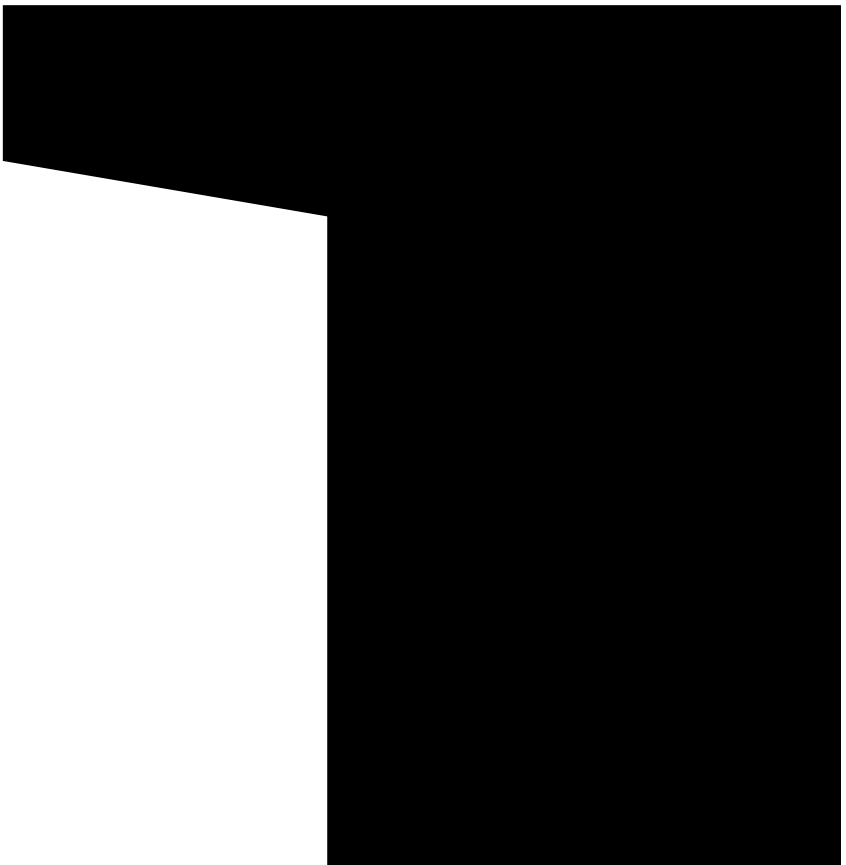


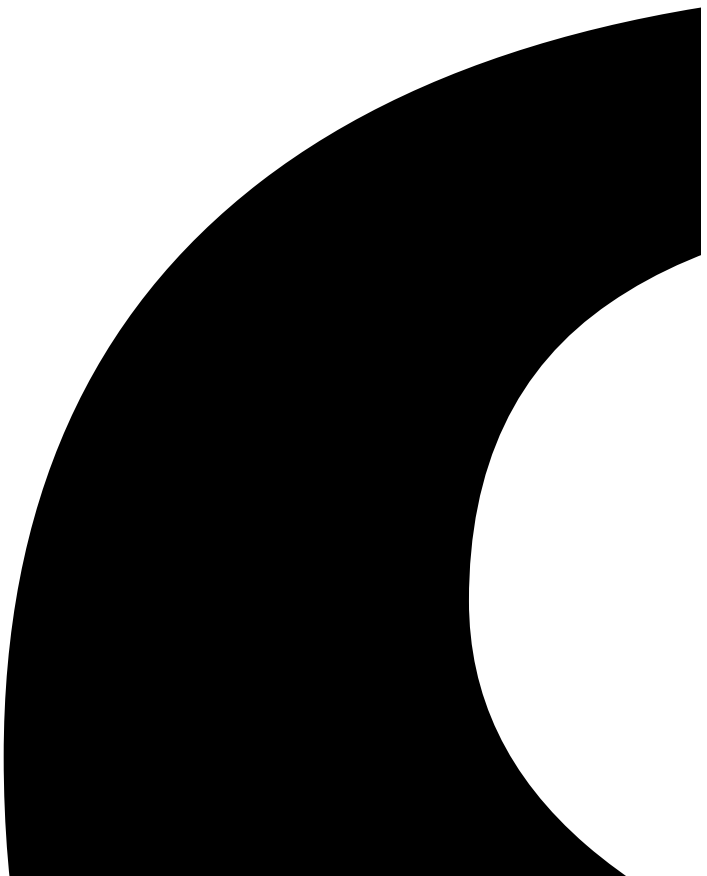


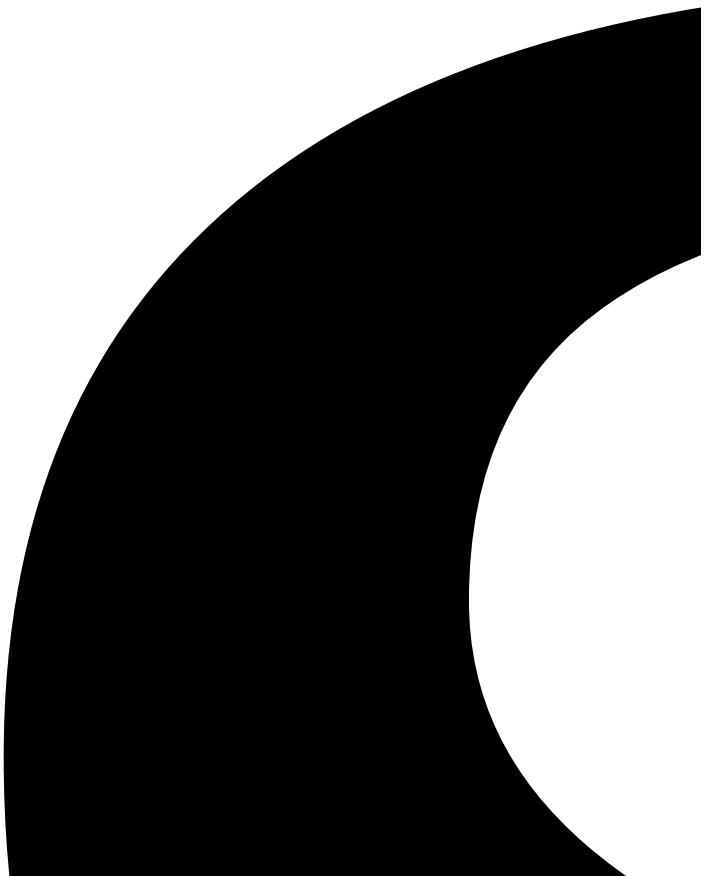


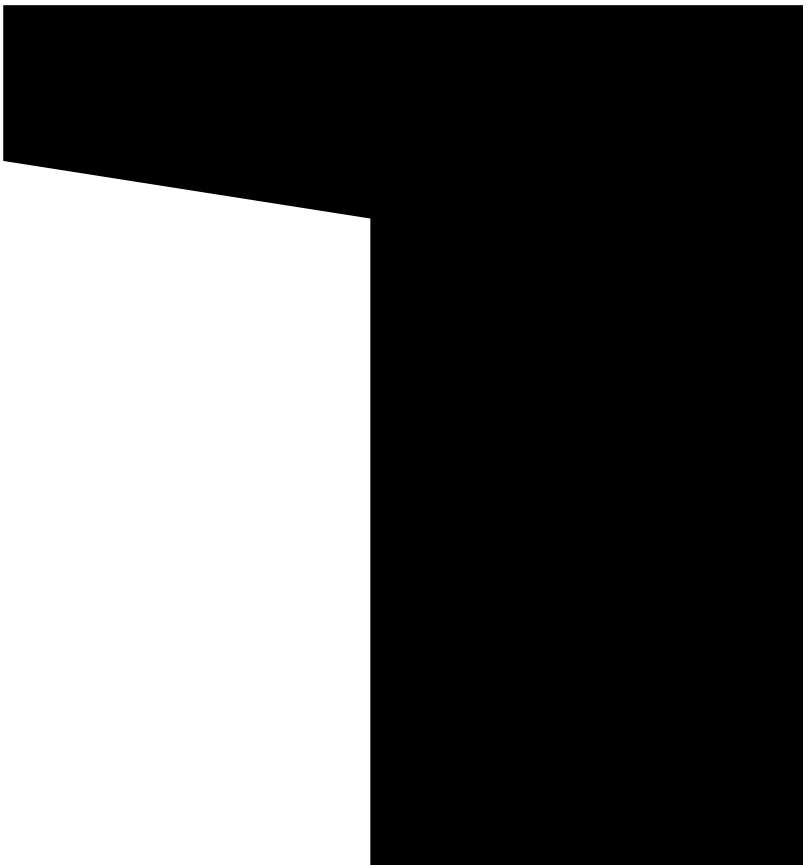


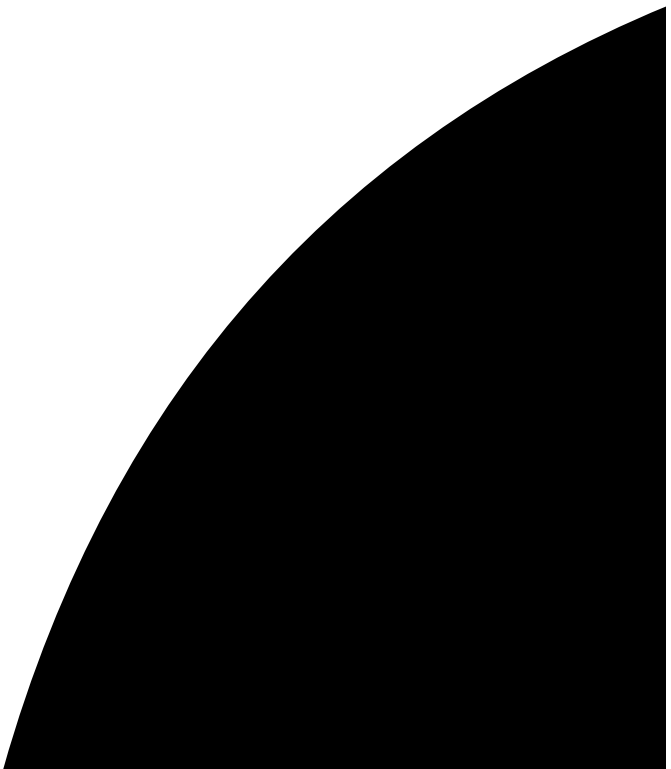


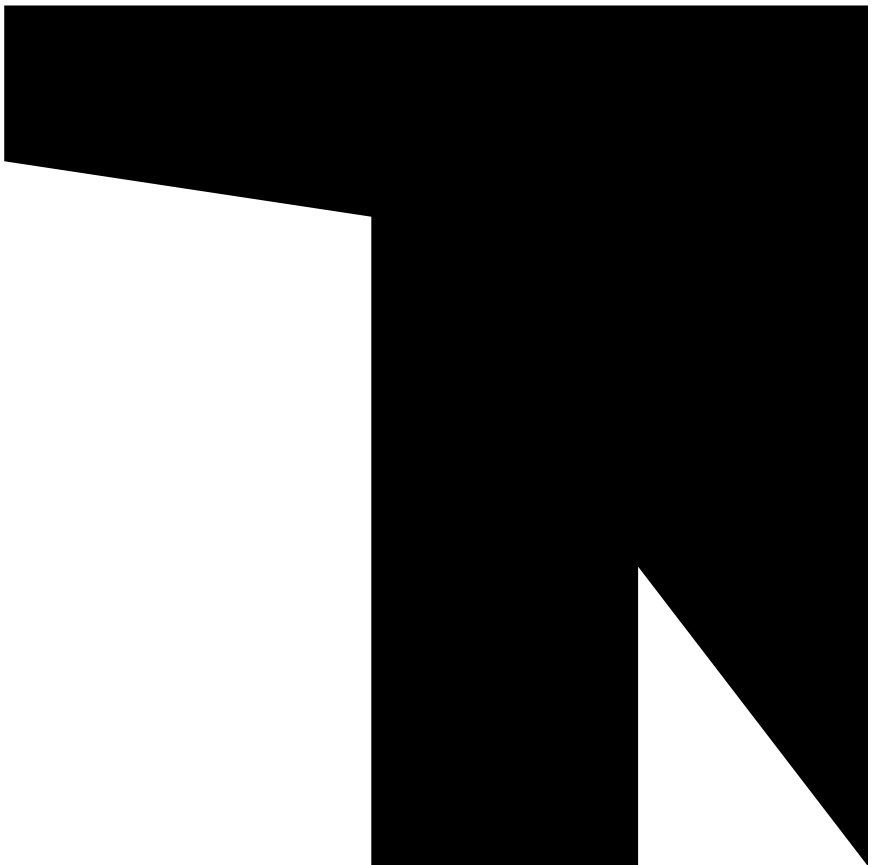


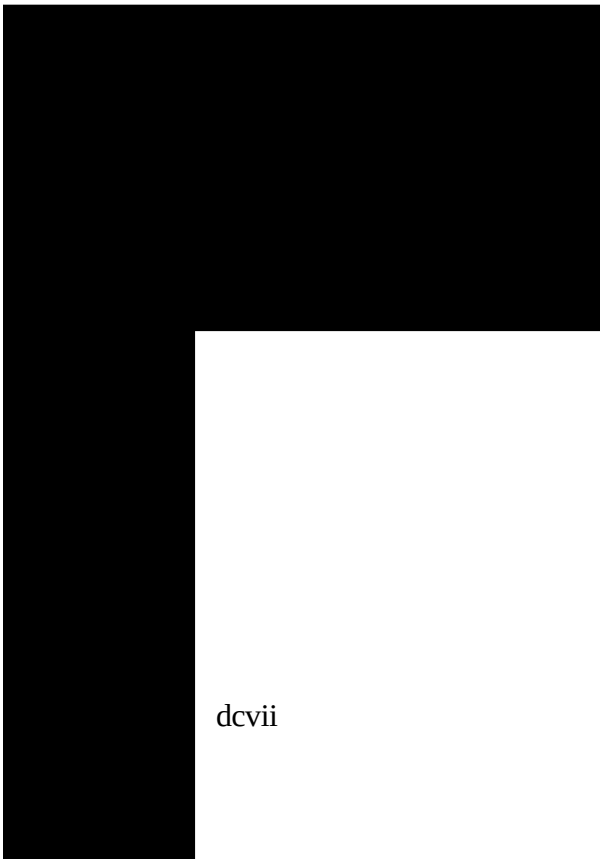


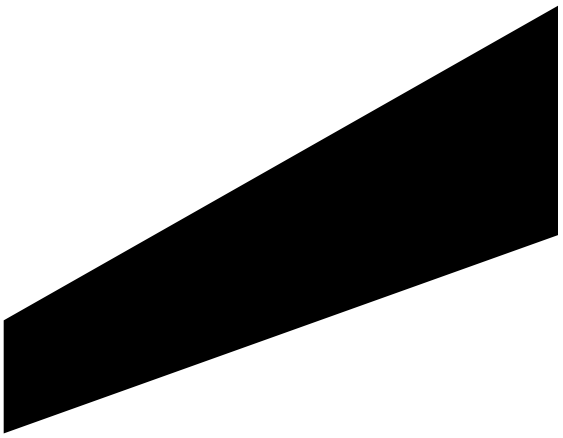


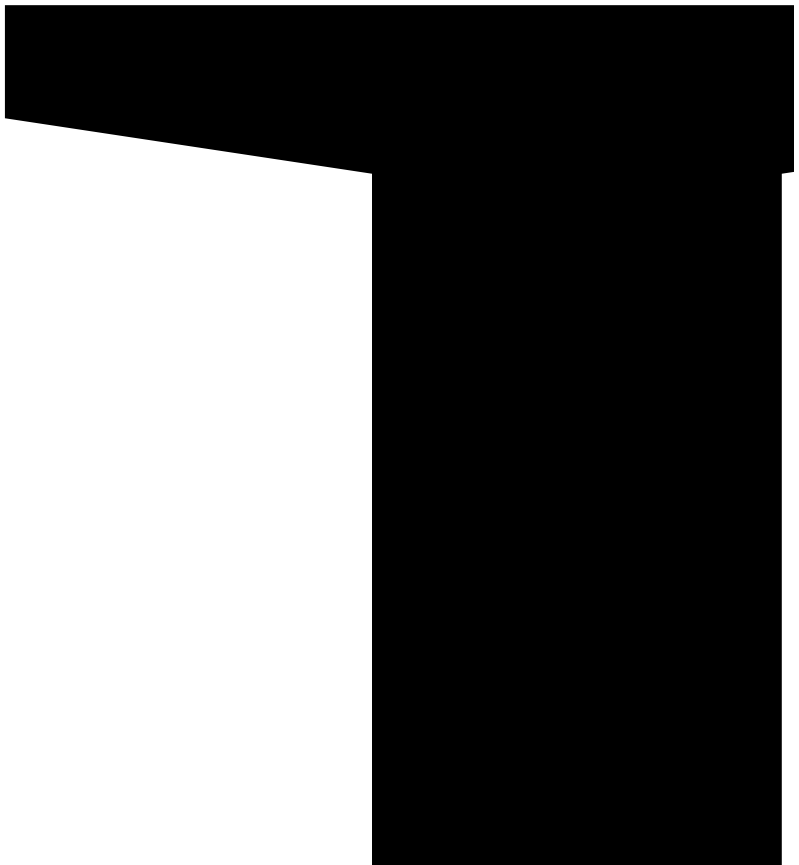








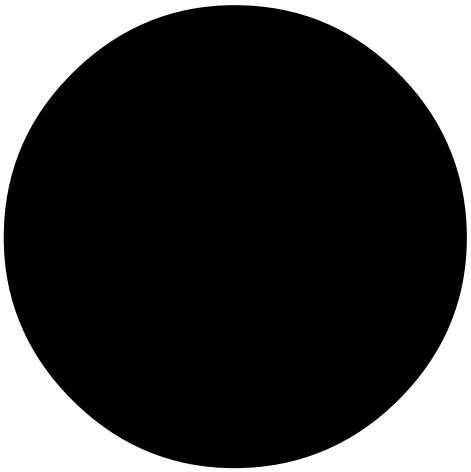


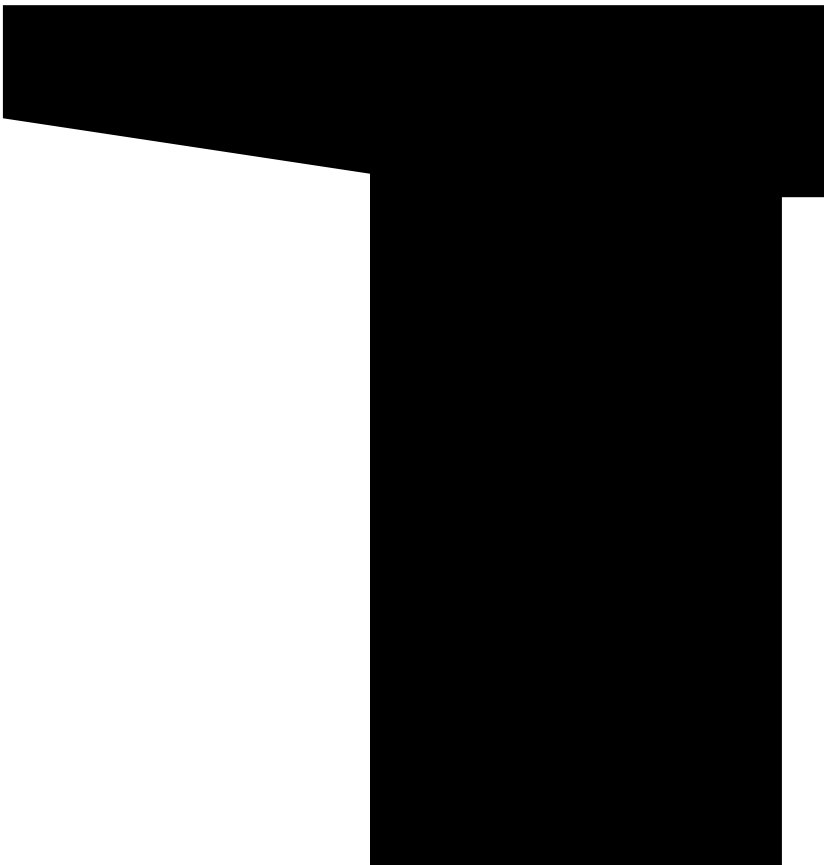




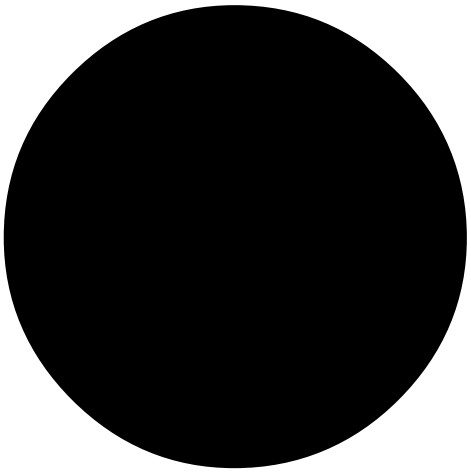




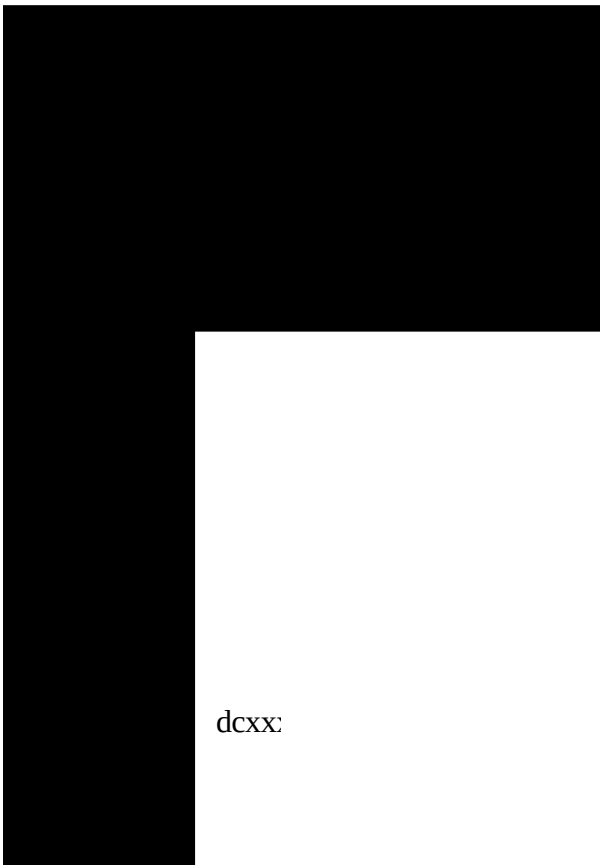




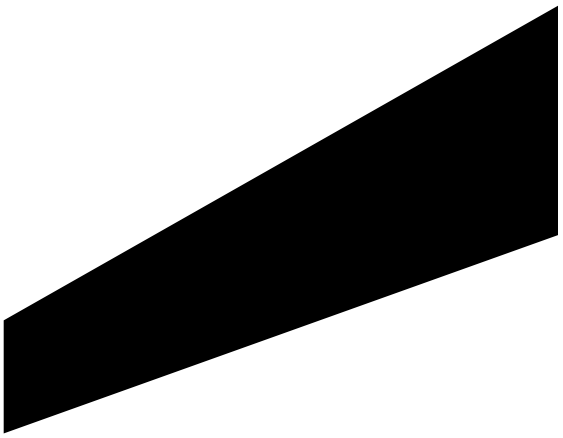


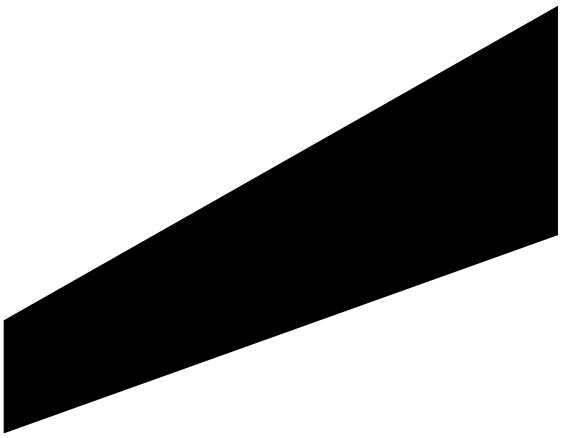


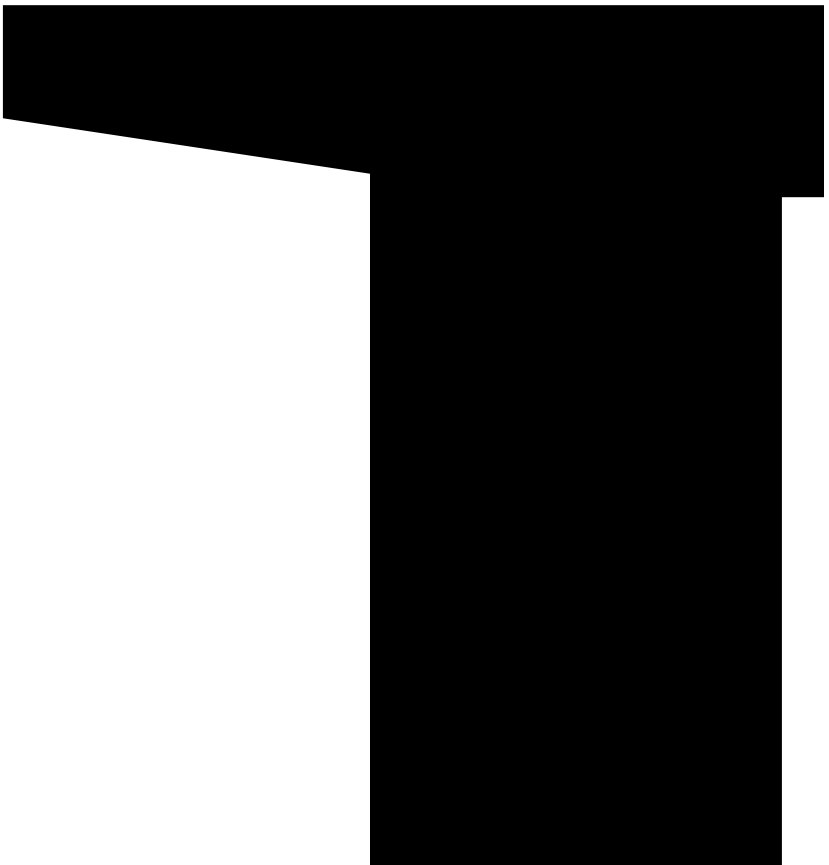




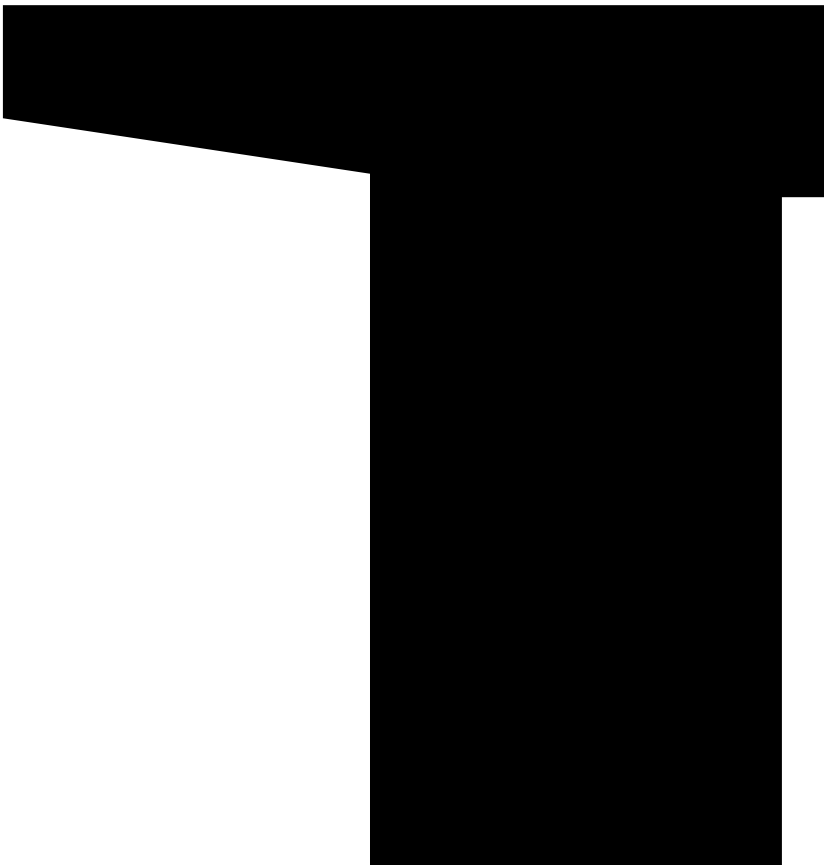
dcxx



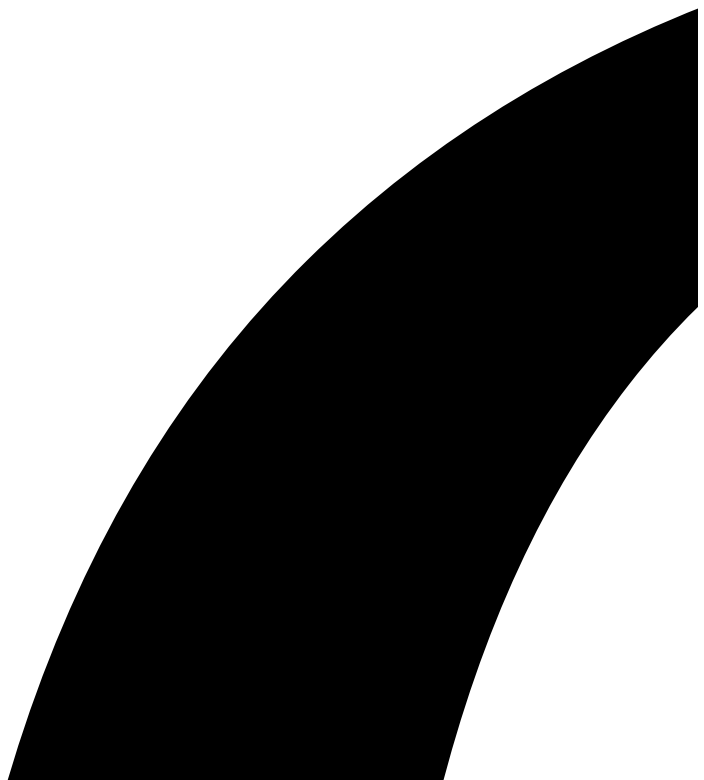






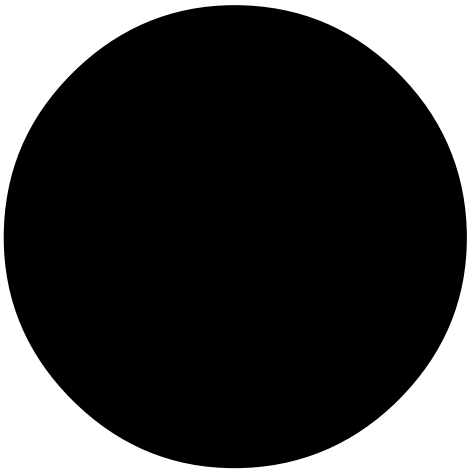




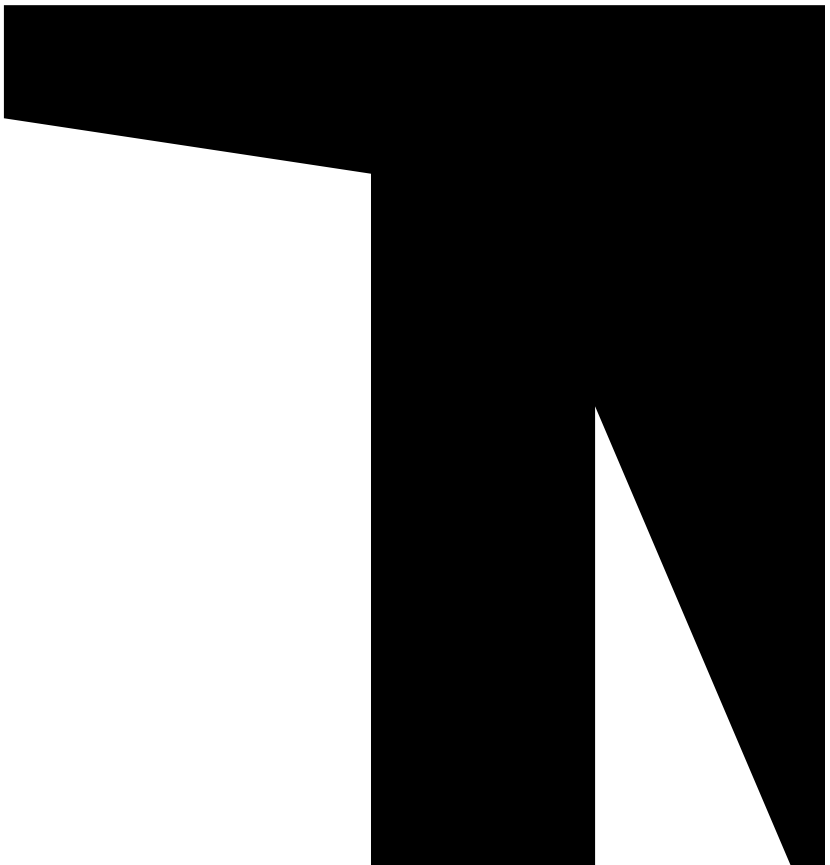


dcl

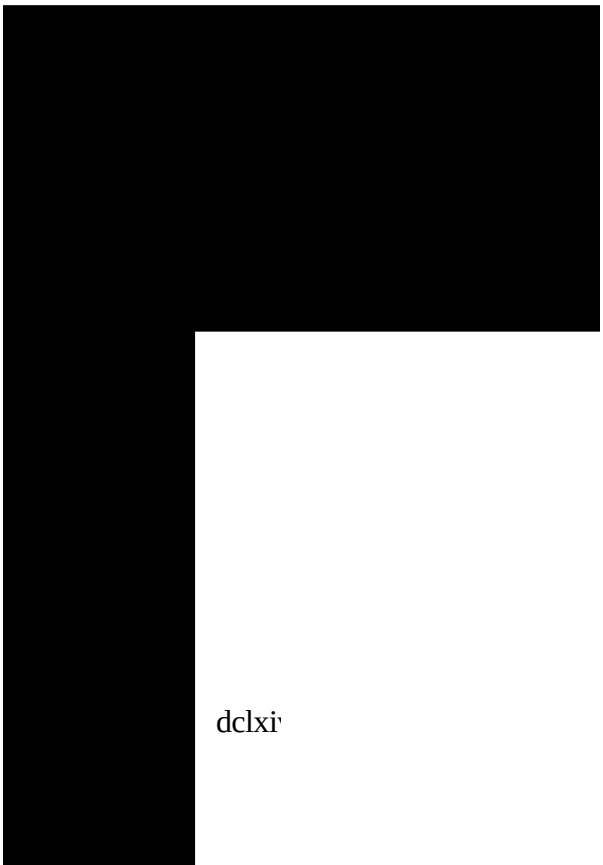




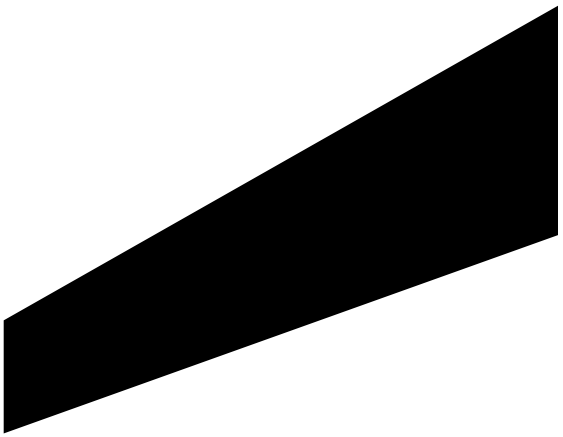
dclv

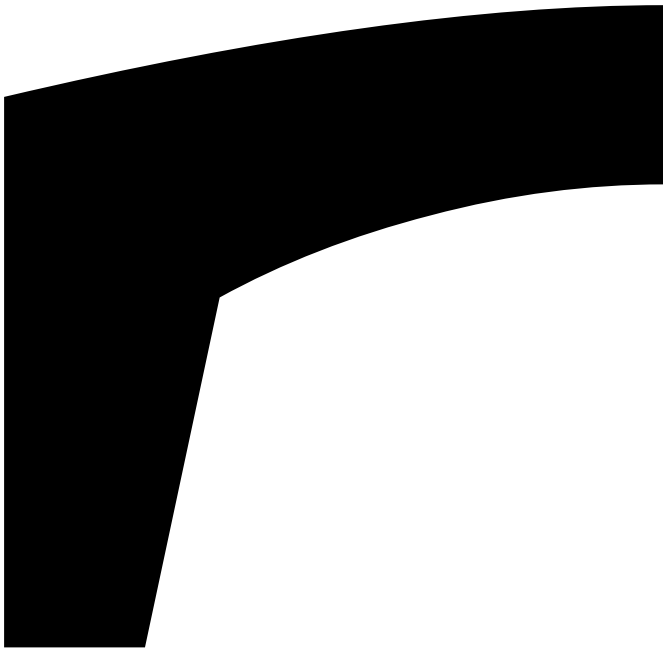


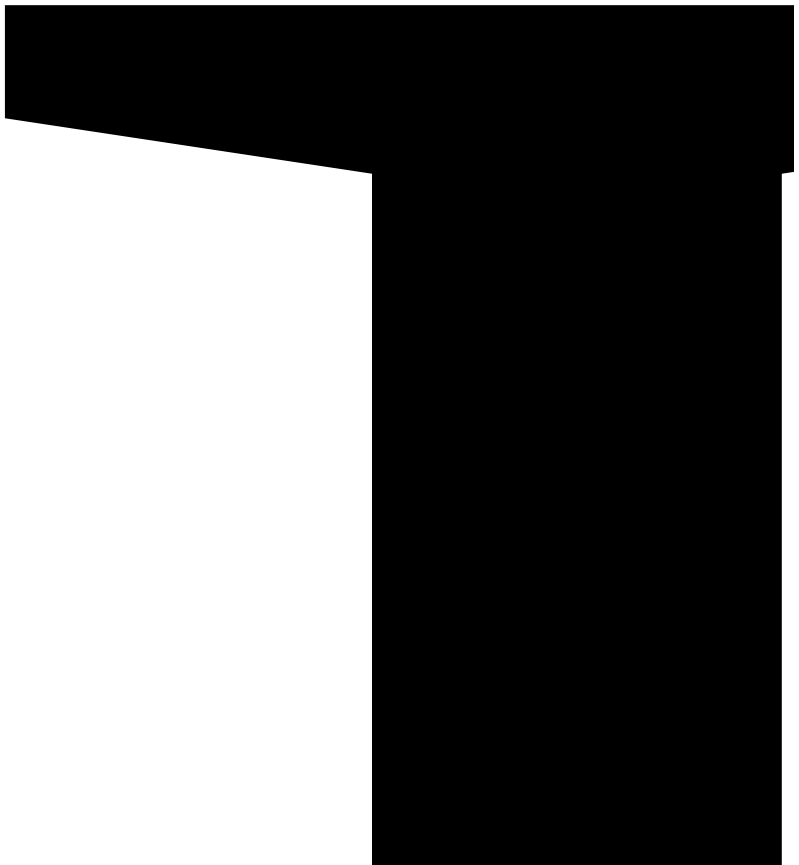




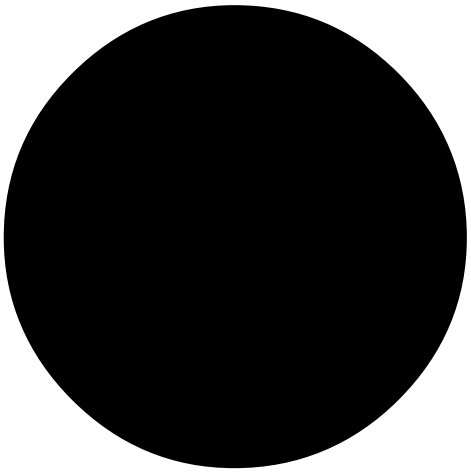
dclxi



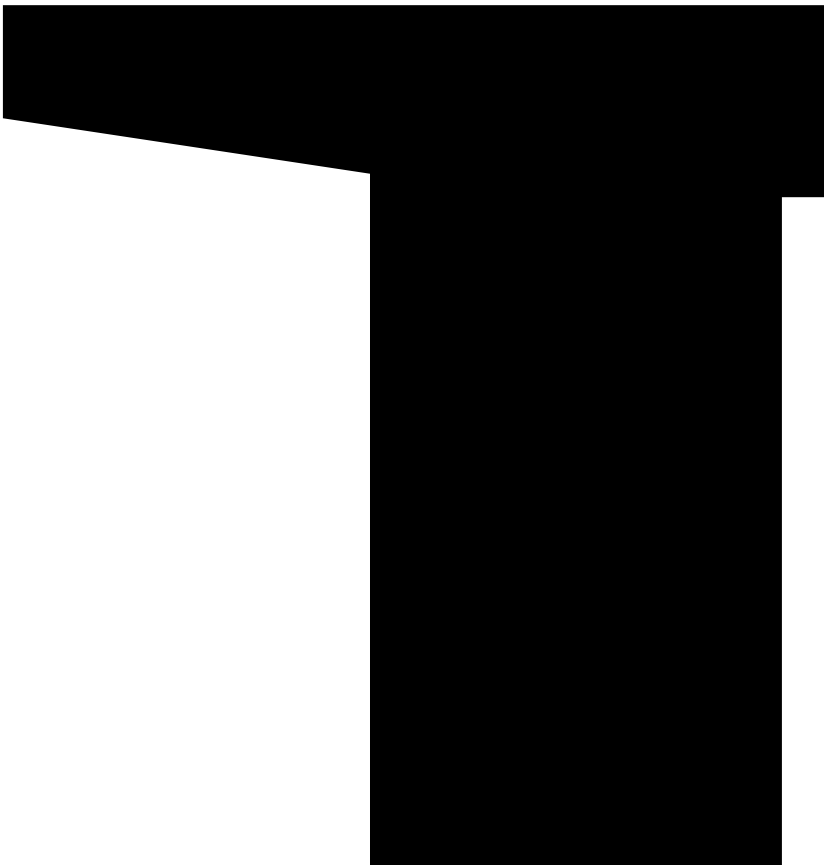


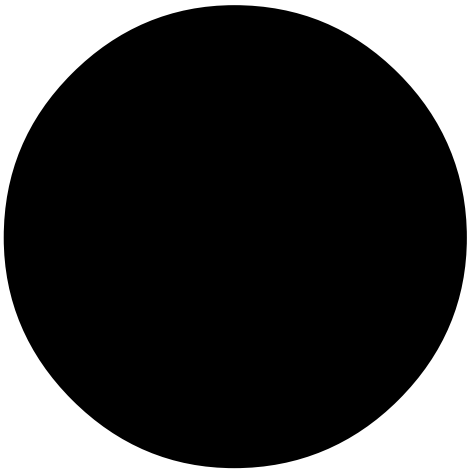




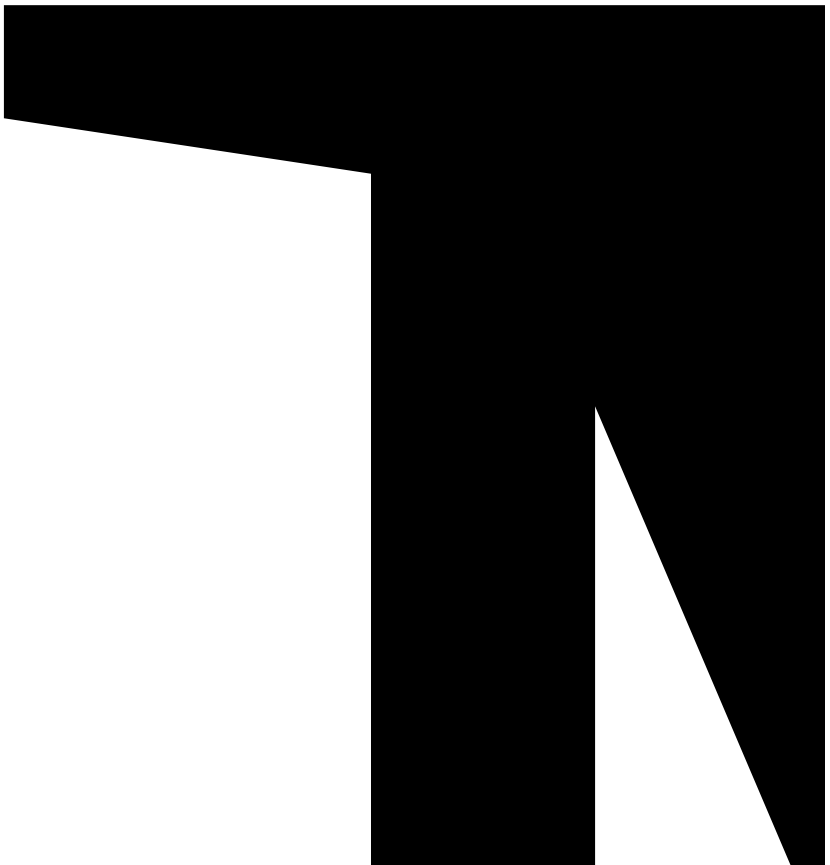


dclxx

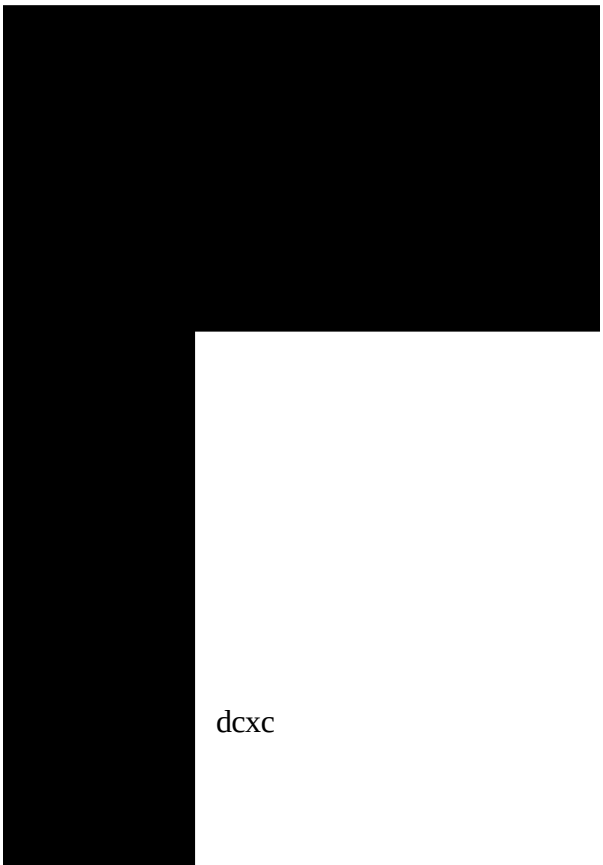




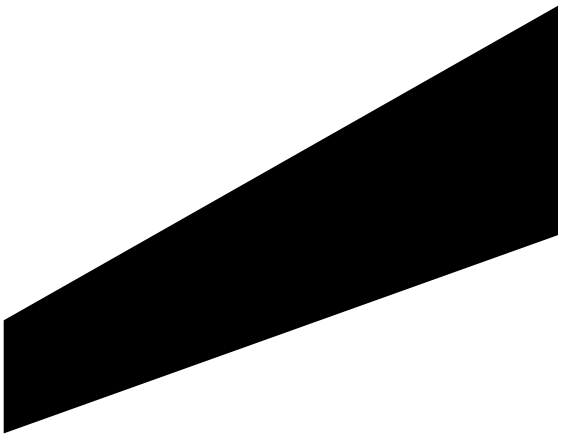
dclxx

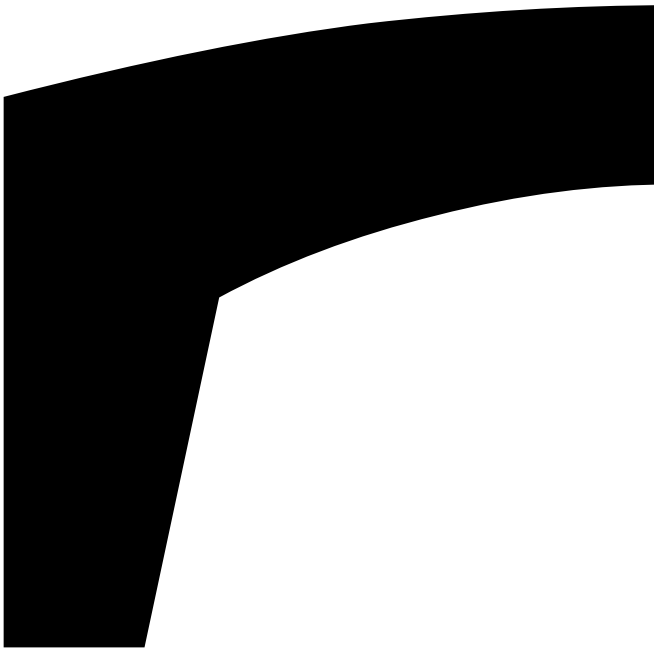


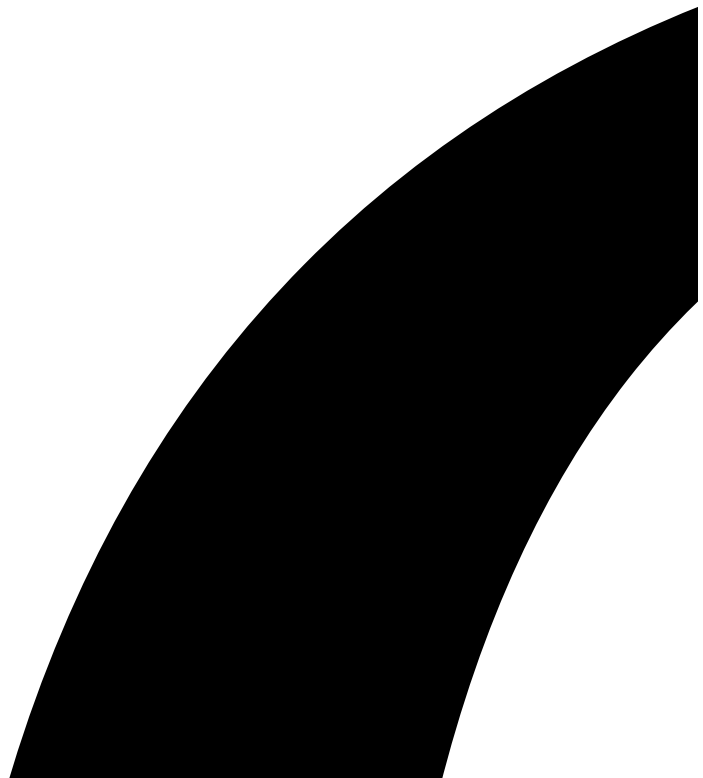




dcxc



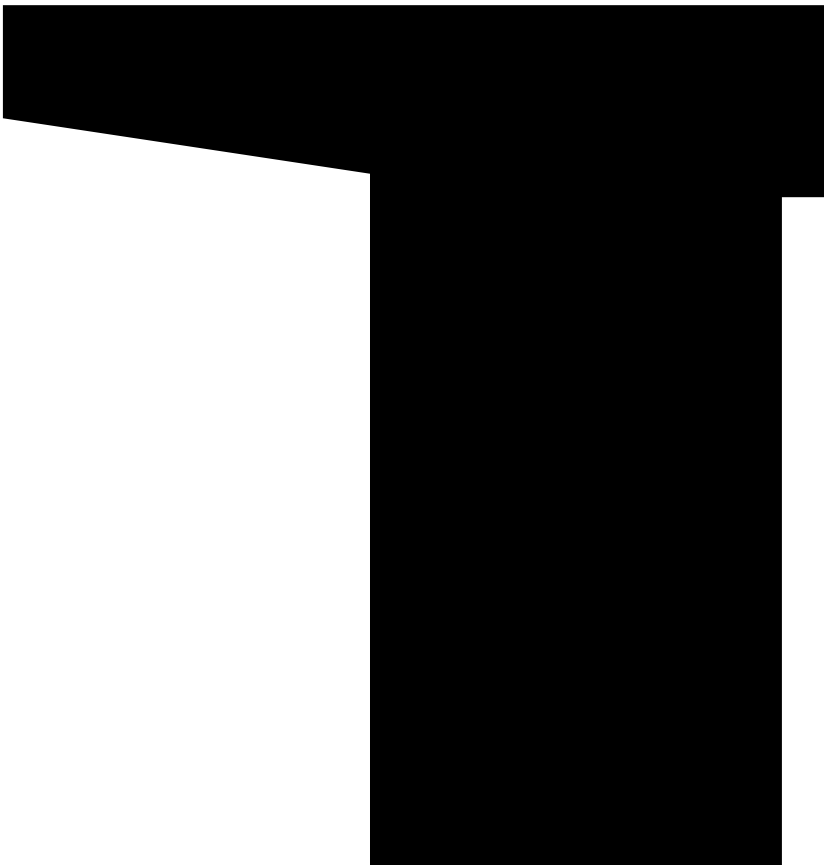




dcxcv

dcxcv

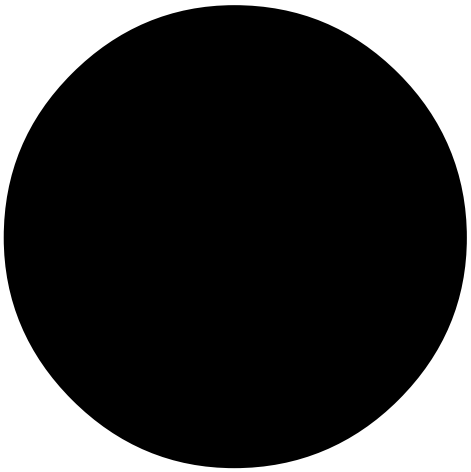




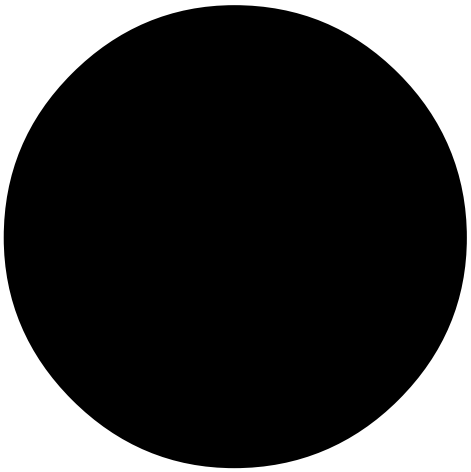
dcc

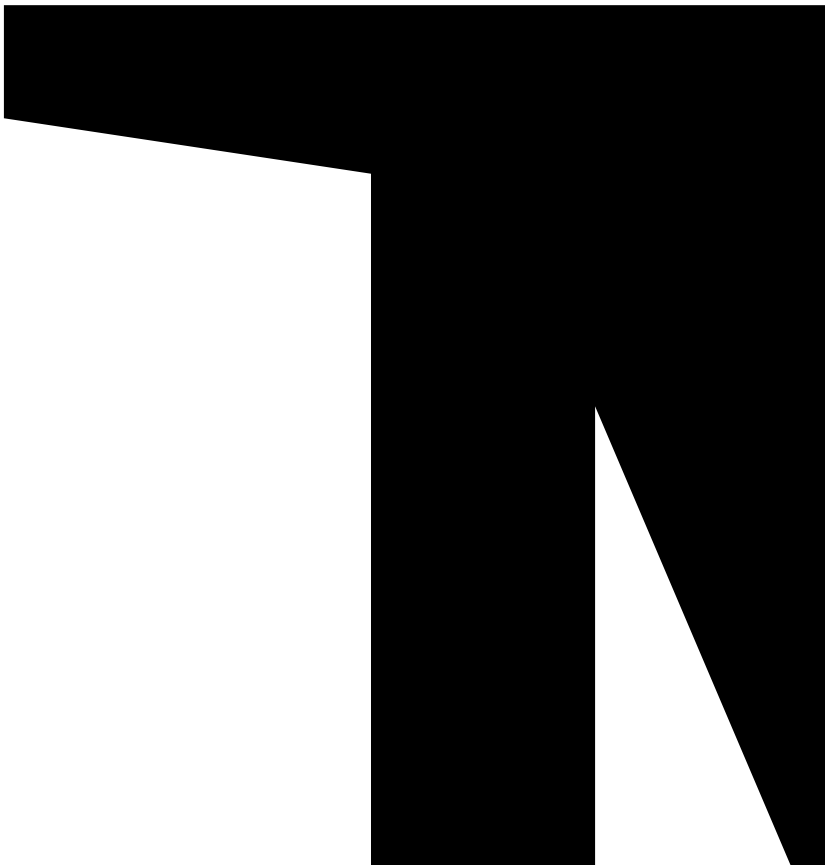




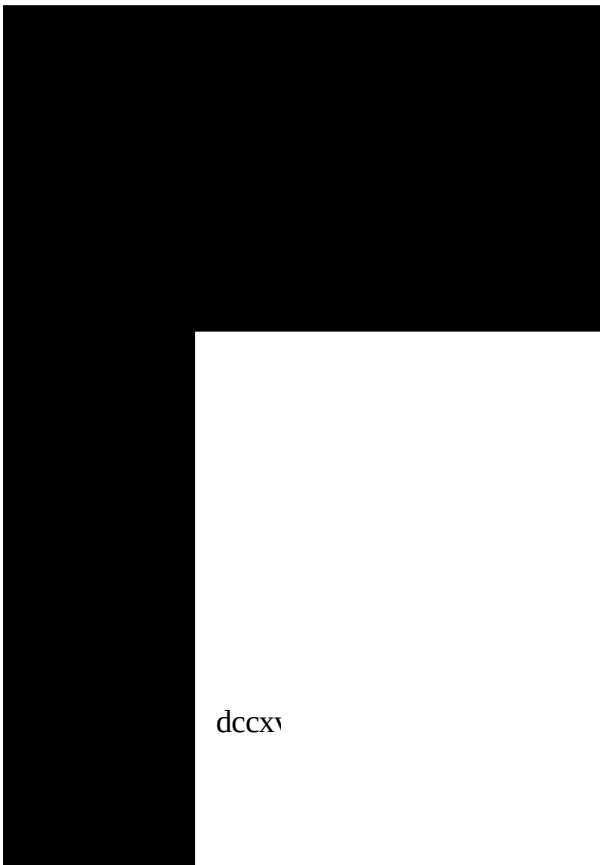




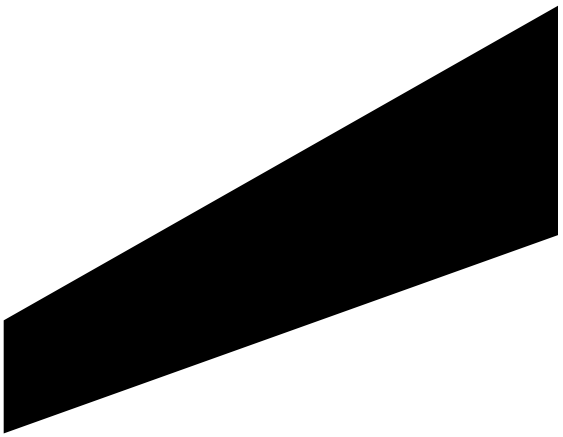




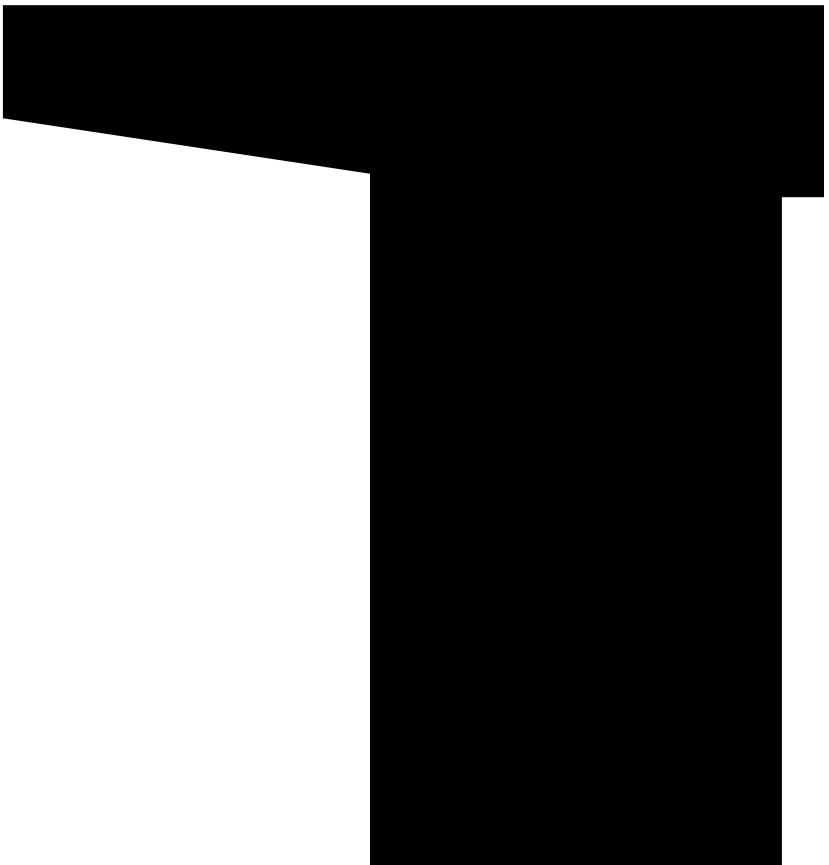




dccxv



dccxx

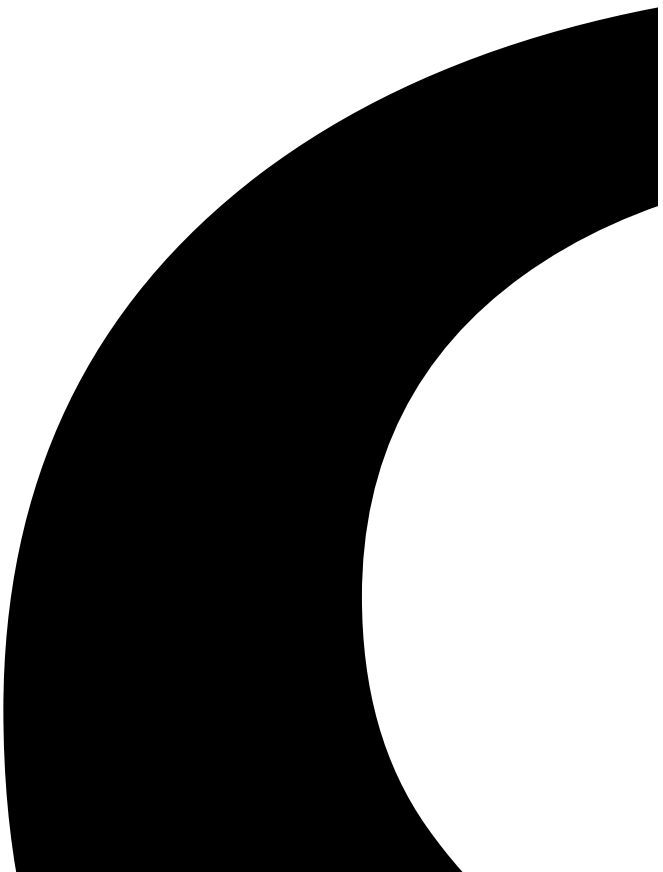




dccx

dccxx

dccxx

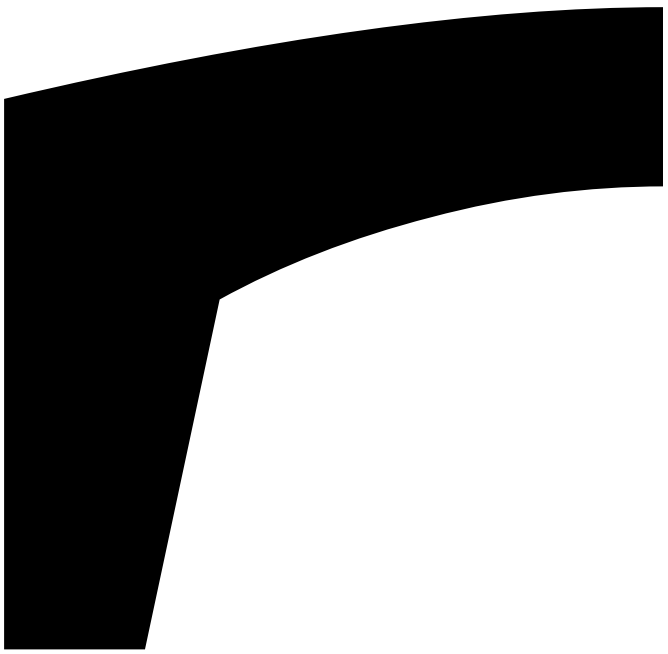




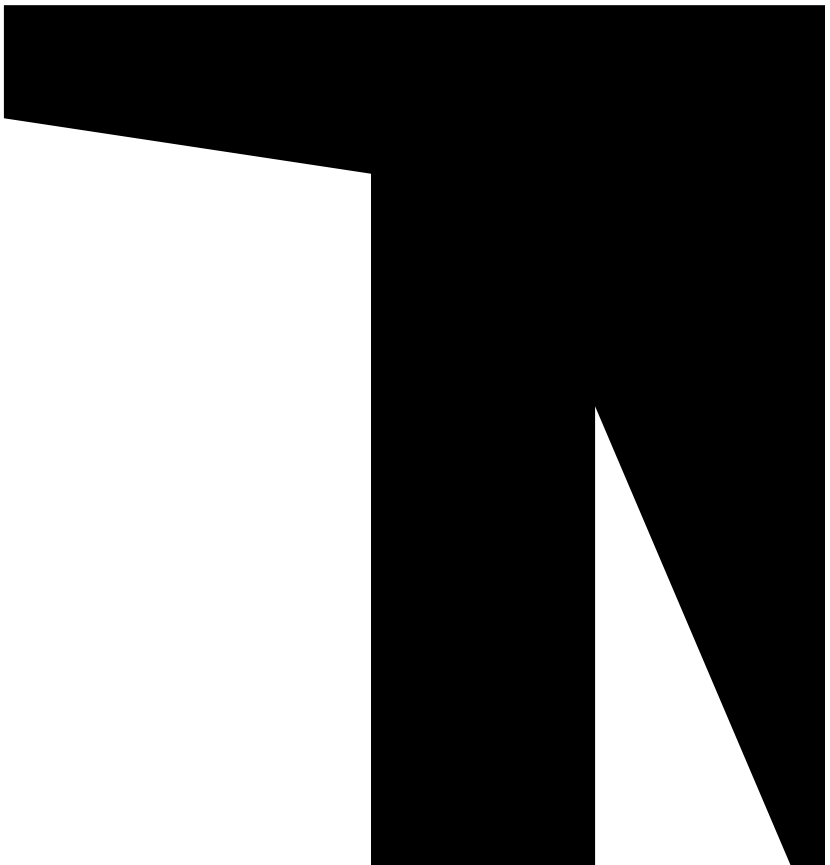
dccx

dccx

dccxx

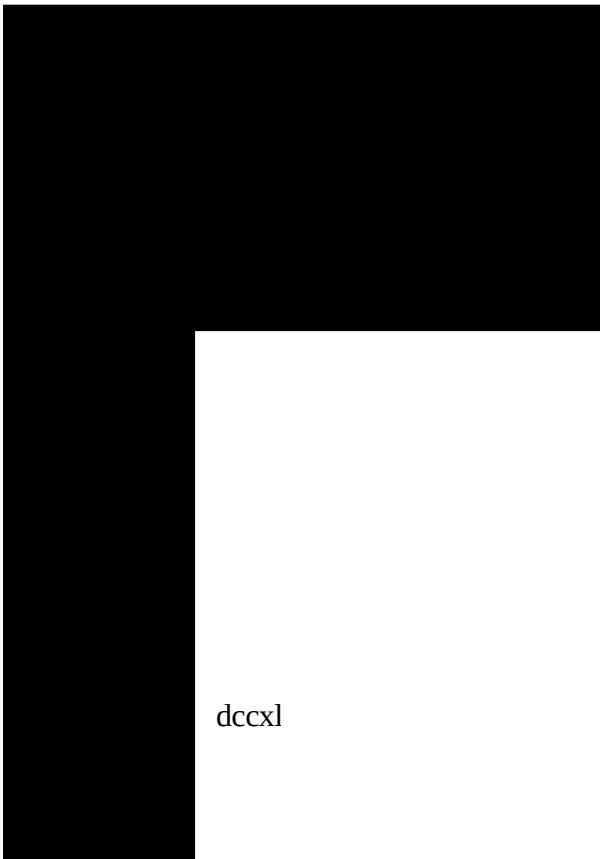


dccx

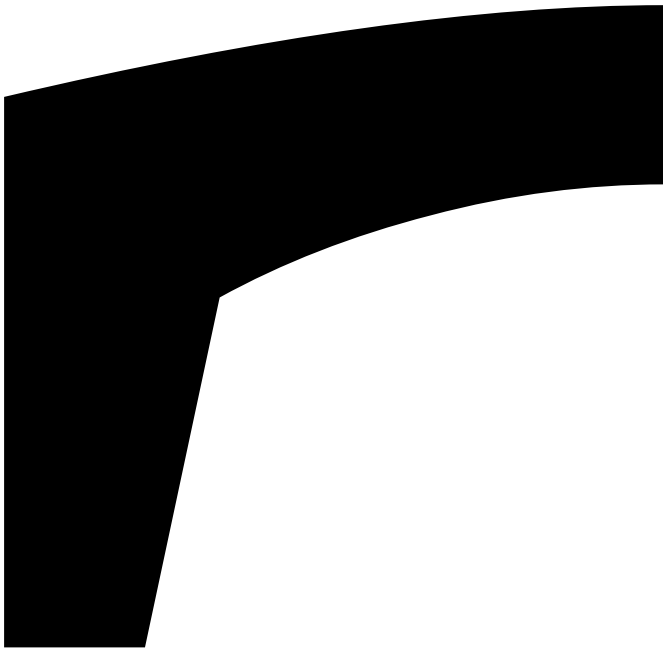


dccxx





dccxl

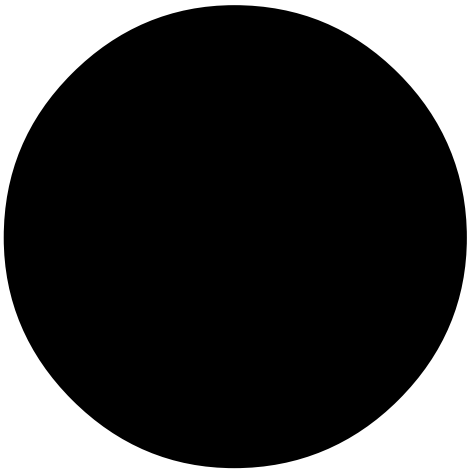


dccxl

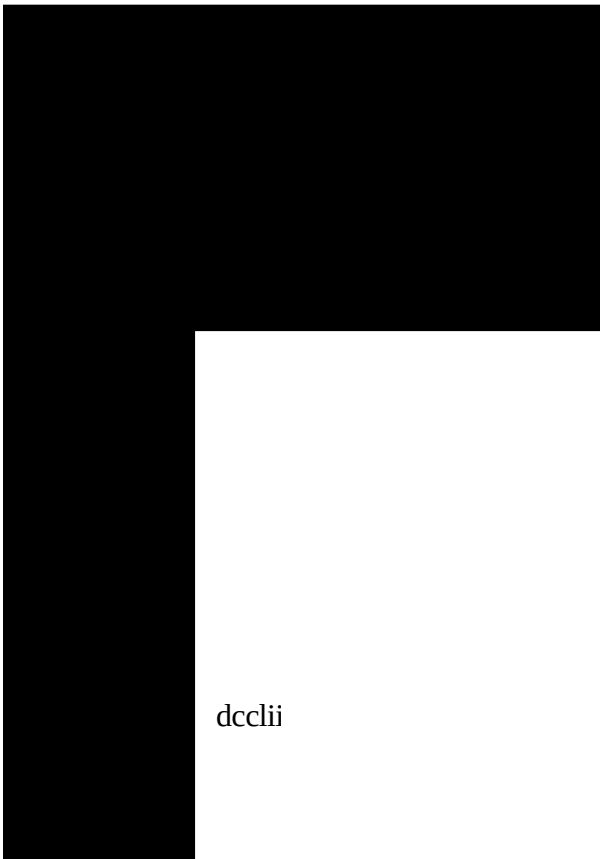


dccxl

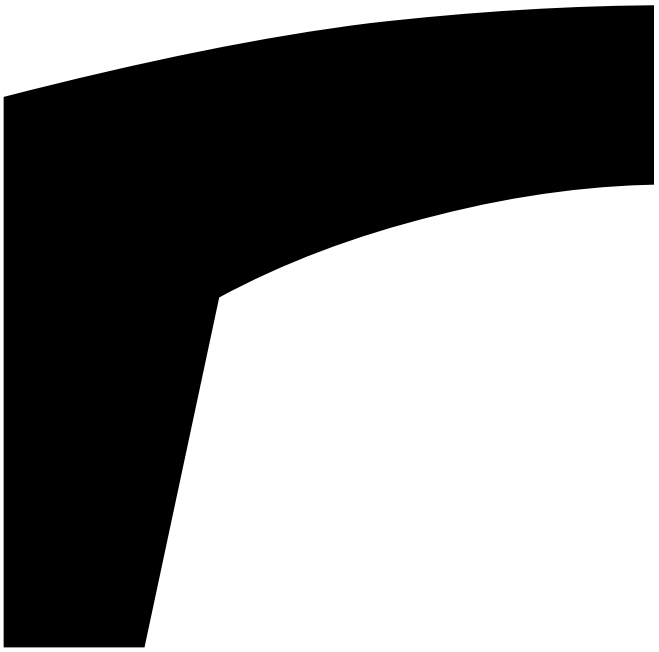




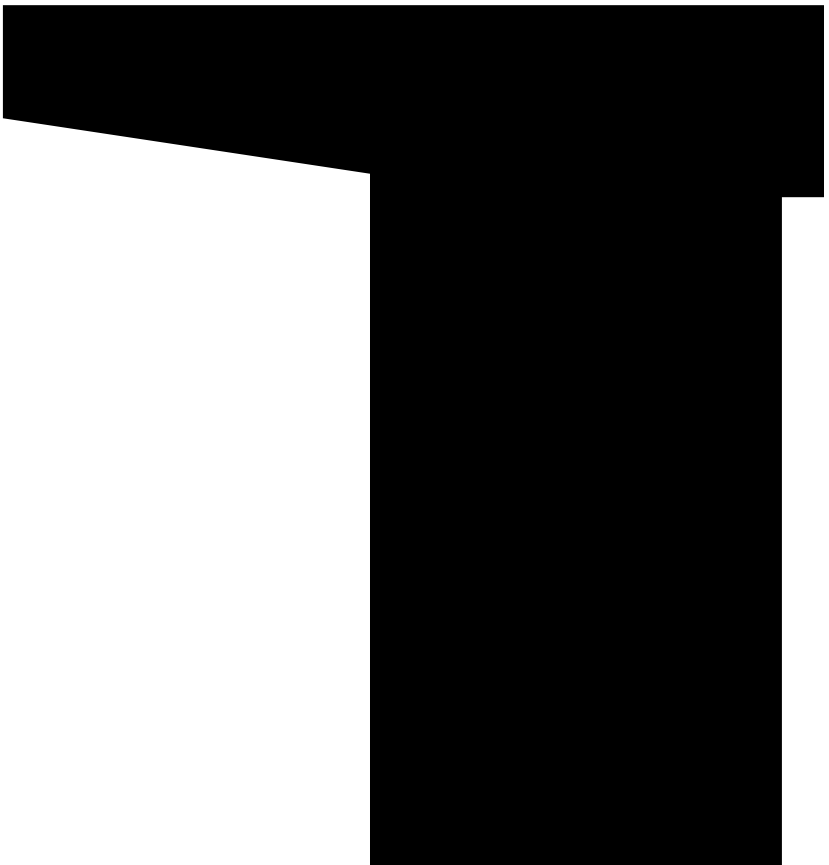
dccl



dcclii

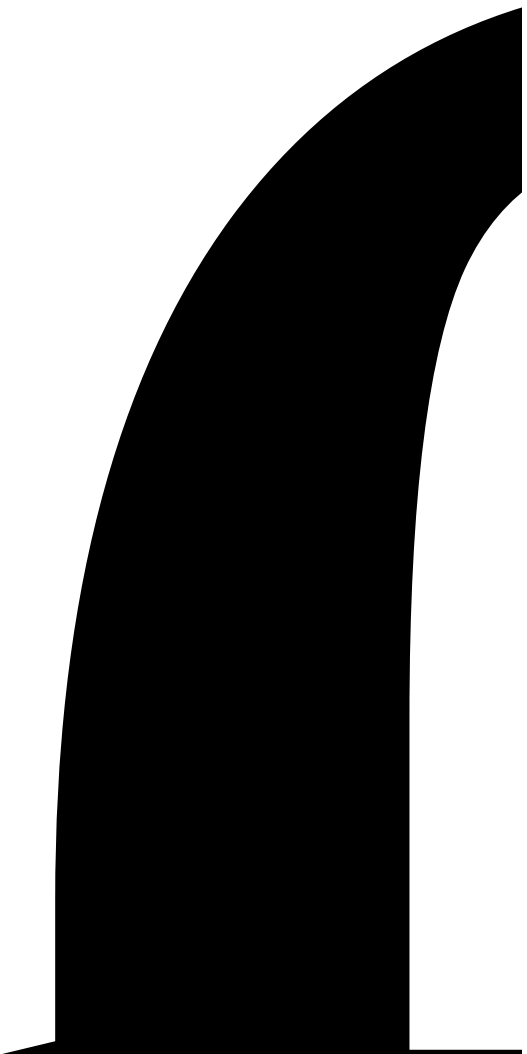


dccliv



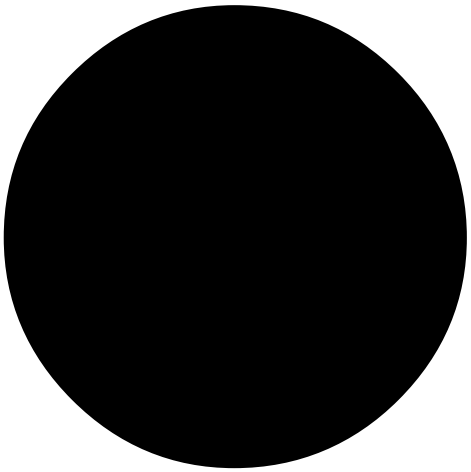
dcclv

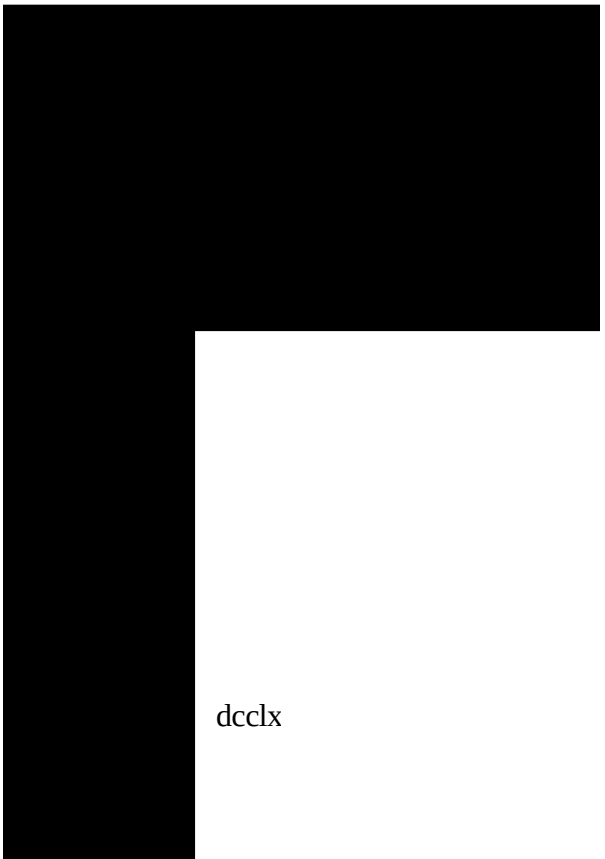




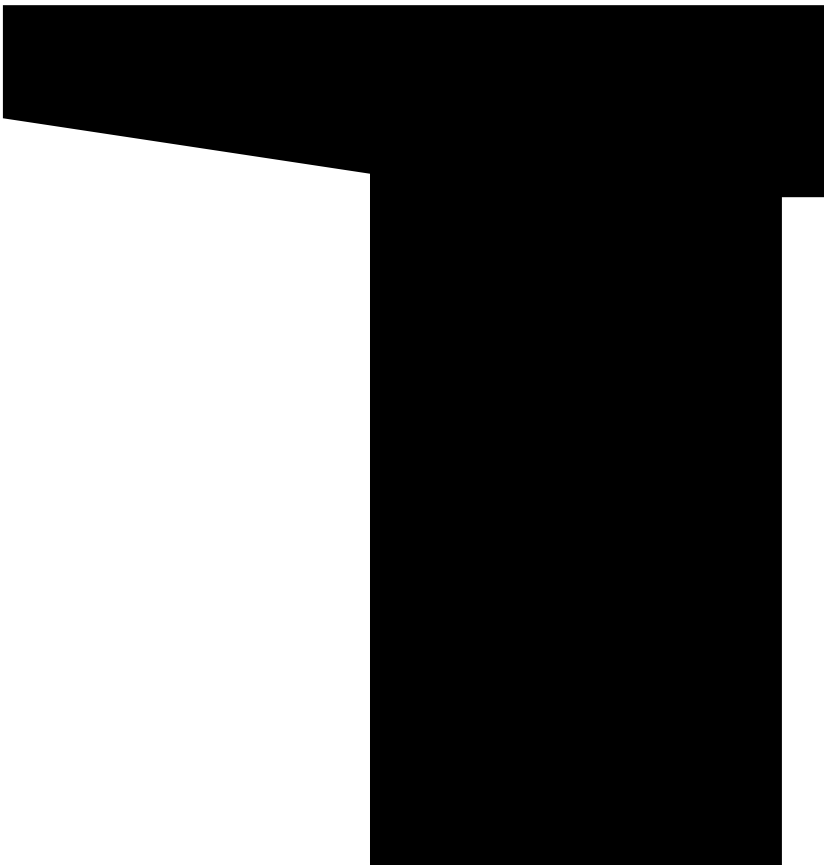
dccli

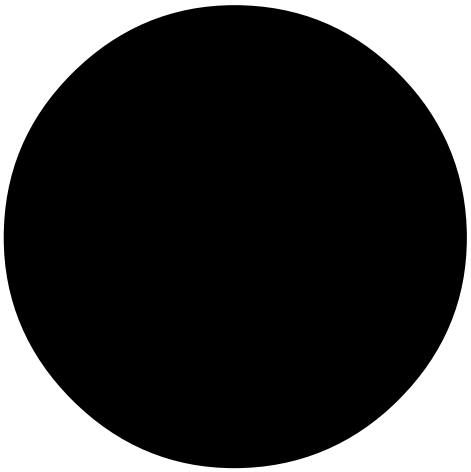


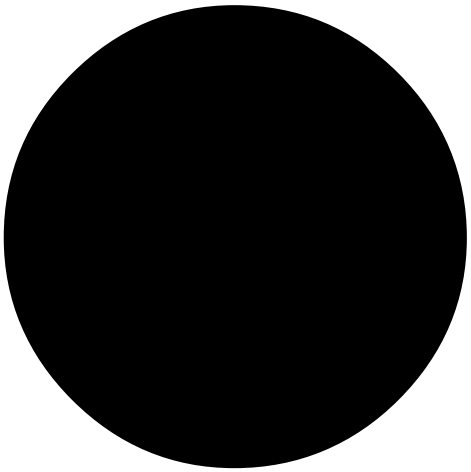


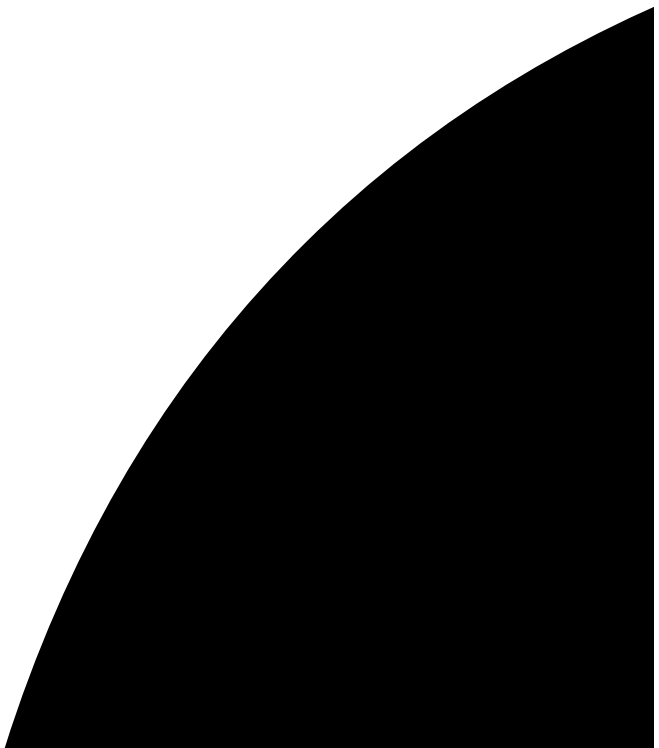


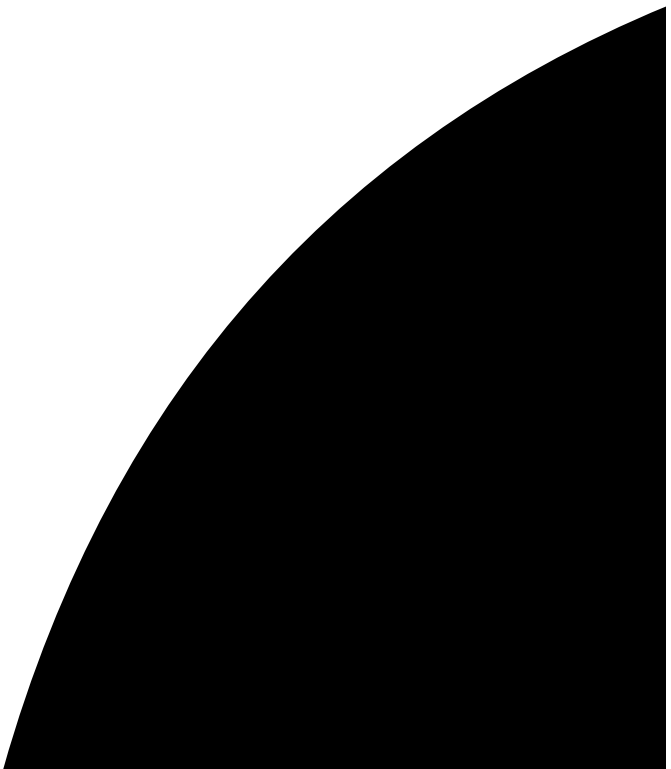
dcclx

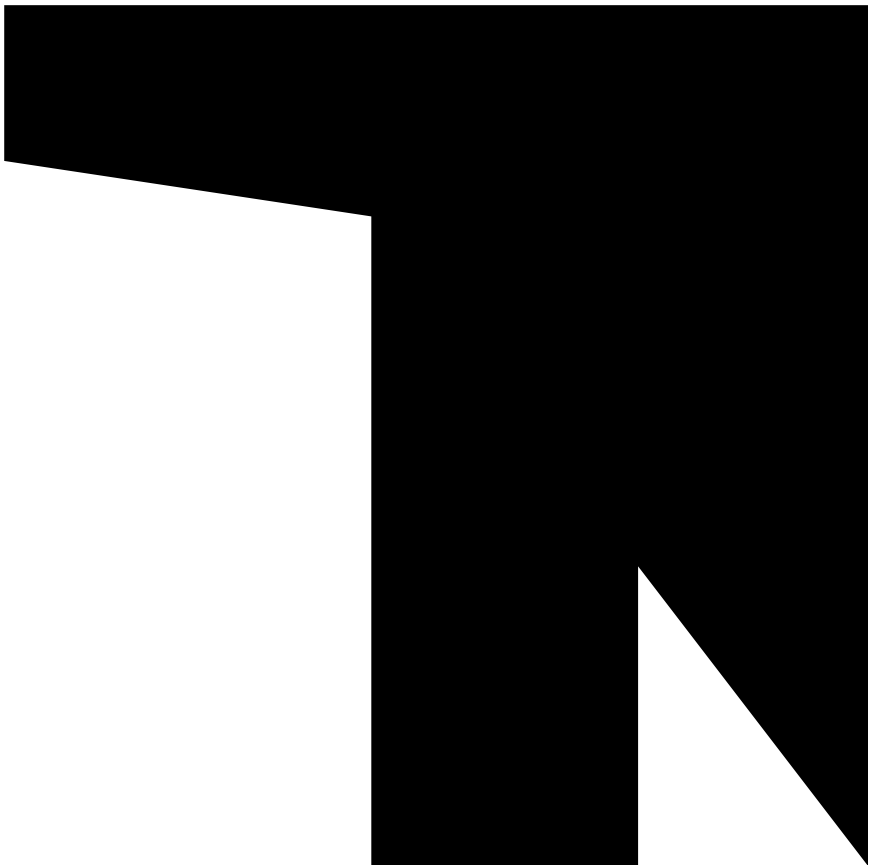


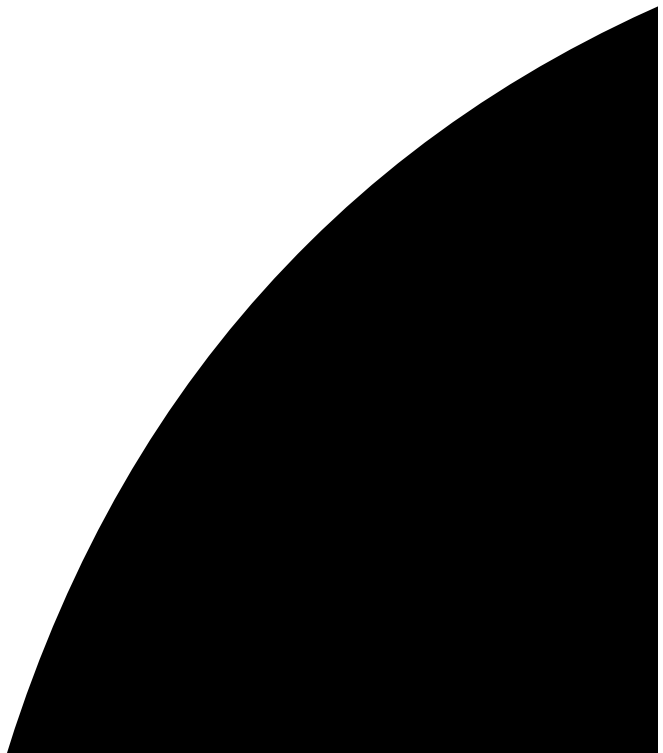


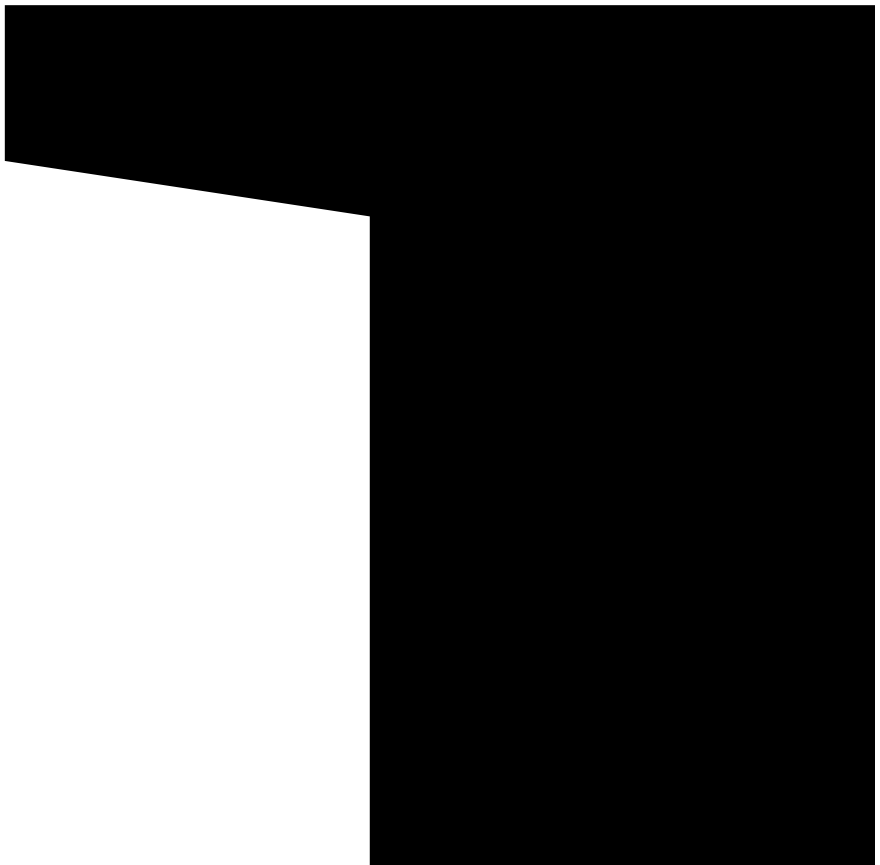


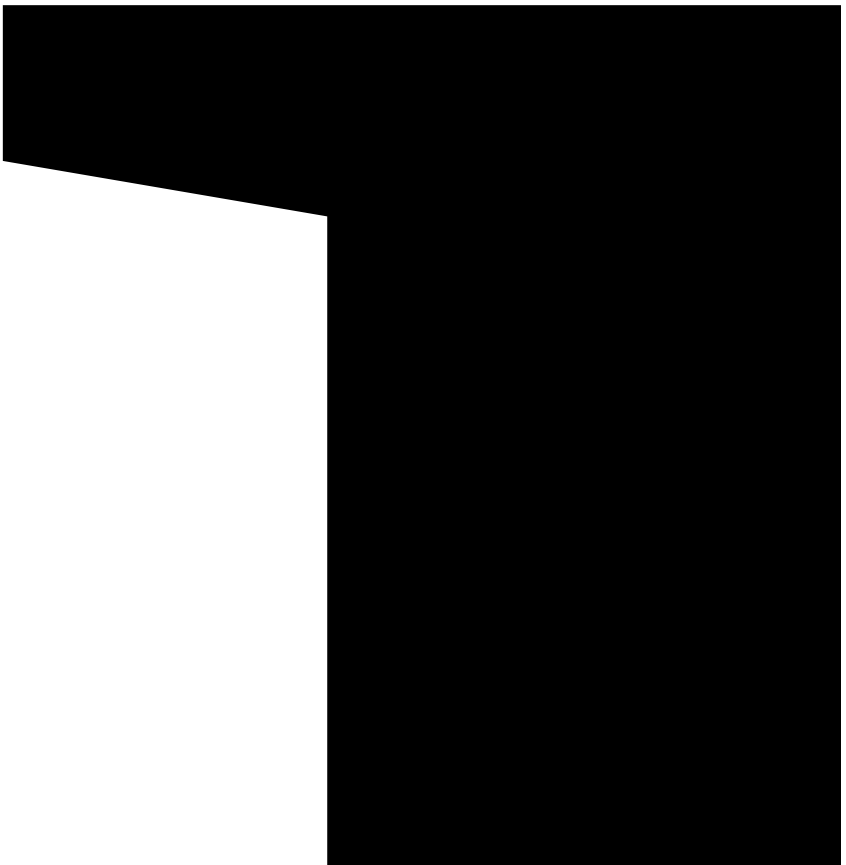


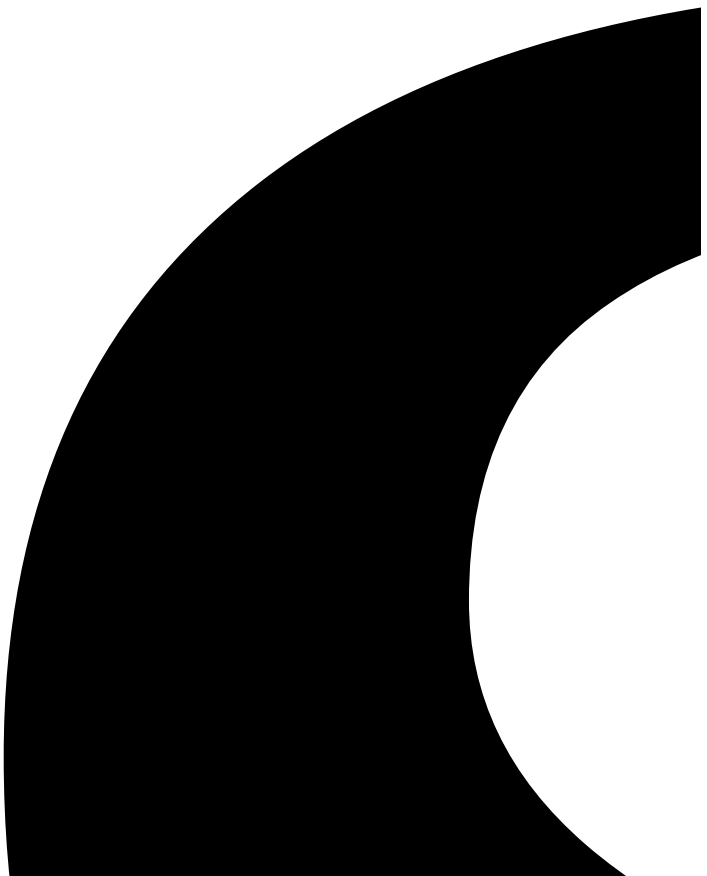


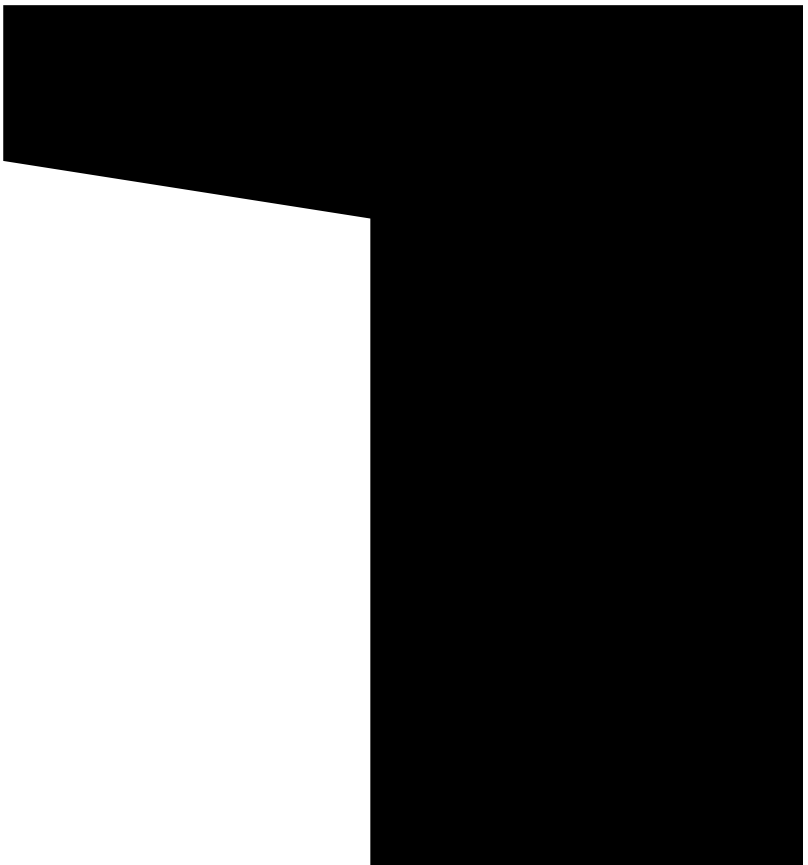


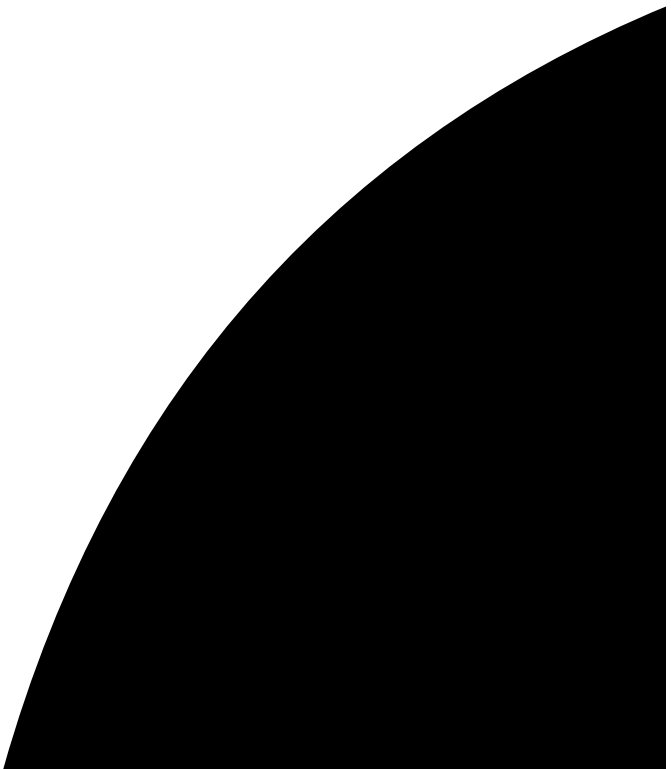


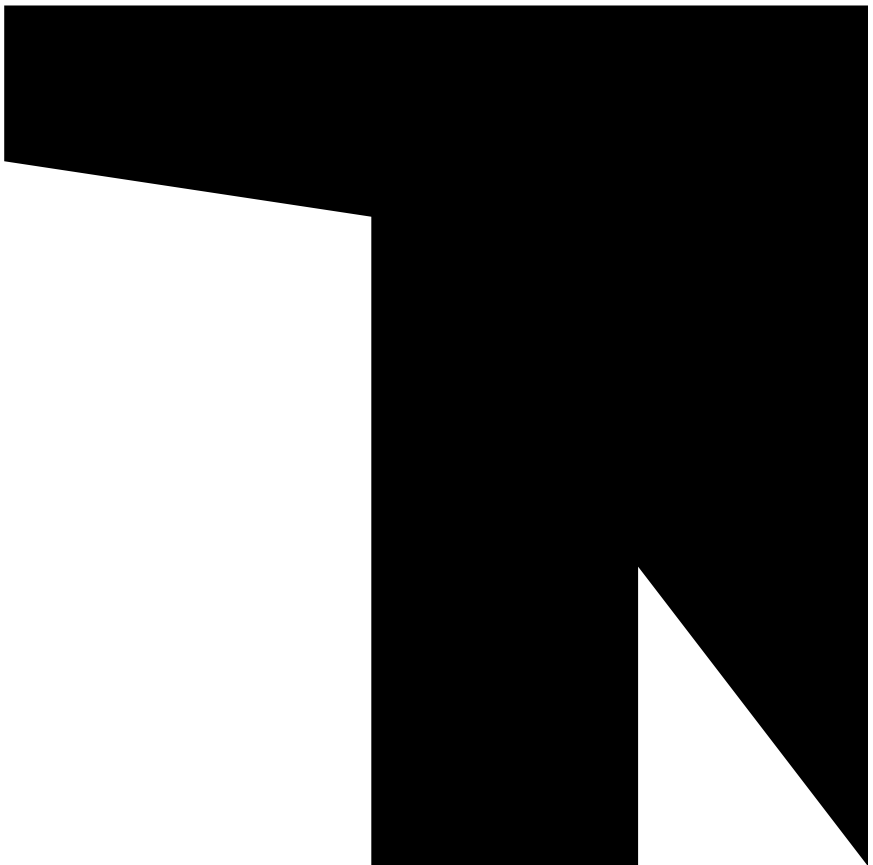




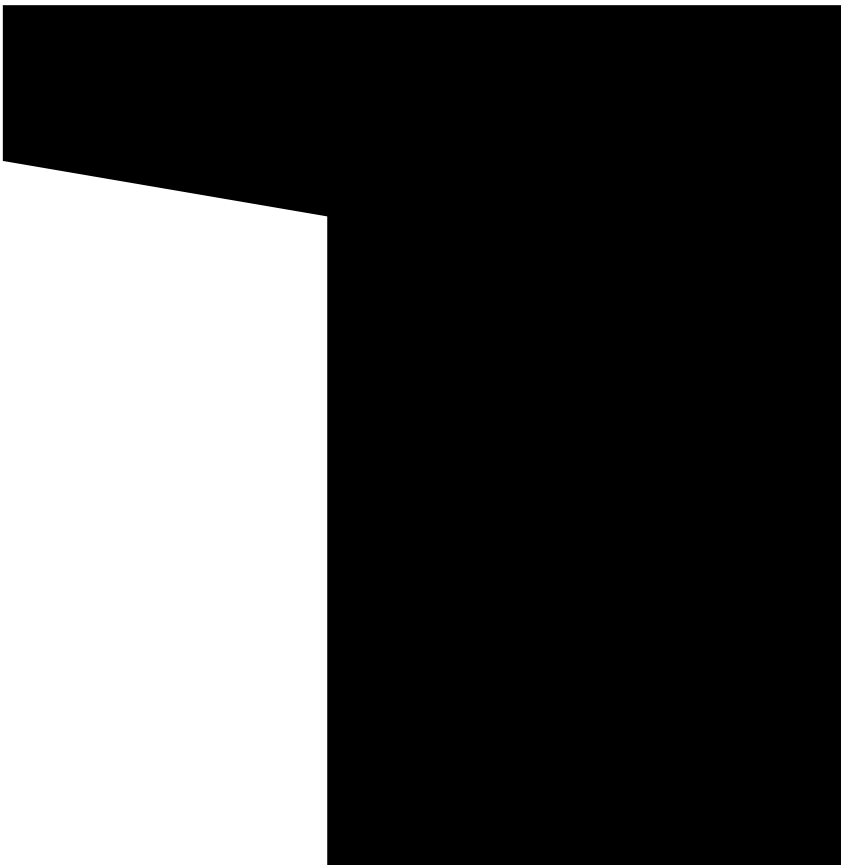


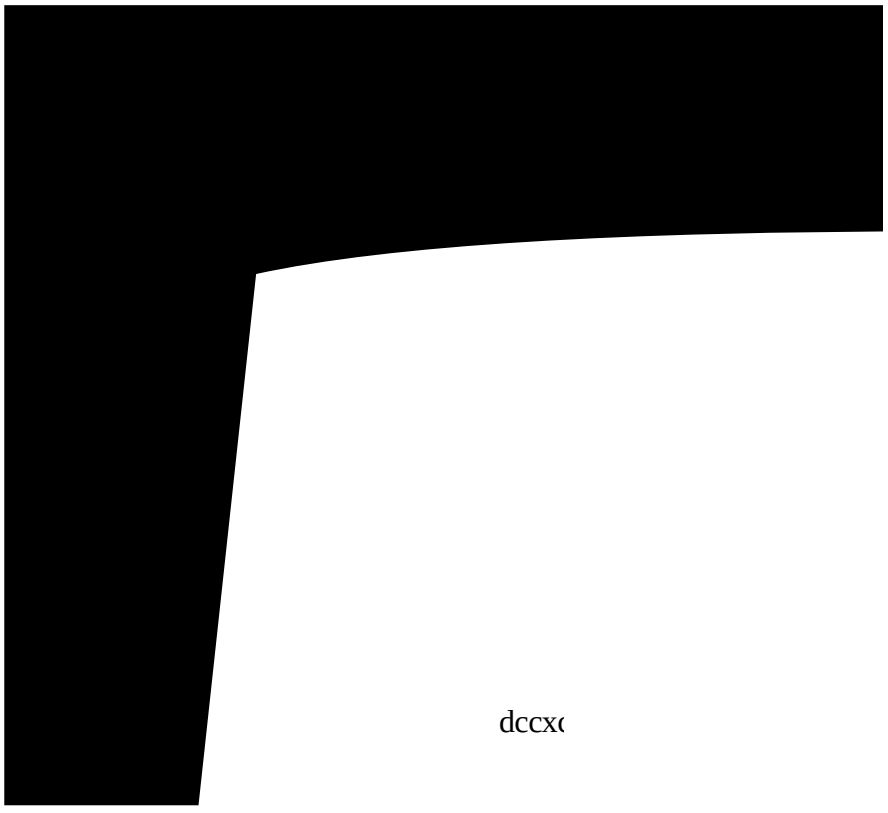




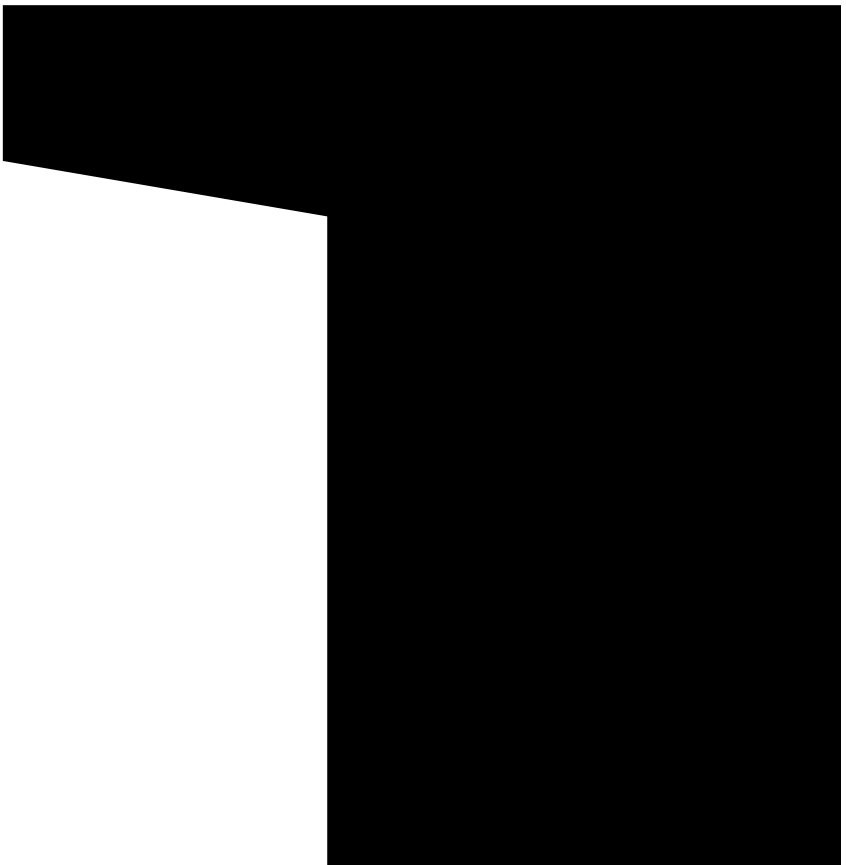


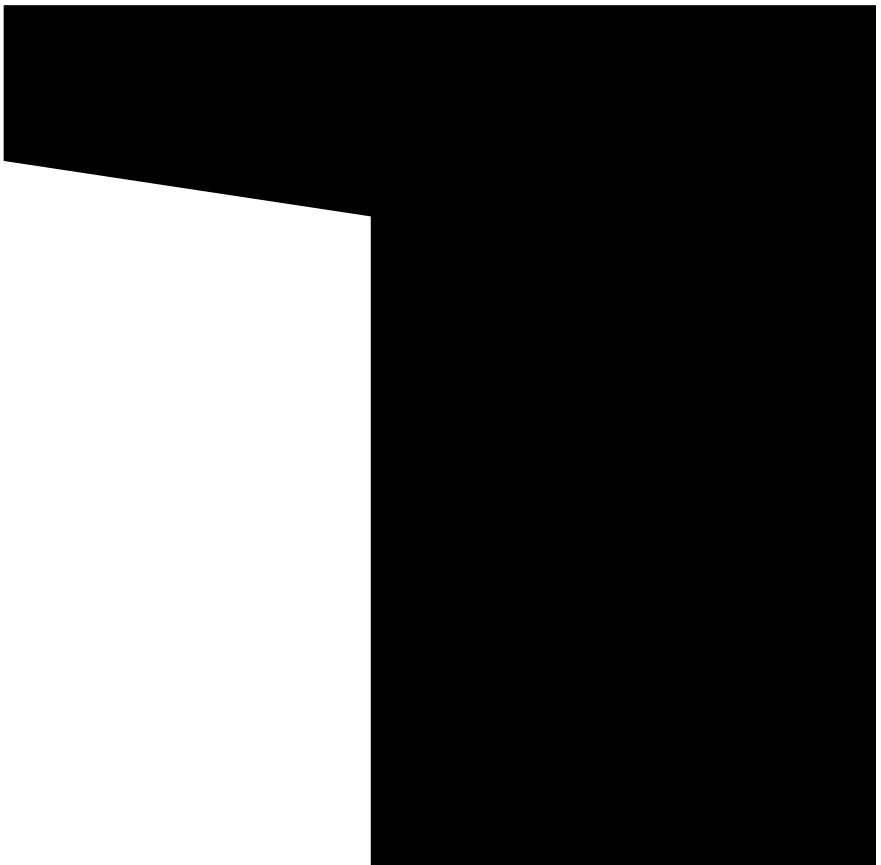


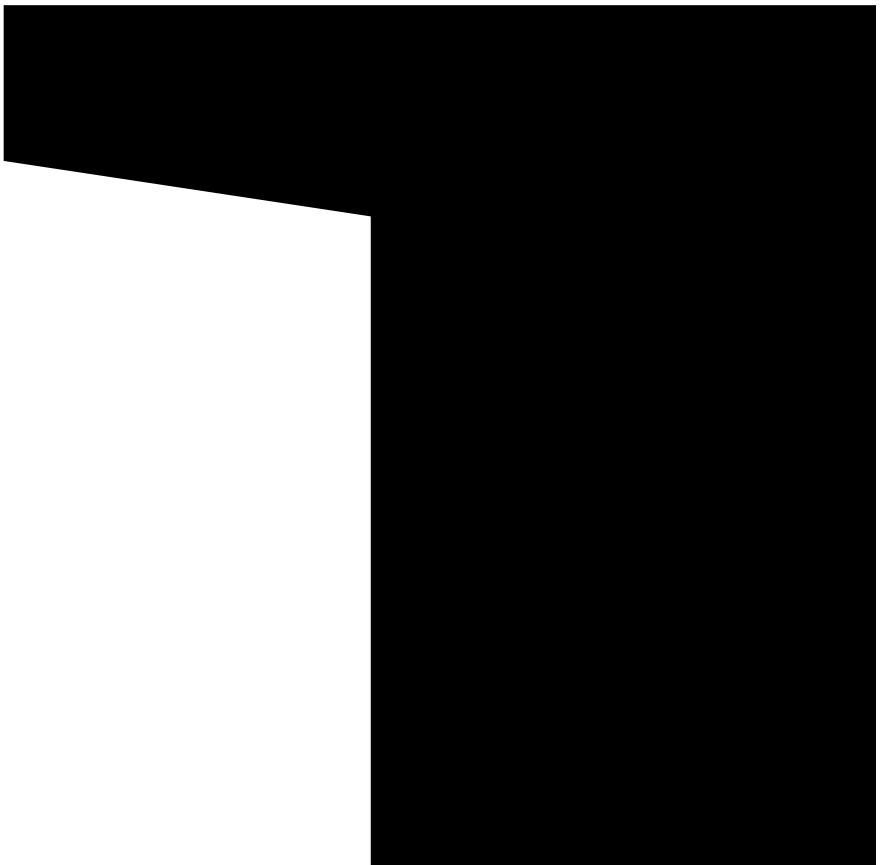


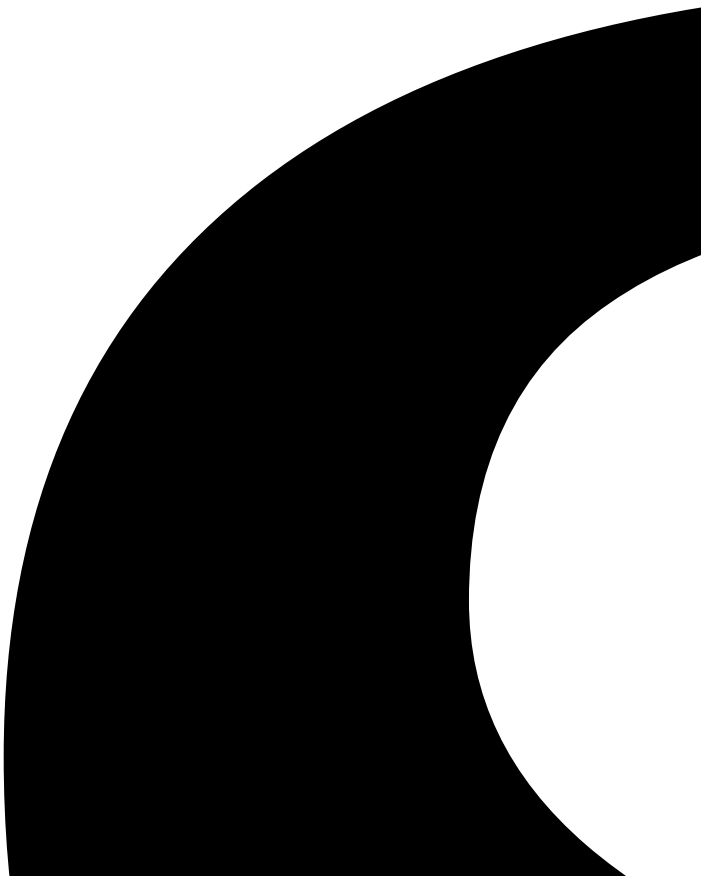


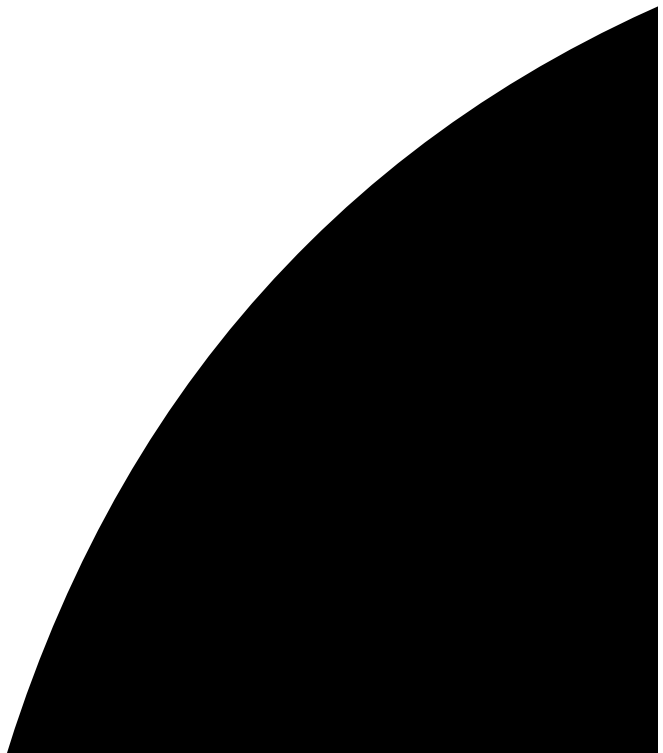
dccx‹

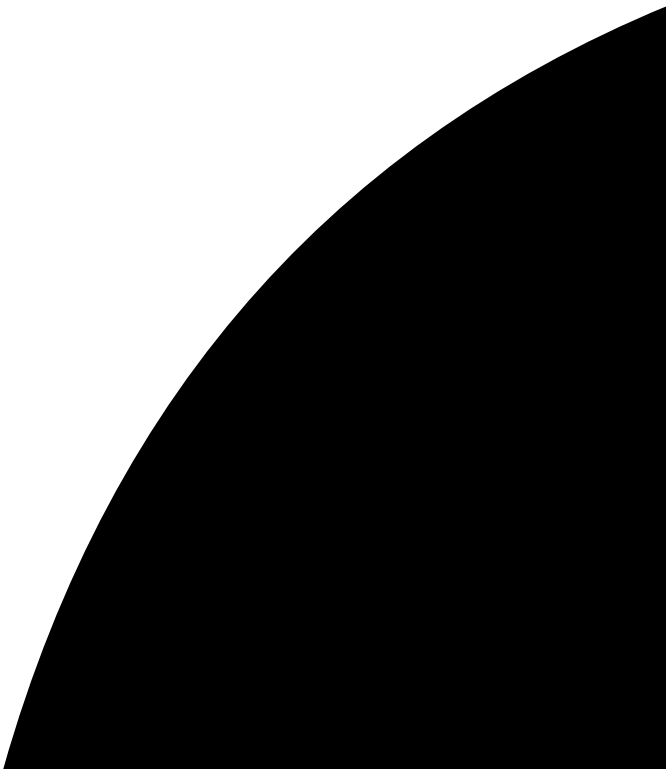


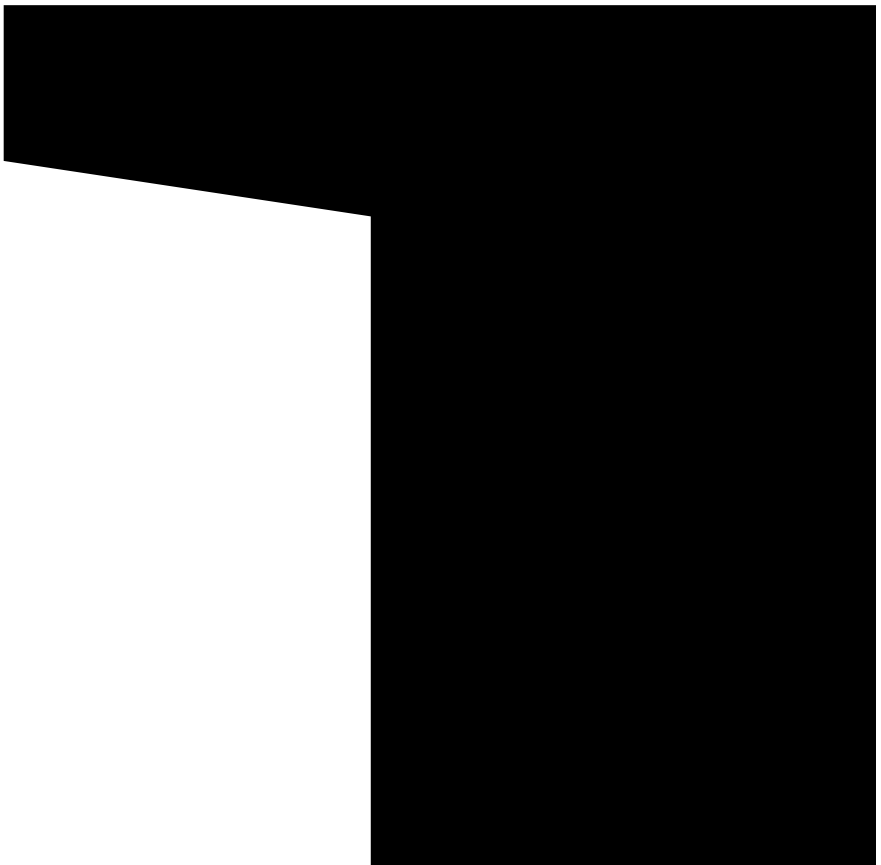


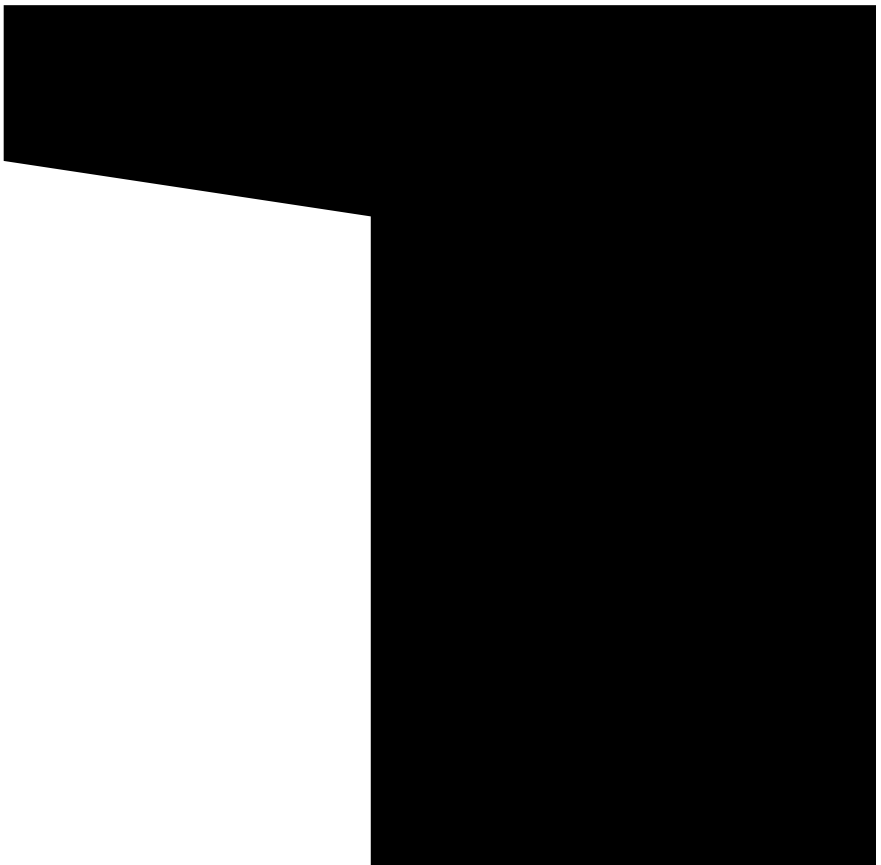


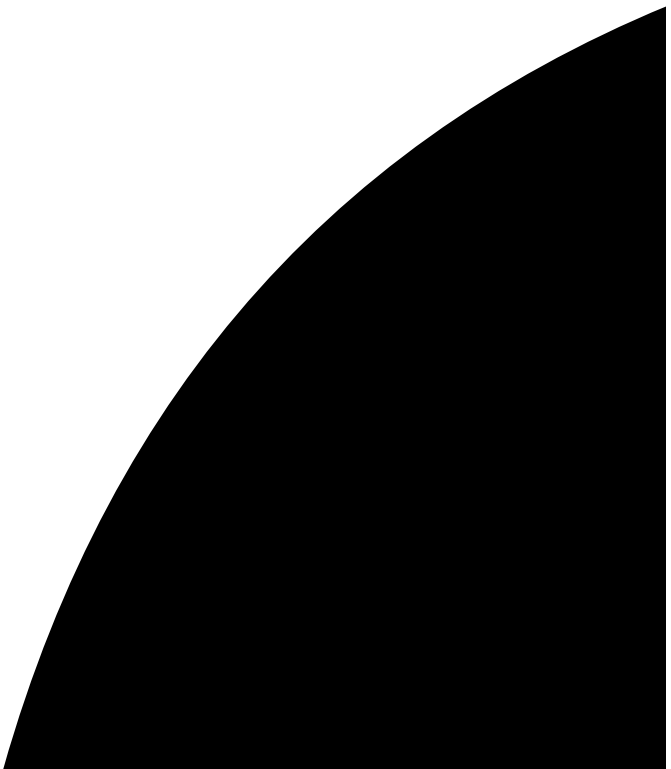


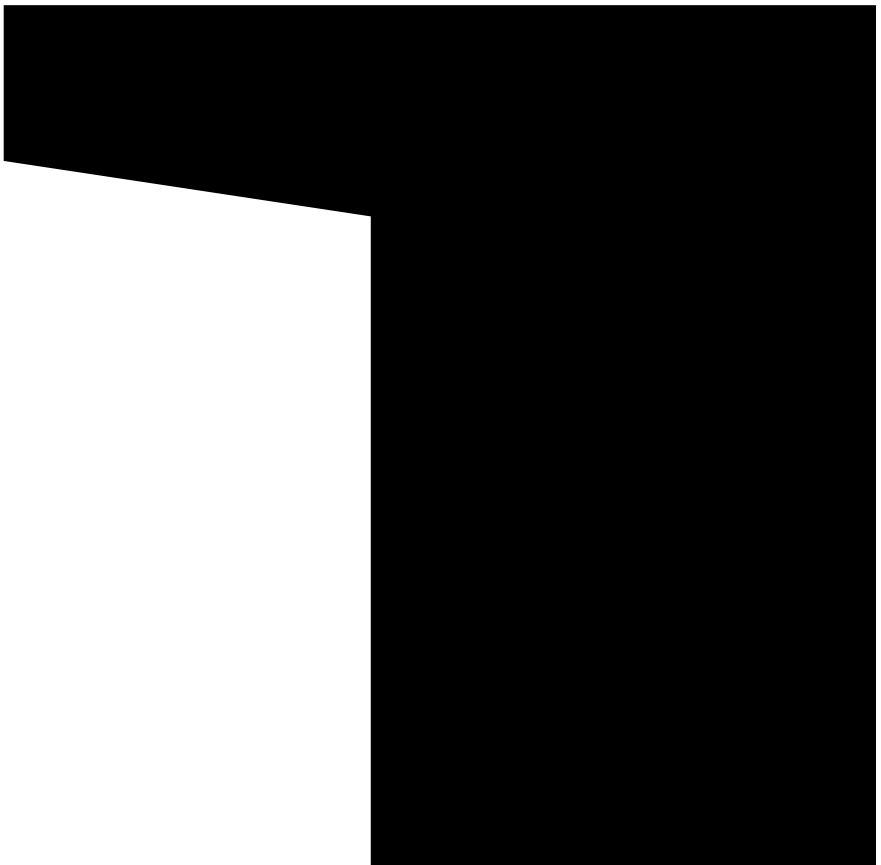






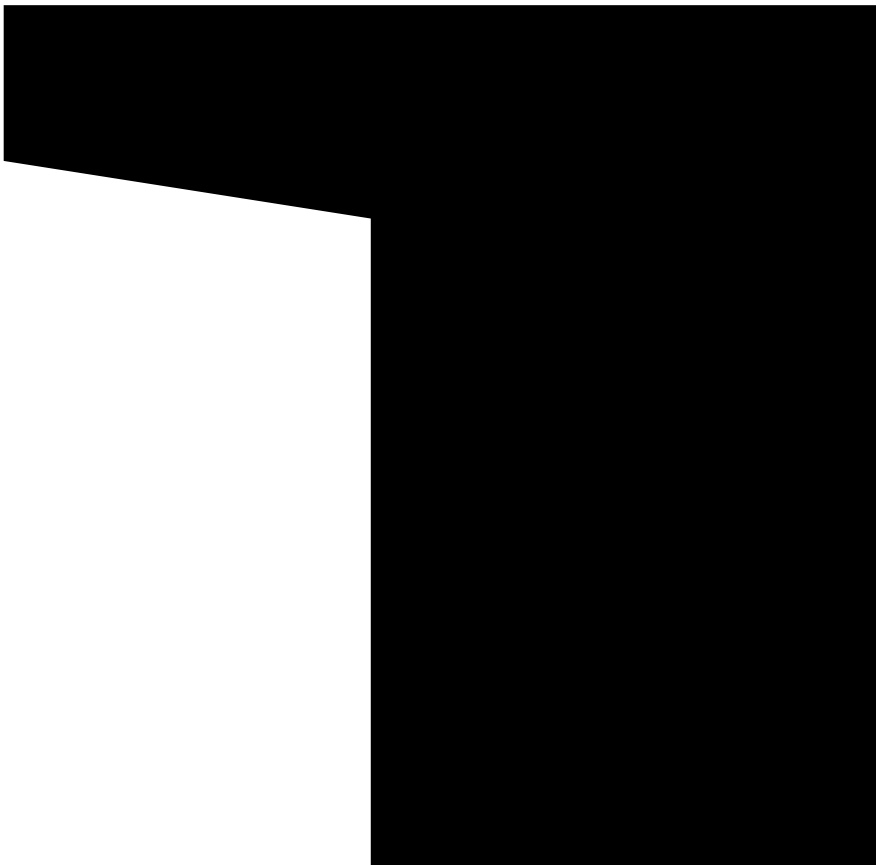


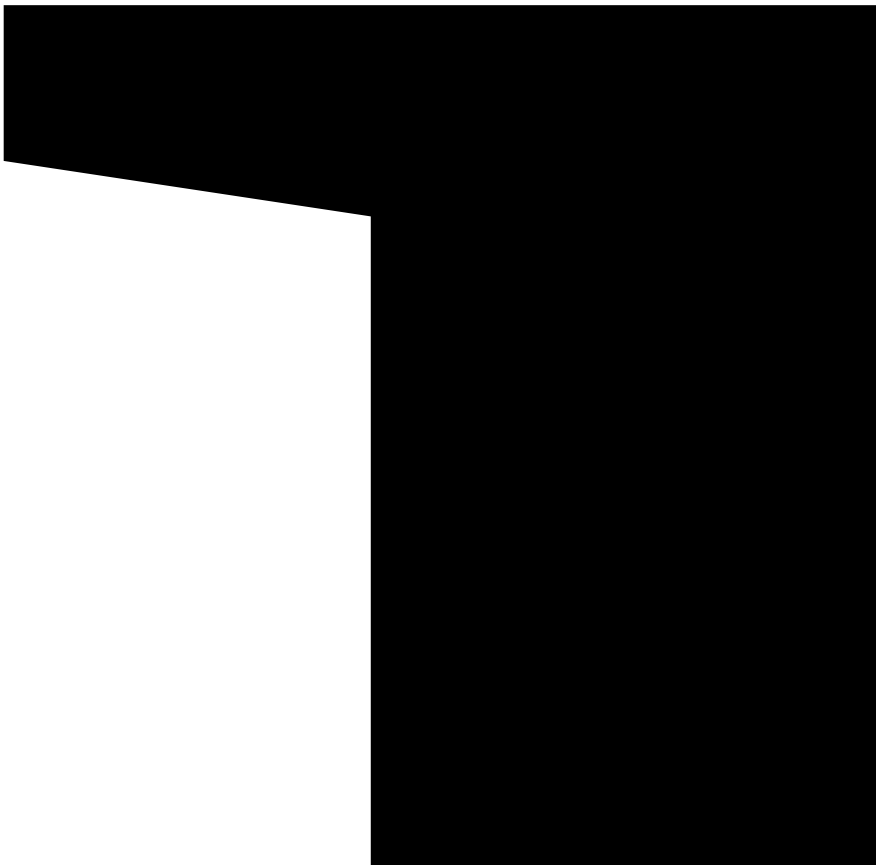


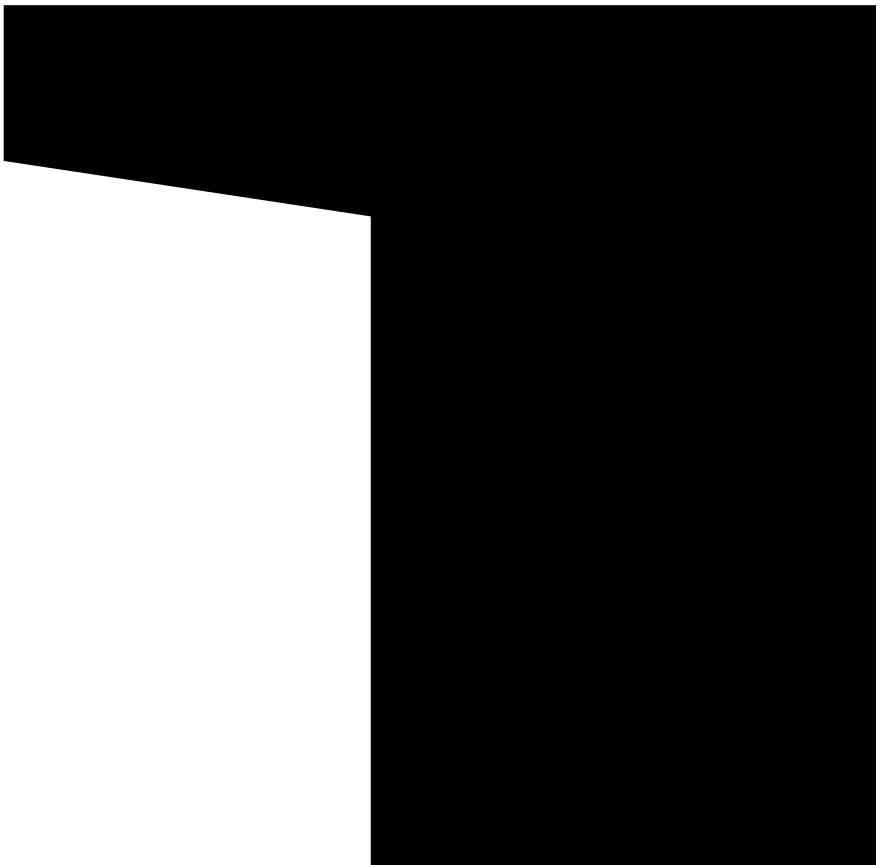


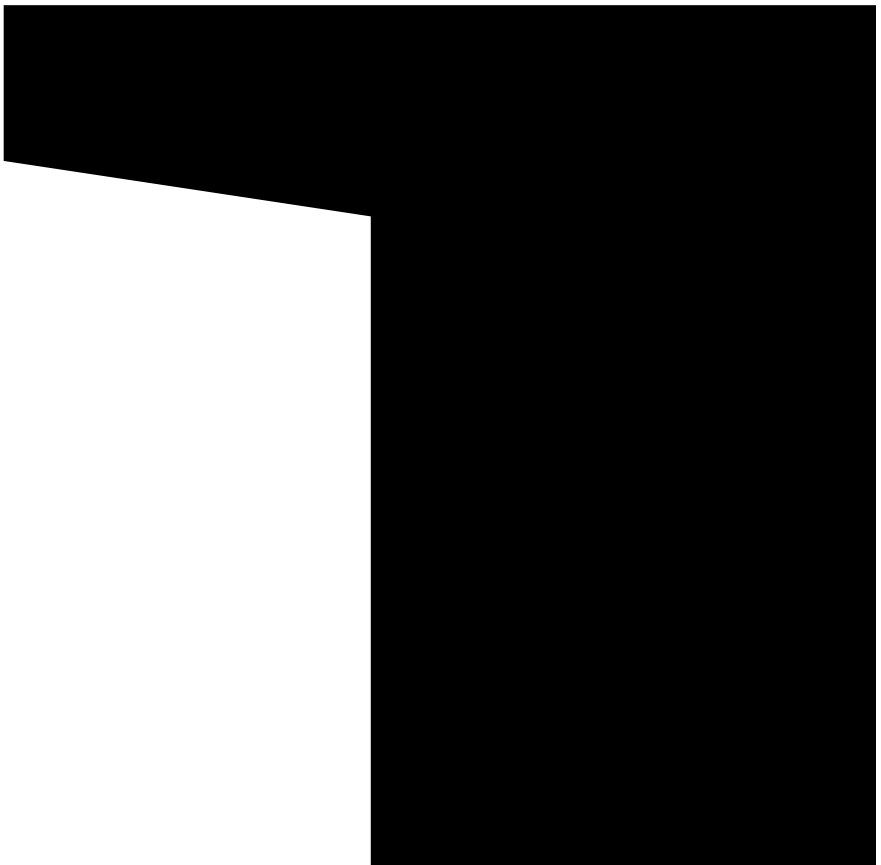


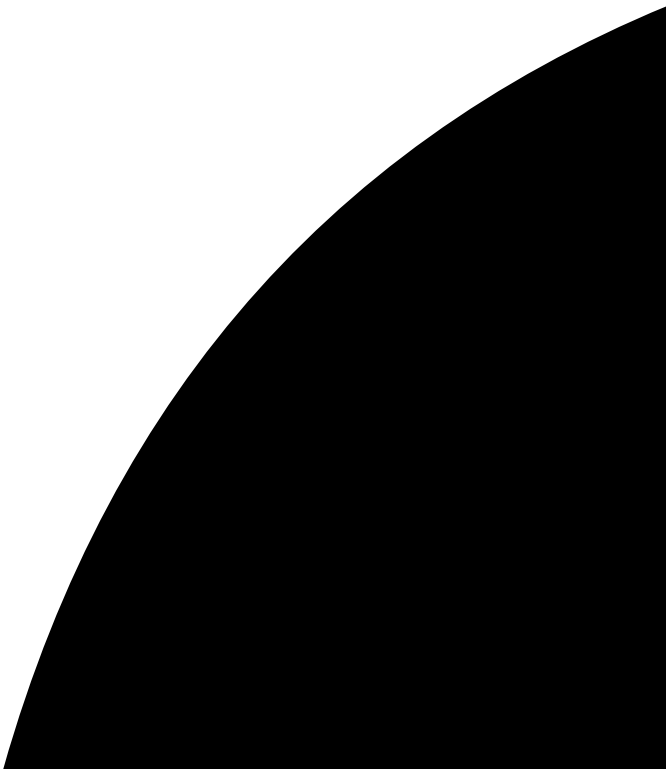
dcccx

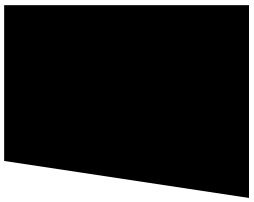




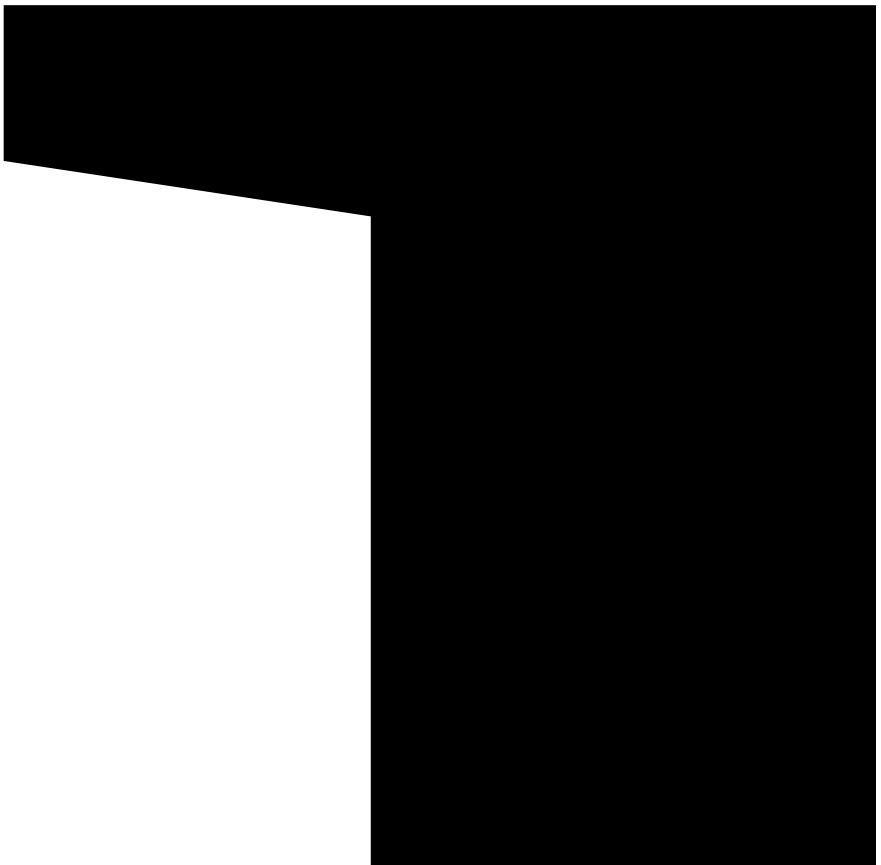


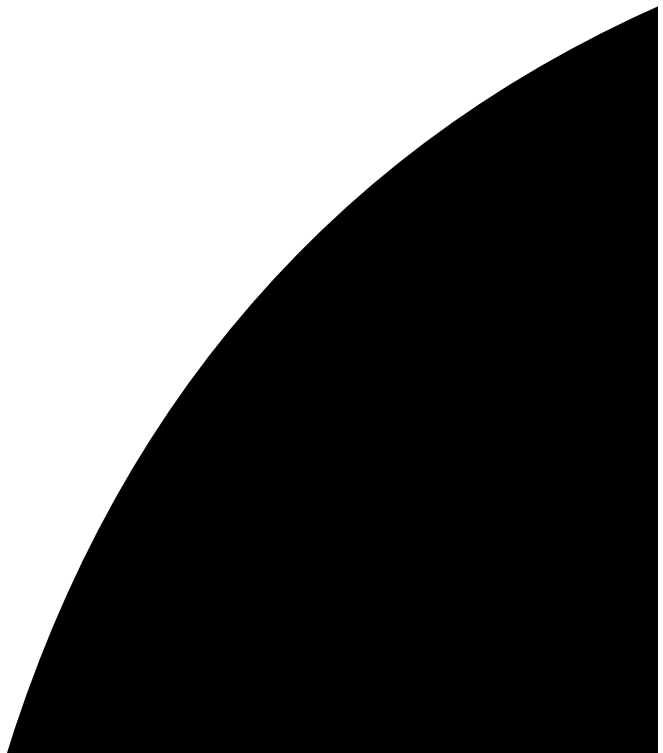






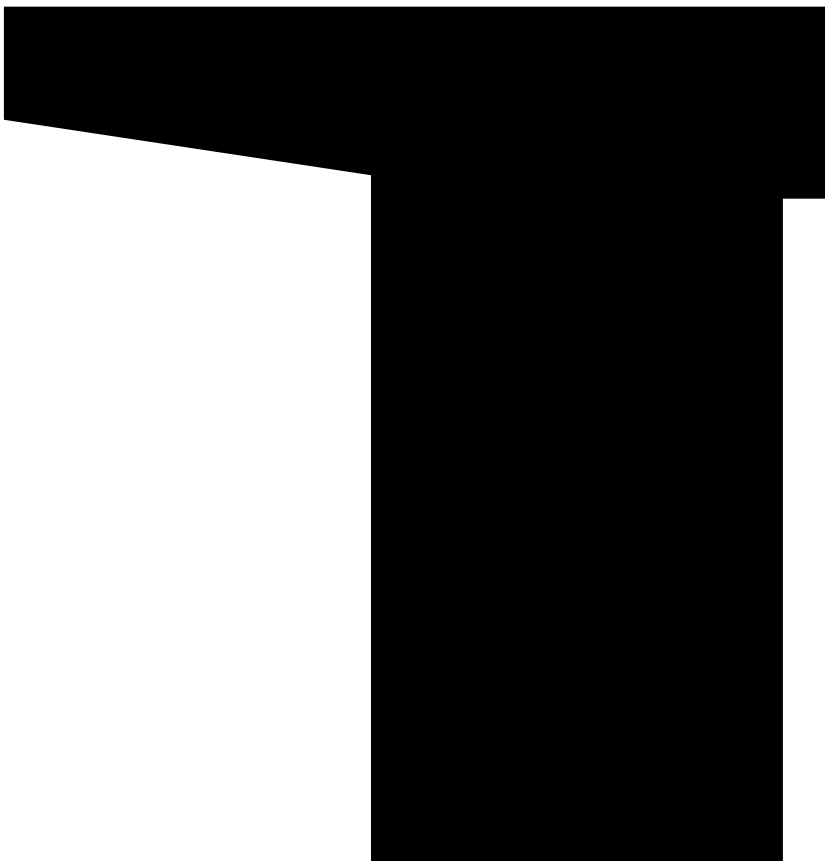
dcccx

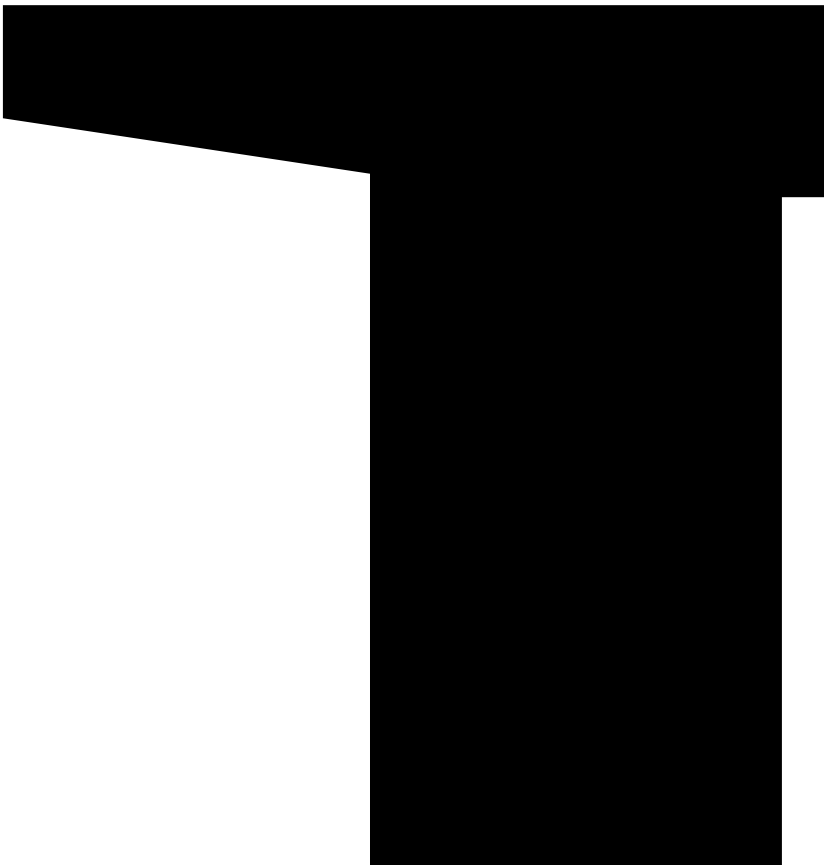


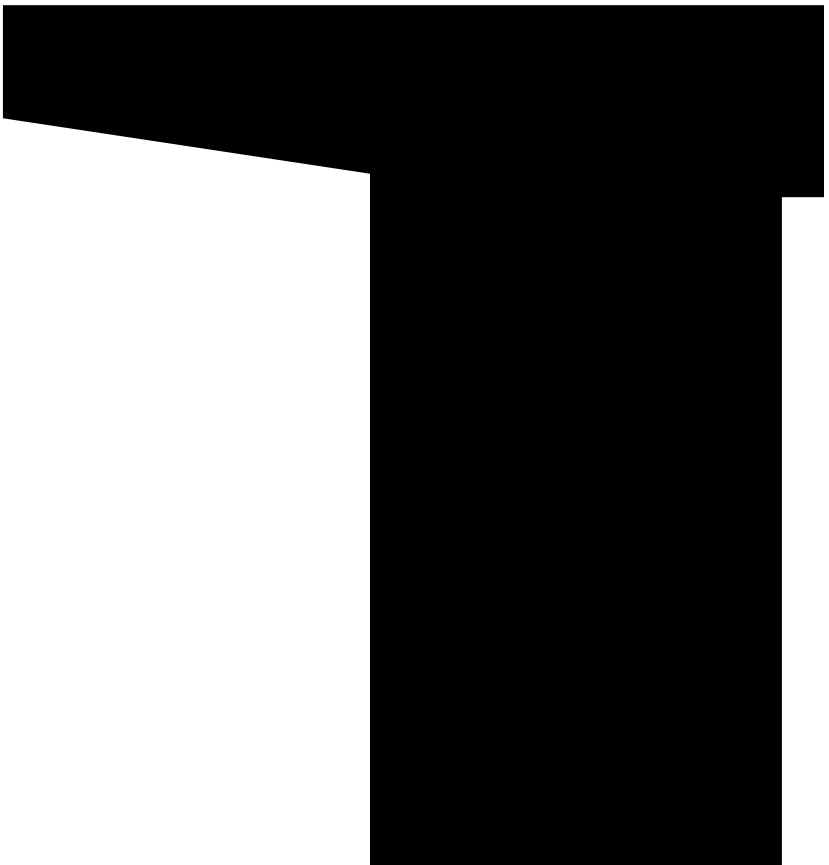


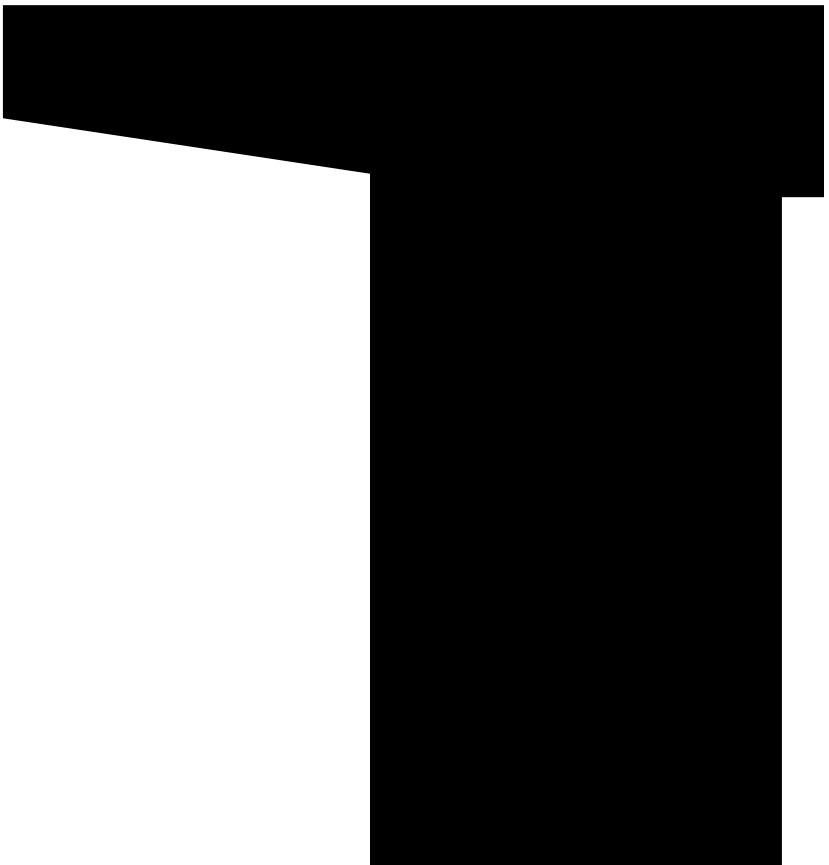


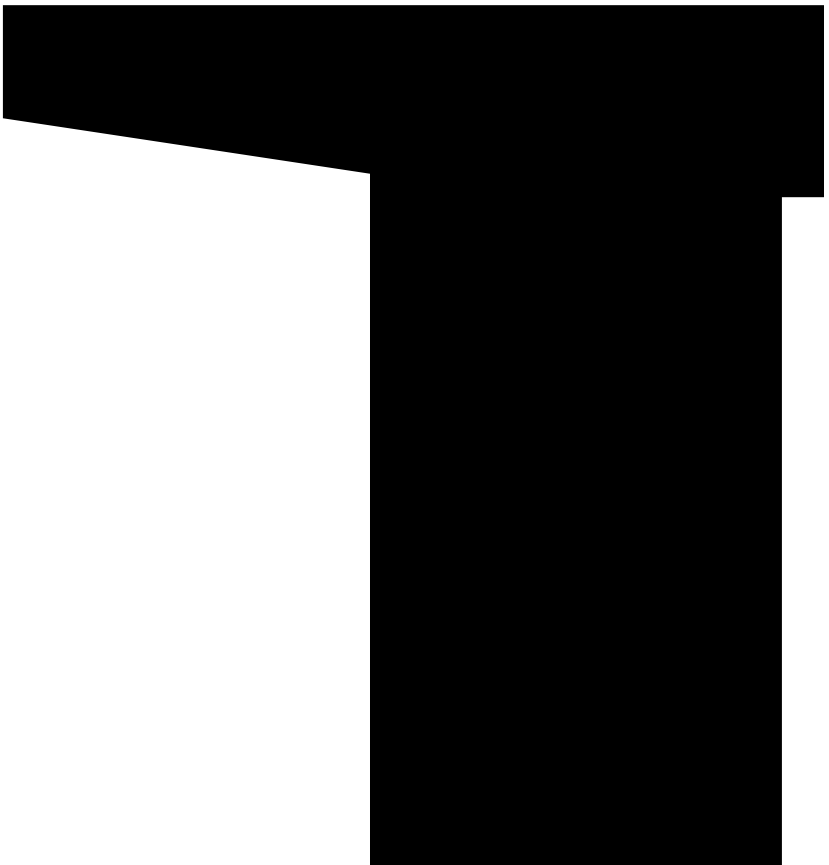
dcccx

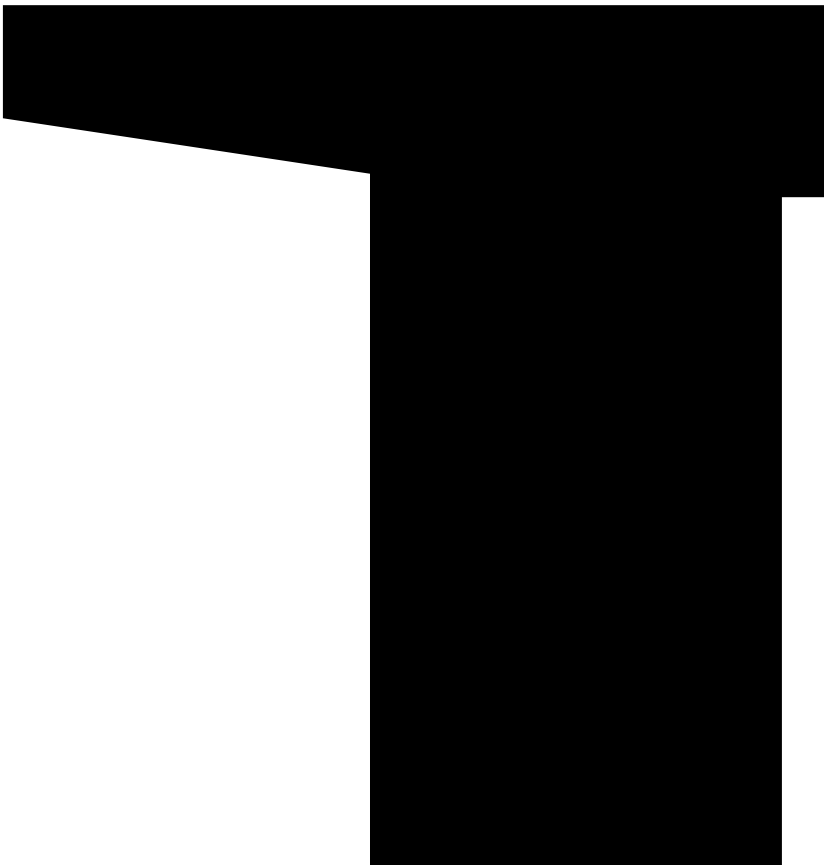


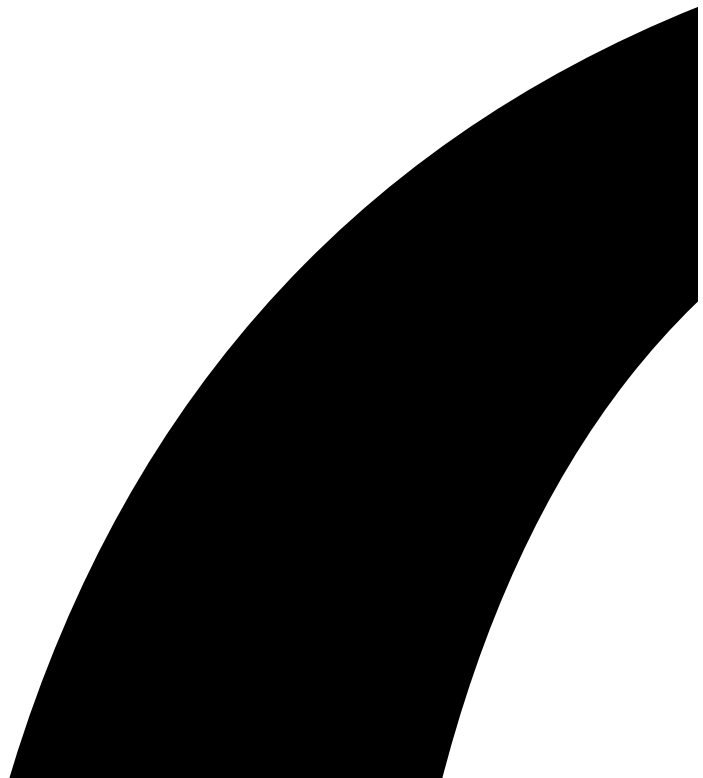


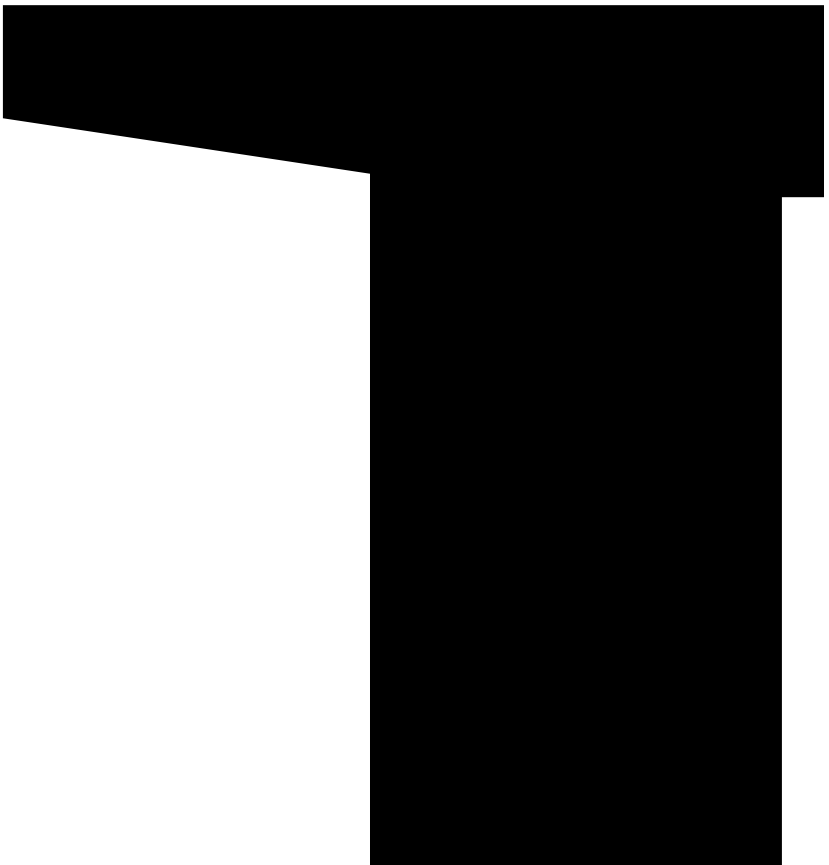


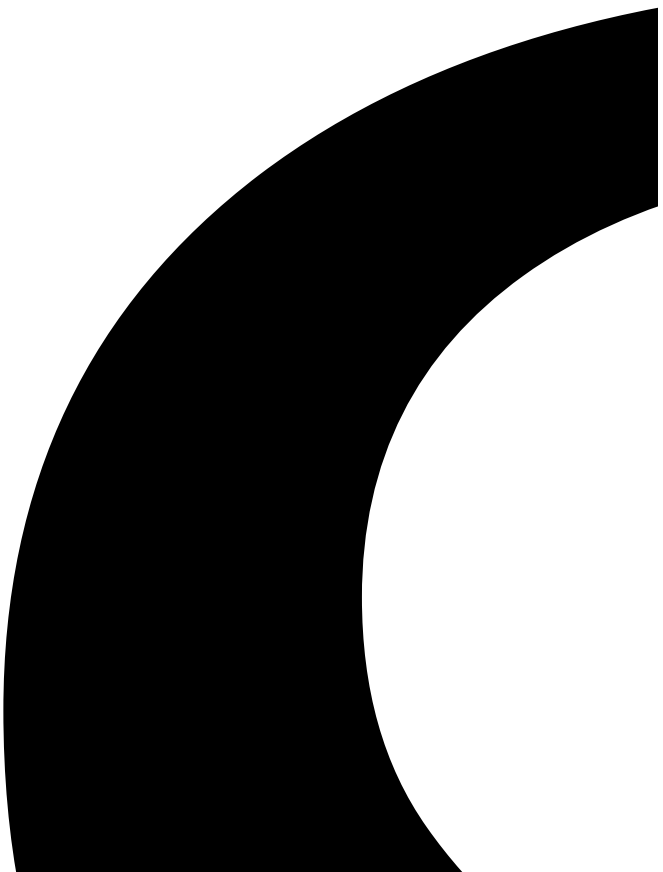




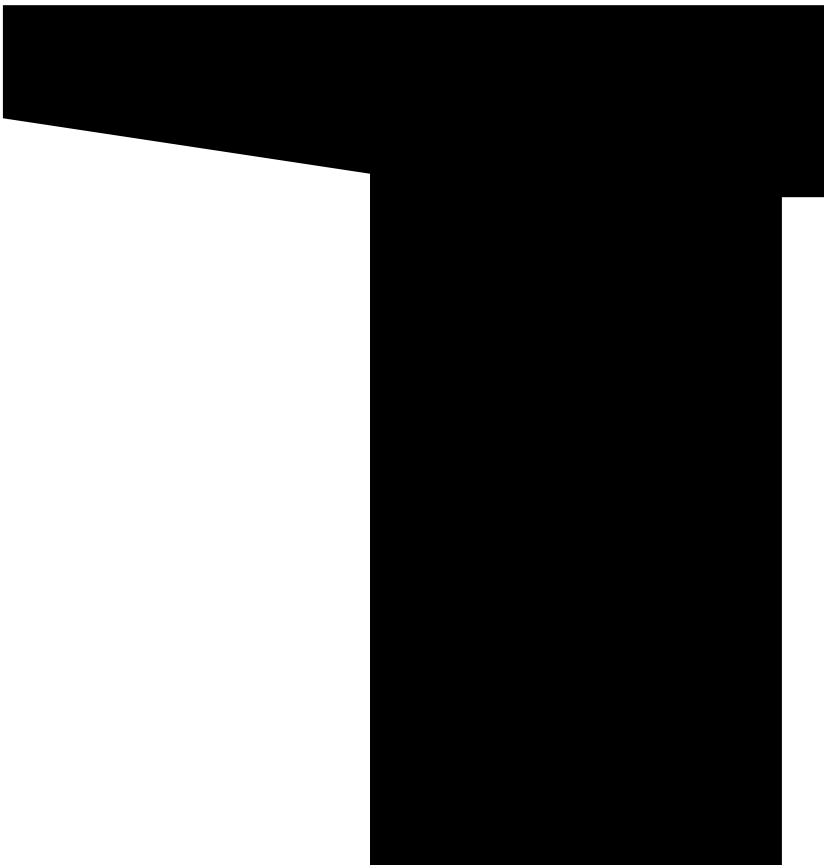


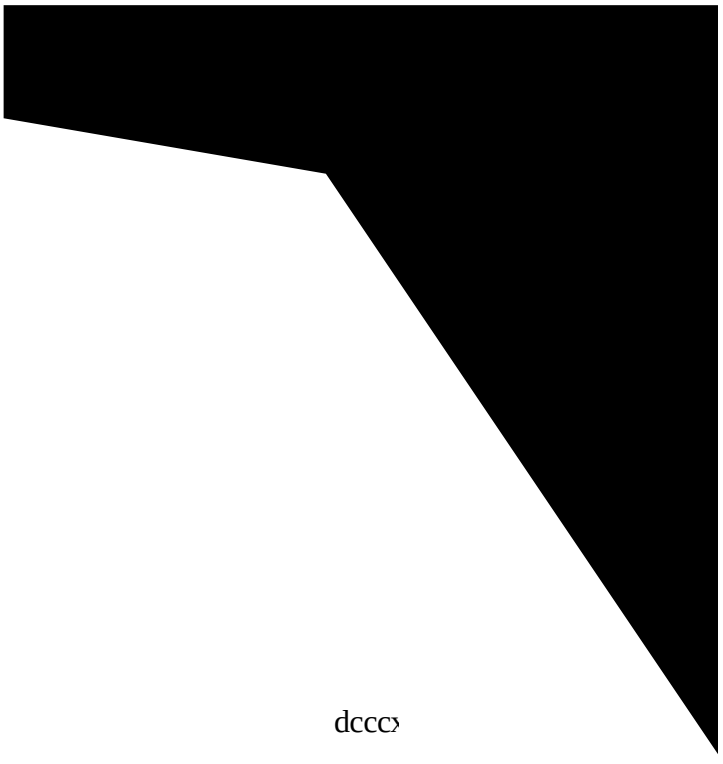




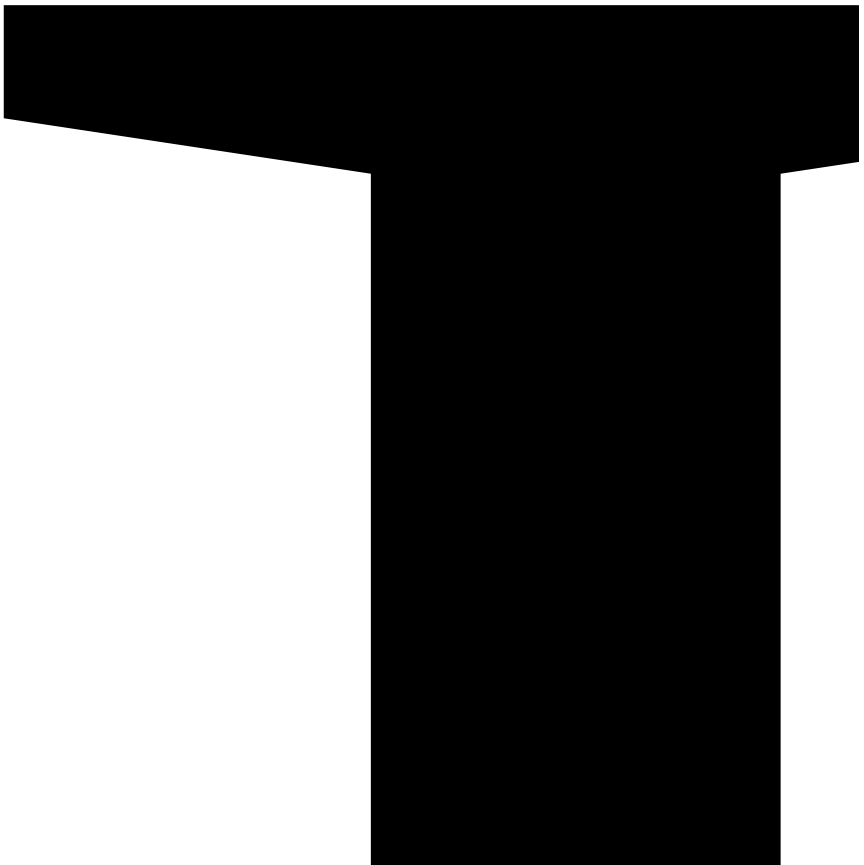


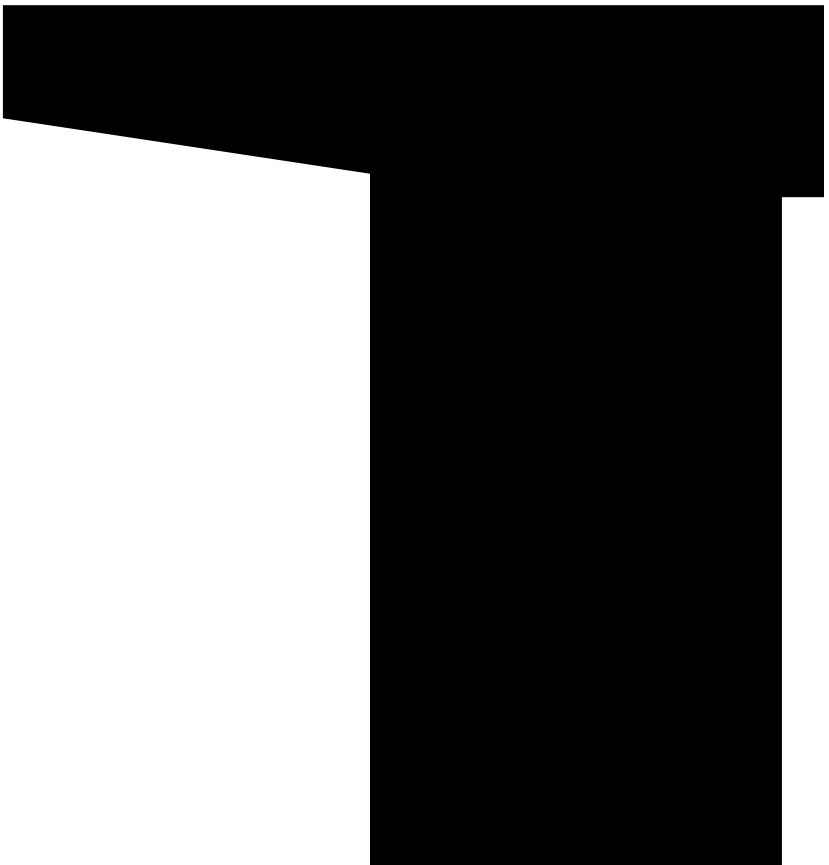
dcccx

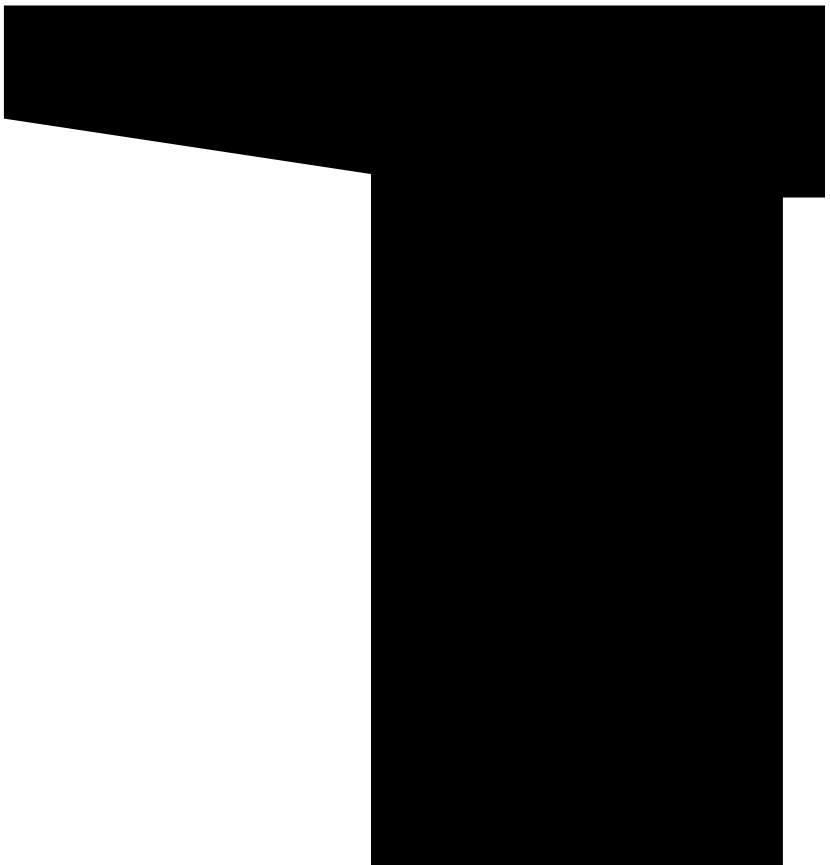




dcccx







dcccx

5 RESULT ANALYSIS AND DISCUSSION

5.1 Implementation Details 28

5.2 Testing 39

5.3 Performance Analysis 41

5.4 Discussion 42

6 CONCLUSION 43

7 FUTURE SCOPE OF THE PROJECT 44

REFERENCES 45

ANNEXURES(source code) **47**

LIST OF FIGURES



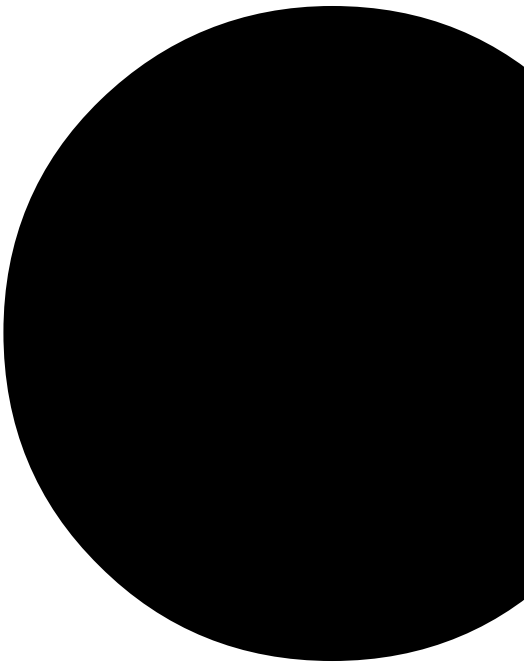
dcccx

dcccx

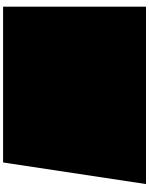
dcccx



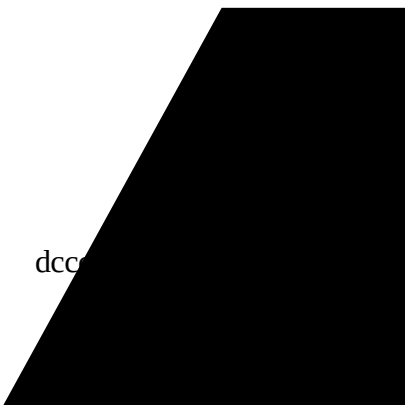
dccc›

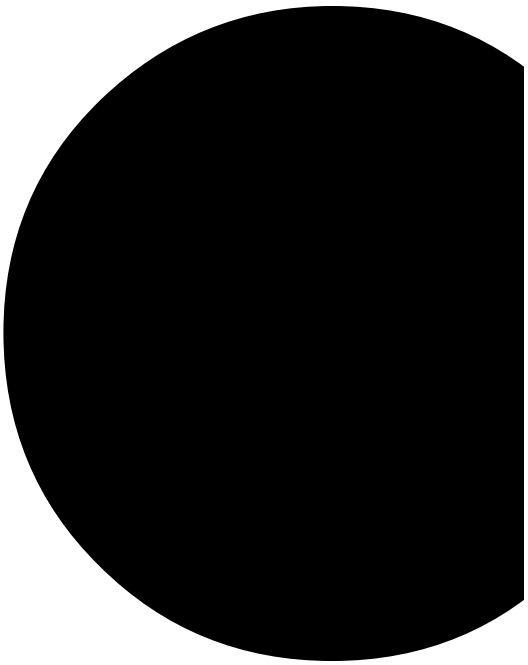


dcccx



dcccx

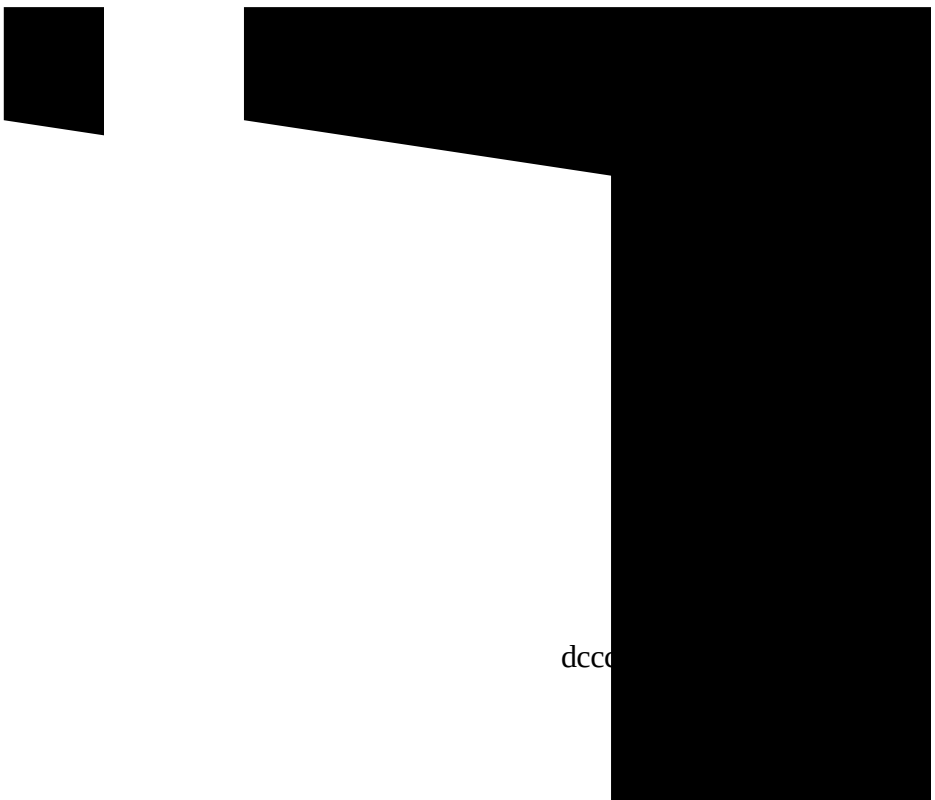


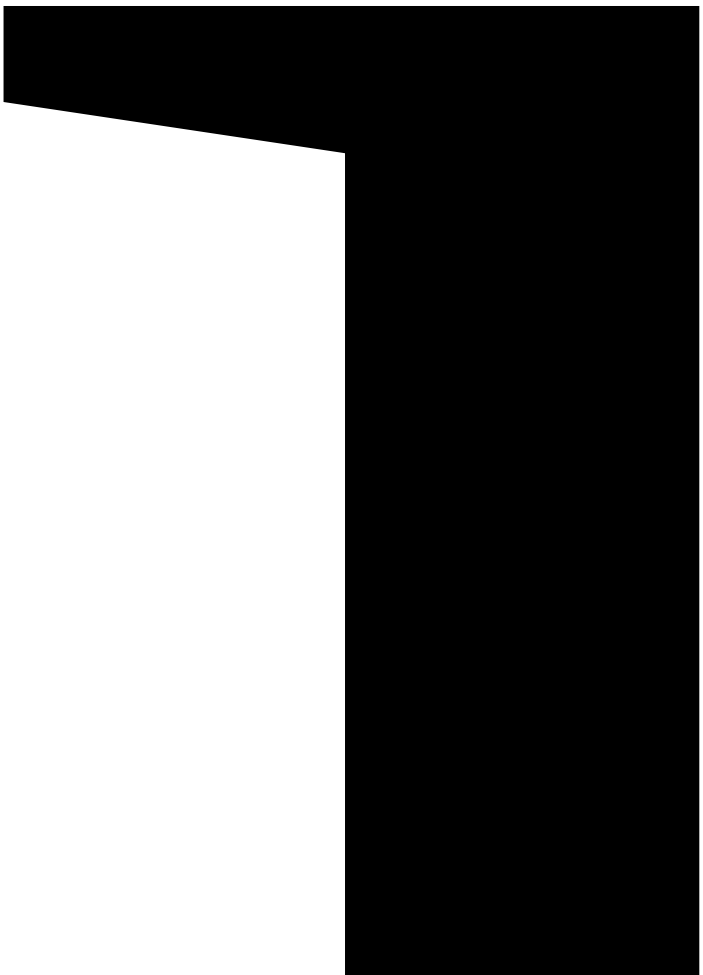


dcccx

dcccl

dcccl



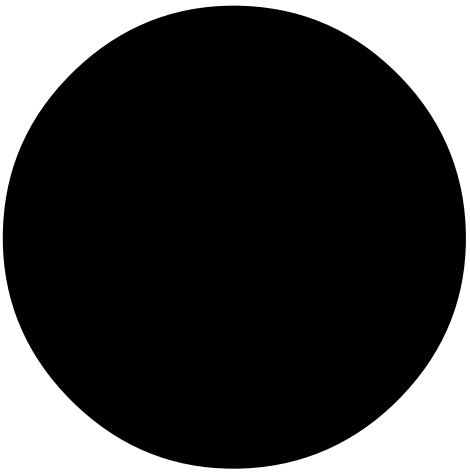


dcccl

dcccl



dcccl



dcccl

dcccl

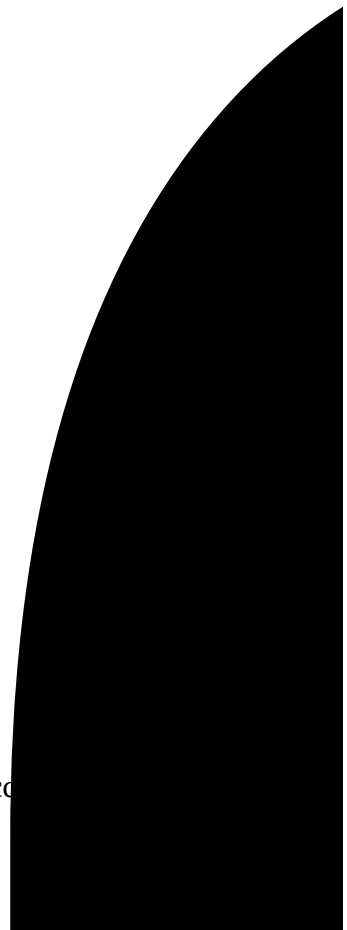
dcccl

dcccl

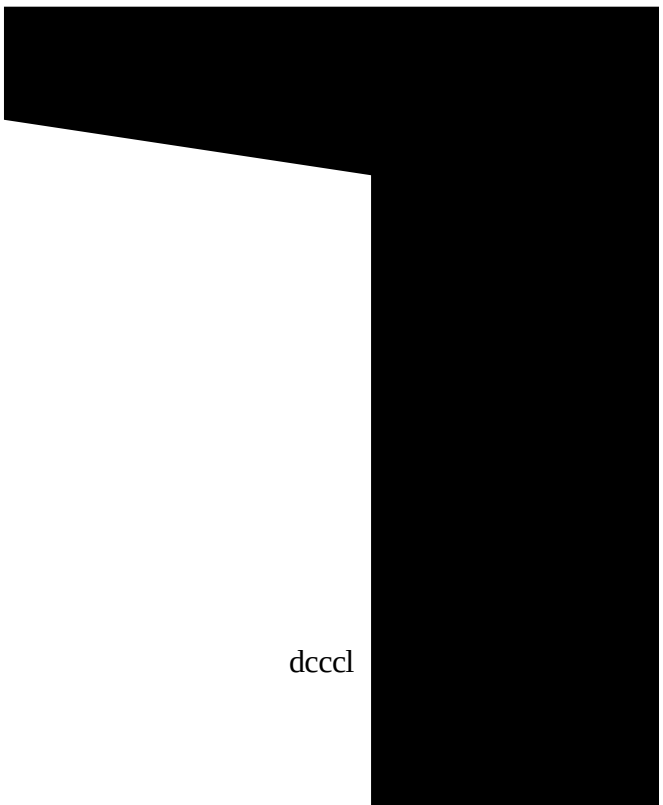
dcccl

dcccl

dcccl



dcccl



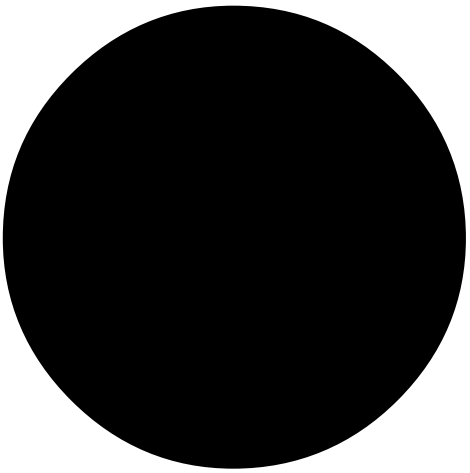
dcccl

dcccl

dcccl

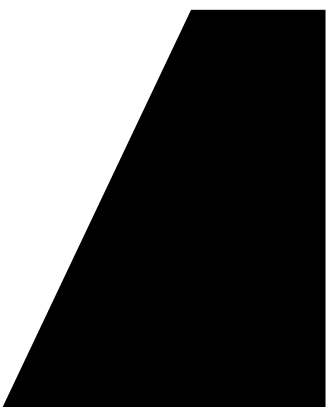
dcccl

dcccl



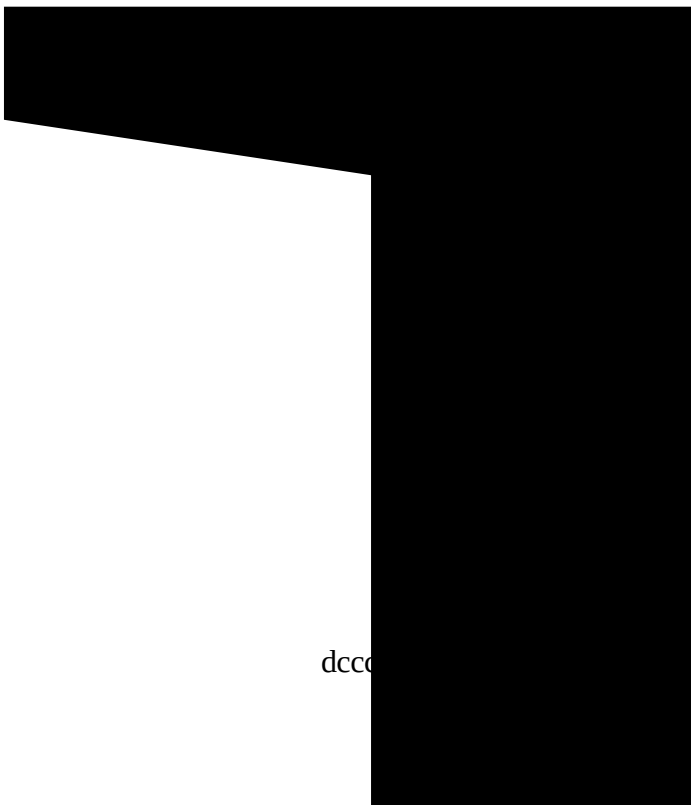
dcccl

dcccl



dcccl

dcccl



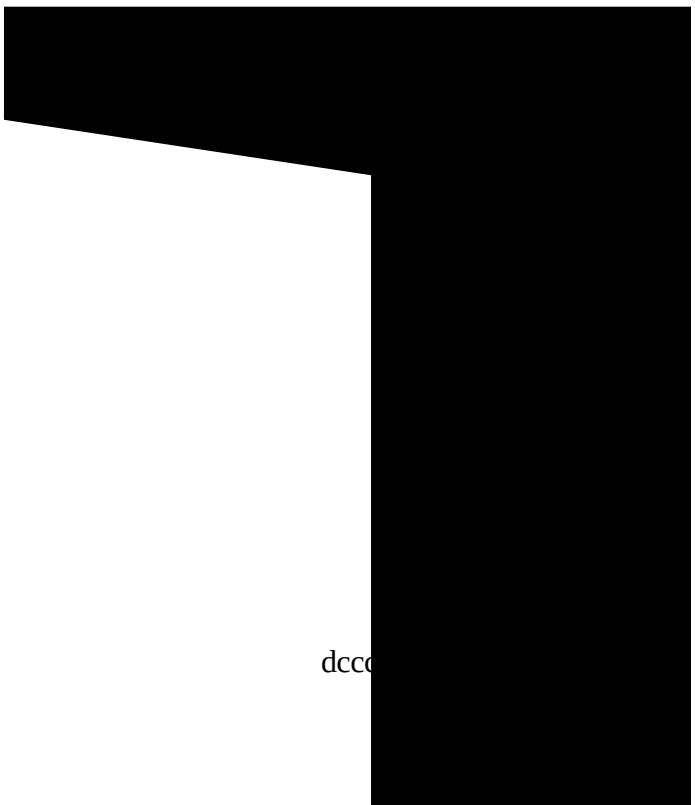
dccc

dcccl

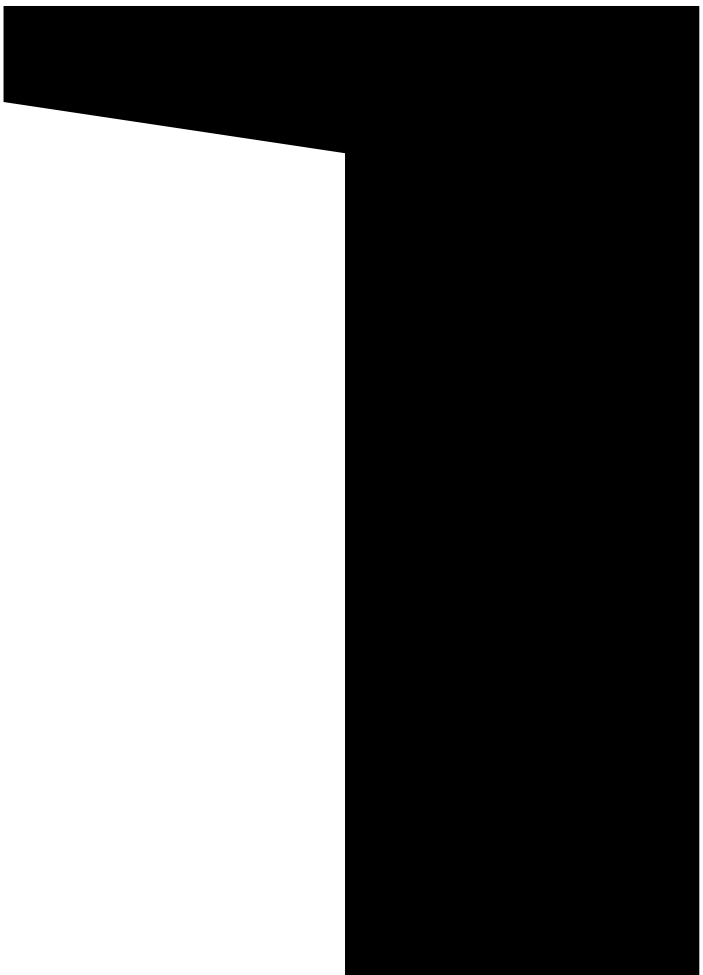
dcccl

dcccl

dcccl



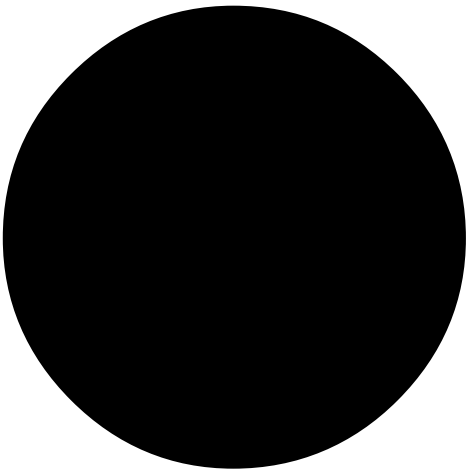
dccc



dcccl

dcccl

dcccl



dcccl

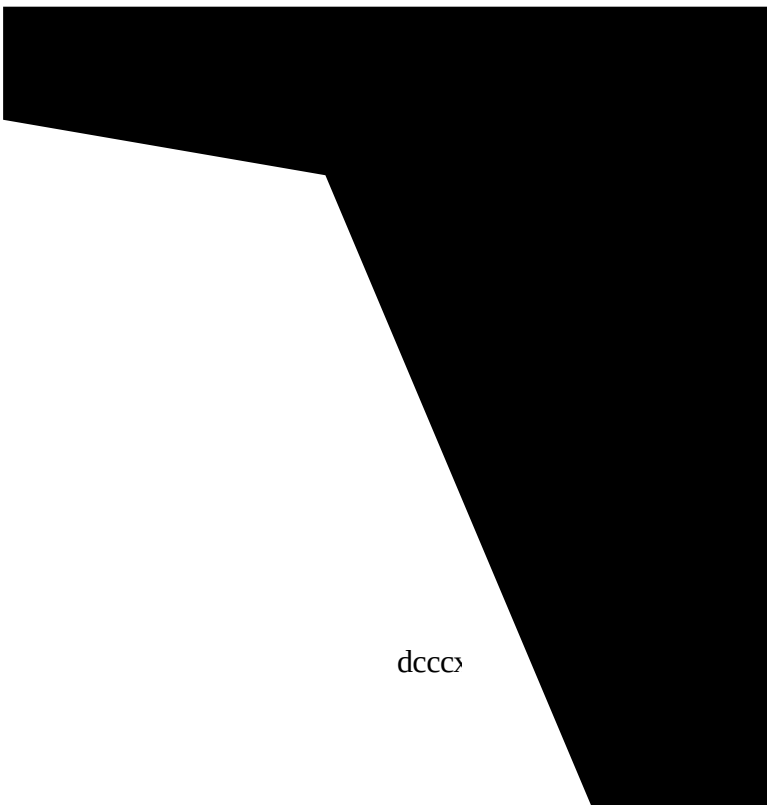
dcccl

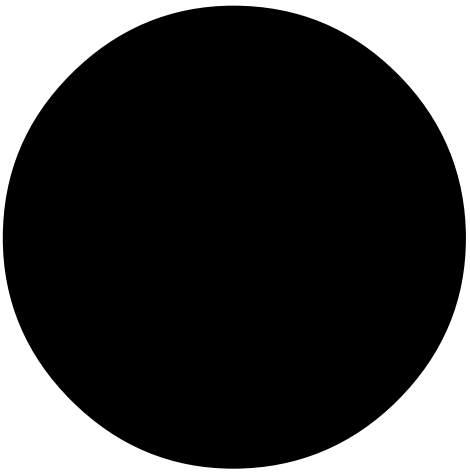
dcccl

dcccx

dcccx

dcccx


dcccx

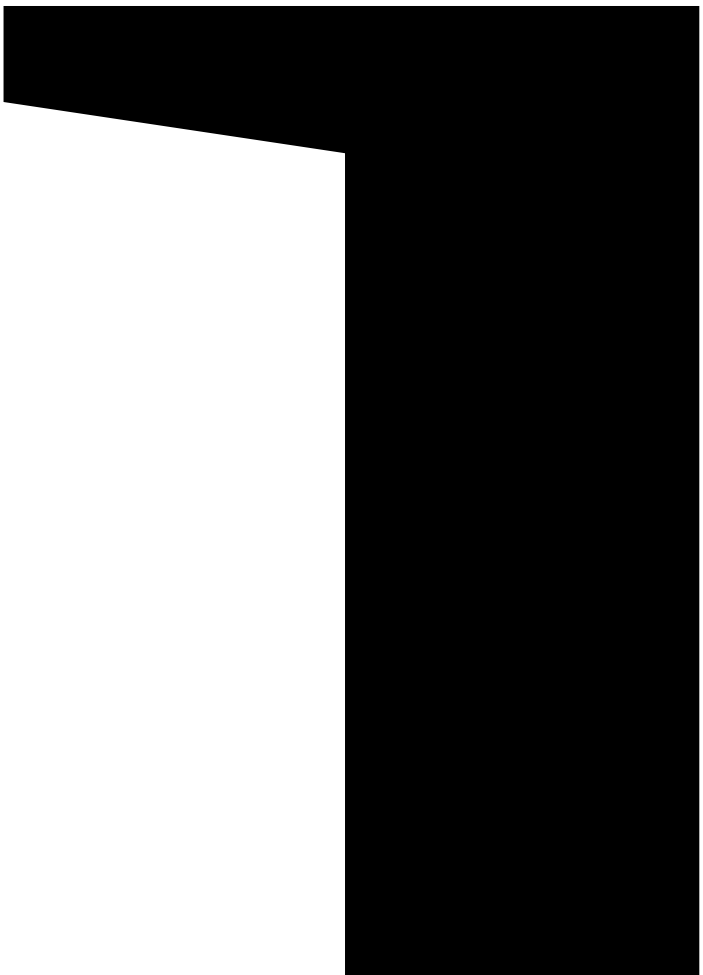


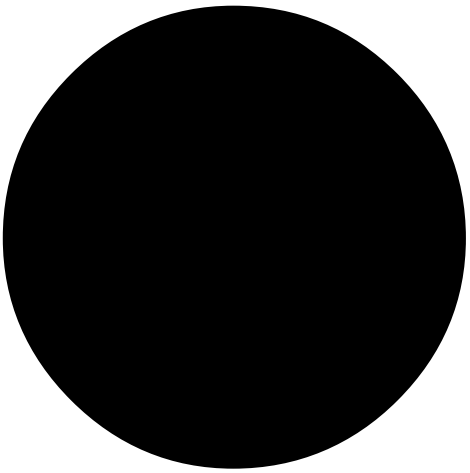
dcccx

dcccx

dcccx

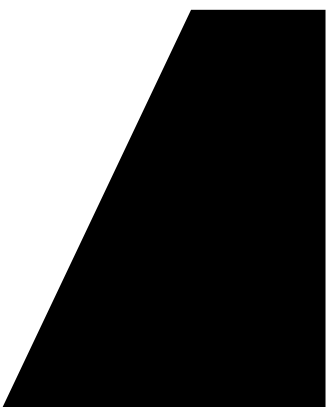
dcccx

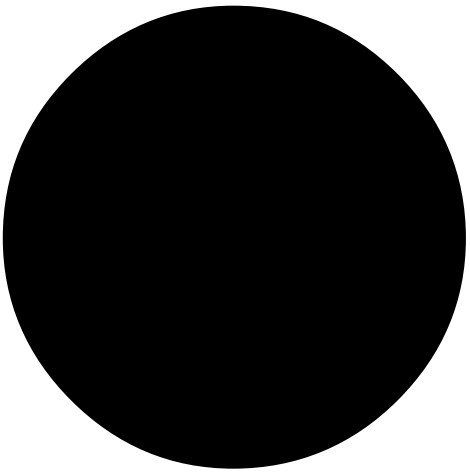




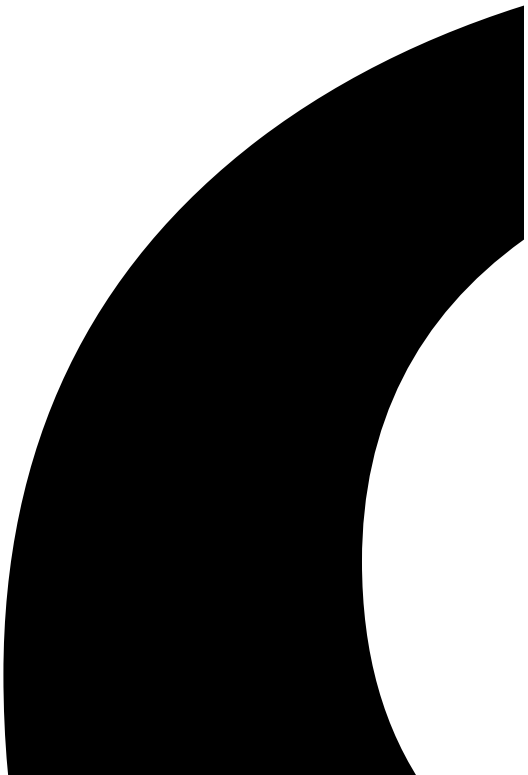
dcccx

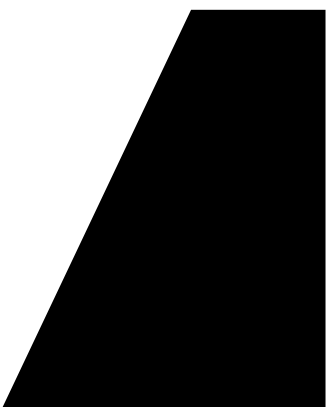
cm





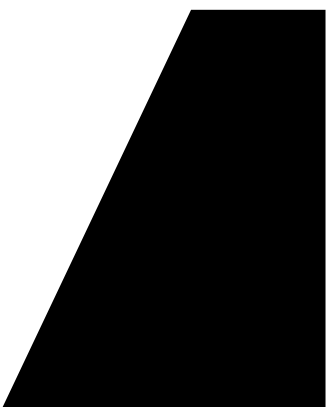
cmv



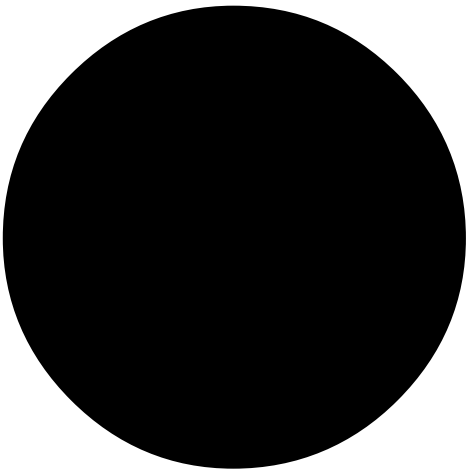


cmx

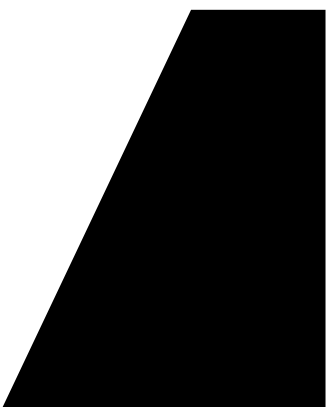


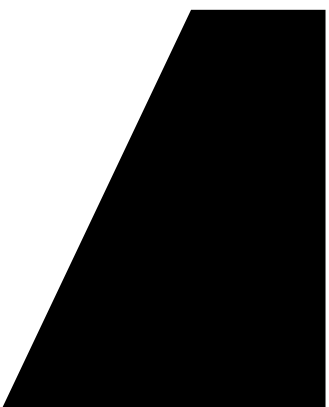


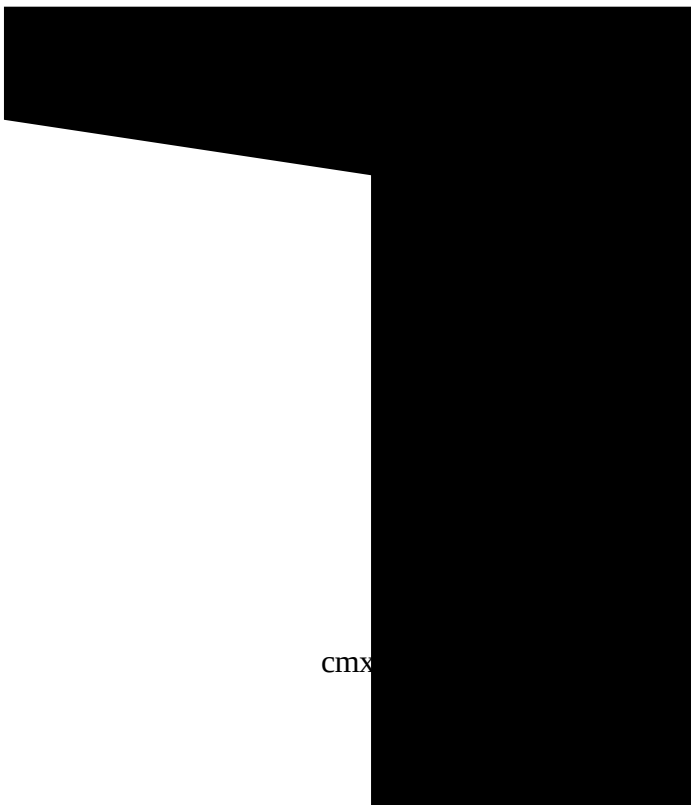




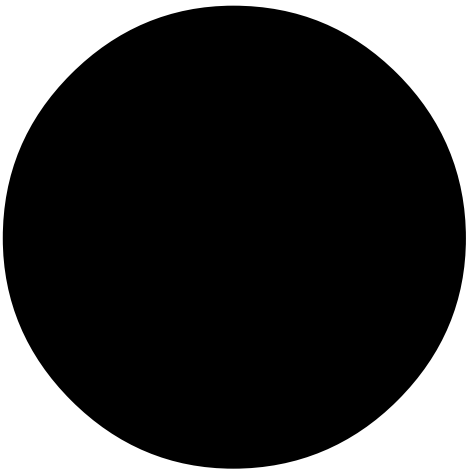
cmxx



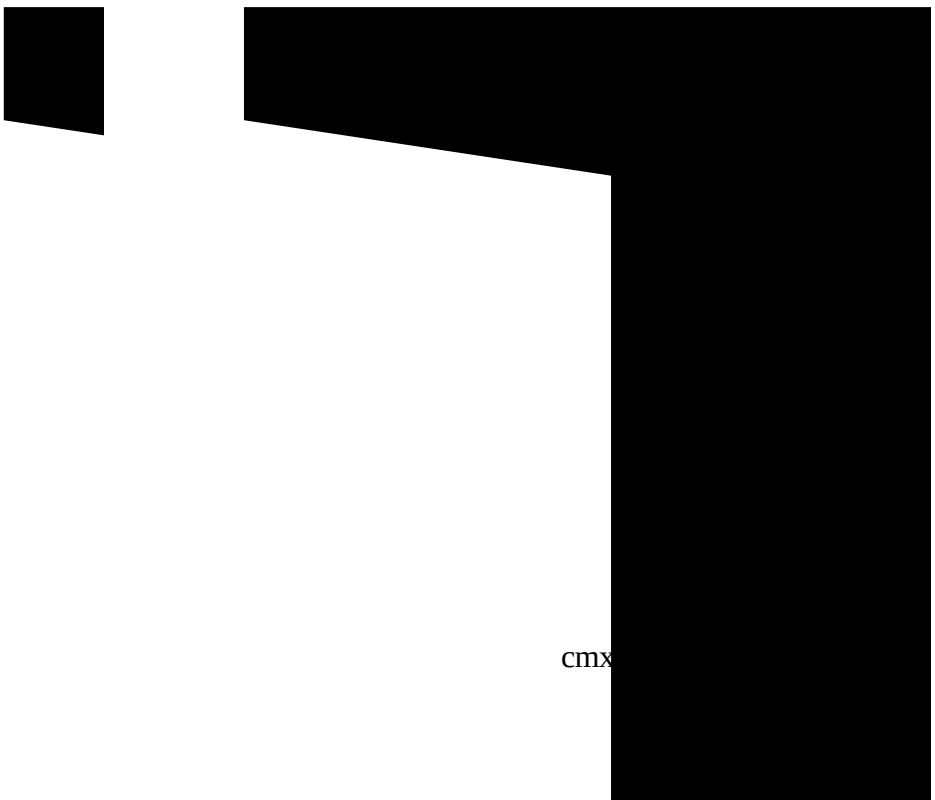




cmx

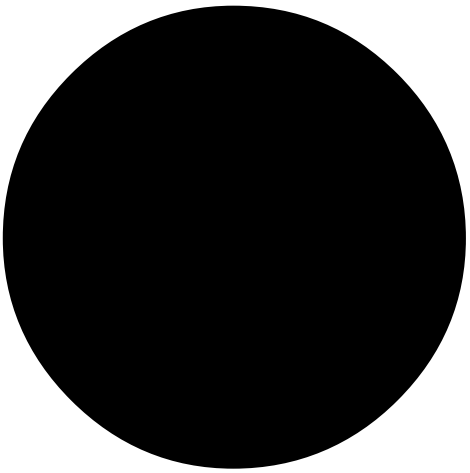


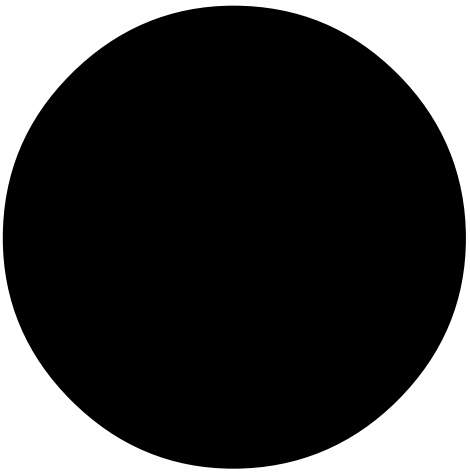
cmxx



cmx

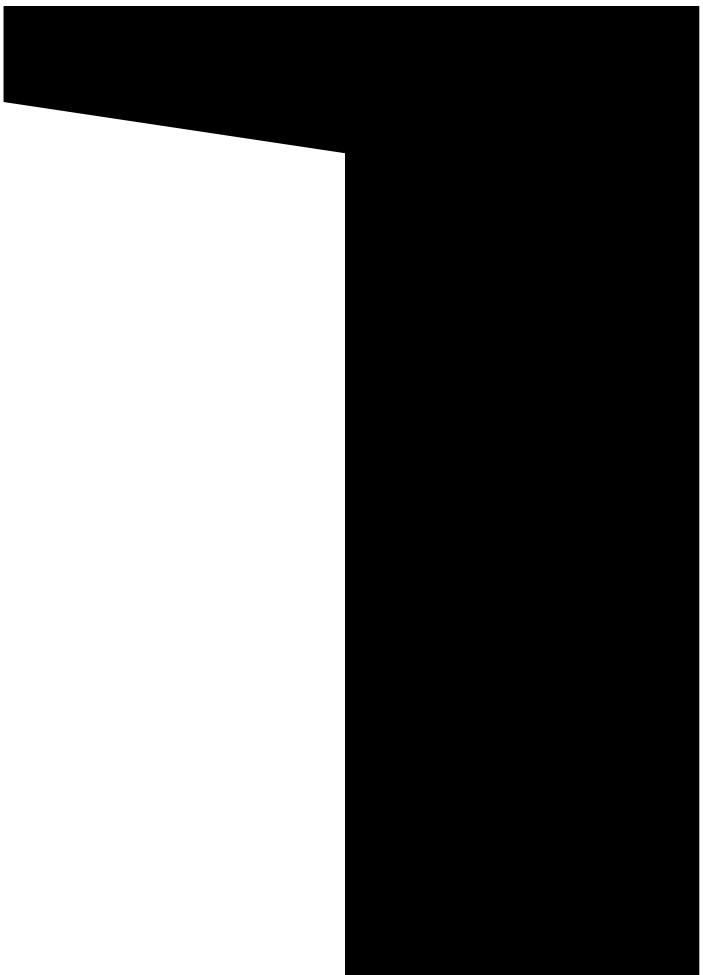
cmxl

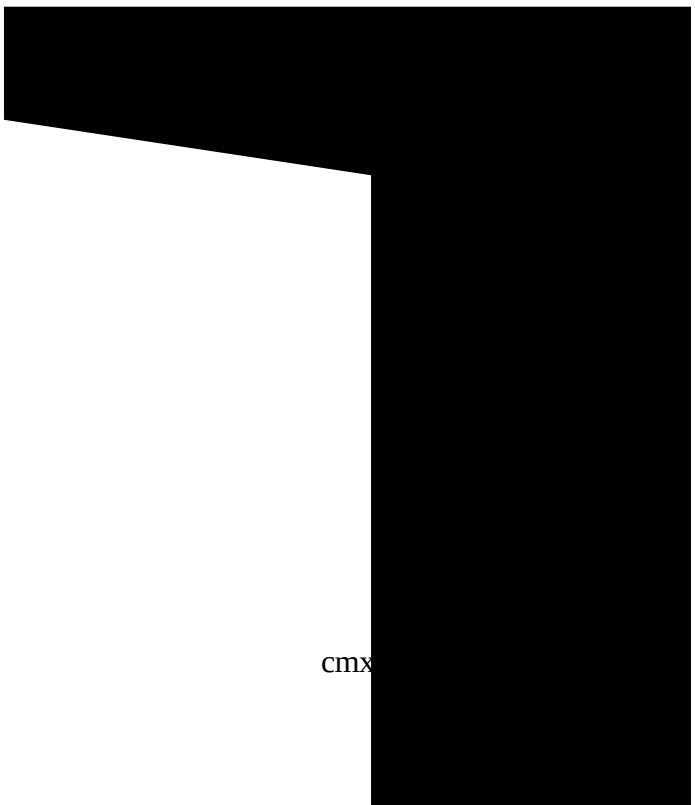




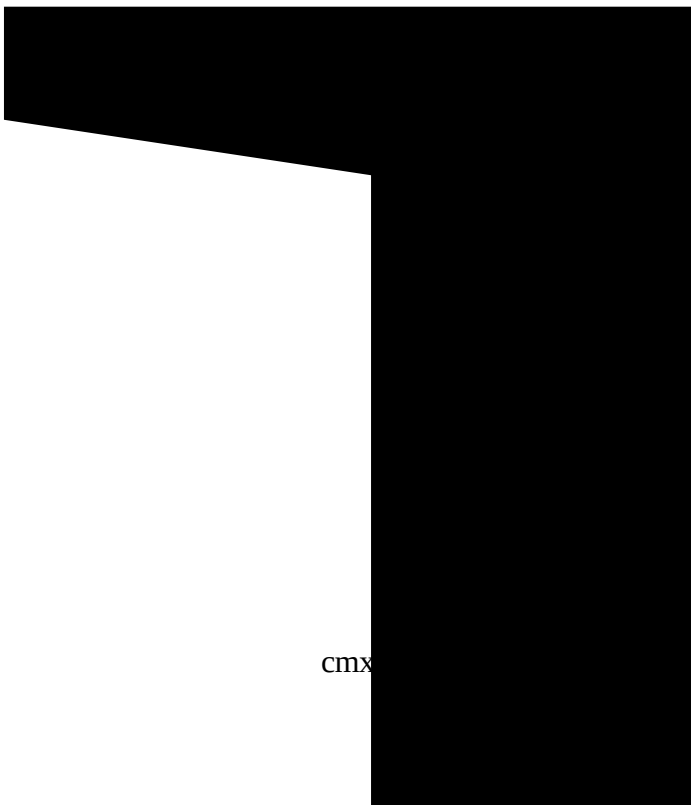
cmxl

cmxl

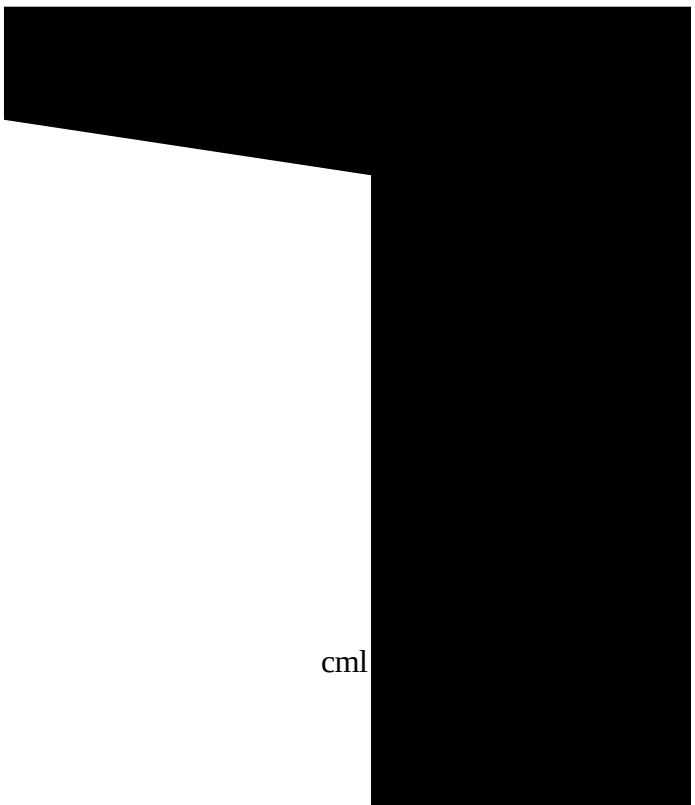




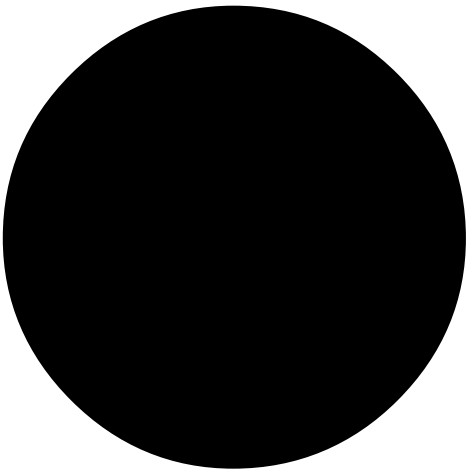
cmx



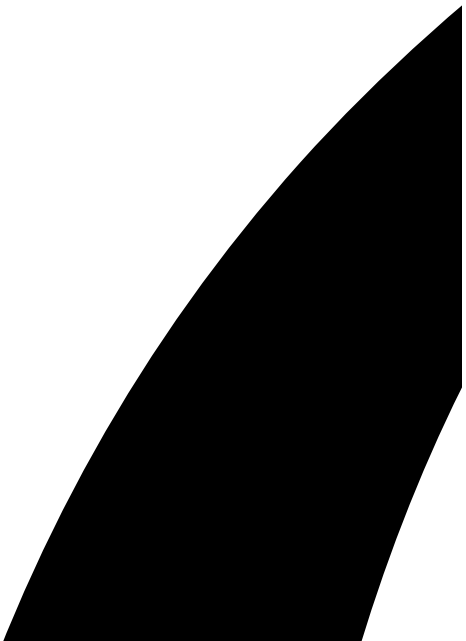
cmx

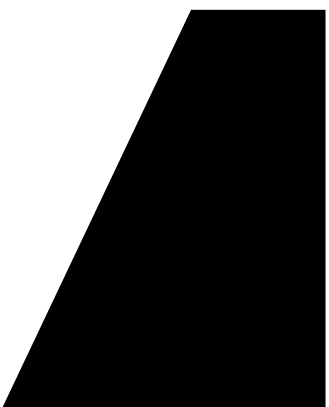


cml

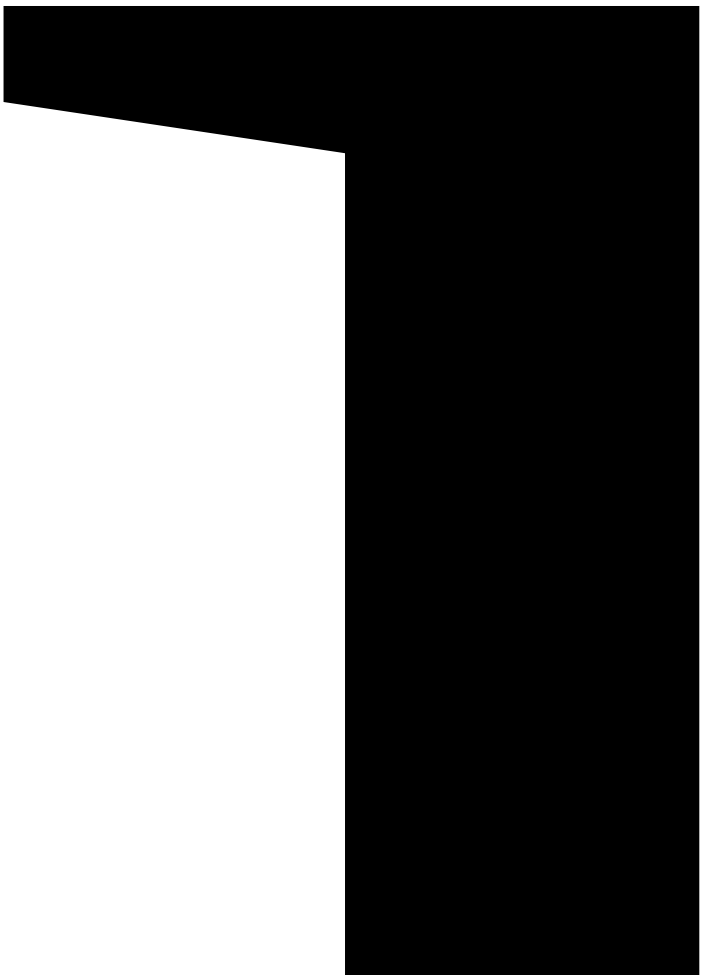


cmlv









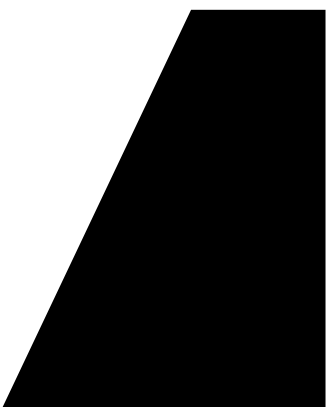
cmlx

cmlx

cmlx



cmlx



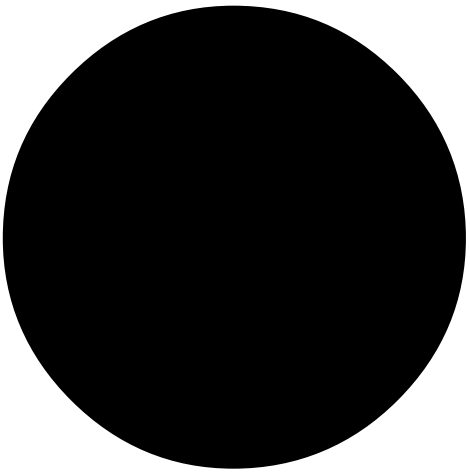


cmlx

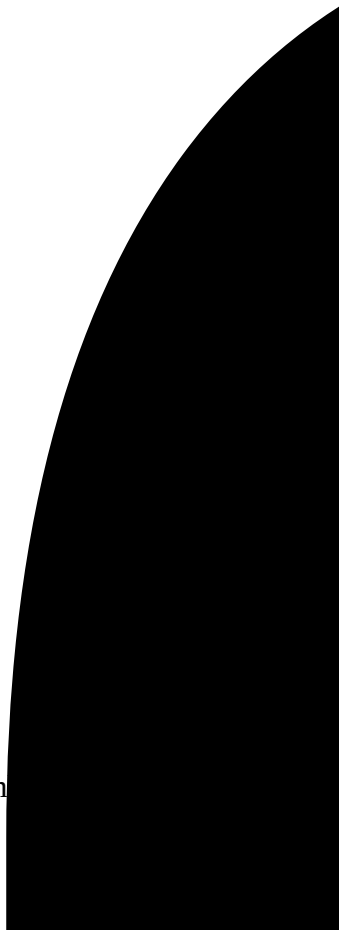
cmlx

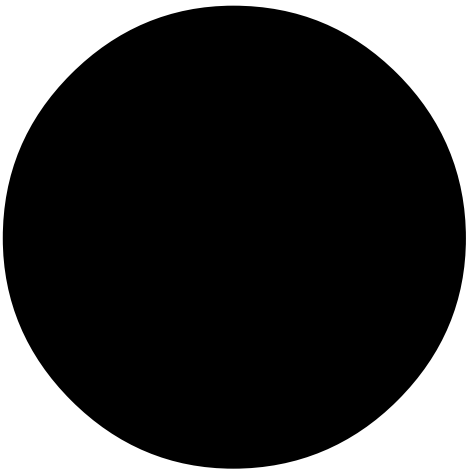
cmlx

cmlx



cmlx:

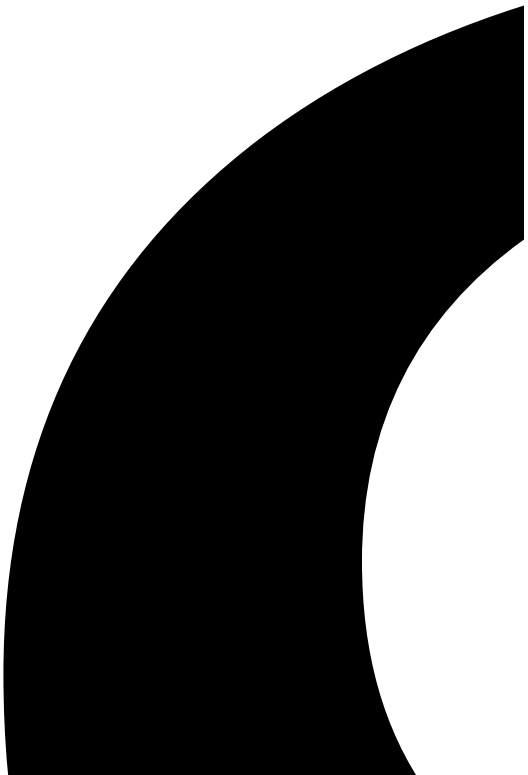

cm



cmlx:

cmlx

cmlx



cmlx

cmlx

cmlx



cmlx

cmlx

cmxc

cmxc

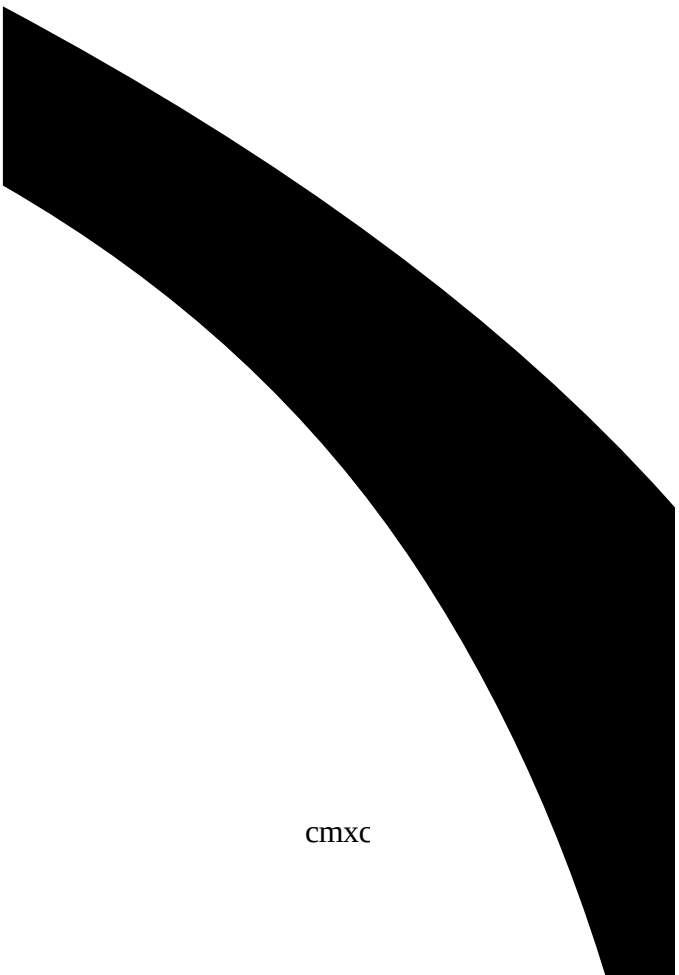
cmxc



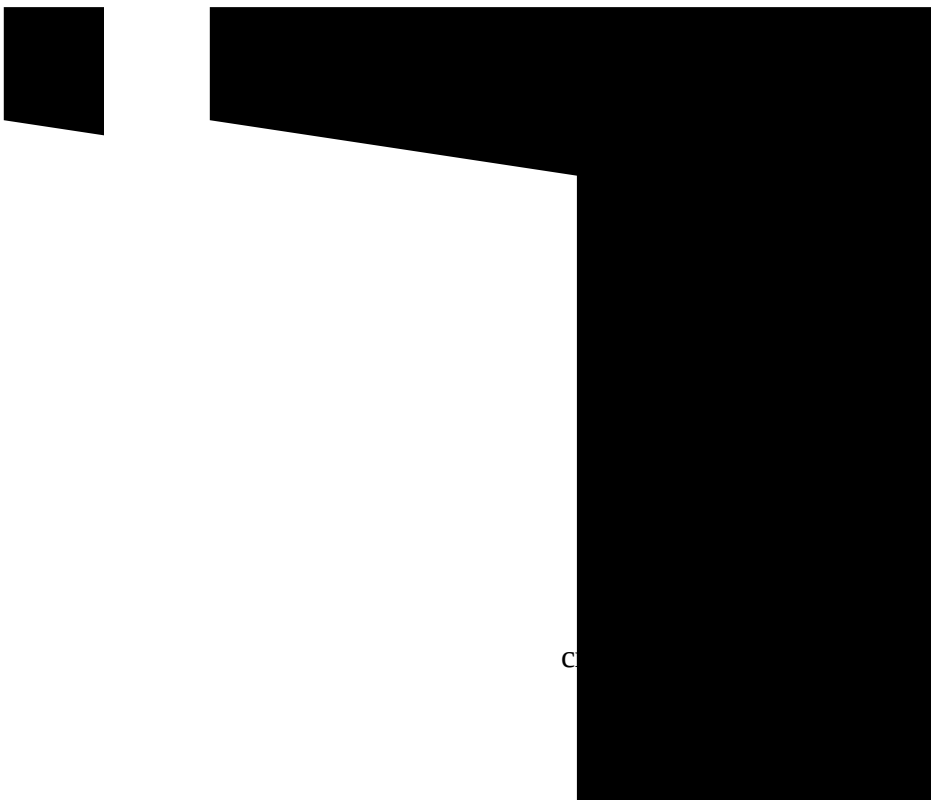
cmxc

cmxc

cmxc

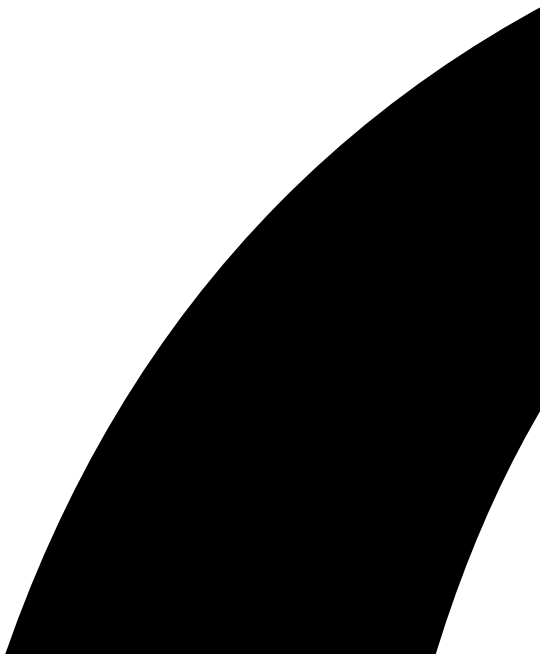


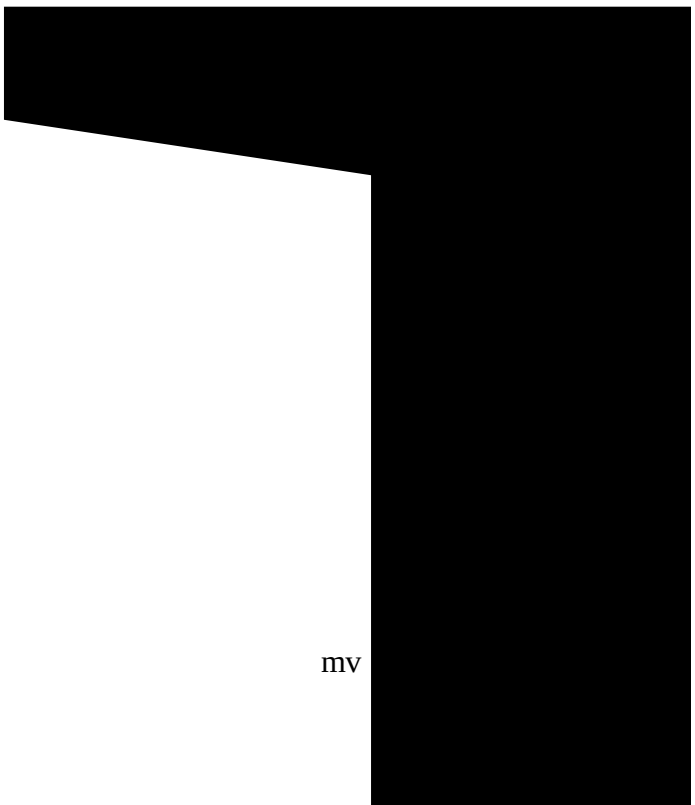
cmxc

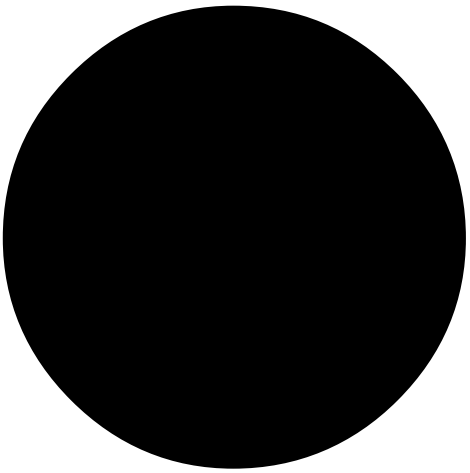


cmxc

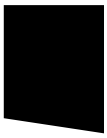
m

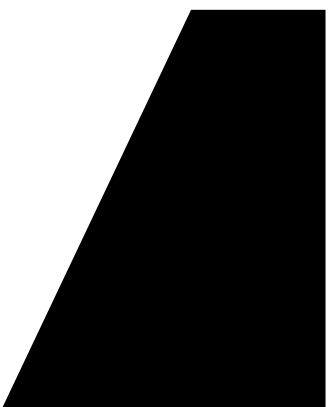


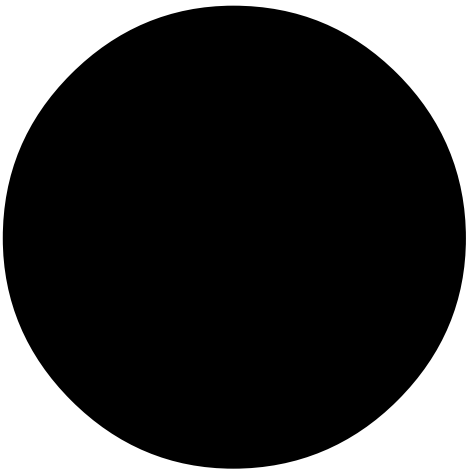

mv

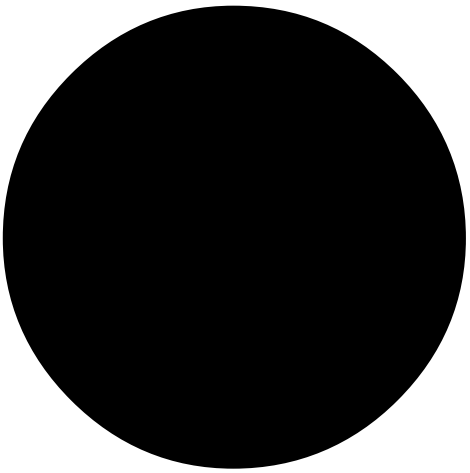


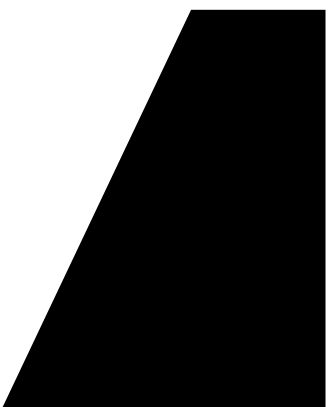
mix

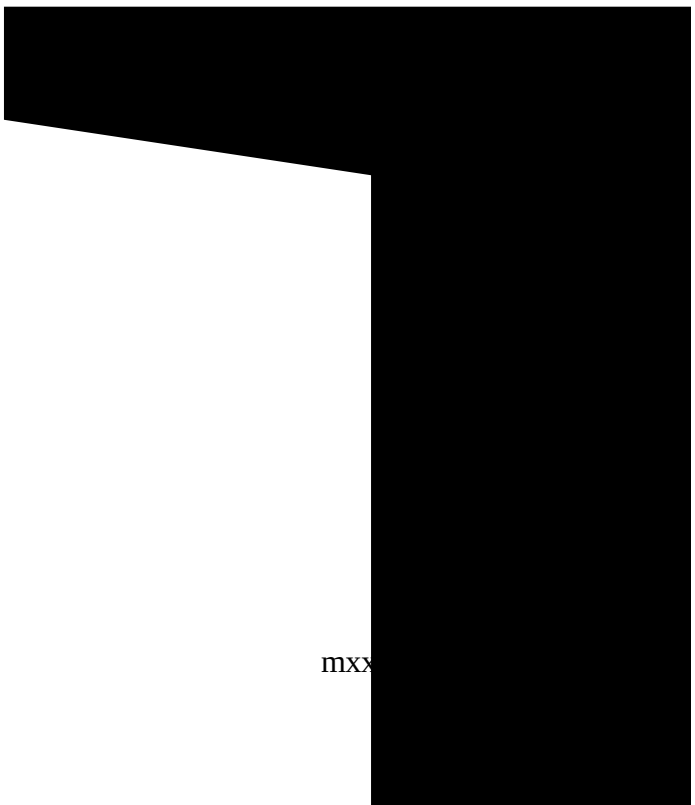




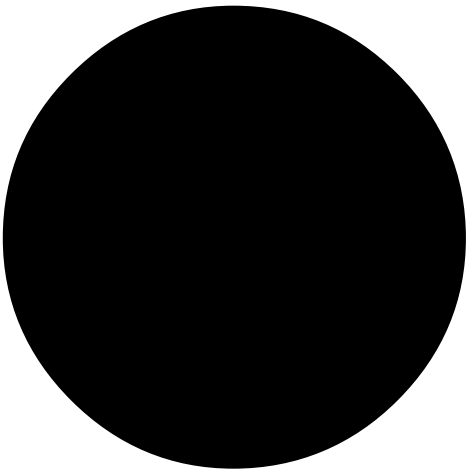


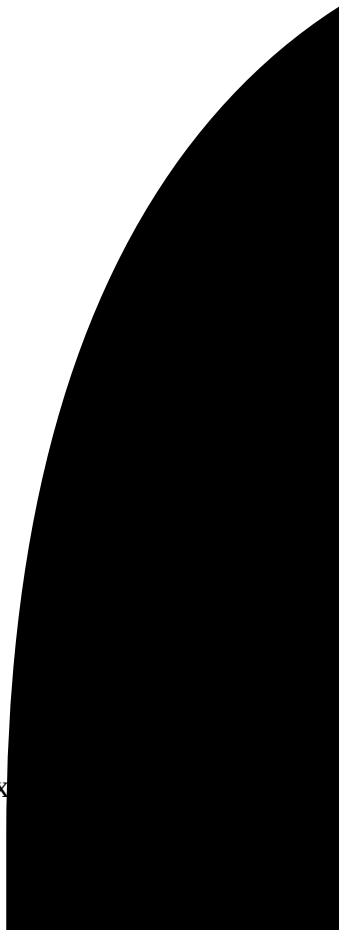




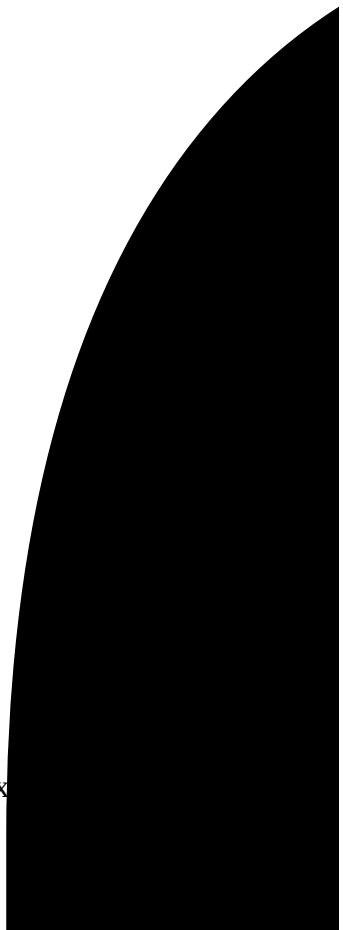


mxx

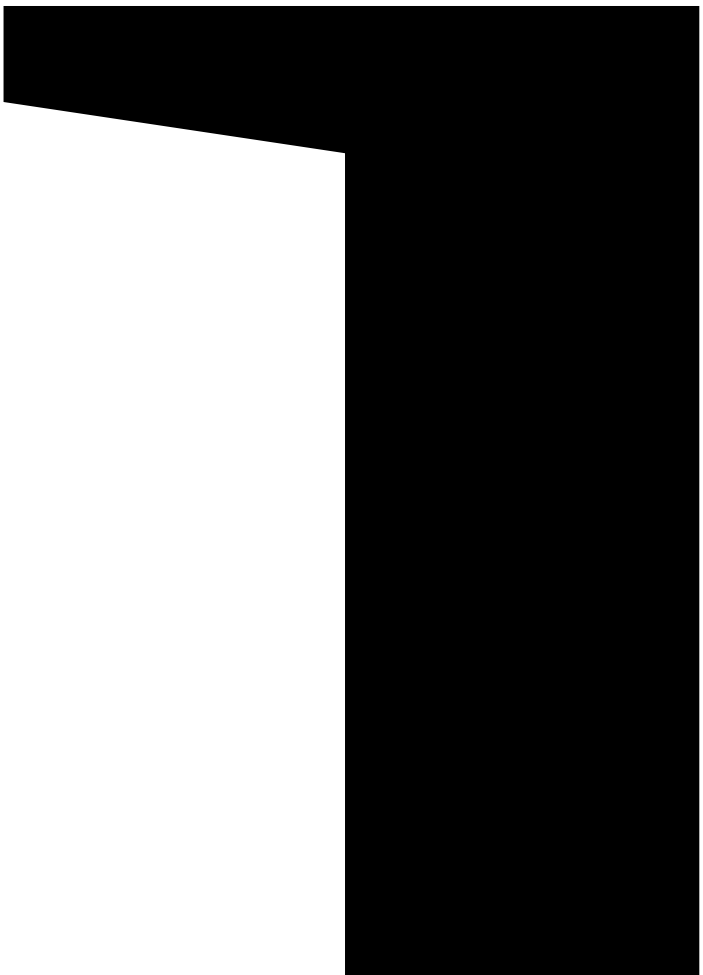


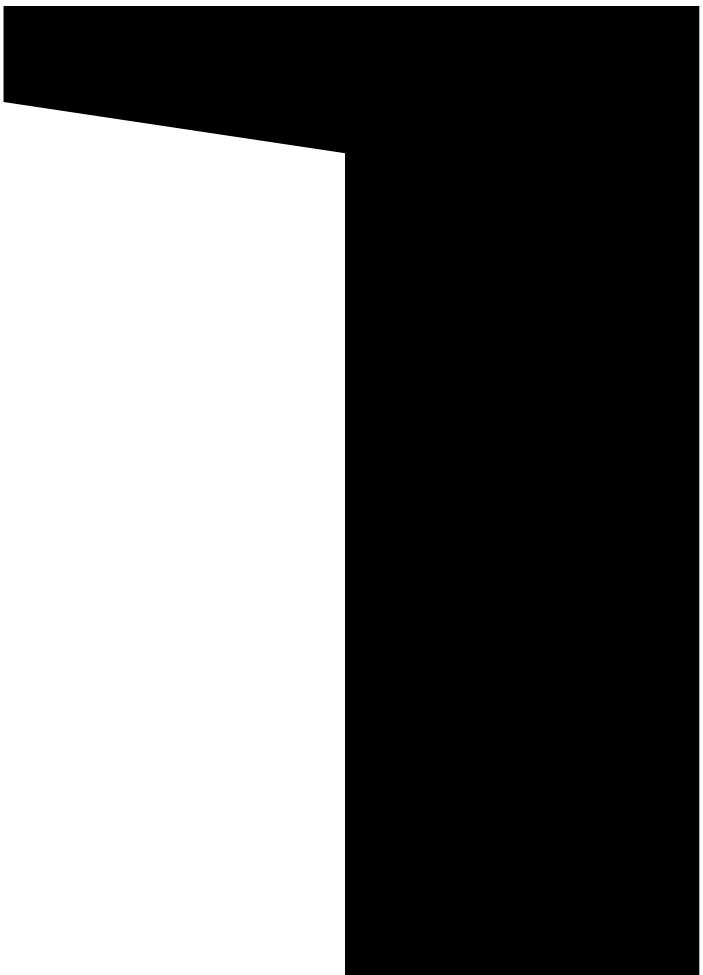


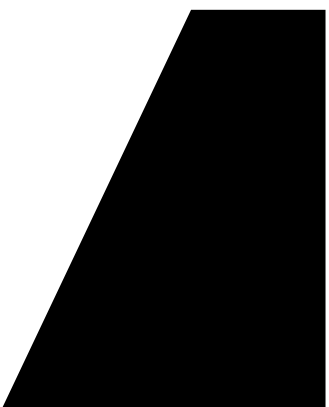
mx

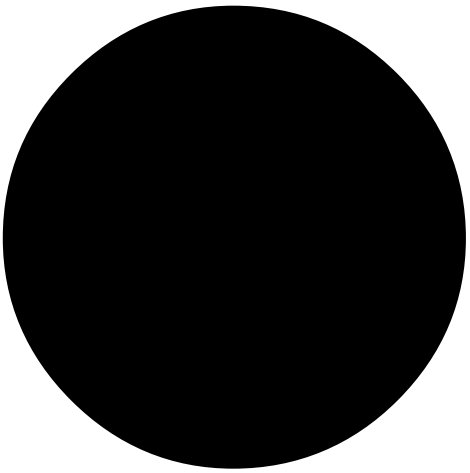


mx

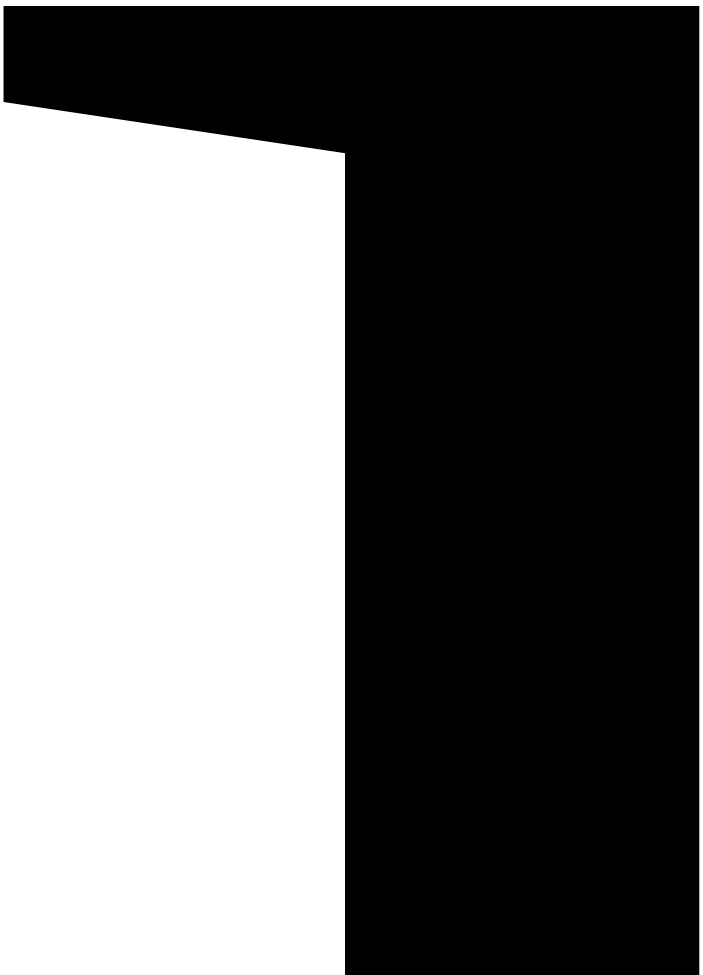


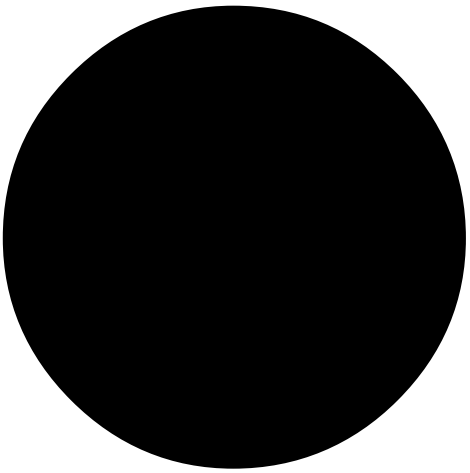


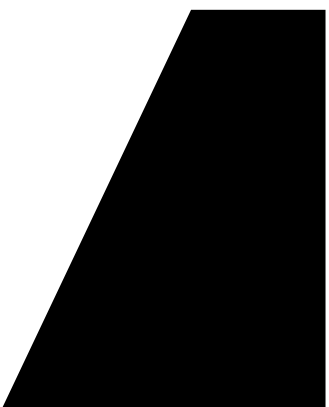


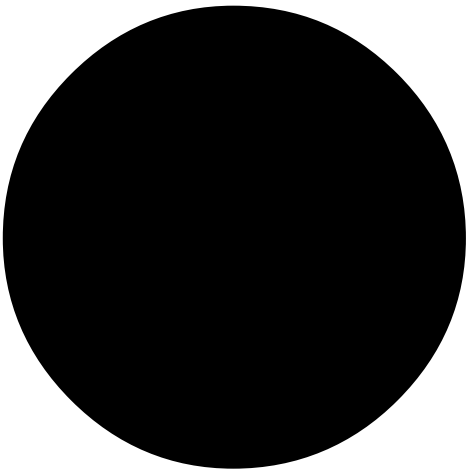


ml



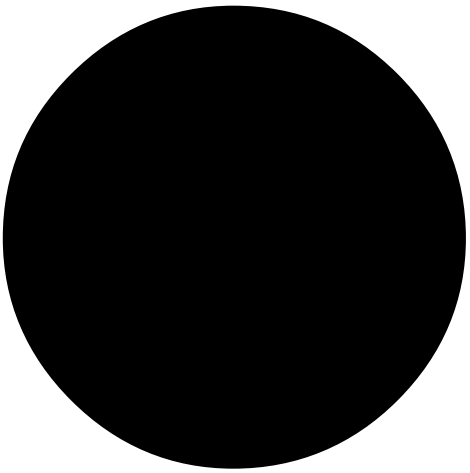




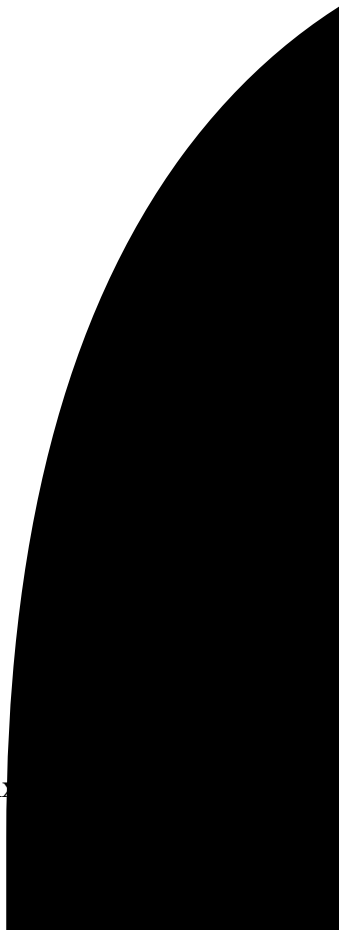


mlx

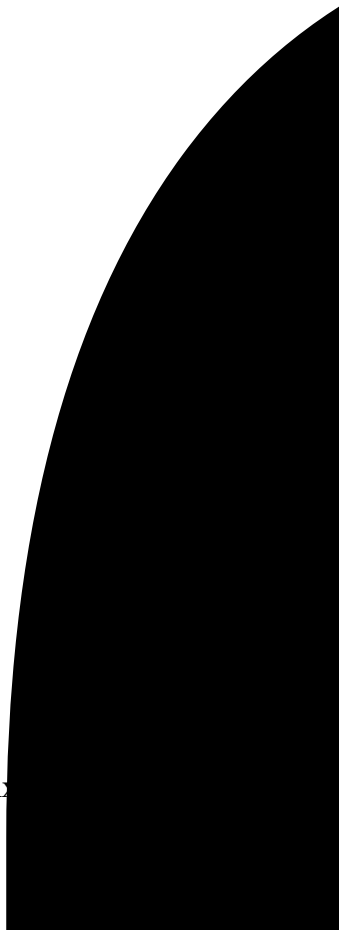




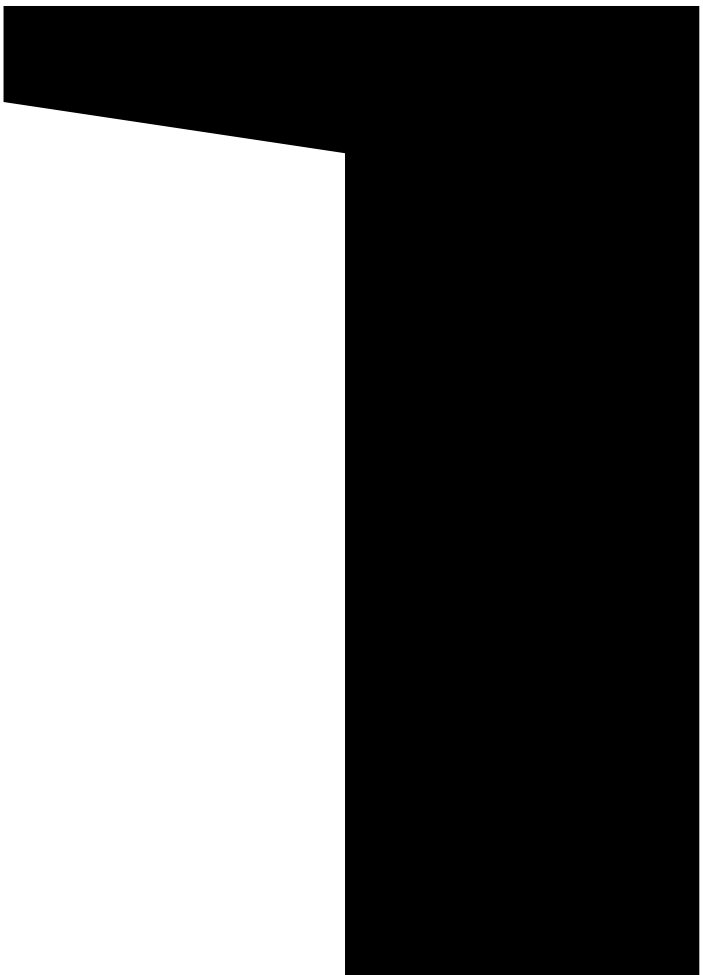
mlxx

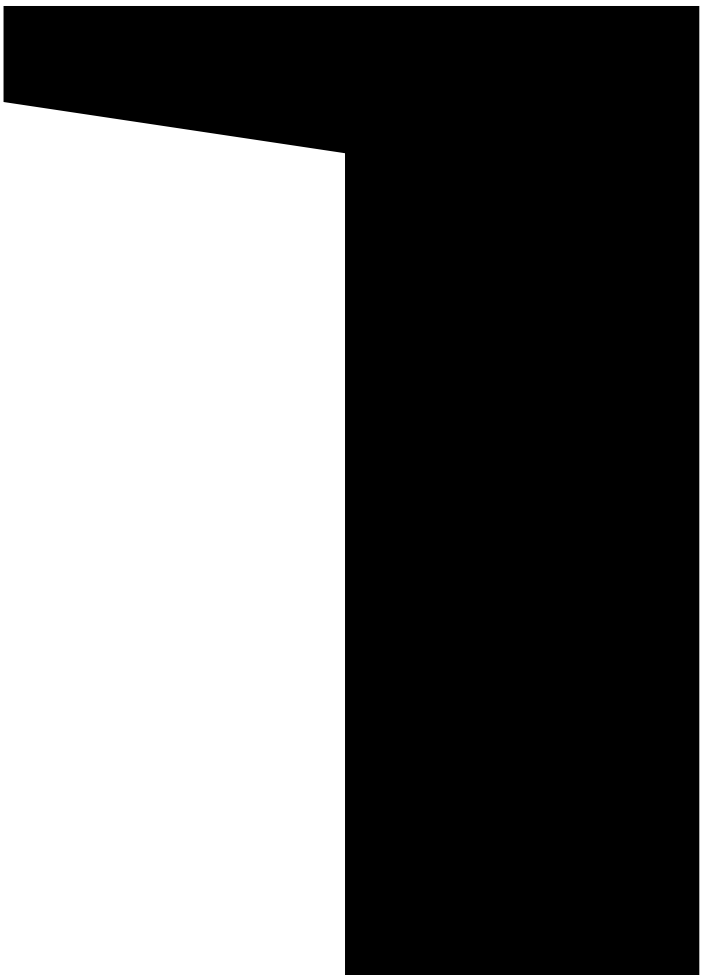


mlx

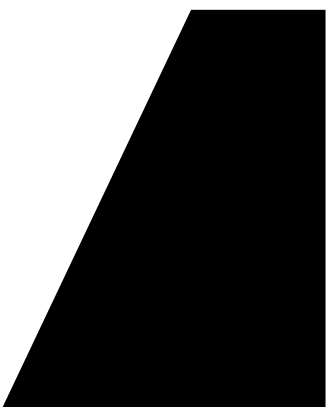


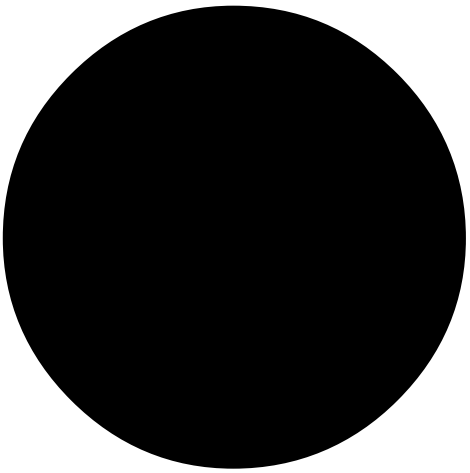
mlx





mxci

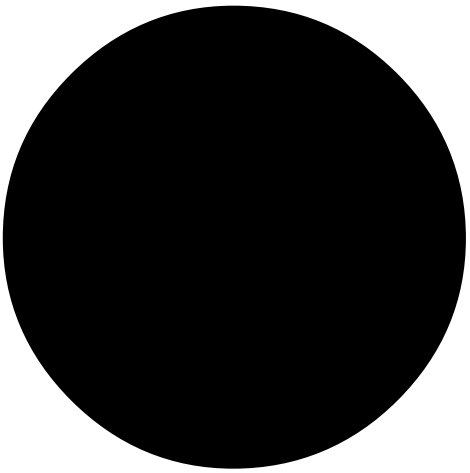




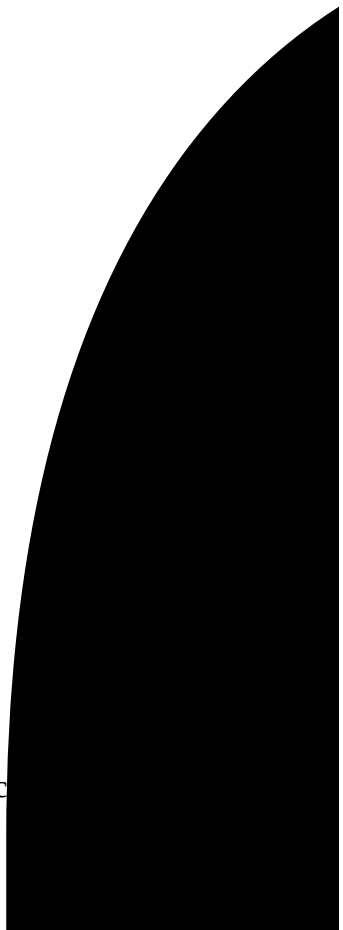
mxcv

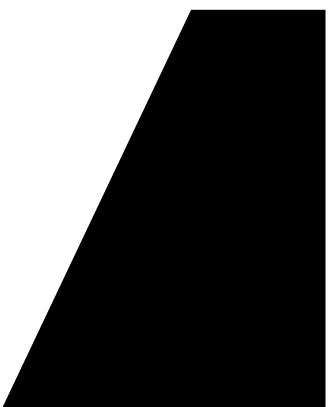
mc

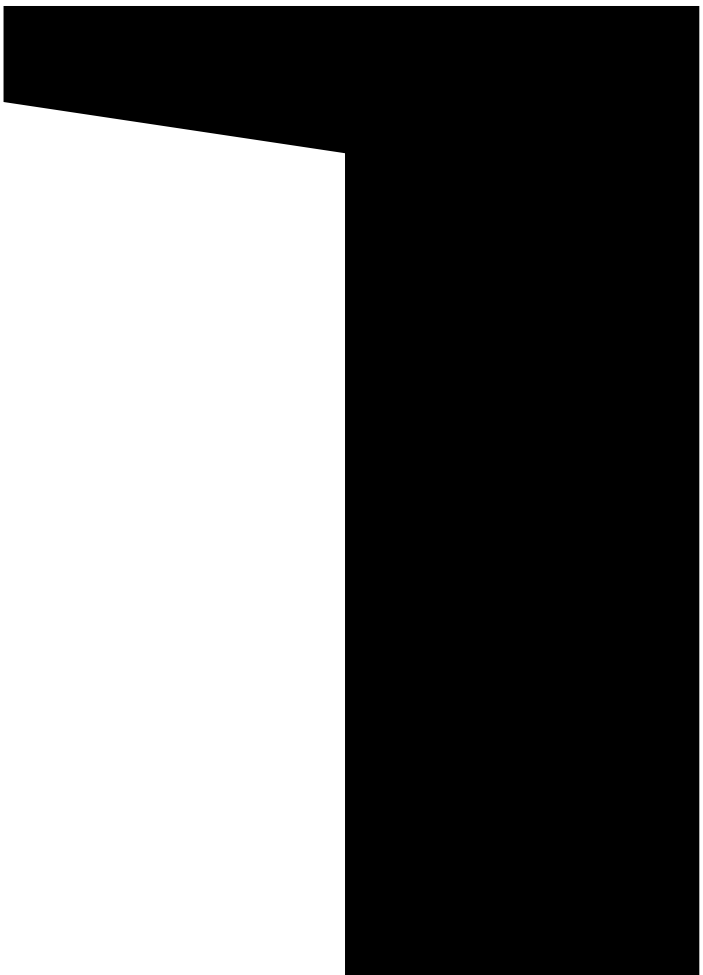


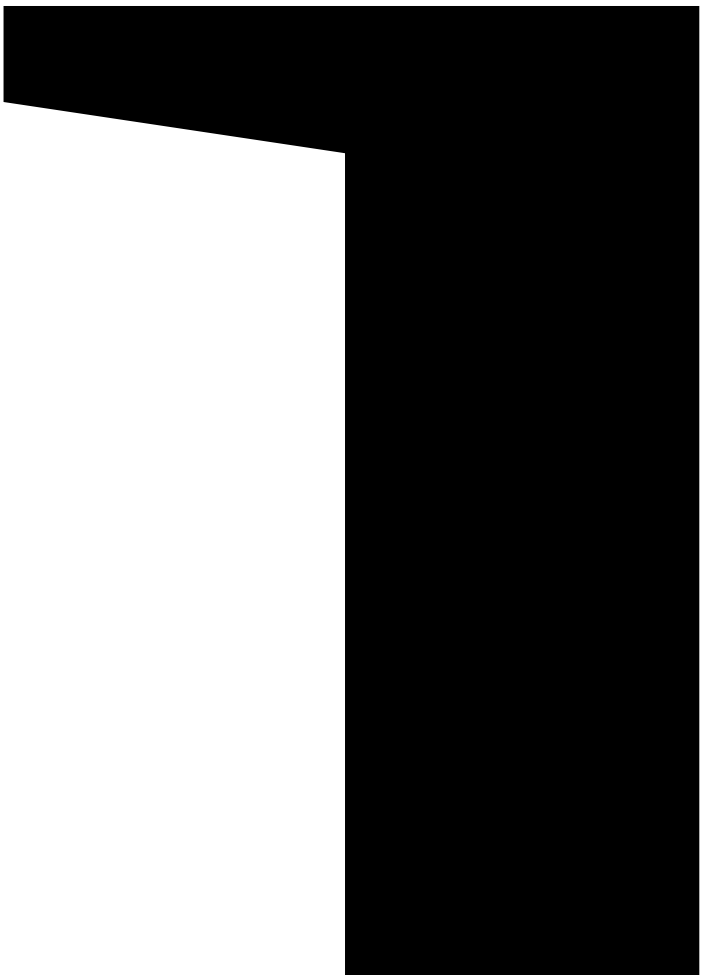


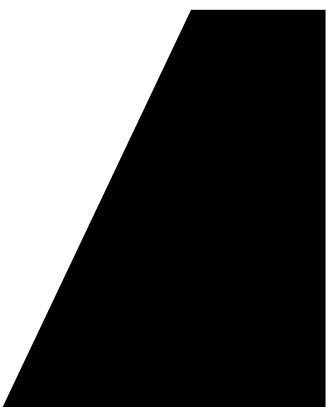
mcx

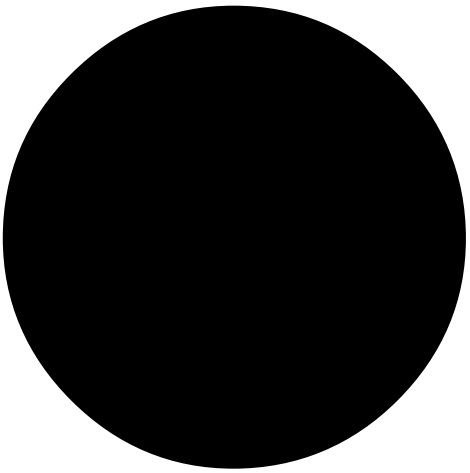


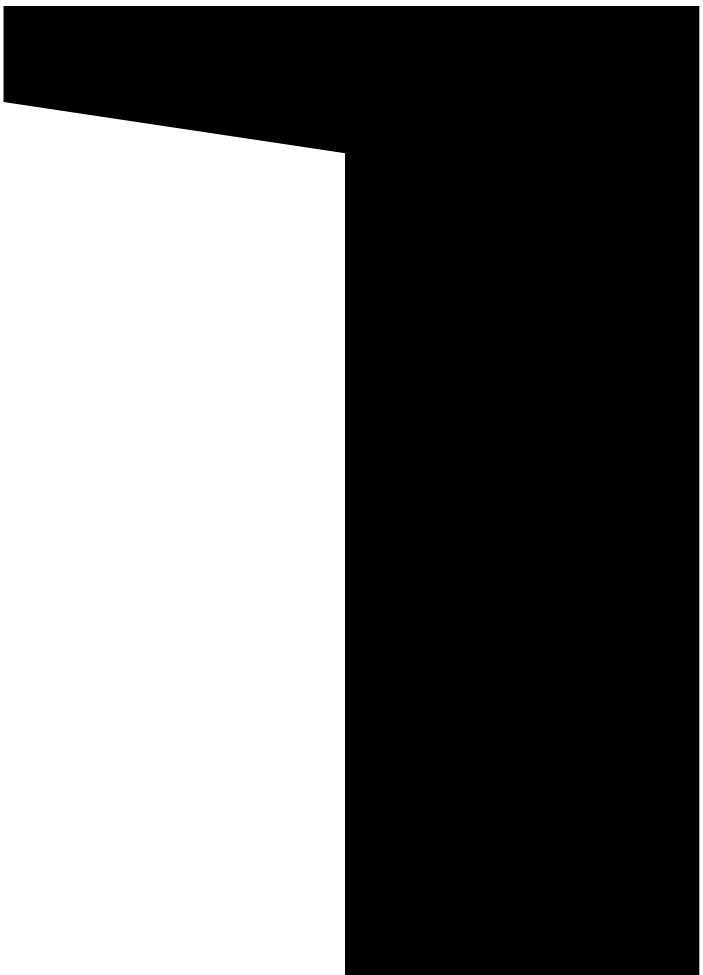


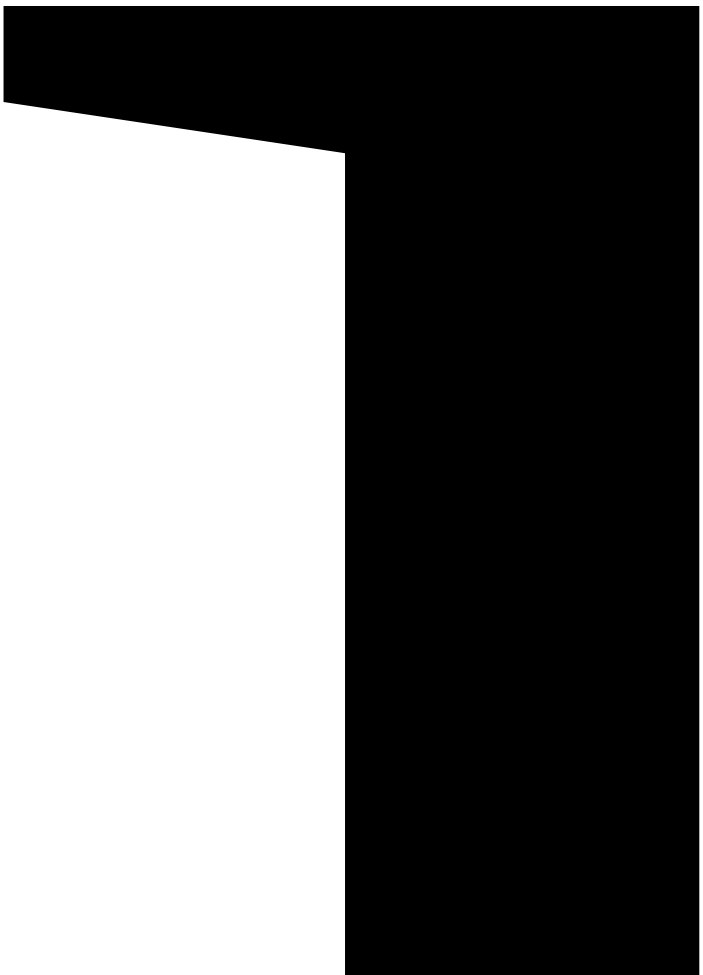


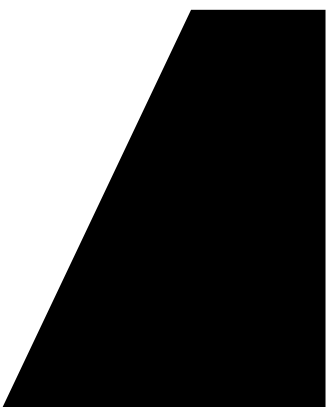


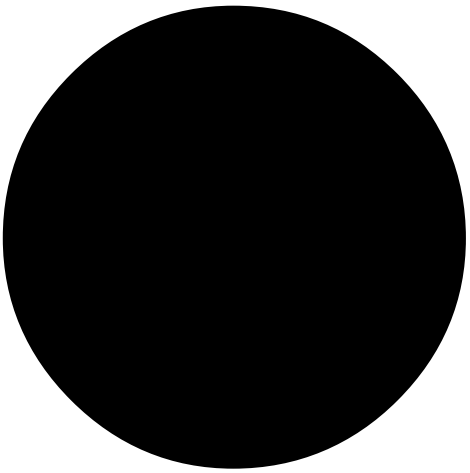




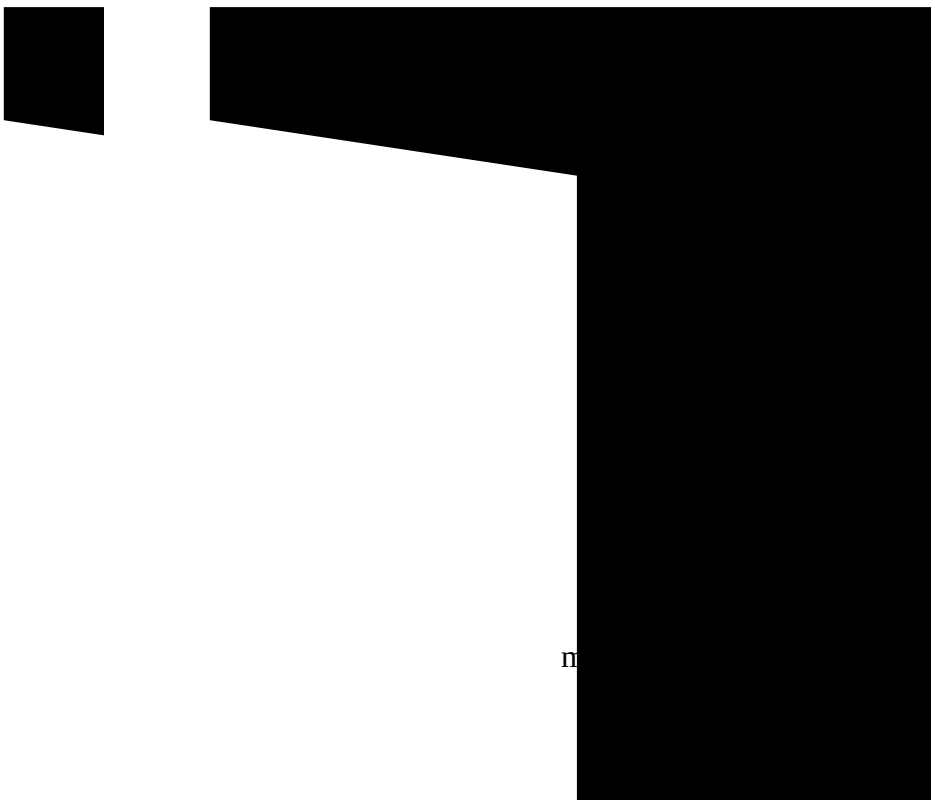


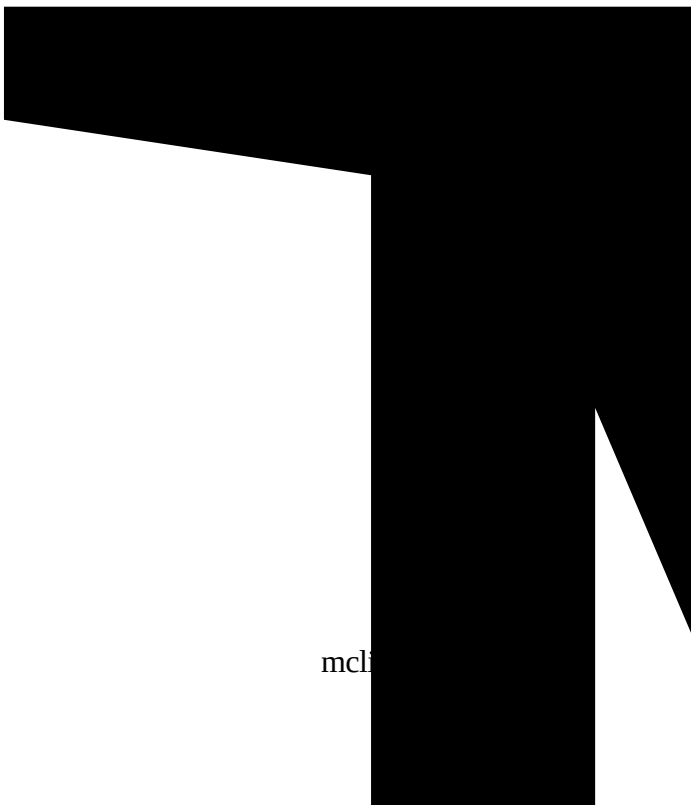




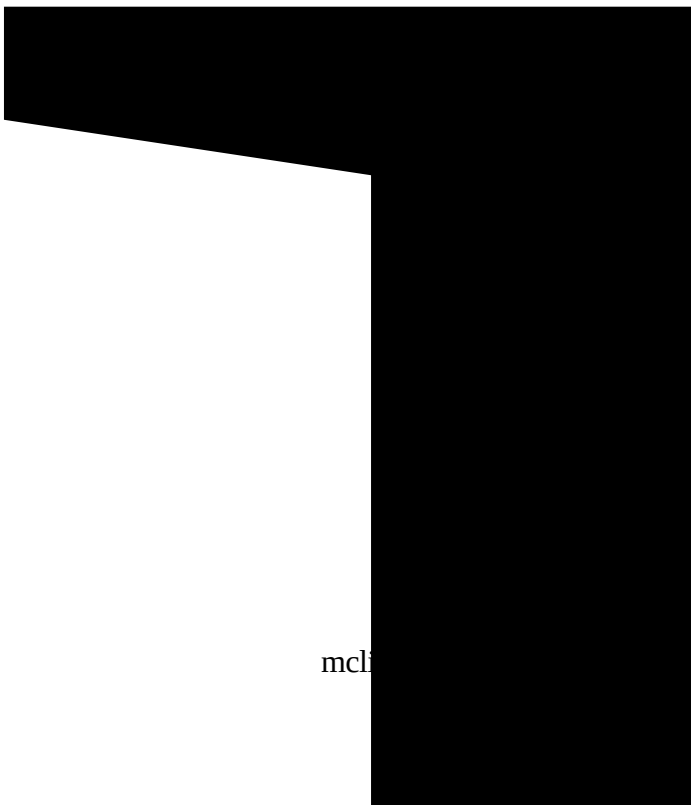


mcxl

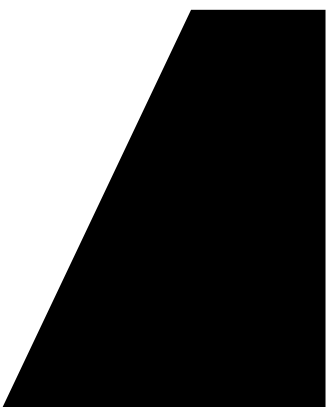


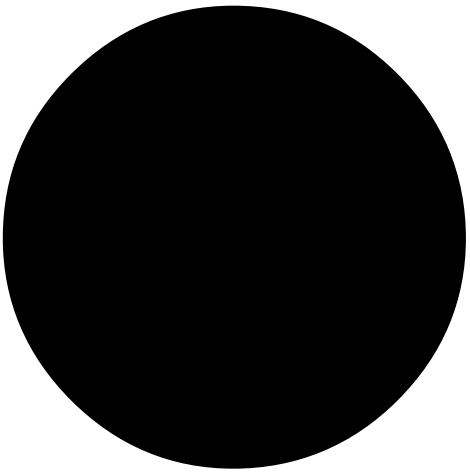


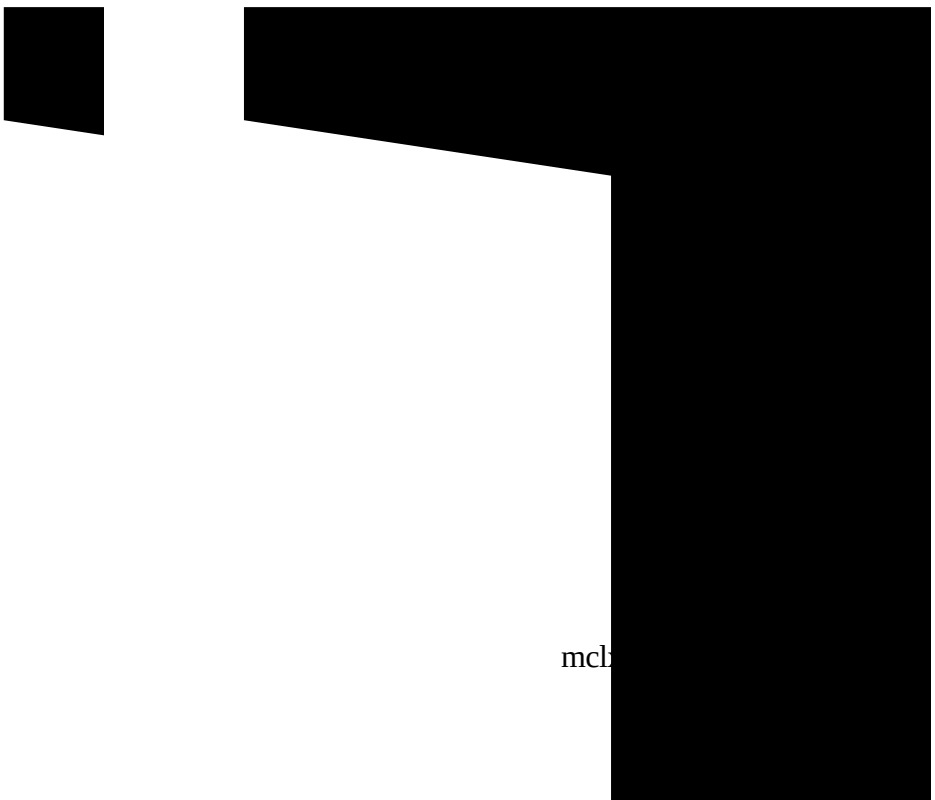
mcli

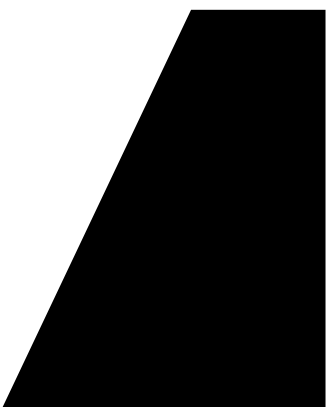


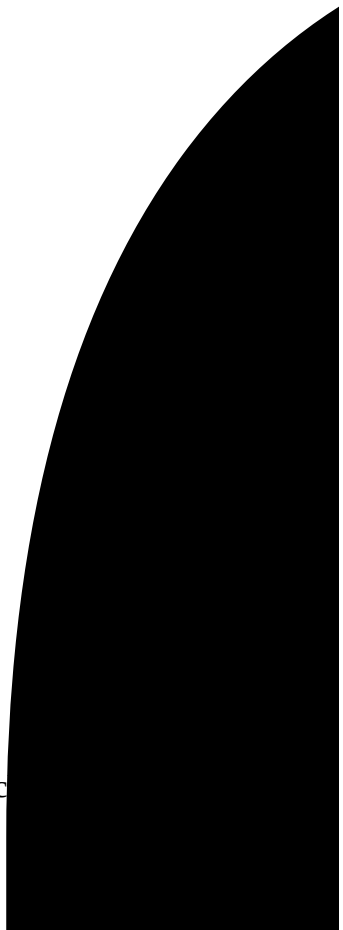
mcli

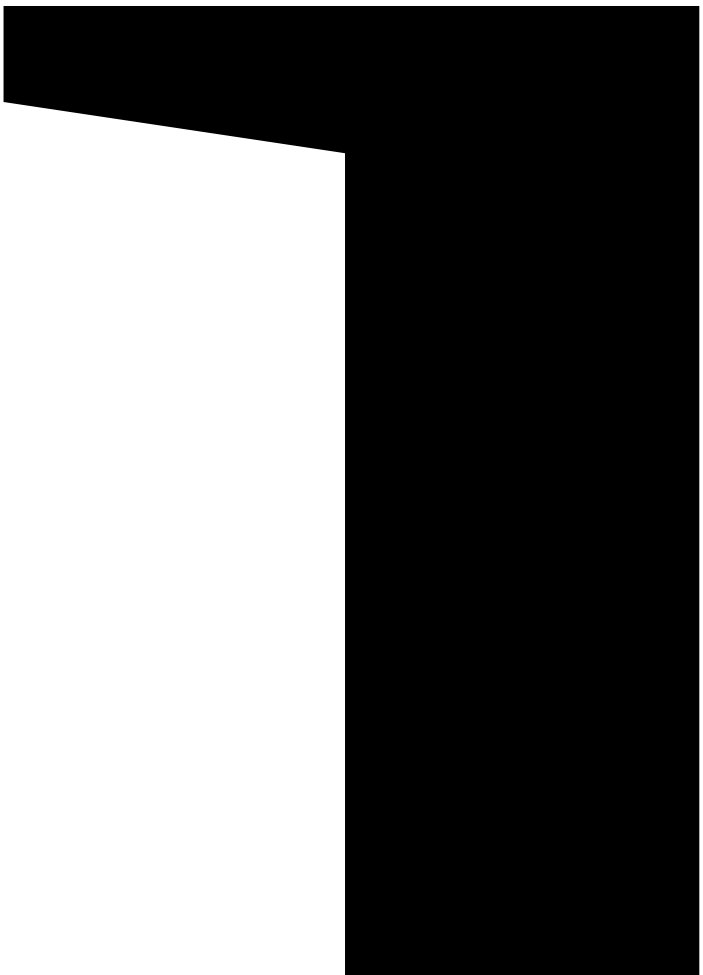


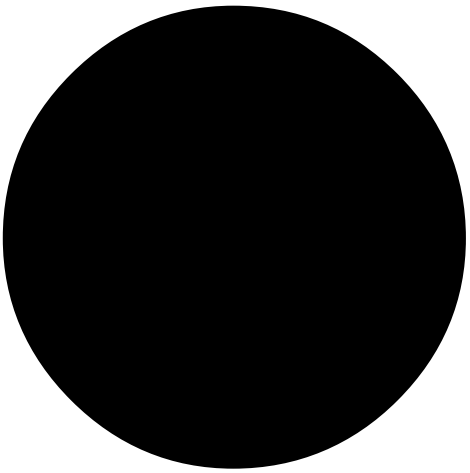




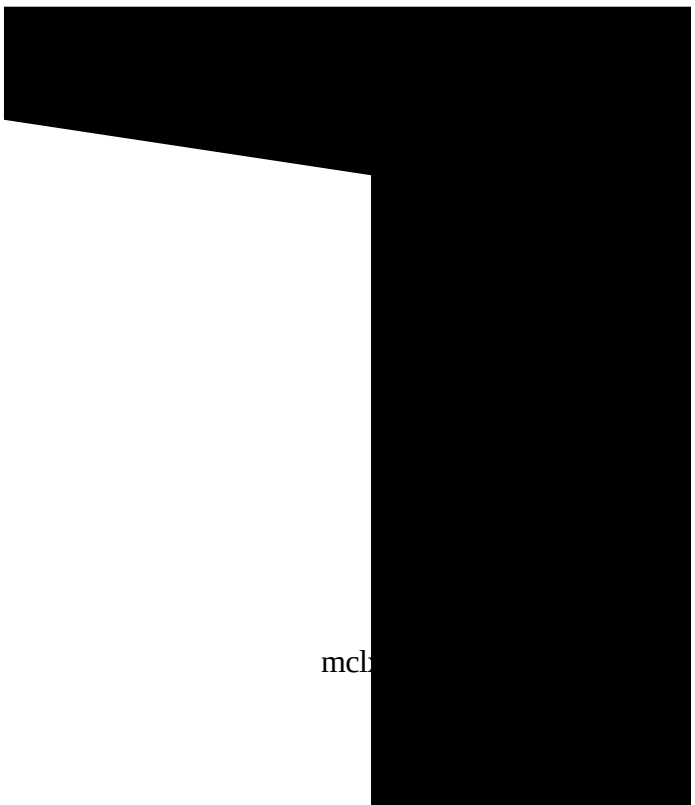




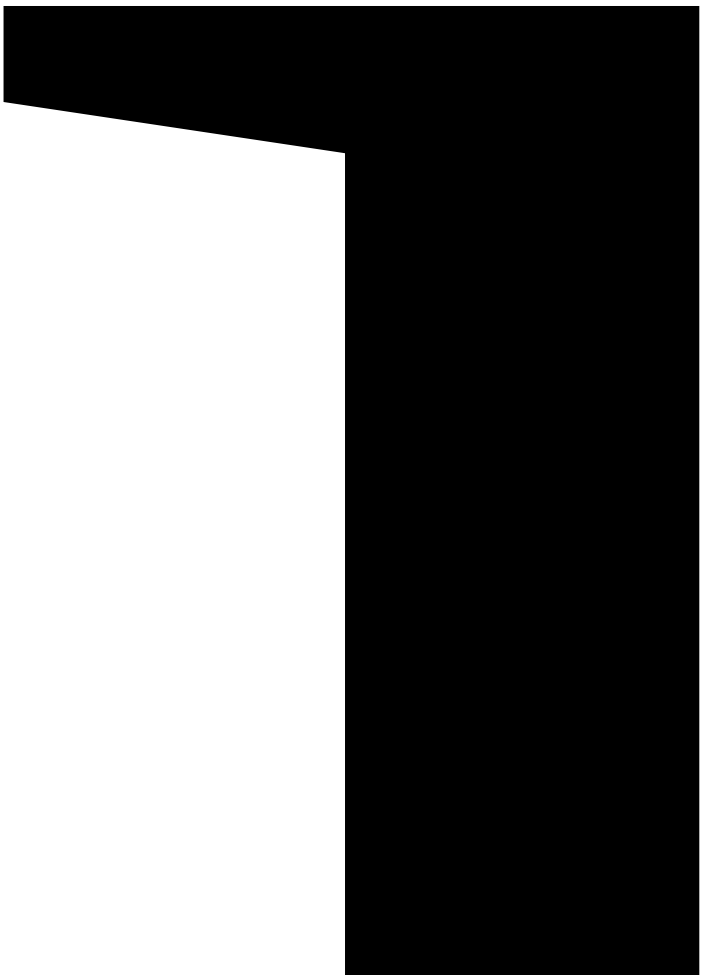


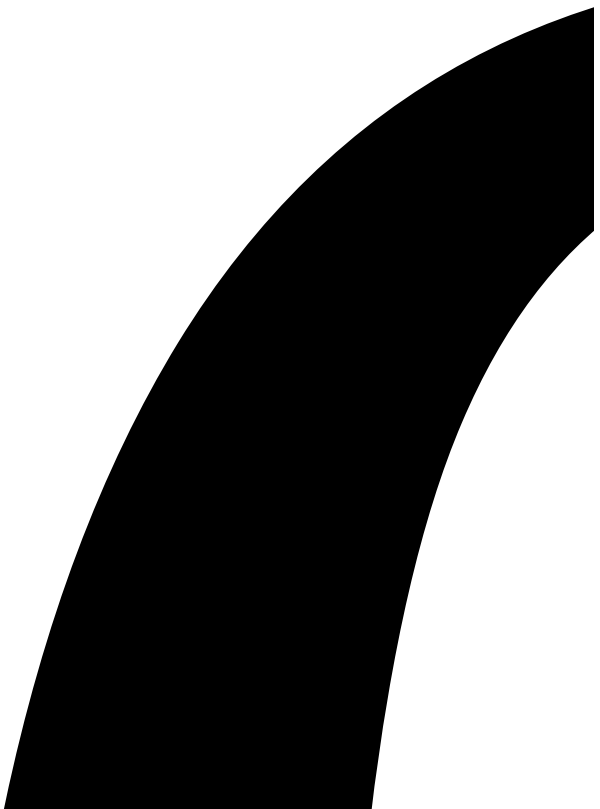


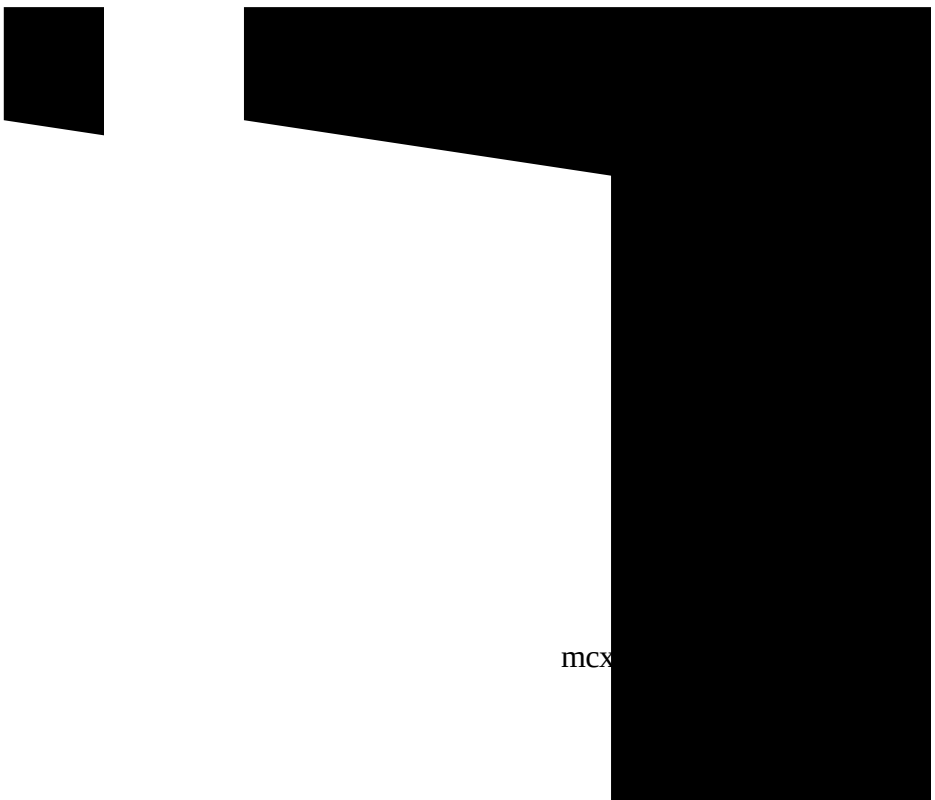
mclx



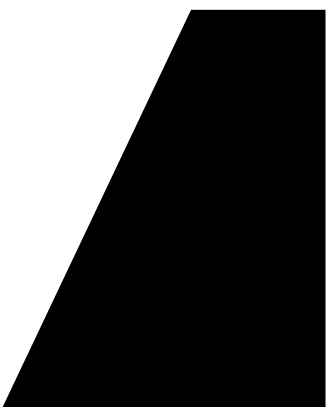
mcl

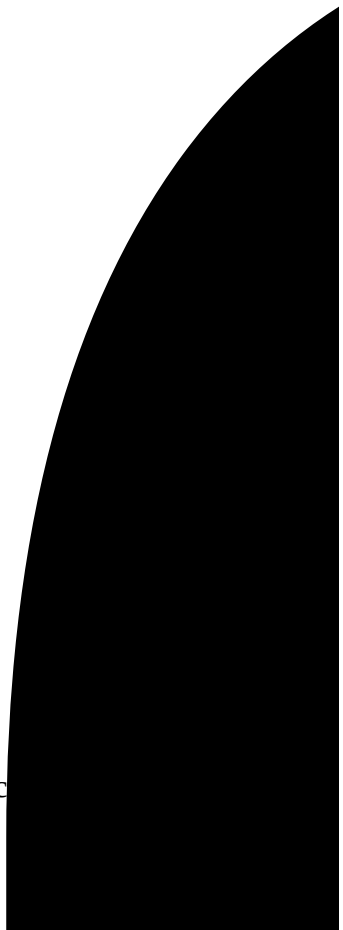


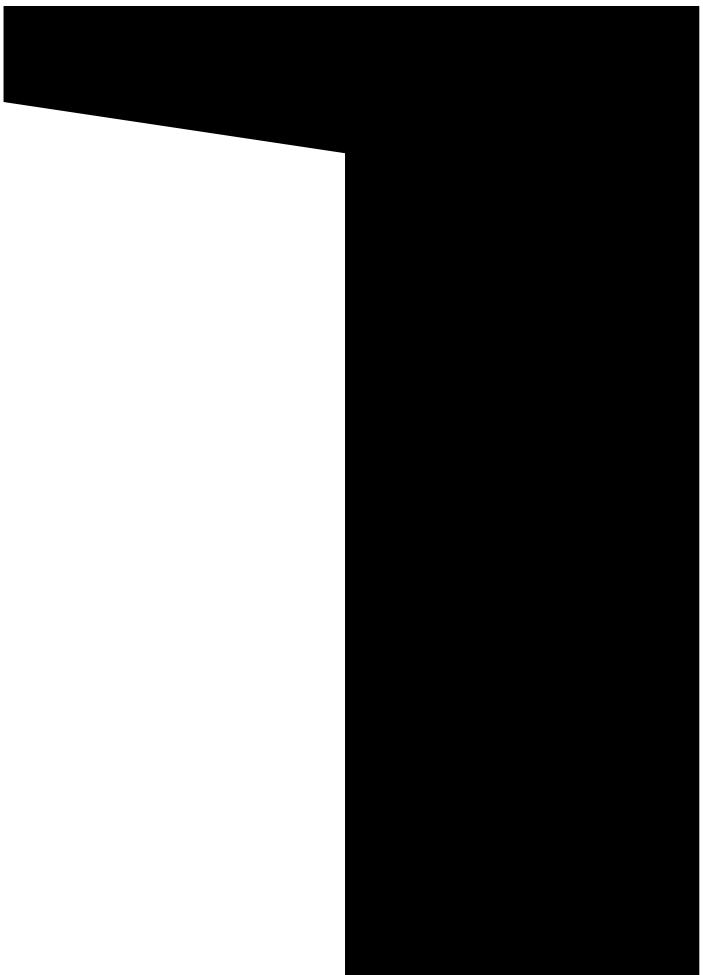


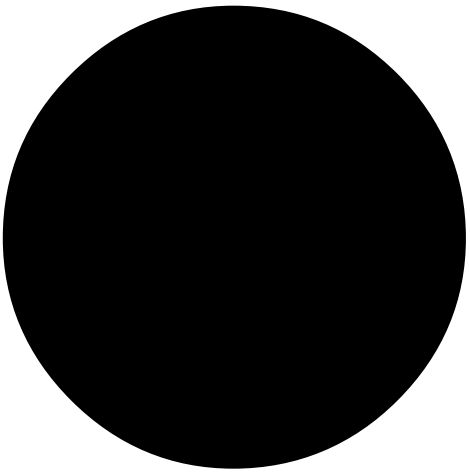


mcx

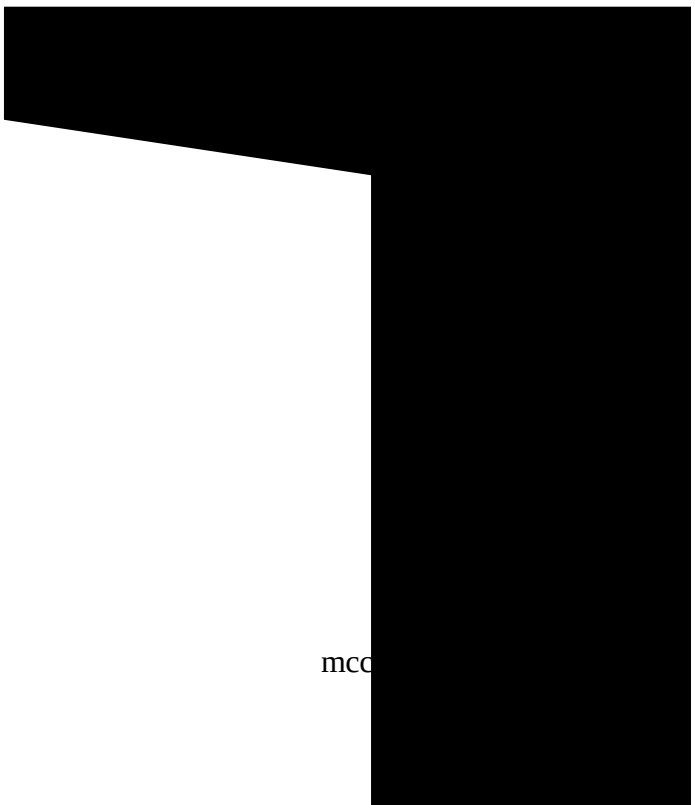




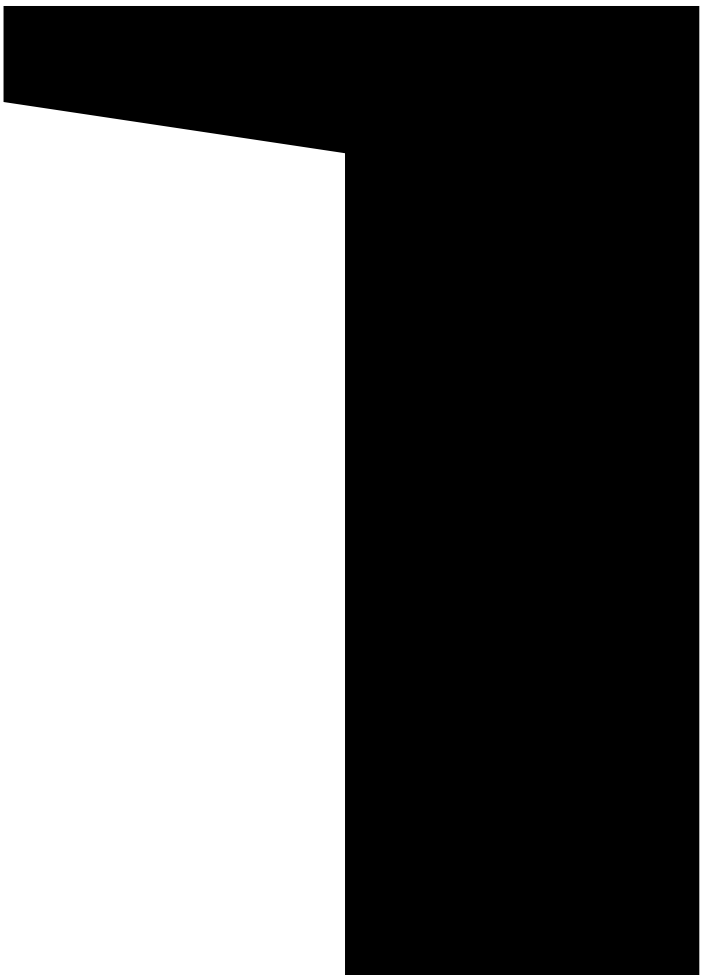




mcc

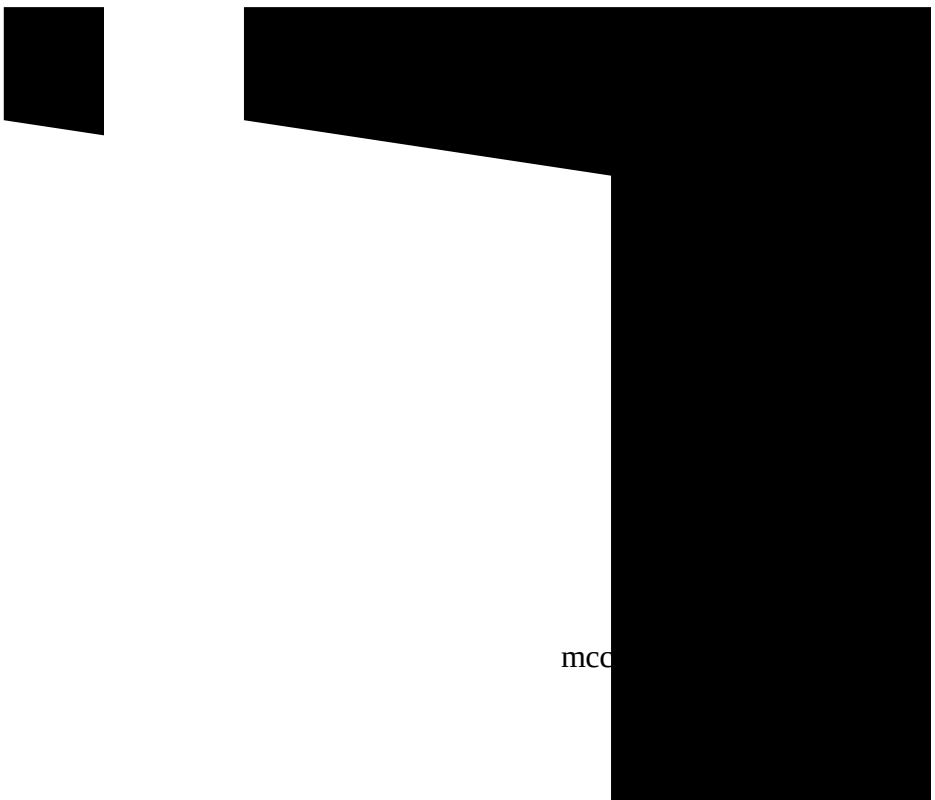


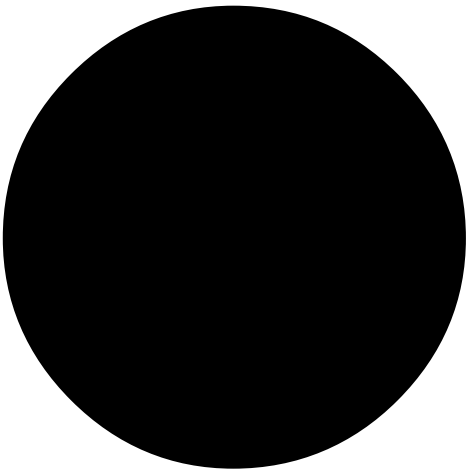
mcc



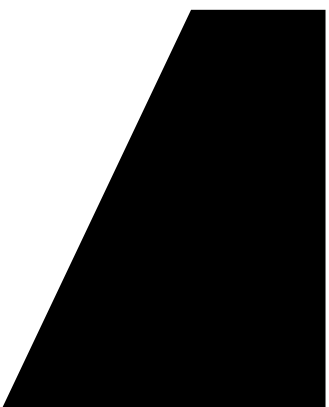


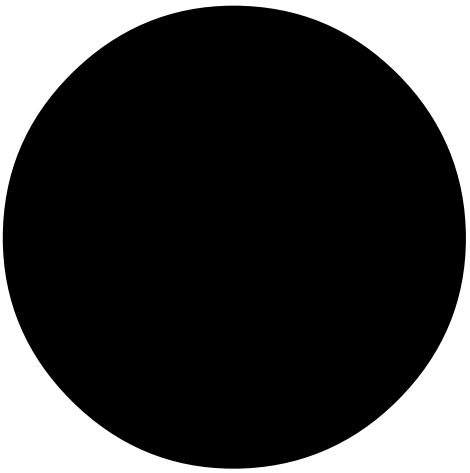
mccx



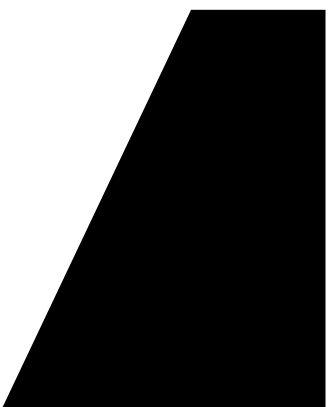


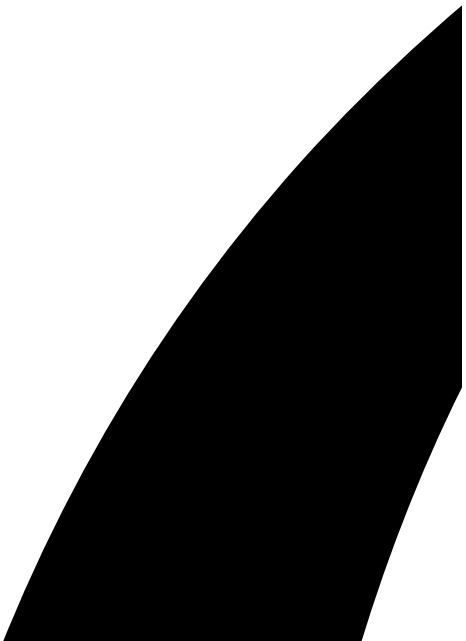
mccx



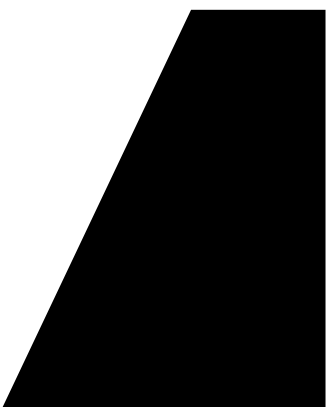


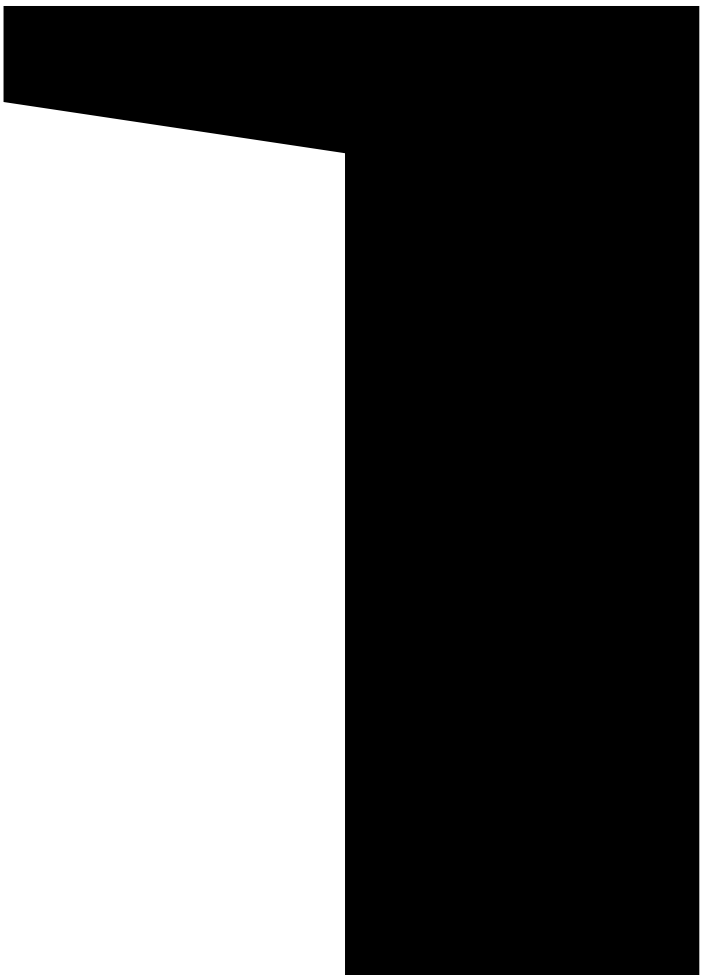
mccx



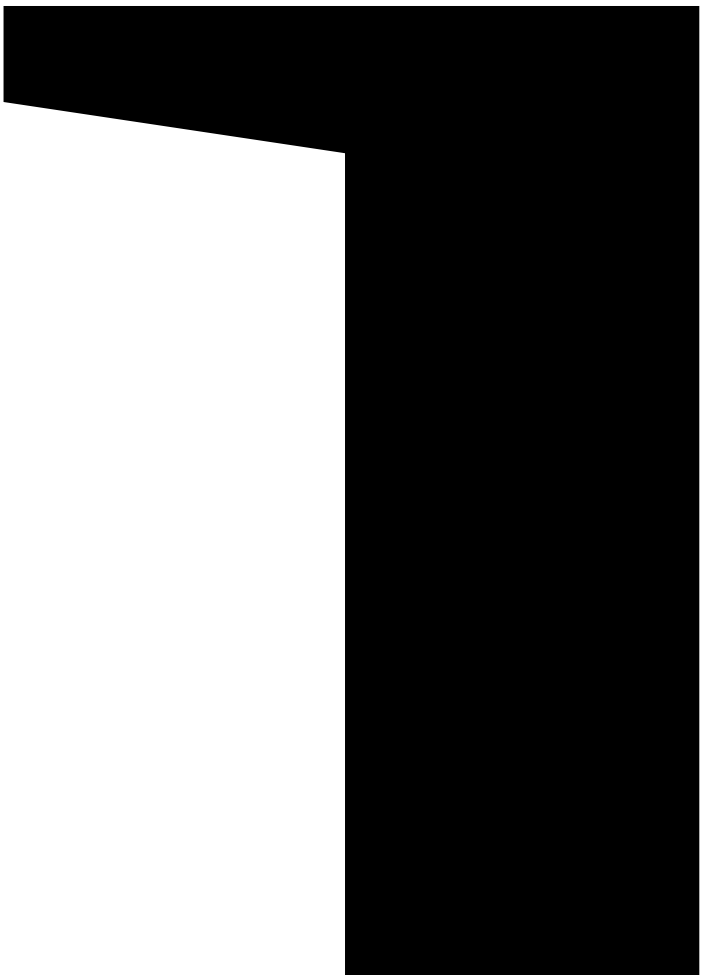


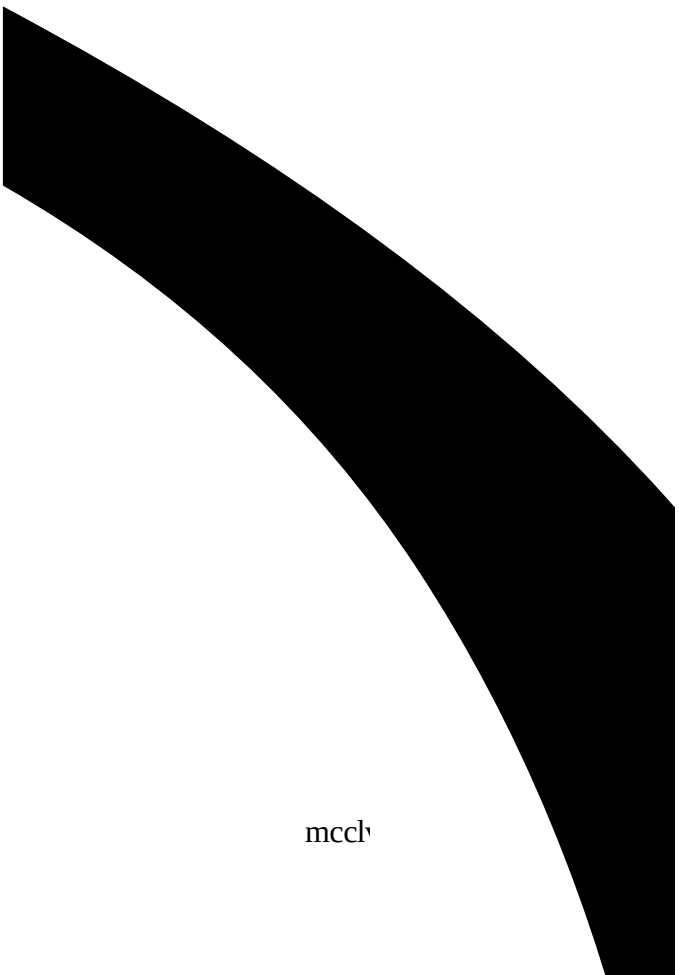




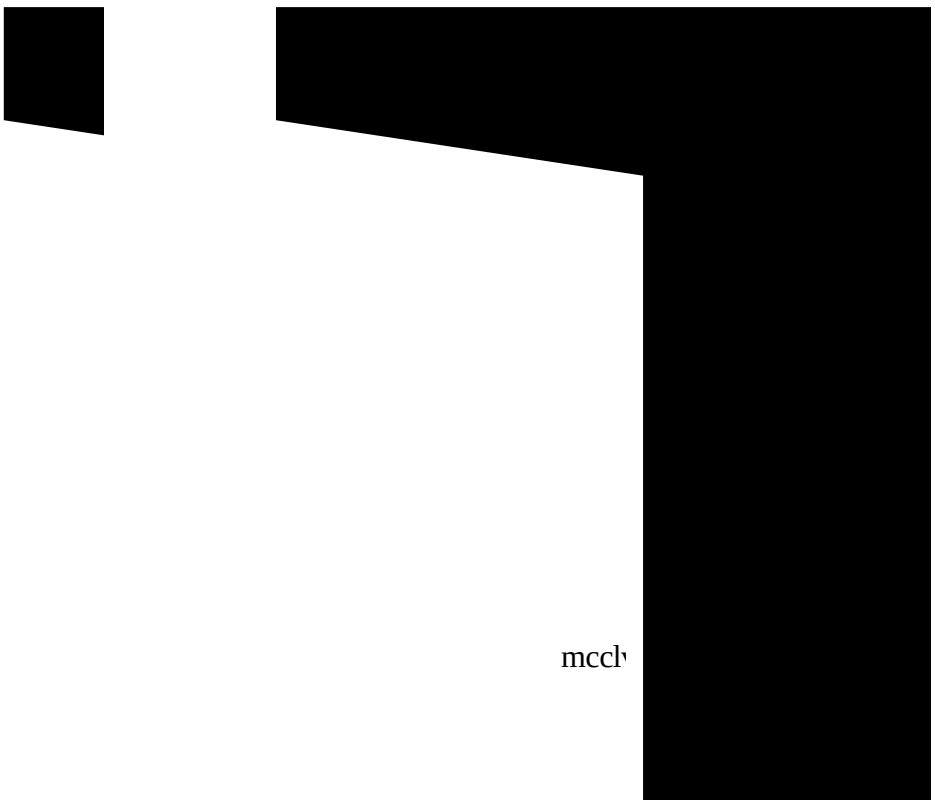




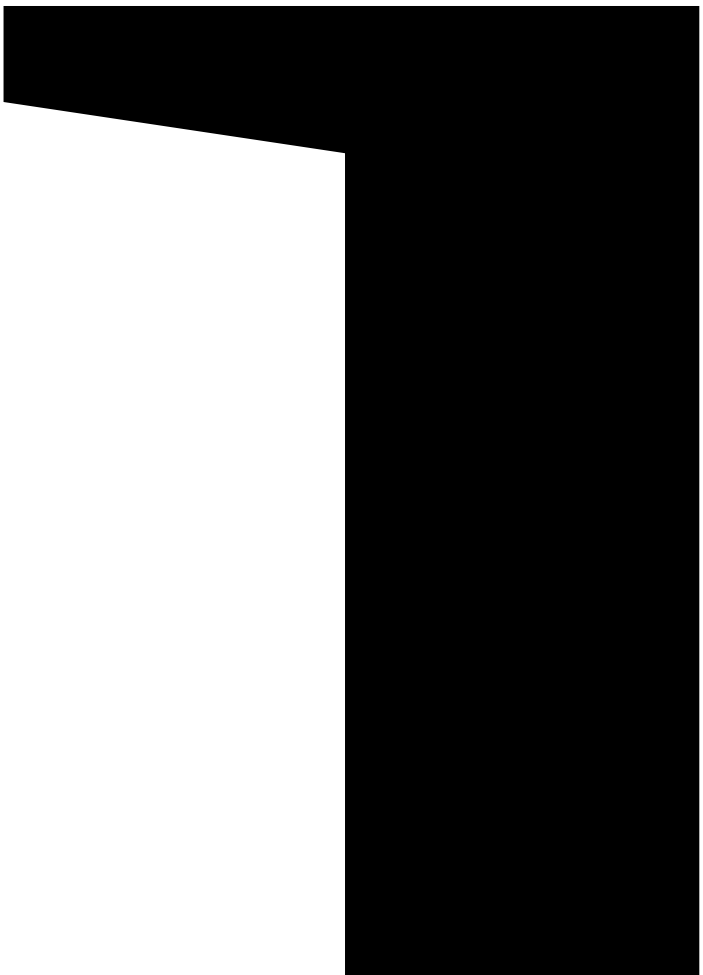


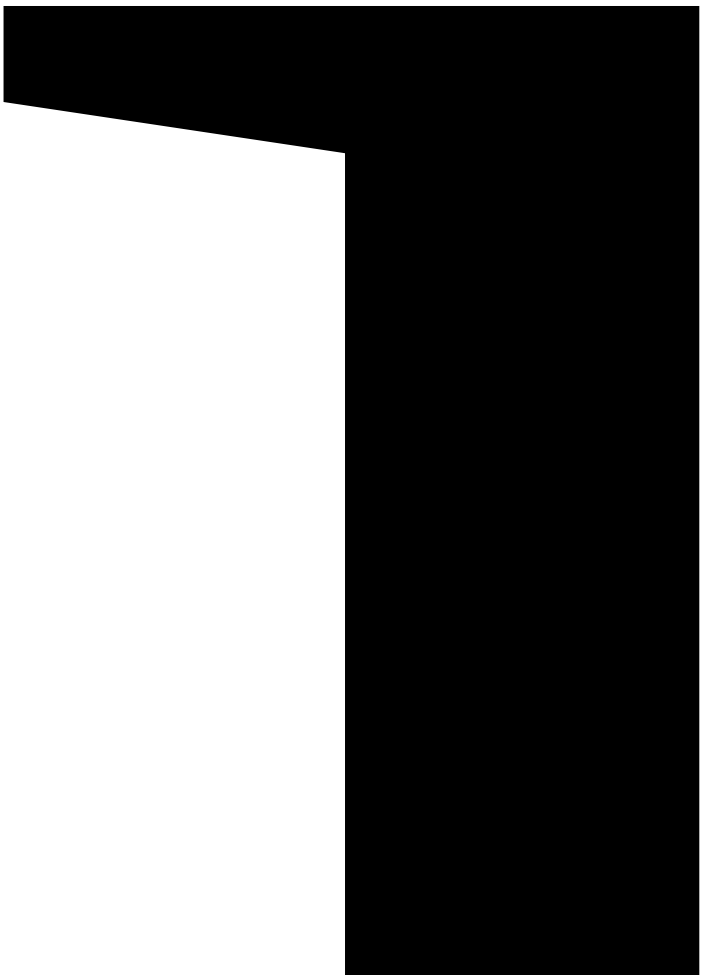


mccl



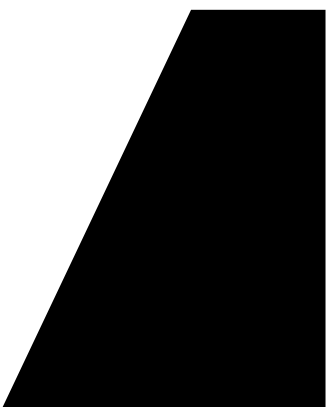
mccl





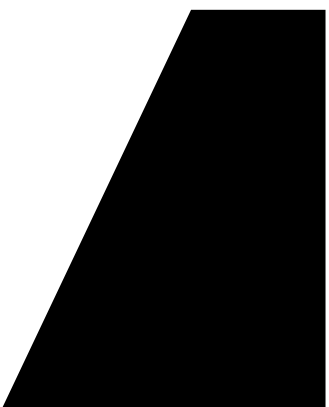
mccl:

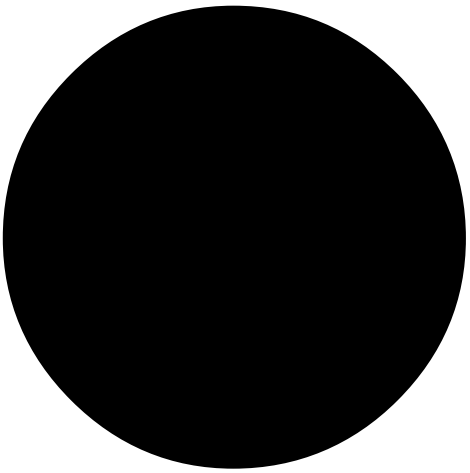
mccl:



mccl:

mccl:





mccl:

mccl:

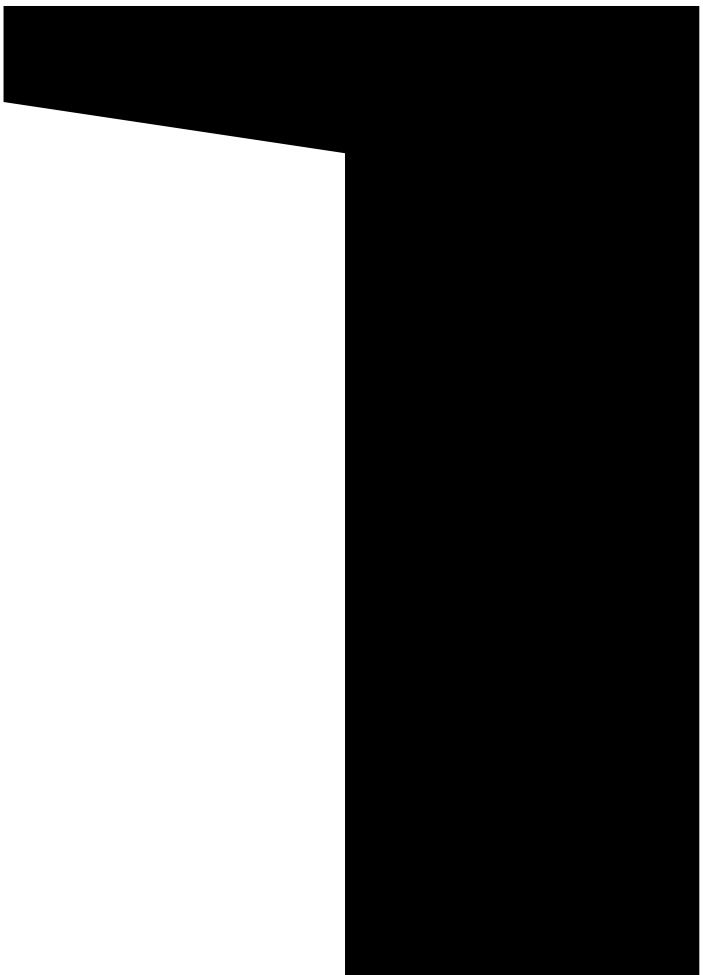
mccl:

mccl:

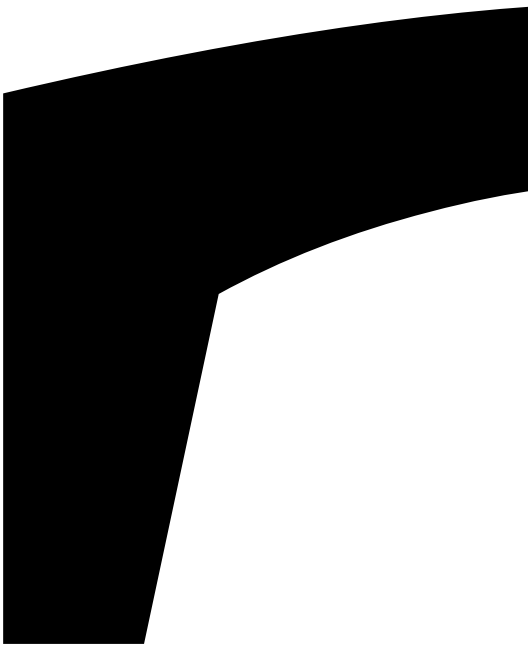
mccl:

mccl:

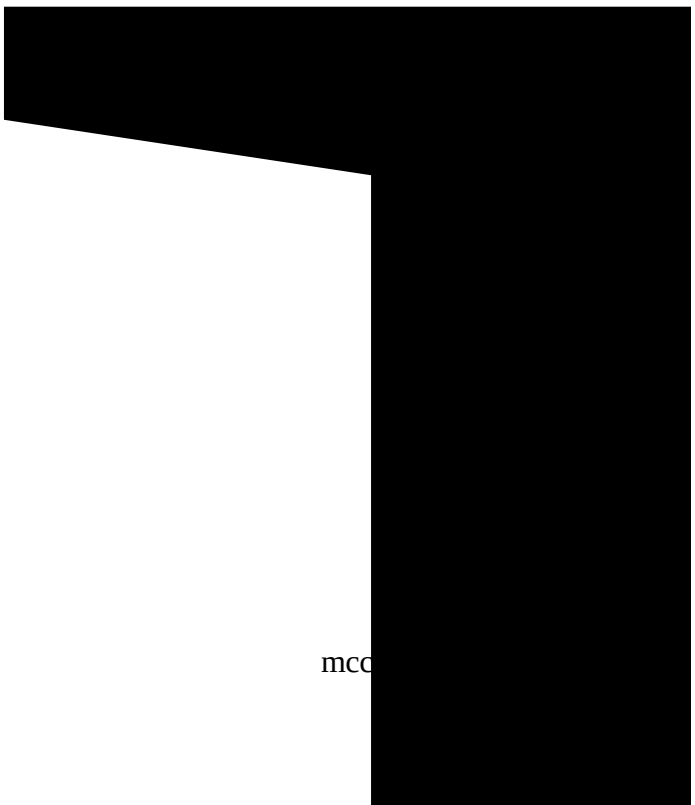
mccl:



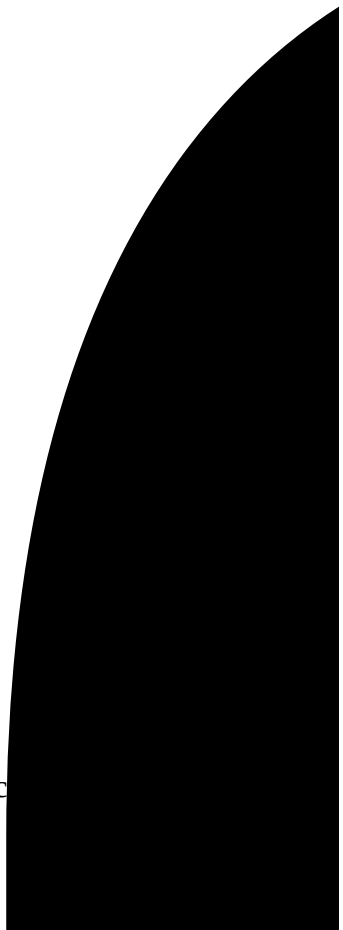
mccl:

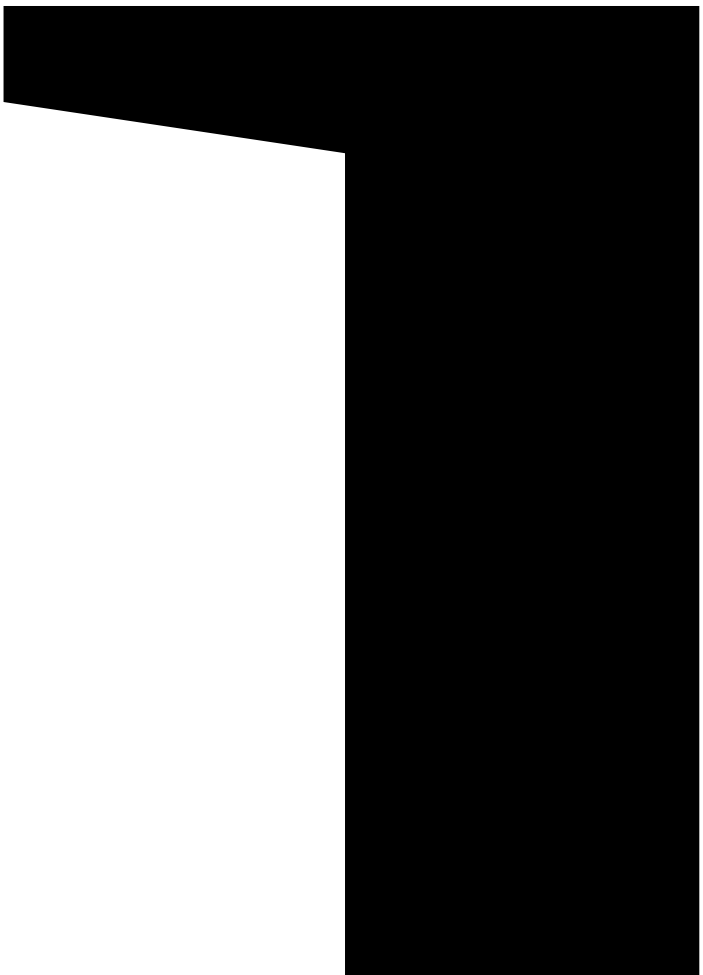


mccl:



mcc



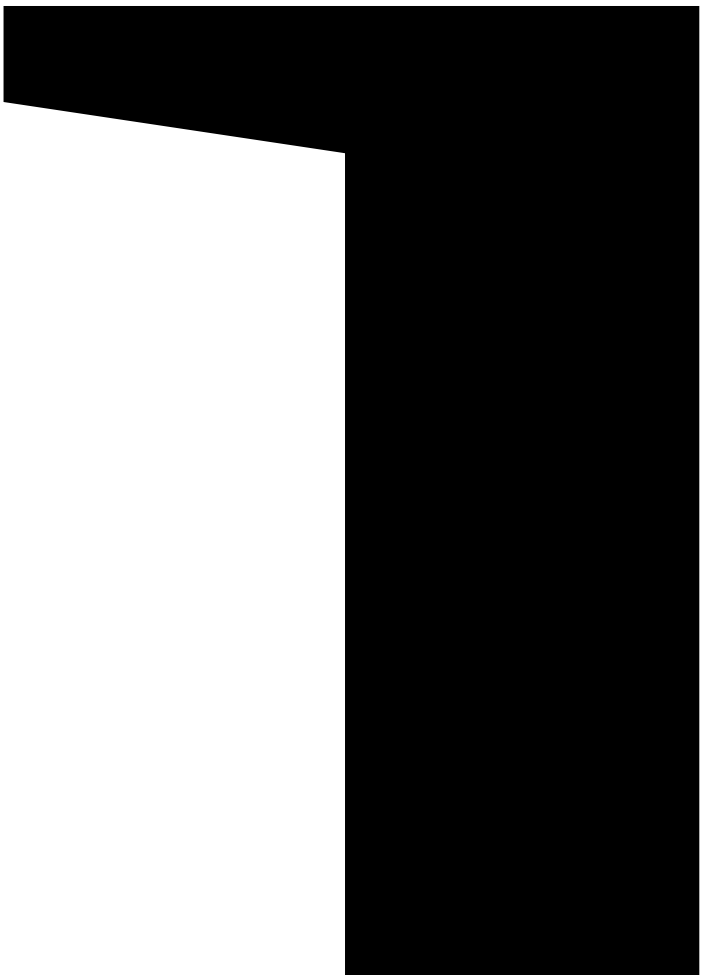


mccl:

mccl:

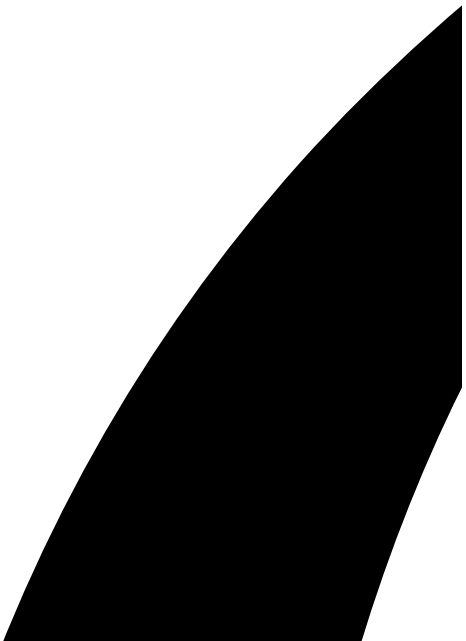
mccl:

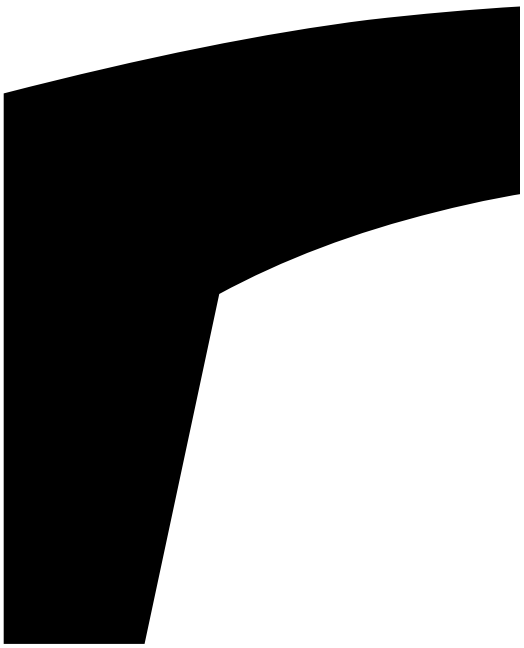
mccl:



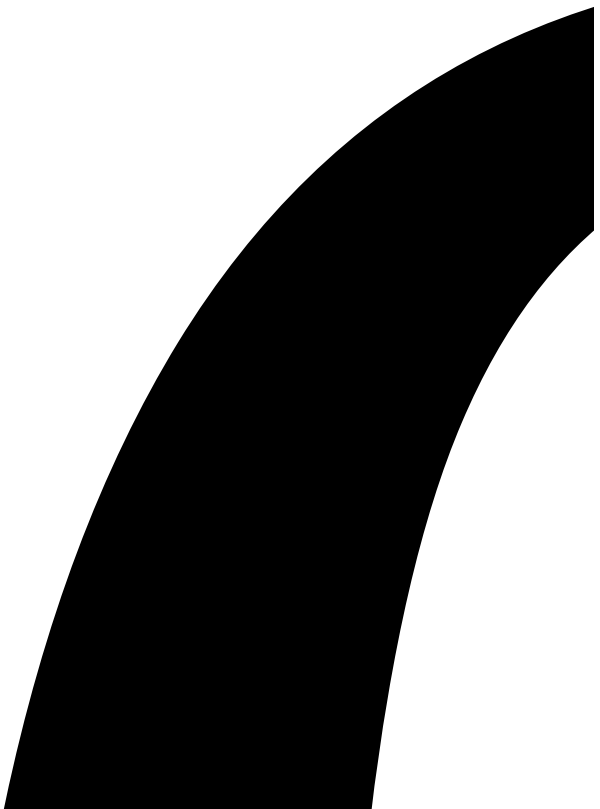
mccl:

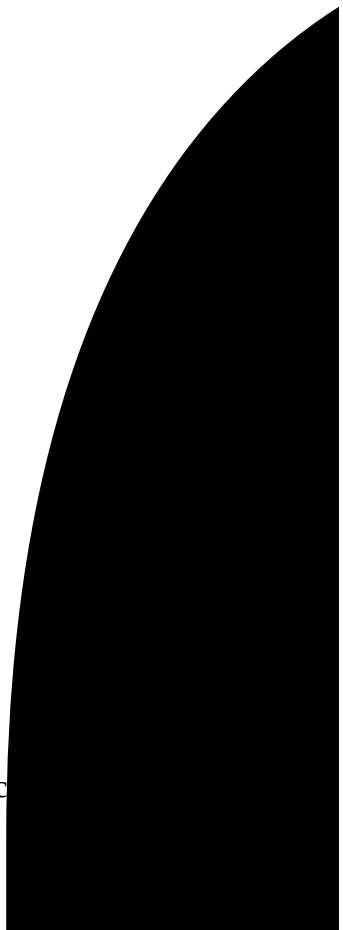
mccl:

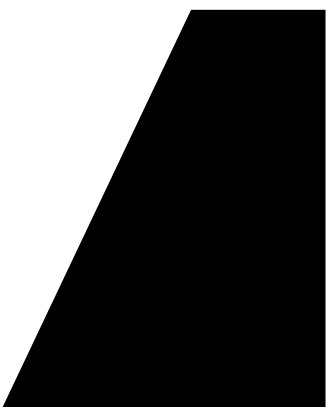


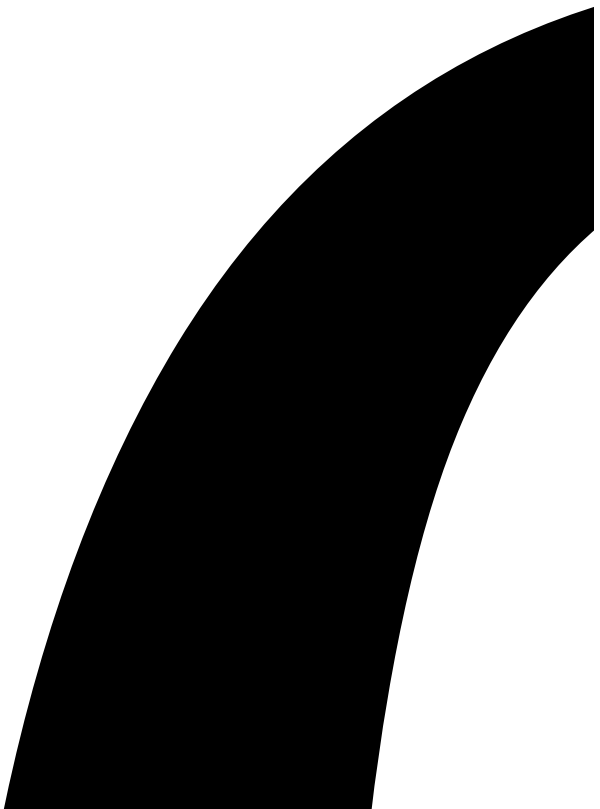


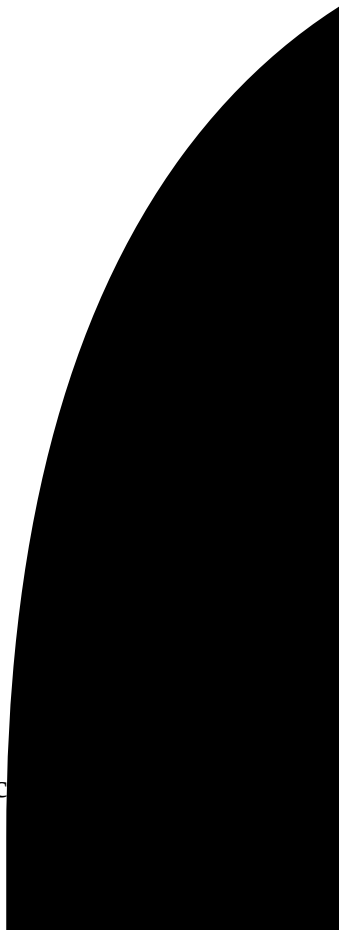
mccl:

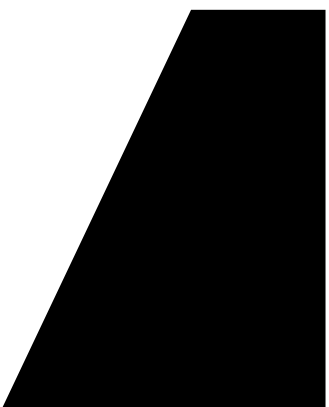


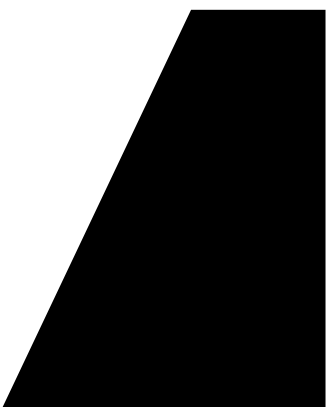




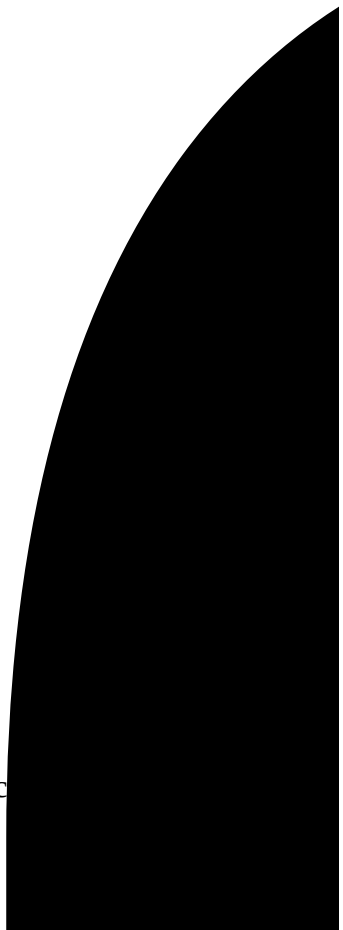


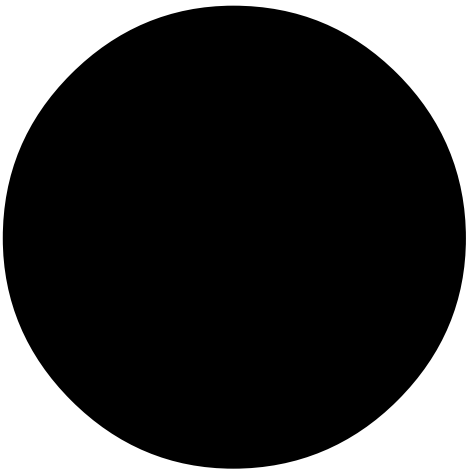




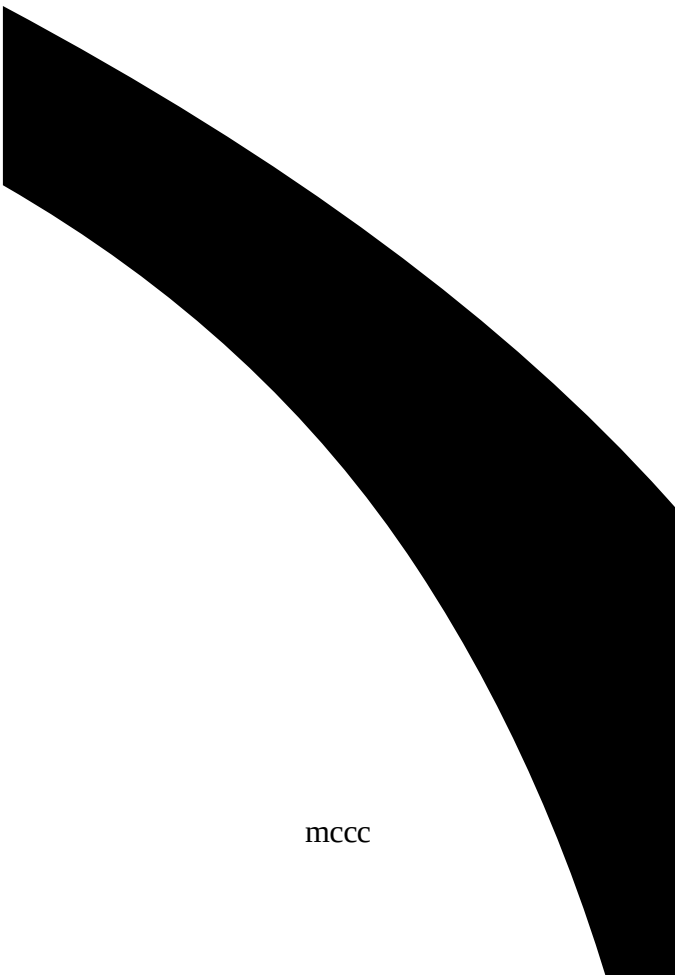




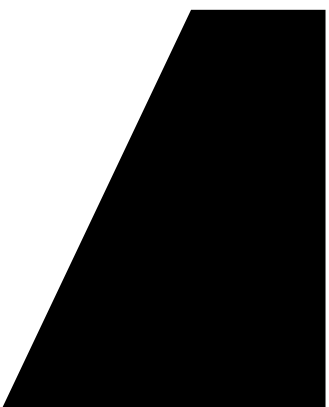


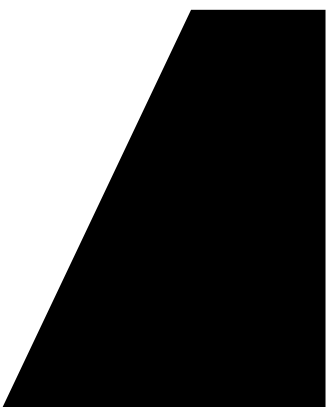


mccc

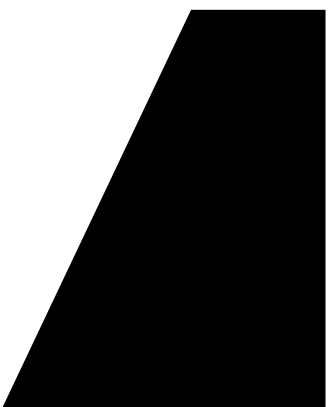


mccc

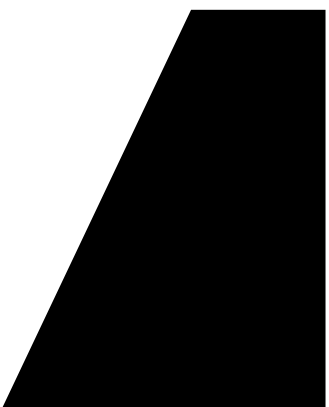




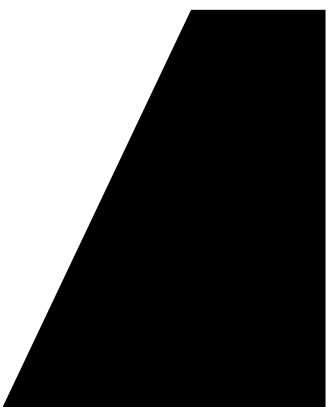


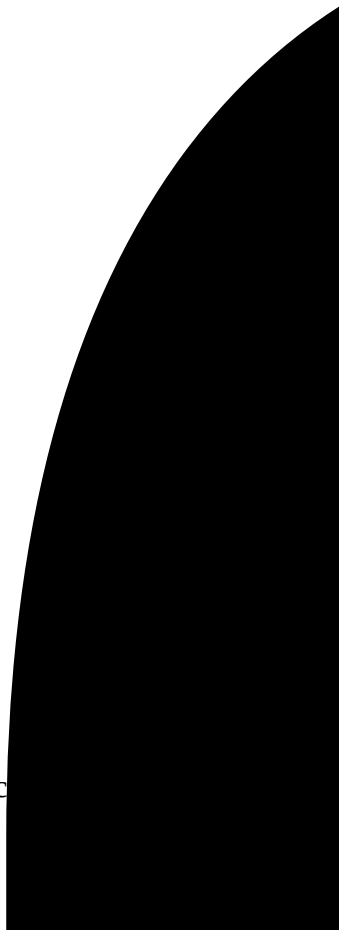


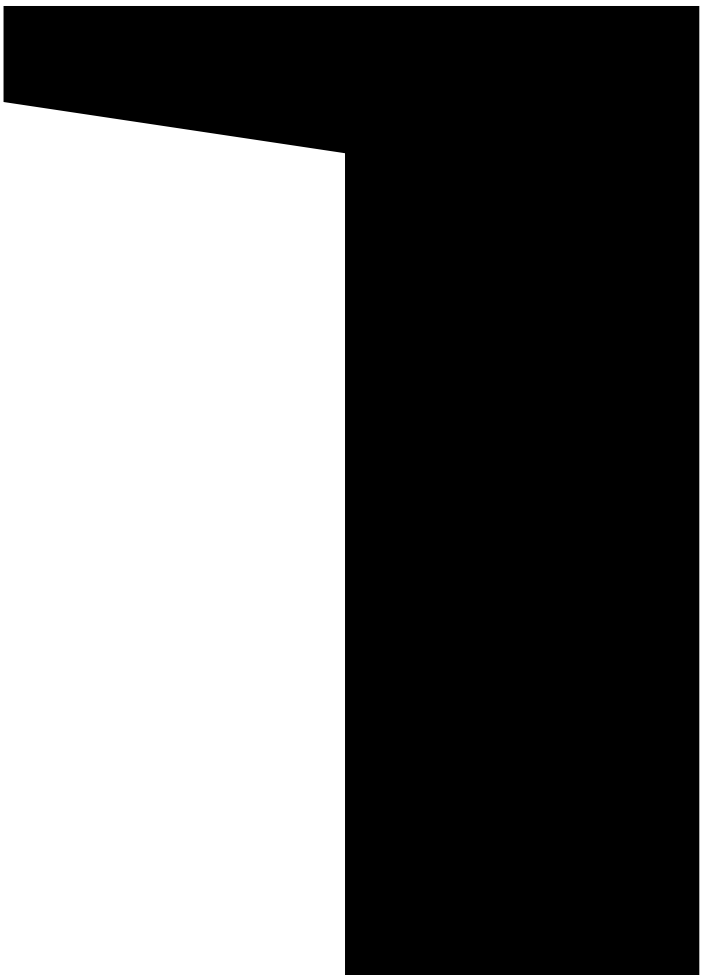


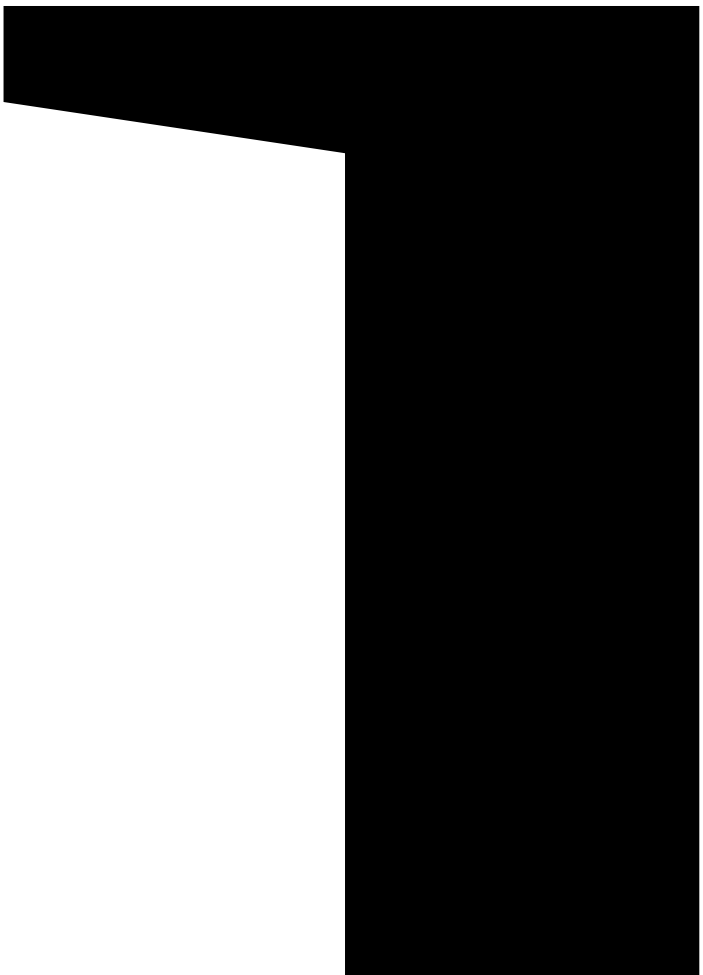


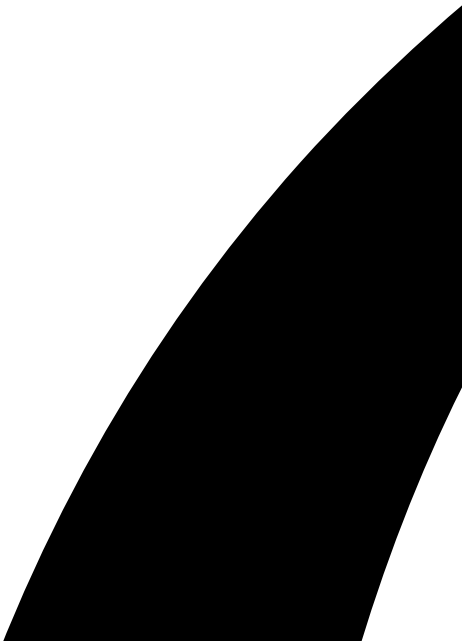




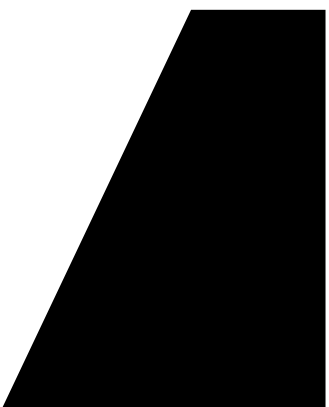


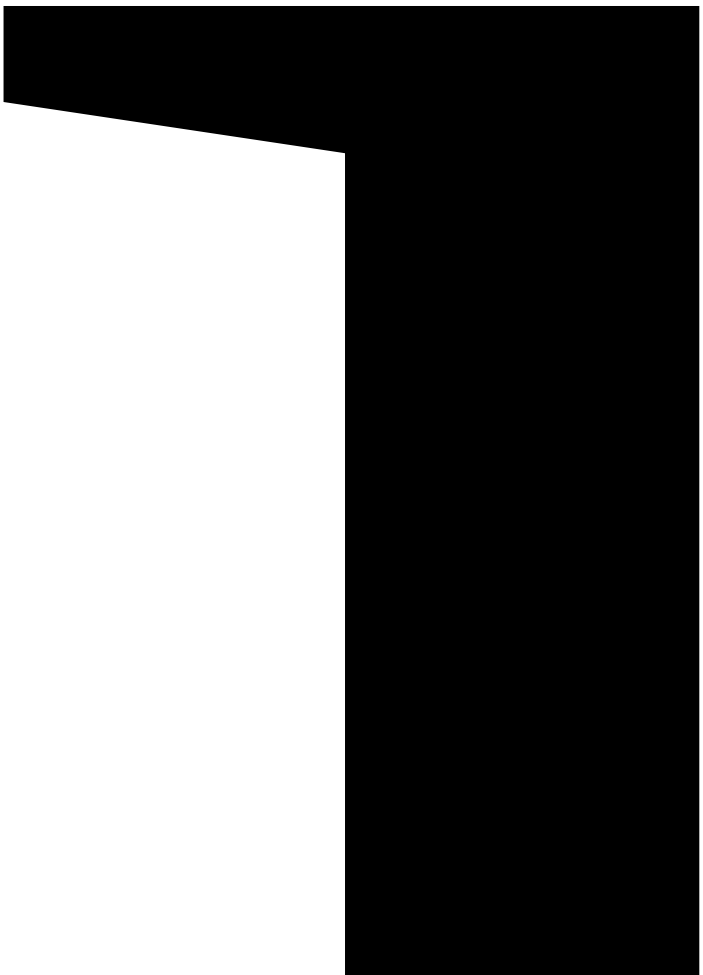




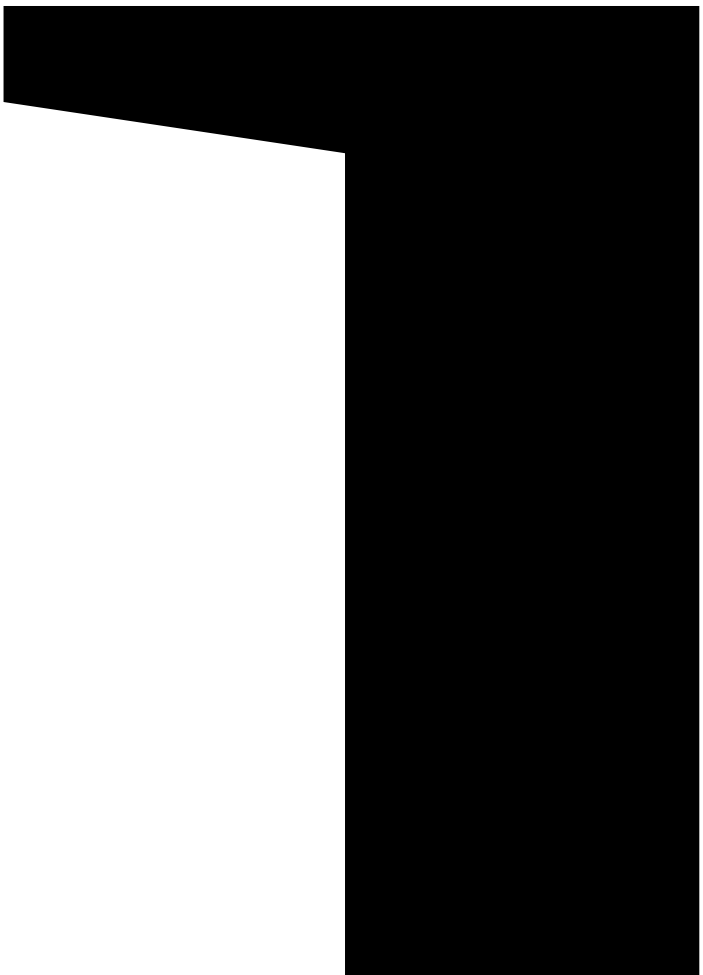


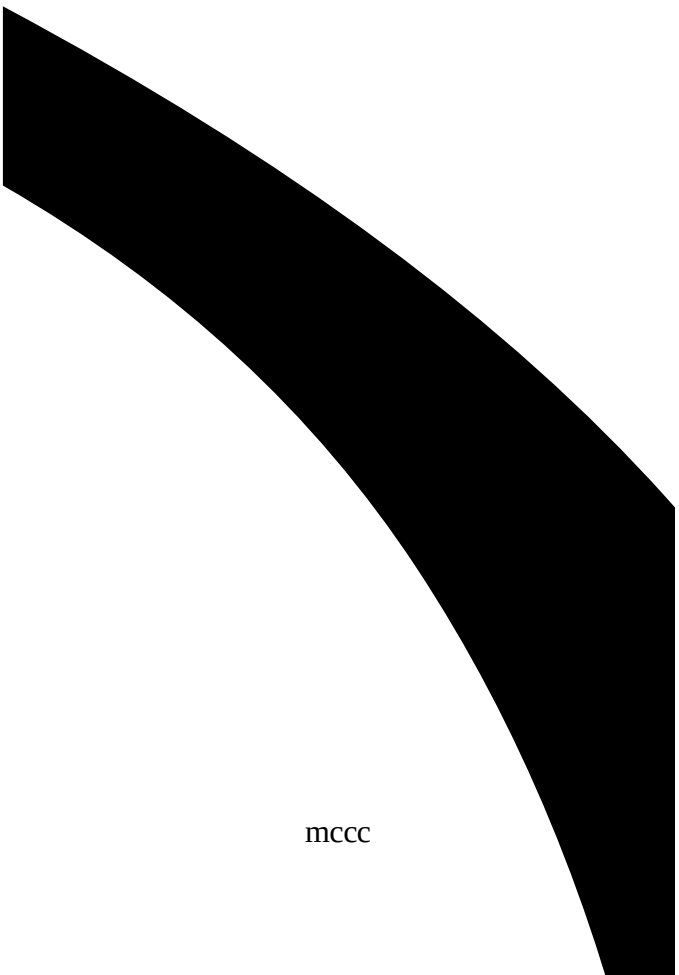












mccc



mccc



mccc



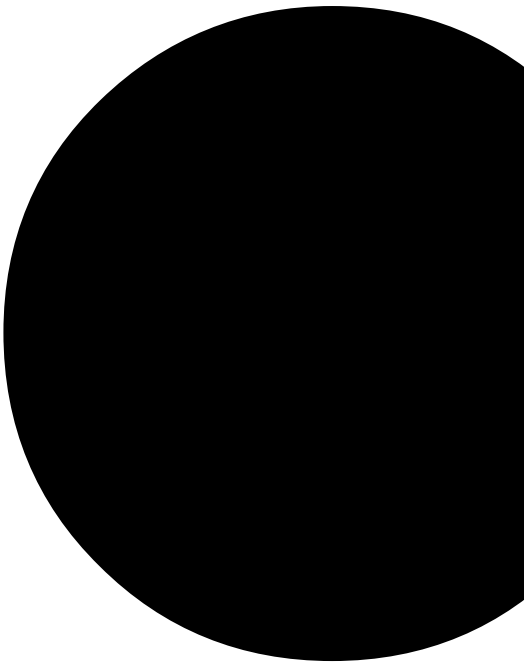
mccc



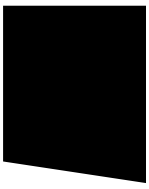
mccc





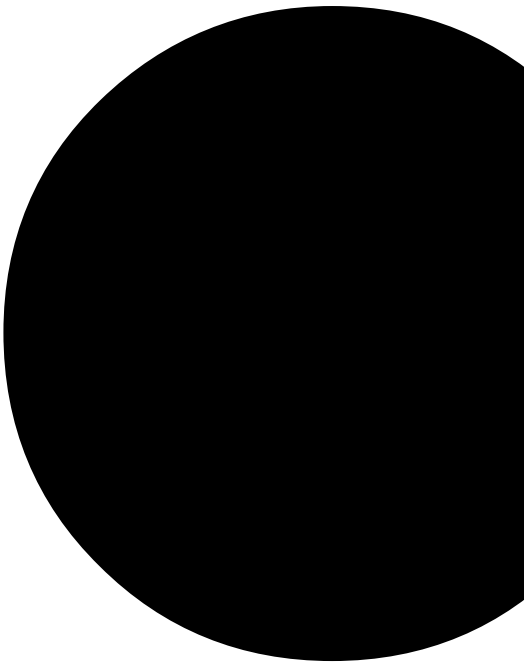


mccc









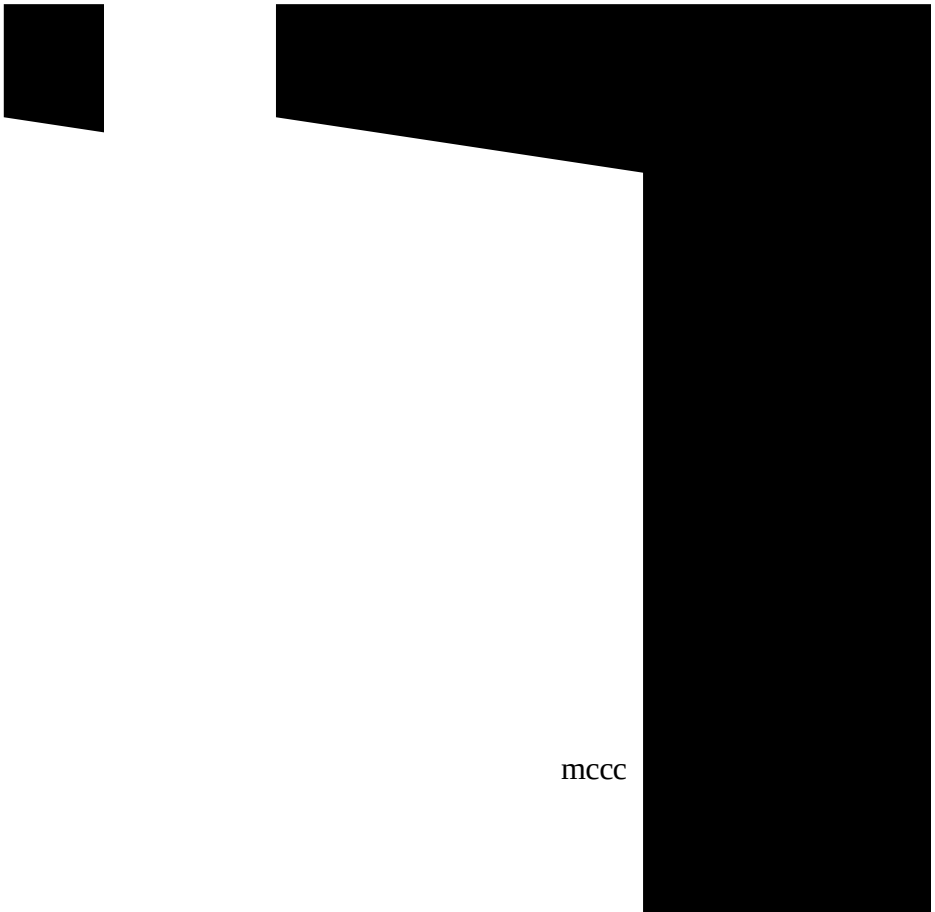
mccc

mccc



mccc

[redacted]



mccc

mccc



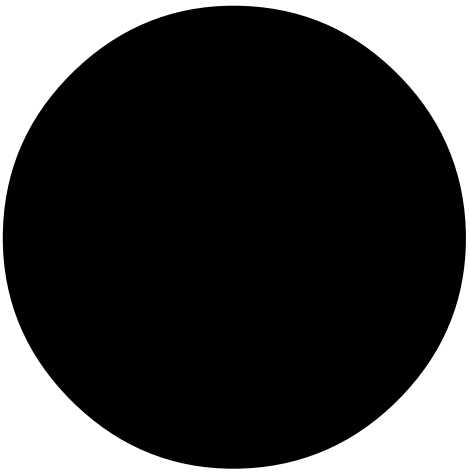


mccc



mccc





mccc

mccc



mccc



mccc

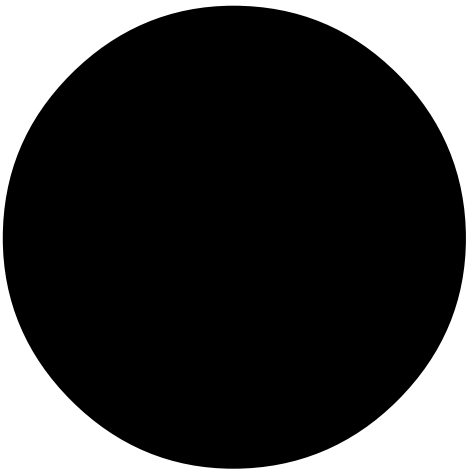


mccc



mccc





mccc


mcd

mcdi



mcdi



mcdv



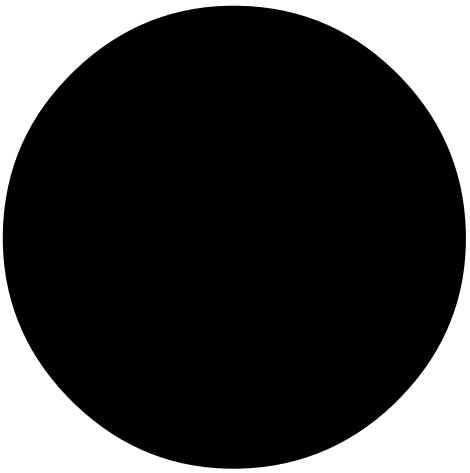
mcdv



mcdv



mcdv

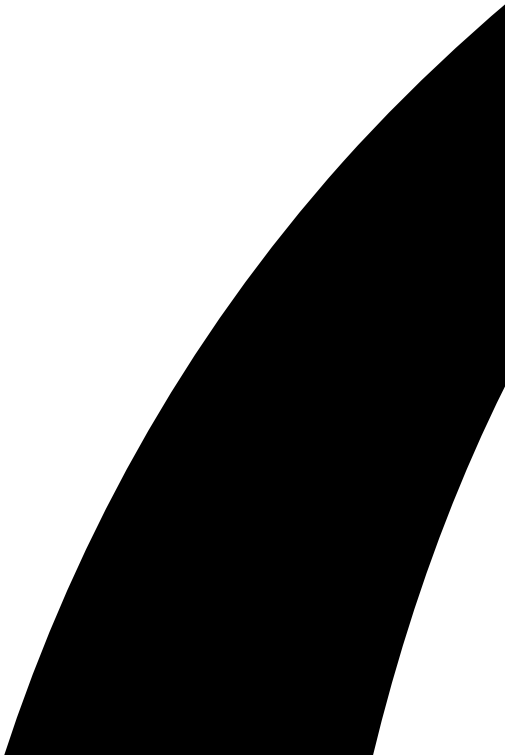


mcdi

mcdx

mcdx





mcdx

mcdx

mcdx

mcdx

mcdx

mcdx





mcdx

mcdx



mcdx



mcdx

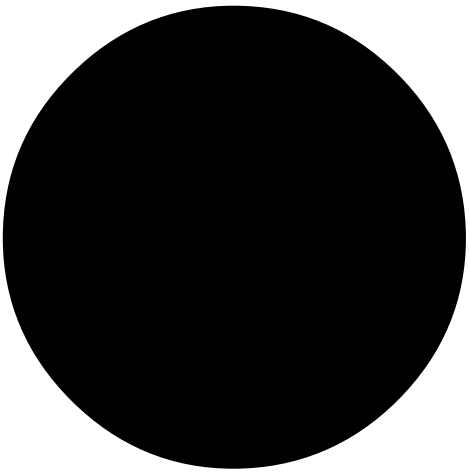


mcdx

mcdx

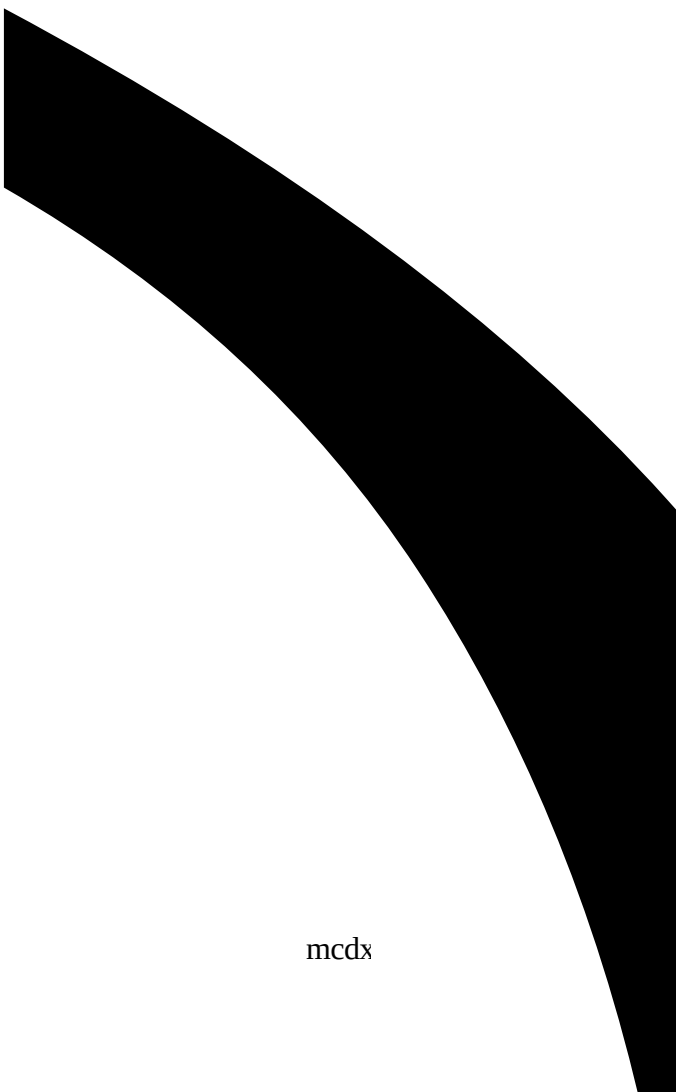
mcdx



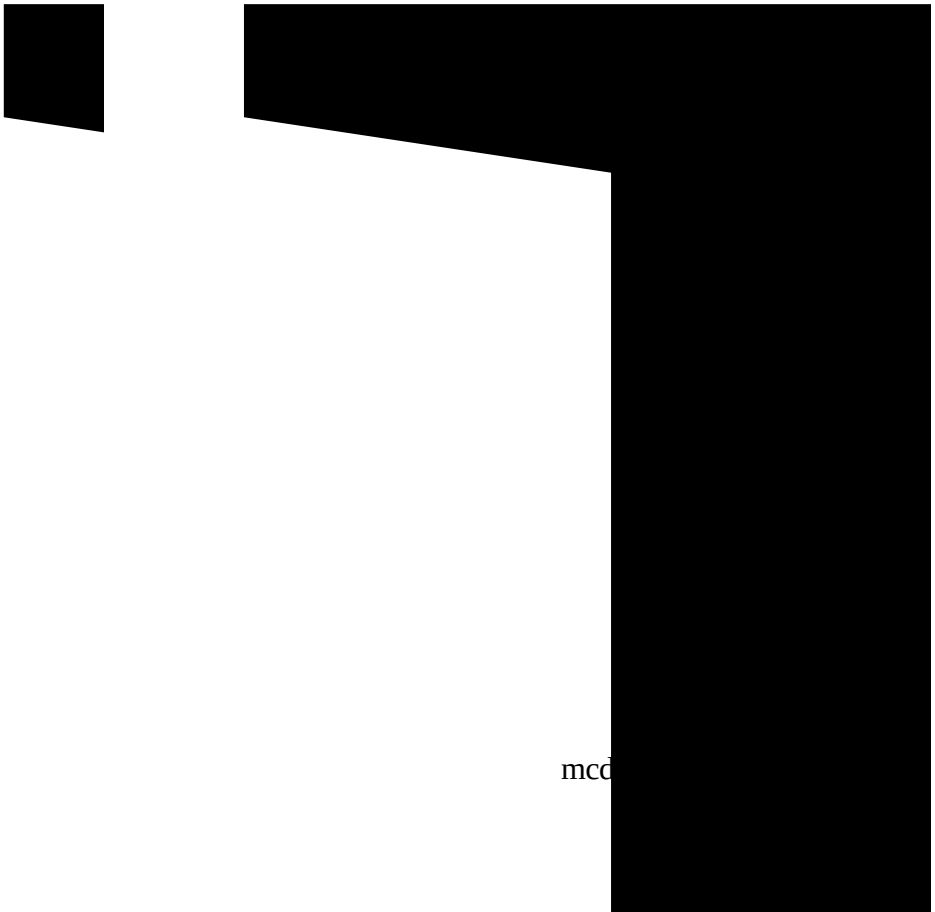


mcdx

mcdx



mcdx



mcd

mcdx

mcdx

mcdx

mcdx

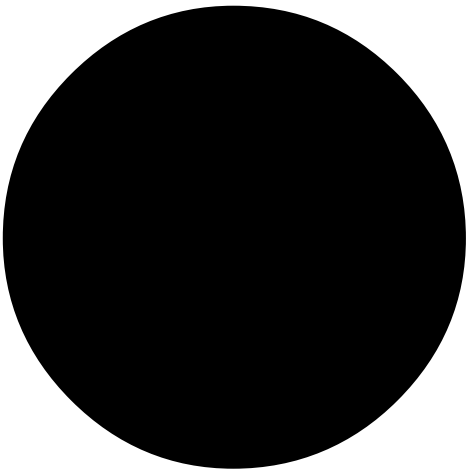


mcdx

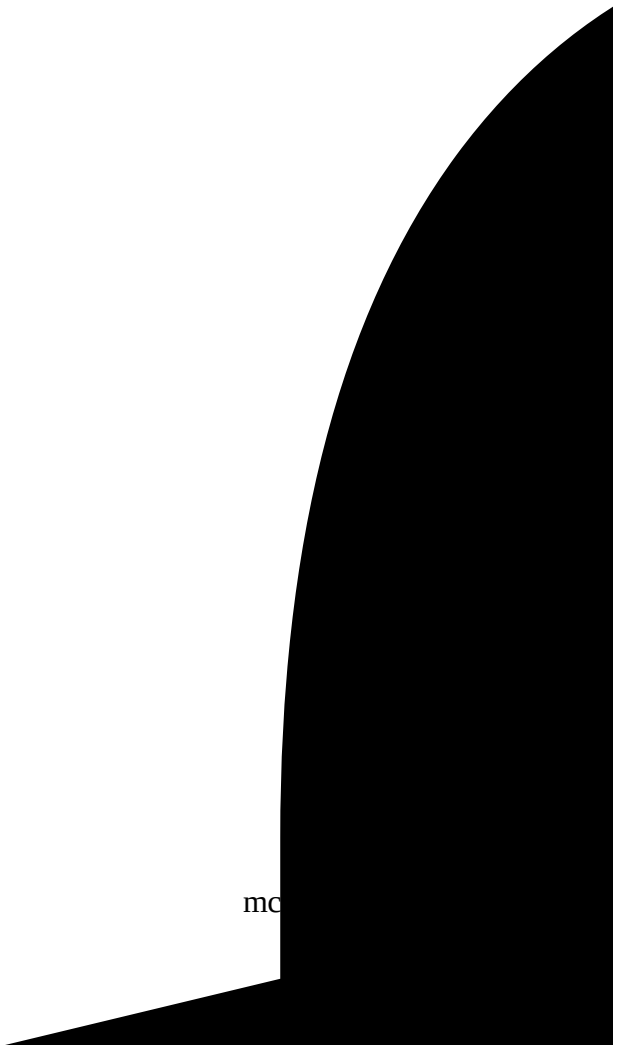


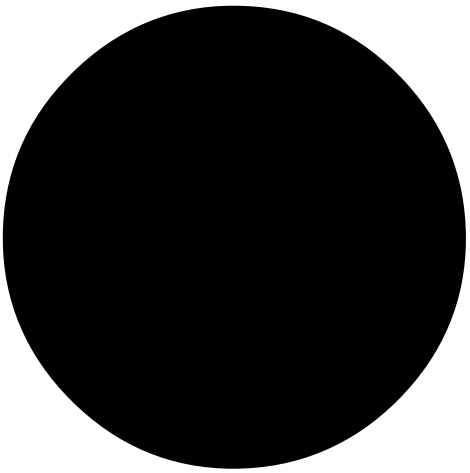
mcdx

mcdx



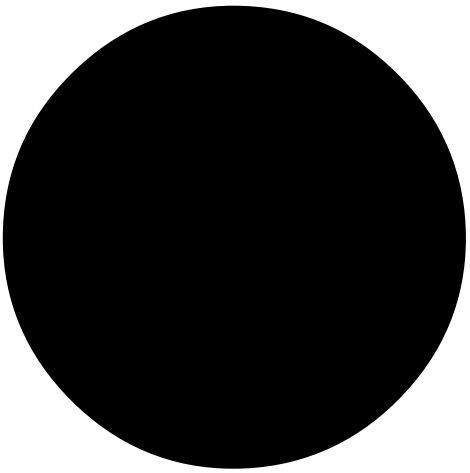
mcdx





mcdx

mcdx



mcdx

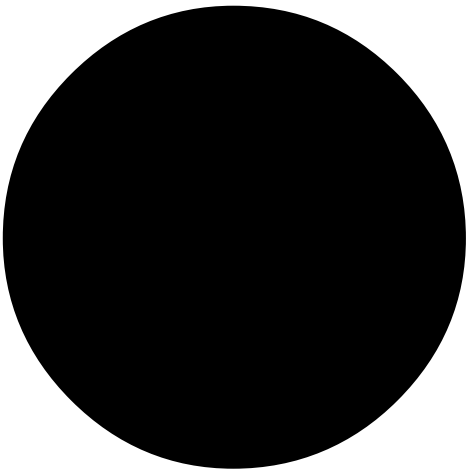
mcdx



mcdx

mcdl





mcdli

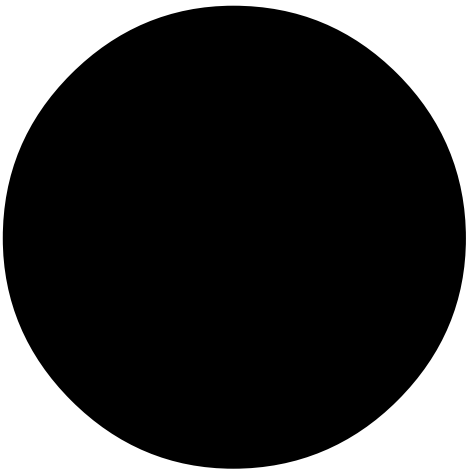


mcdli









mcdl







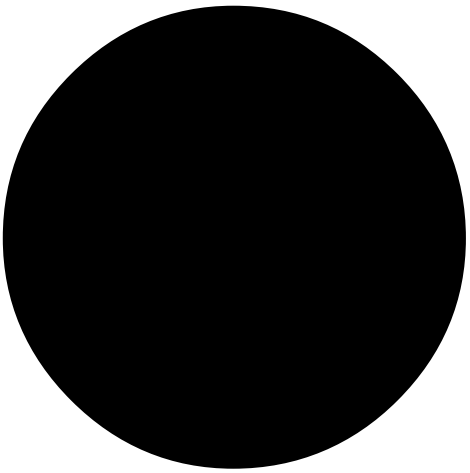






mcdl

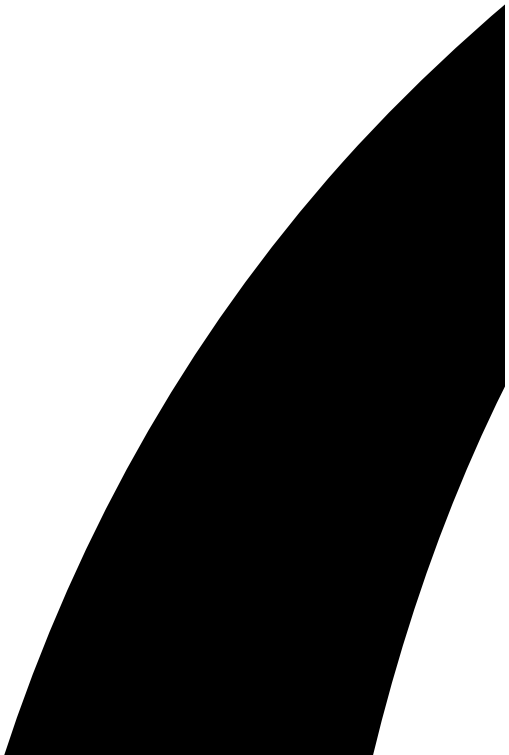




mcdl:

mcdl

mcdl



mcdl

mcdl

mcdl

mcdl



mcdl:



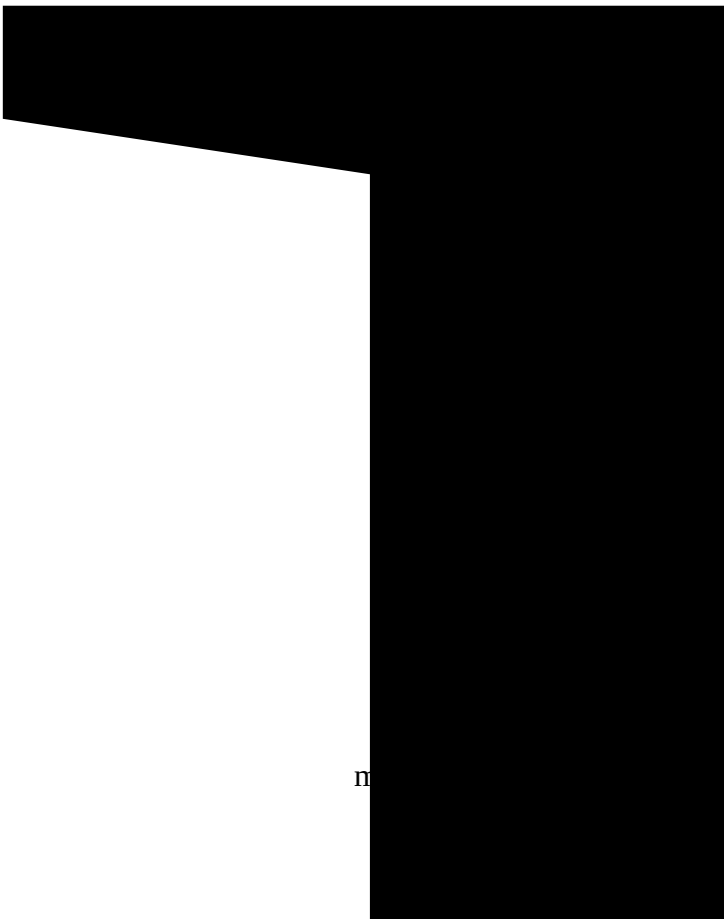


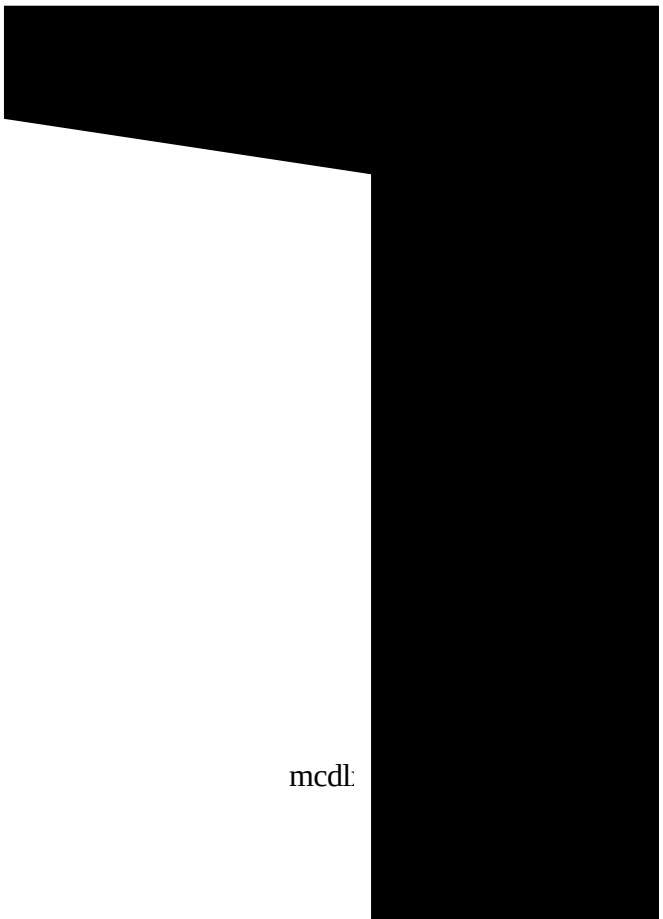
mcdl:



mcdl







mcdl



mcdl

mcdl

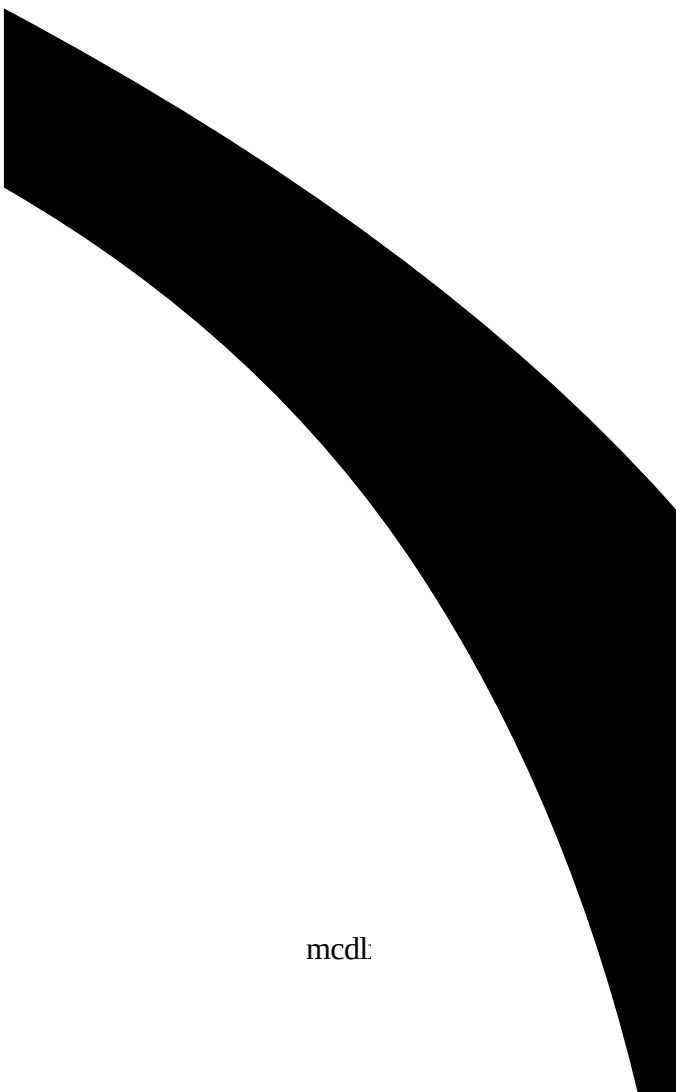


mcdl:

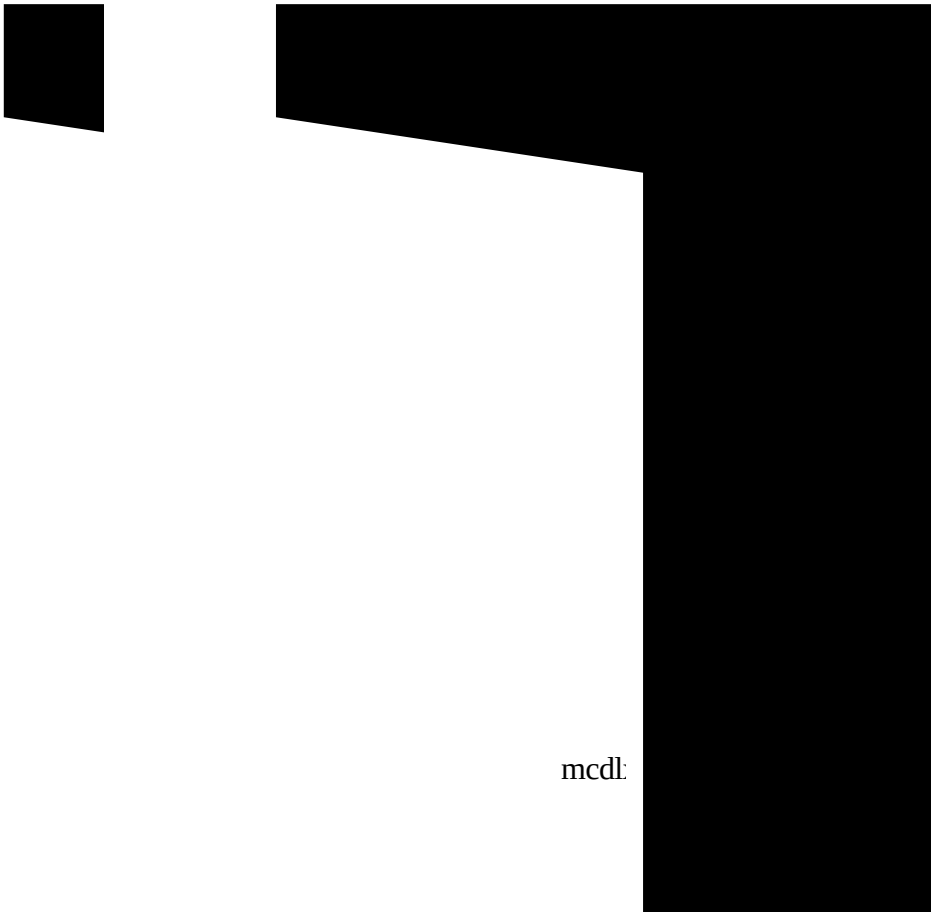


mcdl:





mcdl



mcdl

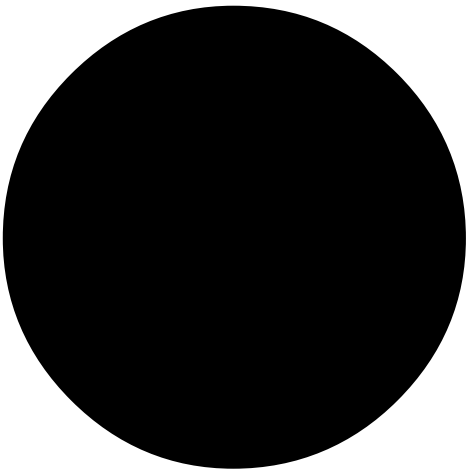
mcdx





mcdx

mcdx



mcdx

mcdx

mcdx

mcdx

mcdx

mcdx

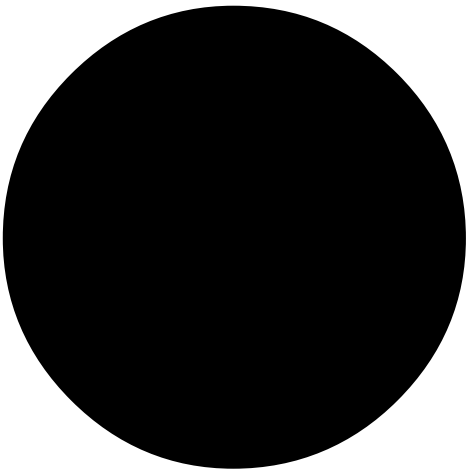




mdi







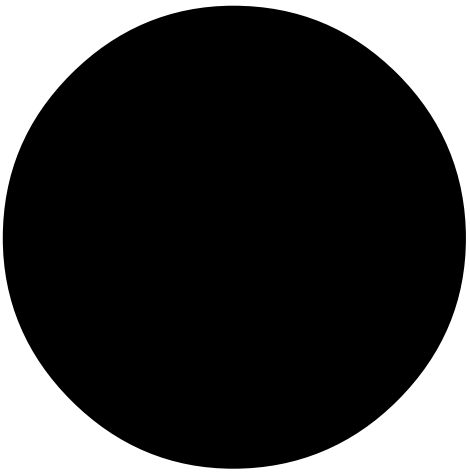
mdiv



mdv







mdvi



mdvi





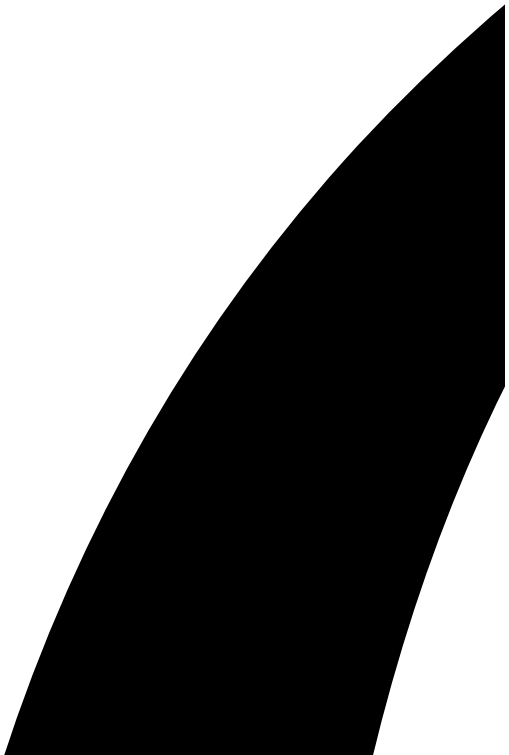
mdx











mdxx

mdx

mdxx

mdxx

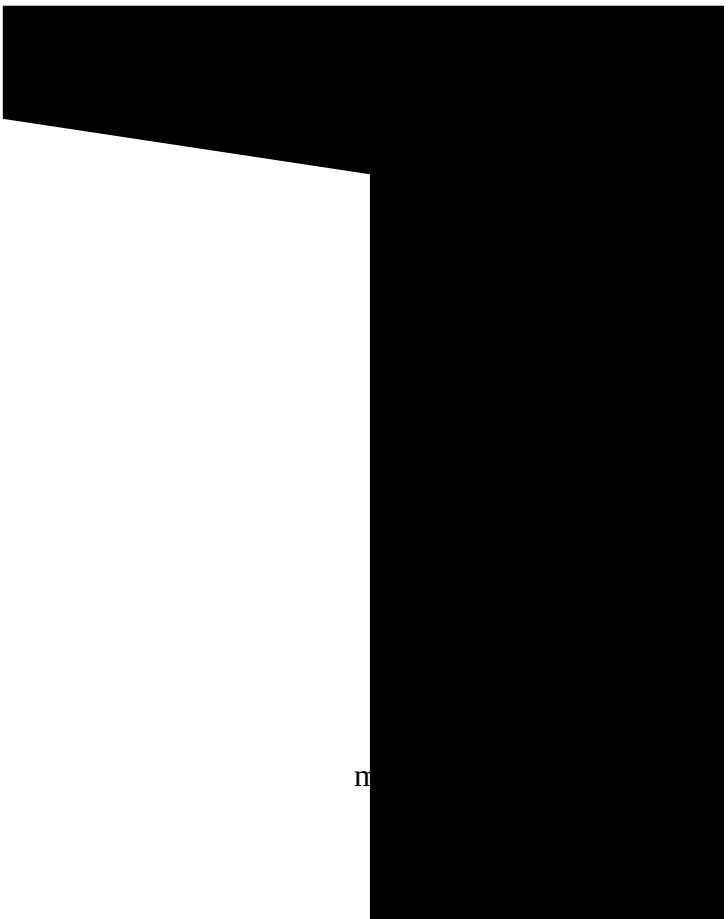


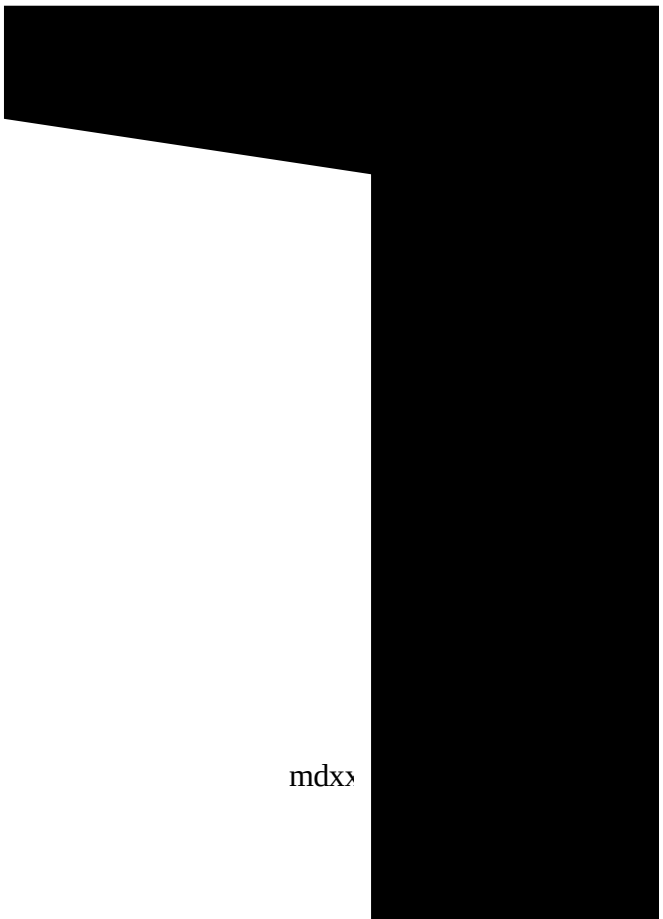
mdx



mdx







mdx

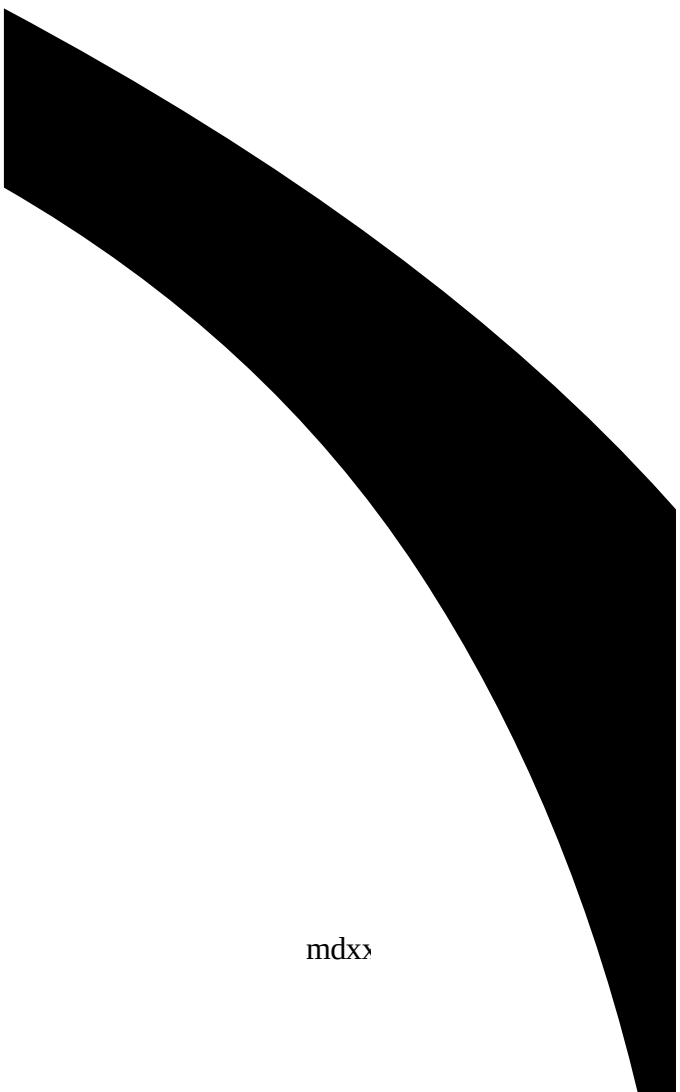
mdx

mdxx

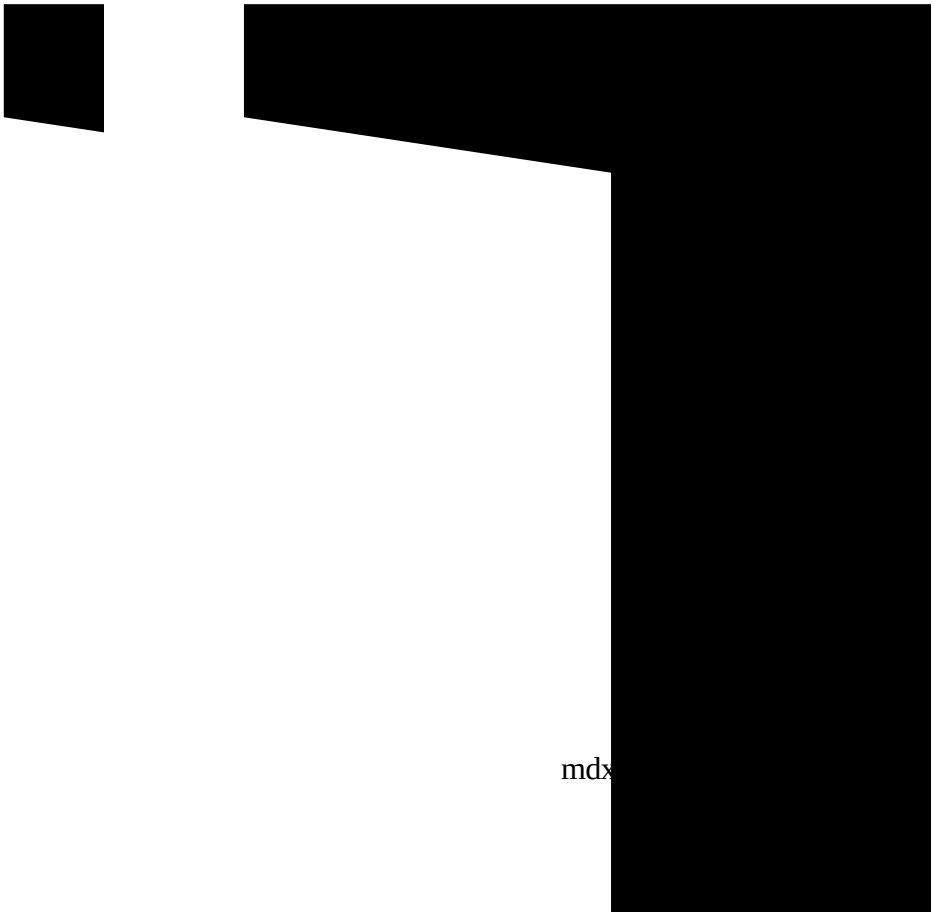
mdxx



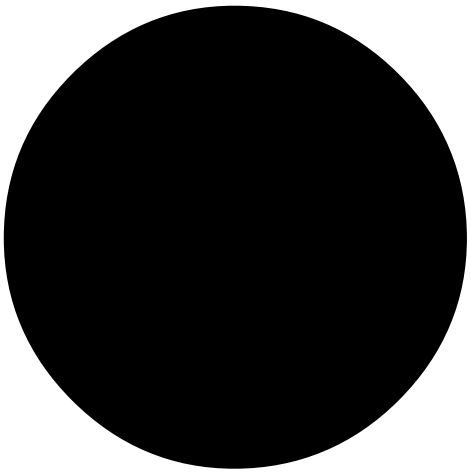
mdx



mdx›



mdx



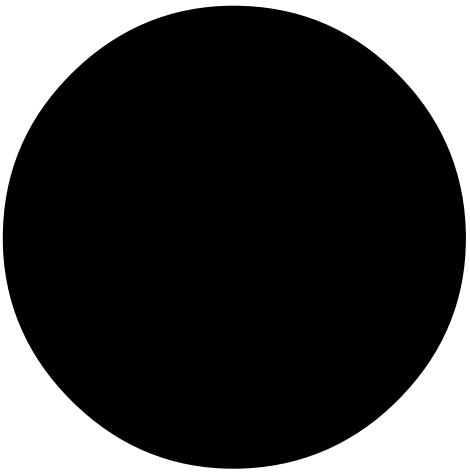
mdx


mdxl

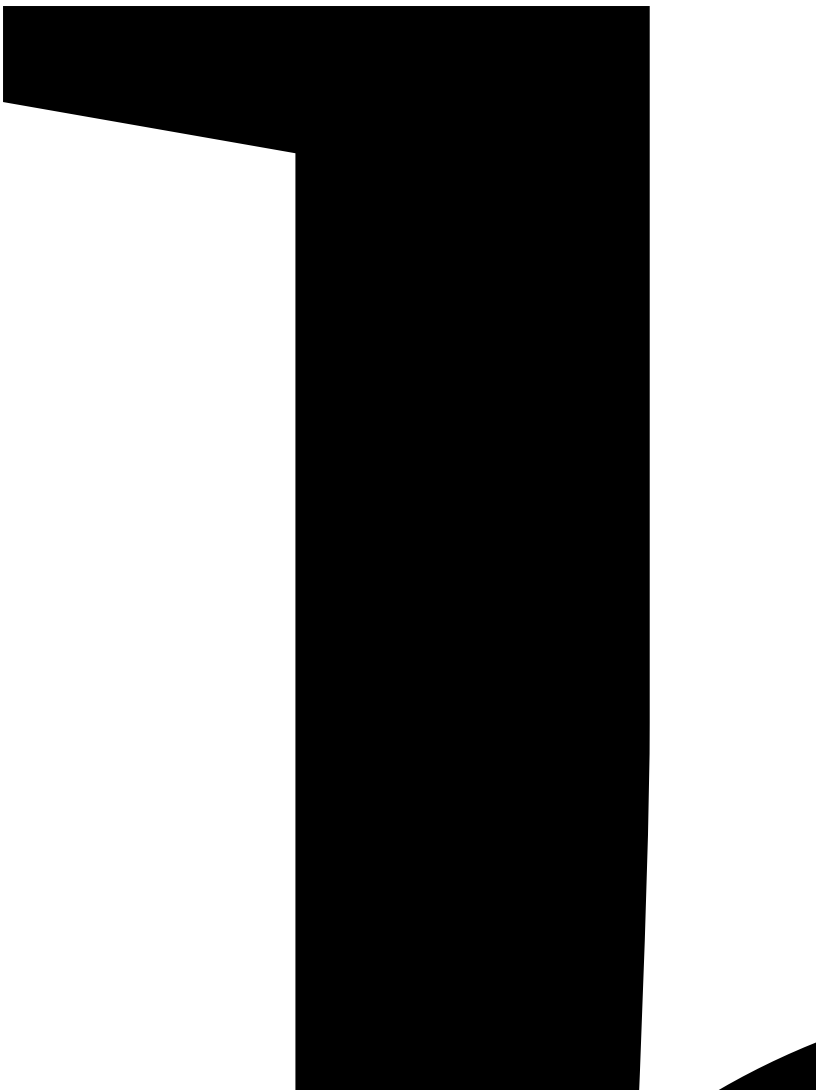


mdxl



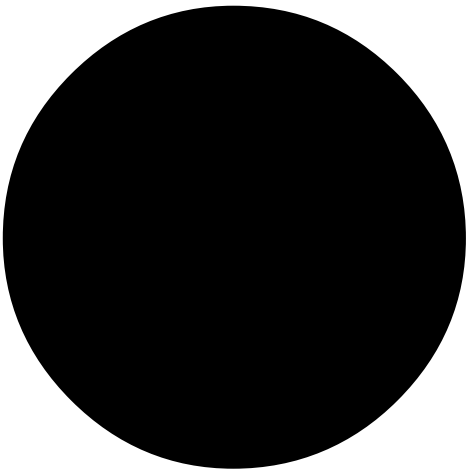


mdxl



mdxl



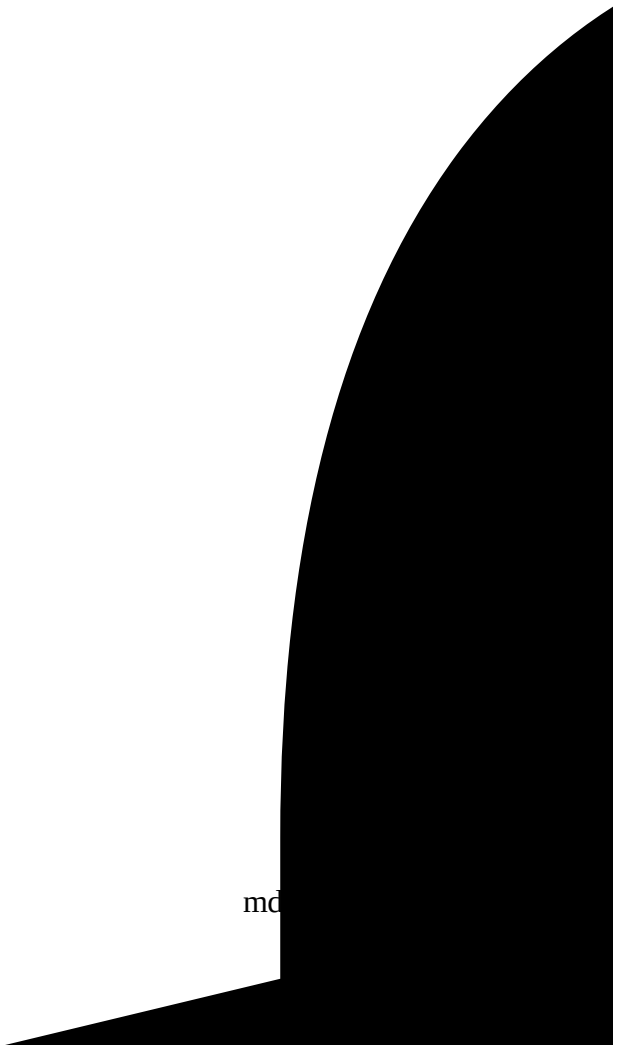


mdxl

mdxl

mdxl

mdl

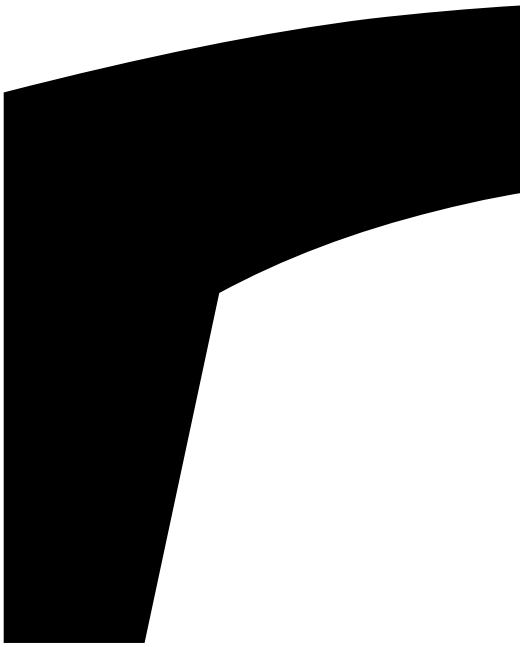


md

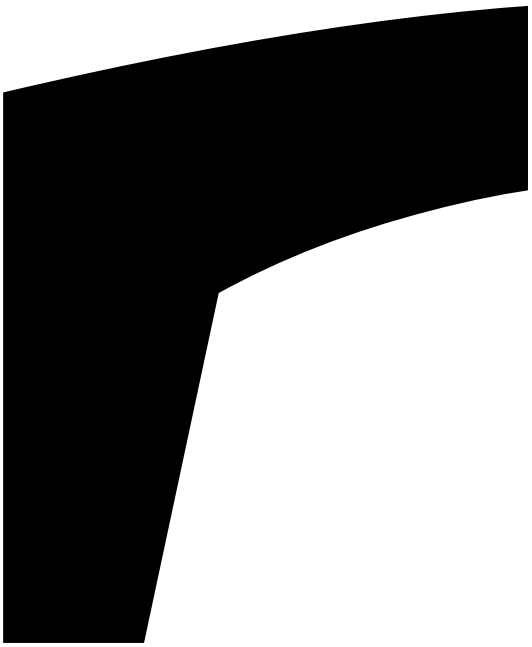


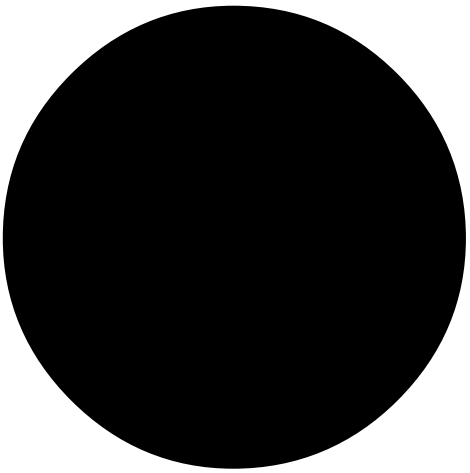


mdliv



mdlv





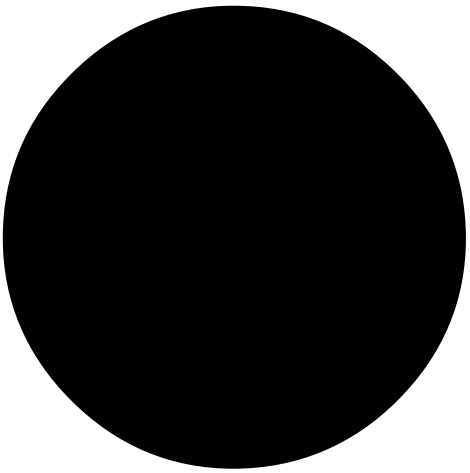
mdlv





mdlx

mdlx



mdlx

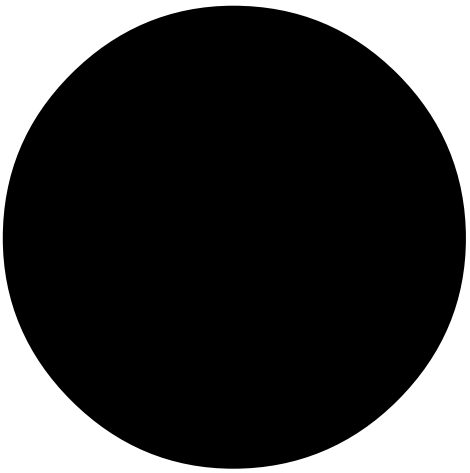
mdlx

mdlx

mdlx

mdlx


mdlx



mdlx

mdlx



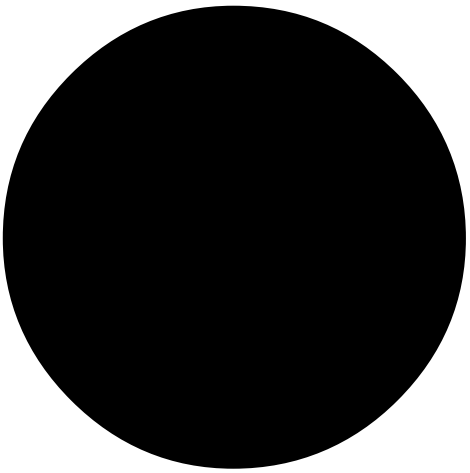
mdlx

mdlx

mdlx







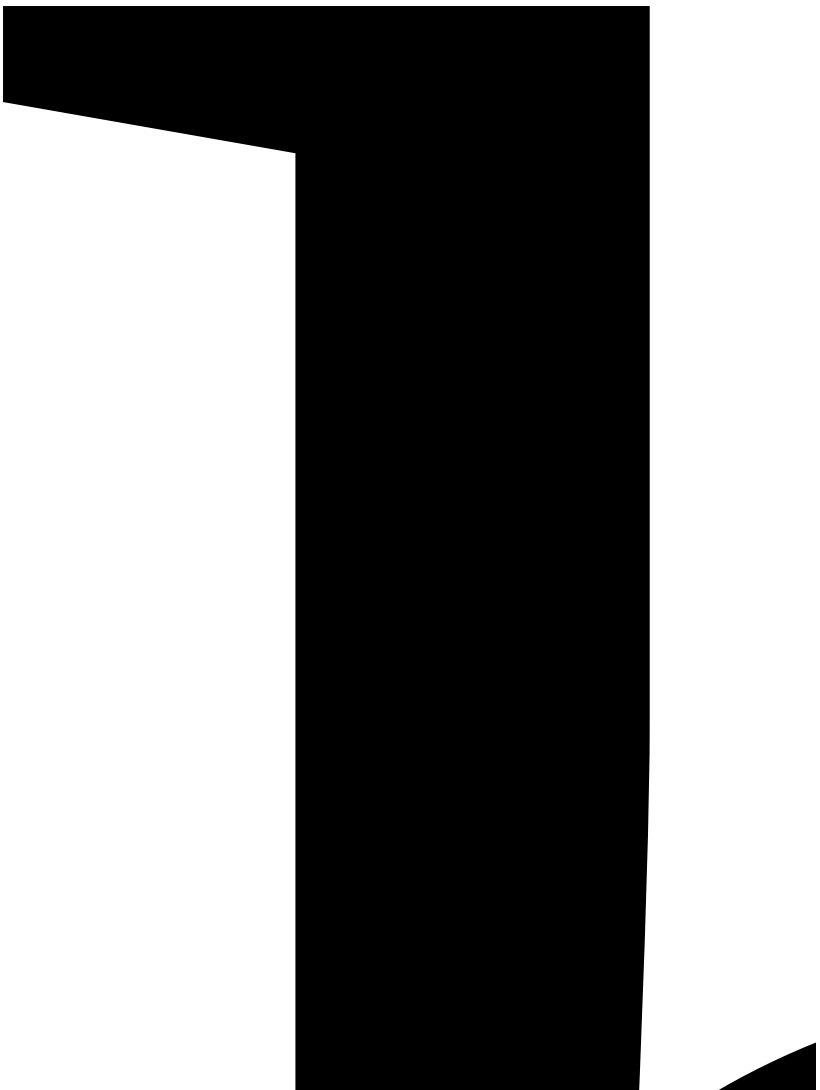
mdlx

mdlx

mdlx

mdlx





mdlx

mdlx

mdlx





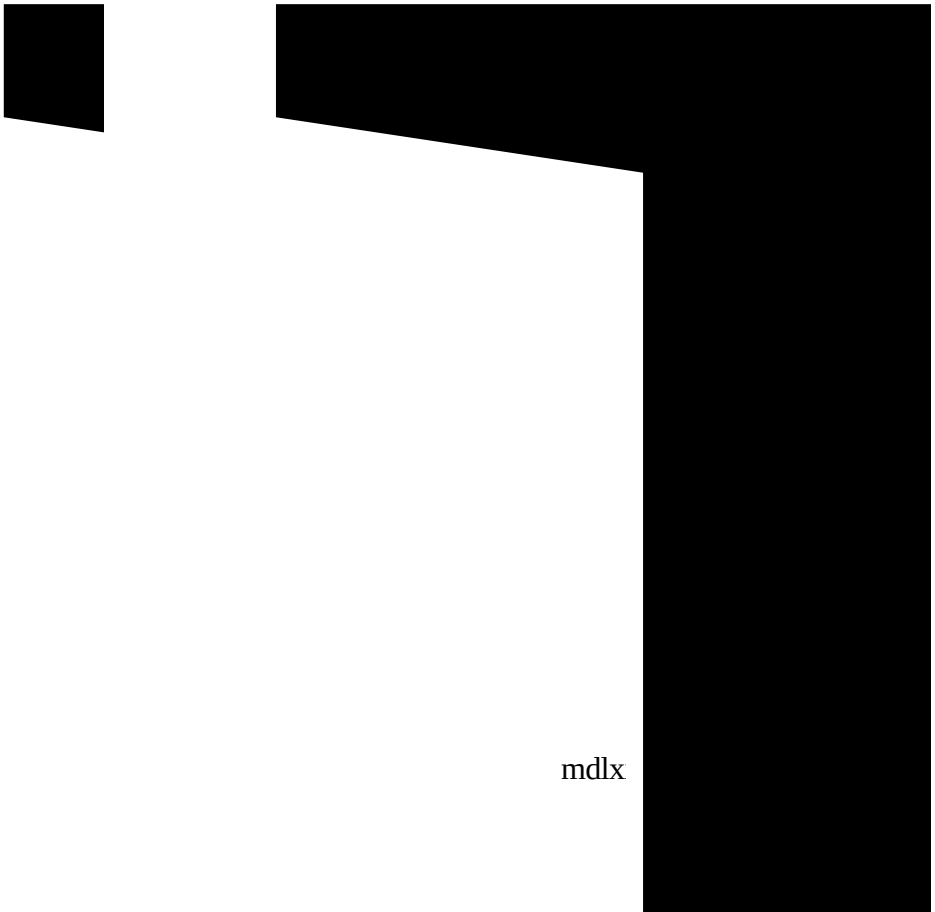
mdlx

mdlx

mdlx

mdlx

mdlx



mdlx

mdlx

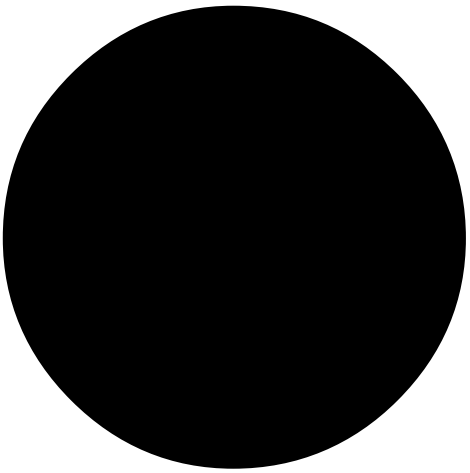




mdxc



mdxc



mdxc

mdxc



mdxc

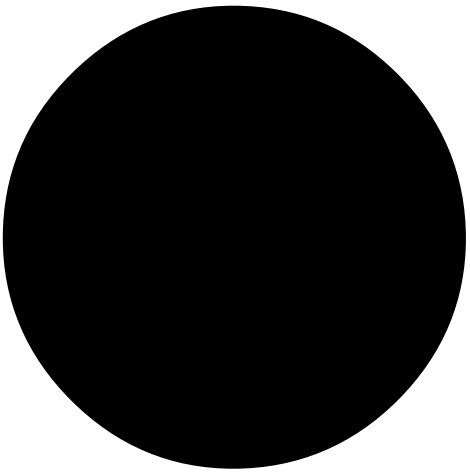
mdxc

mdxc



mdxc





mdxc







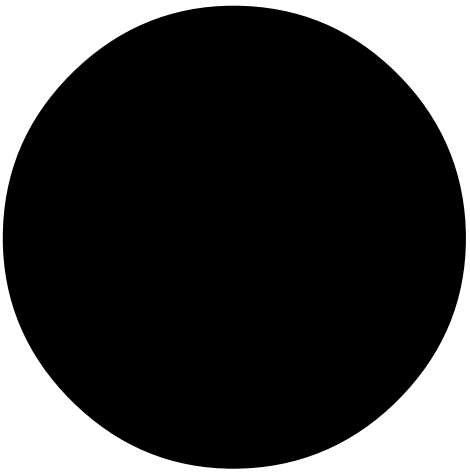


mdcv



mdcv



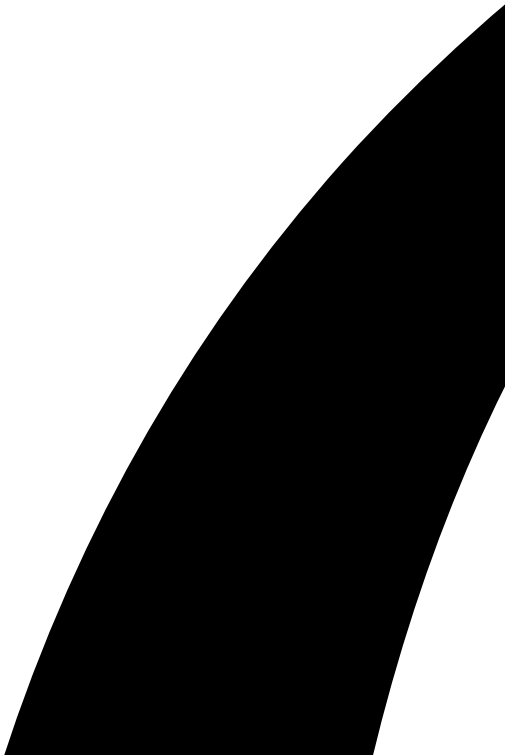


mdcv























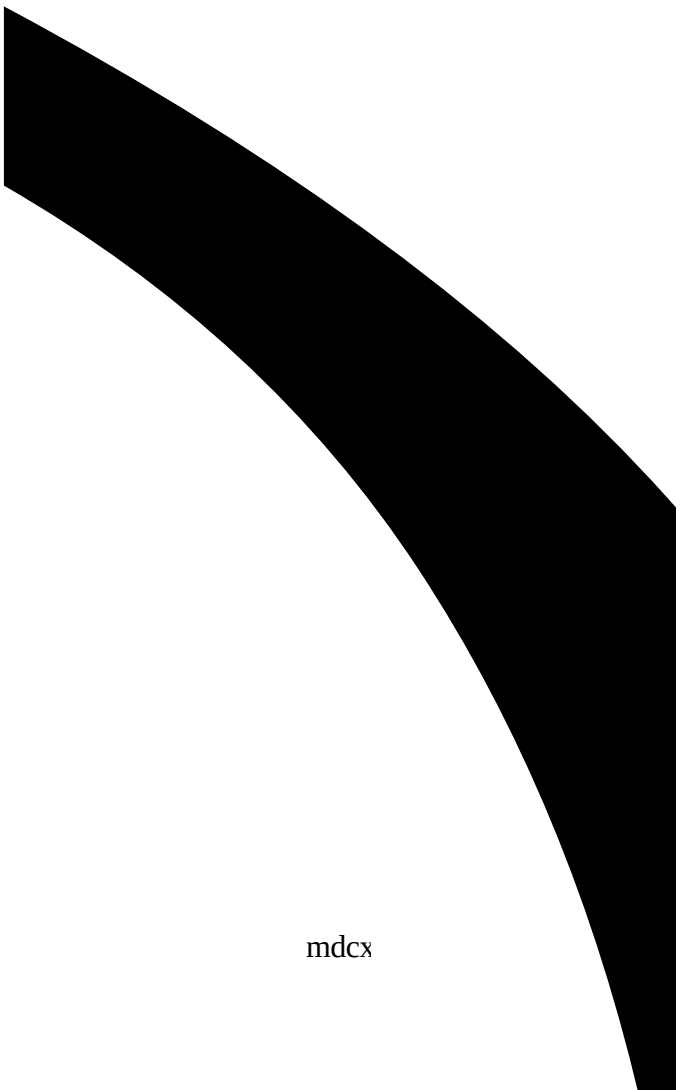














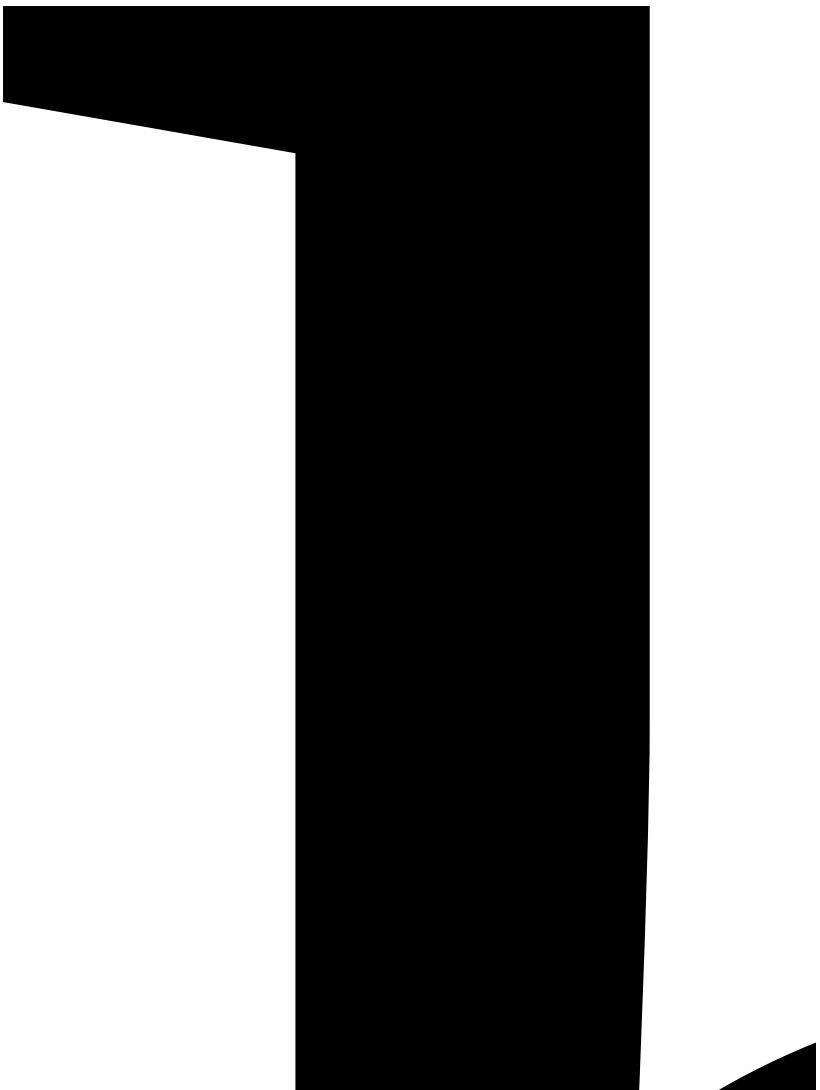






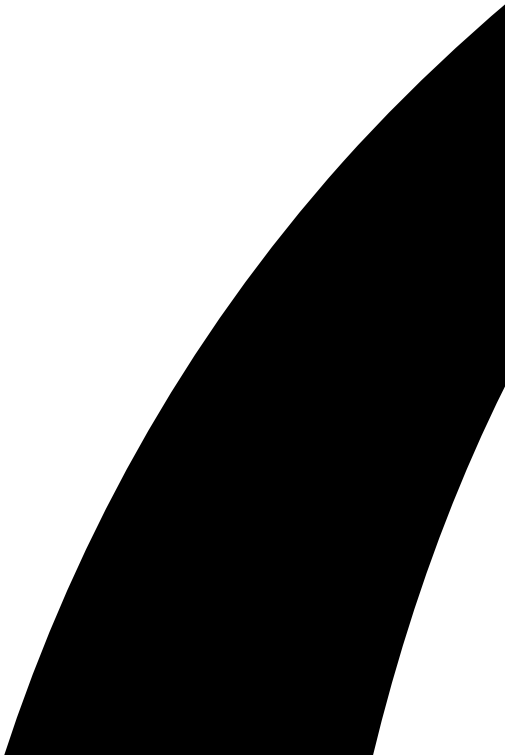






mdcl





mdcl

mdcl

mdcl

mdcl

mdcl



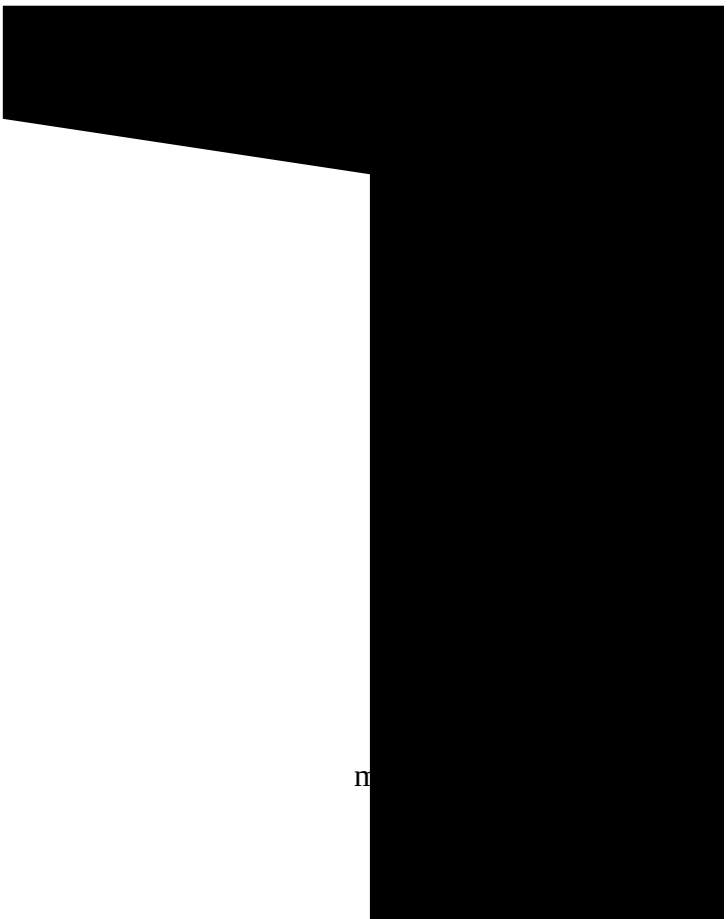


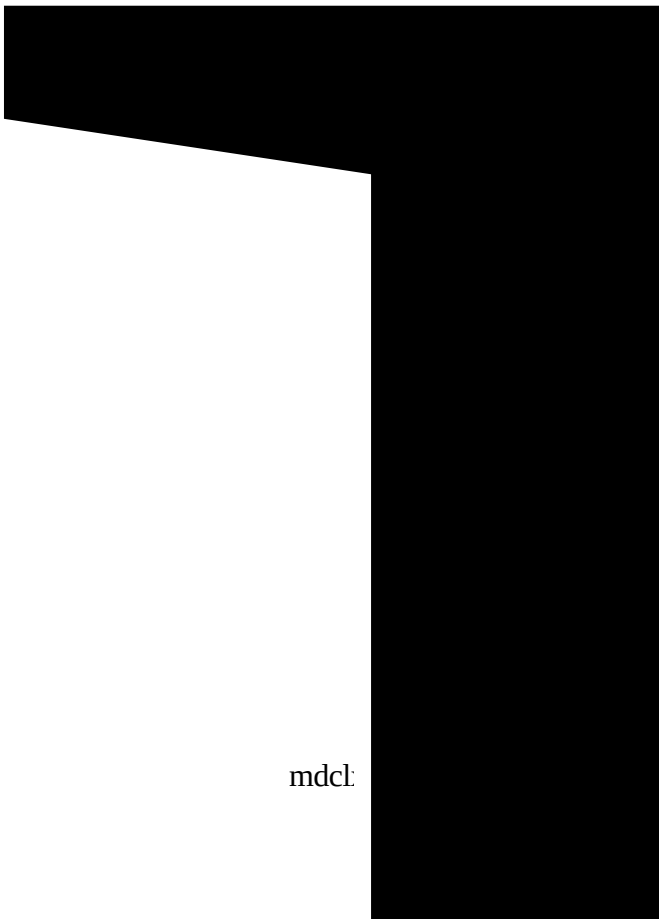
mdcl

mdcl:









mdcl

mdcl

mdcl

mdcl

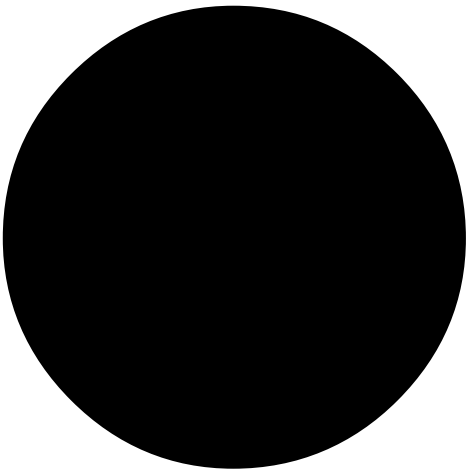


mdcl

mdcl:



mdcl



mdcl





mdcl:



mdcl

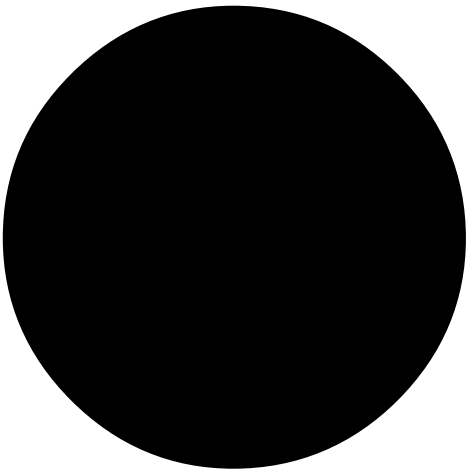
mdcl





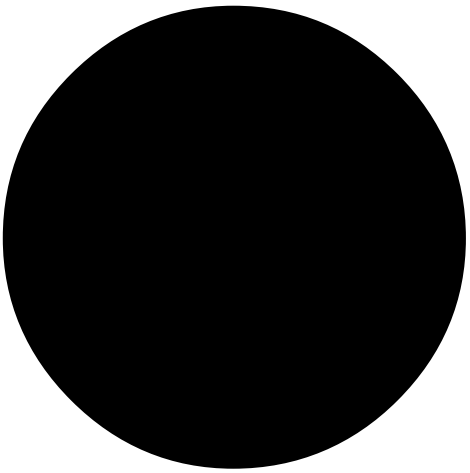
mdcl





mdcl

mdcl



mdcl





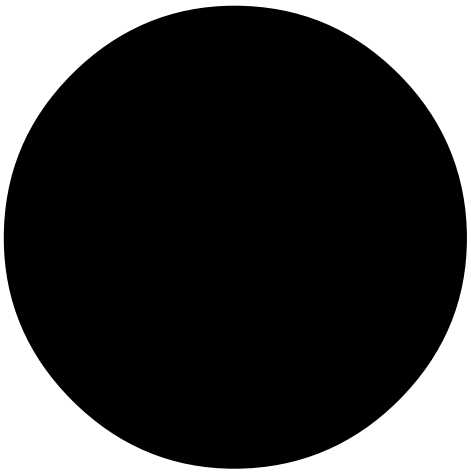
mdcl

mdcl:









mdcx



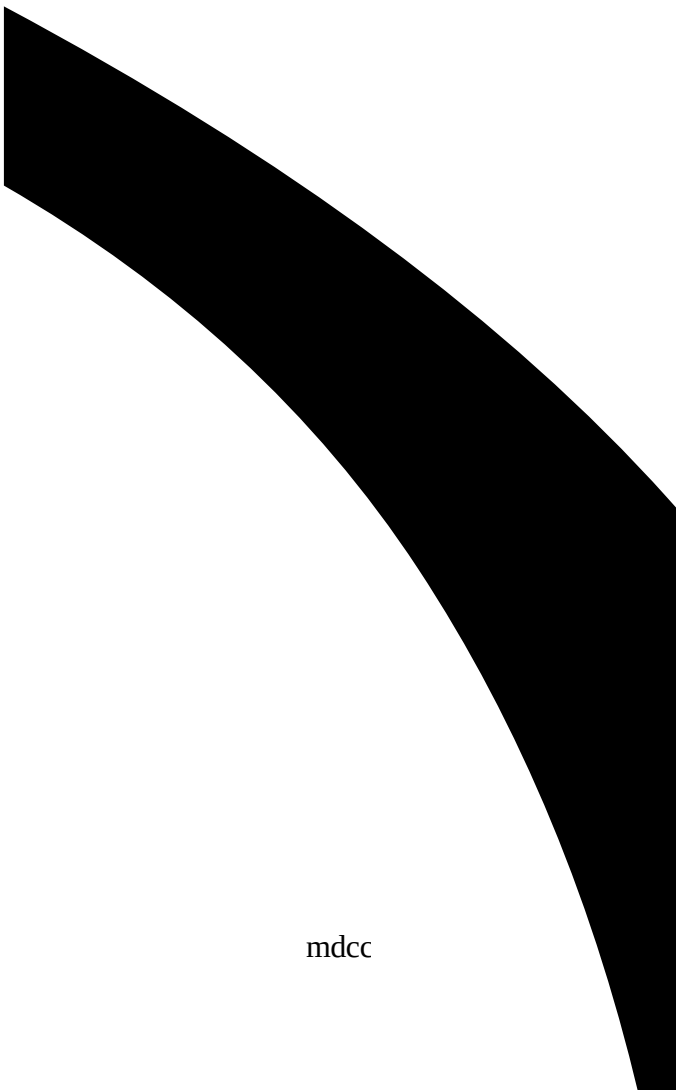


mdcc

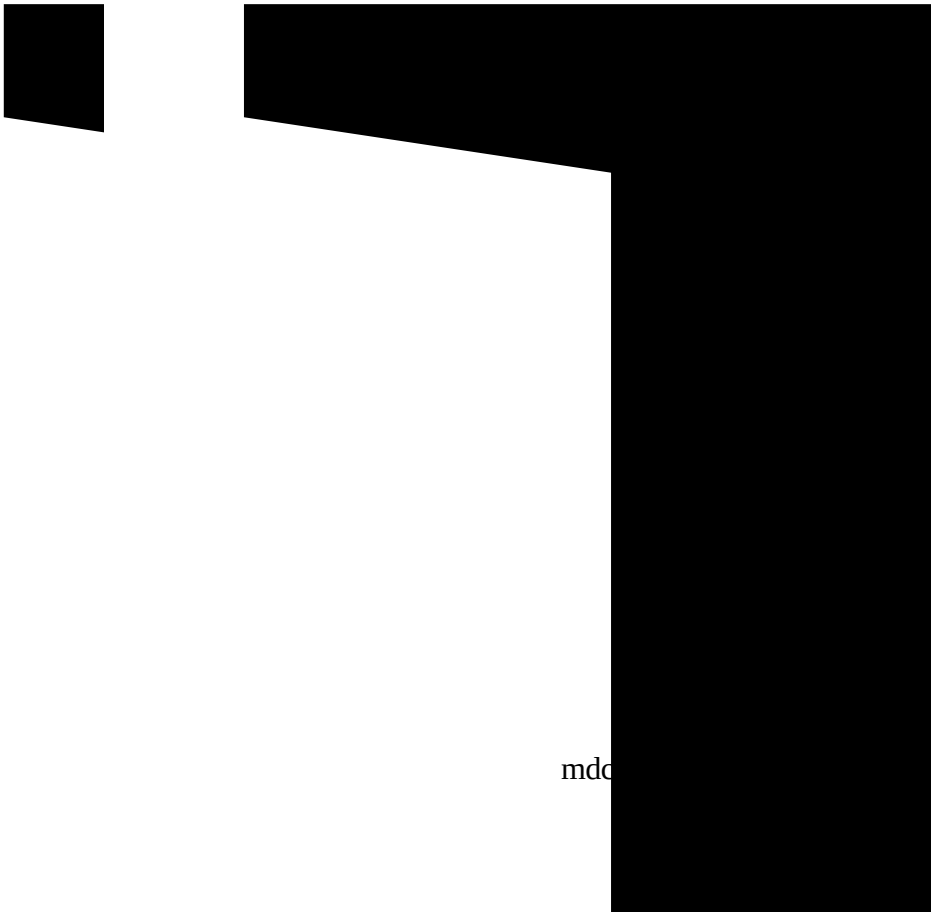


mdcc





mdcc



mdcc

[redacted]

mdcc



mdcc



mdcc





mdcc



mdcc

mdcc



mdcc





mdcc

mdcc

mdcc





mdcc



mdcc



mdcc



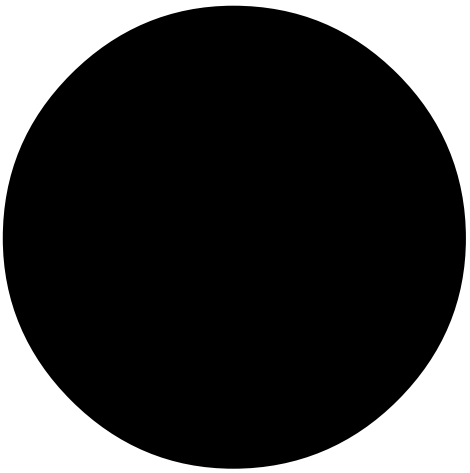
mdcc



mdcc

[redacted]

mdcc



mdcc

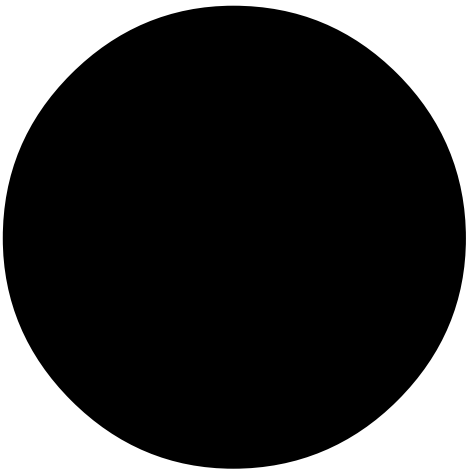
mdcc



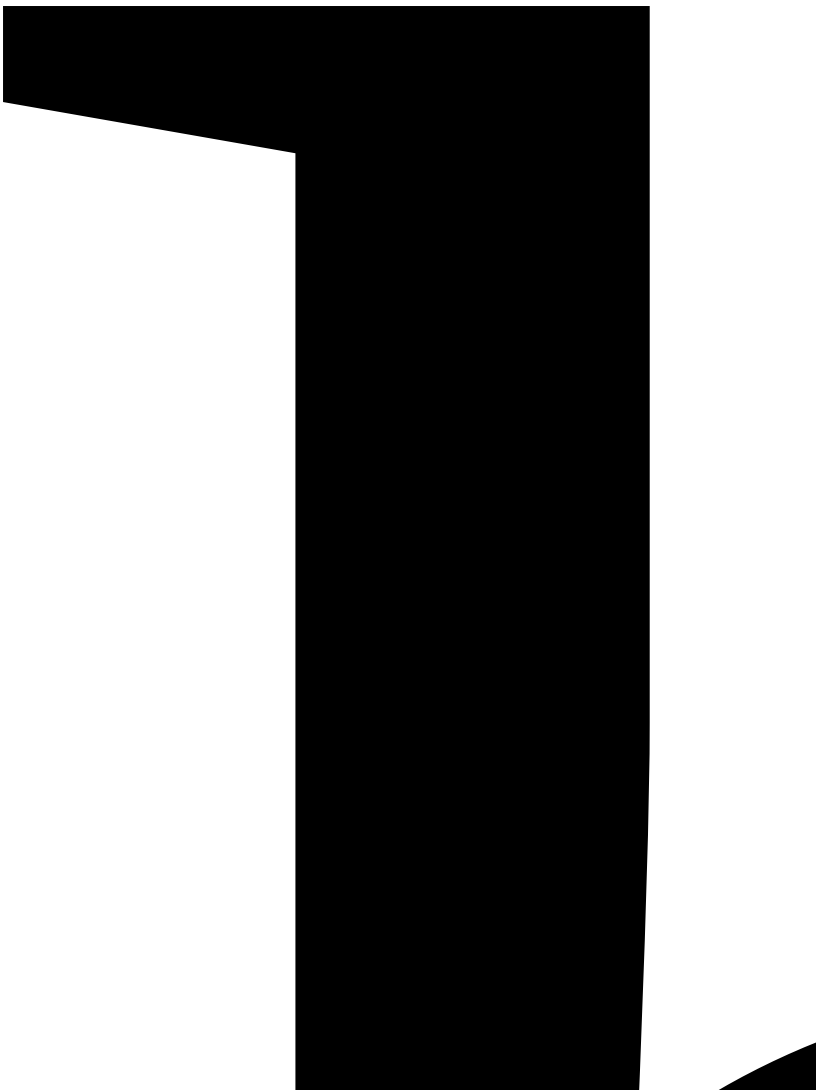


mdcc





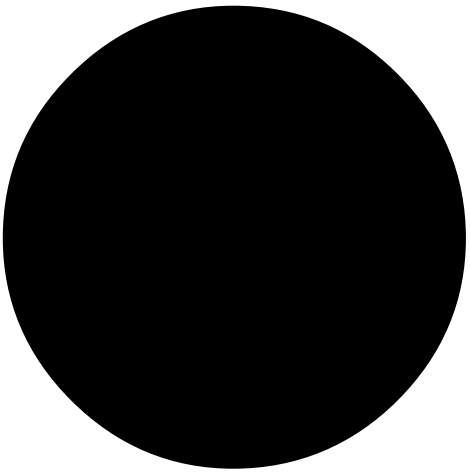
mdcc



mdcc







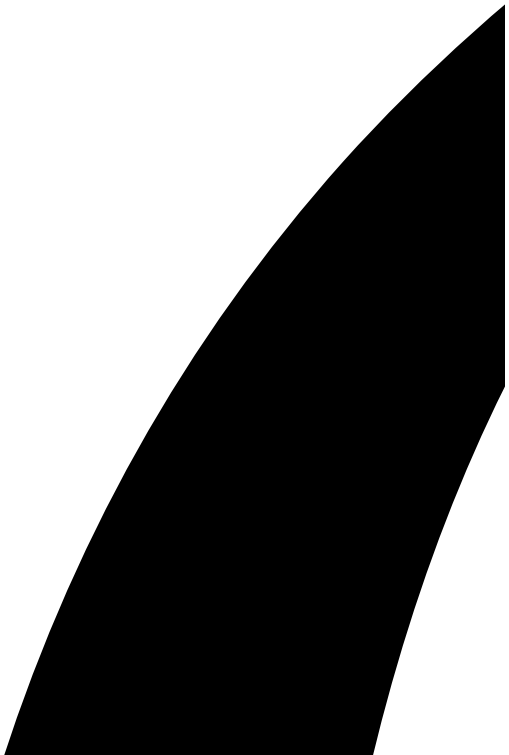
mdcc

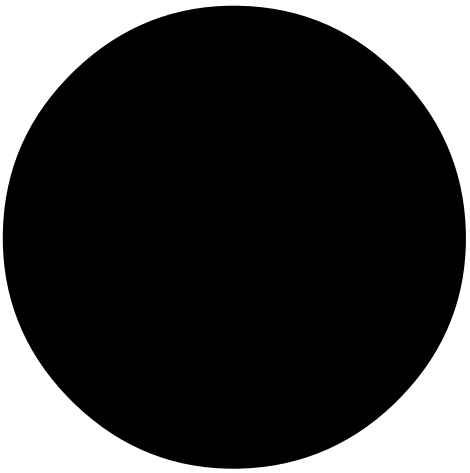
mdcc



mdcc







mdcc





mdcc



mdcc





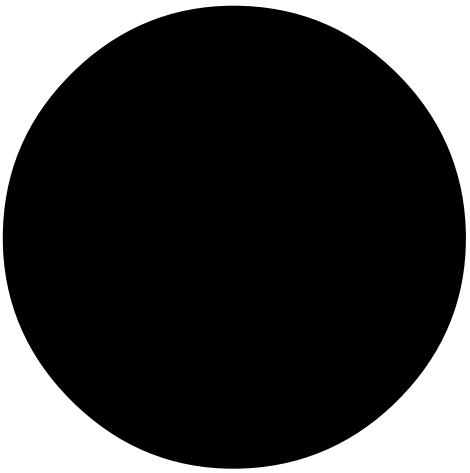
mdcc



mdcc







mdcc

mdcc

mdcc



mdcc





mdcc



mdcc



mdcc



mdcc



mdcc

mdcc



mdcc





mdcc

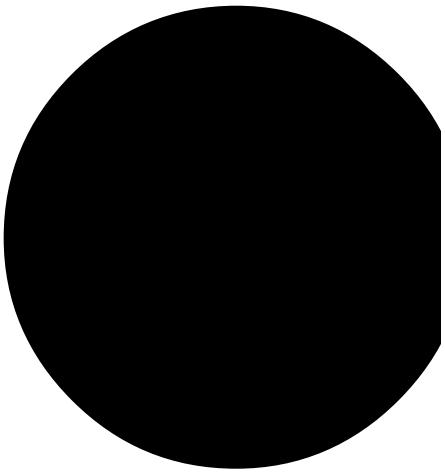


mdcc



mdcc





mdcc

mdcc



mdcc



mdcc

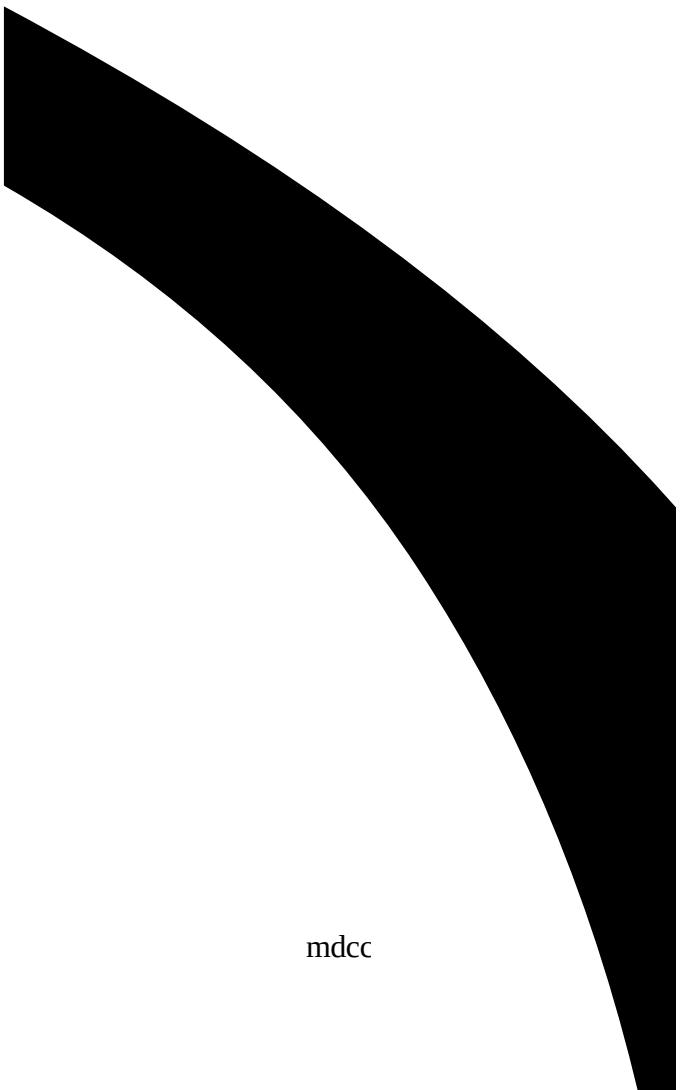
mdcc





mdcc





mdcc



mdcc



mdcc





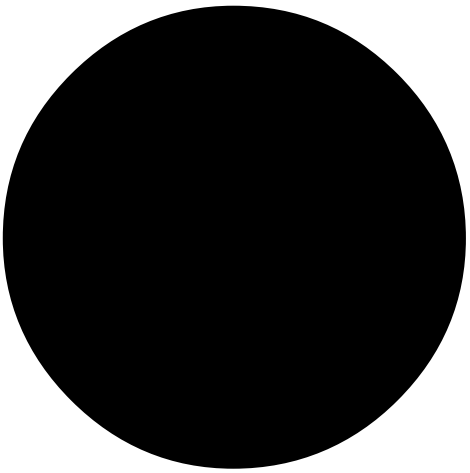
mdcc



mdcc







mdcc

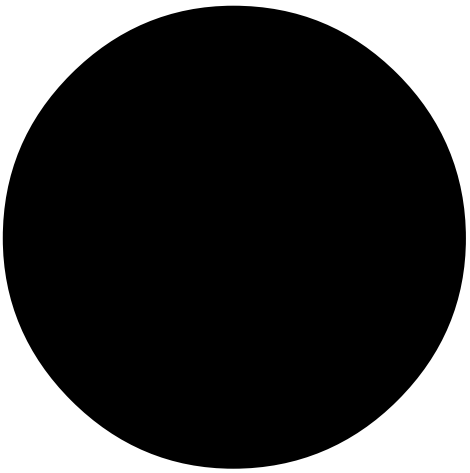
mdcc



mdcc







mdcc

mdcc



mdcc



mdcc



mdcc

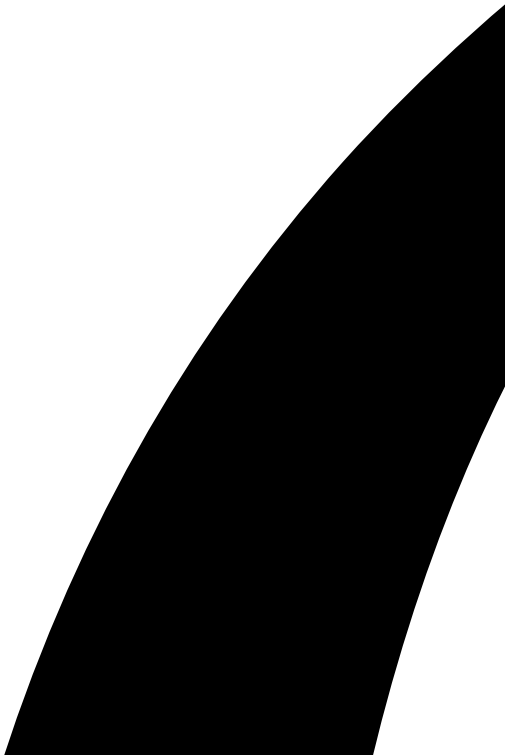


mdcc



mdcc





mdcc



mdcc

mdcc



mdcc



mdcc



mdcc



mdcc





mdcc





mdcc





mdcc



mdcc

mdcc





mdcc





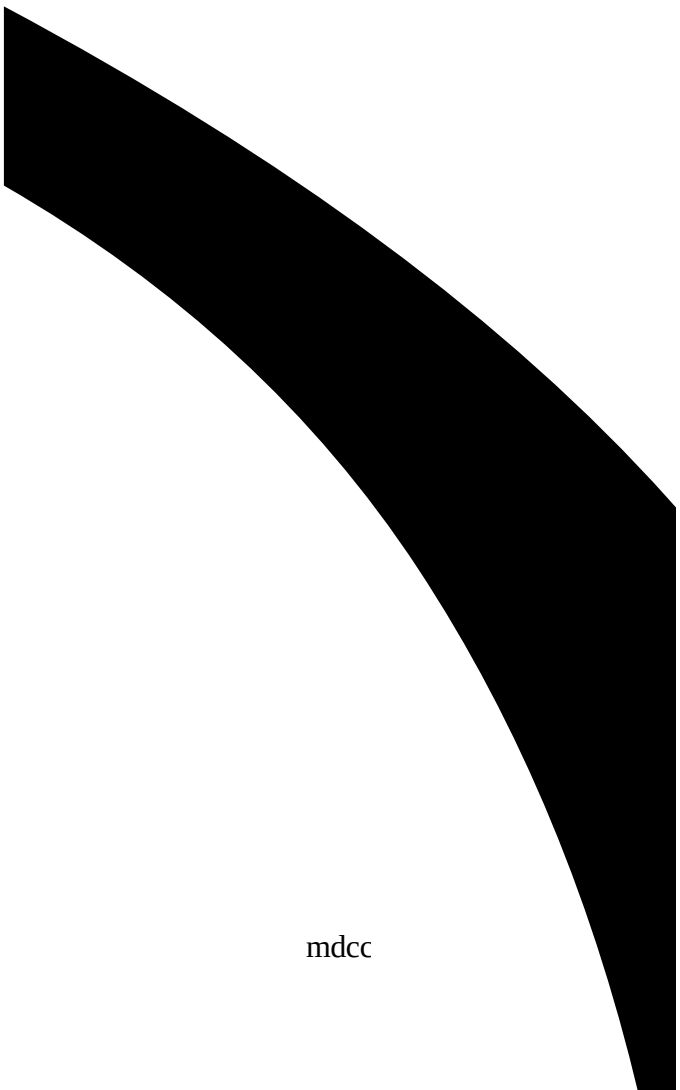
mdcc

mdcc

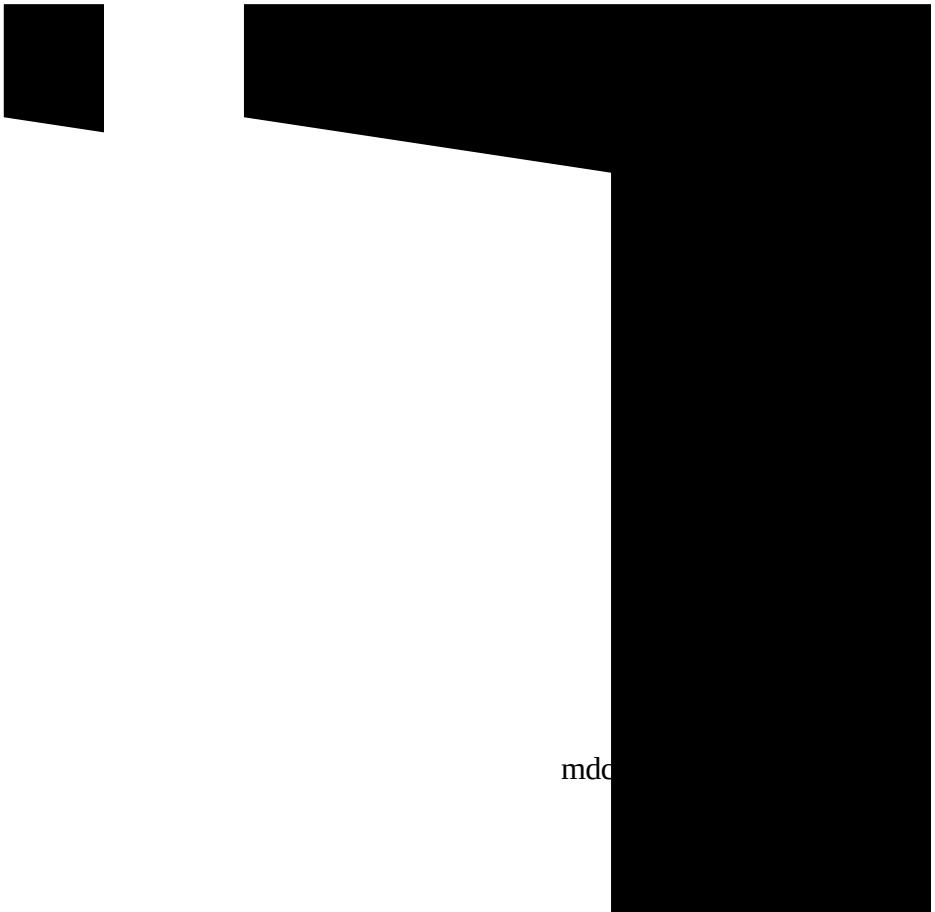


mdcc





mdcc



mdcc

[redacted]

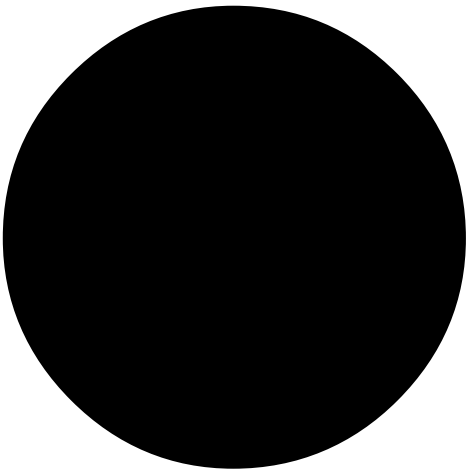


mdcc



mdcc





mdcc

mdcc



mdcc



mdcc

mdcc





mdcc



mdcc



mdcc



mdcc

mdcc



mdcc



mdcc

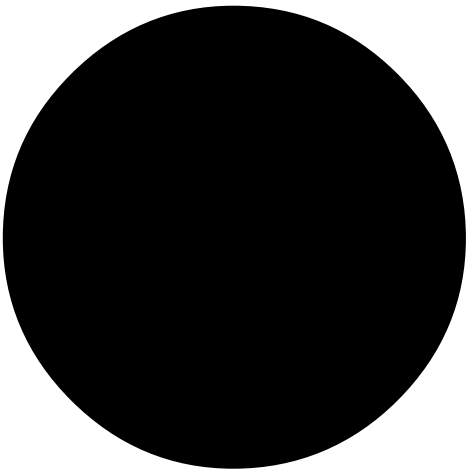


mdcc



mdcc





mdcc

mdcc

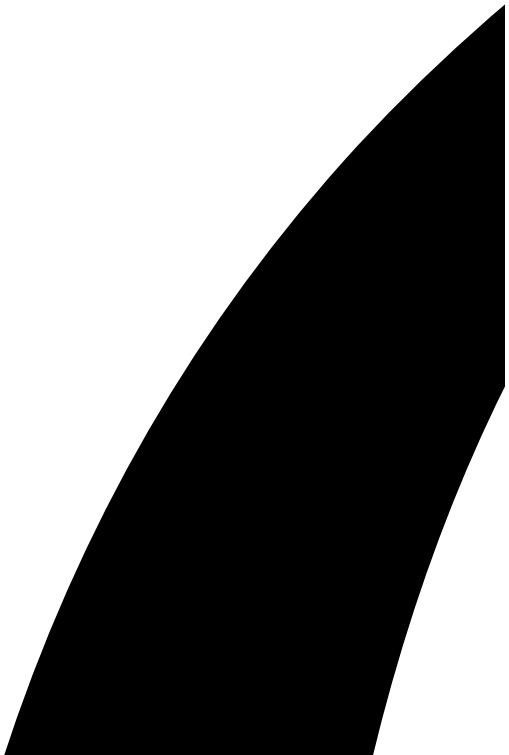


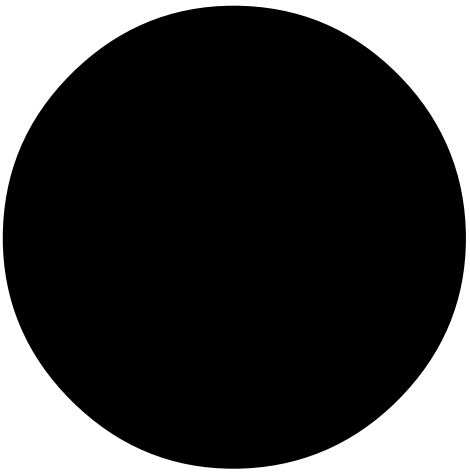
mdcc



mdcc







mdcc





mdcc



mdcc





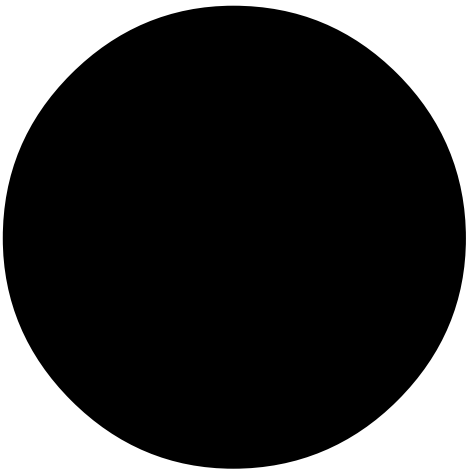
mdcc



mdcc







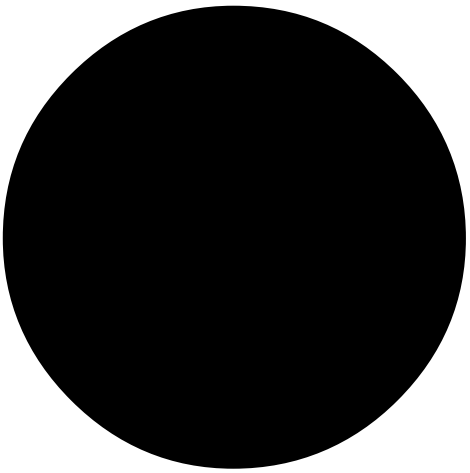
mdcc

mdcc

mdcc



mdcc



mdcc

mdcc



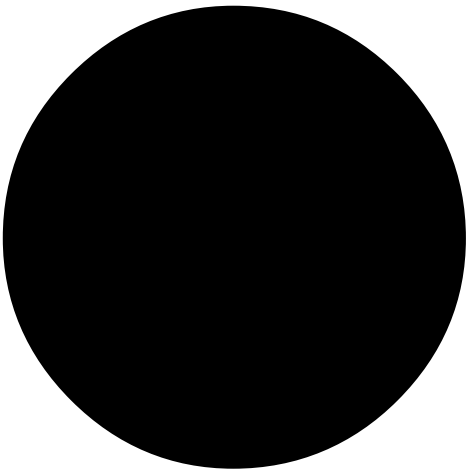
mdcc



mdcc







mdcc

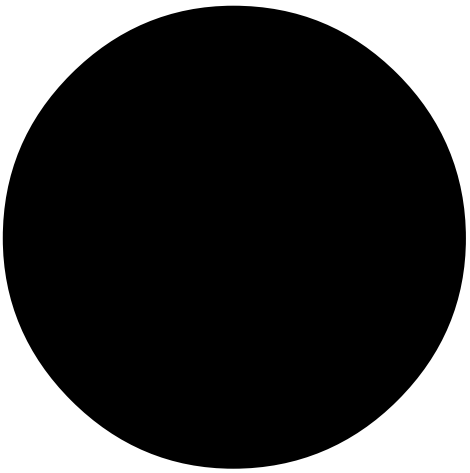
mdcc



mdcc







mdcc

mdcc



mdcc



mdcc



mdcc





mdcc



mdcc



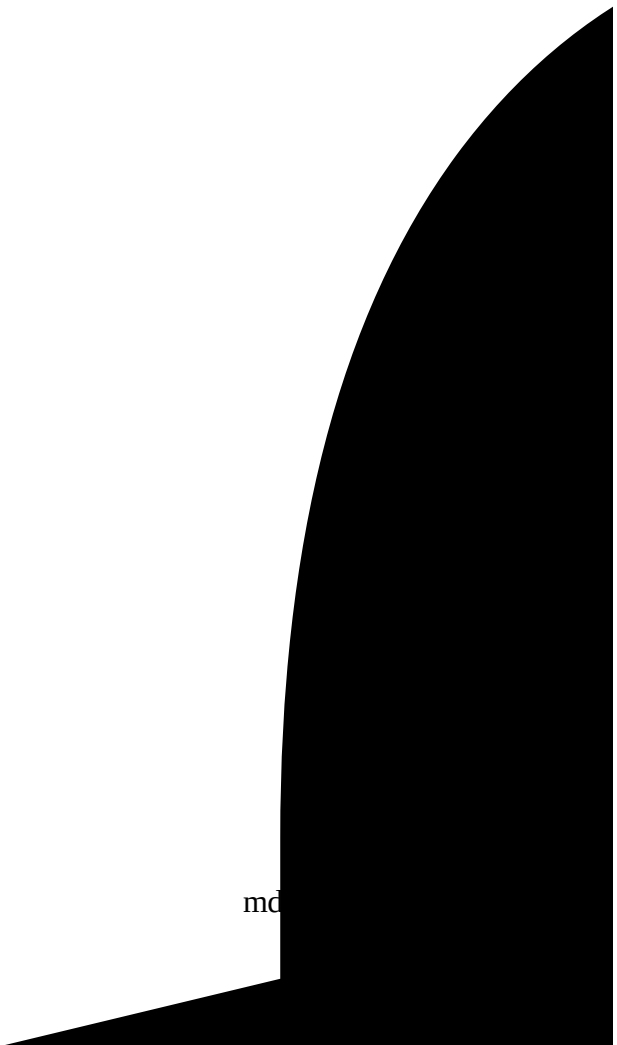
mdcc



mdcc



mdcc



md

mdcc



mdcc





mdcc

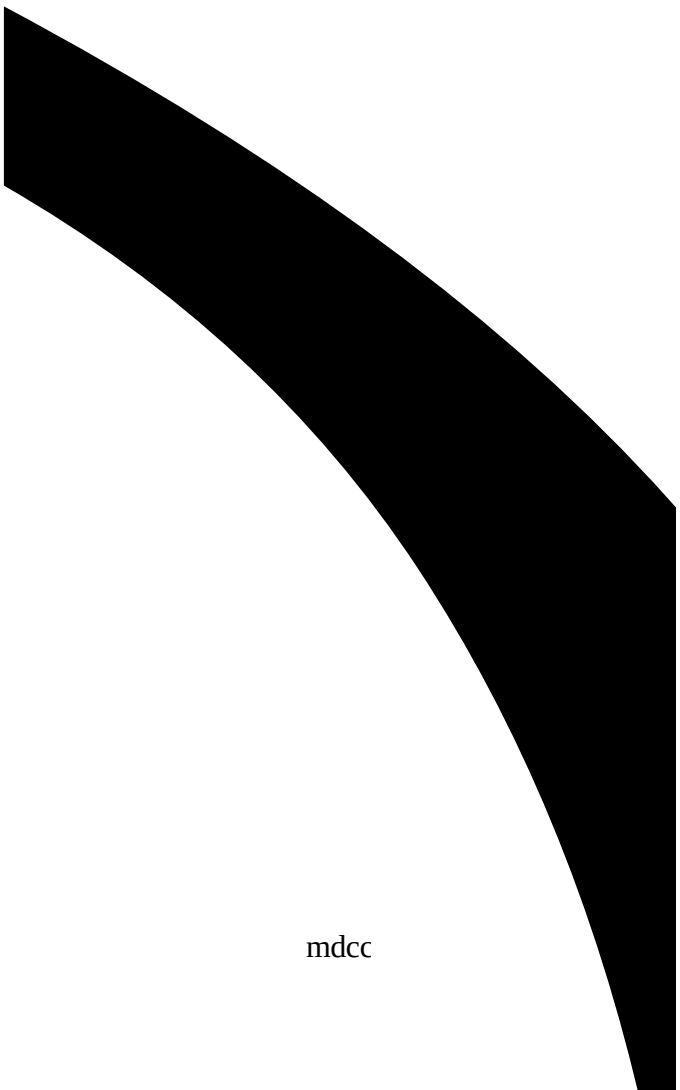


mdcc

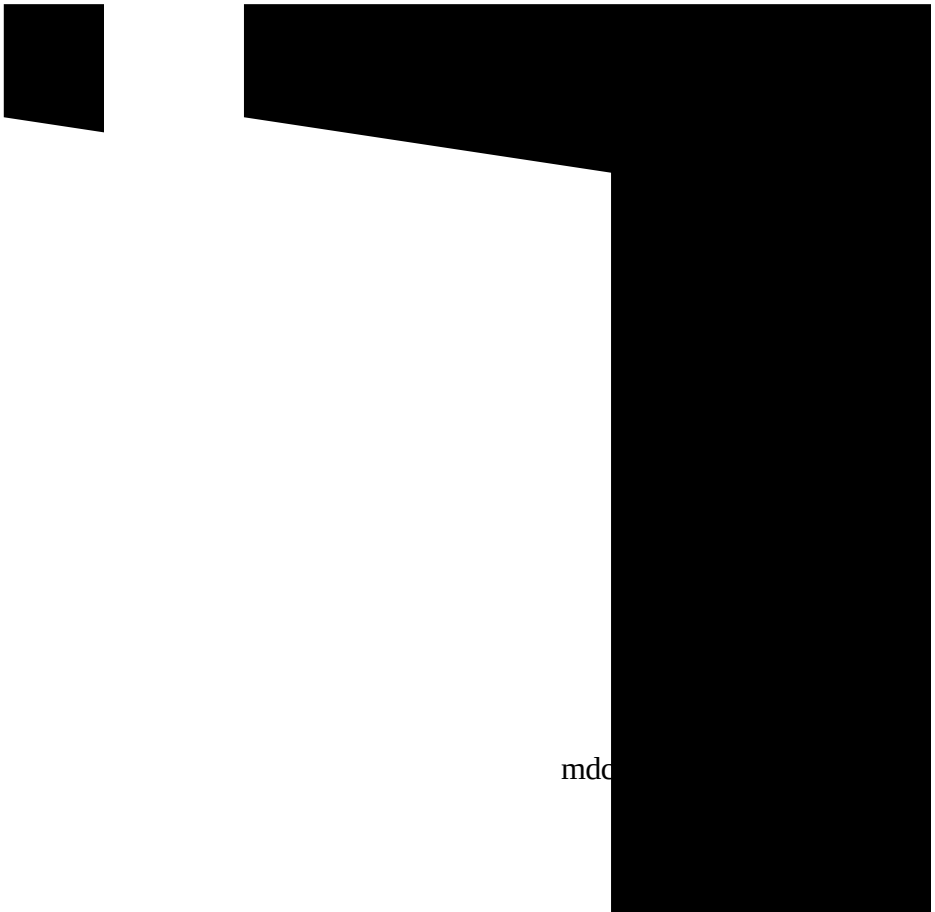


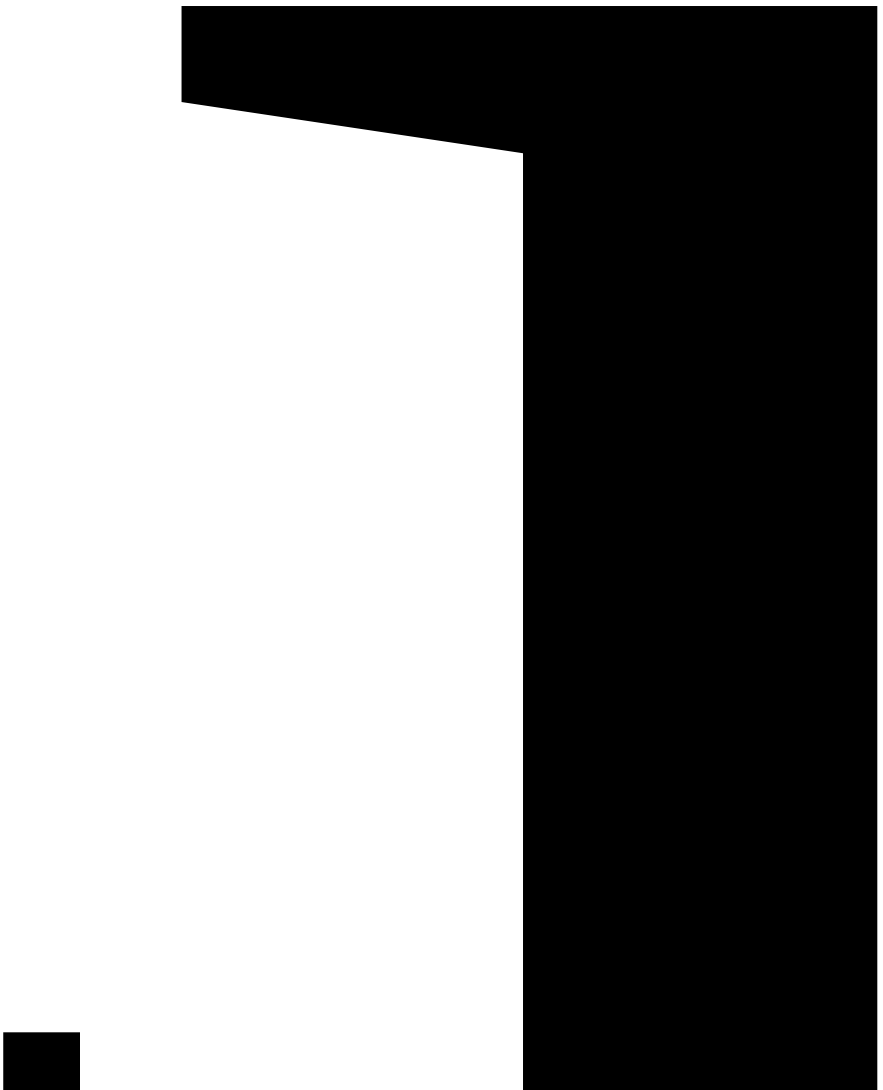
mdcc





mdcc





mdcc



mdcc



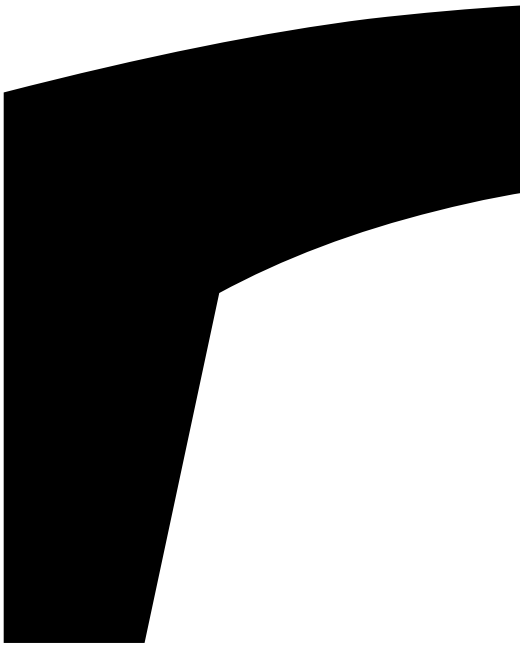
mdcc



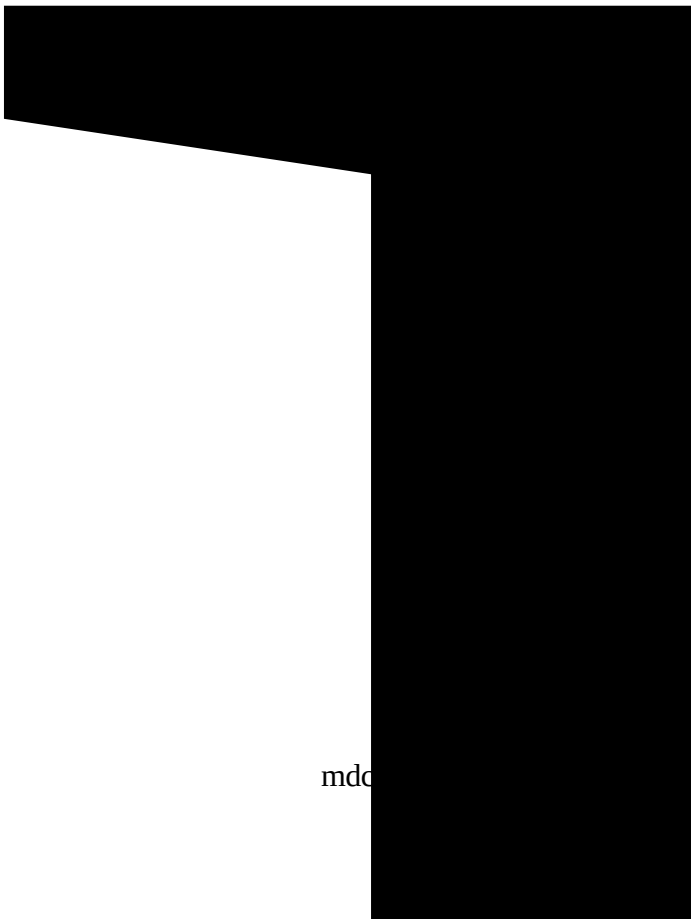
mdcc



mdcc



mdcc



mdc



mdcc



mdcc



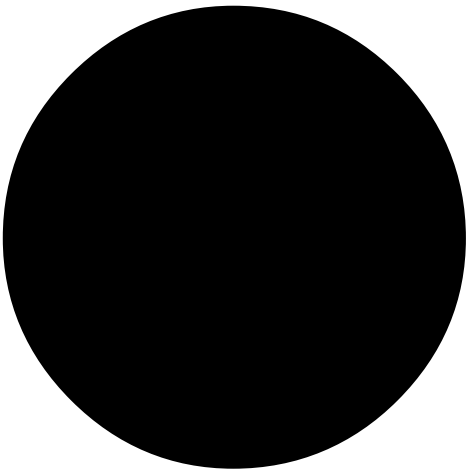
mdcc



mdcc



mdcc



mdcc

mdcc



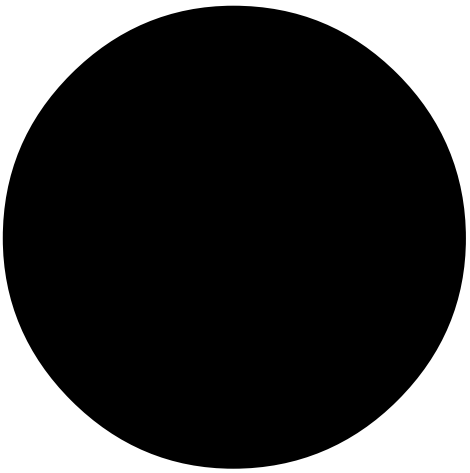
mdcc



mdcc







mdcc

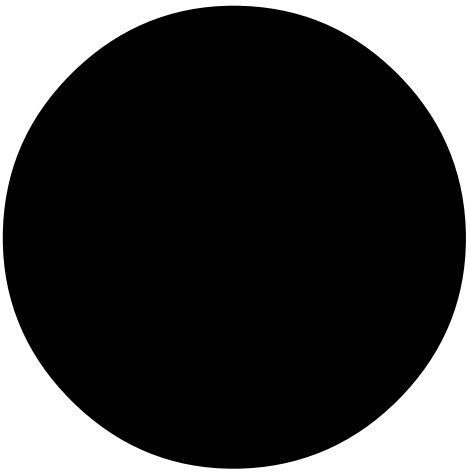
mdcc



mdcc







mcm

mcm

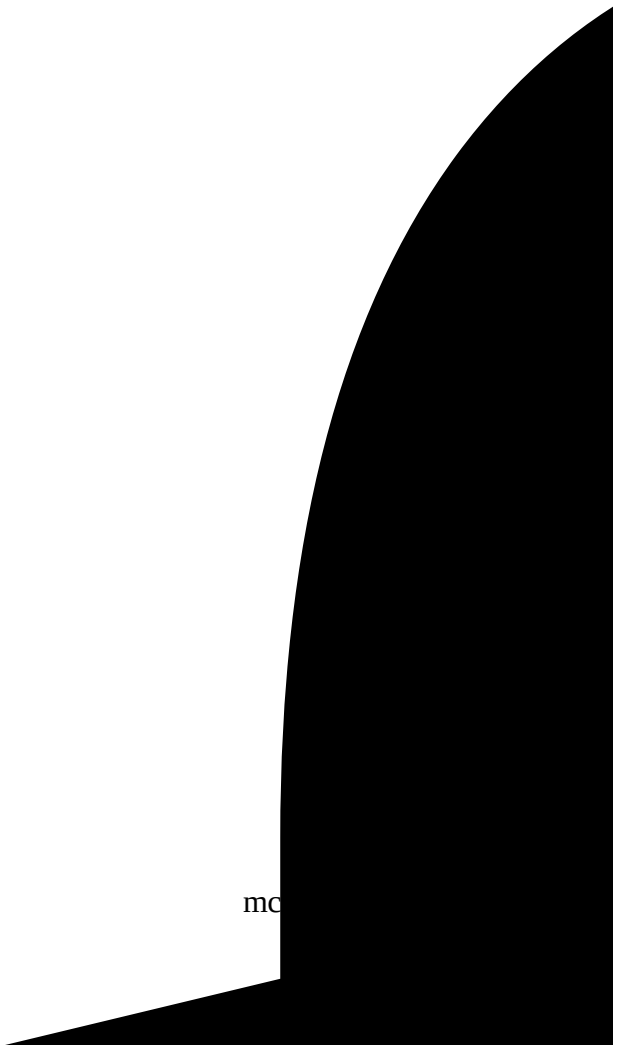
mcm

mcm

mcm

mcm

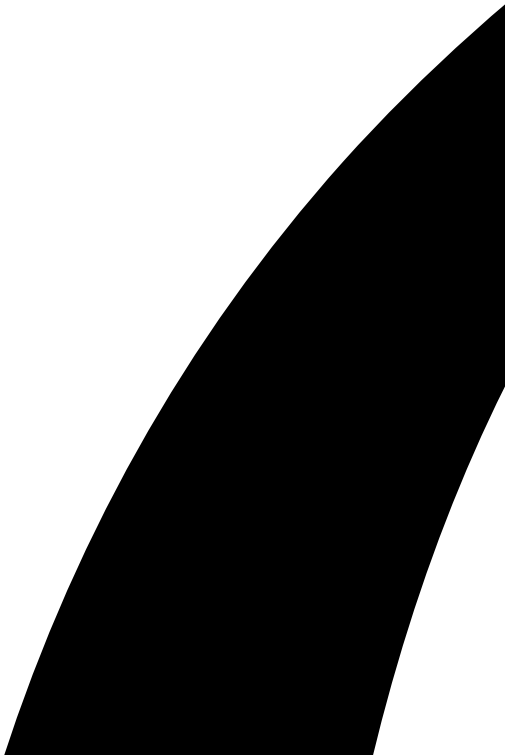






mcm

mcm



mcm

mcm

mcm

mcm

mcm

mcm

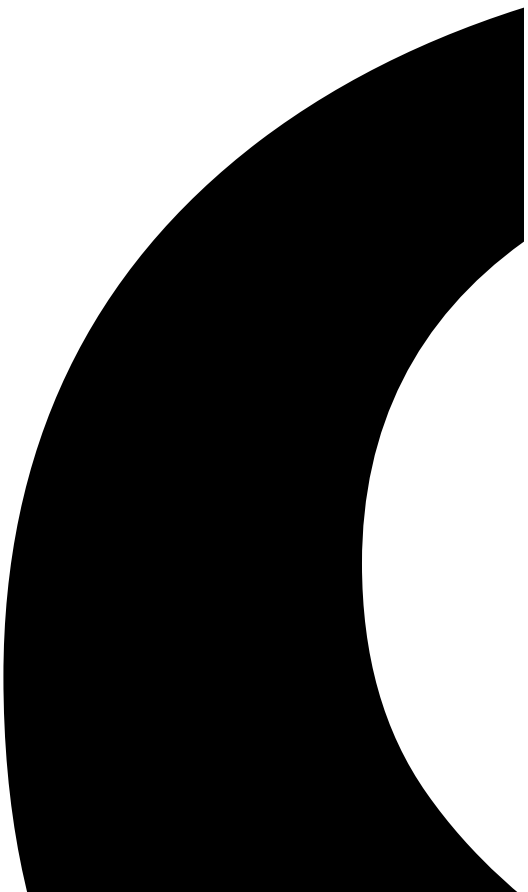


mcm

mcm



mcm



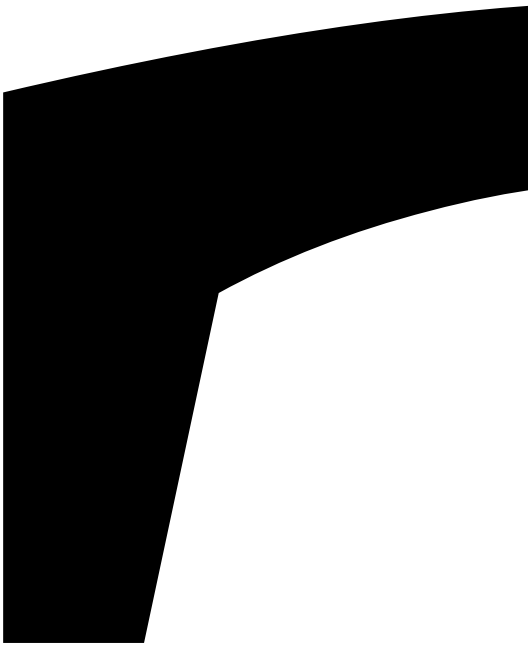


mcm

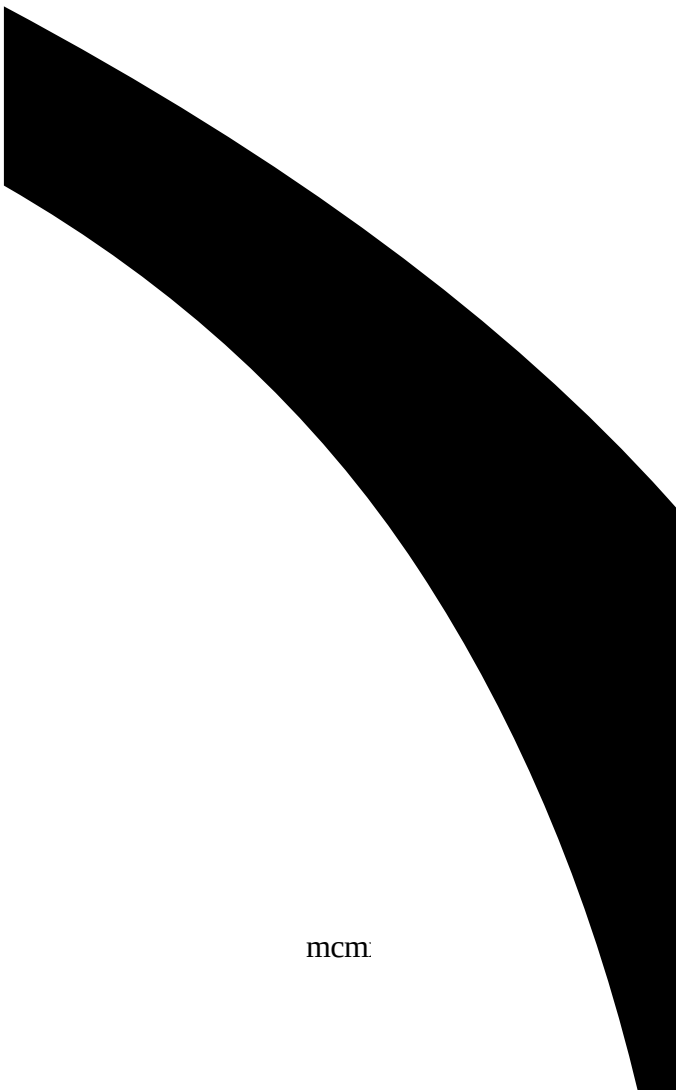


mcm

mcm



mcm



mcm



mcm

mcm

mcm



mcm





mcm



mcm



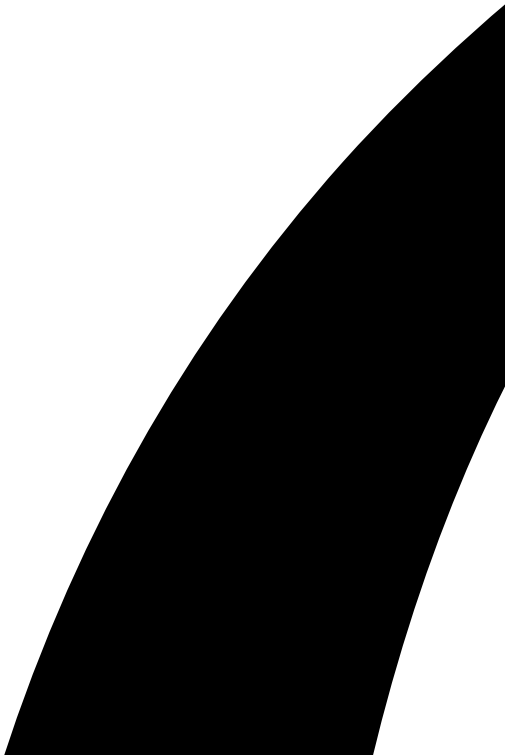
mcm

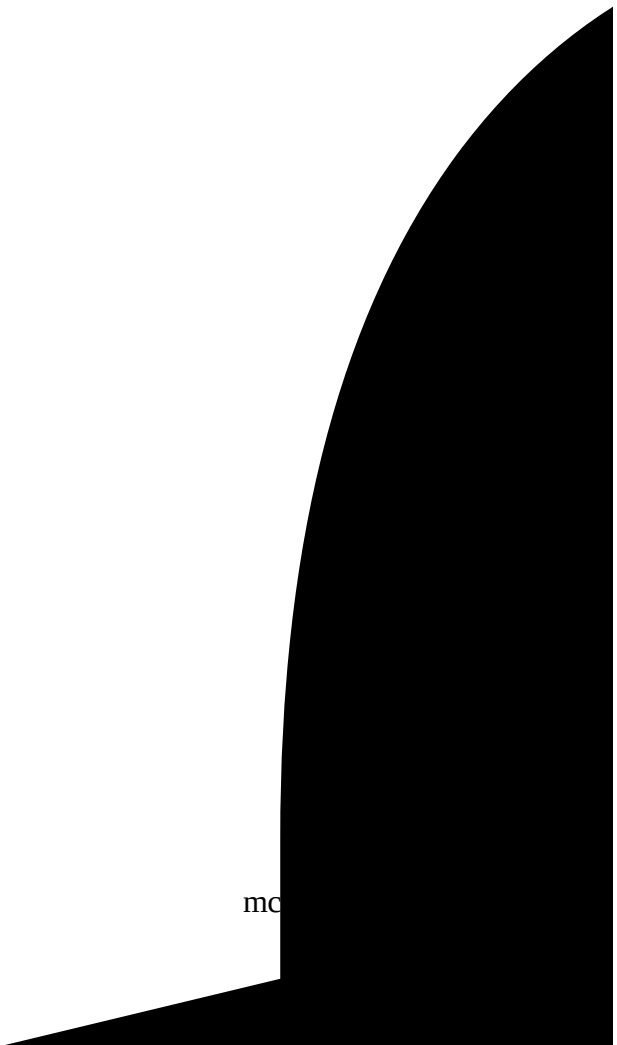


mcm

mcm







mcm

mcm

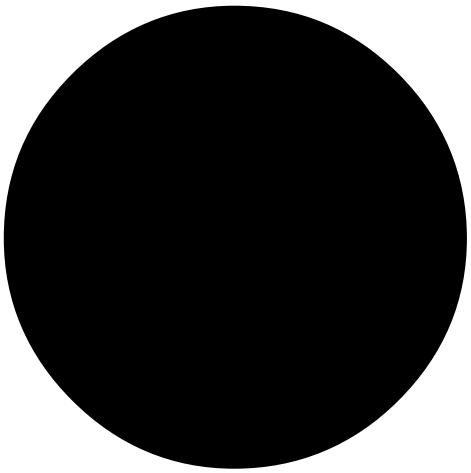


mcm

mcm

mcm





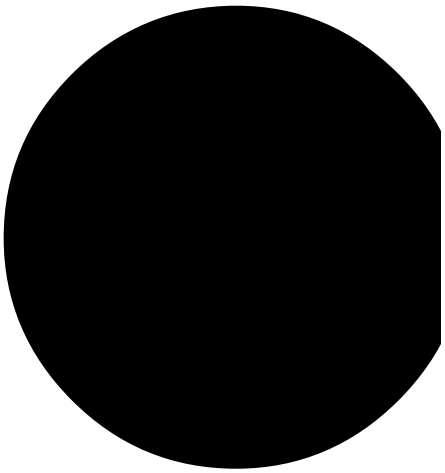
mcm

mcm

mcm

mcm

mcm



mcm

mcm

mcm



mcm

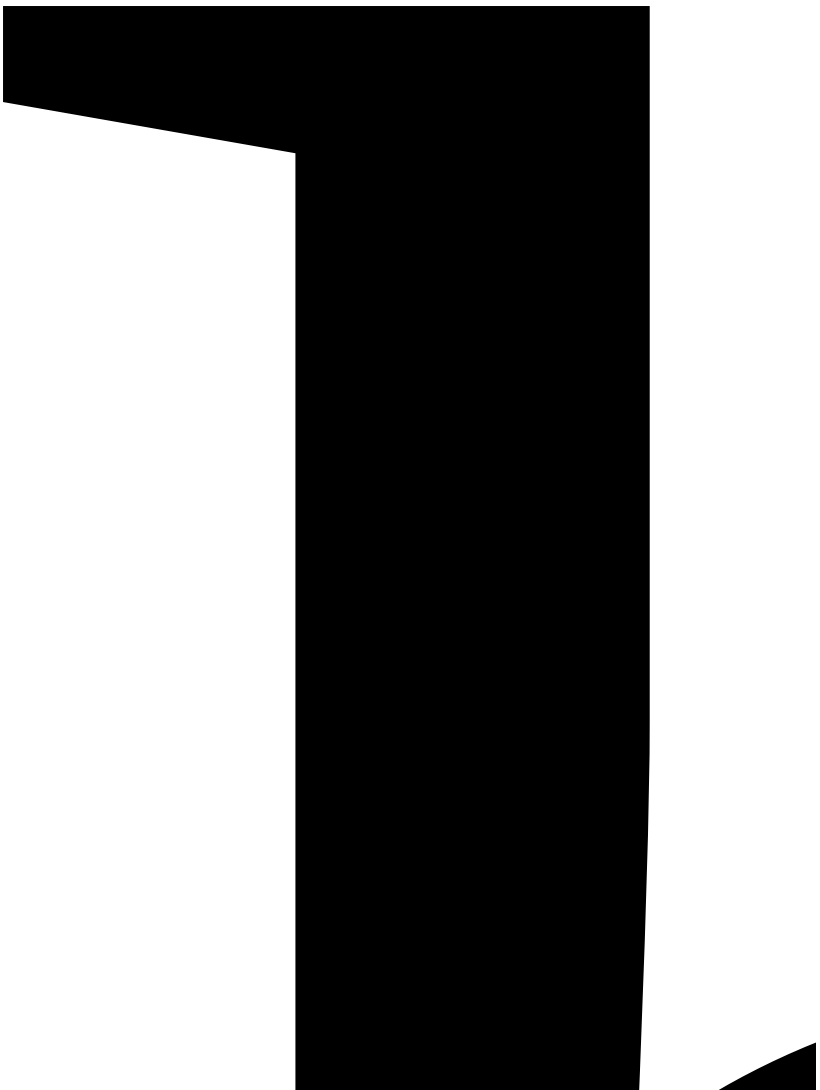


mcm



mcm

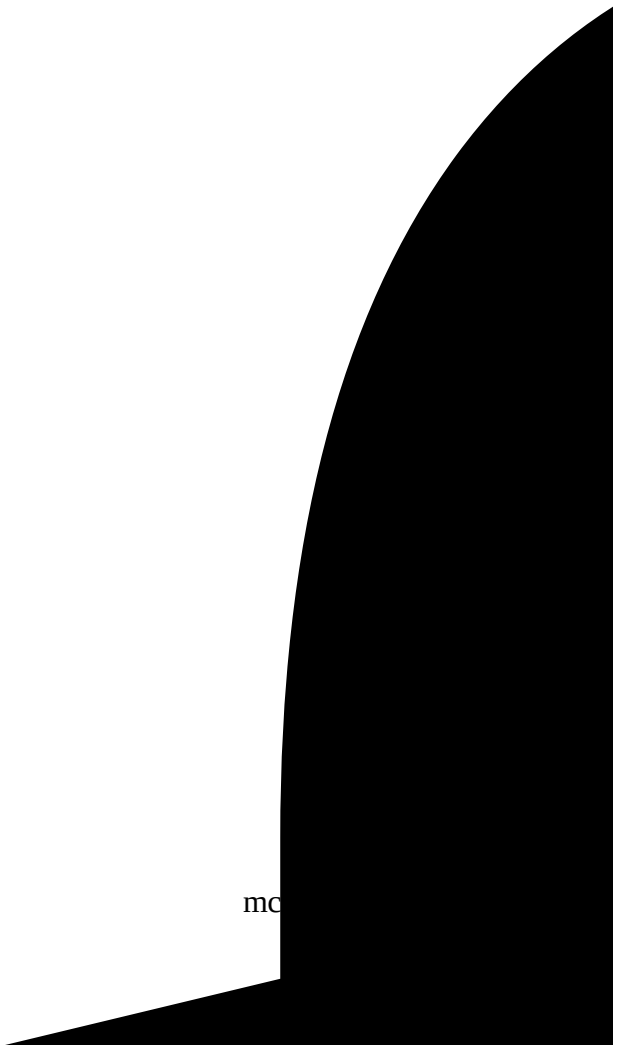


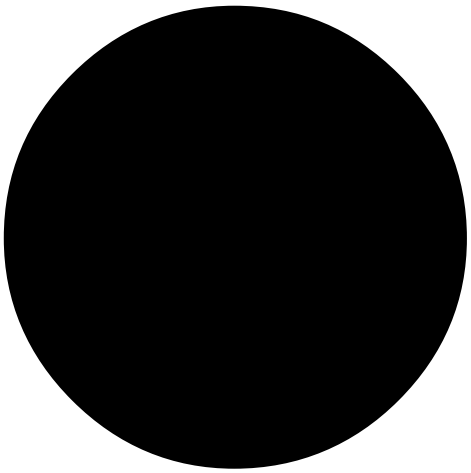


mcm

mcm

mcm





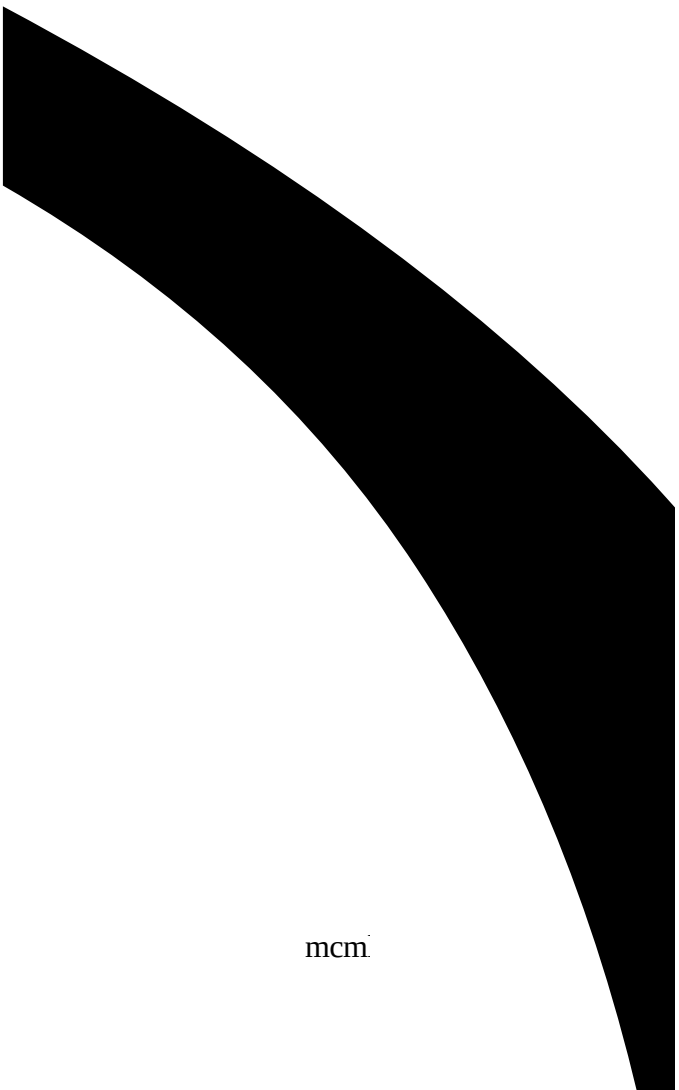
mcm

mcm

mcm

mcm

mcm



mcm



mcm

mcm

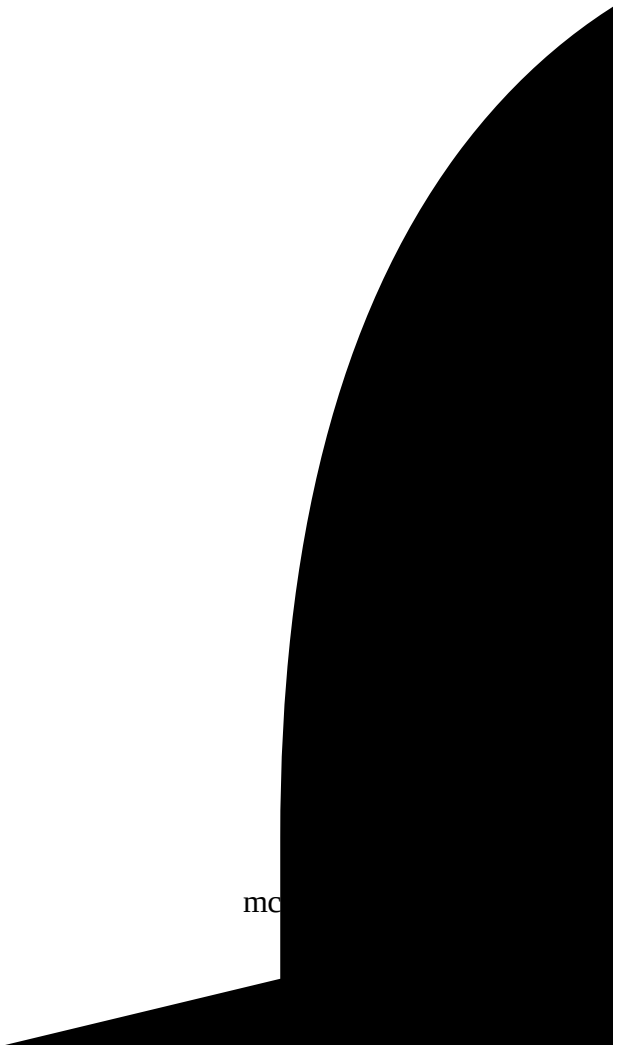
mcm

mcm

mcm

mcm

mcm



mcm

mcm

mm



mmi



mmii





mmiv



mmv





mmv



mmv





mmx



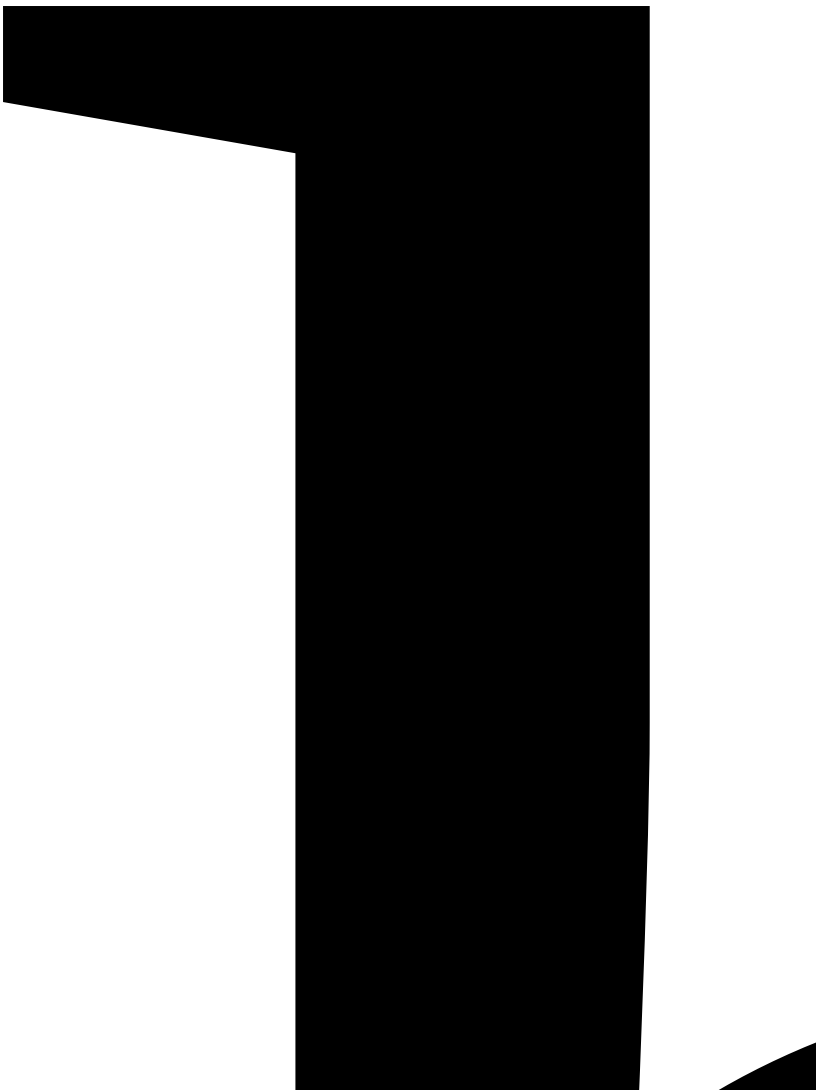
mmx



mmx

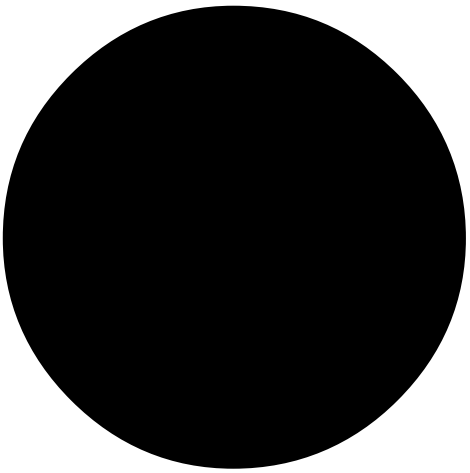
[redacted]

mmx



mmx

[redacted]



mmx



mmx

LIST OF TABLES

Table No.	Description	page No.
Table 2.1	Summary of Reviewed Work	8
Table 3.1	Project Schema (provisional)	18
Table 3.2	Floor Plan and Room Schema (provisional)	18
Table 3.3	Design Selection Schema (provisional)	18
Table 3.4	Cost Estimate Schema (provisional)	18
Table 5.1	Interior Reference Designs By Room Category	32
Table 5.2	Implementation Status of Modules	37
Table 5.3	Reported Test and Build Results	40

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The AI-Assisted House Planning and Visualization System is a web-based application designed to connect the main decisions involved in building a house: the floor plan, the interior design, the exterior design and the construction cost. Today these decisions are usually made in separate tools and often by separate people. A drafter or architect prepares the plan, an interior designer or a catalogue supplies finishes, and a contractor prepares a cost estimate. When a home-owner changes one decision, the others have to be repeated by hand.

This system provides one workflow for the whole process. The user enters the plot dimensions or area, the facing direction, the number of floors and the required rooms. The system generates a 2D floor plan with room divisions, dimensions, doors, windows, staircase placement and circulation areas, and checks the plan geometrically. The user can then browse 32 interior design references grouped by room category, select designs for the rooms that actually exist in the plan, customize flooring, wall finishes, ceilings and lighting, and see a construction cost estimate that is based on quantities taken from the plan.

The system uses constraint-based planning with OR-Tools CP-SAT and geometric validation with Shapely. It can use Gemini for AI-assisted architectural reasoning when it is available, and it keeps a deterministic fallback so that planning continues when the AI provider is unavailable. The cost module uses editable material and labour rates, so every line of the estimate can be inspected and changed, and the estimate can be recalculated without running the floor-plan solver again.

The complete workflow, which also includes exterior design, paint-colour selection and a coherent 3D visualization, is the vision of the project. This report separates the work that is reported as implemented in Stage-1 from the work that is partial or planned for Stage-2, and it does not describe the whole application as finished or production-ready.

1.2 PROBLEM STATEMENT

Planning a house involves a chain of linked decisions. The plot size and facing direction shape the floor plan, the floor plan decides which rooms exist and how large they are, the rooms decide which interior designs are relevant, and the finishes selected for those rooms decide a large part of the construction cost. Most home-owners, students and small builders do not have a single tool that follows this chain from start to end.

The existing way of working is mostly manual and disconnected. Plans are drawn by hand or in drafting software, interior ideas are collected separately from catalogues and websites, and cost estimates are prepared in spreadsheets or quoted as a single figure. Interior references are generic and are not tied to the rooms of the plan. A change in the plan or in a finish normally means redoing the later steps, so only a few alternatives are ever compared.

Existing research on automated floor-plan generation and on text-driven 3D house generation shows that plans and 3D scenes can be produced automatically, but these prototypes do not estimate construction cost, and the cost-estimation research does not generate plans. Therefore, there is a need for an integrated, low-cost system that turns a short list of requirements into a validated 2D plan, lets the user choose interiors for the rooms of that plan, and gives a transparent cost estimate that updates when the choices change.

1.3 OBJECTIVES

The main objective of the AI-Assisted House Planning and Visualization System is to develop a reliable and user-friendly web application that connects floor-plan generation, interior design selection, exterior customization, 3D visualization and construction cost estimation. The specific objectives are:

- To accept plot dimensions or area, facing direction, number of floors, required rooms, parking, staircase and additional requirements through a responsive web interface built with React and Vite.
- To generate a 2D floor plan with room divisions, dimensions, doors, windows, staircase placement and circulation areas.
- To improve spatial consistency through constraint-based planning with OR-Tools CP-SAT and geometric validation with Shapely.
- To use Gemini for AI-assisted architectural reasoning when it is available, while preserving a deterministic fallback.
- To provide 32 interior design reference images grouped by room category and let the user select designs only for rooms present in the generated plan.
- To estimate construction cost from geometry-derived quantities and editable material and labour rates, and to recalculate it without re-running the floor-plan solver.
- To plan, for Stage-2, exterior design, paint-colour selection and a coherent 3D visualization that reflects the selected interior, exterior and colours.

1.4 SCOPE OF THE PROJECT

The scope of the project is to develop a web-based application that helps users plan a residential building from a short list of requirements. The system covers the generation and validation of 2D floor plans, the selection and customization of interior designs for the rooms in the plan, and the estimation of construction cost from the geometry of the plan. It is intended for home-owners who want an early idea of a layout, interior style and budget, for architecture and civil engineering students who want to explore layout alternatives, and for small builders who need a quick and editable preliminary estimate.

Within the scope of Stage-1 (reported as implemented or partly implemented) are the requirement workspace, the floor-plan planning and validation services, the AI integration with fallback, the interior catalogue of 32 reference images, room-level selection and customization, persistence of the selections, and the construction cost estimation. Outside the scope of Stage-1 (planned for Stage-2) are dedicated interior design pages, exterior design, paint-colour selection, coherent 3D visualization, and the integration of the cost estimate with all of these choices.

The interior reference images are used as visual design inspiration only. They are not geometrically exact renderings of a user's room. The cost figures produced by the system are illustrative estimates based on benchmark rates for Hyderabad, Telangana for 2026. They are not guaranteed quotations and not an engineer-certified bill of quantities.

1.5 BLOCK DIAGRAM

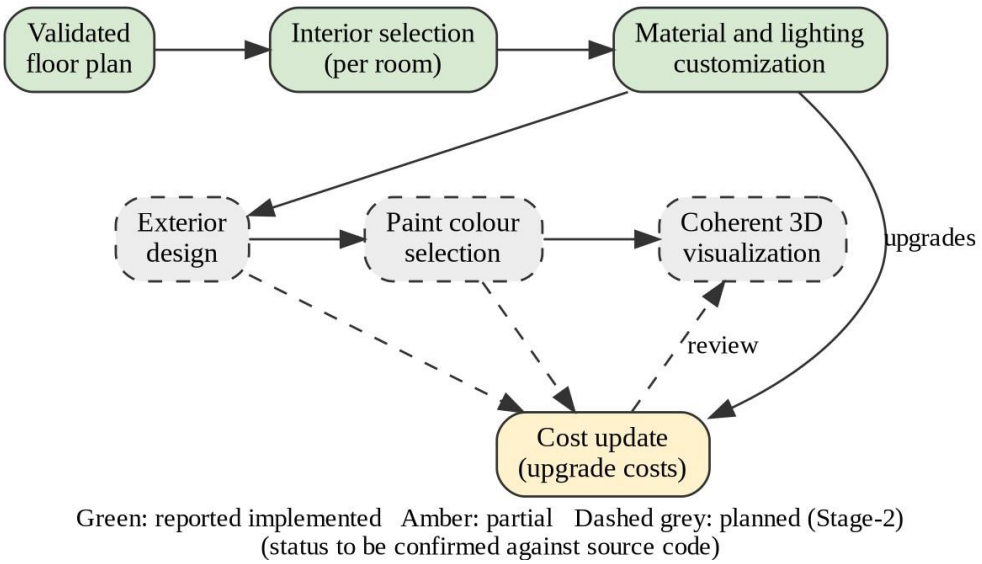


Fig 1.1 Block diagram of AI-Assisted House Planning and Visualization System

Green blocks are reported as implemented, amber blocks are partial and dashed grey blocks are planned for Stage-2.

CHAPTER 2

LITERATURE SURVEY

2.1 REVIEW OF THE EXISTING WORK

The problem of generating residential floor plans automatically has been studied by many researchers, and their findings show both the possibilities and the limits of current methods. Liu et al. (2017) [1] studied floor plans that exist only as raster images. They used a neural network to detect wall and door junctions and integer programming to assemble them into a consistent vector plan, and they used the result to produce 3D pop-up models. Their work shows that a vector representation with consistency constraints supports 3D visualization, but it analyses existing drawings and does not create new plans from user requirements.

Nauata et al. (2020) [2] proposed House-GAN, a relational generative adversarial network that generates house layouts from a graph of room types and adjacencies. It was evaluated on about 117,000 real floor plans and improved realism, diversity and agreement with the input graph, but the rooms are output as axis-aligned boxes and real plot dimensions are not modelled. Nauata et al. (2021) [4] extended this with House-GAN++, which refines its own output iteratively and handles doors and non-rectangular rooms. Architects in their study noted that window placement depends on daylight and neighbouring buildings, which the data did not contain. Shabani et al. (2023) [6] proposed HouseDiffusion, a diffusion model that produces vector floor plans directly and controls the number of corners per room, although it still needs a bubble diagram as input and gives no guarantee about plot size.

Para et al. (2021) [5] combined learning with constraints. Their method generates layout elements and the constraints between them with transformers and then solves for final positions with constrained optimization, and it was applied to floor plans and furniture layouts. The conditioning inputs are soft, so a requested condition is not guaranteed. This is the closest published idea to the present project, in which constraints decide the geometry and a solver enforces them.

Text-driven generation has also been studied. Chen et al. (2020) [3] built Intelligent Home 3D, which turns a written description into a structural graph, predicts the room layout, synthesizes floor and wall textures and renders a 3D house. They created a Text-to-3D House Model dataset of 2,000 houses with 13,478 rooms. Fu et al. (2024) [7] proposed AnyHome, in which a large language model produces a structured representation of a house that is then textured and edited. Both works focus on visual quality and do not address dimensioned plans or construction cost.

For construction quantities and cost, Mattern et al. (2018) [8] describe BIM-based quantity take-off, where quantities are extracted automatically from a building model, which is faster and less error-prone than manual take-off but needs a detailed model that does not exist at the early planning stage. Elmousalami (2021) [9] compared twenty artificial-intelligence techniques for conceptual cost prediction on a case study of field canal improvement projects and showed that early cost models depend on scarce and unstable data.

2.2 SUMMARY OF THE LITERATURE SURVEY

From the literature survey, it is observed that automatic generation of floor plans and 3D houses is an active research area, and that learned generators such as House-GAN, House-GAN++ and HouseDiffusion produce realistic layouts from room graphs, while constraint-based approaches can enforce explicit geometric requirements. Text-driven systems link written requirements to 3D scenes, and BIM-based quantity take-off and machine-learning cost models address the estimation of cost. Table 2.1 summarizes the reviewed work.

Table 2.1 : SUMMARY OF REVIEWED WORK

Ref.	Year	Venue	Main focus	Cost / quantities	3D / interior
[1]	2017	IEEE ICCV	Raster to vector plans	No	3D pop-up
[2]	2020	Springer ECCV	Graph-constrained layout (House-GAN)	No	No
[3]	2020	IEEE CVPR	Text to 3D house	No	Yes
[4]	2021	IEEE CVPR	Layout refinement (House-GAN++)	No	No
[5]	2021	IEEE ICCV	Constraint graphs and optimization	No	Furniture layouts
[6]	2023	IEEE CVPR	Diffusion floor plans	No	No
[7]	2024	Springer ECCV	LLM-guided 3D homes	No	Yes
[8]	2018	Springer	BIM quantity take-off	Quantities	From BIM
[9]	2021	IEEE TEM	AI cost prediction	Yes	No

However, none of the reviewed generation papers estimates cost, and the cost papers do not generate plans. Learned generators give only soft guarantees about plot size and room requirements. Text-driven systems stress appearance over buildability. Early-stage cost estimation needs quantities from the plan itself, whereas BIM-based take-off needs a finished model and machine-learning models need historical data. Interior, exterior, colour and cost are not linked in any of the reviewed systems. Therefore, the proposed system combines constraint-based planning with geometric validation, optional language-model reasoning with a deterministic fallback, interior selection, and a transparent quantity-based cost estimate that can be recalculated without re-solving the plan.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 INTRODUCTION

The proposed methodology of the AI-Assisted House Planning and Visualization System focuses on building a web-based platform that guides a user from the first requirements to a validated floor plan, a set of interior choices and a cost estimate. The user enters the plot size or area, the facing direction, the number of floors and the required rooms through a simple interface. The backend converts these requirements into a constraint model, solves it, validates the result and returns the plan to the browser.

The methodology ensures that the later steps depend on the earlier ones. Interior designs can be selected only for rooms that exist in the generated plan, and the cost estimate is computed from the geometry of that plan. By using React, FastAPI, OR-Tools, Shapely and Supabase, the system provides an easy, reliable and user-friendly solution for early house planning.

PROPOSED SYSTEM

The proposed system is a web application with a React frontend and a Python FastAPI backend. The backend contains the planning and validation services (OR-Tools CP-SAT and Shapely), an optional Gemini-based reasoning step with a deterministic fallback, an interior catalogue service and a cost engine. Supabase provides persistence. The user enters requirements, receives a validated 2D plan, selects interiors for the rooms in the plan, customizes materials and lighting, and sees a cost estimate that can be edited and recalculated.

3.1.1 OBJECTIVES OF THE PROPOSED SYSTEM

The main objective of the proposed system is to connect plan generation, interior selection and cost estimation in one workflow so that a change in one step is reflected in the next. The working of the system begins with the requirements form, where the user enters plot dimensions or area, facing direction, number of floors, required rooms, parking, staircase and any additional requirements. After the input is validated, the planner builds a constraint model with room areas, adjacency preferences and plot boundaries, asks Gemini for reasoning hints when it is available, and solves the layout with CP-SAT.

The candidate plan is then checked with Shapely for containment, overlap and access, and the solver is re-run with adjusted constraints until the plan is valid. The valid plan is stored, and the user continues to the interior design step, where designs are chosen for each room, and then to the cost step, where quantities taken from the plan are multiplied by editable rates. The advantages of the proposed system are given below.

- One workflow connects plan, interior selection and cost.
- Constraint solving and geometric validation give more consistent plans than unchecked generation.
- The deterministic fallback keeps the planner usable when the AI provider is unavailable.
- Cost is derived from geometry and editable rates, so every line item can be inspected and the estimate can be recalculated without re-solving the plan.
- Selections are limited to rooms that actually exist, which avoids irrelevant choices.

3.2 SYSTEM ARCHITECTURE

System architecture refers to the overall structure of a system and explains how its components such as the user interface, services, database and external providers are organized and interact with each other. In the proposed system, the architecture consists of a presentation layer, an application layer containing the planning and cost services, a data layer and an external AI provider. The presentation layer is the React frontend, built with Vite in JavaScript and JSX, where the user enters requirements, views plans, selects interior designs and reads the cost estimate.

The application layer is the FastAPI backend. It validates every request with Pydantic models and calls the planning engine (OR-Tools CP-SAT with Shapely validation), the interior catalogue service and the cost engine. The planning engine can ask Gemini for architectural reasoning, and a deterministic fallback planner is used when Gemini is unavailable or fails. The data layer is Supabase, which stores projects, plans, selections and estimates. Fig 3.1 shows the overall architecture, with planned components drawn dashed.

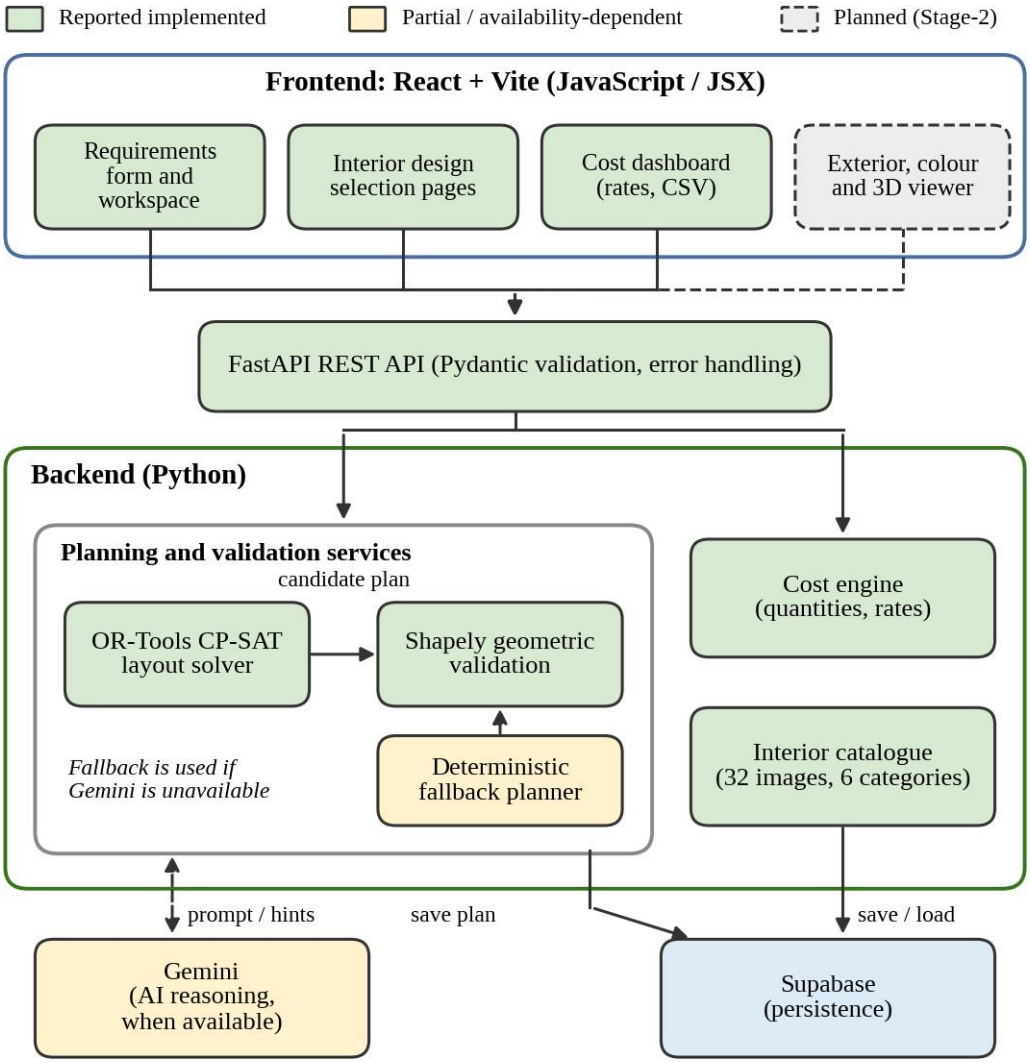


Fig 3.1 System Architecture Diagram

3.3 MODULE DESCRIPTION

The AI-Assisted House Planning and Visualization System consists of several modules that work together to take a user from requirements to a cost estimate. The status of each module is given as reported in the project notes: Implemented, Partial or Planned.

1. Requirements Input Module (Implemented, reported)

This module collects and validates the requirements of the user.

- Users enter plot dimensions or area, facing direction and number of floors
- Users select the required rooms, parking, staircase and additional requirements
- Pydantic models validate the data before planning starts

2. Floor-Plan Generation Module (Partial)

This module builds and solves the layout model.

- Room placement is modelled as a constraint problem and solved with OR-Tools CP-SAT
- The plan contains room divisions, dimensions, doors, windows, staircase placement and circulation areas
- Plan quality is limited and is a Stage-2 improvement item

3. Geometry Validation Module (Partial)

This module checks the candidate plan.

- Shapely checks that rooms lie inside the plot, do not overlap and remain accessible
- Violations are reported to the planner, which adjusts constraints and solves again

4. AI Reasoning Module (Partial)

This module provides optional architectural reasoning.

- Gemini is used for AI-assisted reasoning when it is available
- A deterministic fallback keeps planning usable when Gemini is unavailable
- Improving provider reliability is a Stage-2 task

5. Interior Design Module (Implemented, reported)

This module lets the user select and customize interiors.

- 32 reference images grouped by room type, with category filtering
- Room-level selections and plan-aware previews, only for rooms that exist in the plan
- Customization of supported flooring, wall finishes, ceiling designs, lighting and other materials
- Incremental upgrade cost and persistence of the selections

6. Cost Estimation Module (Implemented, reported)

This module estimates the construction cost.

- Geometry-derived quantities and area schedules
- Editable material and labour rate profiles, cost line items and categorized totals
- Contingency, contractor overhead and cost per square foot
- CSV export and recalculation without re-running the floor-plan solver

7. Exterior, Colour and 3D Visualization Module (Planned)

This module is planned for Stage-2.

- Exterior design and paint-colour selection with side-by-side comparison
- A coherent 3D visualization that reflects the selected plan, interior, exterior and colours

3.4 TOOLS AND TECHNOLOGIES USED

SOFTWARE REQUIREMENTS

1. Functional Requirements

- Users can enter plot, facing, floor and room requirements (Implemented, reported).
- The system generates a 2D plan with rooms, dimensions, doors, windows, staircase and circulation (Partial).
- The system validates geometry and applies constraint-based planning (Partial).
- Gemini reasoning is used with a deterministic fallback (Partial).
- Users can browse 32 interior designs by category and select designs for existing rooms (Implemented, reported).
- Users can customize materials and lighting and see the incremental upgrade cost (Implemented, reported).
- Design selections are persisted (Implemented, reported).
- The system produces a cost estimate with editable rates, recalculation and CSV export (Implemented, reported).
- Exterior design, paint-colour selection, 3D visualization and cost integration are planned (Planned).

2. Non-Functional Requirements

- Usability: responsive interface and room-aware choices.
- Reliability: the planner still works when the AI provider is unavailable.
- Transparency: every cost line shows quantity, rate and total, and rates are editable.
- Accuracy disclaimer: costs are illustrative estimates, not quotations or a certified bill of quantities.
- Maintainability: automated tests for cost, interior and geometry logic.
- Security: validate all input, keep API keys on the server and restrict database access (to be confirmed).
- Performance: no benchmark has been measured.

3. Recommended Software Stack

- Frontend: React, Vite, JavaScript and JSX
- Backend: Python, FastAPI, Pydantic
- Planning and geometry: OR-Tools CP-SAT, Shapely
- AI integration: Gemini API (when available)
- Persistence: Supabase

- Testing: Pytest, Vite production build
- Client: a modern web browser

HARDWARE REQUIREMENTS

These values are suggested minimums for a typical development laptop; they are not measured requirements.

- Processor: dual-core 2 GHz or better (CP-SAT solving benefits from more cores)
- RAM: 8 GB recommended for development
- Storage: 2 GB free for tools and dependencies
- Display: 1366 x 768 or higher
- Internet connection for Supabase and the Gemini API

3.5 DATABASE DESIGN

Database design is the process of organizing and structuring data in a systematic way so that it can be stored, managed and accessed efficiently. The AI-Assisted House Planning and Visualization System uses Supabase for persistence. The tables below are a provisional design derived from the features described in the project notes. They have not been checked against the actual Supabase schema, so the table and column names must be replaced with the verified schema before final submission. Fig 3.2 shows the provisional entity-relationship diagram.

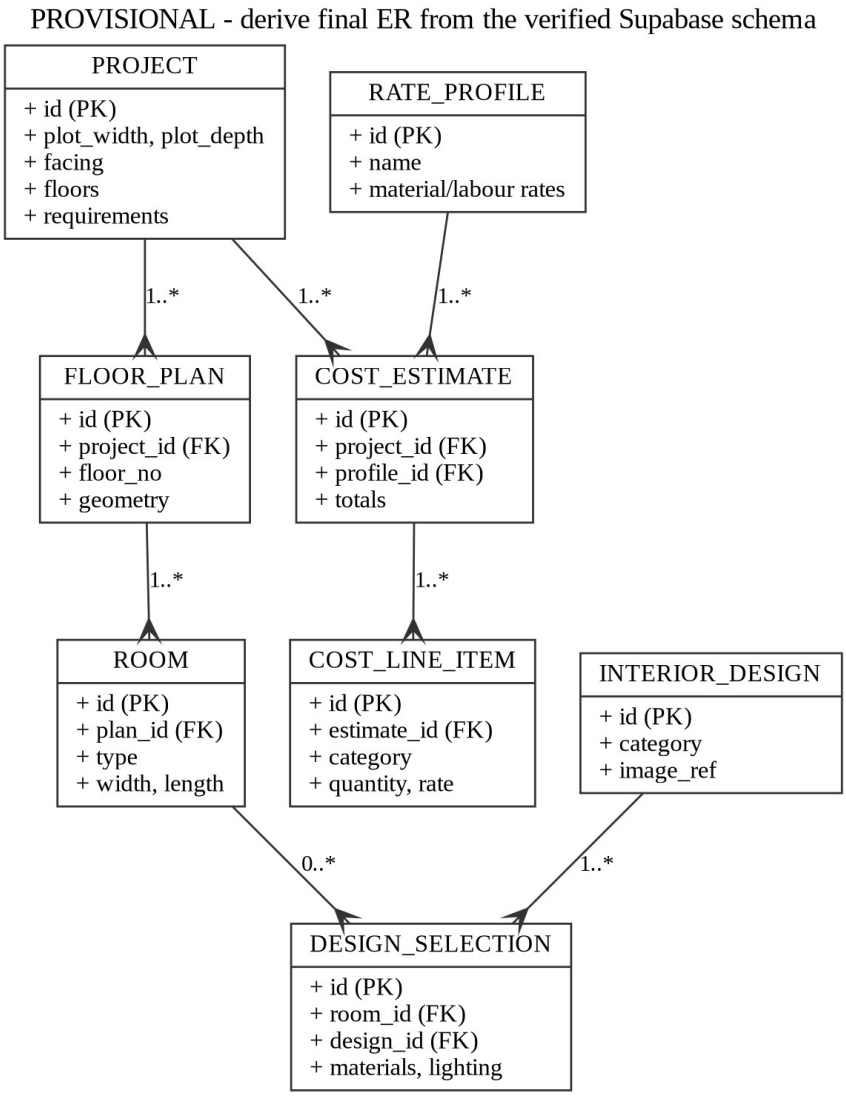


Fig 3.2 Provisional ER Diagram (to be replaced by the verified Supabase schema)

Table 3.1 : PROJECT SCHEMA (PROVISIONAL)

Field Name	Data Type	Description
id	UUID (Primary Key)	Unique ID for each project
plot_width, plot_depth	NUMERIC	Plot dimensions
facing	TEXT	Facing direction of the plot
floors	INT	Number of floors
requirements	JSON	Rooms and extra requirements

Table 3.2 : FLOOR PLAN AND ROOM SCHEMA (PROVISIONAL)

Field Name	Data Type	Description
floor_plan.id	UUID (Primary Key)	Unique ID for each plan
floor_plan.project_id	UUID (Foreign Key)	Project the plan belongs to
floor_plan.geometry	JSON	Generated plan geometry
room.id	UUID (Primary Key)	Unique ID for each room
room.type, width, length	TEXT, NUMERIC	Room type and size

Table 3.3 : DESIGN SELECTION SCHEMA (PROVISIONAL)

Field Name	Data Type	Description
id	UUID (Primary Key)	Unique ID for each selection
room_id	UUID (Foreign Key)	Room for which the design is chosen
design_id	TEXT (Foreign Key)	Interior catalogue entry
materials, lighting	JSON	Customization choices

Table 3.4 : COST ESTIMATE SCHEMA (PROVISIONAL)

Field Name	Data Type	Description
estimate.id	UUID (Primary Key)	Unique ID for each estimate
estimate.project_id	UUID (Foreign Key)	Project estimated
estimate.totals	JSON	Category totals, contingency, overhead
line_item.category, quantity, rate	TEXT, NUMERIC	Cost line items
rate_profile.name, rates	TEXT, JSON	Material and labour rate profiles

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

The AI-Assisted House Planning and Visualization System is a web-based application designed to reduce the effort of early house planning by connecting floor-plan generation, interior selection and cost estimation. Every day, many people plan houses with scattered tools, and every change in the plan means repeating the later steps. This system solves that problem by keeping the plan, the interior choices and the cost together.

Through this system, users can enter requirements, generate and validate a 2D plan, select and customize interiors for the rooms of the plan, and view and edit a cost estimate. The system stores the plans, selections and estimates in a database, and it uses a deterministic fallback so that planning does not depend on the availability of the AI provider.

The main objective of the system design is to describe how the modules interact. For this purpose the UML diagrams and the data flow diagrams in this chapter show the actors, the activities, the message sequences and the components of the system. The endpoint names used in the sequence diagrams are placeholders and must be replaced with the actual routes of the application.

4.2 UML DIAGRAMS

Unified Modeling Language (UML) is a standard visual language used to describe the structure and behaviour of a software system. UML diagrams help developers, students and evaluators understand how a system works without reading its code. In this project the use case, activity, sequence and component diagrams are used.

4.2.1 USE CASE DIAGRAM

The Use Case Diagram represents how the user interacts with the system and which functions are available. The main use cases are entering plot and room requirements, generating a 2D floor plan, viewing the validation result, browsing interior designs by category, selecting a design per room, customizing materials and lighting, viewing and editing the cost estimate, and exporting the cost as CSV. Choosing the exterior and paint colour and viewing the 3D visualization are planned use cases and are drawn dashed. Gemini and Supabase appear as external services.

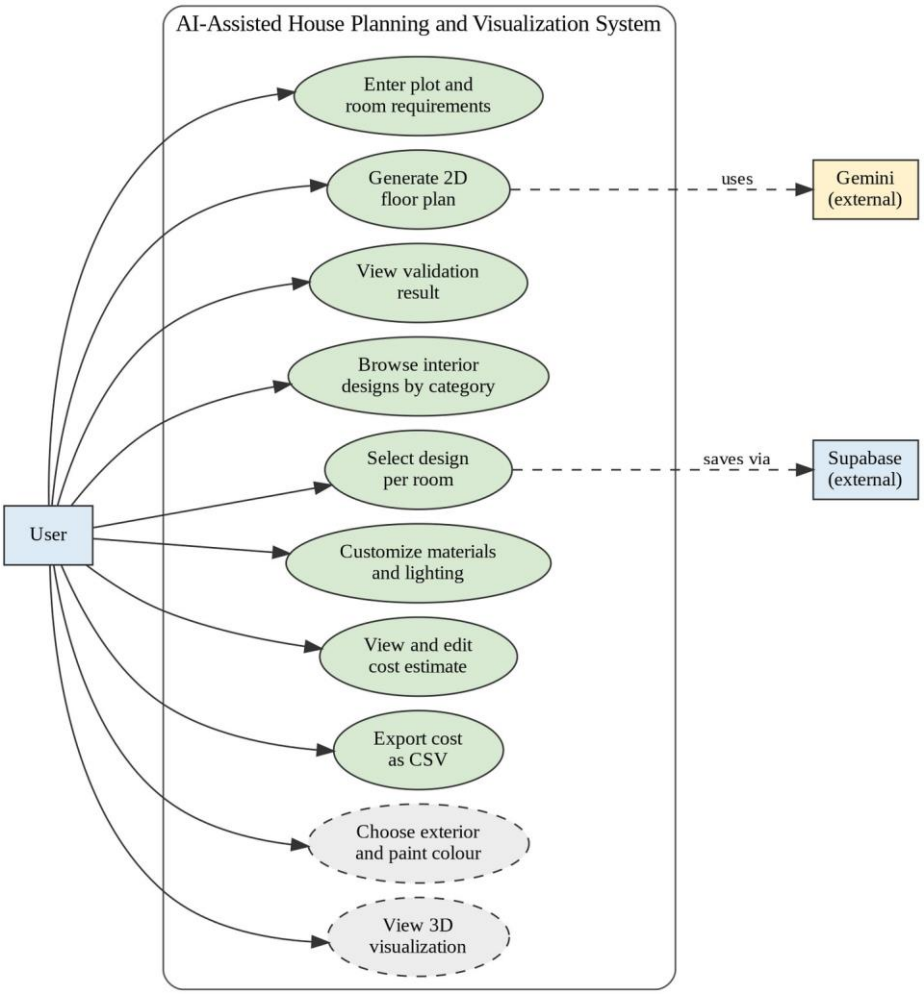


Fig 4.2.1 Use Case Diagram

4.2.2 ACTIVITY DIAGRAM

The Activity Diagram shows how a floor plan is generated and validated. The input is validated first. If AI reasoning is available the planner obtains hints from Gemini, otherwise it applies deterministic rules. The CP-SAT solution is checked with Shapely, and the solver is re-run with adjusted constraints until the plan is valid.

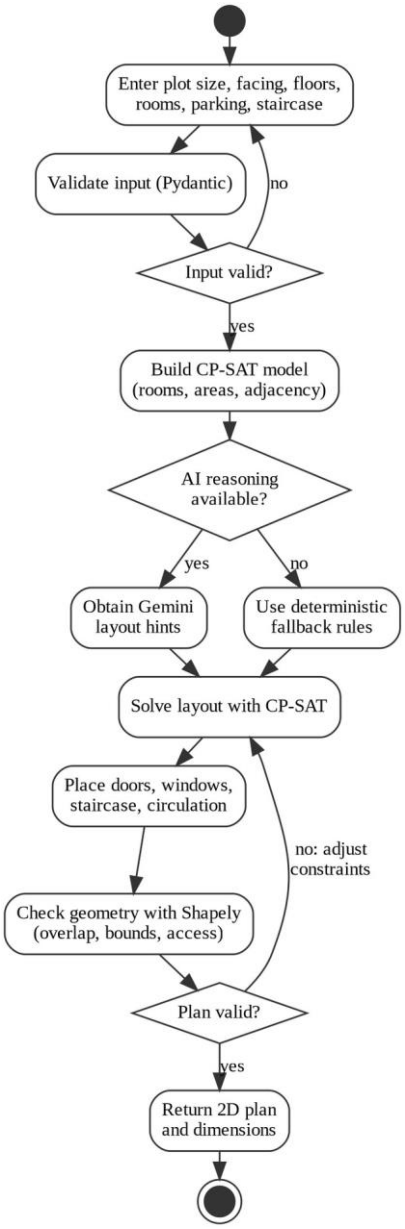


Fig 4.2.2 Activity Diagram for floor-plan generation and validation

4.2.3 SEQUENCE DIAGRAMS

The Sequence Diagram for floor-plan generation shows the messages exchanged between the user, the React frontend, the FastAPI API, the planning engine, Gemini, the Shapely validator and Supabase. The Sequence Diagram for cost calculation shows how the cost engine loads rates, computes quantities and totals, saves the estimate and later recalculates it from stored quantities without calling the floor-plan solver.

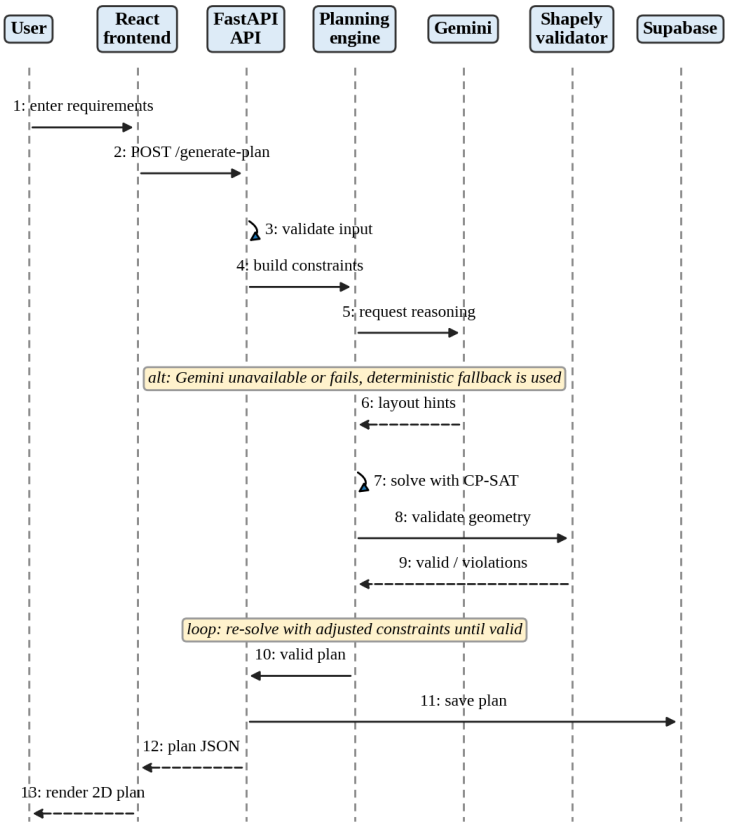


Fig 4.2.3 Sequence diagram for floor-plan generation

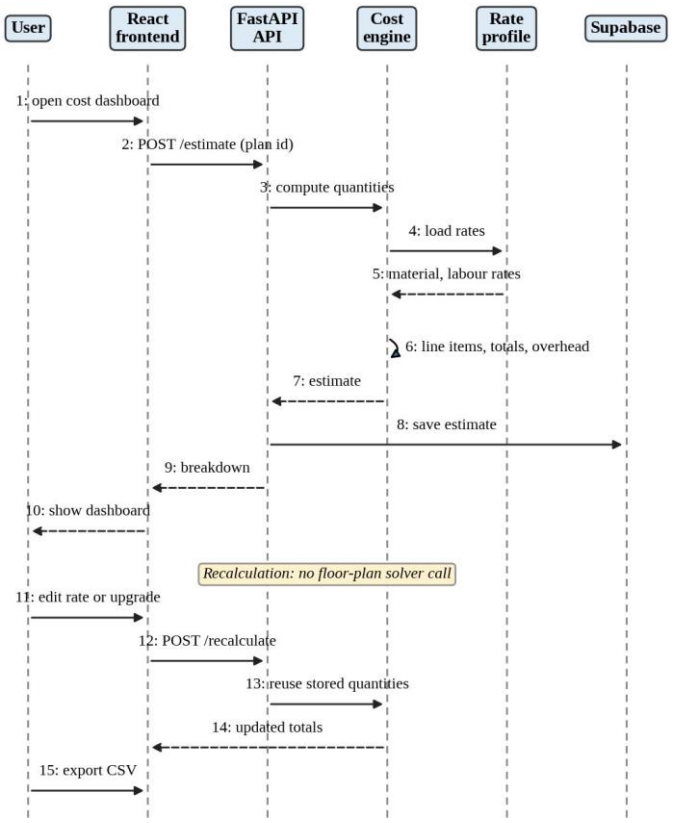


Fig 4.2.4 Sequence diagram for cost calculation and recalculation

4.2.4 COMPONENT DIAGRAM

The Component Diagram shows the main components and their dependencies: the React frontend, the FastAPI layer, the planning engine, the cost engine, the interior catalogue, the persistence layer and the AI provider adapter. The 3D and exterior module is planned and is drawn dashed.

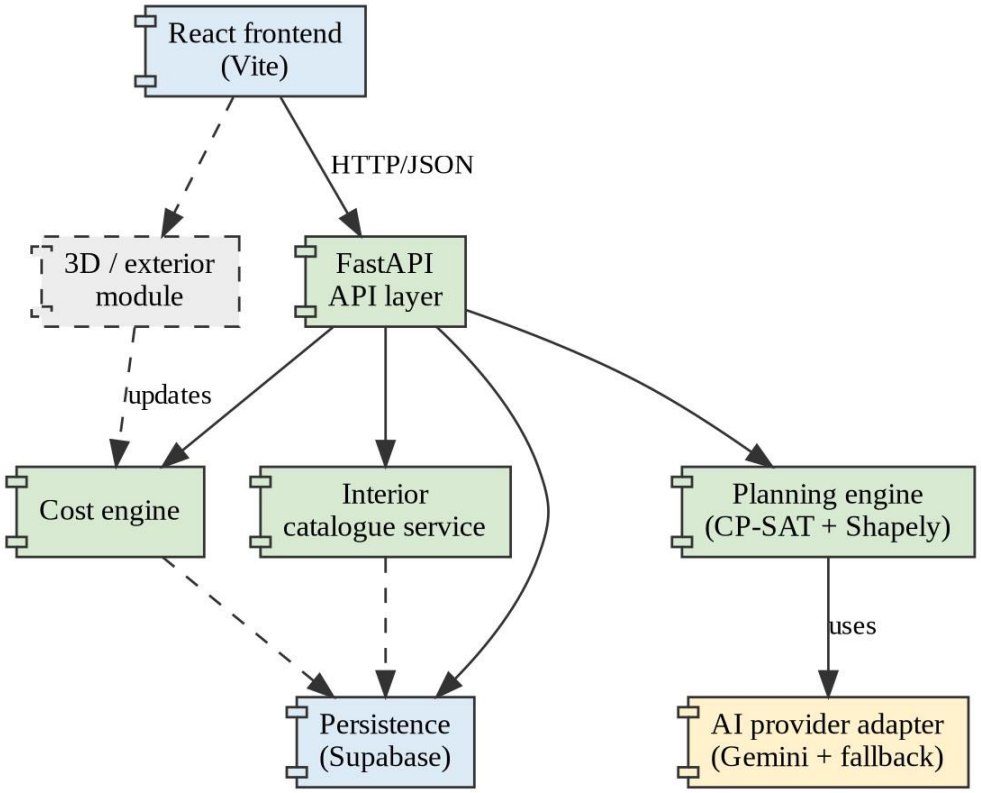


Fig 4.2.5 UML component diagram

4.3 DATA FLOW DIAGRAMS

A Data Flow Diagram (DFD) shows how data moves between the user, the processes of the system and the data stores. The Level 0 diagram shows the whole system as one process with the user, Gemini and Supabase as external entities. The Level 1 diagram decomposes the system into five processes: capturing requirements, generating the floor plan, validating geometry, selecting interior design and estimating cost, with three data stores for projects and plans, the interior catalogue and rate profiles.

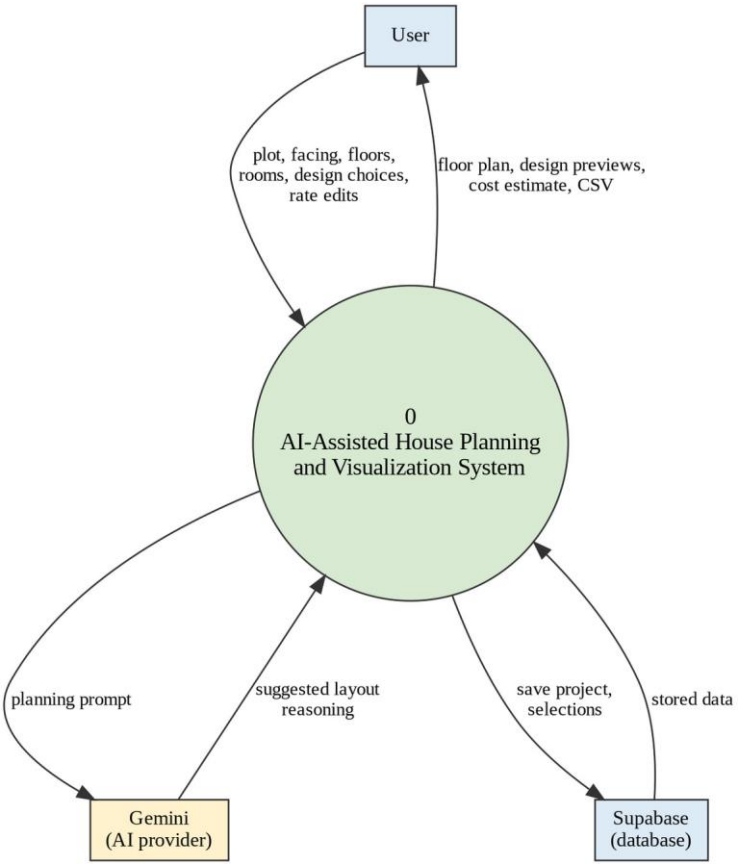


Fig 4.3.1 Data flow diagram, Level 0

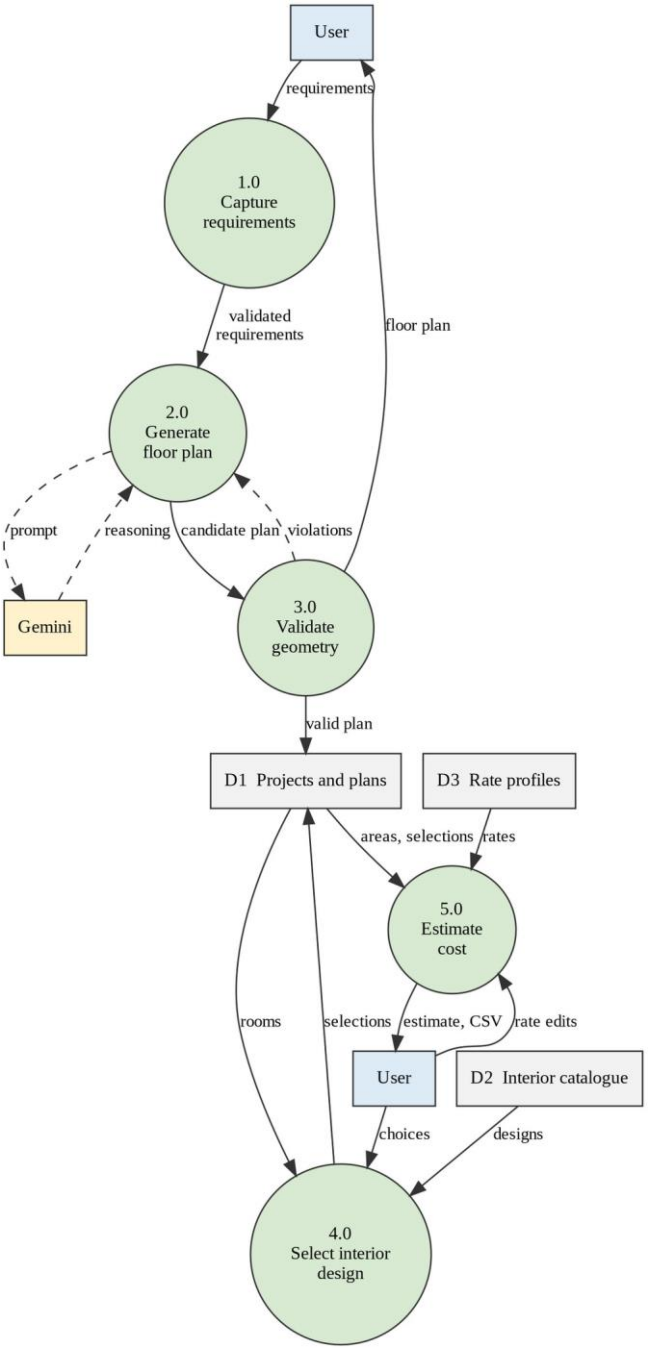


Fig 4.3.2 Data flow diagram, Level 1

CHAPTER 5

RESULT ANALYSIS AND DISCUSSION

5.1 IMPLEMENTATION DETAILS

The AI-Assisted House Planning and Visualization System was designed to connect floor-plan generation, interior design selection and cost estimation. This section describes the implemented and partly implemented parts as reported in the project checkpoint. Figures that are marked as conceptual or illustrative are not screenshots of the application. Places where an application screenshot must be inserted are marked with a placeholder box.

1. REQUIREMENTS INPUT MODULE

The project workspace in the React frontend collects the plot dimensions or area, the facing direction, the number of floors, the required rooms, parking, staircase and additional requirements. The Pydantic models of the backend validate the data before planning starts.

[INSERT SCREENSHOT: requirements form with plot size, facing, floors and rooms]

Fig 5.1 Requirements input page (screenshot placeholder)

5.1.1 FLOOR-PLAN GENERATION MODULE

Room placement is modelled as a constraint problem and solved with OR-Tools CP-SAT. Shapely checks the geometry of the candidate plan. Gemini is used for AI-assisted reasoning when it is available, and a deterministic fallback is used otherwise. Plan quality and AI reliability are Stage-2 improvement items, so this module is classed as partial. Fig 5.2 shows an illustrative sample plan with plot dimensions, facing direction, doors, windows and a staircase. It is a sample drawing, not an output of the system.

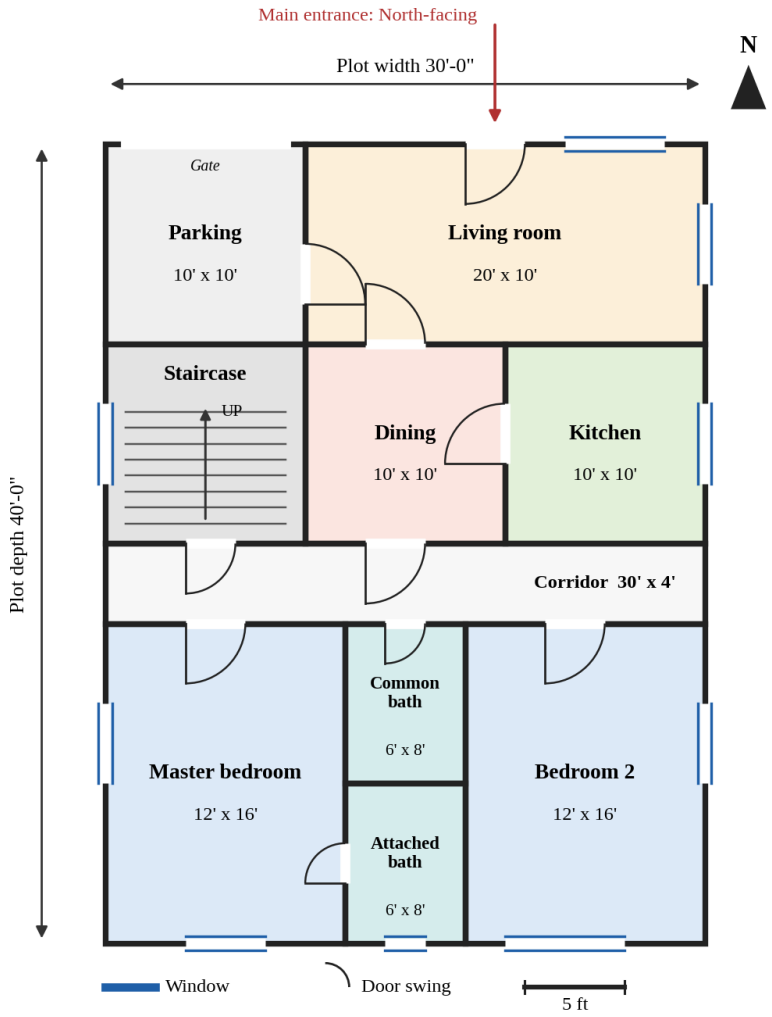


Fig 5.2 Illustrative sample 2D floor plan (30 ft x 40 ft, north-facing); not generated by the system

[INSERT SCREENSHOT: a floor plan generated by the system, with room labels and dimensions]

Fig 5.3 Generated floor plan (screenshot placeholder)

5.1.2 INTERIOR DESIGN MODULE

The catalogue contains 32 reference images grouped by room type: bedroom 8, living room 6, kitchen 6, guest room 4, bathroom 4 and dining room 4. The user filters by category, selects a design for a room that exists in the generated plan, and customizes supported flooring, wall finishes, ceiling designs, lighting and other materials. An incremental upgrade cost is calculated for the options chosen, and the selections are persisted. Fig 5.4 gives an overview of the categories, Fig 5.5 shows category filtering and room-level selection, and Fig 5.6 shows the customization controls. These three figures are conceptual mockups, not application screenshots.

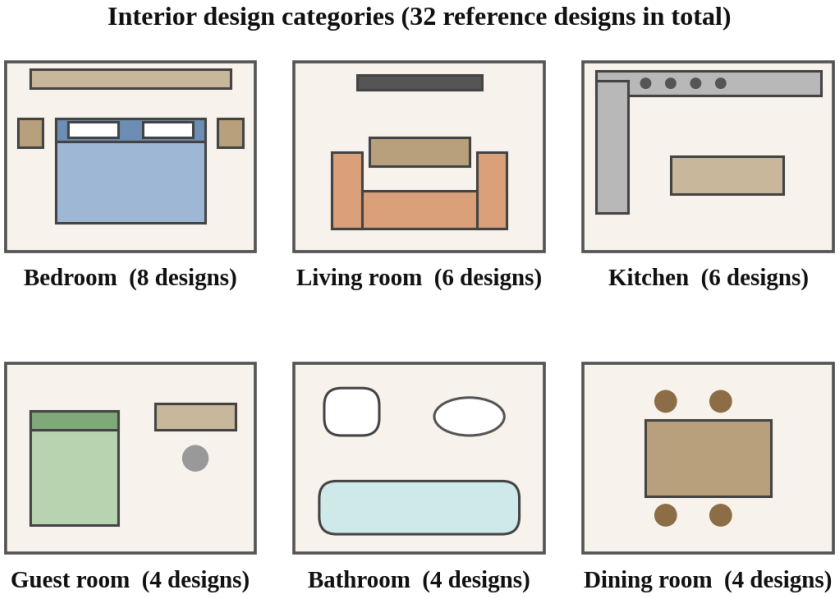


Fig 5.4 Interior design categories (conceptual schematic)

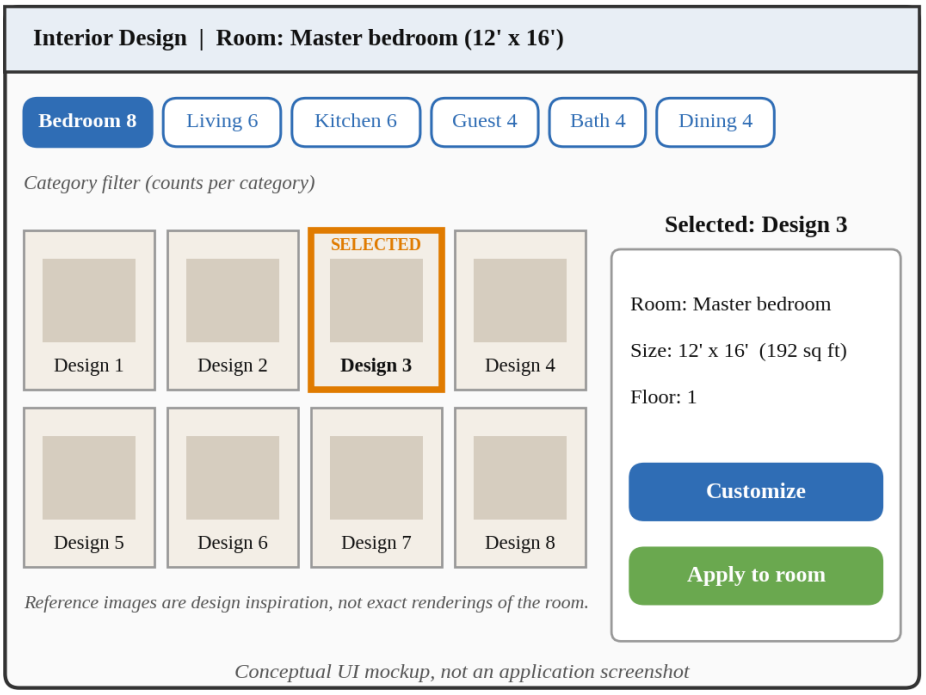


Fig 5.5 Room-specific interior design selection (conceptual UI mockup)

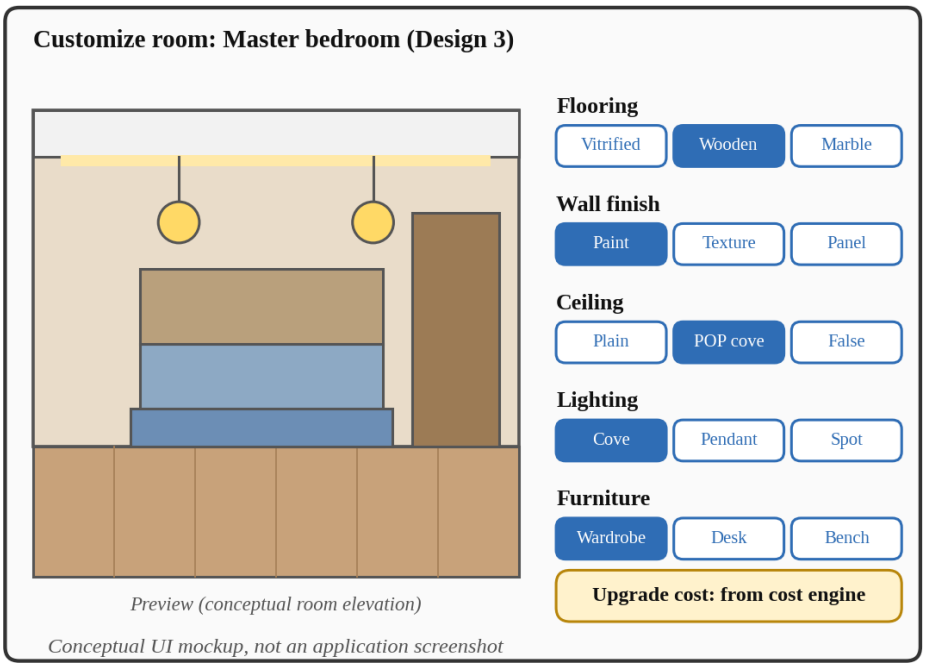


Fig 5.6 Interior customization controls (conceptual UI mockup)

Table 5.1 : INTERIOR REFERENCE DESIGNS BY ROOM CATEGORY

Room category	Number of designs
Bedroom	8
Living room	6

Room category	Number of designs
Kitchen	6
Guest room	4
Bathroom	4
Dining room	4
Total	32

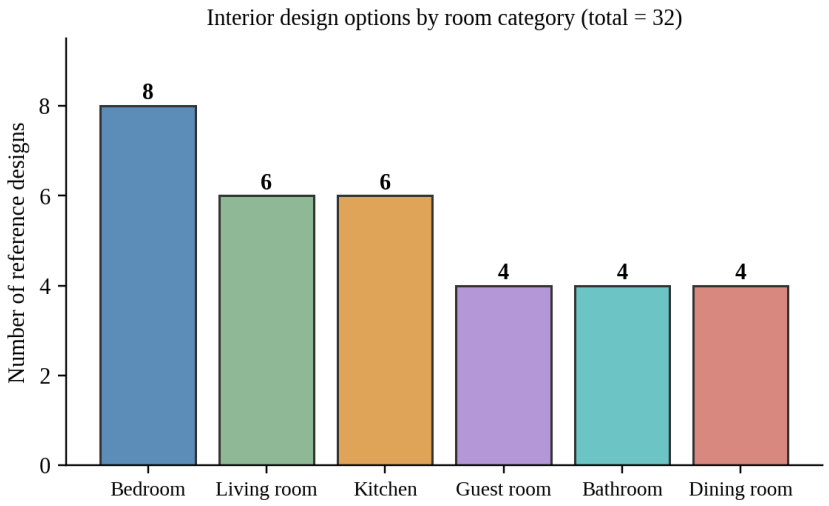


Fig 5.7 Distribution of the 32 interior design options by category

[INSERT SCREENSHOT: interior selection page showing category filter and designs]

Fig 5.8 Interior design selection page (screenshot placeholder)

5.1.3 COST ESTIMATION MODULE

The cost module derives quantities and area schedules from the plan geometry. It supports material and labour rate profiles that can be edited, produces cost line items and categorized totals, applies contingency and contractor overhead, and calculates cost per square foot. The estimate can be exported as CSV and can be recalculated without re-running the floor-plan solver. The benchmark rates are illustrative Hyderabad, Telangana estimates for 2026, not guaranteed quotations or an engineer-certified bill of quantities. The calculation follows these steps: line cost = quantity x unit rate; total = subtotal + contingency + contractor overhead; cost per sq ft = total / built-up area. An upgrade chosen in the interior module adds the quantity affected multiplied by the difference between the upgrade rate and the base rate.

Fig 5.9 shows a conceptual cost dashboard and Fig 5.10 shows an example cost-category distribution. Both use synthetic, illustrative values and are not project data.

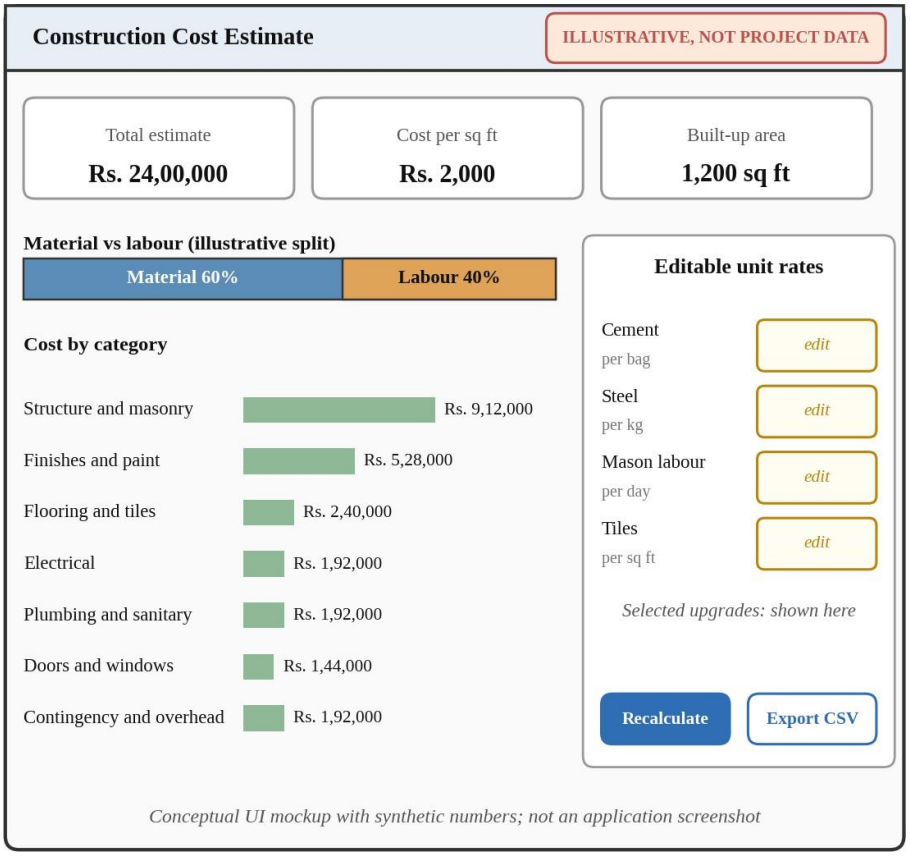


Fig 5.9 Cost dashboard (conceptual UI mockup with synthetic, illustrative values)

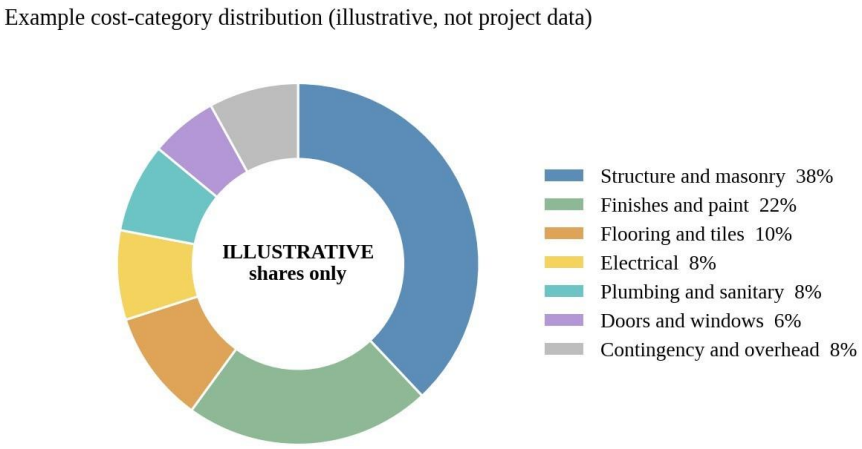


Fig 5.10 Example cost-category distribution (illustrative shares, not project data)

[INSERT SCREENSHOT: cost estimate with line items, editable rates and total]

Fig 5.11 Cost estimation page (screenshot placeholder)

5.1.4 PLANNED STAGE-2 MODULES

Exterior design, paint-colour selection and a coherent 3D visualization are not yet implemented. Fig 5.12 illustrates how a user could compare two exterior paint palettes on the same building form, and Fig 5.13 shows the planned workflow in which the floor plan, interior selections, exterior design and paint colours are combined into one 3D view with an updated cost. Both figures are conceptual illustrations of planned features.

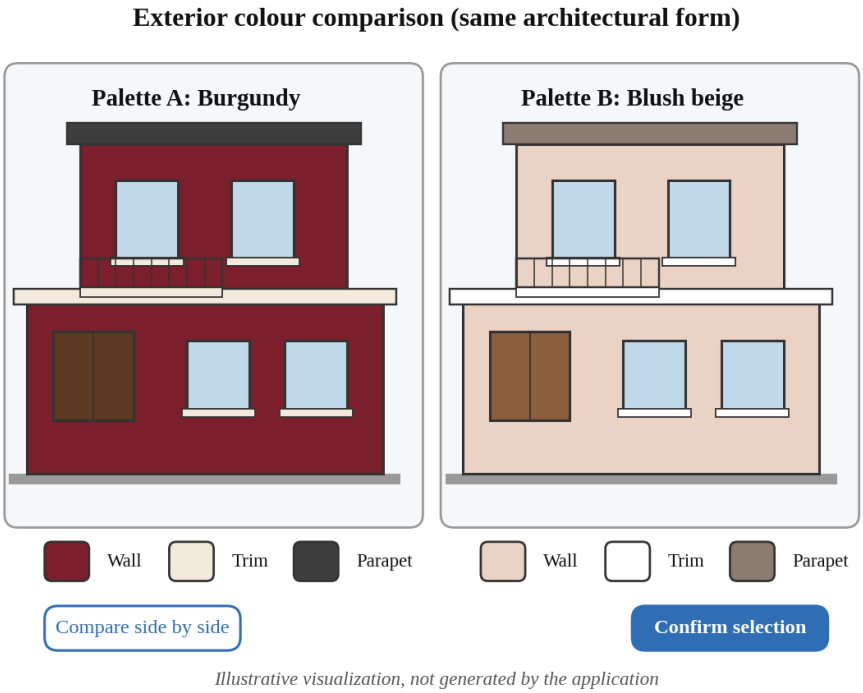


Fig 5.12 Exterior colour comparison (illustrative visualization; planned feature)

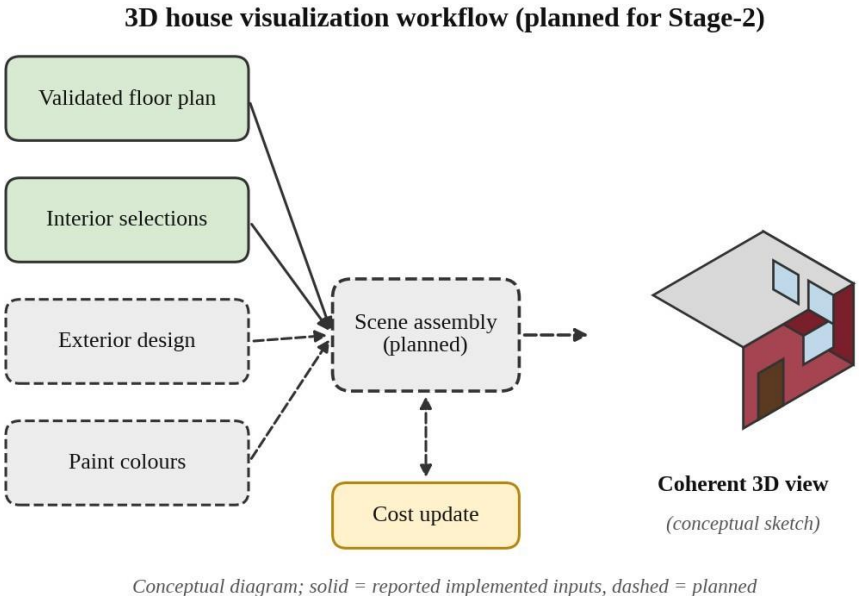


Fig 5.13 Planned 3D house visualization workflow (conceptual; Stage-2)

Table 5.2 : IMPLEMENTATION STATUS OF MODULES

Module	**Status**	**Basis**
Requirements workspace	Implemented	Reported
Floor-plan planning and validation	Partial	Quality improvement planned
AI integration with fallback	Partial	Reliability work planned
Interior catalogue and selection	Implemented	Reported
Material and lighting customization	Implemented	Reported (supported subset)
Persistence of selections	Implemented	Reported
Construction cost estimation	Implemented	Reported; 15 tests
Dedicated interior pages	Planned	Stage-2
Exterior design and colour selection	Planned	Stage-2
Coherent 3D visualization	Planned	Stage-2
Cost integration across all choices	Planned	Stage-2

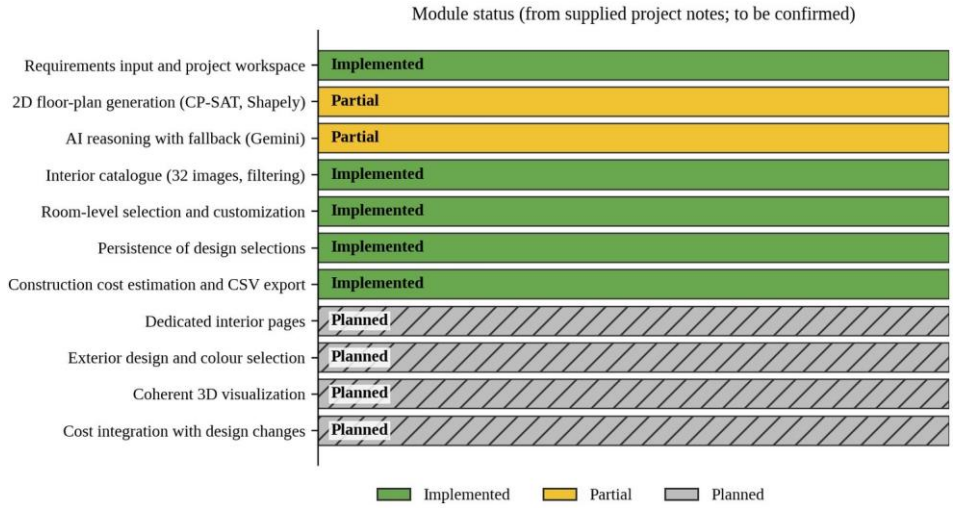


Fig 5.14 Module status chart (from supplied project notes; to be confirmed)

5.2 TESTING

Testing is the process of checking a system to ensure that it works correctly, meets the requirements and is free from errors. It helps find mistakes, verify features and improve the quality and reliability of the system.

Purpose:

- Detect and fix errors.
- Ensure the system meets user requirements.
- Improve usability and reliability.
- Ensure data security and accuracy.

a) Unit Testing

- Tests individual components and modules of the system.
- Examples: the construction cost estimation module (quantities, rates, line items, totals, contingency, overhead and cost per square foot) and the geometry logic of the planner.
- Expected outcome: each module works independently without errors.

b) Integration Testing

- Tests whether the different modules work together smoothly.
- Examples: interior selection with cost estimation (incremental upgrade cost), and geometry with cost quantities.
- Expected outcome: modules interact without data loss or errors.

c) Build Testing

- The Vite production build of the React frontend is run to check that the frontend compiles and bundles.
- Expected outcome: the production build completes successfully.

Table 5.3 gives the results reported in the project checkpoint. These are reported results and have not been independently verified. They cover cost estimation, interior and geometry logic and the frontend build. They do not show that the whole application is production-ready.

Table 5.3 : REPORTED TEST AND BUILD RESULTS

Check	Scope	Reported result
Cost estimation tests	Construction cost estimation module	15 passed
Combined tests	Interior, cost estimation and geometry	36 passed
Production build	Vite frontend build	Successful

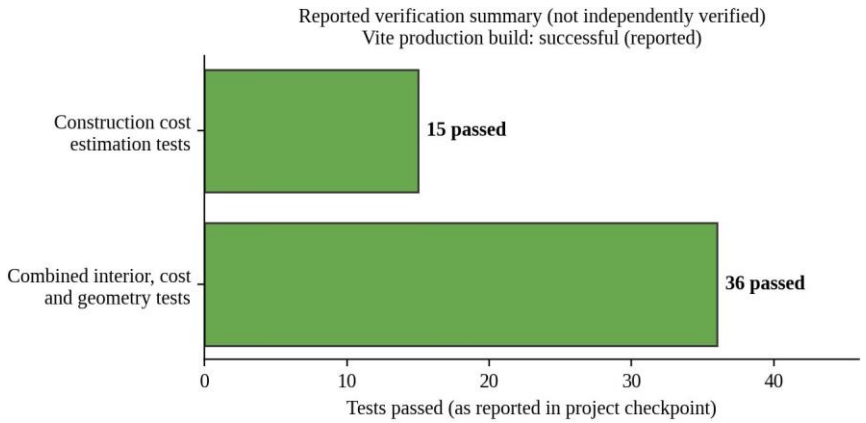


Fig 5.15 Summary of reported tests and build

Testing still to be done in Stage-2

- End-to-end tests of the whole workflow from requirements to cost estimate.
- Plan-quality evaluation of the generated floor plans.
- Tests of the AI provider failure and fallback path.
- Security testing of input handling and database access rules.
- A small user trial.

5.3 PERFORMANCE ANALYSIS

Performance analysis evaluates how efficiently and effectively the system operates. No performance benchmark, load test or user study has been carried out in Stage-1, so this section describes the design properties that affect performance and does not report measured values.

1. System Efficiency

The system reduces repeated work by storing the quantities derived from the plan. When a rate or an upgrade changes, only the cost arithmetic is repeated and the floor-plan solver is not run again. This makes recalculation lighter than a full plan generation.

2. Reliability

The deterministic fallback allows the planner to continue when the Gemini provider is unavailable. The reliability of the AI-assisted path and the quality of the generated plans are still to be improved and evaluated in Stage-2.

3. Transparency and Accuracy of the Estimate

Every cost line shows its quantity, rate and total, and the rates are editable, so the user can inspect and correct the estimate. The estimates use illustrative benchmark rates and must not be treated as quotations.

4. Scalability

The system can be extended in Stage-2 with exterior design, colour selection, 3D visualization and broader testing. The scalability of the solver for large plots and many rooms has not been measured.

5.4 DISCUSSION

The AI-Assisted House Planning and Visualization System connects the planning of a house with the choice of interiors and the estimation of cost. The reported interior and cost modules show that quantities derived from plan geometry, editable rates and room-level selections can be combined in a web application, and the reported tests and production build passed. At the same time, the floor-plan quality and the reliability of the AI integration are not yet at the level needed for the complete workflow, and exterior design, colour selection and 3D visualization are still planned.

Compared with the reviewed research, the system does not claim better plan quality than learned generators. Its contribution is the combination of constraint-based planning, validation, interior selection and a transparent cost estimate in one workflow. The results in this report are reported by the team and need to be confirmed with screenshots and further tests before they are presented as final.

CHAPTER 6

CONCLUSION

The AI-Assisted House Planning and Visualization System provides an organized approach to early house planning by connecting the floor plan, the interior design and the construction cost in one web application. Stage-1 has defined the architecture and the modules, and it has reported working interior and cost modules on a React, FastAPI and Supabase stack.

Through modules such as requirements input, floor-plan generation, geometry validation, interior selection and cost estimation, the system supports consistent plans, room-aware interior choices and an inspectable cost estimate that can be recalculated without re-solving the plan. The literature survey shows that plan generation and cost estimation are normally studied separately, which is the gap this project addresses.

The limitations of the present work are that the floor-plan quality is limited, the AI integration is not yet fully reliable, exterior design, paint-colour selection and 3D visualization are not implemented, and the test results are reported by the team and have not been independently verified. The cost rates are illustrative and the interior images are inspiration only. The database schema and API routes in this report are provisional until they are checked against the code.

CHAPTER 7

FUTURE SCOPE OF THE PROJECT

The AI-Assisted House Planning and Visualization System has significant potential for future development. The planned work for Stage-2 includes the following areas:

1. Better Floor-Plan Quality

Improve room proportions, circulation, door and window placement and the validation rules, and evaluate the plans against clear quality criteria.

2. AI Provider Reliability

Handle failures, timeouts and unusable output from the AI provider, and compare Gemini-assisted plans with fallback plans.

3. Dedicated Interior Design Pages

Provide separate interior design pages for each room, with larger previews and more customization options.

4. Exterior Design and Colour Selection

Add elevation options for the building and paint-colour palettes with side-by-side comparison before the user confirms a choice.

5. Coherent 3D Visualization

Create one 3D view that reflects the selected floor plan, interior, exterior and paint colours.

6. Cost Integration

Update the cost estimate automatically when the interior, exterior or colour choices change.

7. Broader Testing

Add end-to-end tests, plan-quality evaluation, security tests and a small user trial.

REFERENCES

1. C. Liu, J. Wu, P. Kohli, and Y. Furukawa, "Raster-to-vector: Revisiting floorplan transformation," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2195–2203, doi: 10.1109/ICCV.2017.241.
2. N. Nauata, K.-H. Chang, C.-Y. Cheng, G. Mori, and Y. Furukawa, "House-GAN: Relational generative adversarial networks for graph-constrained house layout generation," in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science. Springer, 2020, pp. 162–177, doi: 10.1007/978-3-030-58452-8_10.
3. Q. Chen, Q. Wu, R. Tang, Y. Wang, S. Wang, and M. Tan, "Intelligent Home 3D: Automatic 3D-house design from linguistic descriptions only," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, doi: 10.1109/CVPR42600.2020.01264.
4. N. Nauata, S. Hosseini, K.-H. Chang, H. Chu, C.-Y. Cheng, and Y. Furukawa, "House-GAN++: Generative adversarial layout refinement network towards intelligent computational agent for professional architects," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 13632–13641, doi: 10.1109/CVPR46437.2021.01342.
5. W. Para, P. Guerrero, T. Kelly, L. J. Guibas, and P. Wonka, "Generative layout modeling using constraint graphs," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 6690–6700, doi: 10.1109/ICCV48922.2021.00662.
6. M. A. Shabani, S. Hosseini, and Y. Furukawa, "HouseDiffusion: Vector floorplan generation via a diffusion model with discrete and continuous denoising," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 5466–5475. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2023/html/Shabani_HouseDiffusion_Vector_Floorplan_Generation_via_a_Diffusion_Model_With_Discrete_CVPR_2023_paper.html
7. R. Fu, Z. Wen, Z. Liu, and S. Sridhar, "AnyHome: Open-vocabulary generation of structured and textured 3D homes," in *Computer Vision – ECCV 2024*. Springer, 2024, pp. 52–70.
8. H. Mattern, M. Scheffer, and M. König, "BIM-based quantity take-off," in *Building Information Modeling*, A. Borrmann, M. König, C. Koch, and J. Beetz, Eds. Cham, Switzerland: Springer, 2018, doi: 10.1007/978-3-319-92862-3_23.
9. H. H. Elmousalami, "Comparison of artificial intelligence techniques for project conceptual cost prediction: A case study and comparative analysis," *IEEE Trans. Eng. Manag.*, vol. 68, no. 1, pp. 183–196, 2021.
10. React documentation – https://react.dev/
11. Vite documentation – https://vite.dev/guide/
12. FastAPI documentation – https://fastapi.tiangolo.com/
13. Google OR-Tools (CP-SAT) documentation – https://developers.google.com/optimization

14. Shapely documentation – https://shapely.readthedocs.io/
15. Supabase documentation – https://supabase.com/docs

ANNEXURE

SOURCE CODE:

Note: the source code of the application has not been supplied for this report. Replace the entries below with the actual source code of the project before submission. The pseudocode in this annexure only illustrates the logic described in Chapter 3 and Chapter 5; it is not the source code of the system.

FILES TO BE ADDED (SOURCE CODE):

FRONTEND (React, Vite, JavaScript and JSX): requirements form, project workspace, interior selection pages, cost dashboard

BACKEND (Python, FastAPI, Pydantic): API routes, request and response models

PLANNING: OR-Tools CP-SAT layout model, Shapely geometry validation, deterministic fallback planner

AI INTEGRATION: Gemini adapter

COST ESTIMATION: quantity take-off, rate profiles, totals, CSV export

INTERIOR CATALOGUE: category data and selection logic

PERSISTENCE: Supabase client and table definitions

TESTS: Pytest test files

ILLUSTRATIVE PSEUDOCODE: FLOOR-PLAN GENERATION

```
validate(requirements)
model = build_cpsat_model(plot, rooms, adjacency)
hints = ask_gemini(requirements) if gemini_available else fallback_rules(requirements)
loop:
    plan = solve(model, hints)
    violations = check_geometry(plan)   # containment, overlap, access
    if no violations: return plan
    model = adjust_constraints(model, violations)
```

ILLUSTRATIVE PSEUDOCODE: COST ESTIMATION

```
quantities = take_off(plan)              # areas, lengths, counts
for each item in quantities:
```

```
    line_cost = item.quantity * rate_profile[item.category]
subtotal = sum(line_cost)
total = subtotal + contingency + contractor_overhead
cost_per_sqft = total / built_up_area
upgrade_cost = quantity_affected * (upgrade_rate - base_rate)  # on interior selection
recalculate(): reuse stored quantities, apply edited rates; do not call the solver
```